附光盘

GPU精粹

——实时图形编程的技术、技巧和技艺

GPU Gems

Programming Techniques, Tips, and Tricks for Real-Time Graphics

〔美〕Randima Fernando 编
熊 勇 王小导 译
黄敏敏 贾东涛 审校

人民邮电出版社
POSTS & TELECOM PRESS

美术编辑／胡平利

Addison Wesley

GPU Gems

本书汇集了当今国际上前沿开发者们经多年研究和实践得出的实用的实时图形技术成果。本书主要基于当今图形处理器（GPU）的可编程图形管线，除了讲述用于创建高级视觉效果的基础知识和性能优化技术外，还着重介绍了前沿开发者们总结出来的快速技巧。本书全体撰稿者和编辑人员将他们多年的经验通过这本书贡献给读者，期待能够启发并帮助大家进入神奇的可编程实时图形世界。

本书涵盖以下主题

- 自然效果
- 光照和阴影
- 材质
- 图像处理
- 性能及实践
- 超越三角形

本书令人难忘之处在于其内容精深、覆盖面广。书中包括以产品为主的案例研究、从未发表过的最新研究成果、全面的指导以及大量代码示例和演示程序。

——Eric Haines，*Real-Time Rendering*作者

本书就是一个高级图形技术工具箱。无论是编程新手还是图形大师，都会在其中找到实用的、有益的、令人难以释手的“瑰宝”。

——Tim Sweeney，Epic Games公司Unreal系列的主力开发人员

www.PearsonEd.com

光盘内容

ISBN 7-115-14176-2

9 787115 141767 >

人民邮电出版社网址 www.ptpress.com.cn

PEARSON Education

分类建议：计算机/程序设计/游戏开发

ISBN7-115-14176-2/TP·5076

定价：78.00元(附光盘)

GPU 精粹

——实时图形编程的技术、技巧和技艺

[美] Randima Fernando 编

姚 勇 王小琴 译

龚敏敏 黄东海 审校

人民邮电出版社

图书在版编目（CIP）数据

GPU 精粹：实时图形编程的技术、技巧和技艺 /（美）费尔南多（Fernando, R.）编；姚勇，王小琴译．—北京：人民邮电出版社，2006.1

ISBN 7-115-14176-2

Ⅰ．G…　Ⅱ．①费…②姚…③王…　Ⅲ．图像处理—硬件　Ⅳ．TP303

中国版本图书馆 CIP 数据核字（2005）第 137268 号

版 权 声 明

Simplified Chinese edition Copyright © 2005 by PEARSON EDUCATION ASIA LIMITED and POSTS & TELECOMMUNICATIONS PRESS.

GPU Gems (ISBN:0321228324)

By Randima Fernando

Copyright © 2004

All Rights Reserved.

Published by arrangement with the original publisher, Pearson Education, Inc., publishing as Addison Wesley.

This edition is authorized for sale only in People's Republic of China (excluding the Special Administrative Region of Hong Kong and Macao).

本书封面贴有 Pearson Education（培生教育出版集团）激光防伪标签，无标签者不得销售。

GPU 精粹

——实时图形编程的技术、技巧和技艺

◆ 编　　[美] Randima Fernando

译　　姚　勇　王小琴

审　　校　龚敏敏　黄东海

责任编辑　李　岚

◆ 人民邮电出版社出版发行　　北京市崇文区夕照寺街 14 号

邮编　100061　　电子函件　315@ptpress com cn

网址　http //www ptpress com cn

北京密云春雷印刷厂印刷

新华书店总店北京发行所经销

◆ 开本：787×1092　1/16

印张：33 5

字数：800 千字　　2006 年 1 月第 1 版

印数：1 – 3 500 册　　2006 年 1 月北京第 1 次印刷

著作权合同登记号　图字：01-2004-4419 号

ISBN 7-115-14176-2/TP・5076

定价：78.00 元（附光盘）

读者服务热线：(010)67132705　印装质量热线：(010)67129223

序

目前，计算机图形学正处于前所未有的发展时期。近 5 年来，GPU 技术以令人惊异的速度在发展。渲染速率（以每秒钟所渲染的像素计）每 6 个月就翻一番。成倍地增长已让人兴奋，而翻番再翻番的奇迹就更令人激动了。性能 5 年来翻番了 10 次，也就是（2 的 10 次方比 2）提高了上千倍！

与此同时，不仅操作性能得到了提高，计算质量和图形编程的灵活性也逐渐得以改善。5 年前，PC 和计算机工作站只有图形加速器，没有图形处理器（GPU），而图形加速器只能简单地加速图形渲染。此外，加速器这个名词还意味着计算机执行与以前完全相同的渲染操作，只不过更快速而已。一旦 GPU 取代了图形加速器，我们就应该摈弃图形加速器的旧观念。让我们欢迎图形处理器的到来，同时也迎接它能够产生新图形运算和效果的崭新理念！

如今我们已经进入可编程的 GPU 时代。图形硬件流水线在前 20 年没有显著的变化，现在固有的组成和元件被打破，再造为可编程的、平行传递信息的处理器。在硬件流水线中，三角形顶点被转换和照明，三角形被光栅化，而像素的着色用到漫反射照明、镜面高光求幂、雾混合以及帧缓冲区混合等技术。在可编程的流水线中，每个操作都要进行组分内存访问和数学运算。虽然程序员也可以写这种程序，像硬件流水线一样进行同样的运算（实际上，这样的程序常常首先是学生程序员为可编程的处理器所写），但是现在的情况就比那样要棒多了。

第一个效果是快，可编程的 GPU 是很灵敏的。以前困难的图形问题在性能提高 1000 倍的设备上变得轻而易举。离线渲染技术的交互式应用也变得更为现实。在历史上，平滑（brute force）就是一种常用的运算方式；简单地投射图形使其在 GPU 上能够运行，是粗糙的平滑应用，这样虽然可以工作，但是明显地留下了改善的机会。此外，算法的发展更为有趣。图形程序员现在可以用新的算法和技术做实验，这在像 CPU 那样缓慢的单线串行处理器上是不可能的。GPU 的并行流处理性质加之灵活性和可编程性，允许开发新的算法并且将其投入实验。不仅是图形问题，任何计算所要求的问题，凡是可以利用大规模并行处理性、更大的存储器空间以及几何性能增长的问题，都可以在 GPU 的执行中得到很好的实验。作为一个研究课题，我们正在摸索什么问题可能从 GPU 中获得最大益处。

可编程 GPU 革命的真正受益人是计算机图形算法的设计者、研究者和应用者，而最终是终端使用者或用户：游戏玩家和电影观众等。因为 GPU 的编程设计领域年轻而充满活力，许多开发者正在一次又一次地重复创建相同的技术，所以《GPU 精粹》是一本及时的书。这本书包含的简单应用

是众所周知的技术对权威 GPU 结构的实践，也包含了一些创新的算法和先前不可能达到的效果。

现在是 GPU 程序规划开发的历史关头，许多问题尚待解决，并且将要面临更多挑战。因此，《GPU 精粹》的第一卷仅是方法的简述和一些迄今为止成功实现的例子，远非一个可以插入应用程序的完整特效的使用说明书。这本书能使今天的程序员从以前令人振奋的进步中受益，也将使明天的开发者攀上 GPU 先驱者巨人的肩头。今后的改革创新大任将要由他们来承担!

David Kirk
NVIDIA 公司首席科学家

前　言

本书收集了有关实时计算机图形的内容广泛、实践性强的文章，汇集了工业界和学术界专家们的知识和经验。《GPU 精粹》是一本短文集，它以相同的风格汇集了精粹作品。对于那些欲披挂上阵、为 GPU 摩拳擦掌的今日开发者来说，这本书具有独特而宝贵的意义。

首先也是最重要的，就是这本书直接把重心集中在实时可编程的图形上，即与 GPU 相关的技术上。每章都经过精心挑选，介绍了对交互式应用（例如电脑游戏）直接有用的思路和技术。各章都提供了作者的领悟和理解，而不是仅仅集中在低水平的 API 调用，或专门的数学技巧上。其次，每章都用了很多彩色图表和图像，来举例说明主要概念。最后，撰稿人的经验和多样性将有助于开拓读者的思路；而来自这个领域的专家们的忠告，更能坚定读者的信心。

NVIDIA 公司最宝贵的资产是人才，这个集体的专业知识的深度和质量，使我们萌发了编撰《GPU 精粹》的想法。我们感到，既然有这么多知识和专业技术在手，团队的思想和洞察力已经促进了实时图形的极大的新进展，也必定会产生一本了不起的指导性著作。

与这样一些人共同工作已经是一种荣幸，他们是游戏制造业、工具开发业、电影工作室和学术机构的领军人物，他们正在形成实时计算机图形学的未来。在这里我们要强调的是他们对《GPU 精粹》的贡献。通过广泛公开号召参加本书编写工作的倡议，我们将大量的精英和清新的远见聚积在一起。

无论是正在创建新的效果，构筑图形引擎，还是正为操作性能挤出最后一个 bit，我们都希望本书能为你提供有价值的指导，使人们能从作者所面对的挑战中得到教益。我们全体工作于《GPU 精粹》的同仁，希望这本书能够帮助人们开拓新的思路，激励人们投身于开发图形学的新阶段。

我们期待的读者

这本书给读者提供了有用的信息，使他们在项目开发中能够获得帮助。《GPU 精粹》超越了高级着色法的基本原理，旨在破除复杂效果和高级 GPU 程序的神秘，使现有的项目向前发展。随着实时着色语言的迅速进步，实时图形开发者可用的算法也比过去多得多。通过编辑和发布本书的信息，我们希望高质量、高性能的图形学被更广大的读者群所接受，包括游戏开发者、技术指导、教授和学生等。

代码样例

本书的许多章都含有代码样例，目的是更具体地说明有关内容。作者使用的是任何他们想用的着色法语言，但是最后代码样例显示为 DirectX 9 的高级着色语言（HLSL）或 Cg，在本项目中只是广泛使用这两个高级着色法语言。几乎这里介绍的每件事，都可以使用这两个语言中的一种，或以后出现的 OpenGL 着色语言。可以在随书的 CD 盘中找到代码样例及其绝妙的例子。这使得综合或实验各种不同的例子很容易。更新的代码样例和另外的补充材料，可以从书的网站上得到：http:// developer.nvidia.com/GPUGem/。

致谢

没有许多人的努力，《GPU 精粹》这样大的项目是不可能完成的。首先，我要感谢撰稿人，没有他们这本书就不可能产生。与这些富有经验和能力的人们合作是我的荣幸。但是随之而来的挑战是他们每个人也特别忙。我感谢他们于百忙之中抽出时间从事这个项目，以及他们为了最佳结果所付出的努力。也感谢他们所在的单位允许他们参加这个项目。还要感谢那些响应我们倡议并且为此提供了有价值的建议的人们。

编委 Kevin Bjorke、Cem Cebenoyan、Sim Dietrich、Simon Green、Juan Guardado 和 John Spitzer 做出了极大贡献，他们多次和作者讨论，帮助审阅书稿，而且监督确保文章的进度。对于他们全体志愿协助这个项目我深表感谢。Chris Seitz 对于这个项目也是功不可没，他照顾法律事务、审稿，并且随时抽出时间与我讨论有关项目的问题。

每章都经过了广泛的审阅，包括资深的审稿人、编辑和外国审稿人的评述。撰稿人和编委们做了极好的工作，批评和帮助改善了书的内容。我尤其要感谢 Matt Pharr（他超越倡议的要求，花费周末时间帮助审阅一些章节，还另写了 3 章），感谢 Kevin Bjorke（他是编委，另外还撰写了几章）、Larry Gritz、Eric Haines 和 Matthias Wloka 担任严格的外部评委，而且提供了很多宝贵的、有洞察力的见解。我们对不知名的外部评委也表示感谢，他们很好地完成了所负责的任务。

Caroline Lie、Spender Yuen、Dana Chan 和 Melvin Chong 发挥了他们的专长，制作了书的封面、透明图样纸和图表。我特别要感谢 Spender 耐心地为本书制作了 100 多个图表（另加 200 多个屏幕截图）。Catherine Kilkenny、Debra Valentine 和 Teresa Saffaie 帮助改进和撰写文稿。David Kirk 为本书撰写了远见卓识的序。

Christopher Keane 做了复制和合成手稿的工作。也感谢 Jacquelyn Doucette、John Fuller、Bernard Gaffney、Curt Johnson 和 Heather Mullane，以及在 Addison-Wesley 工作的其他人从始至终对这个项目的帮助。特别是 Peter Gordon 给截止期限提供的奖励帮助我们按时出书。

从构建 GPU 到帮助开发者创建令人兴奋的新内容，正是由于 NVIDIA 经过无数的努力，才有了这本书所包含的对图形业有用的知识。因此，我要感谢 NVIDIA 中的每个人。没有他们的努力，就没有这本《GPU 精粹》。也要对 Mark Daly、Dan Vivoli 和 Jen-Hsun Huang 表示特别的感谢，是他们创建了团队和资源，才使得所有这一切可能发生。

最后，我想要特别感谢我的父母和姐妹，是他们给我以非凡的支持，使我得到这个特殊

的机会。

我相信，集众多才智之士的努力、创造力和关注而诞生的《GPU 精粹》，一定会为我们的读者很好地服务的。

Randima (Randy) Fernando
NVIDIA 公司

作 者 简 介

Curtis Beeson，NVIDIA

Curtis Beeson 5 年前从 SGI 转到 NVIDIA 演示小组，他的研究集中在美术流程、物体模型和 NVIDIA 演示引擎的 DirectX 渲染。进入 Carnegie Mellon 大学后，他开始从事 3D 方面的工作，为头戴式显示器产生了高分辨率的显示环境。Curtis 专攻 NVIDIA 演示小组的场景即图形 API（应用编程接口）的美术流程和物体模型，同时对过度急迫的市场需求管理进行了研究。

Kevin Bjorke，NVIDIA

Kevin Bjorke 在 NVIDIA 的技术开发组工作，正在开发和促进下一代美术和娱乐工具，对于可编程着色硬件的可能性具有独到的眼光。在进入 NVIDIA 之前，他曾在电影、电视和游戏等多个行业中从事过工作，是 *Final Fantasy*（最终幻想）、*The Spirits Within* 和 *The Animatrix* 的开发和影像的督导，在 *Toy Story* 和 *A Bug's Life* 担任过多个技术主管并负责了动画的编排工作，在差不多每个商业平台上开发过游戏，开发过主题公园游艺设施；为众多电视节目制作动画，还参与过电视脱口秀节目。他进过几个大学，最后毕业于 California Institute of the Arts film school（加州艺术电影研究院）。Kevin 还是 SIGGRAPH、GDC 和此类机构过去 10 年的正式发言人。

Rod Bogart，Industrial Light & Magic

Rod Bogart 在 Pacific Data Images（太平洋数据图像）担任了 3 年软件工程师，于 1995 年来到 Industrial Light & Magic（ILM）。他早期的关于摄像机跟踪的作品发表在 *Graphics Gems II* 上。在 ILM，Rod Bogart 开发了交互合成工具的核心引擎，并监管了许多 ILM 其他图像处理和图像观察的技术设计和实施。

Ian Buck，Stanford University

Ian Buck 正在斯坦福大学图形实验室攻读计算机科学博士，现正在研究 CPU 的多用途计算模型。他的研究重点是图形硬件的编程语言设计，

以及多用途计算应用对图形硬件体系结构的影射。他于 1999 年获得普林斯顿大学的计算机科学学士学位，并且获得了斯坦福工程学院和 NVIDIA 的奖学金。

Michael Bunnell，NVIDIA

Michael Bunnell 毕业于 Southern Methodist（南方卫理公会教）大学，获计算机科学和电机工程学位。他为 LynuxWorks 的前身 Macintosh、Atari ST、Apple IIGS 编写了 Megamax C 编译器。在研究即时操作系统 9 年之后，他转到 Silicon Graphics，重点集中在图像处理及视频与图形软件上。然后，他去了 Gigapixel 工作，以后又到 3dfx，现在他供职于 NVIDIA，负责开发 shader 编译器。

Cem Cebenoyan，NVIDIA

Cem Cebenoyan 是开发技术组的软件工程师。他平时研究图形技术，同时帮助游戏开发者最有效地使用图形硬件。他曾在游戏开发者会议上就人物动画、图形性能和 NPR 做过报告。在进入 NVIDIA 之前，Cem 是 Georgia（乔治亚州）技术学院图形、视觉应用实验室中的学生和助研。

Joe Demers，NVIDIA

Joe Demers 曾获得卡内基·美隆大学的学士和南加州大学的硕士学位。1999 年他来到 NVIDIA 工作，在 NVIDIA 曾制作过一个简单而强大的 3D 模型 Nendo 和 Nichimen Graphics 的绘画包以及多个演示程序。

Eugene d'Eon，Waterloo 大学

Eugene d'Eon 最近在 Waterloo 大学获得应用数学和计算机科学学士学位。他现在居住在加州，供职于 NVIDIA 的演示小组。他爱好钢琴，有时深夜还在办公室弹奏。

Sim Dietrich，NVIDIA

Sim Dietrich 是 NVIDIA 的美国技术开发组的经理。Sim 已为 *Game Programming Gems 1*（游戏编程精粹 I）和 *Game Programming Gems 2*（游戏编程精粹 II）撰写数章，并负责 *Gems 2* 中关于图形显示章节的编辑工作。Sim 是 CgFX 的主要撰稿人，首次将即时 shader 引入 3ds max、Maya 和 Softimage|XSI。Sim 的兴趣集中于新的阴影技术和通过 Cg 与 CgFX 改善图形的工作流程。

William Donnelly，Waterloo 大学

William Donnelly 是安大略省 *Waterloo* 大学的数学和计算机科学本科生。他根据 Okino

计算机图形学来为 NuGraf ray 扫描工具扩展功能，并获得了实习医师学位，与 NVIDIA 一起为 GeForce FX 创建即时图形演示。虽然他 8 岁时发现以乐高（塑胶拼图板）建筑为生只是梦想，但仍然自小就树立了以计算机图形学为事业的志向。

Randima Fernando，NVIDIA

Randima（Randy）Fernando 从 8 岁起就爱上了计算机图形学。现在在 NVIDIA 的技术开发组工作，同时教授开发者如何利用最新的图形处理技术（GPU）。Randy 在康奈尔大学获得了计算机科学学士学位和计算机图形学硕士学位。他已经在 SIGGRAPH 上发表过文章，并且是 *The Cg Tutorial: The Definitive Guide to Programmable Real-Time Graphics* 的合著人（与 Mark Kilgard 合著）。

Mark Finch，Cyan Worlds

Mark Finch 曾获得乔治亚（Georgia）技术学院物理学学士学位和 Chapel Hill 的北卡罗莱那大学（North Carolina）的计算机科学硕士学位。他的专业图形工作开始于为星球大战（Star Wars）进行图像处理和渲染。但他很快转到游戏编程，他从中发现了同样的挑战性和更多的建设性。他现在是 Cyan's *Uru* 项目的图形程序师。

Philipp S.Gerasimov，iXBT.com

Philipp Gerasimov 是一名俄罗斯的 3D 程序师和游戏设计者。他目前在研究 RightMark 3D 的创建，即 iXBT 的公开的 3D 硬件基准。

R.Steven Glanville，NVIDIA

Steven Glanville 于 1977 年获得加州大学伯克利分校的计算机科学博士。1978 年，他创办硅谷软件（Silicon Valley Software）并担任总裁 15 年，在那他开发了 C 和 FORTRAN 编辑器。在加入 SIGGRAPH 多年后，他最终于 1995 年转向图形学，并开始开发 OpenGL 驱动器。现在他再创辉煌，是 NVIDIA Cg 语言的主要设计者和实施者之一。

Simon Green，NVIDIA

Simon Green 是 NVIDIA 的技术开发组的工程师。他毕业于英国 Reading 大学的计算机科学专业，在电视游乐业工作两年后移居美国，为 Silicon Graphics 工作。他出席过游戏开发者（Game Developer）和苹果世界开发者（Apple World Wide Developer）会议。他的兴趣包括 OpenGL、便携（cellular）自动装置、基于图像的渲染和模拟合成器。他正在 NVIDIA 发明使用图形硬件的有趣的新方法。

Juan Guardado，NVIDIA

Juan Guardado 曾骑骆驼穿越撒哈拉沙漠，他曾休闲于巴西海岸，但他更乐于献身于计算机游戏业的图形技术。在获得 McGill 大学计算机工程学的学士学位后，他进入了 Matrox Graphics，并在硬件加速置换贴图（hardware-accelerated displacement mapping）的开发中取得很大成绩。现在他供职于 NVIDIA 在英国的技术开发组。他已经在业内会议做过多次口头发言，包括 GDC（美国和欧洲），并在 Shader X 和 Gamasutra 等网上杂志中发表过文章。

Charles Hansen，Utah 大学

Charles Hansen 于 1981 年获得了孟菲斯州立大学的计算机科学学士学位，1987 年获得犹他大学计算机科学博士学位。他现在是 *Utah* 大学计算机科学副教授。从 1989 到 1997 年，他是 Los Alamos 国家实验室的高级计算机实验室（ACL）的技术雇员，ACL 的可视成果由他组织和指导而取得。1987 年和 1988 年他在法国 INRIA Rocquencourt 从事 Bourse Chateaubriand 博士后研究。他的研究兴趣包括大规模科学可视学和计算机图形。

Mark J.Harris，University of North Carolina at Chapel Hill

Mark Harris 1998 年获得 Noire Dame 大学的学士，2003 年获得北卡罗莱那大学（UNC）计算机科学博士。在 UNC，Mark 的研究涵盖了多种计算机图形课题，包括即时云模拟和渲染、CPU 的各种计算、全局照明、非相片拟真的渲染和虚拟环境等。在研究生期间，Mark 曾为 Intel、iROCK Games 和 NVIDIA 做过一些工作，现在就职于 NVIDIA 英国的技术开发组。

Drew Hess，Industrial Light & Magic

Drew Hess 是 Industrial Light & Magic 软件研究和开发部门的软件开发员。过去他曾是 Intel 公司 IA-64 结构组的成员。Drew 曾获得伊利诺斯大学（在 Urbana-Champaign）计算机工程学学士学位和斯坦福大学计算机科学硕士学位。

Milan Ikits，University of Utah

Milan Ikits 是犹他大学科学计算专业的博士生和科学计算及图像学研究所的助研。他目前的研究兴趣在计算机图形、科学可视学、沉浸环境和人机交互作用的领域。他 1997 年毕业于布达佩斯技术和经济大学的计算机科学专业。Milan 是流行的 OpenGL Extension Wrangler（开放式绘图界面扩展库，GLEW）的创始人。

Greg James，NVIDIA

Greg James 成长在 3D 图形电影和计算机开放的家庭，具有开阔的视野，他从很小时就

痴迷于图形学。虽然他未能在学习数学和科学上取得成功，但是在 1995 年获得物理学学士并兼修了摄影艺术。并且幸运的是，Greg 在 NVIDIA 开发者协会顶级项目中发现了安全的环境，它帮助自己和其他人解决了难题。他已经在许多游戏的视觉效果方面做出过贡献。其中包括 Morrowind 中的水反射动画、Tron 2.0 中的光晕和一个即将发行品牌的体积半透明效果。他的著作已经发表在各种计算机图形和物理学的出版物上，他特别热衷于物理学和计算机图形的结合。

Frank Jargstorff，NVIDIA

Frank Jargstorff 是 NVIDIA 技术开发组的软件工程师。他研制用于数字容量创建的工具，并帮助 DCC 公司把新技术整合到他们的产品中。在进入 NVIDIA 之前，他曾在 IBM、CoCreate 和 Fraunhofer 研究所从事过 3D 流、主机操作系统等研究工作。Frank 在 1997 年从德国 Tübingen 大学获得计算机科学学位。

Alexandre Jean Claude，Softimage

AlexandreJean Claude 目前在领导 Softimage 的渲染和流水线小组。过去几年，他主要研究了游戏开发流水线和硬件渲染技术。他毕业于蒙特利尔魁北克大学计算机科学专业。

Florian Kainz，Industrial Light & Magic

Florian Kainz 在 1995 年加入 Industrial Light & Magic（ILM），成为研究和开发组的成员。目前是计算机图形首席工程师，他领导着 ILM 内部计算机动画系统核心机构的小组。Kainz 已经开发了微粒、茸毛和模糊表面的渲染、一个以网络规程为基础的容错的分布式批处理系统。Kainz 是 OpenEXR 文件格式的创始者之一。在进入 ILM 之前，Kainz 曾在德国慕尼黑担任 Steiner Film 的软件工程师，并为新加坡的 Twenty-Five Frames 编写图像处理和 3D 渲染软件。他于 1992 年在德国慕尼黑技术大学获得计算机科学学位。

Joe Kniss，University of Utah

Joe Kniss 1999 年在爱达荷州立大学获学士学位，2002 年获犹他大学计算机科学硕士学位。他现在正在 Utah 大学攻读计算机科学博士，并且是科学计算和图像学会的成员。他的研究兴趣包括计算机图形、光在多种媒质中的传播、人机交互作用和沉浸环境等。

Simon Kozlov, SoftLab-NSK

Simon Kozlov 于 2003 年 6 月从 Novosibirsk 州立大学毕业，专攻物理学和计算机科学。自 2001 年以来，Simon 一直在俄罗斯的游戏开发室 SoftLab-NSK 工作。Simon 目前的项目是 *Rig'n'Roll*，一个卡车模拟游戏。

Aaron Lefohn，University of California，Davis

Aaron Lefohn 是 Davis 的 California 大学计算机科学系的博士生，又是 Pixar 动画工作室的图形软件工程师。他的研究兴趣包括图形硬件、相片拟真渲染和身体的基础动画等一般计算。Aaron 于 2003 年获得犹他大学计算机科学的硕士学位，2001 年获得犹他大学理论化学硕士学位，1997 年获得 Whitman（惠特曼）学院的化学学士学位。Aaron 还曾获得计算机科学 NSF（国家科学基金会）研究生奖学金。

Jörn Loviscach，Hochschule Bremen

Jörn Loviscach 于 1993 年获得数理物理学博士头衔，此前他已在流行计算机杂志中发表了许多关于编程和电子音乐的文章。他曾在几个计算机杂志社工作。在担任德国计算机杂志 c't 的副总编 3 年后，他于 2000 年成为 Hochschule Bremen（应用科学大学）的教授，从事计算机图形的教学与研究工作。

Stephen Marshall，Sony Pictures Imageworks

Stephen Marshall 在 Sony Pictures Imageworks 担任软件工程师，现在是效果技术的指导。在进入 Imageworks 之前，Stephen 是工程动画公司（Engineering Animation）的软件动画部门的开发者。

Christopher Maughan，NVIDIA

Chris Maughan 已经在图形硬件业工作 10 年了。最初他为一家公司开发视频数字转换产品，后来很快转入 3Dlabs，在那他与微软合作，为 Windows 95 写了第一个 OpenGL 驱动器程序。接着 Chris 开发了公司的 DirectX 驱动器。5 年后他进入 NVIDIA，在技术开发组工作，集中为游戏开发者提供工具和样品软件。Chris 从英国北方的 York 的家去工作，他与妻子 Stacey 居住在那里。他对家庭、工作和地点都很满意。

David McAllister，NVIDIA

David McAllister 自 2000 年以来就是 NVIDIA 的计算机图形设计师。2002 年他获得北卡罗莱那大学计算机科学博士头衔，在那里他进行了基于图形的渲染、反射系数的测量和表达式，以及图形硬件的研究。1997 年夏，他在 Hewlett-Packard 的 PixelFlow 图形超级计算机上工作。从 1989 到 1996 年 David 在 Evans & Sutherland 工作，并在 1995 年获得犹他（Utah）大学的学士学位。David 现居住在犹他州盐湖城。

Morgan McGuire，Brown University

Morgan McGuire 领导着布朗（Brown）大学的游戏研究组。他的博士研究课题是游戏的即时全局照明模型。他已经获得了麻省理工学院和布朗大学的硕士学位，他是 graphics3d.com 上开源 G3D 库的项目经理。

Hubert Nguyen，NVIDIA

Hubert Nguyen 是 NVIDIA 演示组的软件工程师。他的工作是寻找新奇效果，展示 NVIDIA 最近的图形处理器特征。他曾在游戏开发者会议上就产生 NVIDIA 演示的技术做过发言。在进入 NVIDIA 之前，Hubert 是 3dfx Interactive 的工程师、Voodoo 的创造者。在那之前，Hubert 是法国巴黎 Cryo InteractiveR&D 部门的成员。在欧洲 demoscene 时，Hubert 就开始开发 3D 图形程序。他曾获得计算机科学学士学位。

John O'Rorke，Monolith Productions

John O'Rorke 从 6 岁起开始创建游戏。在高中时，他就在 HP 公司的固件部门兼职，后来又进入 DigiPen 技术学会。他现在是 Monolith Productions 的引擎设计师，他已经参加过 *Aliens vs. Predator 2*（外星人和掠夺者 2）、*Tron 2.0*、*No One Lives Forever 2*（没有人永远活着 2）、*A Spy in H.A.R.M.'s Way*（间谍在 H.A.R.M.路上）的研制，目前正在为将来的产品开发引擎技术。

Fabio Pellacini，Pixar Animation Studios

Fabio Pellacini 出生在意大利，并在那里度过了 24 年。这期间，他获得了 Parma 大学的物理学 Laurea 学位，并在 Milestone 工作了一年，构建了一个物理引擎。有了经验之后，他决定搬到美国追求对计算机图形的兴趣，在美国他获得康乃尔大学的计算机图形学博士头衔，并发表了数篇论文（其中 4 篇由 SIGGRAPH 发表）。目前他正在 Pixar 动画工作室的黑暗办公室中设计和实现下一代的照明技术。

Kurt Pelzer，Piranha Bytes

Kurt Pelzer 是 Piranha Bytes 的资深软件工程师，他开发过个人计算机游戏 *Gothic*、销售头名的 *Gothic II*。在 2001 和 2002 年分别获得德国颁发的 RPG（角色扮演游戏）和资料片 *Gothic II：The Night of the Raven.*（掠夺之夜）的年度奖。这之前，他是 Codecult 的资深程序师，并开发了几个即时模拟和建立在 Codecult 的高端 3D 引擎 Codecreatures 上的技术演示程序（例如为西门子 AG 和著名的 *Codecreatures Benchmark Pro* 而制作的上海磁悬浮项目的模拟）。他还在《ShaderX2》和《*Game Programming Gems 4*》（游戏编程荟萃 4）中发表了文章。

Ken Perlin，New York University

Ken Perlin 是纽约大学计算机科学系的教授，管理教育媒体研究实验室和高级技术中心的工作。由于他的过程纹理运算法则（广泛地用于游戏和其他相关方面），他已获得学院奖（Academy Award）、由于科学和技术优秀的纽约市市长奖、NYU 表彰杰出理科教职员的 Sokol Faculty 奖和一个国家科学基金会的年轻调查员总统奖。Ken 还获得了纽约大学计算机科学博士头衔、哈佛的理论数学学士学位。他在 R/GA 和 Mathematical Applications Group，Inc（数学应用公司）负责软件开发，从 Tron 开始，他已经完成了多部电影制作。他还是纽约软件业协会董事会的成员。

Matt Pharr，NVIDIA

Matt Pharr 是 NVIDIA 的技术职员，他研究的课题关系到高质量互动图形、可编程的描影和 Cg 语群中的语言特性。此前，他是 Exluna 的共同创办人，开发了离线渲染软件，并为图形硬件调研过高级描影运算法则。他曾是斯坦福大学图形实验室的博士生，在那里他研究过用于渲染的系统问题和渲染的理论基础。他还在 SIGGRAPH 上就这个问题发表过一系列文章。他与 Greg Humphreys 合著了 *Physically-Based Rendering: From Theory to Implementation*（基于物理的渲染：从理论到实践），该书在 2004 年出版。

Fabio Policarpo，Paralelo Computação Ltda.

Fabio Policarpo 从里约热内卢（Rio de Janeiro）的 UFF 联邦大学毕业，曾写过关于经典和即时的计算机图形的书和应用软件。Fabio 曾与 Alan Watt 合著了 *The Computer Image*（计算机图形）、*3D Games Volume1*（3D 游戏第 1 册）和 *3D Games Volume 2*（3D 游戏第 2 册）等书（均由 Addison-Wesley 出版）。

Tim Purcell，Stanford University

Tim Purcell 正在斯坦福大学攻读计算机科学博士学位。他于 2001 年获得斯坦福大学的计算机科学硕士学位，1998 年获得犹他大学的计算机科学学士。他还获得过国家科学基金会研究生研究奖学金，而且是 NVIDIA 奖学金获得者。他目前的研究兴趣包括流体（stream）编程、光线追踪和 GPU 的多用途计算。

Daniel Sánchez-Crespo，Universitat Pompeu Fabra

Daniel Sánchez-Crespo 在西班牙和欧洲是游戏研究的领军人物之一。他的学术生涯开始于 1997 年在西班牙巴塞罗纳 Politècnica de Catalunya 大学进行用户接口和虚拟现实的研究。2001 年他在西班牙巴塞罗纳 Universitat Pompeu Fabra 创办欧洲第一个视频游戏的硕士点。在

那里他担任教师和硕士生导师。他已经写了 3 本书，最近的一本是 *Core Techniques and Algorithms in Game Programming*（游戏设计的核心技术和运算法则），由 New Riders/Prentice-Hall 出版。他的最新挑战是创办 Novarama 游戏开发工作室，焦点是大众娱乐的新形式，最近被认为是西班牙 2003 年第三大的创新公司。

Dean Sekulic，Croteam

认识到制作电脑游戏可能比仅仅来玩更有趣之后，1993 年 Dean Sekulic 与他的两个最好的朋友一起创办了 Croteam。最近两年，他专攻声音和视觉的编码。1996 年他从商业信息和计算机设计的 Zagreb 大学毕业，他希望永远从事计算机游戏设计。

John Spitzer，NVIDIA

John Spitzer 是 NVIDIA 欧洲技术开发部的主管，他在 3D 软件开发界监督工具、技术、艺术和教育材料的开发。John 对游戏开发者每天的游戏技术进行评估，并协助他们实现高级的视觉效果。John 是许多与 3D 图形有关的工业标准委员会成员，包括 OpenGL 结构评估董事会（ARB），而且是 SPEC OpenGL 运行特色组织（SPECopc）的创办人和主席。在 SPECopc 任职时，John 定义、设计和实施了 SPECglperf 基点的工业标准。John 每年出席许多开发教育活动，主题包括运行性能最佳化及先进描影技术等。John 曾从 Rice 大学获得计算机科学的学士和硕士学位。

Jos Stam，Alias Systems

Jos Stam 是 Alias 系统的研究科学家。他具有多伦多大学计算机科学的博士头衔，对计算机图形的很多领域都感兴趣。他是 SIGGRAPH 和其他会议的定期撰稿人。他在以物理学为基础的动画、渲染、纹理映射和细分表面等方面进行了很好的研究。

Marc Stevens，Softimage

Marc Stevens 具有布朗大学计算机科学硕士学位，在图形业有超过 12 年的工作经验。Marc 现在受雇于 Softimage/AVID，过去 9 年他在那里的软件开发中担任过各种不同职务。他曾是 Softimage|XSI 的主要设计师之一，现在是 Softimage 的研究和开发及特别项目的主管。

Thilaka S. Sumanaweera，Siemens Medical Solutions USA，Inc.

自 1980 年以来，Thilaka Sumanaweera 就开始对 GL、后来对 OpenGL 感兴趣，他在计算机视频、图像处理和医学图像方面创建过从 2D 到 4D 的应用程序。1992 年他获得斯坦福大学电机工程博士头衔，然后在斯坦福的放射线科学实验室进行博士后课题研究，之后担任副研究员，开发了 CT/MRI 混合图像和神经外科图像指南。他目前是 Siemens Medical Solutions

（西门子医学解决方案）在美国公司超声波分部的特别会员，他具有多项医学图像和可视性的专利，并且在医学杂志中发表过许多文章。

Kiril Vidimce，Pixar Animation Studios

Kiril Vidimce 是 Pixar 研究和开发组的图形软件工程师，目前正在研制下一代的渲染技术。此前，他曾在密西西比州立大学可视实验室，在 David Banks 教授的指导下，制作了 3D 图形并发表了文章。Kiril 的工作已经发表在 SIGGRAPH、IEEE Visualization、IEEE 计算机图形与应用、图形接口等杂志上。他曾在 SGI、SRI 获得了计算机科学和数学学士，在加州理工学院完成了实习医师学位。他来自马其顿的首都和最大城市斯科普里，现在住在加州旧金山。

内 容 提 要

本书由引领全球计算机图形芯片技术发展的NVIDIA公司组稿，汇集当今国际上前沿开发者们经多年研究和实践得出的实用的实时图形技术成果。本书主要基于当今图形处理器（GPU）的可编程图形管线，除了讲述创建高级视觉效果的基础知识和性能优化技术外，还着重介绍了前沿开发者们总结出来的快速技巧。全书由 42 篇文章组成，根据主题不同分为 6 个部分，包括自然效果、光照和阴影、材质、图像处理、性能及实践、超越三角形。

本书适合于实时渲染相关行业的技术人员阅读，也适合作为相关科研院所的学习研究参考资料。

译 者 序

在翻译本书时，正值 NVIDIA 公司的 GEFORCE 6800 系列显卡在中国发布。GPU 发展到今天已经基本具备了 CPU 运算的大部分功能，32bit 浮点精度、分支循环以及条件判断等也在 Vertex & Pixel Shader 中实现。20 世纪 90 年代末 VOODOO 显卡进入中国，10 多年来 PC 平台上的实时渲染（Realtime Rendering）软件技术框架已经从纯 CPU 上软件优化的光栅化渲染经过硬件光栅化加速渲染发展到 GPU 渲染。实时渲染的主要应用领域，如 3D 电子游戏、虚拟漫游的软件设计思路，已经从软件优化为主导过渡到围绕 GPU 硬件功能，以充分发挥 3D 硬件机能为目的编写渲染架构及设计数据结构上。实时渲染越来越朝着离线渲染的电影级质量迈进，这和 3D 渲染硬件的飞速发展是分不开的。在我国，实时渲染的理论研究与行业实践都不算发达，GPU Gems 这本书的引进是一件非常有意义的事情，使我国实时渲染相关行业的众多技术人员得以及时跟踪学习国外先进技术的发展，这将有助于提高我国实时渲染相关行业软件的质量和从业技术人员水平。译者在实时渲染方面艰苦实践了 5 年，深知一本代表国外先进技术的书籍对于专业人员的重要性。在国内，代表国际先进水平实时渲染技术方面的书籍基本为零，人民邮电出版社引进的这本书，及时地填补了这方面的空白。

姚勇

2005 年 10 月

目　　录

第 1 部分　自然效果

简介 …… 2

第 1 章　用物理模型进行高效的水模拟 …… 4

1.1　目标和范围 …… 4
1.2　正弦近似值的加和 …… 5
1.2.1　波的选择 …… 5
1.2.2　法线和切线 …… 6
1.2.3　几何波 …… 8
1.2.4　纹理波 …… 11
1.3　编辑 …… 13
1.3.1　深度的使用 …… 13
1.3.2　重载 …… 14
1.3.3　边长的过滤 …… 15
1.3.4　纹理坐标 …… 15
1.4　运行时的处理 …… 16
1.4.1　凹凸环境映射参数 …… 16
1.4.2　顶点和像素的处理 …… 18
1.5　小结 …… 19
1.6　参考文献 …… 19

第 2 章　水刻蚀的渲染 …… 21

2.1　引言 …… 21
2.2　刻蚀的计算 …… 22
2.3　方法 …… 24
2.4　使用 OpenGL 实现 …… 25
2.5　使用高级着色语言实现 …… 25
2.6　小结 …… 30
2.7　参考文献 …… 30

第 3 章　Dawn 演示中的皮肤 …… 31

3.1　引言 …… 31
3.2　皮肤着色 …… 32
3.3　场景的照明 …… 32

3.3.1 高动态范围的环境 …… 32
3.3.2 遮挡 …… 34
3.4 皮肤如何对光进行响应 …… 35
3.5 实现 …… 36
3.5.1 顶点 Shader …… 36
3.5.2 像素 Shader …… 41
3.6 小结 …… 43
3.7 参考文献 …… 43

第 4 章 Dawn 演示中的动画 …… 44

4.1 简介 …… 44
4.2 网格的动画 …… 45
4.3 变形网格对象 …… 45
4.3.1 高级语言中的变形网格对象 …… 45
4.3.2 变形网格对象的实现 …… 47
4.4 蒙皮 …… 48
4.5 小结 …… 50
4.6 参考文献 …… 50

第 5 章 改良的 Perlin 噪声的实现 …… 51

5.1 噪声函数 …… 51
5.2 最初的实现 …… 52
5.3 最初实现的缺点 …… 52
5.4 对噪声函数的改进 …… 54
5.5 如何在像素 shader 中产生好的假噪声 …… 56
5.6 不考虑相邻顶点制作凹凸贴图 …… 57
5.7 小结 …… 58
5.8 参考文献 …… 58

第 6 章 Vulcan 演示中的火 …… 59

6.1 创建逼真的火焰 …… 59
6.2 动画精灵的实现 …… 61
6.2.1 火焰和烟的动画 …… 61
6.2.2 使火焰增加多样性 …… 62
6.2.3 动画的存储 …… 63
6.2.4 火焰和烟的混合 …… 64
6.3 粒子运动 …… 65
6.4 性能 …… 65
6.4.1 层次合成 …… 65

6.4.2 定制的精灵 …… 67
6.5 渲染后的效果 …… 67
6.5.1 辉光 …… 68
6.5.2 热微光 …… 68
6.5.3 颗粒 …… 70
6.5.4 最终的程序 …… 71
6.6 小结 …… 72

第 7 章 无数波动草叶的渲染 …… 73

7.1 引言 …… 73
7.2 概述 …… 73
7.3 草体的准备 …… 74
7.3.1 草的纹理 …… 74
7.3.2 草体 …… 74
7.4 动画 …… 76
7.4.1 一般思路 …… 76
7.4.2 每丛草体的动画 …… 77
7.4.3 每个顶点的动画 …… 79
7.4.4 每个草体的动画 …… 80
7.5 小结 …… 82
7.6 参考文献 …… 82

第 8 章 衍射的模拟 …… 84

8.1 什么是衍射 …… 84
8.1.1 波动光学 …… 84
8.1.2 衍射的物理学 …… 85
8.2 实现 …… 86
8.3 结果 …… 89
8.4 小结 …… 90
8.5 参考文献 …… 90

第 2 部分 光照和阴影

简介 …… 92

第 9 章 有效的阴影体渲染 …… 94

9.1 引言 …… 94
9.2 程序结构 …… 96
9.2.1 多遍渲染 …… 96

9.2.2 顶点缓冲器结构 …… 99
9.2.3 在无限远处工作 …… 99
9.3 详细的讨论 …… 101
9.3.1 数学 …… 101
9.3.2 代码 …… 103
9.3.3 markShadows 方法 …… 103
9.3.4 findBackfaces 方法 …… 104
9.3.5 亮罩和暗罩 …… 105
9.3.6 侧面 …… 106
9.4 调试 …… 107
9.5 几何优化 …… 108
9.5.1 方向光 …… 108
9.5.2 点光源和聚光灯 …… 108
9.5.3 剔除阴影体 …… 109
9.5.4 除罩操作 …… 109
9.6 填充率的优化 …… 110
9.6.1 有限的体积 …… 110
9.6.2 XY 裁剪 …… 111
9.6.3 Z-边界 …… 111
9.7 将来的阴影 …… 112
9.8 参考文献 …… 113

第 10 章 电影级的光照 …… 114

10.1 引言 …… 114
10.2 直射光照明模型 …… 115
10.2.1 选择 …… 116
10.2.2 颜色 …… 116
10.2.3 造型 …… 116
10.2.4 阴影 …… 117
10.2.5 纹理 …… 118
10.2.6 结果 …… 118
10.3 泛光 Shader …… 119
10.4 性能分析 …… 124
10.4.1 速度 …… 124
10.4.2 开销 …… 124
10.4.3 优化 …… 124
10.5 小结 …… 124
10.6 参考文献 …… 125

第 11 章　阴影贴图反走样 …… 126

11.1　引言 …… 126
11.2　靠近的百分比过滤 …… 126
11.3　平滑滤波的实现 …… 127
11.4　较少地取样 …… 128
11.5　工作原理 …… 129
11.6　小结 …… 131
11.7　参考文献 …… 131

第 12 章　全方位的阴影映射 …… 132

12.1　引言 …… 132
12.1.1　模板阴影 …… 133
12.1.2　阴影映射 …… 133
12.2　阴影映射的算法 …… 133
12.2.1　条件 …… 133
12.2.2　算法 …… 134
12.2.3　纹理格式 …… 135
12.2.4　阴影贴图的尺寸 …… 135
12.2.5　几何体的数值范围 …… 135
12.3　实现 …… 135
12.3.1　系统需求 …… 135
12.3.2　资源创建 …… 136
12.3.3　渲染阶段 1：渲染到阴影贴图 …… 136
12.3.4　渲染阶段 2：基本渲染 …… 137
12.3.5　光照计算 …… 137
12.3.6　阴影的计算 …… 137
12.3.7　技巧和窍门 …… 138
12.3.8　最终的着色遍（Lighting × Shadow） …… 138
12.4　添加模糊的阴影 …… 138
12.5　小结 …… 139
12.6　参考文献 …… 139

第 13 章　使用遮挡区间映射产生模糊的阴影 …… 140

13.1　加油站 …… 140
13.2　算法 …… 141
13.3　创建映射 …… 142
13.4　渲染 …… 143
13.5　局限性 …… 144

13.6 小结 145
13.7 参考文献 146

第 14 章 透视阴影贴图 147

14.1 引言 147
14.2 PSM 算法的问题 148
14.2.1 虚拟摄像机 148
14.2.2 光源摄像机 152
14.2.3 偏置 157
14.3 获得更好阴影映射的技巧 160
14.3.1 过滤器 160
14.3.2 模糊 161
14.4 结果 164
14.5 参考文献 165

第 15 章 逐像素光照的可见性管理 166

15.1 GPU 书中的可见性 166
15.2 批和逐像素光照 166
15.2.1 逐像素光照的例子 166
15.2.2 究竟需要多少批 167
15.3 作为集合的可见性 168
15.3.1 可见集合 168
15.3.2 光源集合 168
15.3.3 照明集合 168
15.3.4 阴影集合 168
15.4 各集合的生成 169
15.4.1 可见集合的生成 169
15.4.2 光源集合的生成 169
15.4.3 照明集合的生成 169
15.4.4 阴影集合的生成 170
15.5 可见性改善填充率 172
15.6 实际的应用 173
15.7 小结 173
15.8 参考文献 173

第 3 部分 材质

简介 176

第 16 章　次表面散射的实时近似 …… 177

16.1　次表面散射的视觉效果 …… 177
16.2　简单的散射近似 …… 177
16.3　用深度映射模拟吸收 …… 179
16.3.1　实现细节 …… 182
16.3.2　更精密的散射模型 …… 183
16.4　纹理空间的漫反散 …… 183
16.5　小结 …… 187
16.6　参考文献 …… 187

第 17 章　环境遮挡 …… 188

17.1　概述 …… 188
17.2　预处理步骤 …… 189
17.3　硬件加速计算遮挡 …… 190
17.4　用环境遮挡贴图来渲染 …… 191
17.5　小结 …… 194
17.6　参考文献 …… 195

第 18 章　空间的 BRDFs …… 198

18.1　什么是 SBRDF …… 198
18.2　表达式的详述 …… 198
18.3　使用离散光的渲染 …… 200
18.4　使用环境贴图的渲染 …… 202
18.4.1　算法 …… 202
18.4.2　shader 代码 …… 204
18.5　小结 …… 207
18.6　参考文献 …… 207

第 19 章　基于图像的光照 …… 208

19.1　基于图像光照的局部化 …… 208
19.2　顶点 Shader …… 211
19.3　片元 Shader …… 213
19.4　漫反射 IBL …… 215
19.5　影子 …… 215
19.6　使用局部立方体贴图作背景 …… 216
19.7　小结 …… 217
19.8　参考文献 …… 217

第 20 章 纹理爆炸 ····· 219

20.1 纹理爆炸 101 ····· 219
20.1.1 求单元 ····· 219
20.1.2 对图像采样 ····· 220
20.1.3 相邻单元中的图像 ····· 220
20.1.4 图像优先级 ····· 221
20.1.5 程序化图像 ····· 222
20.1.6 图像的随机选择 ····· 223
20.2 技术上的考虑 ····· 224
20.3 高级特性 ····· 225
20.3.1 缩放和转动 ····· 225
20.3.2 可控的变量密度 ····· 225
20.3.3 程序化的 3D 爆炸 ····· 226
20.3.4 随时间变化的纹理 ····· 227
20.3.5 Voronoi 相关的细胞法 ····· 227
20.4 小结 ····· 229
20.5 参考文献 ····· 229

第 4 部分 图像处理

简介 ····· 232

第 21 章 实时辉光 ····· 234

21.1 技术概述 ····· 234
21.2 渲染辉光的步骤 ····· 237
21.2.1 辉光源的指定和渲染 ····· 237
21.2.2 模糊辉光源 ····· 238
21.2.3 分步卷积 ····· 238
21.2.4 GPU 上的卷积 ····· 239
21.3 特定硬件的实现 ····· 240
21.3.1 Direct3D 9 ····· 240
21.3.2 Direct3D 8 ····· 242
21.3.3 Direct3D 7 ····· 242
21.4 模糊的其他用途 ····· 243
21.5 把效果加入一个游戏引擎 ····· 243
21.5.1 渲染场景 ····· 243
21.5.2 走样问题 ····· 244
21.5.3 DirectX 7 的精度问题 ····· 244

21.5.4 残留图像效应 …… 245
21.5.5 渐变效果 …… 245
21.6 小结 …… 246
21.7 参考文献 …… 246

第 22 章 颜色控制 …… 248

22.1 引言 …… 248
22.2 基于通道的颜色校正 …… 248
22.2.1 级别 …… 248
22.2.2 曲线 …… 250
22.3 多通道的彩色校正和变换 …… 252
22.3.1 灰度变换 …… 252
22.3.2 彩色空间的变换 …… 253
22.4 参考文献 …… 255

第 23 章 景深：技术综述 …… 256

23.1 什么是景深 …… 256
23.1.1 模糊圈的计算 …… 257
23.1.2 主要技术 …… 257
23.2 光线跟踪的景深 …… 258
23.3 累积缓冲区的景深 …… 258
23.4 分层的景深 …… 259
23.5 向前映射的 z 缓冲区景深 …… 260
23.6 反向映射的 z 缓冲区景深 …… 261
23.7 小结 …… 265
23.8 参考文献 …… 266

第 24 章 高质量的过滤 …… 267

24.1 质量与速度 …… 267
24.2 对 GPU 求导的理解 …… 277
24.3 解析的反走样和纹理化 …… 278
24.4 小结 …… 284
24.5 参考文献 …… 284

第 25 章 用纹理贴图进行快速过滤宽度的计算 …… 285

25.1 在 shader 中求导的需求 …… 285
25.2 用纹理计算过滤宽度 …… 287
25.3 讨论 …… 288
25.4 参考文献 …… 289

第 26 章 OpenEXR 图像文件格式 ····· 291

26.1 什么是 OpenEXR ····· 291
26.1.1 高动态范围图像 ····· 291
26.1.2 “半精度”（Half）格式 ····· 293
26.1.3 可表示的数值范围 ····· 293
26.1.4 彩色分辨率 ····· 294
26.1.5 C++接口 ····· 294
26.2 OpenEXR 文件结构 ····· 294
26.2.1 文件头 ····· 294
26.2.2 像素 ····· 294
26.3 OpenEXR 数据压缩 ····· 295
26.4 OpenEXR 的使用 ····· 295
26.4.1 OpenEXR 图像的读和显示 ····· 295
26.4.2 一个 OpenEXR 图像的渲染和写入 ····· 296
26.5 线性像素值 ····· 300
26.6 创建和使用 HDR 图像 ····· 302
26.7 小结 ····· 303
26.8 参考文献 ····· 304

第 27 章 图像处理的框架 ····· 305

27.1 引言 ····· 305
27.2 框架设计 ····· 306
27.2.1 操作器和过滤器 ····· 306
27.2.2 图像数据 ····· 307
27.2.3 丢失的块 ····· 308
27.3 实现 ····· 310
27.3.1 Image 类 ····· 311
27.3.2 ImageFilter 类 ····· 314
27.3.3 过滤的实现 ····· 315
27.4 一个示例应用程序 ····· 318
27.5 性能和局限性 ····· 319
27.6 小结 ····· 320
27.7 参考文献 ····· 321

第 5 部分 性能及实践

简介 ····· 324

第 28 章　图形流水线性能 …… 326

28.1　概述 …… 326
28.1.1　流水线 …… 326
28.1.2　方法 …… 326
28.2　定位瓶颈 …… 327
28.2.1　光栅操作 …… 328
28.2.2　纹理带宽 …… 328
28.2.3　片元着色 …… 328
28.2.4　顶点处理 …… 329
28.2.5　顶点和索引传输 …… 329
28.3　优化 …… 329
28.3.1　在 CPU 上优化 …… 329
28.3.2　减少顶点传输的开销 …… 330
28.3.3　顶点处理的优化 …… 331
28.3.4　加速片元着色 …… 331
28.3.5　减小纹理带宽 …… 332
28.3.6　优化帧缓冲带宽 …… 333
28.4　小结 …… 334
28.5　参考文献 …… 334

第 29 章　有效的遮挡剔除 …… 335

29.1　什么是遮挡剔除 …… 335
29.1.1　遮挡查询 …… 335
29.1.2　早期 z 值拒绝 …… 335
29.2　遮挡查询如何工作 …… 336
29.3　初步使用遮挡查询 …… 336
29.3.1　恰当地使用遮挡查询 …… 337
29.3.2　遮挡物和被遮挡物的比较 …… 337
29.4　更进一步的应用 …… 337
29.4.1　将物体排序 …… 339
29.4.2　一个防止误解的说明 …… 339
29.5　关于包围盒 …… 339
29.5.1　静态的物体 …… 340
29.5.2　动画的物体 …… 340
29.6　其他问题 …… 341
29.6.1　CPU 消耗太高 …… 341
29.6.2　高分辨率的渲染 …… 341
29.6.3　快速深度写入的性能 …… 342

29.6.4 锥体剔除 …… 342
29.7 一点小忠告 …… 343
29.8 一个应用：透镜耀斑 …… 343
29.8.1 渲染透镜耀斑的旧方法 …… 344
29.8.2 渲染透镜耀斑的新方法 …… 345
29.9 小结 …… 345
29.10 参考文献 …… 346

第 30 章 FX Composer 的设计 …… 347

30.1 工具的开发 …… 347
30.2 设计初衷和使用对象 …… 347
30.3 对象设计 …… 348
30.4 文件格式 …… 352
30.5 用户接口 …… 353
30.6 Direct3D 图形的实现 …… 353
30.6.1 设备窗口 …… 353
30.6.2 Direct3D 效果 …… 354
30.6.3 ID3DXEffectCompiler …… 354
30.6.4 ID3DXEffect …… 355
30.7 场景管理 …… 355
30.8 小结 …… 356
30.9 参考文献 …… 356

第 31 章 FX Composer 的使用 …… 357

31.1 开始 …… 357
31.1.1 材质面板 …… 358
31.1.2 场景图形面板 …… 359
31.1.3 编辑窗口 …… 360
31.1.4 Shader Perf 面板 …… 361
31.1.5 属性面板 …… 361
31.1.6 场景面板 …… 363
31.1.7 纹理面板 …… 364
31.1.8 任务面板 …… 365
31.1.9 日志面板 …… 365
31.2 项目示例 …… 366
31.3 小结 …… 367

第 32 章 Shader 接口入门 …… 368

32.1 shader 接口的基础 …… 369

32.2 一个灵活的光源描述 …… 371
32.3 材质树 …… 373
32.4 小结 …… 376
32.5 参考文献 …… 376

第 33 章 将产品的 RenderMan shader 转化为实时的 shader …… 377

33.1 引言 …… 377
33.2 光照 …… 378
33.2.1 光源 …… 378
33.2.2 光源 shader …… 378
33.2.3 其他的光源参数 …… 379
33.3 顶点程序与片元程序的比较 …… 379
33.4 使用顶点和片元程序 …… 380
33.5 片元程序的优化技术 …… 381
33.5.1 把代码转移到应用层 …… 381
33.5.2 把代码转移到顶点程序 …… 381
33.5.3 通过纹理查询优化 …… 382
33.5.4 向量化的优化 …… 383
33.5.5 最终的优化 …… 383
33.6 小结 …… 384
33.7 参考文献 …… 386

第 34 章 将硬件着色整合进 Cinema 4D …… 387

34.1 引言 …… 387
34.2 把 Cinema 4D 连接到 CgFX 上去 …… 389
34.3 shader 和参数管理 …… 390
34.4 模拟离线渲染 …… 391
34.5 结果和性能 …… 393
34.6 收获和教训 …… 394
34.7 参考文献 …… 395

第 35 章 在实时应用程序中使用高质软件渲染效果 …… 396

35.1 引言 …… 396
35.2 用于硬件渲染的内容流水线 …… 397
35.3 硬件渲染的组件 …… 398
35.3.1 几何数据 …… 398
35.3.2 属性映射 …… 398
35.4 组件的产生 …… 400
35.4.1 创建几何图形 …… 400

35.4.2 对纹理和顶点的渲染 …… 400
35.5 试验情况和结果 …… 403
35.6 小结 …… 408
35.7 参考文献 …… 408

第 36 章 将 Shader 整合到应用程序中去 …… 409

36.1 引言 …… 409
36.2 关于 shader …… 409
36.3 一个 effect 文件的剖析 …… 411
36.3.1 变量 …… 412
36.3.2 结构体 …… 412
36.3.3 pass …… 412
36.3.4 technique …… 412
36.3.5 评注 …… 412
36.4 shader 数据的类型 …… 413
36.4.1 场景信息 …… 413
36.4.2 材质 …… 414
36.4.3 渲染的场景 …… 414
36.4.4 顶点数据 …… 414
36.5 与 shader 的通信 …… 414
36.5.1 场景信息 …… 414
36.5.2 材质的参数 …… 415
36.5.3 顶点格式 …… 416
36.5.4 场景 …… 416
36.5.5 就场景对 technique 和 pass 的比较 …… 417
36.6 effect 文件格式的扩展 …… 417
36.6.1 对预处理程序的支持 …… 417
36.6.2 对 shader 变化的支持 …… 418
36.6.3 shader 继承的添加 …… 418
36.7 小结 …… 419
36.8 参考文献 …… 419

第 6 部分 超越三角形

简介 …… 422

第 37 章 用于 GPU 计算的工具箱 …… 424

37.1 用 GPU 进行计算 …… 424
37.1.1 编程模型 …… 425

37.1.2 并行编程 …… 426
37.1.3 高级的 GPU 程序 …… 426
37.2 约减 …… 426
37.2.1 并行的约减 …… 427
37.2.2 有关约减的注意事项 …… 428
37.3 排序和搜索 …… 428
37.3.1 Bitonic 归并排序 …… 428
37.3.2 二分搜索 …… 430
37.4 挑战 …… 432
37.4.1 有限的输出 …… 432
37.4.2 缓慢的回读 …… 433
37.4.3 GPU 和 CPU 的比较 …… 433
37.5 小结 …… 433
37.6 参考文献 …… 433

第 38 章 在 GPU 上的快速流体动力学模拟 …… 435

38.1 引言 …… 435
38.1.1 目的 …… 436
38.1.2 假设 …… 436
38.1.3 方法 …… 436
38.2 数学背景 …… 436
38.2.1 不可压缩流体的 Navier-Stokes 方程式 …… 437
38.2.2 Navier-Stokes 方程式的各项 …… 438
38.2.3 矢量微积分的简要复习 …… 438
38.2.4 解 Navier-Stokes 方程式 …… 439
38.3 实现 …… 443
38.3.1 CPU-GPU 的类比 …… 444
38.3.2 片运算 …… 445
38.3.3 片元程序的实现 …… 445
38.4 应用 …… 450
38.4.1 模拟液体和气体 …… 450
38.4.2 浮力和对流 …… 450
38.5 扩展 …… 451
38.5.1 旋涡状态的限制 …… 452
38.5.2 三维 …… 452
38.5.3 交错排列的网格 …… 452
38.5.4 任意边界 …… 452
38.5.5 流体的自由表面 …… 453
38.6 小结 …… 453

38.7 参考文献 453

第 39 章 体渲染技术 454

39.1 引言 454
39.2 体渲染 455
39.3 基于纹理的体渲染 456
39.4 实现细节 459
39.4.1 数据的表达和处理 459
39.4.2 代理几何体 460
39.4.3 渲染 461
39.5 高级技术 463
39.5.1 体光照 463
39.5.2 程序化渲染 467
39.6 对性能的考虑 467
39.7 小结 469
39.8 参考文献 469

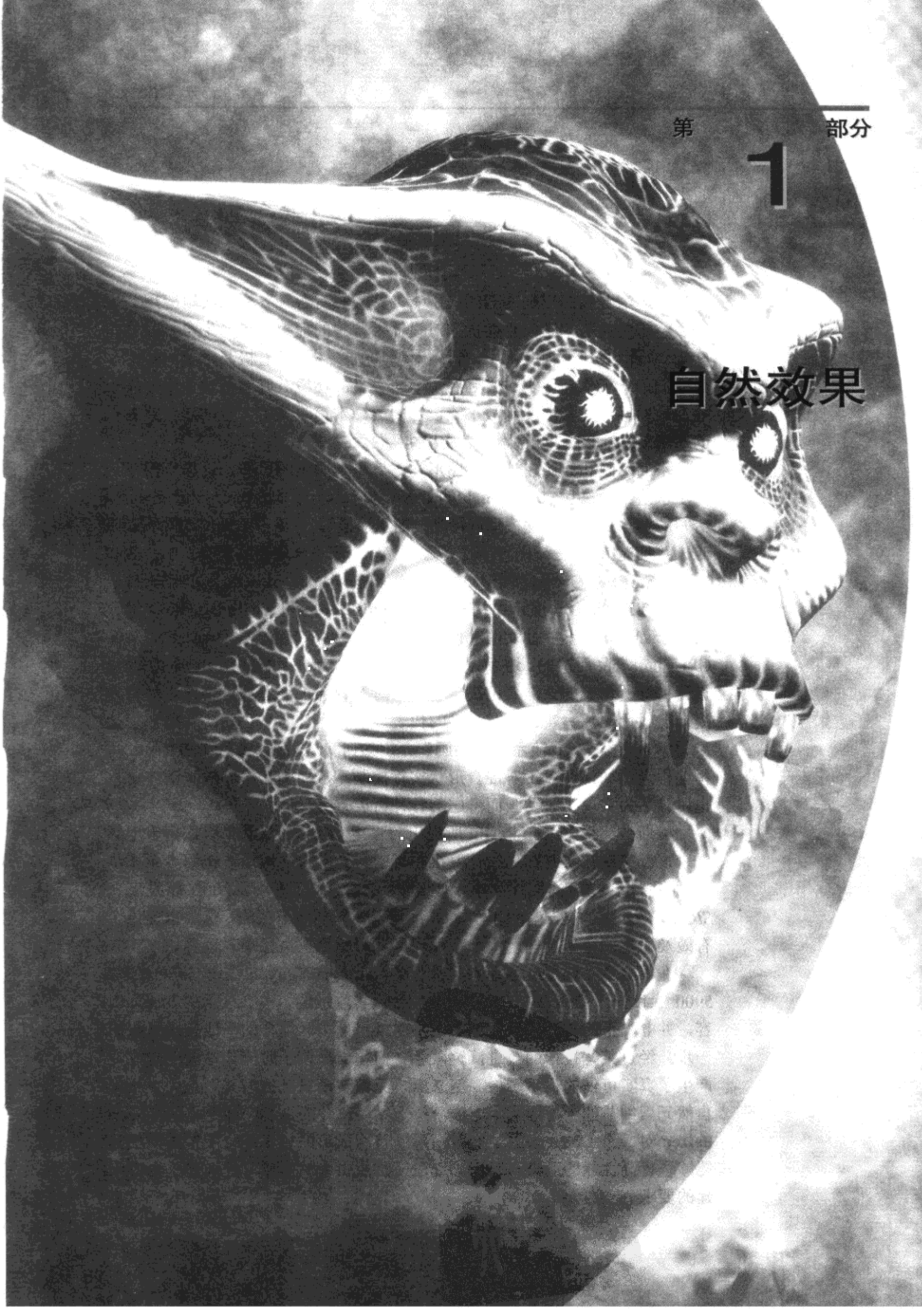
第 40 章 用于三维超声波可视化的实时着色 471

40.1 背景 471
40.2 引言 473
40.2.1 笛卡尔网格数据的体渲染 473
40.2.2 体渲染锥体网格中的数据 475
40.3 结果 480
40.4 小结 480
40.5 参考文献 480

第 41 章 实时立体图 481

41.1 什么是立体图 481
41.1.1 立体摄影 481
41.1.2 随机点立体图 481
41.1.3 单个图像的立体图 483
41.2 单个图像立体图的创建 484
41.2.1 参数 484
41.2.2 渲染 485
41.2.3 动画的单个图像立体图的创建 486
41.2.4 片元程序 488
41.3 示例应用程序 489
41.4 参考文献 490

第 42 章　变形 ····· 491
42.1　什么是变形 ····· 491
42.2　在 GPU 上的变形 ····· 492
42.2.1　变形的公式 ····· 492
42.2.2　写顶点程序 ····· 492
42.2.3　法线的变形 ····· 493
42.3　局限性 ····· 494
42.4　性能 ····· 495
42.5　例子：波浪变形 ····· 495
42.6　小结 ····· 497



第 1 部分

自然效果

简 介

纵观消费级图形加速器的应用史，特效具有各种各样的实时应用；更重要的是，它们帮助玩家对设计者预想的虚拟世界产生身临其境的感觉。在游戏和可视应用中，特效对增加玩家体验起着极其重要的作用，为他们描绘了从超现实主义到照片写实主义的环境。好的特效使玩家终止他们的疑惑，使其沉浸到故事或设定中去。本篇收集了大量创建自然效果的实用技术，以帮助解决这些历史的难题。

水的动画和光照在计算机图形学中属于最困难的工作，这里有两章是关于水渲染的。由 **Mark Finch** 的 **Cyan Worlds**（经典游戏 *Myst* 的制作者）所写的**第 1 章“用物理模型进行高效的水模拟”**，探索了水面的动画和光照明，并且提出改进反射的有用技巧。在**第 2 章“水刻蚀的渲染”**中，我和 **Daniel Sanchez-Crespo** 指出如何使用类似的水动画基础，渲染可信的刻蚀。

第 3 章“Dawn 演示中的皮肤”，**Curtis Beeso** 和 **Kevin Bjorke** 详细介绍用于 *Dawn* 演示中仙女的着色技术，这是为 NVIDIA GeForce FX 5800 GPU 上市产品而写的。这章为开发过程提供了有价值的观点——特别是由设计目标所影响的关键着色决定——当时 NVIDIA 创造的这个电影品质的演示已成为照片写实、实时渲染的代表。

第 4 章“Dawn 演示中的动画”介绍如何实现动画。**Curtis Beeson** 解释编程人员如何使美工对混合形状实行控制，从而创建不同的表情。这章也讨论了为实现实时动画而考虑的各种折衷方案。

奥斯卡奖得主 **Ken Perlin** 所提出的噪声（Noise）算法自首次用于电影以来，已屡次在实时和离线计算机图形中得到多方面应用。在**第 5 章“改良的 Perlin 噪声的实现”**中，Ken 详细阐述最新进展，2002 年发表在 *SIGGRAPH* 上的文章，纠正了 Ken 最初工作中的两个缺陷。这章也提供了有效及稳定的框架结构，用于在现代可编程硬件上执行噪声运算。

在**第 6 章“Vulcan 演示中的火”**中，**Hubert Nguyen** 描述 GeForce FX 5900 上市演示 *Vulcan* 中火的渲染。尽管其效果也如本书这章的大部分内容，并非真正的物理模拟，但的确是对《指环王》那样的离线技术紧跟的一步。经过某些技术的改进，突破了光栅化大量粒子时操作性能的限制，产生了真实可信的火焰图像。**第 7 章“无数波动草叶的渲染”**是关于巨量自然元素细节的。为应战“无垠草原上晃动草叶的描绘”这个难题，**Kurt Pelzer** 对一个 Codecretures 实时演示中首次成形的技术进行了扩展，使其能够进行高性能的渲染，以更好地适应游戏引擎的需要。他详述了内容创建的要求。

最后，在**第 8 章“衍射的模拟”**中，**Jos Stam** 考虑了更精密的细节，比如压缩光盘上的刻线。这章讲述简化的 Jos 衍射光照模型（它最初发表在 *SIGGRAPH* 1999 上）。这个模型以光的物理性质为基础，把光当作波建模，从而创建出多彩的干涉条纹。

Juan Guardado, NVIDIA

第 1 章 用物理模型进行高效的水模拟

Mark Finch
Cyan Worlds

本章介绍在 GPU 中模拟和渲染大的水体的系统。它把基本网格的几何波动与动态法线贴图的生成结合起来。经证明这个系统适用于实时游戏的场合，并且已经大量地应用于 Cyan World 的 *Uru : Age Beyond Myst*，见图 1-1。

图 1-1 安静的池塘
一个取自 *Uru : Ages Beyond Myst* 的场景

1.1 目标和范围

近几年，实时渲染技术渐渐从离线渲染的世界脱离出来。正如 2001 年 Tessendorf 所述的那样，快速傅立叶变换（FFT）技术在大尺寸取样栅格中产生了难以置信的逼真效果；在消费级的 PC 上，对中等大小的取样栅格也可以进行实时处理。还能够基于体素（Voxel）对简化的 Navier-Stokes 方程求解（Yann 2003）。虽然尚未达到 Enright 等人 2002 年创建的离线流体

模拟那样的尖端水平，但是差距正在缩小。在本章发表之时，傅立叶变换（FFT）库将能适用于顶点 shader 和像素 shader，但是写作本文时这些技术的实时版本也还只限于 CPU 的执行。

同时，在 GPU 上运行的水模拟简单模型也得到了改进。Isidoro 等人 2002 年提出在一个 vertex shader 中加和 4 个正弦波以计算水面的高度和方位。Laeuchli 在 2002 年发表一个 shader，使用 3 个 Gerstner 波计算水面高度。

我们由计算简单正弦函数之和开始，逐步扩展到更复杂的函数。我们也将技术扩展到像素 shader 领域，使用周期波函数的加和，来创建动态的平铺（tiling）凹凸贴图，从而获得水面的细节。

本章集中解释了系统参数的物理意义，说明用正弦波的和来近似水面，并不像通常认为的那么非同一般。我们对从基本模型引到实际操作的数学予以特别的关注，因为它是扩展执行的关键。

这个系统设计的水体是从小海湾或岛屿观看的水体，所包含的范围从小池塘一直到大海。它虽然还不是严格的物理学模拟，但也算得上是令人信服的、灵活的和动态的水渲染。模拟完全是在 GPU 上运行的，因此它在人工智能和物理过程两方面都不会与 CPU 发生竞争。因为系统参数具有物理意义，脚本的书写更容易，无须经过试验和误差来发现参数。除了构成系统的波是动态的以外，整个系统动态化将会增加额外的活力。

1.2　正弦近似值的加和

我们需要运行两个表面模拟：一个是表面网格的几何波动，另一个是网格上法线图的扰动。两个模拟本质上是相同的。水面高度由简单的周期波叠加表示。我们从求正弦函数之和开始，接着将接触到更有趣的波形。

正弦函数叠加后得到了一个连续函数，这个函数描述了水面上所有点的高度和方向。在处理顶点时，我们基于每个顶点的水平位置对函数取样，使得网格细分形成连续水面。在几何分辨率之下，将该技术继续应用于纹理空间。通过对近似正弦叠加的法线取样，用简单像素 shader 渲染到渲染目标纹理（render target texture），从而产生表面的法线图。为每帧渲染法线图，允许有限数量的正弦波组相互独立地运动，这大大地提高了渲染的逼真度。

事实上，水纹理的波纹好坏决定着模拟的逼真度。波纹表面的几何波动提供呈现纹理的精细框架结构。于是，对纹理波纹我们有不同的几何选择标准。

1.2.1　波的选择

这里需要一个参数组来定义每个波。如图 1-2 所示，参数如下。

- 波长（L）：世界空间中波峰到波峰之间的距离。波长 L 与角频率 ω 的关系为 $\omega = 2\pi / L$。
- 振幅（A）：从水平面到波峰的高度。
- 速度（S）：每秒钟波峰移动的距离。为了方便，把速度表示成相常数 $\varphi = S \times 2\pi/L$。
- 方向（D）：垂直于波阵面的水平向量。波阵面为波峰沿着它运动的面。

波的状态定义为水平位置（x，y）和时间（t）的函数：

$$W_i(x, y, t) = A_i \times \sin\left(\mathbf{D}_i \cdot (x, y) \times \omega_i + t \times \varphi_i\right) \tag{1}$$

而包括所有的波 i 的总表面是：

$$H(x, y, t)=\sum\left(A_i \times \sin\left(\boldsymbol{D}_i \cdot(x, y) \times \omega_i+t \times \varphi_i\right)\right) \tag{2}$$

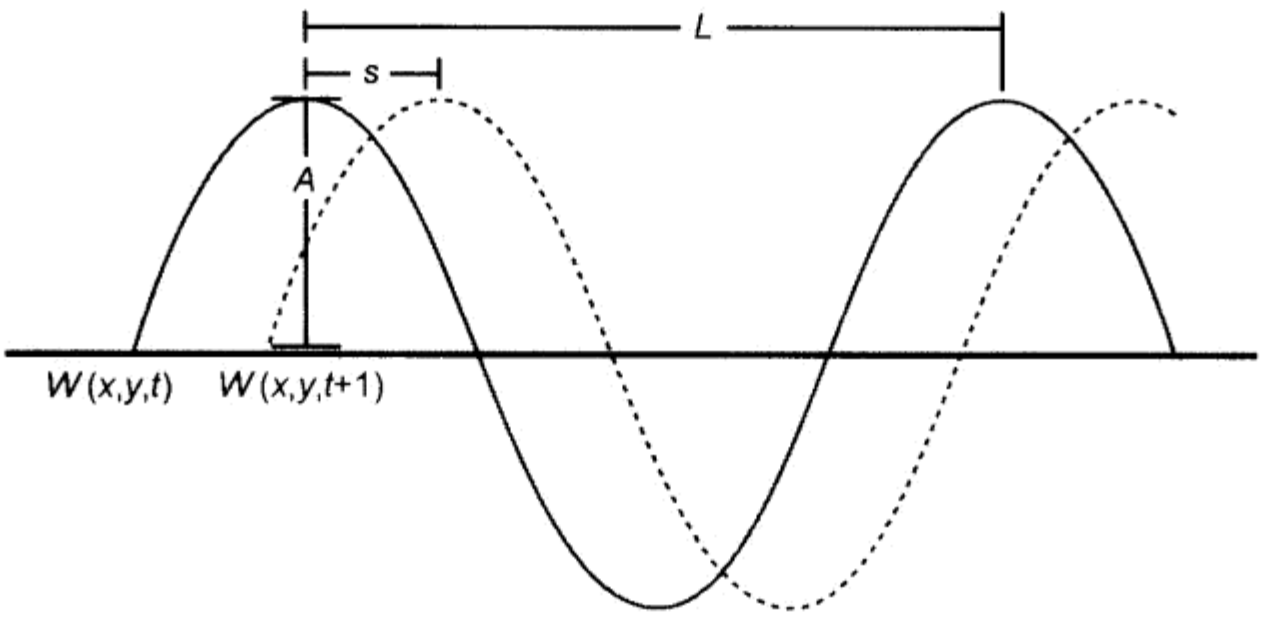


图 1-2 单个波函数的参数

为了提供场景动力学的变量，我们将在约束中随机产生这些波参数。随着时间的变化，我们会不断地将一个波淡出，然后再以一组不同的参数将其淡入。此过程中这些参数是相关的。必须仔细地产生一个完整的参数组，才能使各个波以可信的方式结合在一起。

1.2.2 法线和切线

因为表面是一个显式函数，所以能在任意给定点直接计算表面方向，不必使用有限差分技术。副法线 $\boldsymbol{B}$ 和切线 $\boldsymbol{T}$ 向量分别为 x 和 y 方向的偏导数。对在 2D 水平面中的任何点 (x, y)，在表面上的三维位置 $\boldsymbol{P}$ 是：

$$\boldsymbol{P}(x, y, t)=(x, y, H(x, y, t)) \tag{3}$$

x 方向上的偏导数是：

$$\boldsymbol{B}(x, y)=\left(\frac{\partial x}{\partial x}, \frac{\partial y}{\partial x}, \frac{\partial}{\partial x}(H(x, y, t))\right) \tag{4a}$$

$$\boldsymbol{B}(x, y)=\left(1, 0, \frac{\partial}{\partial x}(H(x, y, t))\right) \tag{4b}$$

同样地，切线向量是：

$$\boldsymbol{T}(x, y)=\left(\frac{\partial x}{\partial y}, \frac{\partial y}{\partial y}, \frac{\partial}{\partial y}(H(x, y, t))\right) \tag{5a}$$

$$\boldsymbol{T}(x, y)=\left(0, 1, \frac{\partial}{\partial y}(H(x, y, t))\right) \tag{5b}$$

法线由副法线和切线的叉积给出：

$$\boldsymbol{N}(x, y)=\boldsymbol{B}(x, y) \times \boldsymbol{T}(x, y) \tag{6a}$$

$$N(x,y)=\left(-\frac{\partial}{\partial x}(H(x,y,t)),-\frac{\partial}{\partial y}(H(x,y,t)),1\right) \tag{6b}$$

在代入函数 H 的偏导数之前，请注意公式 3～6 的形式是很方便的。两个偏导数的计算给出切线空间的 9 个基本分量。这是使用高度场近似表面的直接结果。即 $\boldsymbol{P}(x,y)\cdot x=x$ 和 $\boldsymbol{P}(x,y)\cdot y=y$ 在偏导数中变成 0 和 1。但这只是对这种高度场有意义，为通用起见，这里选择使用函数 $H(x,y,t)$。

对于在第 1.2.1 节中所述的高度函数，计算偏导数是特别方便的。因为对所有的波 i，函数加和的导数是各个函数的导数之和：

$$\begin{aligned}\frac{\partial}{\partial x}(H(x,y,t))&=\sum\left(\frac{\partial}{\partial x}(W_i(x,y,t))\right)\\&=\sum\left(\omega_i\times\boldsymbol{D}_i.x\times A_i\times\cos(\boldsymbol{D}_i\cdot(x,y)\times\omega_i+t\times\varphi_i)\right)\end{aligned} \tag{7}$$

直接叠加正弦波产生的波浪有太多的“卷”，而真实波浪的波峰比较尖，波谷比较宽。解决的办法是使用正弦函数的简单变形，从而可控地产生这个效果。我们将正弦函数修改为非负状态，并且赋予它一个指数 k。函数及其对于 x 的偏导数是：

$$W_i(x,y,t)=2A_i\times\left(\frac{\sin(\boldsymbol{D}_i\cdot(x,y)\times\omega_i+t\times\varphi_i)+1}{2}\right)^k \tag{8a}$$

$$\begin{aligned}\frac{\partial}{\partial x}(W_i(x,y,t))=k\times\boldsymbol{D}_i.x\times\omega_i\times A_i\times&\left(\frac{\sin(\boldsymbol{D}_i\cdot(x,y)\times\omega_i+t\times\varphi_i)+1}{2}\right)^{k-1}\\&\times\cos(\boldsymbol{D}_i\cdot(x,y)\times\omega_i+t\times\varphi_i)\end{aligned} \tag{8b}$$

图 1-3 显示了有幂常数 k 的函数所产生的波形。这是我们为产生纹理波所使用的函数，但是为了简化起见，我们仍然借助简单的正弦之和表示波，并且在必须考虑基本波形变化时予以标注。

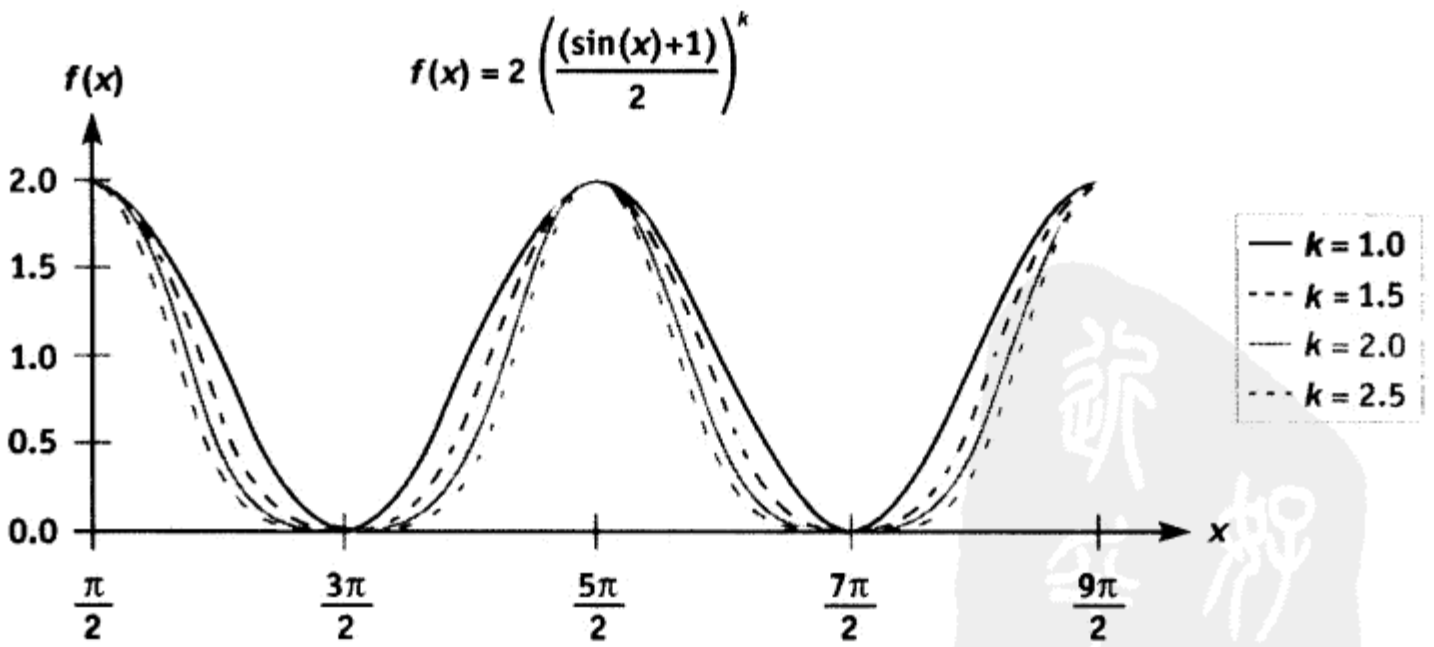


图 1-3　变化的波形
当指数 k 大于 1 时，波峰变尖，而波谷变平

1.2.3 几何波

我们把几何波限制为 4 个。添加更多的波并不能增加新概念，只不过增加更多相同的顶点 shader 处理指令和常数而已。

方向波或圆形波

我们对图 1-4 所示的方向波或圆形波进行选择。方向波需要的顶点 shader 处理指令较少，但是究竟选择何种波要取决于模拟的场景。

对于方向波，公式（1）中的每个 $\boldsymbol{D}_i$ 是波寿命常数。对于圆形波，方向必须在每个顶点计算，它是从波中心 $\boldsymbol{C}_i$ 到顶点的规范化向量：

$$\boldsymbol{D}_i(x, y) = \left(\frac{(x, y) - \boldsymbol{C}_i}{\left|(x, y) - \boldsymbol{C}_i\right|} \right)$$

对于大的水体，方向波往往更好一些，因为它们是风吹动产生的波的较好模型。对于较小的池塘中的水，产生波的原因不是由于风（例如是由于瀑布），圆形波则更好一些。圆形波还有一个很好的特性，即它们的干涉条纹从不重复。两种波的实现相当类似。对于方向波，波的方向是在风向的一定范围内任意画的；对于圆形波，波中心是在某些限定的范围内任意画的（例如瀑布击落在水面形成的线）。下面集中讨论方向波。

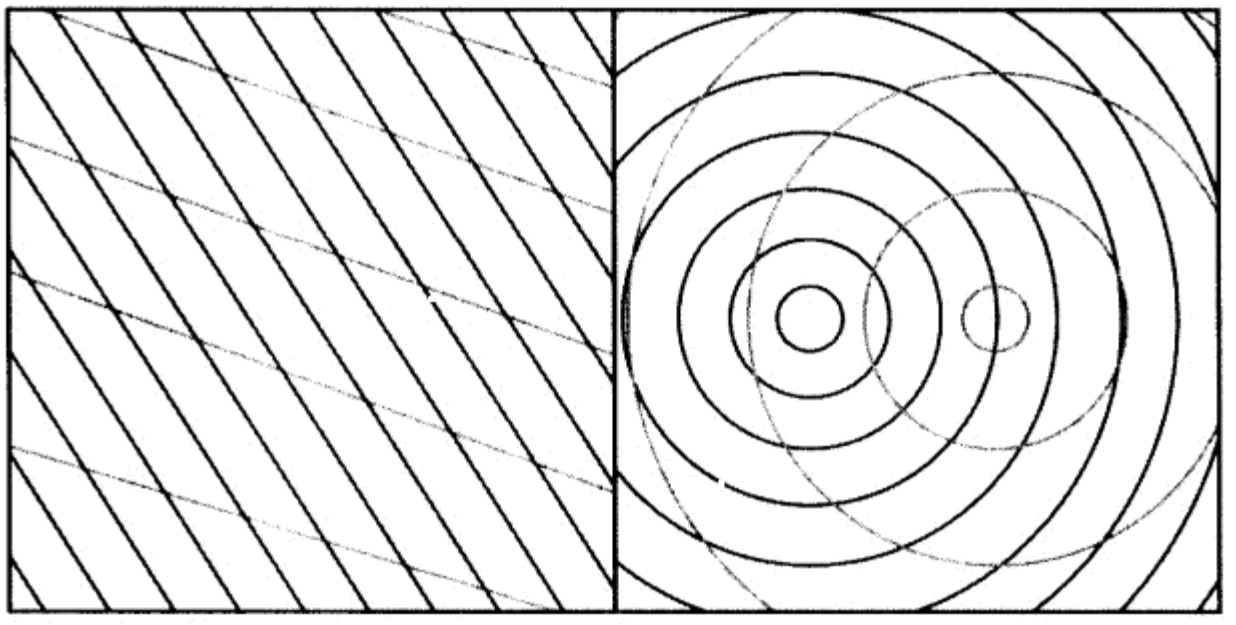


图 1-4 方向波和圆形波
对于两种类型的波，视觉兴趣和复杂性都是由干涉条纹引起的

Gerstner 波

为了有效模拟，需要控制波的陡度。如前所述，正弦波看起来圆滑，这可能是绘制一个平静的、田园诗般的池塘所需要的。但是对于粗犷的海洋，需要形成较尖的浪头和较宽的浪槽。使用公式（8a）和（8b）可以产生所需要的形状，但是最后我们选择了 Gerstner 波。早在计算机图形出现之前就研发出 Gerstner 波函数，在物理学基础上为海水建模。Gerstner 波可以提供一些表面的微妙运动，虽不很明显但是很可信（详细的描述可见 Tessendorf 2001）。我们选择 Gerstner 波，是因为它们具有一种经常被忽略的性质：它们把顶点朝着每个浪头移

动形成更尖的峰。因为在表面上浪头最尖（即频率最高），我们正是想要把顶点集中那个地方，如图 1-5 所示。

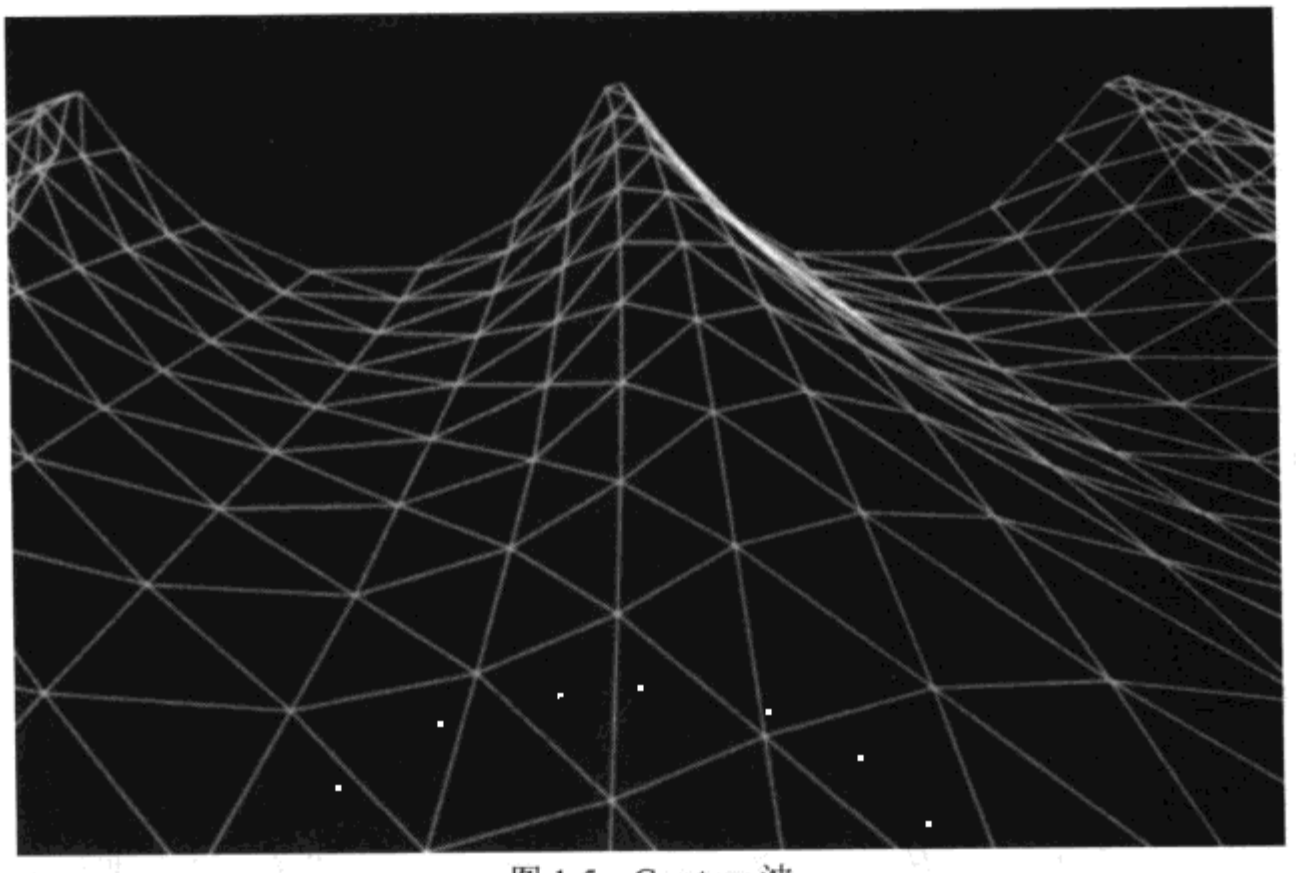


图 1-5　Gerstner 波

顶点被推向浪尖，最密的细分（tessellation）出现在尖形浪头处，这恰是所需要的附带效应

Gerstner 波函数是：

$$\boldsymbol{P}(x, y, t) = \begin{pmatrix} x + \sum(Q_i A_i \times \boldsymbol{D}_i.x \times \cos(\omega_i \boldsymbol{D}_i \cdot (x, y) + \varphi_i t)) \\ y + \sum(Q_i A_i \times \boldsymbol{D}_i.y \times \cos(\omega_i \boldsymbol{D}_i \cdot (x, y) + \varphi_i t)) \\ \sum(A_i \sin(\omega_i \boldsymbol{D}_i \cdot (x, y) + \varphi_i t)) \end{pmatrix} \tag{9}$$

这里 Q_i 是控制波陡度的参数。对于单个的波 i，$Q_i = 0$ 给出正常的正弦波，而 $Q_i = 1 / (\omega_i A_i)$ 给出尖峰的波形。应当避免选用较大的 Q_i 值，因为它们将会在波峰上形成环。事实上，我们可以把 Q 留给美工作为控制“陡度”的参数，其范围为 0～1，使用 $Q_i = Q / (\omega_i A_i \times numWaves)$ 可以产生变化的波形，从完全平滑的波到最尖锐的波。

请注意公式（3）和公式（9）之间惟一的差别是顶点的横向运动。它们的高度是相同的。这意味着不再有严格的高度函数，即 $\boldsymbol{P}(x, y, t) \cdot x \neq x$。但是，函数还是容易求导的，并且可以方便地去掉一些项。求导后得到切线空间的基础向量是：

$$\boldsymbol{B} = \begin{pmatrix} 1 - \sum(Q_i \times \boldsymbol{D}_i.x^2 \times WA \times S()) \\ -\sum(Q_i \times \boldsymbol{D}_i.x \times \boldsymbol{D}_i.y \times WA \times S()) \\ \sum(\boldsymbol{D}_i.x \times WA \times C()) \end{pmatrix} \tag{10}$$

$$\boldsymbol{T} = \begin{pmatrix} -\sum(Q_i \times \boldsymbol{D}_i.x \times \boldsymbol{D}_i.y \times WA \times S()) \\ 1 - \sum(Q_i \times \boldsymbol{D}_i.y^2 \times WA \times S()) \\ \sum(\boldsymbol{D}_i.y \times WA \times C()) \end{pmatrix} \tag{11}$$

$$N=\begin{pmatrix}-\sum(\boldsymbol{D}_i.x\times WA\times C())\\-\sum(\boldsymbol{D}_i.y\times WA\times C())\\1-\sum(Q_i\times WA\times S())\end{pmatrix} \tag{12}$$

其中，

$WA=\omega_i\times A_i$

$S()=\sin(\omega_i\times \boldsymbol{D}_i\cdot \boldsymbol{P}+\varphi_i t)$

$C()=\cos(\omega_i\times \boldsymbol{D}_i\cdot \boldsymbol{P}+\varphi_i t)$

这些式子可能不如式（4b）、（5b）和（6b）那样简洁，但是计算起来相当有效。

对法线的 z 分量做进一步考察，发现上文提到的波峰处形成环的原因很有趣。Tessendorf（2001）从流体力学 Navier-Stokes 描述和“展开变换技术”（Lie Transform Technique）推导“波浪起伏”（choppiness）效果时，得到的最终结果是表达在频率域中的 Gerstner 波的变异。在频率域中，能发现并且消除在波峰顶部的环，而在空间域中，可以清楚地看到这种现象是如何发生的。当 $Q_i\times\omega_i\times A_i$ 的加和大于 1 时，法线的 z 分量在峰顶变成负值，造成波的自我循环。只要选择 Q_i，使其加和总是小于或等于 1，就会形成尖锐的波峰而不再成环。

波长和波速

我们从选择合适的波长开始。不是要追求波在真实世界中的分布，而是要使现有的少数几个波达到最大效果。对波长相似的波进行叠加可以突显水面的活力。于是我们选择中等的波长，然后从它的 1/2 至 2 倍之间产生任意波长。中等波长是在写作过程中指定的，而且随着时间的推移而变化。比如，暴风雨中的波浪显然比风和日丽场景中的波浪要大。注意，我们不能够改变正在活动的波的波长。即使渐渐地改变它，波峰也会从原始状态扩展或压缩，看起来非常不自然。因此，我们只能改变当前的平均波长，在波随着时间的推移淡出场景以后，它们将基于新波长而再生。对方向的改变也是如此。

给定一个波长，我们能很容易地计算出波在表面传播的速度。Tessendorf（2001）给出忽略高次项的水的传播关系：

$$\omega=\sqrt{g\times\frac{2\pi}{L}} \tag{13}$$

这里ω是角频率，g 是重力常数，为国际单位制（例如，9.8m/s^2），而 L 是波峰到波峰的长度。

振幅

有不同的处理振幅的方法。虽然可以把振幅看作波长和当前天气条件的函数来求振幅的导数，我们还是要在编写时指定常数比（或由脚本指定）。更准确地说，是由美工对中等波长指定中等振幅。任何尺寸的波，其振幅对波长之比都要与这个指定的比率相匹配。

方向

波的运动方向与其他参数完全独立，因此可以基于我们所选择的任何标准，自由地选择波的方向。如前所述，我们大致从风向的常数向量开始。然后在恒定的风向角周围任意选择

波的方向。那个恒定的风向角按情节要求产生，或由脚本指定。

1.2.4　纹理波

加和在纹理中的波也像顶点一样要参数化，但是具有不同的约束。首先，在纹理中得到宽频谱更为重要。其次，在纹理中更容易形成不像天然波纹的图案。第三，对给定波长只有某些波方向能保证全部纹理的平铺（tiling）。也就是说，不像在世界空间中仅仅需要注意距离，在纹素（texel）中要注意所有的量。

我们在 2 到 4 个通道中，使用 15 个频率和方位不同的波。虽然 4 个通道听起来有点过分，但是它们是进入 256×256 的渲染目标纹理，而不是主帧缓冲器。实际上，可以忽略生成法线图的填充率所造成的影响。

波长和速度

我们再次从波长选择开始。波长应限于纹理支持的范围。显然，如果要将纹理作为块（tile）平铺，正弦波必须至少重复一次。就是把最大波长设定为 TEXSIZE，这里 TEXSIZE 是目标纹理的尺寸。当波长接近 4 个纹素时，波就退化为锯齿状的图案。所以最小波长应大于 4 个纹素。同样，当几何波动由较长波长近似时，我们乐于选择较短的纹理波长。所选择的典型波长大约为 4～32 个纹素。对于每 50 英尺（约 15m）平铺的凹凸贴图，32 个纹素的波长大约对应 6 英尺。这使得几何波长范围大于 4 英尺，而纹理波长范围小于 6 英尺，只有一点重叠。波速计算和几何学的波相同。式（8a）和（8b）中的指数用来控制波峰的陡度。

振幅和精度

像处置几何波一样，确定每个波的振幅，保持振幅对波长之比为常数 *KAmpOverLen*。这将引起有趣的优化。

这里我们不关心高度函数，只是要构建法线图。我们的查找纹理含有 $\cos(2\pi u)$，这里 u 是范围为 0 到 1 的纹理坐标。我们不是把原始的余弦值存储在法线中，而是存储在查找纹理中，因为实际上把余弦换算成旋转纹理的法线，比存储法线再与纹理一起旋转更容易。

通过从查找纹理到目标的渲染来计算叠加正弦波的法线。对于 *u-v* 空间表达公式（7），我们有：

$$\frac{\partial}{\partial u}W_i = \boldsymbol{D}_i.x \times \omega_i \times A_i \times \cos(\boldsymbol{D}_i \cdot (u,v) + \varphi_i t) \tag{14}$$

这里 u 和 v 在渲染目标中的范围是 0～1。我们在顶点 shader 中计算括弧内的项 $\boldsymbol{D}_i \cdot (u,v) + \varphi_i t$，把结果作为像素 shader 中纹理查找的 u 坐标。将括弧外的项 $\boldsymbol{D}_i.x \times \omega_i \times A_i$ 和 $\boldsymbol{D}_i.y \times \omega_i \times A_i$ 作为常数代入。结果的像素 shader 是常数乘上每个波的纹理查找数据。为了使用等式（8a）表达的陡峰波函数，只要简单地用：

$$\left(\frac{\sin(2\pi u)+1}{2}\right)^{k-1} \times \cos(2\pi u)$$

填入查找表代替 $\cos(2\pi u)$，把 $\boldsymbol{D}_i \times \omega_i \times A_i \times k$ 作为括弧外的项即可。

使用查找表既快速又灵活。由于处理器中对寄存器访问的速度加快，出人意料地给 CPU 带来了查找表这个便利工具，我们期待着 GPU 上的同样进展。但愿将来能更好地识别查找纹理以选择来代替直接计算。特别希望在使用查找表前，不用再为改变波的陡度而重生查找表。

在创建纹理波时，我们非常关心精度，这点不同于在顶点 shader 中的几何法线的合成。输出法线的每个分量必须表达为存偏的、有符号的、8bit 精度的定点值。如果表面非常陡，x 或 y 分量将会比 z 分量更大，而且会在 1 处达到饱和。如果表面总是扁平的，x 和 y 分量将总是接近于 0，而且具有量化的误差。在目前的工作中，我们期待后一种情况。如果能严格限定 x 和 y 分量的数值范围，我们就能放大纹理的法线，以得到最大精度；以后使用它们时，就不用再放大了。

考察生成法线的 x 分量，可以发现，余弦函数和方向向量的 x 分量均在[−1, 1]范围中。而频率和振幅的乘积不容易确定，因为每个波的频率和振幅是不同的。

用波长项中的频率表达公式（7），我们有：

$$\frac{\partial}{\partial x}W_i = 2\pi \times \frac{A_i}{L_i} \times \boldsymbol{D}_i.x \times \cos(\cdots)$$

这里高度由振幅较大的波决定，而表面方向则由振幅对波长比率较大的波决定。

首先，这个结果证明所有波的振幅对波长之比是恒定的，而由于我们所使用的波数量有限，略去比率较小的波是合理的。其次，由于那个常数比，使得波的 x 和 y 分量绝对值小于 $kAmpOverLen \times 2\pi$，而总数小于 $numWaves \times kAmpOverLen \times 2\pi$。为了在加和时能够保持分辨率，我们用下式做加和：

$$\frac{1}{numWaves} \times \boldsymbol{D}_i.x \times \cos(\cdots)$$

在使用时再乘以 $numWaves \times kAmpOverLen \times 2\pi$。

方向和平铺

如果在不平铺时，渲染目标具有足够高的分辨率，我们可以在其上累加任意的正弦函数。事实上还可以在上面累加任意法线图。例如，可以在场景中覆盖跟随人物运动的激烈变形。同样地，也可以从比单个正弦波更复杂的函数开始，累加几个波函数得到更为复杂的波。但要注意的是，这种技术的威力在于波的相对运动。复合波图案作为单元的运动，真实性和影响力比不上简单波的独立运动。

这些附加是相当直接的。但是要想让渲染目标能够作为平铺块（tile），就要对所叠加的波函数加上一些约束。圆形波的主要特点是不形成重复图案。假如想要平铺纹理，就需要波形成重复的图案，所以只能选用方向波。

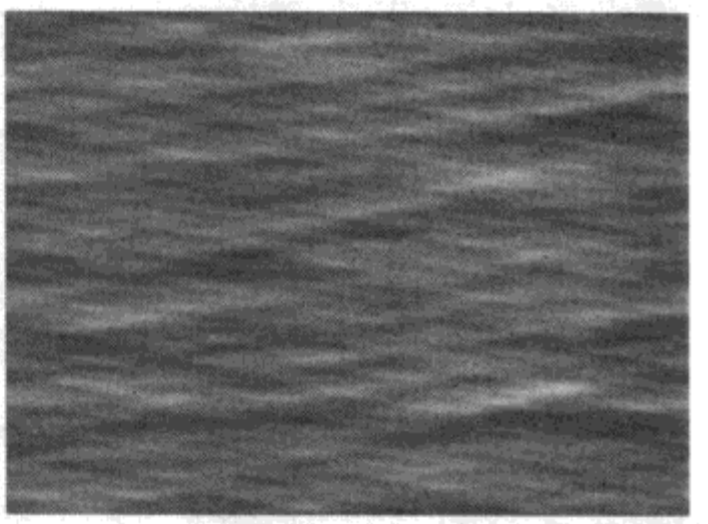
显然只有在纹理重复整数倍时，平铺块才能平铺。同样，如果某个波长的正弦波纹理能够平铺，只有它的某些旋转仍然能平铺。另一个事实是，如果每个旋转的正弦函数能够平铺，那些正弦函数的叠加也能平铺。

因为是通过纹理变换来旋转和缩放正弦函数，所以满足平铺的两个条件，就是保证纹理变换的缩放和旋转元素都是整数。然后在平移变换公式上增加相位分量来平移波。由于纹理实际上是一维的，只需要变换 u 坐标。

1.3　编辑

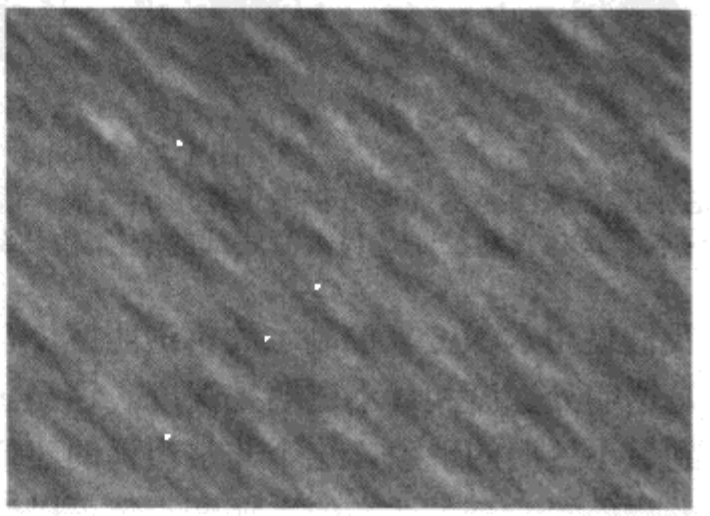
在这节中将简要地讨论如何放置水系统和离线建模。通过对水网格的建模，作者可以在顶点级别控制模拟，见图 1-6。

（a）海水场景



（b）用于渲染海面的法线图

（c）池塘场景



（d）渲染池面的法线图

图 1-6　海水和池水

在实现中，网格数据包括：网格细分、网中每个顶点的水平位置、顶点向下到水体底部的垂直位置，以及 RGBA 颜色。纹理坐标可以显式地指定或在运行中（on the fly）产生，见图 1-7。

首先把在顶点上的水深度作为一个输入参数，这样，在水碰到岸这样的微妙的区域，shader 就能进行自动校正。我们还要谈谈顶点级别的系统重载（overrides），已证明它们是特别有用的。为了避免失真，自动地滤掉那些取样频率不足的波。最后，对于纹理映射需要根据上下文显性指定，而非基于位置上的平面映射隐性导出的情况，这时我们只讨论必要的输入数据。

1.3.1　深度的使用

因为水的高度是要计算的，所以顶点位置的 z 分量就没什么用了。尽管可以利用这点来压缩顶点，但是我们选择把水深度编码在 z 分量中。更确切地说，就是把水体底部的高度放在顶点的 z 分量中，作为常数代入水的高度表中，这样通过相减得到水深度。这也呈现出水

高度的常数表。为了给类似沿山而下的河流等事物建模，我们需要每个顶点显式的 3D 位置和深度。这时，深度可以通过进行离线计算（使用简单的从水面到河床的光线投射法）获得，或者作者显式地把它指定为顶点颜色，但是无论如何，必须把深度看作顶点数据的一个附加部分。

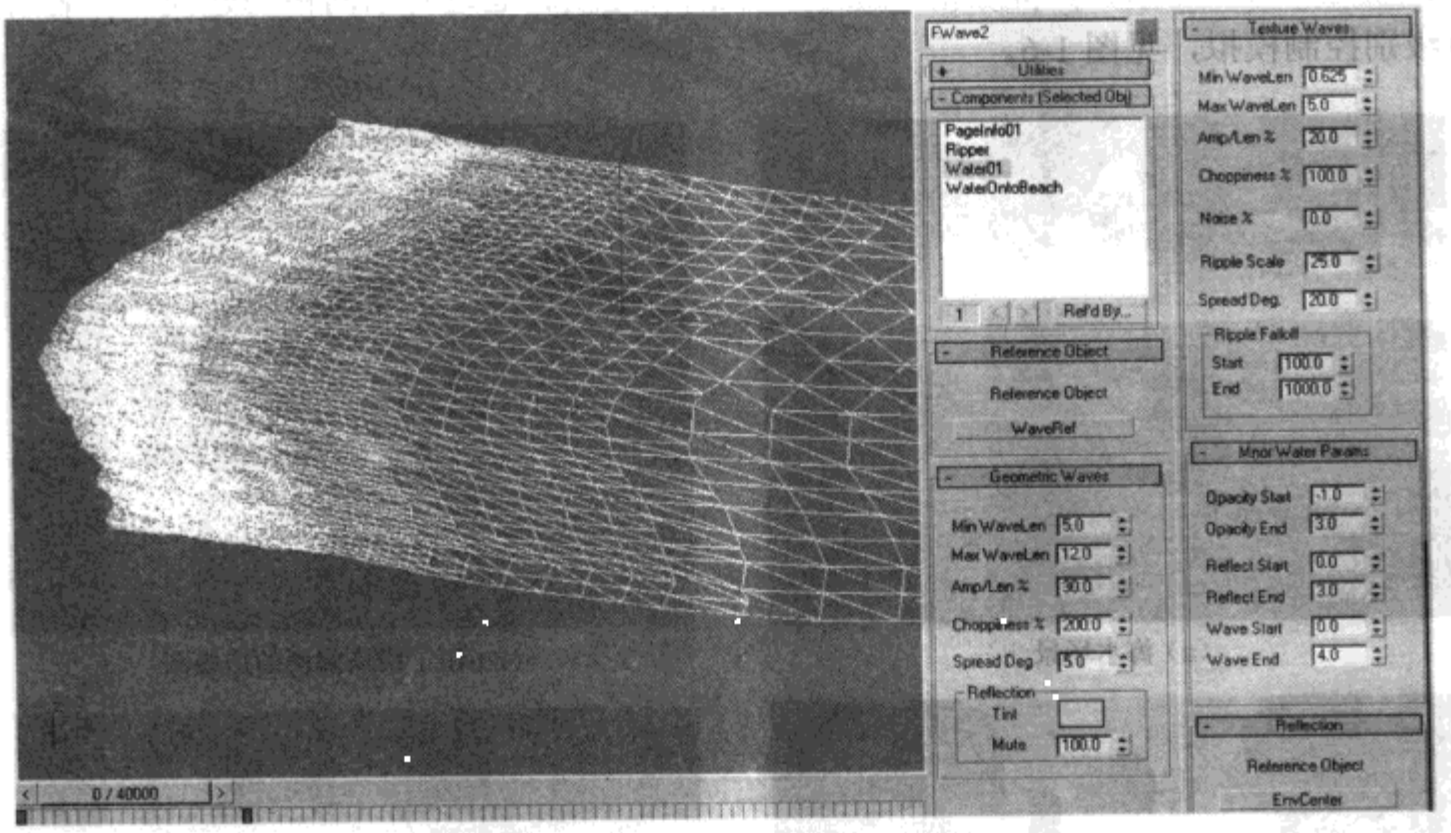


图 1-7 3ds max 编辑工具用户界面

这张屏幕截图显示许多可供美工使用的参数。请注意水网在靠近海岸的地方具有更密集的细分网格，这点不是与表面、而是与大海底部的变化相一致的

我们使用水深度来控制水的不透明度、反射强度和几何波振幅。例如，系统的一对输入参数是透明处深度和最不透明处深度。这显示出深度从离海岸 0 英尺变到 3 英尺时，水从透明变到最不透明的现象。这个非常粗糙的建模表达了一个事实：水浅的地方颜色浅，而水深的地方颜色深。有了适当的水深度，就可以对光的传播效果进行更完善的建模。

几何波振幅的衰减基于深度，对物理建模具有很大的实际意义。在水网格碰到水平面处波浪衰减消失，就可以在水网格碰到陡岸的地方把水顶点“固定”住。也可以使波浪涌向浅滩海岸时逐渐平息消失。因为我们约束顶点永远不能低于其输入高度，于是波浪衰减为 0 的地方总是稍微高于海平面，造成波浪重叠在岸上，这使得我们可以控制波浪冲上岸的距离。

1.3.2 重载

大部分情况下，系统“仅仅是可以工作”，用相同的方法处理所有的顶点。但是内容创作者要求在每个顶点的基础上重载系统行为，也是一种正当要求。我们把这些重载编码为顶点颜色 RGB。让无色为默认值，这些重载对模拟实行全面控制。调节一种效果就是把某个通道降为 0。

红色分量决定全面的透明度，当红色变为 0 时，水面完全透明。绿色调节表面的反射强度，当绿色是 0 的时候，制造水面的涟漪。蓝色基于视角控制不透明度的衰减，因为它影响 Fresnel 项。这些颜色的另一种使用方式，是缩放所计算的每个像素法线的水平分量。这可以使得一些水面较之其他区域变得起伏不平，正如一种时常在港湾中看到的效果。

1.3.3　边长的过滤

假如你熟悉信号处理理论，就会意识到，构建波浪网所能使用的最短波长取决于网格细分的精细度。从 Nyquist 定理可知，网格顶点的间距起码要小于最短波长的 1/2。请参见图 1-8，图中所示的虽然不够严格，但给出了直观的说明。只要网格中三角形的边比高度函数中的波长短，表面就看起来很好。而三角形的边等于或长于 1/2 最短波长时，就会看到表面严重的失真。

通常一个合理的办法是预先决定在高度函数里的最短波长，然后细分网格，以便所有的边都比那个波长短。这里，我们采取另一个办法：考察在顶点周围的边长，然后滤掉那些在邻居中不能被很好表示的“太短”的波。

这个技术有两点直接的好处。第一，可以把任何波长送入顶点处理单元，既不会造成失真，也不必考虑网格的细分。这就使得模拟能够仅仅根据当时的天气条件产生波长。根据衰减值（1 到 0）滤掉比波动网格太短的波长，波长是边长的 4 倍时，衰减值为 1；当波长是边长的 2 倍时，衰减值为 0。第二，网格不需要均匀地细分。可以把较多的三角形集中在感兴趣的区域。那些不太重要的区域，可以使用较少的三角形和较长的边，虽然这会使波动较为平坦，但是不会出现令人讨厌的失真。对小海湾的模拟就是一个例子，靠近海岸的区域使用很密的细分格，当水面向地平线扩展时，就使用越来越大的三角形了。

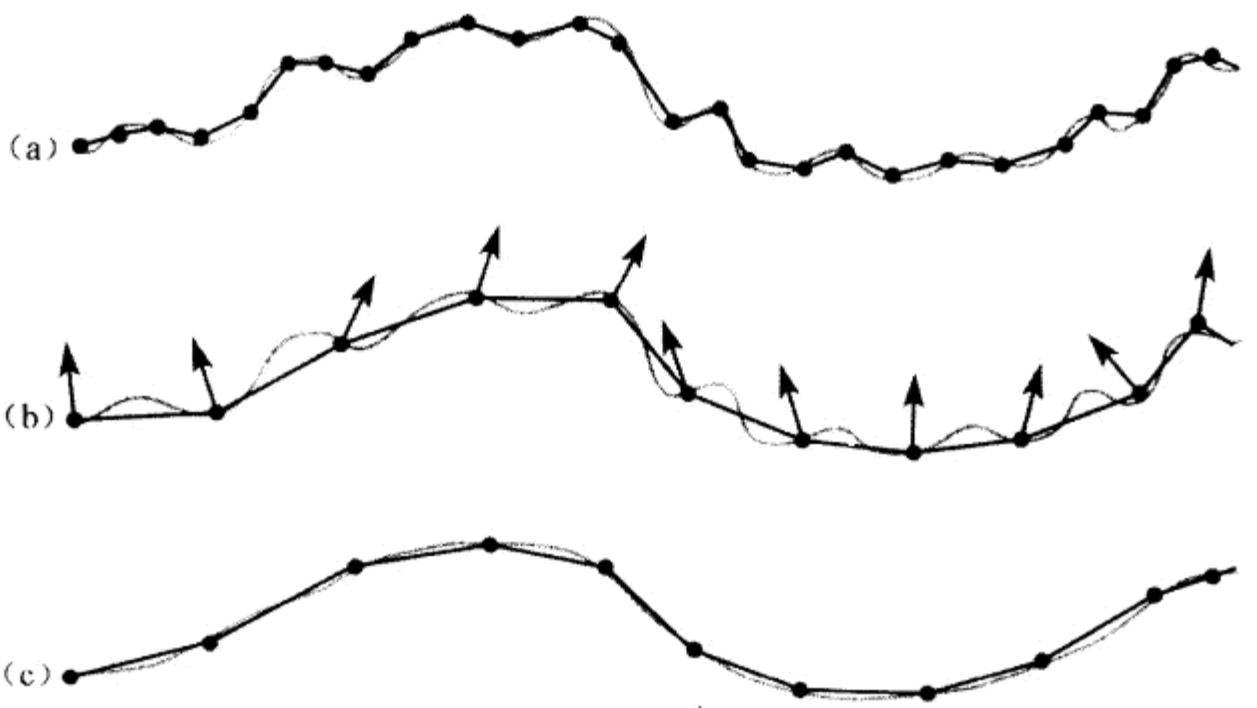


图 1-8　使波频率与细分相匹配
（a）高密度细分网格很好地表现高频和低频混合的波
（b）与（a）相同的波由低密度细分网格表达，效果很差。尤其要注意，虽然表面本身还可以，但由不充分取样产生的法线却很糟糕
（c）在滤去高频波、保持低频波不动之后，低密度细分网格又能很好地工作了

1.3.4　纹理坐标

通常，不需要指定纹理坐标。它们很容易由顶点位置导出，求解时基于一个比例值，它指定法线贴图的单个地砖所覆盖的世界空间距离。

然而，在某些情况下，显式的纹理坐标可能是有用的。例如渲染沿着弯曲河道流动的水。

在这种情况下，当河水转弯时，为了把凹凸贴图的法线从纹理空间转换到世界空间，显式的纹理坐标必须随着切线空间向量而增大。这些值是自动地离线计算的（对纹理坐标相关的位置求偏导数，因为纹理坐标是用凹凸贴图作为标准的）。载于 *Engel* 2002 的“逐像素光照”（Pre-Pixel Lighting）对使用 DirectX 产生切线空间基础向量做出了实际的描述。

1.4 运行时的处理

让我们用顶点和像素 shaders 进行高水平的处理。详情请参见所附的源代码。根据上面的解释，这种处理很简单。只有一个敏感的问题还有待讨论。

对一个动态、波动的几何表面，以及互相影响的波的复杂法线图，我们需要产生适当的凹凸环境映射参数将两者联系在一起。这些参数是将法线从纹理空间到世界空间的变换，以及把表面反映到立方体环境贴图的视线向量。我们先进行推导，然后再做顶点和像素处理。

1.4.1 凹凸环境映射参数

切线空间基础向量

我们借助水表面函数的偏导数计算空间基础向量。公式（10）、（11）和（12）给出副法线 $\boldsymbol{B}$，切线 $\boldsymbol{T}$ 和法线 $\boldsymbol{N}$。

将这些值放入 3 个向量（即输出的纹理坐标）构成一个矩阵，这个矩阵是：

$$Surf2World = \begin{pmatrix} \boldsymbol{B}_x & \boldsymbol{T}_x & \boldsymbol{N}_x \\ \boldsymbol{B}_y & \boldsymbol{T}_y & \boldsymbol{N}_y \\ \boldsymbol{B}_z & \boldsymbol{T}_z & \boldsymbol{N}_z \end{pmatrix}$$

除了可以得到基础向量以外，这也是凹凸映射的通常变换。应考虑到我们的表面是波动的，不是平的，是从波动表面空间转换到世界空间中去。如果法线图的纹理坐标是隐式的，即从顶点位置推导而来，那么我们可以认为在纹理空间和世界空间之间没有旋转。但是如果纹理坐标是显式的，我们就必须在纹理沿着河转弯时考虑旋转。

离线计算了 $\partial \boldsymbol{P}/\partial u$ 和 $\partial \boldsymbol{P}/\partial v$ 后，使它们形成一个旋转矩阵，把纹理空间法线转变成表面空间法线。

$$Tex2Surf = \begin{pmatrix} \partial \boldsymbol{P}/\partial u.x & \partial \boldsymbol{P}/\partial v.x & 0 \\ \partial \boldsymbol{P}/\partial u.y & \partial \boldsymbol{P}/\partial v.y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

这显然是在水平面上的旋转，因为我们在计算梯度之前已经将水网格塌陷为 $z = 0$。在更一般的情形下，当基础面不平坦时，它应为一个填满的 3×3 矩阵。无论何种情况，次序都很重要，所以合并矩阵是 $Surf2World \times Tex2Surf$。

如果想要重新缩放（rescale）法线的 x 和 y 分量，我们还必须进行最后一步操作。这样做是因为把它们写成法线图时，为取得最大精度已经将它们放大。或是因为要反走样，需要基于它们与照相机的距离而重新缩放它们。这两种情况下，都要缩放原始法线的 x 和 y 分量（即未基于以上两种原因进行变换之前的值）。于是假设统一的缩放因子是 s，我们的缩放矩

阵和最终的变换是：

$$重新缩放(Rescale)=\begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

$$最终矩阵(FinalMatrix)=Surf2World\times Tex2Surf\times Rescale$$

视线向量

我们通常以视线向量顶点的视点空间位置（译注：视点坐标系的坐标）为基础，建立环境映射查询。这样处理无限远的环境映射场景很有效。

假如在水中被反射的一切都是无限远的事物（例如天空），那么这种近似是完美的。然而，对于比较小的水体，反射的若是对岸上的物体，这种结果就显得不够好了。假定我们想要反射的不是处于无限远距离的场景，而是与池中心有同样距离的景物。那就是说，我们将环境映射投射到球面上，球心在池中央，球半径与池半径相同。Brennan 于 2002 年为这点给出了非常聪明和有效的近似。这里我们提供另一个方法。

图 1-9 直观地表述了问题以及解决方法。所给的环境映射是从点 C 产生，照相机现在是在点 E 看着一个在点 P 的顶点。我们想要视线向量从 E 经过 P 看见在环境映射中点 A 的物体。但是视线向量实际上是与产生环境映射的点相关的，因此我们在点 B 对环境映射取样。

需要计算一个把我们从 C 带到 A 的视线向量。可以在原始视线向量与球相交的地方发现点 A，于是把视线向量改正为 $A-C$。因为 A 是在半径 r 的球面上，并且 $A=E+(P-E)t$，我们有：

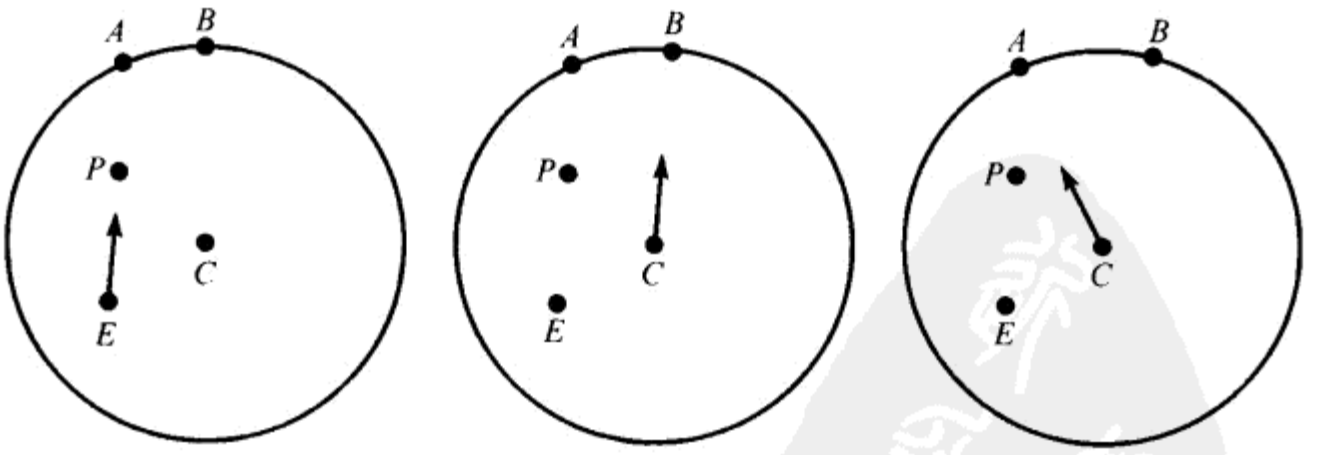
$$r^2=|A-C|^2$$

$$r^2=|E+(P-E)t-C|^2$$

$$r^2=|(P-E)t-(E-C)|^2$$

可以推出：

$$0=(P-E)^2\cdot t^2-2(P-E)(E-C)\cdot t+(E-C)^2-r^2$$



（a）原始视线向量　（b）使用原始视线向量的结果　（c）校正的视线向量

图 1-9　视线向量的校正

使用 2 次方程式求 t，得到：

$$t=\frac{2(P-E)(E-C)\pm\sqrt{\left(4\left((P-E)\cdot(E-C)\right)^2-4(P-E)^2\left((E-C)^2-r^2\right)\right)}}{2(P-E)^2}$$

这里进行一些置换：

$$\boldsymbol{D}=\frac{(P-E)}{|P-E|}$$ 规范化的原始视线向量

$$\boldsymbol{F}=(E-C)$$ 从摄像机指向环境映照中心的向量

$$\boldsymbol{G}=F^2-r^2$$ 为了方便运算的校正常数

代入替换量和舍弃较小的根，得：

$$t=\frac{2\boldsymbol{D}\cdot\boldsymbol{F}+\sqrt{4(\boldsymbol{D}\cdot\boldsymbol{F})^2-4\boldsymbol{D}^2\cdot\boldsymbol{G}}}{2\boldsymbol{D}^2}$$

消去 2 次项并使 $\boldsymbol{D}^2=1$（它被规范化），得：

$$t=\boldsymbol{D}\cdot\boldsymbol{F}+\sqrt{(\boldsymbol{D}\cdot\boldsymbol{F})^2-\boldsymbol{G}}$$

和

$$A-C=E+\left(\boldsymbol{D}\cdot\boldsymbol{F}+\sqrt{(\boldsymbol{D}\cdot\boldsymbol{F})^2-\boldsymbol{G}}\right)\times\boldsymbol{D}$$

我们已经有了规范化的视线向量 $\boldsymbol{D}$，可用它基于视角对水的不透明度进行衰减。$\boldsymbol{F}$ 和 $\boldsymbol{G}$ 是常数。所以虽然一开始计算 t 和校正视线向量很麻烦，但最后只需要 5 个算术操作。这里要强调的是 Fernando 和 Kilgard 2003 年做的重要工作，他们把在网上塌陷为常数的值代入 shader 后，shader 代码获得了令人惊讶的优化。

这个算法本质上是让光线与球面相交，假如光线不与球面相交，显然它就要失败。但是我们并不担忧这一点。如果视点位置或顶点位置在球体内，总是会有一个实根；而且因为球体包围水体周围的风景，水顶点位于球体内是没有问题的。

值得注意的是，有时球面映射对于反射出水体外的环境不能很好地近似，有时动态的环境无法用静态的环境映射表达，Vlachos 等人 2002 年描述的投影方法可以较好地处理这种情况。

1.4.2 顶点和像素的处理

我们从计算 4 个几何波中每个波的正弦和余弦函数开始。

在对它们求和之前，我们从水位的高度常数中减去输入顶点 Z，目的是得到那个顶点上水的深度。依据在输入深度常数之间插值的形式，那个深度构成第一个波的高度衰减因子。

因为存储了这个顶点邻居的最小边长，现在我们将它与波长比较，独立地对波的高度进行衰减。当边长大于 1/2 波长时，就滤掉那个波。

显式的纹理坐标不用再说了。隐式的坐标只是对顶点的 x 和 y 位置的缩放。从纹理空间到世界空间的变换按照第 1.4.1 节所述计算。所用的缩放值是输入常数 *numWaves* × 2π × *kAmpOverLen*，如第 1.2.4 节中“振幅和精度”所述。缩放值由另一个缩放因子调节，随着从顶点到视点的距离逐渐增加，缩放因子渐变为 0。这使得在法线图纹理元（texels）比像素小

得多的地方，其法线会塌陷为几何表面的法线。视线向量是作为凹凸环境映射的最后一步计算的。

每个顶点发出两种颜色。一种是水本身的颜色。另一种是水反射的颜色。首先根据深度函数调整水本身的不透明度，通常深度浅的地方水体更透明。然后根据视角调整不透明度，直视水面比斜视更透明。每个像素的 Fresnel 项以及其他更复杂的衰减效果，可以通过把视线向量引入像素 shader 进行计算，而不必进行顶点级别的计算。反射颜色也根据深度和视角调整，但是对于反射的时间和透明程度要分别独立地予以控制。

在像素处理阶段，需要做的事就不多了。简单的环境映射凹凸查询给出反射颜色。这里我们使用的是立方体环境映射，但是 Vlachos 等人 2002 年所给出的投影面环境映射在许多情形下更为适用。反射的颜色要进行调制，并加到水本身的颜色上去。最后将像素发送到帧缓冲器中并存入混合的 alpha 值。

1.5　小结

在顶点和像素 shaders 的领域里，水模拟是一个有趣的主题：它能将这些截然不同的技术胶结为一个系统。本文把增强的凹凸环境映射与复杂的函数计算结合在顶点和像素 shaders 中。我们希望本章在两个方面有用：第一，对通常认为很特别的系统，详述其后面的物理学和数学，显示系统本身如何能变得更富延展性。只有理解了原始波是如何使用的，才能用更多更好的几何和纹理波函数去改善系统。其次，只使用很少的资源，系统就能生成动态、可管理和生动逼真的水面。系统只要求 vs.1.0 和 ps.1.0 的支持。在目前和未来的硬件上，这留出大量的资源，以便于实现更为复杂的效果，例如光的传播和对其他物体的反射等。

1.6　参考文献

JeffLanders has a good bibliography of water techniques on the Darwin 3D Web site:

http://www.darwin3d.com/vsearch/FluidSim.txt

And another good one, on the Virtual Terrain Project Web site:

http://www.vterrain.org/Water/index.html

Brennan, Chris. 2002. "Accurate Reflections and Refractions by Adjusting for Object Distance." In *ShaderX,* edited by Wolfgang Engel. Wordware. **http://www.shaderx.com**

Engel, Wolfgang. 2002. "Programming Pixel Shaders." In *ShaderX,* edited by Wolfgang Engel. Wordware. **http://www.shaderx.com**

Enright, Douglas, Stephen Marschner, and Ronald Fedkiw. 2002. "Animation and Rendering of Complex Water Surfaces." In *Proceedings of SIGGRAPH 2002.* Available online at **http://graphics.stanford.edu/papers/water-sg02/water.pdf**

Fernando, Randima, and Mark J. Kilgard. 2003. *The Cg Tutorial.* Addison-Wesley.

Isidoro, John, Alex Vlachos, and Chris Brennan 2002. "Rendering Ocean Water." In *ShaderX,* edited by Wolfgang Engel. Wordware. **http://www.shaderx.com**

Laeuchli, Jesse. 2002. "Simple Gerstner Wave Cg Shader." Online article. Available online at

http://www.cgshaders.org/shaders/show.php?id=46

Tessendorf, Jerry. 2001. "Simulating Ocean Water." In *Proceedings of SIGGRAPH 2001.* Course slides available online at **http://online.cs.nps.navy.mil/DistanceEducation/ online. siggraph. org/ 2001/Courses/47_SimulatingNature/Presentation07.html**

Vlachos, Alex, John Isidore, and Chris Oat. 2002. "Rippling Reflective and Refractive Water." In *ShaderX,* edited by Wolfgang Engel. Wordware. **http://www.shaderx.com**

Yann, L. 2003. "Realistic Water Rendering." E-lecture. Available online at **http://www.andyc.org/ lecture/viewlog.php?log=Realistic%20Water%20 Rendering,%20by%20Yann%20L**

第 2 章　水刻蚀的渲染

Juan Guardado
NVIDIA
Daniel Sdnchez-Crespo
Universitat Pompeu Fabra/Novarama technology

2.1　引言

水与光相互作用能产生一种梦幻般的景象：由于反射和折射，弯曲的光线在海底形成舞动的刻蚀，因此在海面上就看到了变换无穷的花纹，见图 2-1。这些现象及其复杂性吸引了许多物理领域的研究者，近年来计算机图形学者也加入了他们的行列。模拟和渲染真实的水就像模拟火一样，令人非常着迷。要以可交互的帧速率得到好的模拟结果是不容易的，所以必须时常考虑创新的方法。

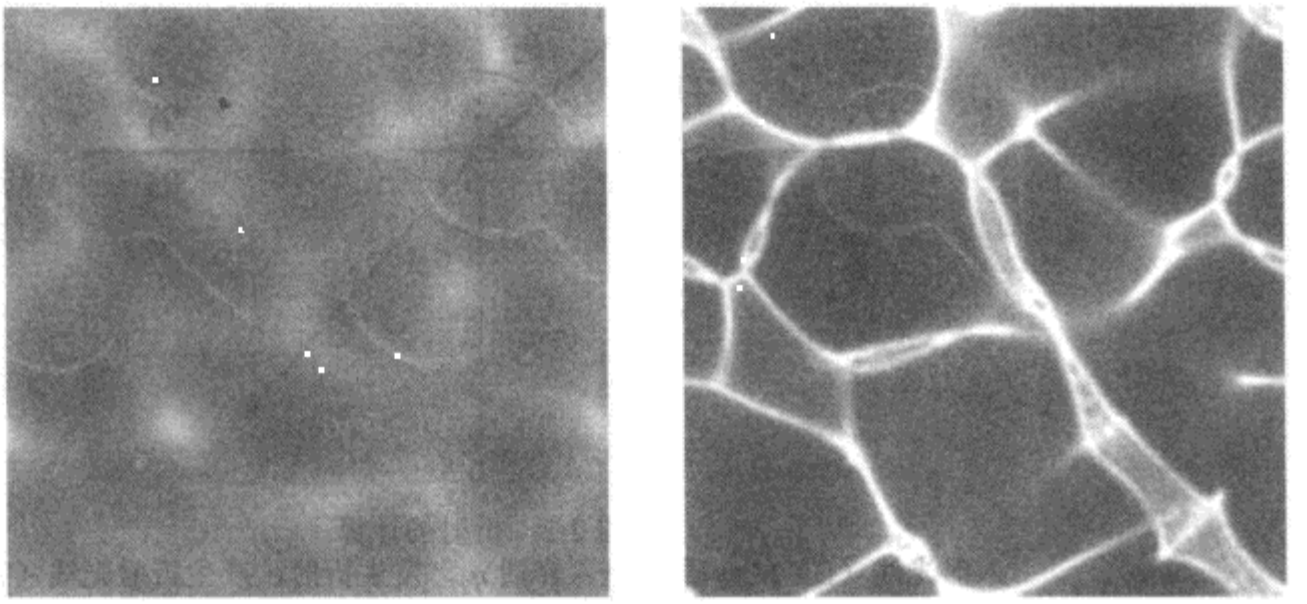


图 2-1　水刻蚀的例子
版权归 Jos Stam 所有

光线从弯曲的表面反射或折射，因此只聚焦在受光面上的某些区域，于是就产生了刻蚀现象。这节介绍了一种从美学角度出发（aesthetics-driven）来实时渲染水中刻蚀的方法。纯美学的方法完全忽略了真实性。虽然结果的场景看上去很好，但可能不能正确地模拟物理装备。在这节中将会看到，使用我们的方法会使结果看起来出奇地逼真，而且很容易在大多数图形硬件上实现。在许多与分形（fractal）相关的目标中，例如对山和云的渲染，

或对树的建模上，这个简化方法已经取得成功。

本章的目的是阐述此实时渲染刻蚀的新技术，从它的物理基础到实施细节予以全面的介绍。因为此技术是过程式的，它可以使用高级着色语言（high-level shading language）优美地实现。

2.2 刻蚀的计算

正确地计算水下的刻蚀是一个复杂的过程：这涉及到数目庞大的光线及其相互作用。为了适当模拟这一过程，必须从光源射出光子作为开始（例如对海洋场景，光源就是太阳）。一部分光子最后碰到海面，或者反射，或者折射。暂且不管反射，来看看透过海面的光线是如何按照 Snell 定律折射的，即：

$$\eta_1 \sin\theta_1 = \eta_2 \sin\theta_2$$

也可以写成：

$$IOR = \frac{\eta_1}{\eta_2} = \frac{\sin\theta_2}{\sin\theta_1}$$

在前面的等式中，η_1 和 η_2 分别是相关材料的折光指数，而 θ_1 和 θ_2 是入射角和折射角，如图 2-2 所示。折射率为 *IOR*，可以简单地写成入射角和折射角的正弦之比。

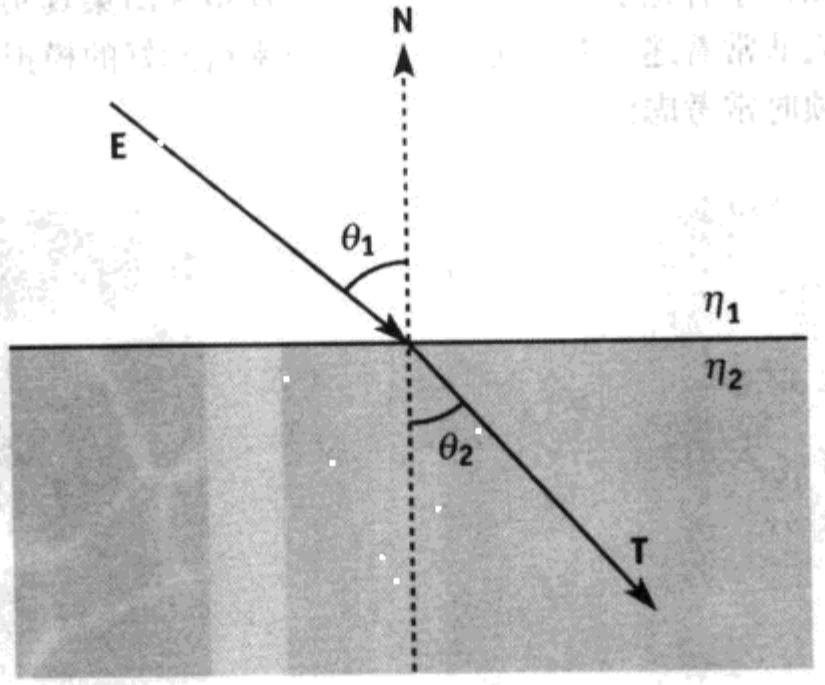


图 2-2 折射的计算
入射光线（E）从介质 η_1 进入介质 η_2，发生折射，产生折射光线（T）

用这样的公式给 Snell 定律编程是不容易的，原因是它只加了一个约束，使得计算折射光线不是很容易。假如入射线、折射线和表面法线是共面的，可以将公式变形为一个容易编程的形式，例如，Foley 等人 1996 年提出：

$$T = N\left(\frac{\eta_1}{\eta_2}(E\cdot N) \pm \sqrt{1+\left(\frac{\eta_1}{\eta_2}\right)^2\left((E\cdot N)^2-1\right)}\right) + \frac{\eta_1}{\eta_2}E$$

这里 T 是折射线，N 是表面法线，E 是入射线，而 η_1、η_2 是折射指数。

光子一旦入水后即发生折射并继续前进，随着入水的深度增加光强度会衰减。最后，一

些光子会碰到海底并将其照亮。由于海面的波纹，经过不同路径入水的光线，可能最终照到海底相同的区域。每当这个现象发生时，在刻蚀中的聚集光线就会形成明亮的光斑，类似于透镜的聚光现象。

从模拟的观点出发，刻蚀其实可以通过正向或逆向光线跟踪计算。在正向光线跟踪中，要跟踪从光源射出并穿过场景的光线，累计其在不连续地区的贡献。这个方法的问题是许多光线不与海面相碰，即使相碰的光线中，实际上也只有很少光线对刻蚀的形成有贡献，因此，它是一种不合理的方法，即使采用空间分割法来加速运算也仍是如此。

逆向光线跟踪，则以相反的过程工作。它从海底开始，按照与入射相反的顺序逆向跟踪光线，计算给定点的所有入射光线总和。在理想情况下，可以求解发自同一点的所有光线的半球积分。实际上，积分的结果是经过 Monte Carlo 取样法得到的。也就是说，先假设一束光线是产生刻蚀的光线。这束待测光线从取样点以半球面向各方向发送，半球面的中心在取样点。如果某光线碰到其他物体（例如一只鲸、一艘船，或一块石头），就把它舍弃。另一方面，碰到海面的光线显然是从外面来的，可以作为继续检测的对象。于是它们按照逆向的 Snell 定律折射。然后这些剩余的光线继续在空中传播，经过检测，以确定每道光线是真的来自光源。只有那些最终碰到光源的光线，才确实是有助于形成刻蚀的，其余的作为无效者舍弃。

这两个方法都非常费时，因为只有极小部分的计算对最终结果有实际意义。在商业刻蚀处理器中，通常有用光线占总光线的比率为 1%～5%。

实时刻蚀是 Jon Stam 首先开发的（Stam 1996）。Stam 的方法使用波理论计算了一个动画的刻蚀纹理，并用它来照亮地面。这个纹理混合叠加在物体的基本纹理上，如图 2-3 所示的那样，好看而且可信。

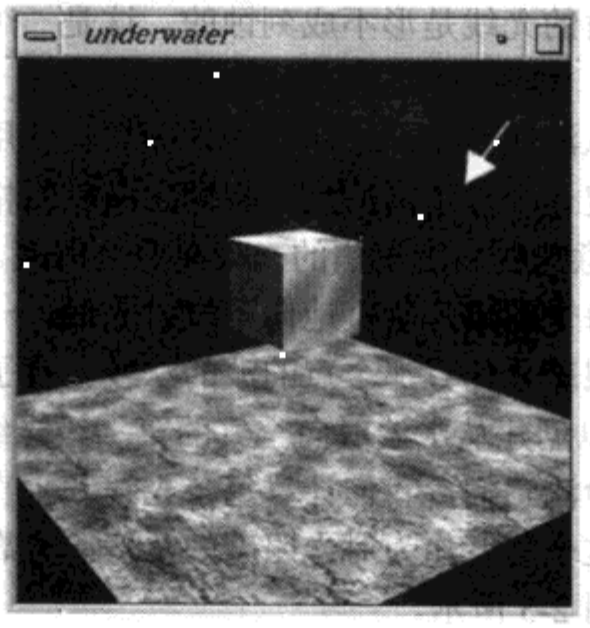


图 2-3　使用 Jon Stam 的投影刻蚀纹理所创建的刻蚀

在 Gamasutra 一篇优秀的论文（Jensen and Golias，2001）中，Lasse Staff Jensen 和 Robert Golias 研究了另外一种有趣的方法。论文不只包括了刻蚀渲染，同时也是一个完整的关于水的动画与渲染框架。平台基于快速傅立叶变换（FFT），来对波函数建模。这个方法在处理反射、折射和刻蚀时，力图分别给它们以物理精度的建模。可以想见，Jensen 和 Golias 处理刻蚀的方法是模拟真实过程的：跟踪从太阳发出的光线，直到水面网格的每个顶点。那些光线遵循 Snell 定律折射，由此产生新的折射光线。

2.3 方法

我们用于模拟水下刻蚀的算法，只是前一节介绍的逆向 Monte Carlo 光线跟踪的一个简化。我们大胆地假设一些光线对刻蚀有贡献，并且只计算到达海底光线的一个子集，因此，该方法计算消耗非常少，却产生了尽管在物理上不“正确”但是非常逼真的刻蚀图样和效果。由于整个效果看起来非常令人信服，尤其是图像质量，使得这个方法非常值得实现。

一开始，我们假定在赤道上计算中午的刻蚀。这意味着太阳正在我们头顶上。运算法则需要计算太阳圆盘（Sun disk）在天空的张角。太阳离地球距离约为 1.47～1.52 亿公里（取决于季节），它的直径是 142 万公里，太阳圆面产生的角度为 0.53 度，如图 2-4 所示。

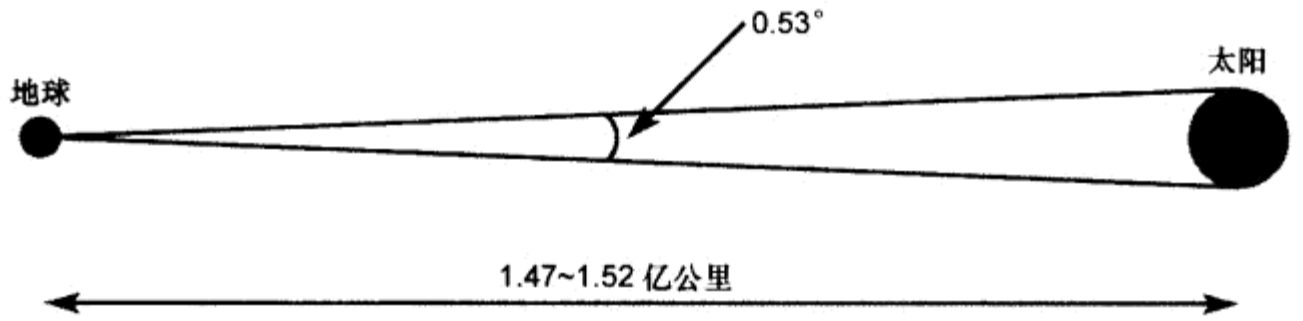


图 2-4 太阳圆面的角度

第二个假设是，照亮我们感兴趣的点的光线必须与海底面垂直。水的透明度为每米 77%～80%，这样在每米距离上，20%～23%的入射光会被介质吸收，化成热量发散。这意味着，那些从入水到碰上海底面，传播距离最短的光线，才会容易形成刻蚀。因此，垂直光线形成刻蚀的可能性最大，而水中斜射的光线是形不成刻蚀的。这是一个大胆的假设，但它是算法成败的关键。

我们的算法工作如下。从海底开始，绘制海底地面后，使用第二次混合叠加渲染来渲染刻蚀。为了这样做，我们创建一个与水网格尺寸相同的海底网格，并且用刻蚀值对其逐顶点地着色：0 意味没有照明；1 意味着高度聚焦的一束光照到海底。为了创建光照，采用逆向光线跟踪：从我们海底网格的每个顶点，垂直地向上发射光线，一直到达正好位于那个顶点之上的水波点。然后，使用有限差分计算那个点的波的法线。有了矢量和法线，使用 Snell 定律（水的折射率为 1.33），可以创建从水波射向空气的第二级光线。这些光线是假设能为海底带来光照的待测光线。为了检测它们，我们计算每条光线和垂直线的夹角。因为太阳圆盘非常远，我们可以简单地使用这个角度作为对光照度的量度：愈靠近垂直的方向，从那个方向进入大海的光线就越多，如图 2-5 所示。

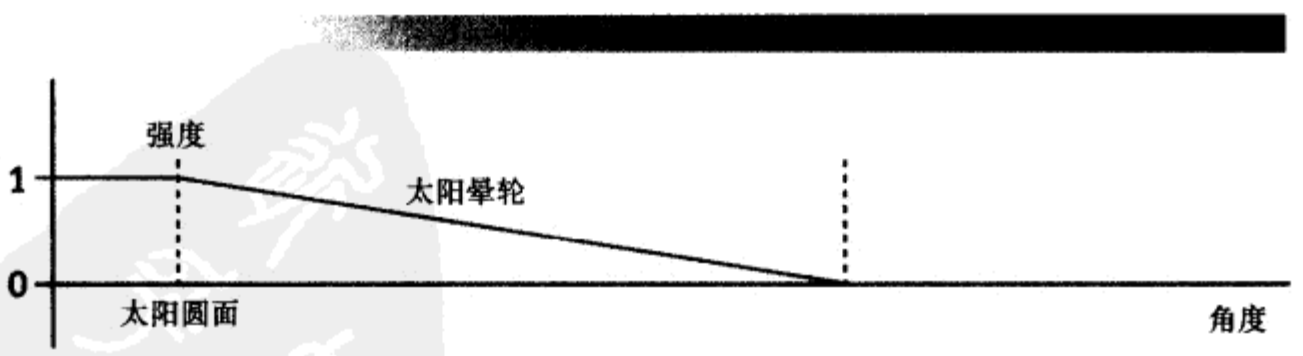


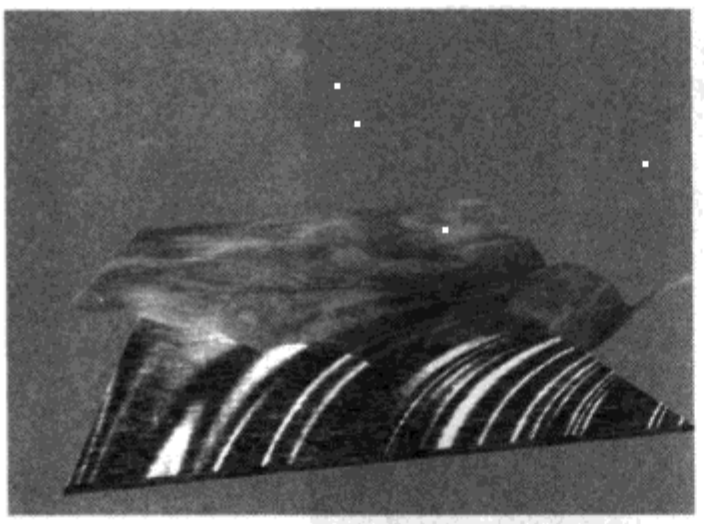
图 2-5 太阳圆面的强度与入射角成反比

2.4　使用 OpenGL 实现

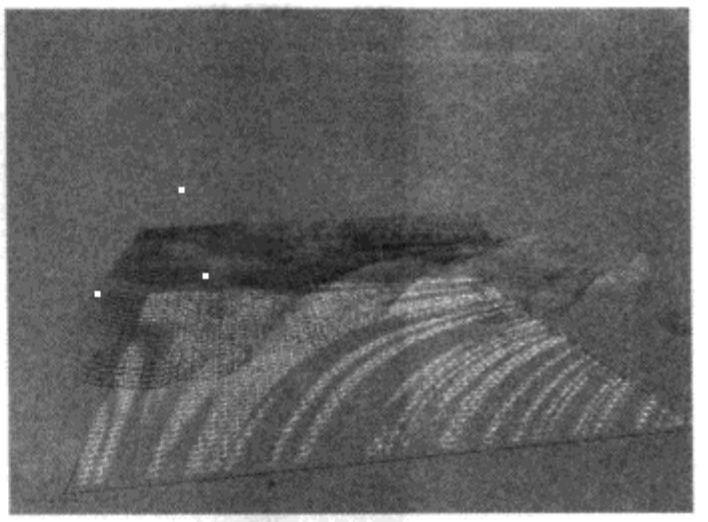
算法的最初实现是直接的 OpenGL 代码，惟一的不同是使用多遍次（multi pass）纹理渲染。第一次渲染使用一个普通纹理四边形渲染海底面。然后，使用精细的网格把同一个海底面再画一次，精细网格的每个顶点是用刻蚀发生器逐一照亮的，如图 2-6（b）所示。我们从海底精细网格的每个顶点，都垂直发射一道射线，当它碰到海表面时，按照 Snell 定律产生折射，然后使用折射的射线进行一次射线-四边形测试，所得结果用来作为纹理贴图坐标索引，如图 2-6（a）所示。最后渲染遍次使用波形生成器来渲染海浪。用一个平面环境反射贴图给这些三角形贴上纹理，这样在上面就反射出一片好看的天空。其他的效果，比如 Fresnel 效果，也可以在上面实现。

该技术的伪码如下。

1. 画海底面。
2. 对于精细网格中的每个顶点：
   a. 反射一道垂直的光线；
   b. 使光线与海的网格相碰；
   c. 逆向使用 Shell 定律计算折射光线；
   d. 使用折射的光线为“太阳”贴图计算纹理坐标；
   e. 把纹理坐标应用到精细网格的顶点。
3. 渲染大海表面。



（a）实体渲染模式



（b）线框渲染模式

图 2-6　使用 OpenGL 实现

2.5　使用高级着色语言实现

这个技术在 GPU（图形处理器）上的实现，既给出了更好的视觉效果，又改善了它的运行性能。一个高级的着色语言（例如 Microsoft 公司的 HLSL，或 NVIDIA 的 Cg），使得我们能用像 C 语言那样的语法，很快地将所有这些计算移植到 GPU（图形处理器）中去。事实上，先前 OpenGL 实现中在 CPU 上执行的波函数，使用一个包含文件（include file）就可以简单地

复制到 pixel 和 vertex shaders 中去；只要做极小的修改，就能适应高级语言的基于矢量的结构。

用逐像素法代替逐顶点法来计算刻蚀，全面地提高了视觉质量，而且减小了几何复杂性。当初打算把技术移植到 GPU（图形处理器）的 pixel shaders 时，考虑过两种方法。它们都使用波函数的偏导产生法线。不像前面提到的方法，要使用有限差分。

第一个方法利用了这样一个事实：在屏幕空间（screen space）中可以渲染过程纹理（procedural texture），因此，当只有小部分像素看得见时，这个方法可以节省渲染时间。但是这也意味着，当有很多像素看得见时，对每个像素都要做同样多的工作，尽管有时候可能感觉不到这些附加的细节。这些大量的工作显然要减小帧速率，所以需要另外的方法以解决这个问题。

第二个方法，在纹理空间（texture space）中渲染一个分辨率固定的渲染目标贴图（render target）。在纹理空间中渲染，能够固定在每帧中的工作量，这是一个优越性；但是却丢掉了另一个好处，不能只渲染看得见的像素了。另外，也很难决定使用多大渲染目标贴图的分辨率对当前场景是最合适的。如果纹理点对屏幕像素（texel to pixel）的比例降得太低，那么对于依靠纹理过滤的点就可能形成所谓的双线过滤十字图案（bilinear filtering cross pattern）。

我们还可以根据以下原因进一步优化运算，即这样两种算法之间的视觉差异很小：一是使用 Snell 定律对入射光线的折射，二是简单地基于从水面到水底的距离，沿着波法线进行的环境映射，见图 2-7。结果，穿过纹理中心的垂直光线是明亮的，而夹角光线则逐渐衰弱。而且，深度越大，环境反射贴图的相对尺寸（即光源的相对尺寸）越小，因为刻蚀会随距离而衰减，所以产生的刻蚀也越细。这个方法允许我们有一个非均匀的地平面，而且可以容易地把变化的深度编码到顶点属性中。

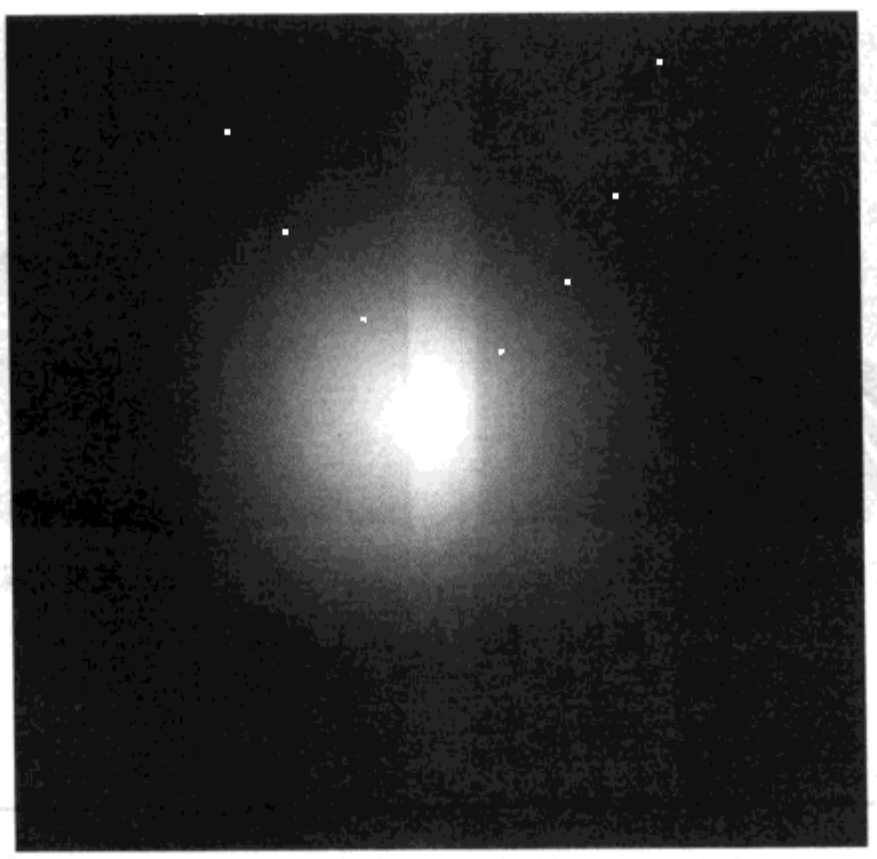


图 2-7 头顶上太阳的环境贴图

因此，这个效果用来计算波函数的法线，以便跟踪光线以得到与海底面的交点。为了产生法线，计算波函数关于 x 和 y 的偏导数。

让我们写下原来方法中使用的波函数：

$$f(x,y)=-\sum_{i=1}^{i=octaves}\sqrt{x^2+y^2}\,\frac{c_1\cos(c_2+2^i c_3 xy)}{2^i}$$

这里，i 表示用来产生波的 8 个一组（octaves）的数目，c_1、c_2 和 c_3 分别是波频率、振幅和速度的常数；x 和 y 表示单位方块中的一个点，范围从 0 到 1。

应用链式法则和积式法则求偏导数的两个公式如下：[1]

$$\frac{\partial f(x,y)}{\partial x}=-\sum_{i=1}^{i=octaves}\sqrt{x^2+y^2}\left(c_1\sin(c_2+2^i c_3 xy)c_3 y\right)-\frac{c_1\cos(c_2+2^i c_3 xy)x}{2^i\sqrt{x^2+y^2}}$$

和

$$\frac{\partial f(x,y)}{\partial y}=-\sum_{i=1}^{i=octaves}\sqrt{x^2+y^2}\left(c_1\sin(c_2+2^i c_3 xy)c_3 x\right)-\frac{c_1\cos(c_2+2^i c_3 xy)y}{2^i\sqrt{x^2+y^2}}$$

偏导数实际上是所计算点的梯度分量。因为函数实际上代表高度，即 z，有关 z 的偏导数是 1。法线是两个梯度向量的叉积，即

$$\boldsymbol{n}=\begin{pmatrix}-\dfrac{\mathrm{d}f}{\mathrm{d}x} & -\dfrac{\mathrm{d}f}{\mathrm{d}y} & 1\end{pmatrix}$$

渲染刻蚀最后所需要的是线与平面的相交公式。 我们能用下式来计算线上一点到线与平面交点的距离：

$$d=\frac{D_{pl}-\boldsymbol{n}_{pl}\cdot p_l}{\boldsymbol{v}_l\cdot\boldsymbol{n}_{pl}}$$

这里 D_{pl} 是从水平面到起始点的距离，$\boldsymbol{n}_{pl}$ 是地平面的法线，v_l 是描述折射光方向的矢量（实际上是沿着波面的法线），而 p_l 是折射光线和海底面的交点，见图 2-8。

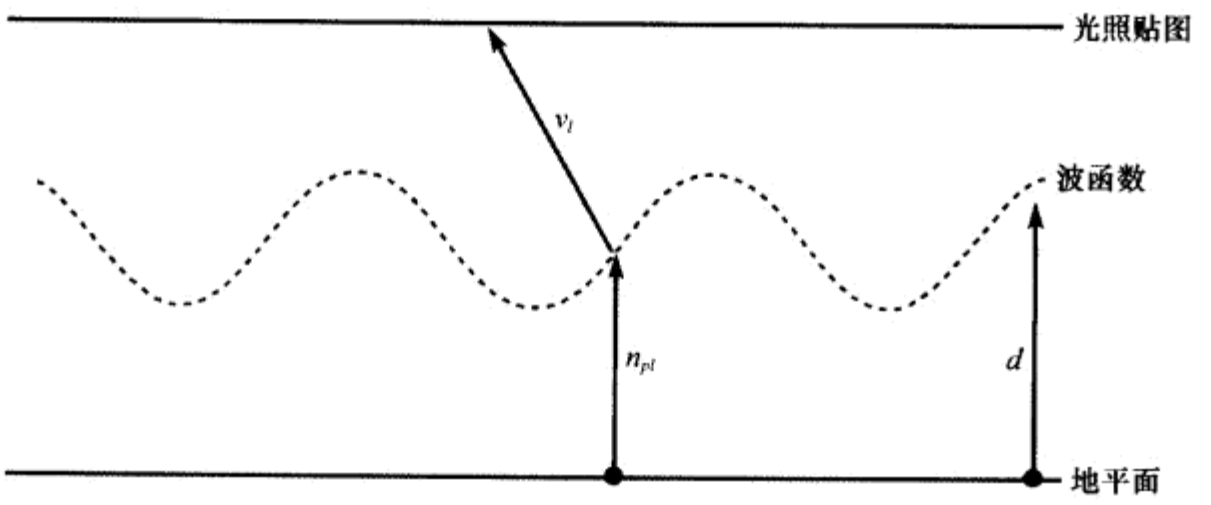


图 2-8　访问环境贴图

显然逐像素地计算是相当复杂的，不过目前的硬件已经提供了灵活的高级着色语言（High Level Shading Language）和像素处理能力。算法实现与快速渲染已不成问题。列表 2-1

1. 可以把 c_1 提取公因式放在加和的外面，如举例中的代码所示。

的例程代码给出了 Cg 语言的实现过程。

列表 2-1 波函数、波函数梯度，以及线与平面相交方程的示范代码

```
// Caustics
// Copyright (c) NVIDIA Corporation. All rights reserved.
//
// NOTE:
// This shader is based on the original work by Daniel Sanchez-Crespo
// of the Universitat Pompeu Fabra, Barcelona, Spain.

#define VTXSIZE 0.01f // Amplitude
#define WAVESIZE 10.0f // Frequency
#define FACTOR 1.0f
#define SPEED 2.0f
#define OCTAVES 5

// Example of the same wave function used in the vertex engine
float wave(float x,
           float y,
           float timer)
{
  float z = 0.0f;
  float octaves = OCTAVES;
  float factor = FACTOR;
  float d = sqrt(x * x + y * y);

  do {
    z -= factor * cos(timer * SPEED + (1/factor) * x * y * WAVESIZE);
    factor = factor/2;
    octaves--;
  } while (octaves > 0);

  return 2 * VTXSIZE * d * z;

}

// This is a derivative of the above wave function.
// It returns the d(wave)/dx and d(wave)/dy partial derivatives.
float2 gradwave(float x,
                float y,
                float timer)
{
  float dZx = 0.0f;
  float dZy = 0.0f;
  float octaves = OCTAVES;
  float factor = FACTOR;
  float d = sqrt(x * x + y * y);

  do {
    dZx += d * sin(timer * SPEED + (1/factor) * x * y * WAVESIZE) * y *
           WAVESIZE -factor *
```

```
            cos(timer * SPEED + (1/factor) * x * y * WAVESIZE) * x/d;
    dZy += d * sin(timer * SPEED + (1/factor) * x * y * WAVESIZE) *
            x * WAVESIZE - factor *
            cos(timer * SPEED + (1/factor) * x * y * WAVESIZE) * y/d;
    factor = factor/2;
    octaves--;
  } while (octaves > 0);

  return float2(2 * VTXSIZE * dZx, 2 * VTXSIZE * dZy);
}

float3 line_plane_intercept(float3 lineP,
                            float3 lineN,
                            float3 planeN,
                            float planeD)
{
  // Unoptimized
  // float distance = (planeD - dot(planeN, lineP)) /
  //                  dot(lineN, planeN);
  // Optimized (assumes planeN always points up)
  float distance = (planeD - lineP.z) / lineN.z;
  return lineP + lineN * distance;
}
```

一旦计算了交点，就可以用相关纹理（dependent texture）读取，从刻蚀的光照贴图（lightmap）中取出（fetch）贴图像素；接着还要把刻蚀添加到地面纹理上，正如在列表 2-2 中显示的。

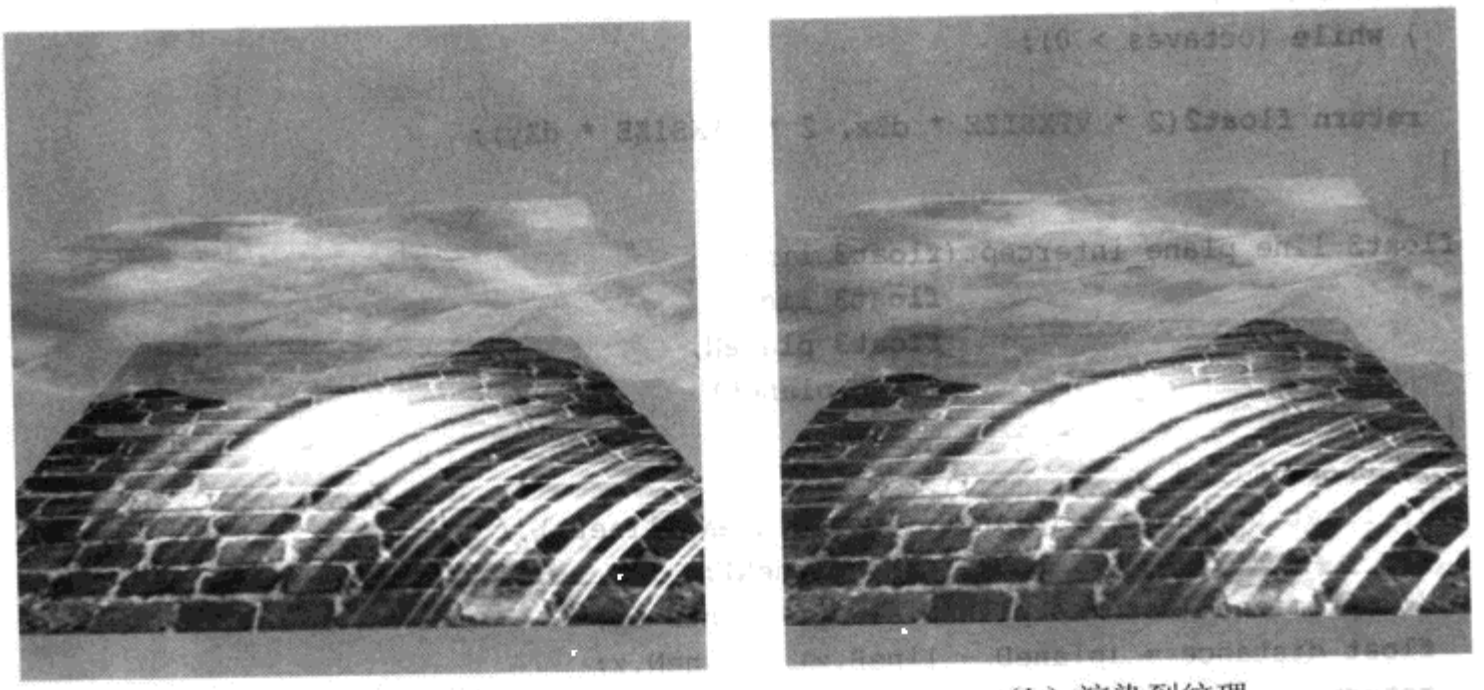
列表 2-2 最终渲染遍次的代码样例，说明相关纹理（dependent texture）的读出操作

```
float4 main(VS_OUTPUT vert,
            uniform sampler2D LightMap : register(s0),
            uniform sampler2D GroundMap : register(s1),
            uniform float Timer) : COLOR
{
  // Generate a normal (line direction) from the gradient
  // of the wave function and intercept with the water plane.
  // We use screen-space z to attenuate the effect to avoid aliasing.
  float2 dxdy = gradwave(vert.Position.x, vert.Position.y, Timer);

  float3 intercept = line_plane_intercept(
                     vert.Position.xyz,
                     float3(dxdy, saturate(vert.Position.w)),
                     float3(0, 0, 1), -0.8);

  // OUTPUT
  float4 colour;
  colour.rgb = (float3)tex2D(LightMap, intercept.xy * 0.8);
  colour.rgb += (float3)tex2D(GroundMap, vert.uv);
  colour.a = 1;
  return colour;
}
```

图 2-9 所示的结果表明，使用逐像素代替逐顶点进行计算，质量得到了改进。同时，纹理的分辨率足够大时，它的质量能够像屏幕空间的渲染一样好，而且在大量像素显示时对性能没有影响，另外，当使用一个渲染目标纹理（render target texture）时，为了减少刻蚀的走样，可以利用自动 mipmap 生成和各向异性滤波器，这是渲染到屏幕空间时不可能办到的。



（a）使用屏幕空间渲染　　（b）渲染到纹理

图 2-9　渲染刻蚀

2.6　小结

在这章中描述了如何利用基于 shader 的技术，将经典的算法升级和提高。随着 shader 处理能力的进一步增强，全部 Monte Carlo 方法将能完全地在图形硬件中运行，那时计算物理正确的刻蚀将会变成现实。

要创建一个完整的、具有美学趣味的水下渲染模型，另外一个值得研究的效果是曙暮辉（crepuscular rays）——水中的光柱。这种效果也以神奇光线著称，看起来像是看得见的光线，是由水中粒子反射所生成。

2.7　参考文献

Foley, James, Andries van Dam, Steven Feiner, and John Hughes. 1996. *Computer Graphics: Principles and Practice,* 2nd ed. Addison-Wesley.

Jensen, Lasse Staff, and Robert Golias. 2001. "Deep-Water Animation and Rendering." Gamasutra article. Available online at **http://www.gamasutra.com/gdce/2001/jensen/jensen_01.htm**

Stam, Jos. 1996. "Random Caustics: Wave Theory and Natural Textures Revisited." Technical sketch. In *Proceedings of SIGGRAPH 1996,* p. 151. Available online at **http://www.dgp.toronto.edu/people/stam/INRIA/caustics.html**

第 3 章 Dawn 演示中的皮肤

Curtis Beeson
NVIDIA
Kevin Bjorke
NVIDIA

3.1 引言

Dawn 是由 NVIDIA 创建的，用来介绍 GeForce FX 产品线的演示程序，它说明如何使用可编程的着色技术（shading）创建出逼真的人类角色，结果如图 3-1 所示。Vertex shaders（在第 4 章“Dawn 演示中的动画”介绍）通过带索引的蒙皮（indexed skining）和形态混合（blend shapes）使一个高分辨率网变形。演示提供了用于设置光照模型的像素 shader 程序（fragment shader）。如果没有可编程的图形硬件 [1]，要想得到由像素 shader 程序（fragment shader）产生的皮肤和翅膀的细节效果，是不可能的。

图 3-1 Dawn

1. 概念和实现曾得到 NVIDIA 的 Gary King 和 Alex Saldaartchouk 的帮助。

这节讨论如何使用可编程图形硬件，来照明与加速角色 Dawn 皮肤的着色。

3.2 皮肤着色

基于多种原因，在计算机图形中模拟皮肤是十分困难的。即使用在电影中应用的高端图形产品模拟出来的真实仿真角色，也经不起近距离的观察。人类可以从中获得大量无言的信息：例如重心的移动，走动的特别习惯，面部的表情，甚至有些人的皮肤泛红等。研究已经表明，婴儿在出生的片刻就能识别并且注视人的面部表情。我们人类可以借助于动作和情感表现，甚至皮肤颜色的细微变化，感觉和察觉到一些无规律的东西。虽然很少有人能理解像“次表面散射”（subsurface scanttering）、“轮廓照明”（rim lighting）这些词汇，但是当把它们渲染错了的时候，几乎任何人都可以指出。

除了着色问题之外，有时人们可能会因为皮肤运动的方式，说皮肤看起来像是塑料。因此角色的设置是很重要的。第 4 章描述了用于 Dawn 演示的技术——使用累积的索引蒙皮（accumulated indexed skinning）来驱动身体，通过混合对象（网格）把表情加到 Dawn 的面部。这些技术对 Dawn 和她的动作提供了尽可能的近似。但仍未能充分地描述附在骨骼上的皮肤的滑动、弯曲和下垂方式。在创建一个角色的时候，重要的是在观察者实际可见的观察范围内，对角色运动和着色（shading）进行建模。

3.3 场景的照明

一旦皮肤有了拟真的运动方式，就可以把注意力集中在着色上。最直接的办法是把一个表面的着色（shading）分解，先考查场景的照明环境，然后再看皮肤对光的响应。

3.3.1 高动态范围的环境

我们想把创建的角色放置在不同于传统的光照设定中。老的图形架构建立在 Gouraud 和 Phong 光照的概念上。即把所有的入射光看成是从无穷小点光源、方向光或聚光灯光源发出的。使用纹理以及相应公式来反映物体表面的漫反射和镜面高光反射。但它们未能描述出一个合适的光照响应范围。

上述方法的一个不理想之处是，场景往往达不到真实世界中的亮度范围。在地面上的一张白纸亮得像是一个白色光源；在复杂环境中的角色常常看起来不对劲，因为它们不是被周围的光照亮，而是由几个点光源照亮的。

Paul Debevec 针对这一问题，已经在高动态范围的图像处理方面做了大量的研究。他的工作可以在多种 SIGGRAPH 出版物中找到。他渲染的影片[2]给出了更为贴切的解释。我们力图借助新型可编程的实时图形架构，把他的概念应用于一个实时渲染的角色。我们尤其希望使用环境光照亮 Dawn，由亮斑照亮肌肤肉色，并且柔化她的轮廓边缘。

因为 Dawn 是一个仙女，我们找到一片合适的森林，拍了大量标准的照片。我们用了一个 iPIX 工具包，包括一个可从相反角度锁定的三脚架、一个数码相机，和一个 183 度的鱼眼

2. Paul Debevec 的许多作品可以在 www. debevec.org 找到。

镜头。使用相机内部的宏操作，以多个快门拍摄两张半球环景照片。

我们使用 iPIX 软件把对应的半球半景照片缝合为一个全景，然后用 Debevec 的 HDRShop 软件在每个给定角度对环境光的颜色和强度进行编码，创建高动态范围的全景，见图 3-2。随后 HDRShop 可以创建一个漫反射查找表，它执行一个卷积，用给定的表面粗糙度为表面创建漫反射或镜面高光反射映射图（过去这项操作使用 Phong 指数模型）。

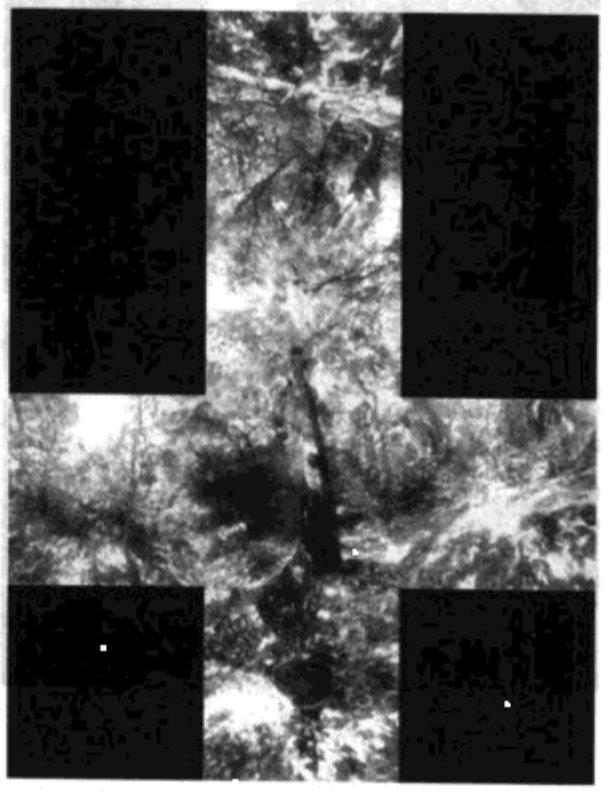


图 3-2　立方体环境反射贴图

如图 3-3 所示，漫反射映射图（diffuse map）是一个立方体映射贴图（cube map）。它使用网格表面的法线作为索引。每个像素存储了相应法线与入射光夹角的余弦加权平均值（半球入射光）。假如不考虑在半球上给定某点的部分可能被遮挡，加权平均给出了适当的数值用于计算物体表面上某点的漫反射光照。

图 3-3　漫反射立方体环境映射贴图

如图 3-4 所示，镜面高光反射映射图（specular map）也是一个立方体映射贴图（cube map），使用反射矢量作为索引（类似于立方体映射图通常用于的环境反射）。把这个镜面高光反射映射图基于粗糙因子进行模糊，为的是模拟对表面任何给定点上的法线的改变。

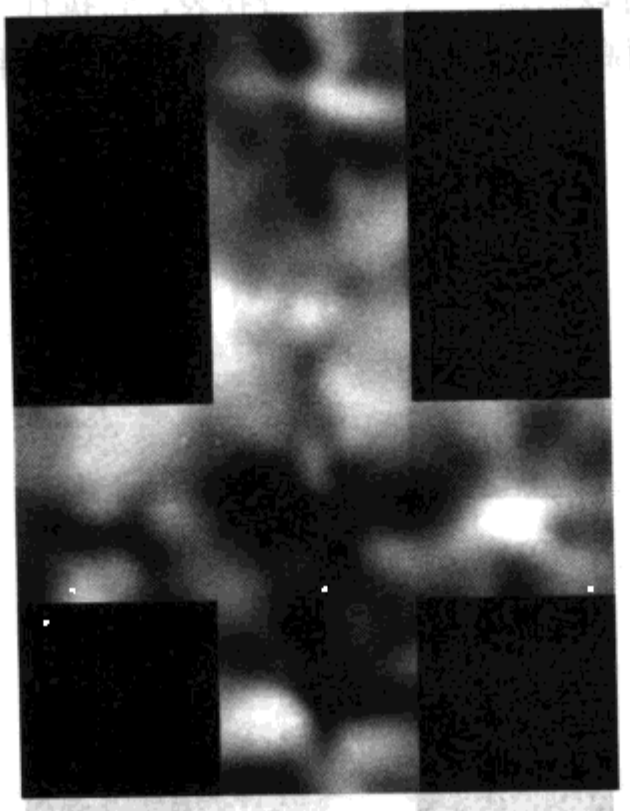


图 3-4 镜面高光立方体环境映射贴图

3.3.2 遮挡

使用环境映射贴图（environment maps）描述环境或照明存在一个问题，即被映射的样本，只对空间中的一个点是正确的。这样，第一个副作用是，如果仙女在镜头附近走，背景会呈现在“无限”远处。因为在这样的情况[3]下仙女很小，背景应该很远，所以这个近似还是合适的。

第二个副作用是，漫反射和镜面高光映射图（diffuse and specular map）考虑了来自所拍照环境的入射光，但是不包括由物体引起的阴影，例如 Dawn 本身的阴影。因此。必须开发一种技术来包含这个信息。

许多实时阴影技术产生的阴影都很清晰，但我们想要特别模糊的阴影。因此，我们生成一个遮挡项值，用来近似表达在每个顶点上半球辐射光中，有多大比率被场景中其他物体所遮挡[4]。

具体做法是，使用一个软件工具，由顶点随机地投射出半球光线，求出其他几何体对光的遮挡。我们将这个技术用于 Dawn 演示和 Ogre 演示（版权属于 Spellcraft Studio）。Dawn 身体网格顶点遮挡项值保持为常数；对她的脸则使用了来自各变形（morph）对象网格遮挡项值的混合值。

在 GeForce FX 5800 的试验品（Dawn 是为该产品上市所创建的演示）中达到了极限效果后，我们就大胆地对每帧中的每个顶点使用遮蔽项，对 Spellcraft Studio 的样本 Yeah! The

3. 可以在不同点所取的全景或偏景之间内插，用于模拟类似 plenoptic 建模的传播（McMillan and Bishop 1995）。
4. 相似的技术在本书第 17 章“环境遮蔽”有概略说明。

Movie 中的角色 Ogre（食人魔鬼）就是这样处理的。

3.4　皮肤如何对光进行响应

皮肤不像大多数在计算机绘图学中建模的表面，因为它是由半透明的表皮、真皮和皮下组织等数层构成的。“次表面的散射”是描述这种现象的一个术语：进入表面的光，通过介质散射，然后从另外一个点射出。这种现象很普遍：当在太阳面前向上举起手时，就能看到穿过皮肤的桔红色的光。皮肤下的散射在所有的角度上显现皮肤形态，使它具有了柔软的、与众不同的特征。可惜的是，这个真实情况违背了图形应用编程接口和结构的一个通常假设，即，物件上某一点的光不能影响其他点的反射。

过去，一些小组尝试使用多层纹理贴图模仿皮肤的复杂性。一般而言，这个方法是难以管理的，并且让纹理美工预想出最终的所混合的颜色而工作也是很难的。相反地，我们使用单个的彩色图，通过着色程序增加其色彩变化，如图 3-5 所示。

图 3-5　Dawn 头部的前半边的漫反射图
纹理分辨率是 1024 × 1024

另外，皮肤具有一些极细小的变化影响其反射特性，特别是当光源正对着照相机位置直射的时候，需要皮肤轮廓边缘的照明和给边缘加上光晕。真正的皮肤具有一些细微的特征，比如汗毛和毛孔能捕捉光。尽管这些细节用于显式地建模是太不明显了，但我们还是希望得到一个合适而完整的皮肤表面。当特写时，增加凹凸贴图可以提供一些额外的细节，特别是小的皱纹；但我们想要的是柔软的皮肤形态，不是有光闪闪的塑料（通常只有当特写时才能看得见凹凸贴图）。

我们可以通过建模来近似这两个着色属性，建模可以是基于表面法线的简单公式，或者

是基于光线或视线矢量的简单公式。我们特别地沿着 Dawn 的轮廓边缘，对她后面发出的光取样，按照视线矢量的索引，让“穿过”Dawn 的光与她的基础皮肤色调混合，从而创建次表面发光和边缘效果的照明效果。尤其是在背景图中更加明亮的区域，见图 3-6。

图 3-6 Dawn 的头部前面的切线空间法线贴图（凹凸贴图）提高了对比度。纹理分辨率是 1024 × 1024

3.5 实现

光照的处理分成顶点 shader 和像素 shader。这是单遍次（one-pass）渲染的光照解决方案——不需要另外的渲染遍次或 alpha 混合来创建皮肤表面。

3.5.1 顶点 Shader

顶点 shader 的主要功能是把坐标转变到投影空间，并且执行那些不能在像素 shader 中执行的数学运算。如第 3.3.2 节所提到，Dawn 演示中顶点 shader 首先用于变形 morph 目标网格（如果需要变形），然后对带有 98 个骨头的骨架和 180 000 个以上顶点的蒙皮进行运算。更多请见第 4 章。

对于每个顶点，我们从 CPU 中接收的数据列于程序表 3-1[5]。

在顶点 shader 中计算、插值后作为像素 shader 的输入数据，包括世界-空间视线方向矢量（WorldEyeDirection）（它描述从观察者的眼睛到任意给定顶点的方向）、切线-空间到世界-空间的 3 × 3 矩阵（tangentToWorld）[6] 和一个称为血液传输项的多项集合的变量（bloodTransmission）。列表 3-2 显示了顶点输出的数据结构。

5. 顶点 shader 的许多数据和代码，属于本书的第 4 节“Dawn 样本的动画”中所描述的蒙皮和变形目标网格。
6. 只需要 3 × 3 的矩阵，因为表面矢量（法线，副法线和切线）变换与顶点变换方式不同。

列表 3-1　从 CPU 应用中接收的每个顶点的数据

```
// Here is the PER-VERTEX data - we use 16 vectors,
// the maximum permitted by our graphics API
struct a2vConnector {
  float4 coord;                    // 3D location
  float4 normal;
  float4 tangent;
  float3 coordMorph0;              // 3D offset to target 0
  float4 normalMorph0;             // matching offset
  float3 coordMorph1;              // 3D offset to target 1
  float4 normalMorph1;             // matching offset
  float3 coordMorph2;              // 3D offset to target 2
  float4 normalMorph2;             // matching offset
  float3 coordMorph3;              // 3D offset to target 3
  float4 normalMorph3;             // matching offset
  float3 coordMorph4;              // 3D offset to target 4
  float4 normalMorph4;             // matching offset
  float4 boneWeight0_3;            // skull and neck bone
  float4 boneIndex0_3;             // indices and weights
  float4 skinColor_frontSpec;      // UV indices
};
```

列表 3-2　输出顶点的数据结构

```
// Here is the data passed from the vertex shader
// to the fragment shader
struct v2fConnector {
  float4 HPOS                  : POSITION;
  float4 SkinUVST              : TEXCOORD0;
  float3 WorldEyeDir           : TEXCOORD2;
  float4 SkinSilhouetteVec     : TEXCOORD3;
  float3 WorldTanMatrixX       : TEXCOORD5;
  float3 WorldTanMatrixY       : TEXCOORD6;
  float3 WorldTanMatrixZ       : TEXCOORD7;
};
```

因为是做凹凸映射，像素 shader 必须得到世界空间的凹凸法线，所以须提供一个方法，把切线空间的凹凸法线（由一个凹凸纹理提供的）转换到世界空间中去。做凹凸映射的通常方法是得到顶点 shader 中的世界空间的法线、副法线和切线，然后把这 3 个矢量作为一个 3 × 3 矩阵，从世界空间转置到切线空间中去进行计算。而在这种情况下，像素 shader 必须使用世界空间的矢量求光照的解，所以使用这个矩阵的转置（旋转矩阵的逆是它的转置）。在顶点 shader 中，使用 9 个 MOV 指令，载入 WorldTanMatrixX、WorldTanMatrixY 和 WorldTanMatrixZ 项。

最后，顶点 shader 的“血液传输”或“皮肤轮廓”项是一个 float4 矢量，它是由遮挡项、表达式（N.V）的各种变化（即，表面法线和观察矢量的点积）和相对于立方体映射光照坐标系旋转的表面法线组成的，见图 3-7。

```
OUT.SkinSilhouetteVec = float4(objectNormal.w,
                               oneMinusVdotN * oneMinusVdotN,
```

```
                oneMinusVdotN,
                vecMul(G_DappleXf, worldNormal.xyz).z);
```

这些步骤只是提供了几个参数化的方式来说明像素离轮廓有多近。计算能力强的读者可能会注意到那个（$N \cdot V$）项不可能通过 Gouraud（线形）着色正确地在大的三角形上插值。在这种情况下，幸运的是 Dawn 是高细分的网格（tessellate），照相机能够非常接近她的脸而不会出现任何问题。这是在预定屏幕范围、对一个特殊模型使用专门指定的 shader 的例子。项（$N \cdot V$）也可以在像素 shader 中运算，但那将非常耗费资源（因此也比较慢）。

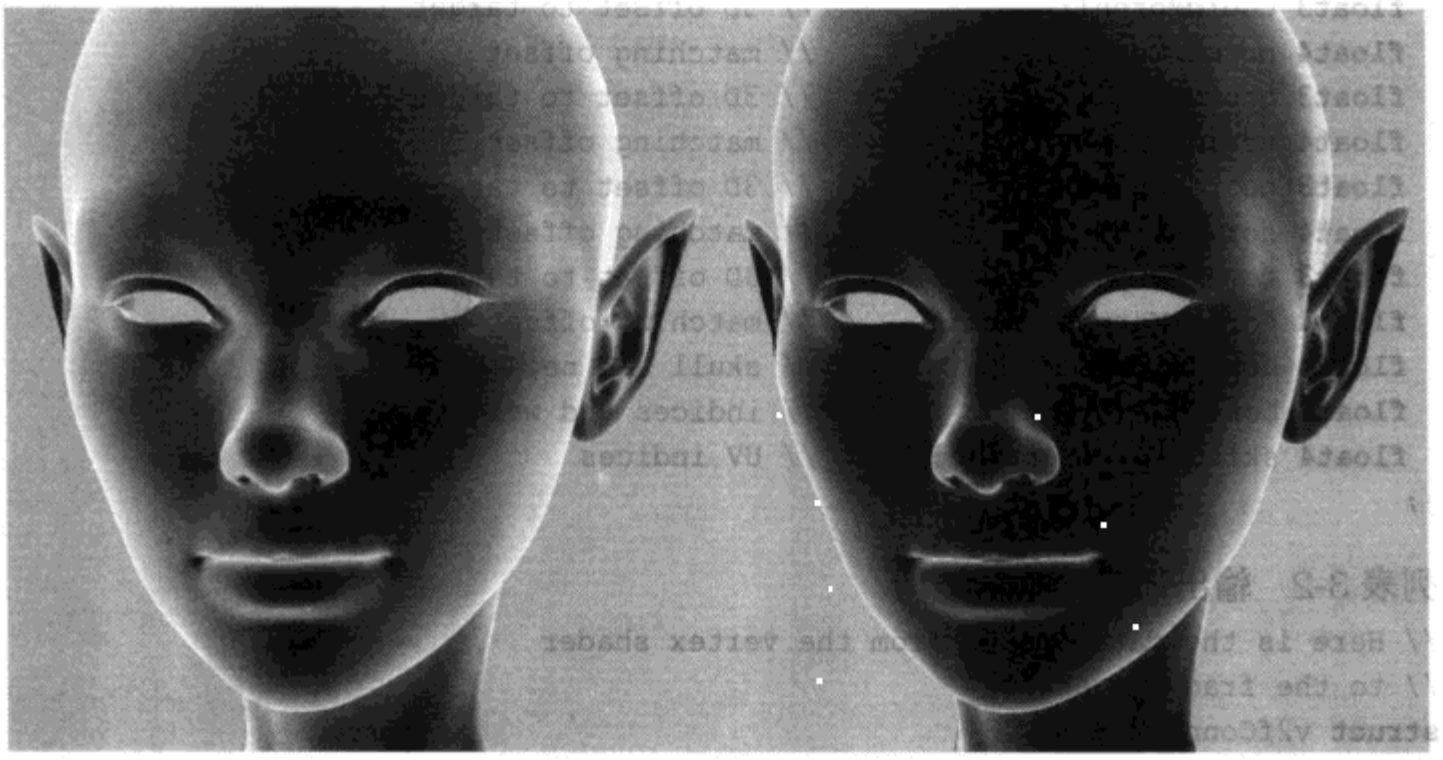


图 3-7 存储在 SkinSilhouetteVec 中的点积
左边，$(1-(N \cdot V))$；右边，$(1-(N \cdot V))^2$

注意 Dawn 演示中所有顶点 shader 是程序产生的。我们在 Maya 中指定像素 shader。同时，用一个顶点 shader 发生器，根据角色设置（骨骼、变形目标网格，等等），以及像素 shader 的输入参数需求，利用一个规则文件，使用 A*搜索[7]来产生最优化的顶点 shader。

列表 3-3 是一个有注释的顶点 shader 的范例，用于 Dawn 的面部（矩阵蒙皮、网格变形连同在像素 shader 中用于颜色计算的值）。

列表 3-3 为 Dawn 面部取样的顶点 Shader

```
// Helper function:
// vecMul(matrix, float3) multiplies like a vector
// instead of like a point (no translate)
float3 vecMul(const float4x4 matrix, const float3 vec)
{
  return(float3(dot(vec, matrix._11_12_13),
                dot(vec, matrix._21_22_23),
                dot(vec, matrix._31_32_33)));
}
```

7. A*是蛮力搜索法的一种变形，它使用一个消耗合计衡量（aggregate-cost metric）去引导搜索达到最佳的结果。

```
// The Vertex Shader for Dawn's Face
v2fConnector faceVertexShader(a2vConnector IN,
  const uniform float MorphWeight0,
  const uniform float MorphWeight1,
  const uniform float MorphWeight2,
  const uniform float MorphWeight3,
  const uniform float MorphWeight4,
  const uniform float4x4 BoneXf[8],
  const uniform float4 GlobalCamPos,
  const uniform float4x4 ViewXf,
  const uniform float4x4 G_DappleXf,
  const uniform float4x4 ProjXf)
{
  v2fConnector OUT;

  // The following large block is entirely
  // concerned with shape skinning.
  // First, do shape blending between the five
  // blend shapes ("morph targets")
  float4 objectCoord = IN.coord;
  objectCoord.xyz += (MorphWeight0 * IN.coordMorph0);
  objectCoord.xyz += (MorphWeight1 * IN.coordMorph1);
  objectCoord.xyz += (MorphWeight2 * IN.coordMorph2);
  objectCoord.xyz += (MorphWeight3 * IN.coordMorph3);
  objectCoord.xyz += (MorphWeight4 * IN.coordMorph4);

  // Now transform the entire head by the neck bone
  float4 worldCoord = IN.boneWeight0_3.x *
                      mul(BoneXf[IN.boneIndex0_3.x], objectCoord);
  worldCoord += (IN.boneWeight0_3.y *
                 mul(BoneXf[IN.boneIndex0_3.y], objectCoord));
  worldCoord += (IN.boneWeight0_3.z *
                 mul(BoneXf[IN.boneIndex0_3.z], objectCoord));
  worldCoord += (IN.boneWeight0_3.w *
                 mul(BoneXf[IN.boneIndex0_3.w], objectCoord));

  // Repeat the previous skinning ops
  // on the surface normal
  float4 objectNormal = IN.normal;
  objectNormal += (MorphWeight0 * IN.normalMorph0);
  objectNormal += (MorphWeight1 * IN.normalMorph1);
  objectNormal += (MorphWeight2 * IN.normalMorph2);
  objectNormal += (MorphWeight3 * IN.normalMorph3);
  objectNormal += (MorphWeight4 * IN.normalMorph4);
  objectNormal.xyz = normalize(objectNormal.xyz);
  float3 worldNormal = IN.boneWeight0_3.x *
                       vecMul(BoneXf[IN.boneIndex0_3.x],
                              objectNormal.xyz));
  worldNormal += (IN.boneWeight0_3.y *
                  vecMul(BoneXf[IN.boneIndex0_3.y],
                         objectNormal.xyz));
```

```
worldNormal += (IN.boneWeight0_3.z *
                vecMul(BoneXf[IN.boneIndex0_3.z],
                       objectNormal.xyz));
worldNormal += (IN.boneWeight0_3.w *
                vecMul(BoneXf[IN.boneIndex0_3.w],
                       objectNormal.xyz));
worldNormal = normalize(worldNormal);

// Repeat the previous skinning ops
// on the orthonormalized surface tangent vector
float4 objectTangent = IN.tangent;
objectTangent.xyz = normalize(objectTangent.xyz -
                              dot(objectTangent.xyz,
                                  objectNormal.xyz) *
                                  objectNormal.xyz);
float4 worldTangent;
worldTangent.xyz = IN.boneWeight0_3.x *
                   vecMul(BoneXf[IN.boneIndex0_3.x],
                          objectTangent.xyz);
worldTangent.xyz += (IN.boneWeight0_3.y *
                     vecMul(BoneXf[IN.boneIndex0_3.y],
                            objectTangent.xyz));
worldTangent.xyz += (IN.boneWeight0_3.z *
                     vecMul(BoneXf[IN.boneIndex0_3.z],
                            objectTangent.xyz));
worldTangent.xyz += (IN.boneWeight0_3.w *
                     vecMul(BoneXf[IN.boneIndex0_3.w],
                            objectTangent.xyz));
worldTangent.xyz = normalize(worldTangent.xyz);
worldTangent.w = objectTangent.w;

// Now our deformations are done.
// Create a binormal vector as the cross product
// of the normal and tangent vectors
float3 worldBinormal = worldTangent.w *
                       normalize(cross(worldNormal, worldTangent.xyz));

// Reorder these values for output as a 3 x 3 matrix
// for bump mapping in the fragment shader
OUT.WorldTanMatrixX = float3(worldTangent.x,
                             worldBinormal.x, worldNormal.x);
OUT.WorldTanMatrixY = float3(worldTangent.y,
                             worldBinormal.y, worldNormal.y);
OUT.WorldTanMatrixZ = float3(worldTangent.z,
                             worldBinormal.z, worldNormal.z);

// The vectors are complete. Now use them
// to calculate some lighting values
float4 worldEyePos = GlobalCamPos;
OUT.WorldEyeDir = normalize(worldCoord.xyz - worldEyePos.xyz);
float4 eyespaceEyePos = {0.0f, 0.0f, 0.0f, 1.0f};
```

```
    float4 eyespaceCoord = mul(ViewXf, worldCoord);
    float3 eyespaceEyeVec = normalize(eyespaceEyePos.xyz -
                                      eyespaceCoord.xyz);
    float3 eyespaceNormal = vecMul(ViewXf, worldNormal);
    float VdotN = abs(dot(eyespaceEyeVec, eyespaceNormal));
    float oneMinusVdotN = 1.0 - VdotN;
    OUT.SkinUVST = IN.skinColor_frontSpec;
    OUT.SkinSilhouetteVec = float4(objectNormal.w,
                                   oneMinusVdotN * oneMinusVdotN,
                                   oneMinusVdotN,
                                   vecMul(G_DappleXf, worldNormal.xyz).z);
    float4 hpos = mul(ProjXf, eyespaceCoord);
    OUT.HPOS = hpos;
    return OUT;
}
```

3.5.2　像素 Shader

给定顶点 shader 的输出（在 Dawn 的身体各处，顶点 shader 输出一致的数据结构），我们可以产生真实纹理的颜色。

列表 3-4 给出了面部所用的完全的像素 shader。

列表 3-4　面部所用的完全的像素 shader

```
float4 faceFragmentShader(v2fConnector IN,
  uniform sampler2D SkinColorFrontSpecMap,
  uniform sampler2D SkinNormSideSpecMap, // xyz normal map
  uniform sampler2D SpecularColorShiftMap, // and spec map in "w"
  uniform samplerCUBE DiffuseCubeMap,
  uniform samplerCUBE SpecularCubeMap,
  uniform samplerCUBE HilightCubeMap) : COLOR
{
  half4 normSideSpec tex2D(SkinNormSideSpecMap,
                           IN.SkinUVST.xy);
  half3 worldNormal;
  worldNormal.x = dot(normSideSpec.xyz, IN.WorldTanMatrixX);
  worldNormal.y = dot(normSideSpec.xyz, IN.WorldTanMatrixY);
  worldNormal.z = dot(normSideSpec.xyz, IN.WorldTanMatrixZ);
  fixed nDotV = dot(IN.WorldEyeDir, worldNormal);
  half4 skinColor = tex2D(SkinColorFrontSpecMap, IN.SkinUVST.xy);
  fixed3 diffuse = skinColor * texCUBE(DiffuseCubeMap, worldNormal);
  diffuse = diffuse * IN.SkinSilhouetteVec.x;
  fixed4 sideSpec = normSideSpec.w * texCUBE(SpecularCubeMap,
                                             worldNormal);
  fixed3 result = diffuse * IN.SkinSilhouetteVec.y + sideSpec;
  fixed3 hilite = 0.7 * IN.SkinSilhouetteVec.x *
                  IN.SkinSilhouetteVec.y *
                  texCUBE (HilightCubeMap, IN.WorldEyeDir);
  fixed reflVect = IN.WorldEyeDir * nDotV - (worldNormal * 2.0x);
  fixed4 reflColor = IN.SkinSilhouetteVec.w *
```

```
                        texCUBE(SpecularCubeMap, reflVect);
    result += (reflColor.xyz * 0.02);
    fixed hiLightAttenuator = tex2D(SpecularColorShiftMap,
                                    IN.SkinUVST.xy).x;
    result += (hilite * hiLightAttenuator);
    fixed haze = reflColor.w * hiLightAttenuator;
    return float4(result.xyz, haze);
}
```

首先，我们得到凹凸映射的表面法线。切线空间的凹凸表面法线存储在 SkimNormSideSpecMap 纹理的 RGB 值中。高光强度贴图（使用灰度值表示高光强度的大小）则包含在此纹理的 alpha 通道（为了代码清楚起见，这里我们将 RGB 表示为 *xyz*）。使用 WorldTanMatrix 旋转切线空间中 *xyz* 的值，将凹凸表面法线完全变换到世界坐标。这是为了世界空间中光照计算的需要。

接着计算表面法线与视线向量的点积。稍后会使用此值。

```
half4 normSideSpec tex2D(SkinNormSideSpecMap,
                         IN.SkinUVST.xy);
half3 worldNormal;
worldNormal.x = dot(normSideSpec.xyz, IN.WorldTanMatrixX);
worldNormal.y = dot(normSideSpec.xyz, IN.WorldTanMatrixY);
worldNormal.z = dot(normSideSpec.xyz, IN.WorldTanMatrixZ);
fixed nDotV = dot(IN.WorldEyeDir, worldNormal);
```

漫反射颜色是皮肤纹理贴图，乘以漫反射立方体映射贴图（diffuse cube map）。同时 *s* 代入 SkinSilhouetteVec 的半球遮挡项值，对颜色进行调制（相乘）。

```
half4 skinColor = tex2D(SkinColorFrontSpecMap, IN.SkinUVST.xy);
fixed3 diffuse = skinColor * texCUBE(DiffuseCubeMap, worldNormal);
diffuse = diffuse * IN.SkinSilhouetteVec.x;
```

角色边缘的镜面高光颜色来自镜面高光立方体映射贴图（specular cube map），乘以高光照明强度。而高光强度是从法线贴图的 alpha 通道中得到（即，在 SkinNormSideSpecMap 的 alpha 通道中）。最后即得到综合的结果。

```
fixed4 sideSpec = normSideSpec.w * texCUBE(SpecularCubeMap,
                                           worldNormal);
fixed3 result = diffuse * IN.SkinSilhouetteVec.y + sideSpec;
```

接着，以向量 WorldEyeDir 为索引，从传统的立方体环境反射贴图中（立方体环境反射贴图也是事先制作好，见图 3-2）得到 Dawn 后面的环境颜色。将这些连同一些美工的“梦幻因素”加到结果中去。

```
fixed3 hilite = 0.7 * IN.SkinSilhouetteVec.x *
                IN.SkinSilhouetteVec.y *
                texCUBE (HilightCubeMap, IN.WorldEyeDir);
fixed reflVect = IN.WorldEyeDir * nDotV - (worldNormal * 2.0x);
fixed4 reflColor = IN.SkinSilhouetteVec.w *
                   texCUBE(SpecularCubeMap, reflVect);
result += (reflColor.xyz * 0.02);
```

```
fixed hiLightAttenuator = tex2D(SpecularColorShiftMap,
                                IN.SkinUVST.xy).x;
result += (hilite * hiLightAttenuator);
```

最后，把一点额外的轮廓信息加进最终输出的 alpha 频道中去，这样在 alpha 混合时，沿着 Dawn 轮廓边缘的“发光”可以看起来更自然些。

```
fixed haze = reflColor.w * hiLightAttenuator;
return float4(result.xyz, haze);
```

3.6　小结

虽然许多在 Dawn 皮肤 shader 上的实施细节，对多数游戏引擎而言可能限制太多，但大部分的概念还是能用其他方法实现的。高动态范围、次表面散射、给皮肤边缘加上光晕，还有其他类似的基本原理，也能用合成的光源或其他的场景信息来计算。

在许多方面，由环境照亮的 Dawn 很难处理。更复杂和更逼真的光照往往要美工更复杂的制作。在这个例子中，我们想要她的肌皮疙瘩（goose）凹凸更加可见，但是环境的漫射太强，为了补偿，我们只能过分地不自然地夸张渲染她表面的凹凸。

假如能第 2 次实现 Dawn，或许可以使用一个混合方法去进行光照，我们会查询漫反射贴图和镜面高光映射贴图（给出平滑的法线），然后使用一个基本的“光方向”去计算凹凸贴图的贡献。这使我们能更加直接地控制凹凸贴图，并且免去在顶点 shader 执行的费时的矩阵转置。

3.7　参考文献

Debevec, Paul. 1998. “Rendering Synthetic Objects into Real Scenes: Bridging Traditional and Image-Based Graphics with Global Illumination and High Dynamic Range Photography.” In *Proceedings of SIGGRAPH 98*, pp. 189～198.

Gritz, Larry, Tony Apodaca, Matt Pharr, Dan Goldman, Hayden Landis, Guido Quaroni, and Rob Bredow. 2002. “RenderMan in Production.” Course 16, SIGGRAPH 2002.

McMillan, Leonard, and Gary Bishop. 1995. “Plenoptic Modeling: An Image-Based Rendering System.” In *Proceedings of SIGGRAPH 1995*.

Stout, Bryan. 1996. “Smart Moves: Intelligent Pathfinding.” *Game Developer*, October 1996. Available online at the Gamasutra Web site:

http://www.gamasutra.com/features/19970801/pathfinding.htm

第 4 章 Dawn 演示中的动画

Curtis Beeson
NVIDIA

4.1 简介

Dawn 是由 NVIDIA 公司创建的演示，目的是介绍 GeForce FX 显卡产品线，同时举例说明如何使用高级语言（如 HLSL 或 Cg）来创建一个逼真的人物角色。顶点 shader 通过索引的蒙皮和变形网格对象（morph target）来使一个高分辨率网格变形。顶点 shader 用于设置像素 shader 中的光照模型。像素 shader 为皮肤和翅膀提供了在高级的可编程图形硬件问世之前难以观察到的范围和细节，见图 4-1。

图 4-1 一个实时 Dawn 的屏幕截图

这节讨论如何使用可编程图形硬件，来加速演示中人物 Dawn 的动画。

4.2　网格的动画

传统上，一般不在 CPU 上执行耗时的复杂网格动画，因为 CPU 已经负担了物理模拟、人工智能和其他必需的计算。而新的图形硬件使用可编程的顶点和像素 shader，取代了传统的固定功能的渲染管道，并可以减轻一些 CPU 的负担。

但有时候还是必须在 CPU 上执行这种运算。许多基于模板（stencil-based）的阴影体技术必须遍历转换过的网格来寻找轮廓边缘；而动态阴影棱台的生成也最好在 CPU 上运行（见第 9 节，“高效的阴影体渲染”）。在场景中的每帧，人物要多次绘制进阴影缓冲和辉光（glow）缓冲，以及其他这种临时缓冲。如果应用程序使用有限的顶点，可能最好还是在 CPU 上进行变形。是在 CPU 还是在 GPU（图形处理器）上进行网格变形，应该针对每个应用、有时甚至要针对每个物体来决定。

Dawn 人物的建模、纹理和动画主要是用 Alias 系统的 Maya 软件包制作。因此我们的网格动画方法是以软件提供的工具集为基础。我们在 discreet 的 3ds max 软件包上，使用同样的技术制作了一个类似的演示（即用于 GeForce FX 5900 上市的演示 Dusk）；这些方法对多种建模软件包都是通用的，并不依赖于任何单一的工作流程。用在这两个演示的方法是（有索引的）蒙皮法，它的顶点受一组加权的矩阵和加权变形对象（Morph Target）的影响，可用来控制 Dawn 的面部表情。

4.3　变形网格对象

使用变形网格对象是表现复杂的网格变形的一个通常的方法，NVIDIA 演示小组已经使用这种技术创建了多个演示。Zoltar 和 Yeah! The Movie 演示（内容由 Spellcraft Studio 提供）从每秒插值 30 网格开始，然后以累积误差原则去除网格关键帧。这使得我们能够缩小文件并减少存储空间，即使减为原始关键帧的 2/3，也几乎不会出现可见的失真（artifacts）。在这种类型的网格插值中，任何给定时间中只有两个插值关键帧激活，而且它们是连续地执行的。

变形网格对象也能够并行地使用。Dawn 是显示这个有用方法的样板。由一个标准头部模型（27 000 个三角形）开始，我们的美工创建这个头的 50 个副本，并且把它们建模成一组变形网格对象，如图 4-2 所示。大约其中的 30 个头对应着情绪（例如快乐、忧愁和深思等等），而另外 20 多个是调节器（modifiers）（例如左边的眉毛上挑、右边的眉毛上挑、嘻嘻地笑等等）。在这种风格的动画中，变形网格对象的权重可以加起来不等于 1，比如说，可能有（0.8* 快乐+1.0*耳朵摆动），毕竟 Dawn 是个仙女。

虽然这种复杂表情面孔的制作，也许可以由更多表情元素的调节器（modifiers）来混合，但我们的美工发现，在所要求的姿势上给脸建模更为直观，因为如果没有配合眼睛、面颊和嘴一起运动，对像皱眉那样的元素孤立建模是很困难的。这种合并也对硬件寄存器的局限性有所帮助，这一点后面将要谈到。

4.3.1　高级语言中的变形网格对象

幸运的是，在 HLSL 或 Cg 中变形网格对象的实现很简单。假定 VertexIn 是包含每个顶

点数据的结构，以线性或连续方式应用变形网格对象是很容易的。

```
float4 position = (1.0f - interp) * vertexIn.prevPositionKey + interp * vertexIn.
                  nextPositionKey;
```

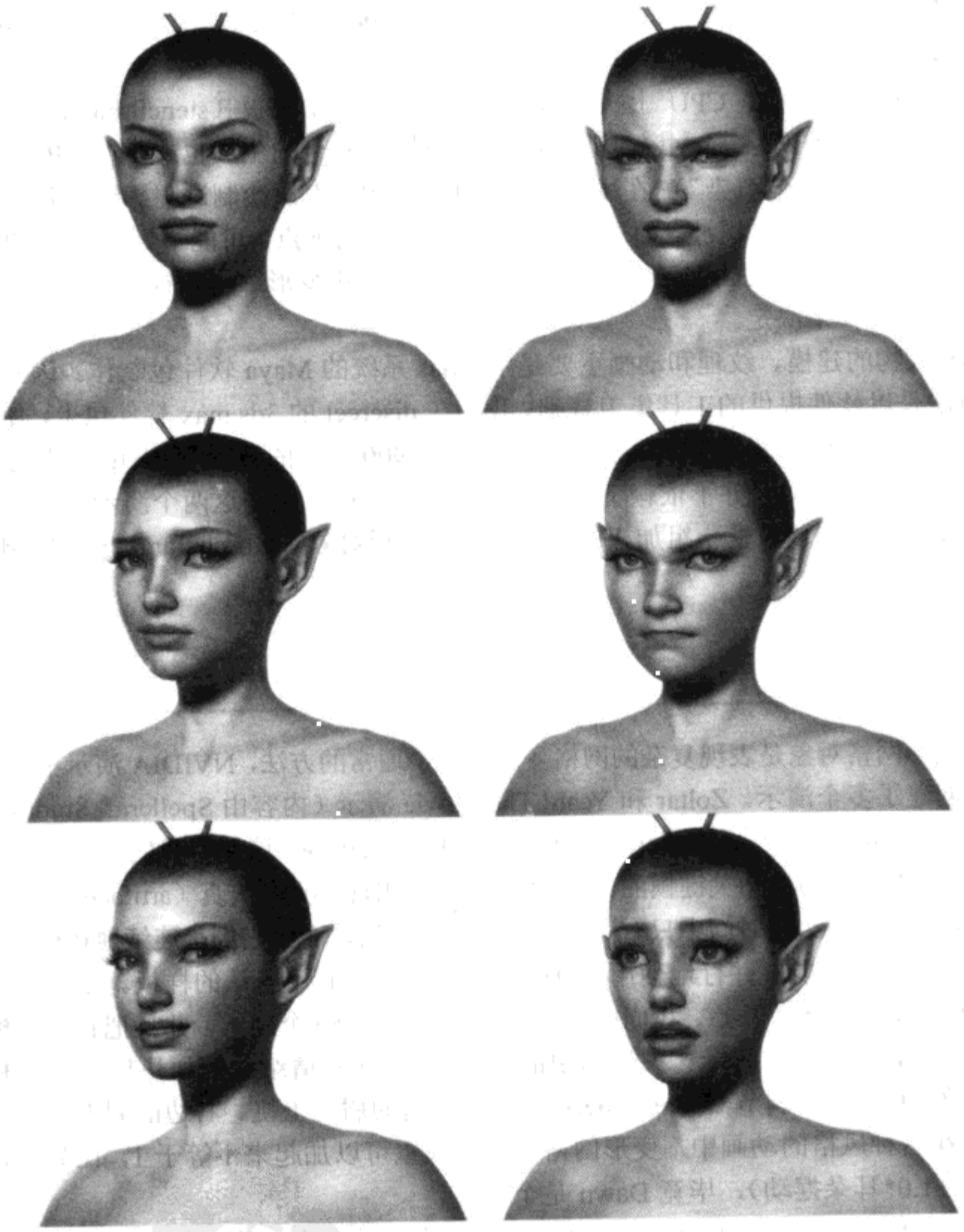


图 4-2 表情的混合对象（混合形状）

在这个代码中，interp 是 shader 中一个常量输入参数，prevPositionKey 和 nextPositionKey 分别是顶点前一时刻和下一时刻的位置。当并行地应用变形网格对象的时候，无表情标准姿态网格（译者注：也就是不带任何表情初始建立的头部网格）和变形网格对象相应顶点的空间位置不同，形成一个空间差向量。然后我们将一个系数加权到此空间差向量上。结果是把 1.0 的加权值应用于每个顶点的偏移，以得到表情变形网格对象。但是对于每个不同表情变形

网格对象可以分别地应用。每个变形网格对象的应用只是一个“乘法-加法”指令。

```
// vertexIn.positionDiffN = position morph target N - neutralPosition
float4 position = neutralPosition;
position += weight0 * vertexIn.positionDiff0;
position += weight1 * vertexIn.positionDiff1;
position += weight2 * vertexIn.positionDiff2;
. . .
```

4.3.2　变形网格对象的实现

我们需要变形网格对象影响顶点的位置以及法线、副法线和切线，这样它们可以影响在像素 shader 中实施的光照。仅仅执行前面提到的代码，来计算顶点位置、法线、副法线和切线，看上去似乎就可以了。但是这样顶点输入寄存器很容易用完。我们在写 Dawn 和 Dusk 演示时，GPU 最多可以映射顶点的 16 个输入属性（vertex attribute）。网格顶点输入属性依次为：中间姿态网格的顶点位置、法线、副法线、纹理坐标，骨骼权重和骨骼索引（稍后描述）。为变形网格对象留下 10 个顶点输入属性备用。虽然也能映射切线，但我们在顶点 shader 中取法线和副法线的叉积得到切线，目的是省下一个额外的顶点输入属性。

因为在设置顶点 shader 顶点输入属性时，每个变形网格顶点空间差向量要占去一个顶点输入属性，我们可以使用 10 个变形网格对象来混合头部网格形态，只取网格顶点位置；或使用 5 个变形网格对象来混合，取它们的顶点位置和法线；或使用 3 个变形网格，取顶点位置—法线—副法线 3 个属性；或只使用两个变形网格对象，取它们的顶点位置—法线—副法线—切线。我们最后选择在顶点 shader 中使用 5 个混合网格形态，这 5 个混合网格形态只改变顶点位置和法线。顶点 shader 对新的法线与无表情网格顶点的切线进行正交规一化，（orthonormalization）来得到新的切线（即，从中间姿态网格顶点切线减去新法线的共线元素，然后规一化），对新切线与新法线求叉积得到副法线。若网格不围绕法线扭曲，正交规一化是一个合理的近似。

```
// assumes normal is the post-morph-target result
// normalize only needed if not performed in fragment shader
float3 tangent = vertexIn.neutralTangent - dot(vertexIn.neutralTangent, normal) * normal;
tangent = normalize(tangent);
```

这样我们有一个带有 50 个变形网格对象的数据集，但是在任何给定时间只可能激活 5 个（即，权重大于 0）。每当一个不同的混合形状（即变形网格）激活时，都要复制网格数据。我们不希望由此加重 CPU 的负担，因此使用 neutralPosition（标准网格顶点位置——无表情网格）、neutralNormal（标准网格顶点法线）、neutralBinormal（标准网格顶点副法线）、textureCoord（顶点贴图坐标），以及 50*（positionDiff+NormalDiff）——标准网格顶点与 50 个变形网格顶点间的位置向量差（positionDiff）与法线向量差（NormalDiff），作为顶点通道，来创建网格。在每帧的基础上，我们可仅仅改变顶点输入属性名，从而有效地输入应该激活变形网格对象的数据，从而忽略那些未激活的。对于每一帧，我们找到 5 个激活变形网格对应的顶点坐标差和法线差，并把它们映射到顶点 shader 中去。

请注意，positionDiff 和 normalDiff 的 w 分量没有存储任何真正有用的插值。我们可以利用这一点，把一个自遮挡项的量存储在 neutralNormal 的 w 分量中，而把遮挡项的差值存储在

每个变形网格的法线中。当提取法线时，我们仅把*.xyz* 用于寄存器。这样可以计算一个动态的遮挡项，其变化只基于 Dawn 的眼睛和嘴是开还是闭，无需额外指令。这一点提供了皮肤照明中的柔和阴影（详细描述请见第 3 节“Dawn 的皮肤”）。

在内容创建方面，最多保持 5 个激活的混合形状，这对美工而言没有任何困难，因为美工主要就是同时在大约 3 个表情之间调整动画，然后添加表情元素的调节器来满足复杂性。我们把头部网格与身体网格分开，以避免做额外的数学运算或存储 0 差值，也就是说，避免出现“快乐的面孔加到 Dawn 的肘上”的情况。因为任何表情变形网格的顶点移动并不触及标准面部网格最靠近脖子的顶点，所以在躯干与头部的不同变形运动下，整体网格还保持无缝。躯干由骨骼蒙皮动画驱动。而面部由变形网格对象驱动。尽管如此，面部网格依然也同时由骨骼蒙皮动画驱动。面部顶点的权重，与身体网格中相匹配顶点的权重相同。这保证了头部与躯干无缝连接的动画。

4.4 蒙皮

蒙皮（Skinning）是一种网格变形方法，对网格中每个顶点指定一组带有权重的矩阵（权重最大可增加到 1.0），权重指明矩阵应该如何约束顶点。举例来说，在二头肌上的顶点可能只受肩关节的制约；但是肘上的一个顶点，可能就是 50%连接到肩关节，50%连接到肘关节；而过了肘弯的顶点变得 100%受肘关节支配了。

为一个网格蒙皮做准备，通常要为网格创建一个中间状态，叫做绑定姿势（bind pose）。这个姿势保持胳膊和腿略微分开，并且尽可能避免弯曲，如图 4-3 所示。首先，我们创建一

图 4-3 Dawn 的绑定姿势

个与这个网格相匹配的转换层次（hierarchy，实际就是指建立骨骼系统，组成骨架），然后通常借助于动画工具的帮助，指定顶点连接的骨骼，以及连接骨骼权重。经常总是要修改定点蒙皮结果以解决围绕肩、肘、臀部等处的问题。然后通过多种技术使这个骨骼动画起来。我们使用了关键帧动画、反向运动学，以及动作捕捉技术，这是由工具集所支持的。

一个蒙皮过的顶点的值，是此顶点连接关节矩阵转换后值的加权和：

$$v' = \sum_{i \in bones} weight(v)[i] \times matrix[i] \times matrix[i]^{-1}_{\text{bindpose}} \times v_{\text{bindpose}}$$

就概念而言，这个方程将顶点从其标准位置取出，放入加权的模型空间，又将每个矩阵转换回世界空间，最后将结果相加。把连接矩阵 $matrix[i] \times matrix[i]^{-1}_{\text{bindpose}}$ 存储为常数参数，把矩阵的索引和权重作为顶点的属性输入。一顶点连接 4 块骨头的蒙皮应用程序如下。

```
float4 skin(float4x4 bones[98],
            float4   boneWeights0,
            float4   boneIndices0)
{
  float4 result = boneWeights0.x * mul(bones[boneIndices.x], position);
  result = result + boneWeights0.y * mul(bones[boneIndices.y], position);
  result = result + boneWeights0.z * mul(bones[boneIndices.z], position);
  result = result + boneWeights0.w * mul(bones[boneIndices.w], position);
  return result;
}
```

在 Dawn 演示中，我们驱动一个超过 180000 个三角形并具有 98 块骨头的网格。我们发现，为了驱动身体和头，每个顶点用 4 个矩阵太多。必须每个顶点存储 4 个骨骼索引和 4 个骨骼权重，作为顶点的输入属性（16 个 *xyzw* 顶点寄存器的最后两个，在第 4.3.2 节中提到）。我们将骨权重和骨索引排了序，这样如果需要更高的顶点计算性能，就可以重写顶点 shader，从而人为地减少在顶点上起作用的骨骼数。注意，如果这样做，也必须重新调节顶点连接骨骼的权重，以便其总和仍是 1。

蒙皮时，为了光照的正确，必须求矩阵的逆，并求经逆矩阵转换的坐标、法线、副法线和切线。如果层次关节的缩放不能够保证在 x、y 和 z 轴的方向都一样，那就必须求矩阵逆的转置。如果是 *xyz* 同轴缩放（均匀缩放），那么矩阵的逆就是它的转置，因此矩阵保持不变。非均匀缩放会产生多方面的问题，因此，我们的引擎不允许这种情况。

如果使用前面的代码计算，则必须为每个顶点属性的每个矩阵调用 mul。在目前的硬件中，实现点与矩阵相乘执行 4 个点积和 3 个加法，矢量相乘是 3 个点积和 2 个加法。因此，4 块骨骼蒙皮的坐标、法线、副法线和切线运算需要：

4×（4 个点积+3 个加法+3×（3 个点积+2 个加法））=52 个点积+36 个加法=88 个指令

一种创建加权矩阵和的技术在 HLSL 或 Cg 中可实现如下。

```
float4x4 accumulate_skin(float4x4 bones[98],
                         float4 boneWeights0,
                         float4 boneIndices0)
{
  float4x4 result = boneWeights0.x * bones[boneIndices0.x];
```

```
    result = result + boneWeights0.y * bones[boneIndices0.y];
    result = result + boneWeights0.z * bones[boneIndices0.z];
    result = result + boneWeights0.w * bones[boneIndices0.w];
    return result;
}
```

虽然这个技术耗费指令来建立累积矩阵（16 个乘法和 12 个加法），但现在只需一个矩阵乘法来蒙皮一个点。先前一样的蒙皮，现在使用指令数量如下：

16 个乘法 + 12 个加法 （建立累积矩阵）
+ 4 个点积 + 3 个加法 （用于坐标）
+ 3 × （3 个点积 + 2 个加法）
13 个点积 + 16 个乘法 + 21 个加法 = 50 个指令

4.5 小结

把网格动画计算从 CPU 上移除，以及利用现代图形硬件提供的可编程顶点管道总是有好处的。尽管已看到了利用 shader 实现蒙皮和变形网格对象，使用流（Streaming）SIMD 扩展（SSE）指令及其他类似的指令，则很容易实现内部循环。因此，在一些情况下，同样的技术保留在 CPU 上，效果也很好。

在 Dawn 演示中，变形网格对象只是用来驱动头部的表情。假如有更多的时间，我们就会在躯干上使用变形网格对象来解决简单蒙皮的问题。甚至一个蒙皮良好的网格也有这种问题：当肘、膝及其他的关节转动时，它们的体积会变小。这是因为当网格弯曲时，关节并没有变“胖”一点，去补偿肌肉对肌肉的压缩。在蒙皮之前或之后，使用一个变形网格对象或其他的网格变形技术，就可以提供这种柔软、肉感的形变，从而创建更为逼真的效果。我们已经使用数码内容制作工具，重新制作了各种网格变形，并期待在将来应用它们。

4.6 参考文献

Alias Systems. Maya 5.0 Devkit. *<installation_directcry>/devkit/animEngine/*

Alias Systems. Maya 5.0 Documentation.

Eberly, David H. 2001. *3D Game Engine Design*, pp. 356～358. Academic Press.

Gritz, Larry, Tony Apodaca, Matt Pharr, Dan Goldman, Hayden Landis, Guido Quaroni, and Rob Bredow. 2002. “RenderMan in Production.” Course 16, SIGGRAPH 2002.

Hagland, Torgeir. 2000. “A Fast and Simple Skinning Technique.” In *Game Programming Gems*, edited by Mark DeLoura. Charles River Media.

第 5 章　改良的 Perlin 噪声的实现

Ken Perlin
New York University

这章的重点是对噪声函数做一个新的改进设计。重新设计要达到 3 个目的：（1）使噪声变得适宜逐步地移植进硬件；（2）在某些重要的方面改进噪声函数的视觉特性；（3）引入一个单个的、标准的版本，让它在所有硬件和软件平台都能够返回相同的值。

这章的构成如下：首先，描述噪声函数本身的目的，并且简短地回顾函数最初的实现；然后讨论为了增加噪声的质量和减少视觉上的失真，我在实现噪声改进中所做的特别选择；最后，是有关逐步移植到硬件中所需要的步骤。一代代 GPU 不断改进从而具有了更强大的指令集，我们的目标也是可以逐渐地使用更少的指令实现噪声函数。

5.1　噪声函数

噪声函数的目的，是在三维空间中，提供一种可以有效率地实现、可重复、伪随机的信号。此信号的能带有限（band-limited，大部分能量集中在一个空间频率附近），而视觉上各向同性（isotropic，统计上不随旋转变化）。

一般的想法是创建某种信号，它类似于一个完全的随机信号（即白噪声），通过低通滤波后，滤除了所有的空间高频率而变得模糊。有如在沙丘里缓慢上升的山包和下落的低谷，如图 5-1 所示。

图 5-1　沙丘

5.2 最初的实现

最早的噪声是 1983 年实现、1985（Perlin 1985）年发表的，使用以下的运算法则定义任意一点（x, y, z）的噪声函数（Noise）。

1．将空间中的每一点（i, j, k）（其坐标是整数）赋值为 0，并赋予一个可以散列得到的伪随机斜率。

2．定义（x, y, z）的坐标为一个整数加上小数部分，$(x, y, z) = (i + u, j + v, k + w)$。围绕这个点的单位立方体的 8 个顶点为，$(i, j, k)$, $(i+1, j, k)$，…… $(i+1, j+1, k+1)$。

3．使一个 Hermite 样条满足穿过这 8 个点。使用 u、v 和 w 作为插值参数，计算样条函数在（x, y, z）的数值。如果预先定义 Hermite 3 次混合函数为 $3t^2 - 2t^3$，制作一个查询表，那么这个运算只需要 7 个线性插值：比如，4 个在 x 方向上，接着 2 个在 y 轴方向，然后 1 个在 z 向上，如图 5-2 所示。

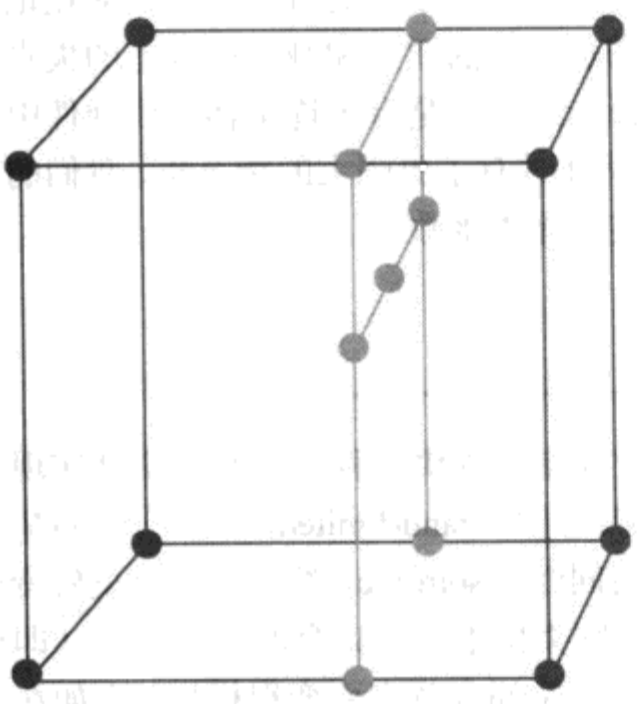


图 5-2 在规则三维格子中从 8 个采样点间内插一个值

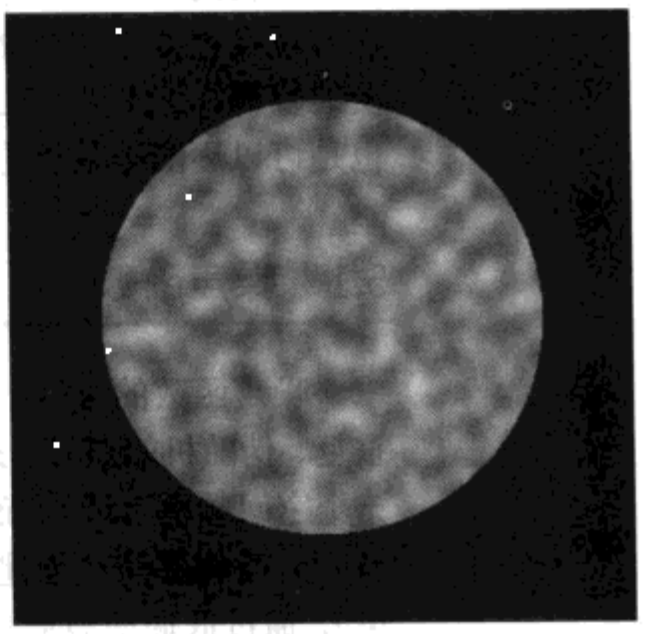


图 5-3 通过噪声函数产生的一个切片图样

这个方法效果很好。通过噪声函数产生的一个二维切片，如图 5-3 所示。

5.3 最初实现的缺点

噪声函数尚存在以下的不足。

- 所选择的插值函数其每维都是 $3t^2 - 2t^2$，它的 2 次导数含有非 0 值。这样在使用噪声的导数时，将导致出现失真的视觉效果，例如制作凹凸贴图时。
- 从（i, j, k）得到斜率，要在一组 256 个伪随机斜率矢量中查找，这组矢量散布在一个 3D 立体球面上；这种分布上的不规则，在噪声函数的结果中产生了不希望的高频。见图 5-4。

回顾一下可以知道，在选择伪随机斜率方向的预处理阶段，应该采用一个不那么严格的

运算法则，从球面剔除一些点，以形成一种更平坦的泊松（Poisson）分布，这样可以减少不希望的空间高频。图 5-5 给出了视觉结果。

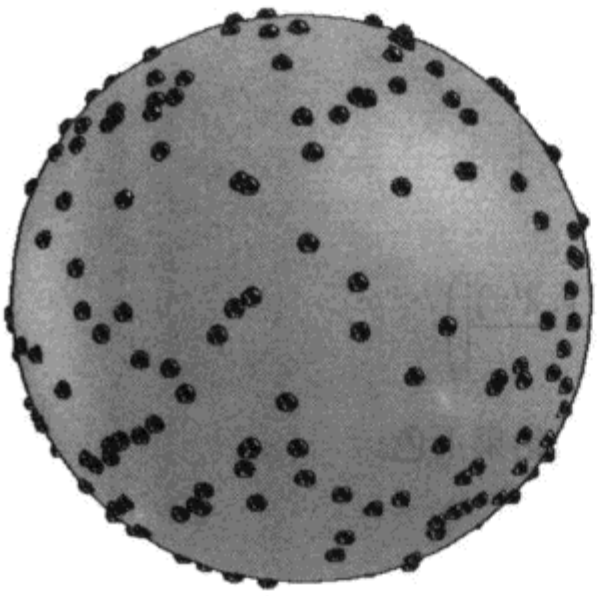


图 5-4　斜率矢量的不规则分布

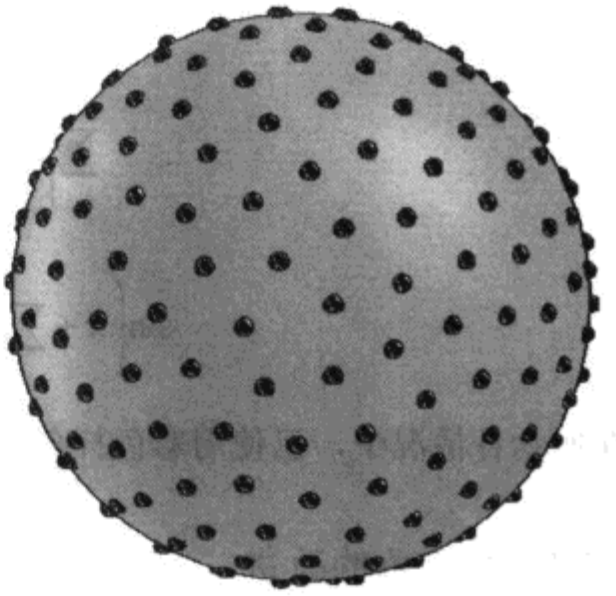


图 5-5　斜率矢量的规则分布

近来，我意识到，这样大量的不同斜率方向在视觉上是不必要的。更重要的是一个斜率方向的均匀统计分布，而不是许多不同的斜率方向。这一点与知觉的研究结果相一致，Grill Spector 等人指出（Grill Spector et al. 1995）：虽然人类视觉对于纹理方向统计上的不规则是高度敏感的，但是对于方向上的间隔尺寸是相对不敏感的，这是因为我们的低级视觉系统，会把一组充分密集的、不连续的方向，转换成连续信号。人类的视觉系统会在极早的阶段就执行这个转换，即下意识地执行。

用统计上平坦的斜率方向分布来实现一个噪声，可以非常直接地建立程序表达式，从而创建看上去复杂的纹理。例如，图 5-6 显示了由噪声函数构造的 4 个纹理，它们一个比一个更复杂。

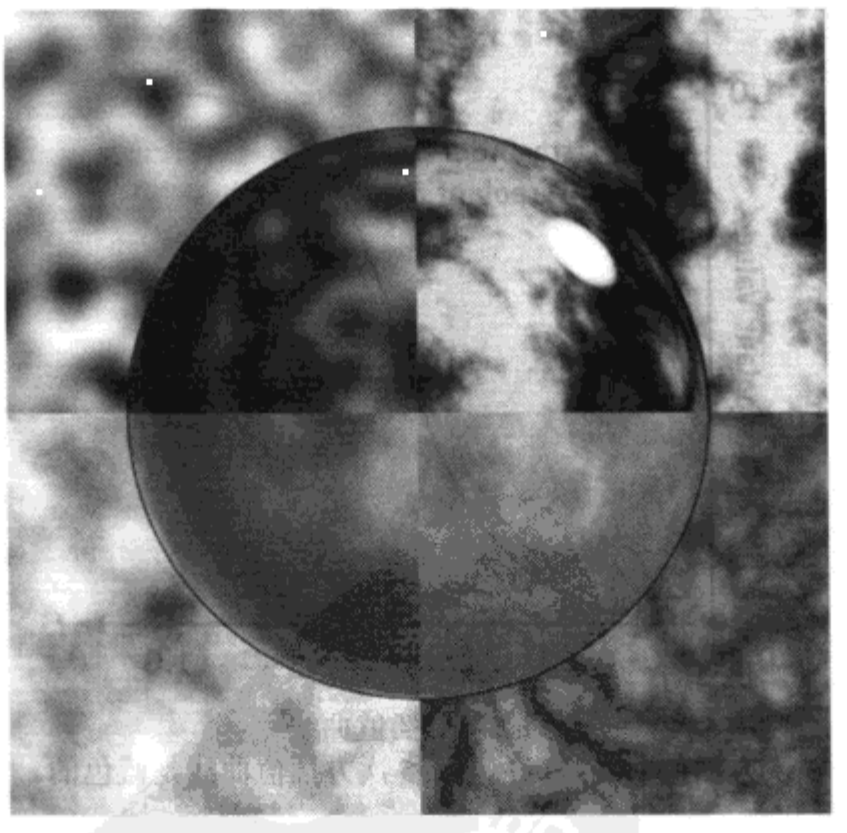


图 5-6　由噪声构造的 4 个纹理表达式

从左上部开始，顺时针转动，它们的表达式是：

$$noise\ (x, y, z)$$

$$\sum \frac{noise(2^i x, 2^i y, 2^i z)}{2^i}$$

$$\sum \left| \frac{noise(2^i x, 2^i y, 2^i z)}{2^i} \right|$$

$$\sin\left(x + \sum \left| \frac{noise(2^i x, 2^i y, 2^i z)}{2^i} \right| \right)$$

在所示各情况中，已使用彩色样条做后期处理，为结果上色。

5.4 对噪声函数的改进

对噪声函数的改进从两个方面进行：插值的特性以及伪随机斜率场的特性。

正如在 Perlin 2002 讨论的，改进插值的关键是消除 2 阶导数的不连续性。最初选择 3 次样条 $3t^2 - 2t^3$ 进行插值，是因为当 $t = 0$ 和 $t = 1$ 时导数为 0。可惜它的 2 阶导数 $6 - 12t$ 在 $t = 0$ 或 $t = 1$ 时不是 0。这会使噪声函数用于凹凸贴图时导致失真。因为凹凸贴图表面的光照效果随相应的高度函数的导数而变，所以 2 阶导数的不连续性是可以看见的。

改为用 5 次样条 $6t^5 - 15t^4 + 10t^3$ 插值，可以解决这个问题：它在 $t = 0$ 和 $t = 1$ 处的 1 阶和 2 阶导数均为 0。

从图 5-7 中可以看到这两个插值间的差别。绿色的是旧的 3 次方插值曲线，2 阶导数在 $t = 0$ 和 $t = 1$ 处不连续。蓝色的是新的 5 次方插值曲线，在 $t = 0$ 和 $t = 1$ 处，2 阶导数没有间断性。

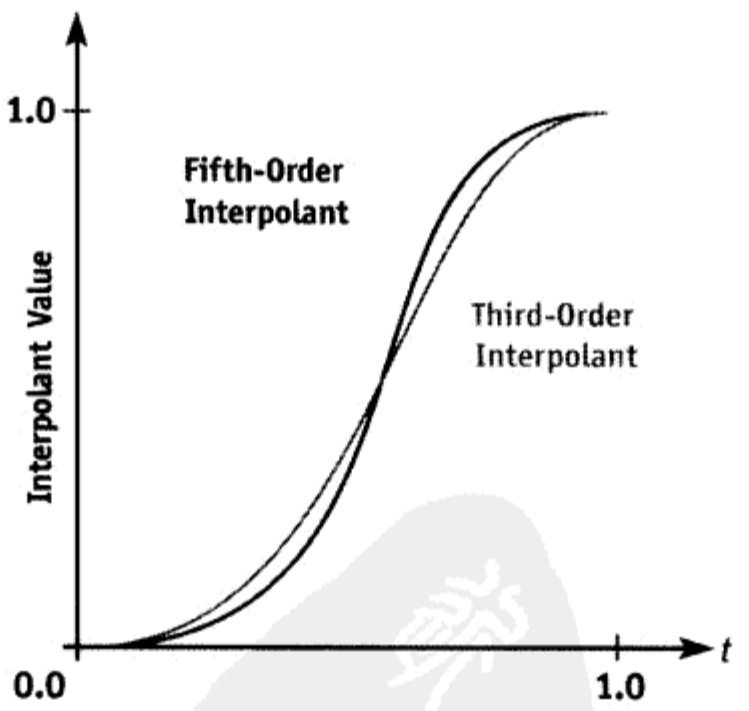
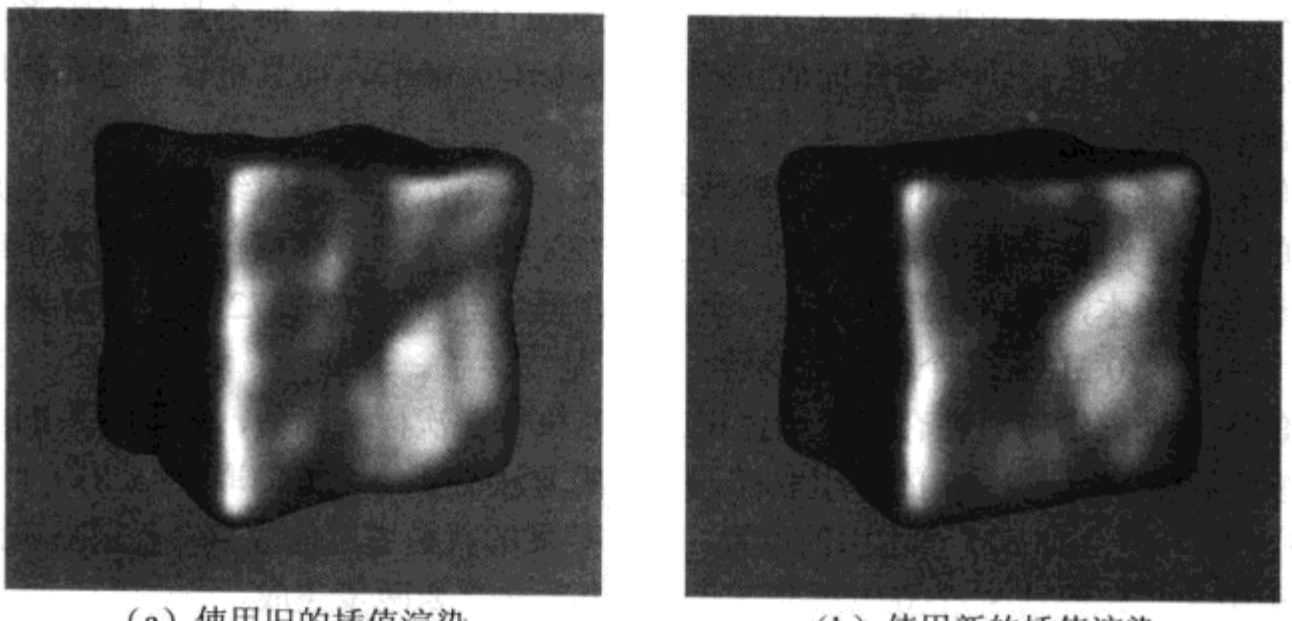


图 5-7 插值曲线
3 次方插值曲线是绿色的，5 次方插值曲线是蓝色的。

使用图形加速器的一个好处是，可以通过一维纹理查找表来执行一维插值。用这个方法

进行高阶插值时，没有额外的开销。在实践中可以发现纹理查找表的大小为 256 已经很好，精确度为 16bits 足矣。甚至尾数为 10bits 的半精度浮点（fp16）就足够了。

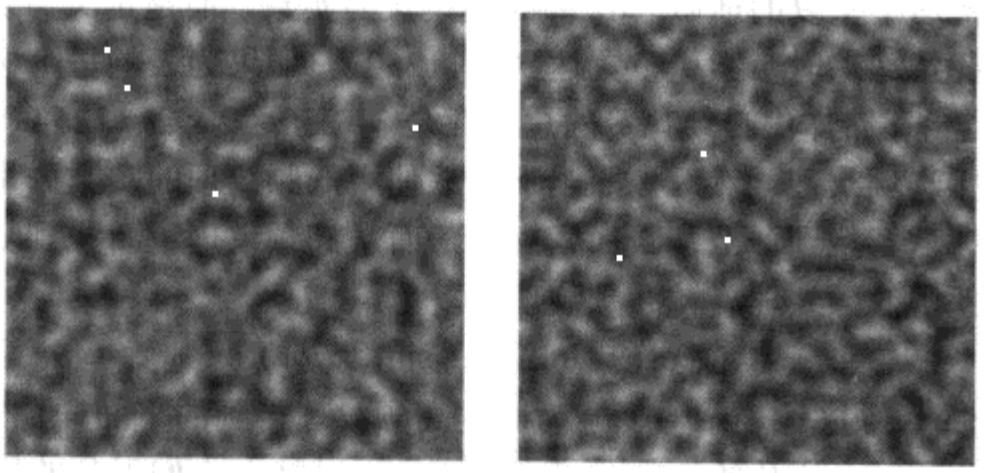
图 5-8 显示了两个不同渲染噪声置换（Noise-replaced）的超二次曲面，两者使用了不同的噪声插值。图 5-8（a）使用旧的 3 次方插值；图 5-8（b）使用新的 5 次方插值。可以发现在形体正面减少了类似 4 × 4 方格的不自然痕迹，在视觉效果上有了改善。



（a）使用旧的插值渲染　　（b）使用新的插值渲染

图 5-8　噪声置换的超二次曲面

另一个改进是用 12 个伪随机斜率代替 256 个，它们由立方体 12 条棱的中点组成：（0, ±1, ±1）、（±1, 0, ±1）和（±1, ±1, 0）。这造成一个看上去更少“污点”的结果，在图 5-9 中可见到两种平面切片的比较，它们分别对应老方法与新方法实现噪声函数的结果。



（a）旧的噪声　　（b）新的污点较少的噪声

图 5-9　斜率分布的改进

改善的原因是 12 个斜率的方向没有彼此离得太近的，因此它们不会挤在一起。相邻的斜率挤在一起，正是引起最初的噪声函数实现显现出污点的原因。

这个方法的另一个好处是，它能够避免在计算斜率函数时作许多乘法，因为（x, y, z）与其他值（比如（1, 1, 0））的点积可以按 $x + y$ 作加法。

为了使硬件尽可能快地逐点求这组斜率，实际上定义了 16 个斜率，所以求斜率的散列函数可以简单地返回一个任意的 4bit 值，额外的 4 个斜率只须简单地重复（1, 1, 0）、（−1, 1, 0）、（0, −1, 1）和（0, −1, −1）即可。

5.5 如何在像素 shader 中产生好的假噪声

如果可以把一些适当的体积噪声图元完全放入像素 shader 中是非常好的，但是直接这样实现却需要相当多的指令。那么怎样利用今天像素 shader 的功能，以非常少的指令去实现对噪声的某些合理逼近？这里假定使用分段 3 线性插值（trilinear），而不是更高阶多项式插值（因为今天的 GPU 直接提供 3 线性插值（trilinear interpolation），而且可以容忍在使用 3 次样条插值时出现的少量斜率间断。

一个解决方案，是把一个噪声的完整体积放入一个 3 次样条插值纹理之内。但是，这会动用太多的样本。例如，从（0, 0, 0）到（72, 72, 72）的一个立方体取样噪声，每个线性的单位距离含有 3 个样本，对终点还要再加一个，即在每个方向有 $(3 \times 72) + 1$ 个样本，会需要 $217 \times 217 \times 217 = 10\,218\,313$ 个索引的位置，实在是太多了。

今天的图形处理器可以在合理的小体积上快速执行 3 次样条插值，可以用不同的方式利用这个事实：例如对 $8 \times 8 \times 8$ 的体积取样，目前的方法需要每个方向取 $(3 \times 8) + 1$ 个样本，或只要 $25 \times 25 \times 25 = 15\,625$ 个索引的位置。现在要做的就是通过倍数参考量把噪声定义到这种中等体积中去。如何从 8^3 个立方得到 72^3 个立方？这需要两步完成。

1. 将空间填满 8^3 的样本的拷贝，每个样本为一个地砖（tiling）。换言之，构造这样一个样本，它形成一个不断重复的连续图案，从上到下、从左到右、从前到后都实现无缝连接。当这些样本在空间中作为立方形砖堆砌的时候，它们将产生一个连续的、重复的空间填充纹理。

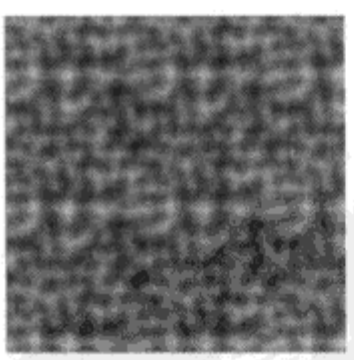
2. 使用这个同样的取样图案，放大 9 倍，做一个低频图案，这样就改变了小尺寸的地砖，使得小尺寸的图案不再重复出现。

细节如下：不把这个重复的地砖（tile）的值定义为实数，而是定义为复数。注意这将使存储查询表的成本翻番（这时，索引位置达到 31 250）。如果为表的每个标量数字使用一个 16-bit 的量，那么为了存储体积必须花费的总内存是 $31\,250 \times 2 \times 2 = 125\,000$bytes（字节）。

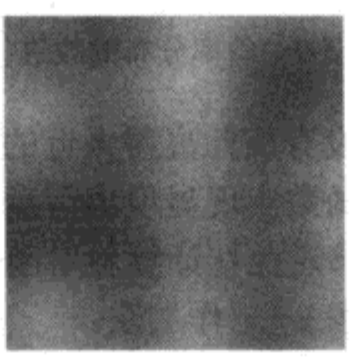
计算低频和高频噪声纹理。然后按照以下方法使用低频纹理的实数分量作为一个旋转相移：我们从低频纹理重新找到一个复数（lo_r, lo_i），而从高频纹理重新找到复数（hi_r, hi_i）。通过计算下式，我们使用低频纹理旋转高频纹理的复数：

$$hi_r \cos(2\pi\, lo_r) + hi_i \sin(2\pi\, lo_r)$$

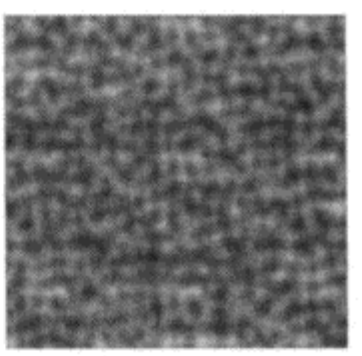
图 5-10 的 3 个图像顺序地表示，（a）为通过高频纹理得到的平面样片的实数分量，（b）为通过低频纹理得到的平面样片的实数分量，（c）为通过结合它们得到的最终的噪声纹理。



（a）通过高频纹理的平面样片的实数分量



（b）通过低频纹理的平面样片的实数分量



（c）通过结合它们得到的最终的噪声纹理

图 5-10 高频纹理与低频纹理的结合

注意在最左边图像上可见的地砖图案，在最右面的图像上不再是可见的。

5.6　不考虑相邻顶点制作凹凸贴图

一旦能在像素 shader 中直接地实现噪声，就能按如下过程实现凹凸贴图。

1．考虑一个点（x, y, z）和一些函数 F（x, y, z），它可以通过一些噪声函数的组合在像素 shader 中定义。

2．选择小的 ε 值，计算下面的量：

$$\boldsymbol{F}_0 = \boldsymbol{F}(x, y, z)$$
$$\boldsymbol{F}_x = \boldsymbol{F}(x + \varepsilon, y, z)$$
$$\boldsymbol{F}_y = \boldsymbol{F}(x, y + \varepsilon, z)$$
$$\boldsymbol{F}_z = \boldsymbol{F}(x, y, z + \varepsilon)$$

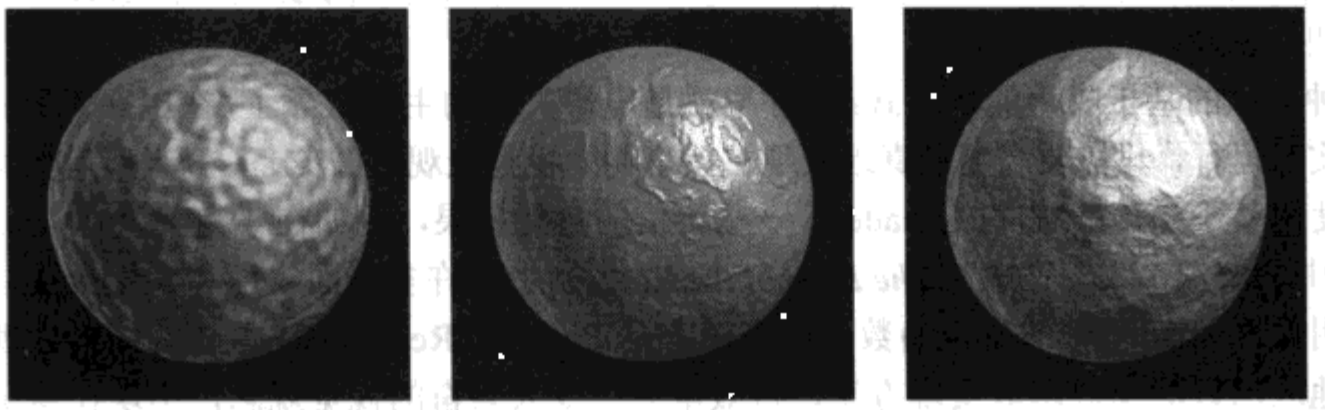
3．可以通过下式，得到 $\boldsymbol{F}$ 在（x, y, z）的斜率（导数）矢量的很好近似：

$$\mathrm{d}\mathbf{F} = \left[\frac{F_x - F_0}{\varepsilon}, \frac{F_y - F_0}{\varepsilon}, \frac{F_z - F_0}{\varepsilon}\right]$$

4．从表面法线减去这个斜率，然后把结果再归一化回单位长度：

$$\boldsymbol{N} = normalize(\boldsymbol{N} - \mathrm{d}\boldsymbol{F})$$

图 5-11 显示的结果，是将这个方法准确地应用到球体上的 3 个不同的程序式凹凸映射：一个简单的多瘤图案、一个饰以浮雕花纹的大理石图案、一个碎片的皱纹图案。



（a）多瘤的图案　　（b）饰以浮雕花纹的大理石　　（c）绉纹图案

图 5-11　球面的凹凸贴图

假设模型是一个单位半径的球面，实现这些凹凸图案的表达式是：

```
.03 * noise(x, y, z, 8);                                    //多瘤的图案
.01 * stripes(x + 2 * turbulence(x, y, z, 1), 1.6);         //大理石
-.10 * turbulence(x, y, z, 1);                              //绉纹
```

在下列的函数上定义：

```
// STRIPES TEXTURE (GOOD FOR MAKING MARBLE)
double stripes(double x, double f) {
  double t = .5 + .5 * sin(f * 2*PI * x);
  return t * t - .5;
}
```

```
// TURBULENCE TEXTURE
double turbulence(double x, double y, double z, double f) {
  double t = -.5;
  for ( ; f <= W/12 ; f *= 2) // W = Image width in pixels
    t += abs(noise(x,y,z,f) / f);
  return t;
}
```

5.7 小结

如果不创建显式的纹理图像，如何才能得到看起来自然的材质呢？这时用体噪声制造程序式纹理就派上用场了。也就是说，不用冥思苦想通过参数将纹理图像映射到形状表面；而基于噪声的纹理的体积性质，就可以让你用像素 shader 计算它们在（x, y, z）的位置。这样，可以有效地在实体材料外面刻画纹理，这常常比做合理的未变形的参数映射更为简单。

基于噪声的纹理，也允许以独立于分辨率的方式工作：即当接近一个物体时，纹理不会变得模糊，而是可以保持清晰的细节，好像把噪声的高频加到定义纹理的程序上一样（texture-defining procedure）。

对程序式纹理永远要注意：所施加的高频不能超过像素的取样率。这种超过像素的频率不但对增加视觉效果无益，还会在纹理动画时导致不希望的斑点出现。

要想有效地在图形处理器上使用基于噪声的纹理，关键在于要真正地利用图形处理器的强大计算功能。这里已经概述了实现方法，而且用一些例子说明了如何以像素 shader 程序式地定义凹凸贴图。

这种体积的基于噪声的程序式纹理，早就是电影大片的主要依靠了，在电影大片中 shader 不需要实时运行，只需要高度逼真地表现自然事物的视觉表观。事实上，今天所有的特效电影都大量使用基于噪声的程序式 shader。一些可信的自然效果，例如在“完美风暴”（*A Perfect Storm*）中的海浪，“指环王”（*The Lord of the Rings*）中的许多表面纹理和大气效果，就依靠大量使用像素 shader 中的噪声函数，编程语言是 Pixar 的 RenderMan。随着图形处理器上可以实时地应用好的噪声函数实现方法，特效电影中令人兴奋的视觉观念，出现在新一代电脑和控制台的互动式娱乐中已是指日可待了。

5.8 参考文献

Grill Spector, K., S. Edelman, and R. Malach. 1995. “Anatomical Origin and Computational Role of Diversity in the Response Properties of Cortical Neurons.” In *Advances in Neural Information Processing Systems* 7, edited by G. Tesauro, D. S. Touretzky, and T. K. Leen. MIT Press.

Perlin, K. 1985. “An Image Synthesizer.” *Computer Graphics* 19(3).

Perlin, K. 2002. “Improving Noise.” *Computer Graphics* 35(3).

第 6 章　Vulcan 演示中的火

Nubert Nguyen
NVIDIA

这章谈谈 Vulcan 演示，它是 NVIDIA 演示小组为 GeForce FX 5900 Ultra 投放市场而创建。受电影“指环王”中的角色 Balrog 的启发，我们的目标是显示一个妖魔，它会发射熊熊大火，如图 6-1 所示。火焰和烟雾必须是极大的，有时是不可遏制的。这章的目的是共享我们创建的效果的经验，并讨论进一步提高的可能。最后，我们希望这项探索能帮助读者在自己的应用中创建更成熟的火效应。

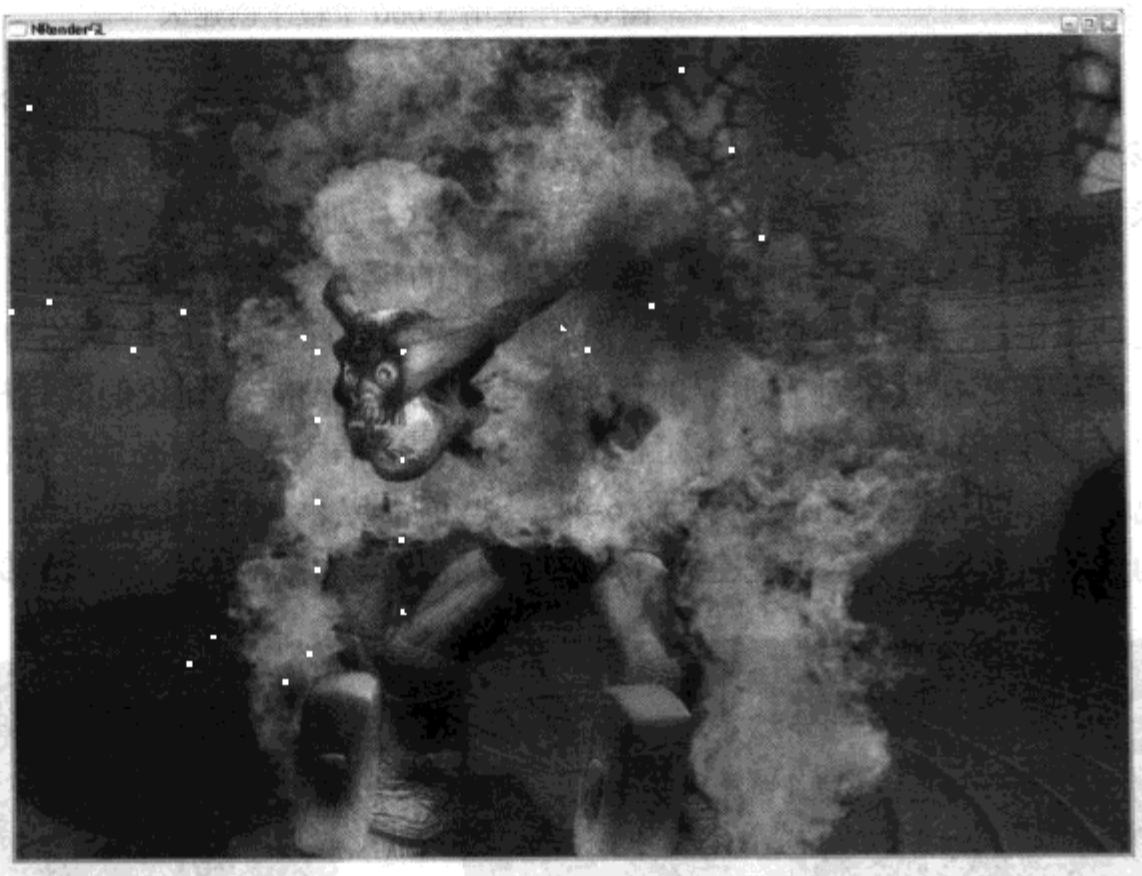


图 6-1　Vulcan 演示中的屏幕截图

6.1　创建逼真的火焰

当我们开始演示工作的时候，首先尝试了两个看起来有希望的方案：完全程序化的火焰和屏幕空间基于变形的二维火焰。

完全程序化的方法只消耗很少的内存，却创造了吸引人的火的效果。为了产生明确的火焰，我们必须显示数以千计的粒子，而处理所有的顶点和像素，对 CPU 和 GPU 是一个沉重的负担。图 6-2 显示了其效果。

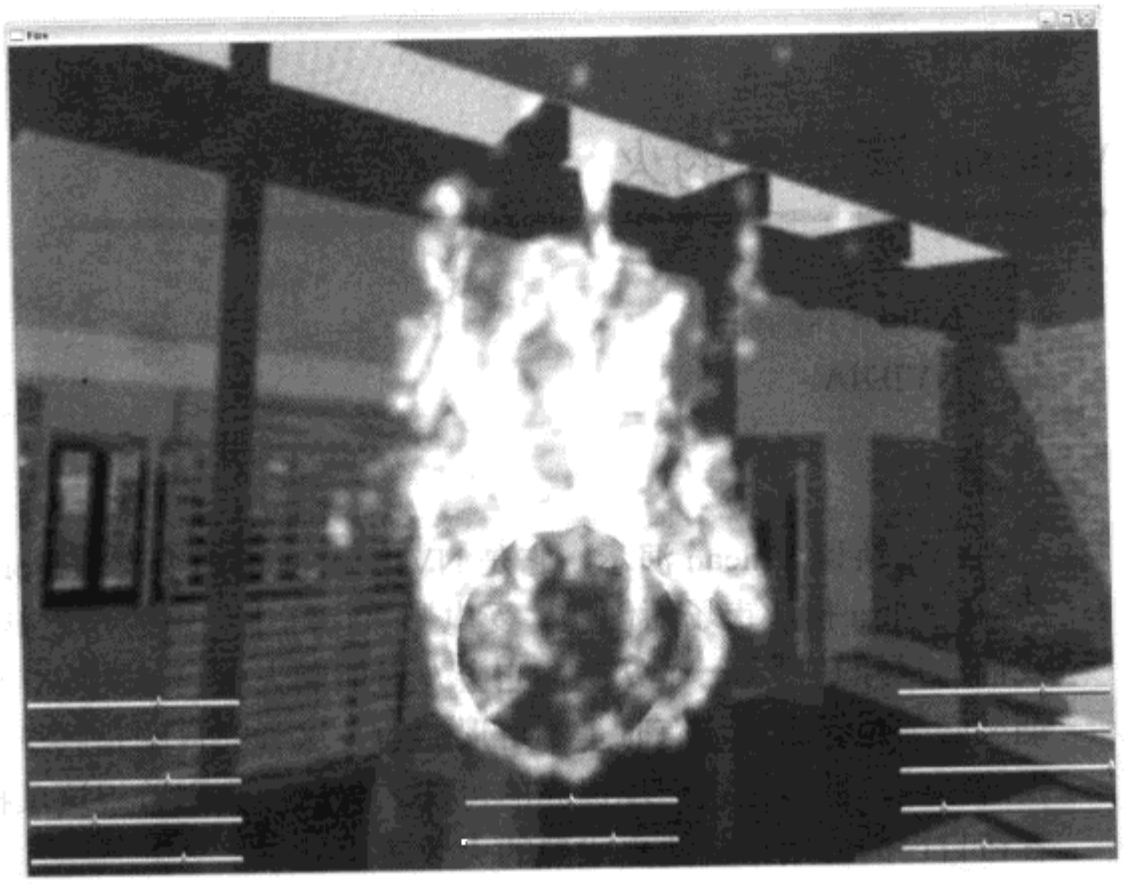


图 6-2 使用 5 000 个粒子创建火

基于变形的 2D 火焰使用 GPU 产生的扰动函数，它改变火焰的形状，使其能逼真地运动。变形涉及到制造几个渲染到纹理的遍和 2D 纹理坐标的转换。虽然它消耗的内存比粒子系统技术多（因为必须安排几个渲染目标），但是它对创建像烛光那样的火焰是很好的，见图 6-3。

效果的屏幕取齐的性质，使其对摄相机的视角（需要对视区的顶和底进行约束）和一般的运动很敏感（在 2D 中有时不能很好地移近和挪远摄相机）。烟的整合也是一个挑战。

两种程序化技术都有明显的优点，但是它们都没能达到我们的目标，我们期待在实时的、用户可控制的环境中创建一个可信的、有烟的熊熊大火。于是为了使火变得更逼真，我们转向视频纹理的精灵（sprite），见图 6-4。一些前沿电影（比如指环王），就使用了基于精灵的火焰构成的特技效果，至于完全程序化的、以物理学为基础的火焰恐怕要等待未来才能生成了。

图 6-3 使用 2D 流体创造的火焰

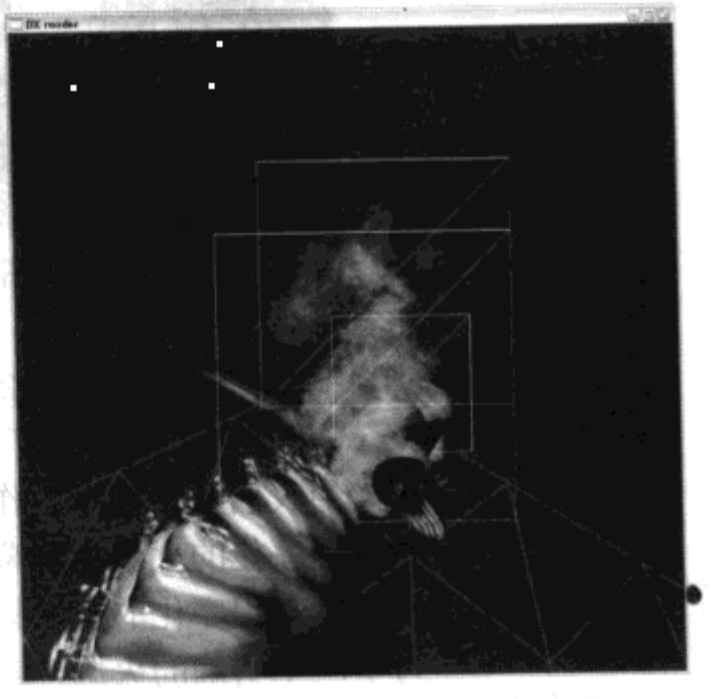


图 6-4 3 个纹理的视频火焰粒子

6.2　动画精灵的实现

为了要创建包围 Vulcan 角色的火，首先需要决定的是火从哪里来。使用建模工具，我们把发射源放在角色身上。每次发射都能调节尺寸、密度（即，每秒钟的粒子数）、纹理和粒子生存期等等。这种灵活性非常有用，因为它允许改变火的形状以适应我们的需要。然后我们可以散布一些粒子并描绘那个火焰，如图 6-5 所示。

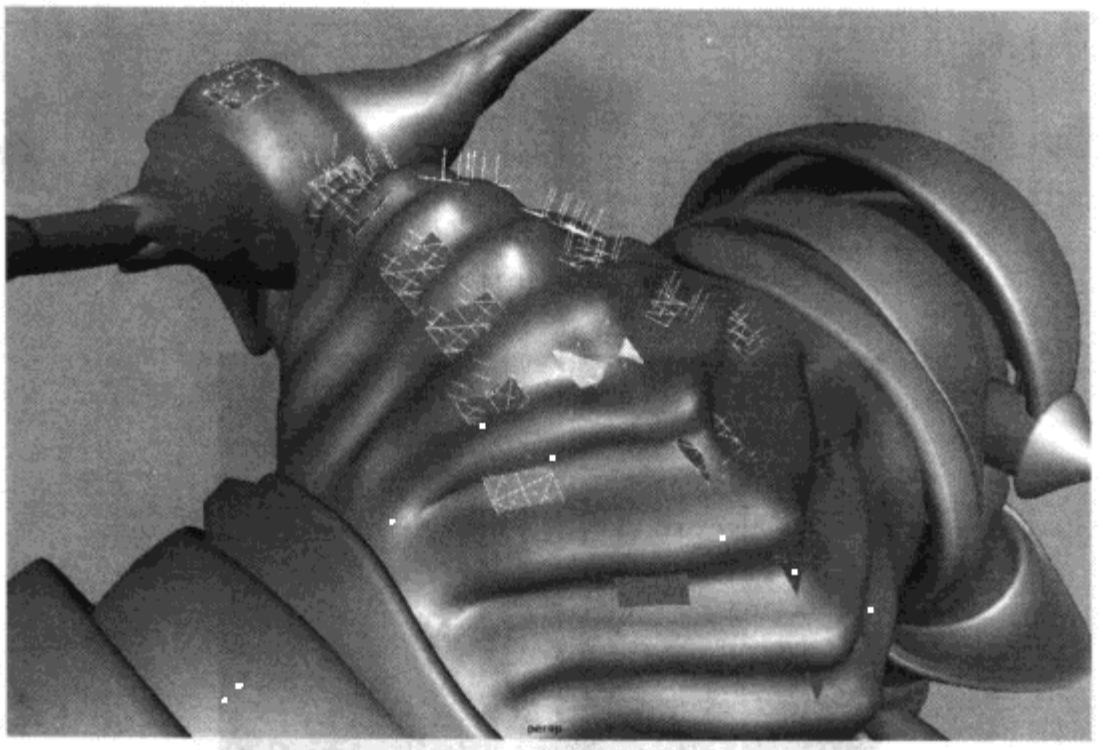


图 6-5　火焰的散布

火是从放在 Vulcan 身上的多边形发射的。火与组成 Vulcan 的多边形一起动画。

6.2.1　火焰和烟的动画

随着使用的视频纹理数目的增多，所消耗的纹理内存会迅速增加。因此我们不得不限制自己只用 3 个动画：2 个用于火焰，1 个用于烟。每个为 64 帧，192 帧动画表现火和烟，还有 64 帧作其他的用途，比如用于演示中的火花飞舞。一个 3D 立体纹理需要的层（这里为帧）数是 2 的幂，因此，我们用了 256 个帧。假如我们想要增加帧数，最少要变为 512 帧，是目前的 2 倍，见图 6-6。

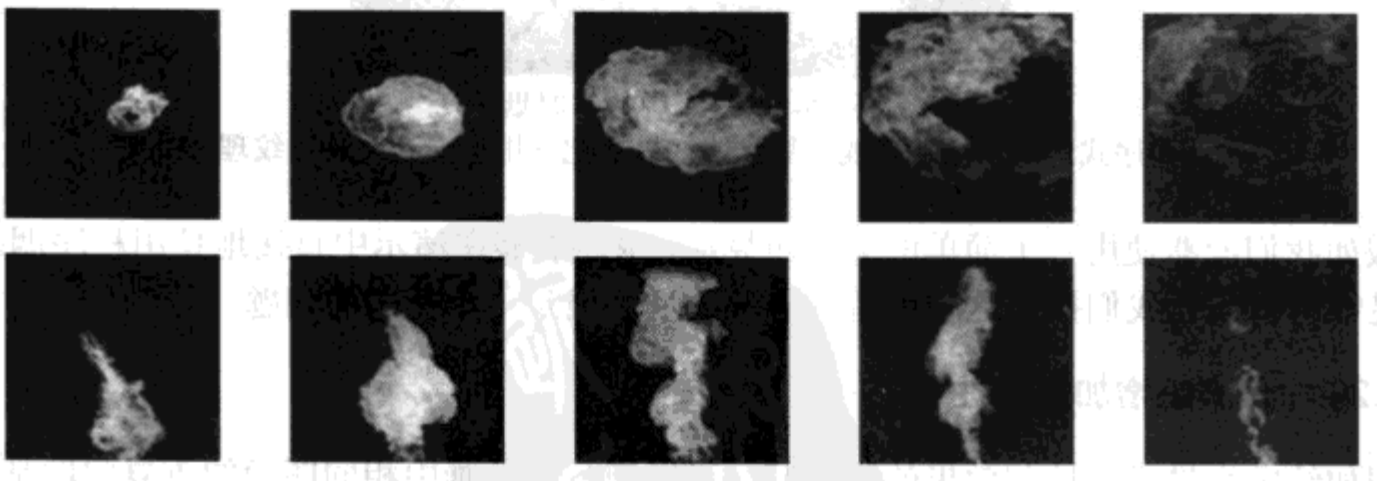


图 6-6　用于创建火效果的视频连续镜头

我们用了两个分开的视频片段（顶部和底部）给最终的效果加上变化。

定制的烟雾生成器

一开始我们想使用视频连续镜头创建烟，像创建火焰那样，但是很难找到合适的单独一股烟的镜头，同时还具备正确的照明、颜色等。

最后我们使用粒子系统创建了一个烟雾生成器，因为所需要的光照可以采用不同的技术达到，例如光线投射。调节光照明以适应火的环境，最重要的是保持对颜色的控制。给烟粒子一个在 x 轴上的旋转运动，这样烟本身似乎是盘旋的。这个细节没有什么，但是它增加了很多全面的烟效应。结果将真正的火的镜头和程序的烟很好地合在一起，见图 6-7 和图 6-8。

图 6-7 程序式地产生烟

图 6-8 演示中的实时烟
程序式地产生烟并离线计算光照，以后把它用作一个动画的纹理。

假如我们能够使用一个简单的烟光照算法，也就能够在演示中直接地使用程序烟。然而，没有足够的时间让我们探索这种可能性，解决烟光照是一个复杂的问题。

6.2.2 使火焰增加多样性

假如给定少量可以控制效果的动画帧，可以想到在动画中相同图像的重复问题是十分严重的。存在一些减少重复性和人为增加更多变化的简单办法。我们从单个的动画开始，它工

作得甚至比两个或多个动画更好。另外，我们创建的粒子其属性应该以任意比例变化，属性包括尺寸、位置、衰减率、动画开始帧、整体透明度，等等。

我们也决定不使用硬件点精灵，而是产生自己的方块。这个技术允许我们创建定制的纹理坐标，在 1 或 2 个轴上翻转动画。在演示中，我们使用了水平和垂直翻转（沿着 u 和 v 轴）。这工作得很好，但是任意旋转可以更好，见图 6-9。

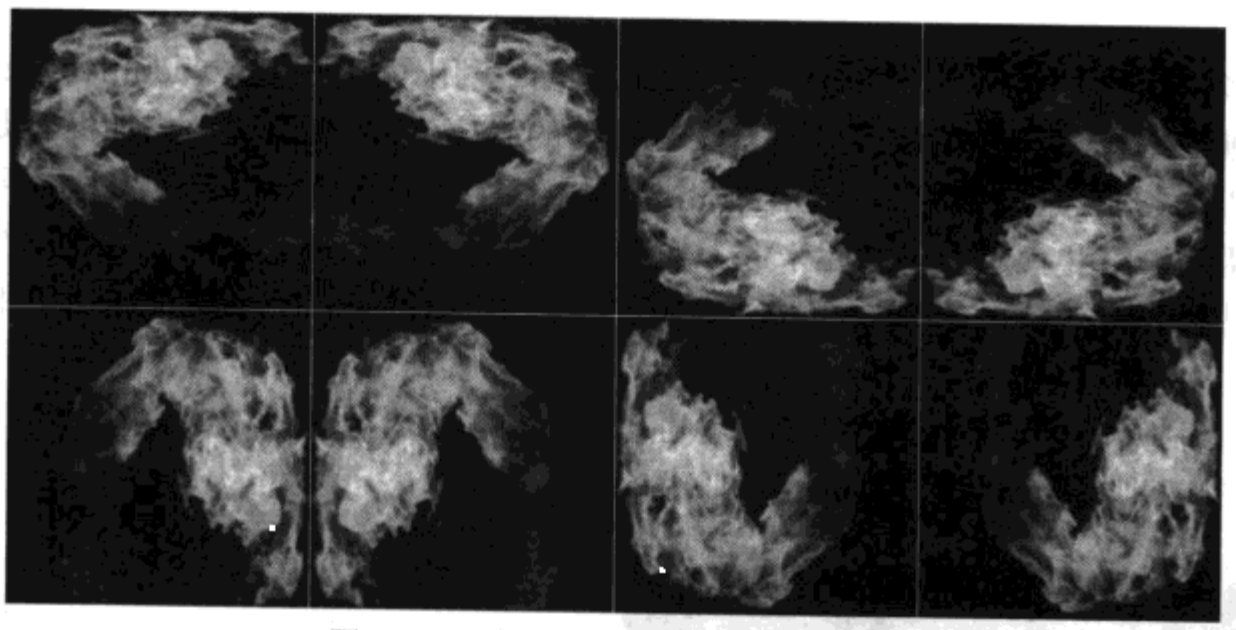
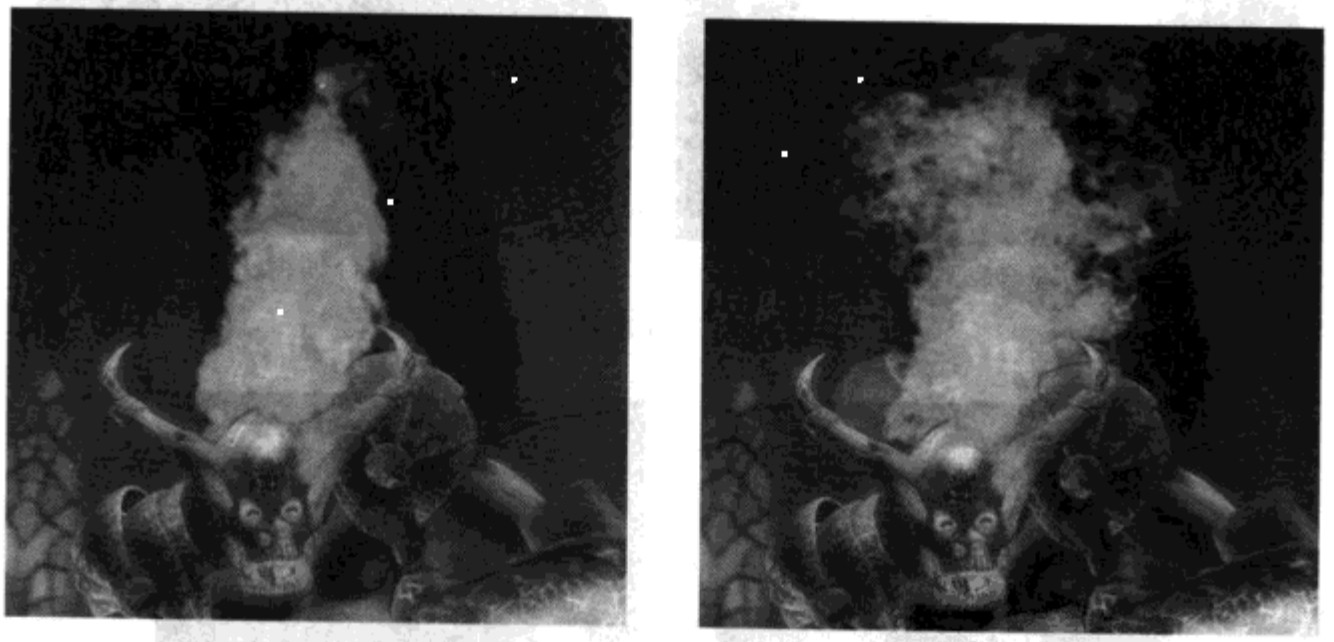


图 6-9　由定制的纹理坐标产生的变化
定制的纹理坐标能给单个动画帧提供更多的变化。任意旋转还会增加更多的变化。

因为有了第 2 个动画（取自图 6-6 中显示的视频图像片段），我们混合了两个类型的精灵：A 和 B。改变 A 型和 B 型不同的百分比，给火焰增加多样性，而且有助于控制效果的外观，见图 6-10。



（a）变化较小的火　　（b）变化较多的火

图 6-10　演示中改进的火

6.2.3　动画的存储

我们发现把火焰存储为一个体纹理是有帮助的（即一个 3D 纹理）。这样可以产生混合中间帧，以补偿动画帧数量较少的问题。因为增加了纹理坐标 z 来“播放”动画，处理起来也很方便。整个动画表现为对于应用的一个单个纹理，如果把粒子进行了分类，动画看起来很

不错。体纹理要求所有的边（宽度、高度和深度）是 2 的幂。在这样的情况下，使用压缩的 256 × 256 × 256 的 B8G8R8A8 纹理，共 16MB。如果可供支配的内存不多（可能正运行在一个低端的卡或控制台上），可以试着压缩体，让体尺寸变成 64 × 64 × 256，结果看起来仍然很不错。

6.2.4 火焰和烟的混合

像许多游戏开发者一样，在我们开始实施火焰时，用了相加混合（additive blending），因为它允许独立地画出所有的精灵（即，没有特别的次序）。在视觉上，相加混合是可以被看出来的，因为它会使颜色饱和，并且由此造成大部分的颜色不能由艺术家所控制，这是我们最不希望的。饱和使得火焰看起来像卡通，显得不真实。也就是说，相加混合使得火焰和烟的混合甚至变得更棘手，渲染达到有限的范围和一个 A8R8G8B8 目标的精确度时，就尤其严重。

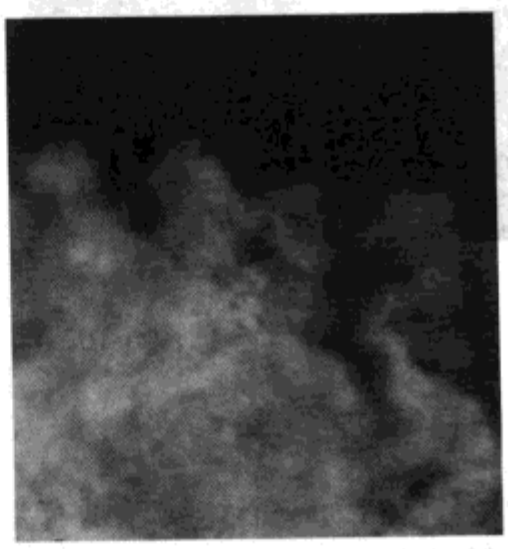
只要对每帧几百个粒子进行排序，就可以进行 alpha 混合来解决这两个问题。排序对操作性能的影响可以忽略不计，见图 6-11 和图 6-12。

（a）用附加的混合，很难结合火和烟

（b）aplha 混合使得火和烟排序

图 6-11 火和烟的混合

（a）附加的混合使颜色饱和



（b）aplha 混合对总的颜色控制

图 6-12 颜色的控制

6.3　粒子运动

我们期望粒子的运动仅仅是简单的向上运动。然而与预期不同，真实的粒子运动需要更多的注意。因为视频纹理包含了我们想要创建效果（即火）的大部分视觉线索，而我们只用了很少的区区几百个粒子。虽然只使用这么少的粒子在几何学方面是了不起的，但是当火源移动的时候，就遇到麻烦了。没有充足的粒子填充火焰移动后的间隙，并且定在空中的粒子会破坏效果。

我们想要的是一个连续流动的火。为了保持火的连续性，我们决定使用权重把粒子束缚在它们的发射源上。原理很简单：当粒子创建的时候，它紧紧地束缚在发射源上。如果发射源移动，粒子也随它移动。当粒子衰老时，发射源的影响就减弱，那么粒子就自由地在空中漫游，见图 6-13。这样仅化费最小的操作代价，就可以使运动变得可信。

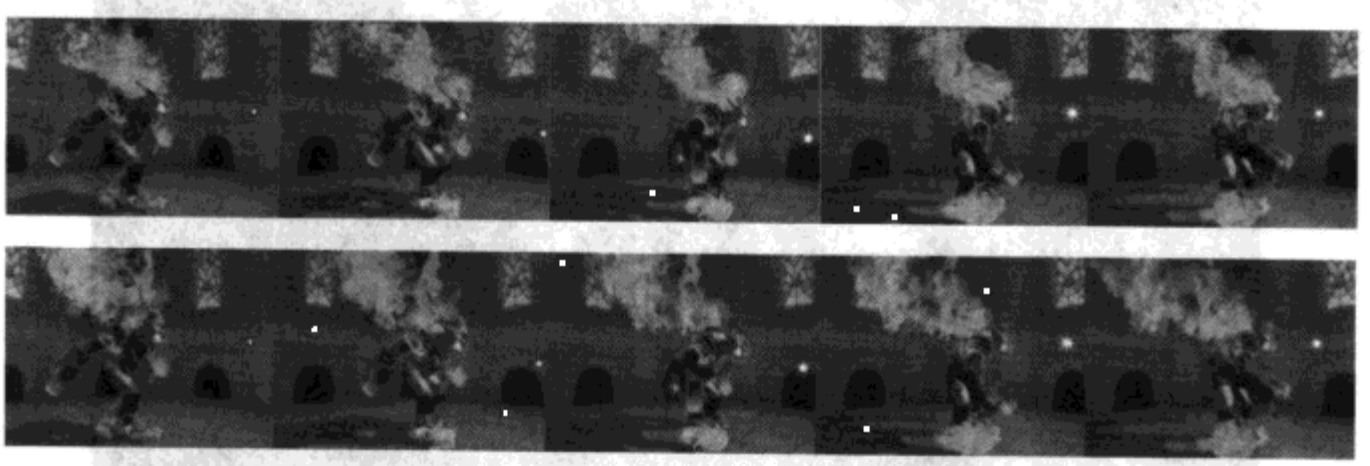


图 6-13　加权粒子使火焰可信

顶部：没有使用加权的几帧火。注意火焰移动得太多并且看起来不自然。底部：同样的帧加了权重

对烟的运动找一个流体力学的解法可能是最好的。当角色移动的时候，它会更新流体力学的解以影响烟的动作。我们花了很多力气来研究这种可能性。结果，流体力学本身是有效的而且可能是很快的，但是不适用于大的面积。我们打算在人物的根局部空间上进行模拟，让流体力学的解“跟随”他。不幸的是，粒子的数量少而尺寸大，我们不能利用流体力学创建微妙的动作。在以后项目中有更多（和较小尺寸）的粒子时，我们再利用这个解。

6.4　性能

从一开始我们就对性能极为关注。使用大的、视频纹理的公告牌耗费了大量的处理像素的能力。如图 6-14 所示，较之屏幕，子画面的平均尺寸是很大的。显然，我们尝试过这样设置混合：alpha 检测可以拒绝 alpha 为 0 的像素（GeForce FX 硬件能自动地做到这一点）。但是仍然有数以百万计的帧缓冲器混合和每帧的纹理操作，我们通过以下的方法减少它们。

6.4.1　层次合成

我们发现使用层次合成是一个解决办法。火源图像只有 256 × 256，因此，我们可以将它渲染进一个低分辨率缓冲区（仅为屏幕分辨率的 1/4）。当我们渲染火焰时混合操作就减少为原来的 1/4，见图 6-15。

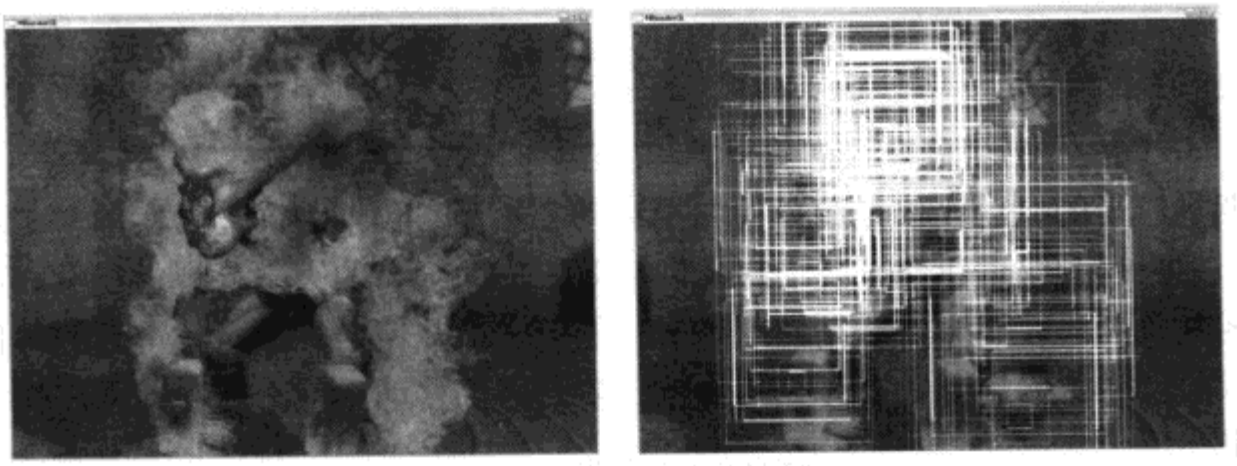
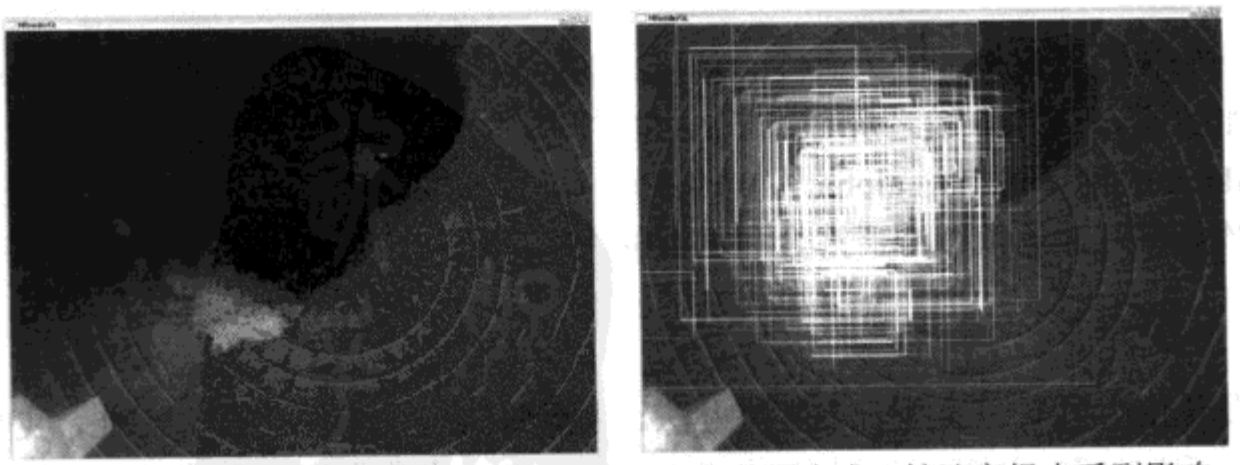


图 6-14 精灵尺寸和性能
数以百计的大的精灵能非常快地消耗填充率。这里，深度复杂性是明显的

图 6-15 使用层次合成
最终的合成图像，带有低分辨率的火源层（插图，右上方）

这对全部图像质量的影响是很小的，但是对性能的影响是极大的。现在有可能把照相机放在火焰的中央，并且仍然有一个很好的帧速率，见图 6-16。



（a）当使用精灵时，最坏的情况　　（b）使用合成，帧速率很少受到影响

图 6-16 使用精灵的最坏情况

6.4.2　定制的精灵

只要创建自己的精灵，我们就决定最好使用它们来提高性能。在 Vulcan 演示中，因为一些粒子是正方形，它只占用纹理表面的 1/3。结果，对于那些没用的纹素（texel），即使不把它们混合到帧缓冲区，读出它们也是浪费。我们可以用矩形代替正方形的精灵，那样能更好地适应某些火焰。更好的是做一遍来“分析”每一帧，以便创建最佳的长方形来适应它，然后增加一个 2D 偏移量，相对于粒子位置把精灵对中，见图 6-17。

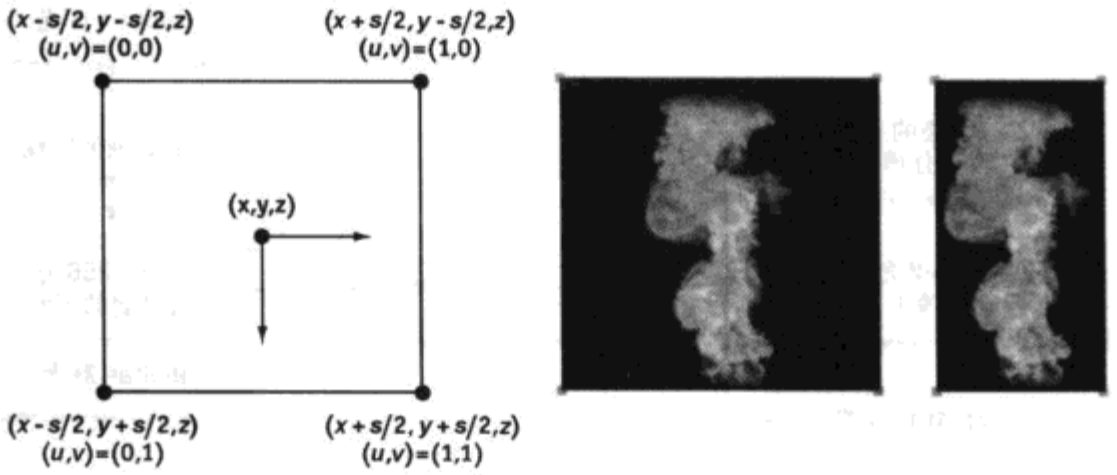


图 6-17　DirectX 9 点精灵的产生规则

6.5　渲染后的效果

使用实时合成迫使我们把场景渲染进一个纹理中。我们决定通过增加渲染后效果将这个限制变成一个优点。我们尝试了辉光、胶片颗粒（film grain）和热微光（heat shimmer）。因为我们更希望把像素处理的功能留给其他效果，比如 2048 × 2048 的阴影贴图，所以有些效果没有进入最后的演示。从概念上说，可以作为渲染后效果以一个线性成本加入许多 2D 过滤器，比如一个颜色校正矩阵，像我们在 Toys 演示中所做的一样。

因为是在一个渲染目标中渲染火，我们需要针对人物对它进行 z 检测。理想情况下，火的渲染应该与渲染的场景共享深度缓冲区，但是由于渲染目标尺寸不同，这就妨碍了对深度缓冲区的共享。我们只好把低分辨率的模型渲染进火焰渲染目标的深度缓冲区，见图 6-18。

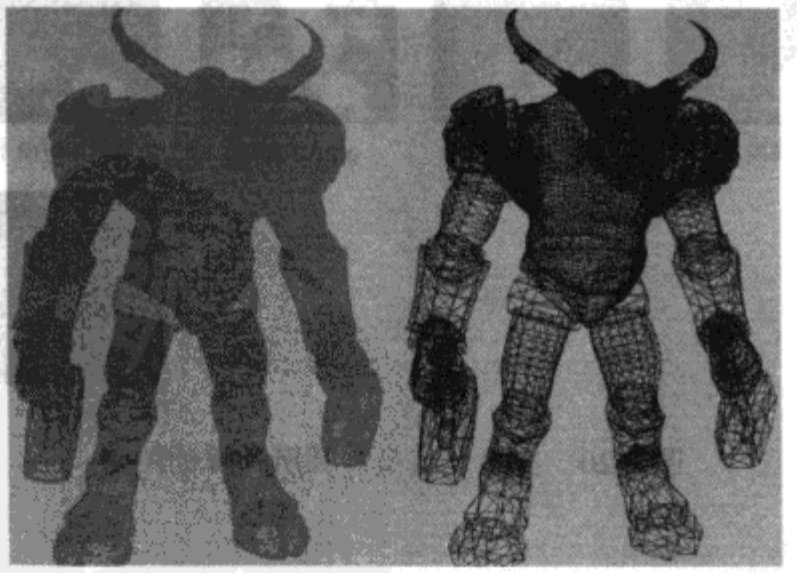


图 6-18
左：高分辨率的模型；右：用于火的 z 检验的低分辨率网格

这给顶点性能加了一点压力，但是省去了像素处理。因为辉光视觉效果最好，所以只有它出现在最后的演示中，见图 6-19。

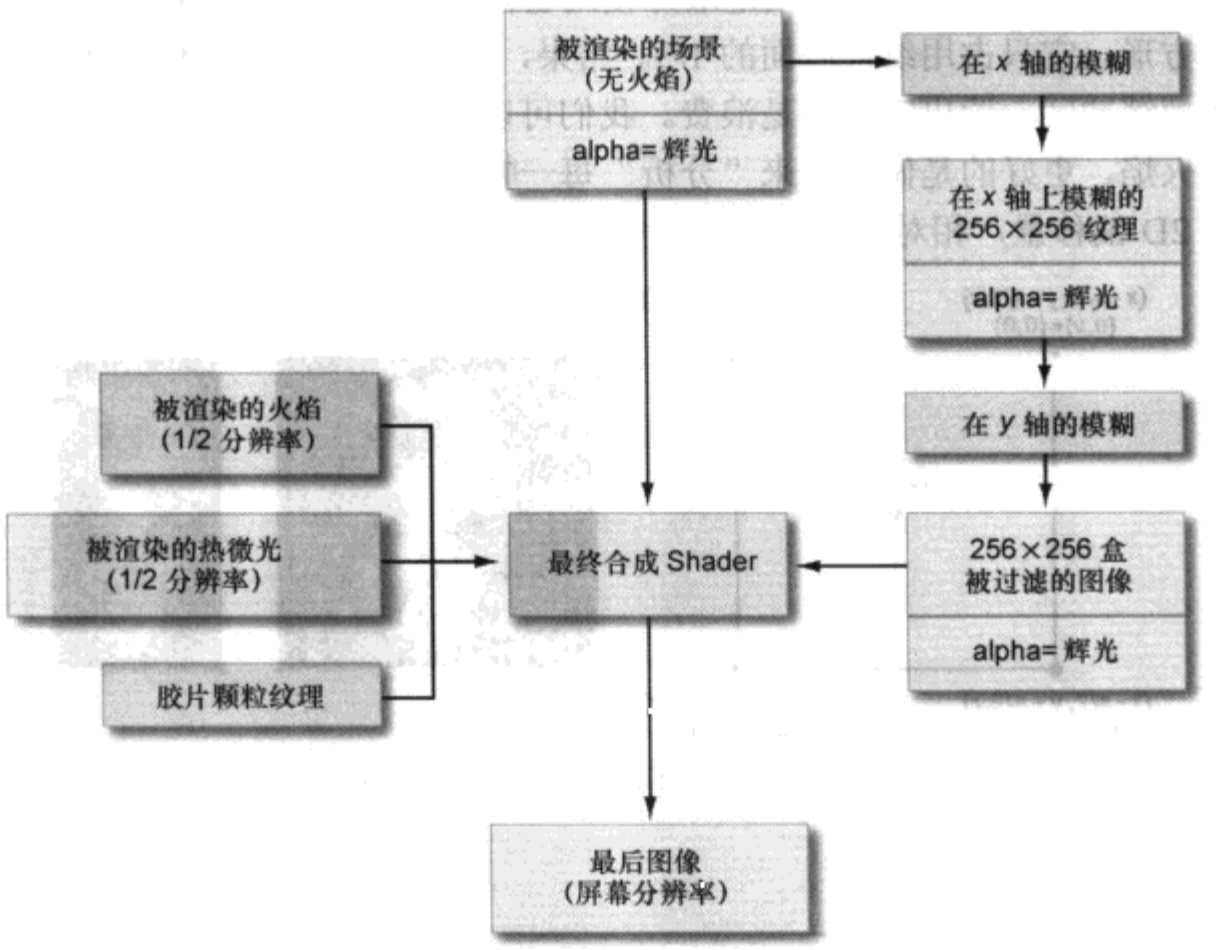


图 6-19 一个渲染流程

6.5.1 辉光

我们在原始场景上选择性地添加一个模糊的场景，以创建辉光。模糊的方法是一个 2 遍的盒滤波器。在第一个通道中，我们在 x 轴上添加一个 8 像素宽的模糊点。然后再把一个 8 像素宽的模糊点加在 y 轴上。我们使用模糊图像的 alpha 通道，选择将要模糊哪个物体。

图 6-20 说明模糊和合成过程。本书的第 21 章“实时辉光”讨论了这个主题的更多细节。

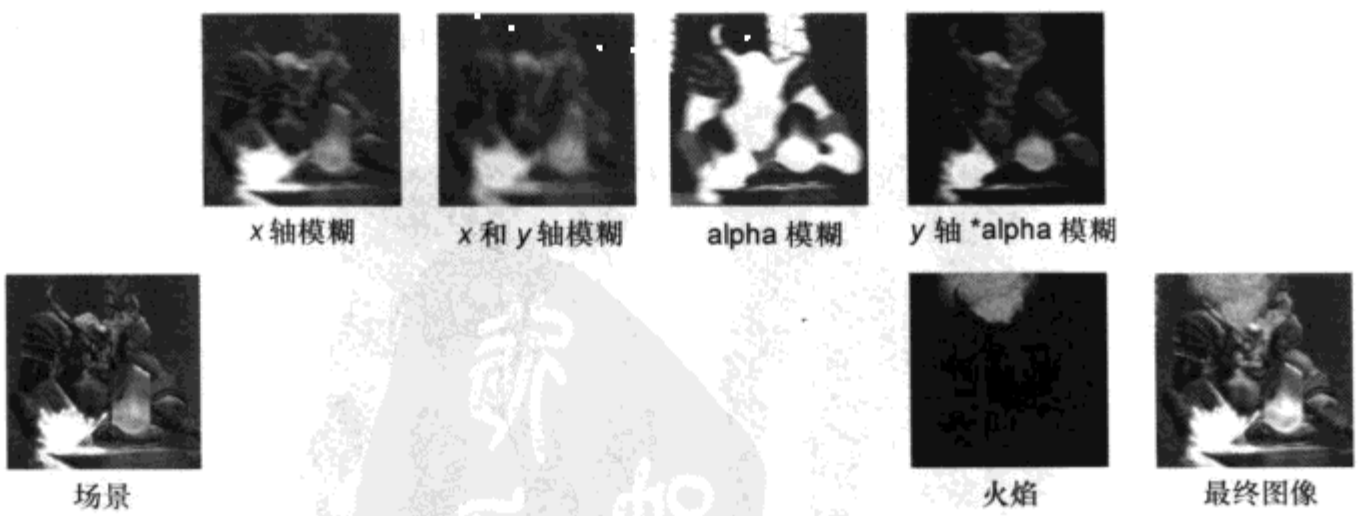


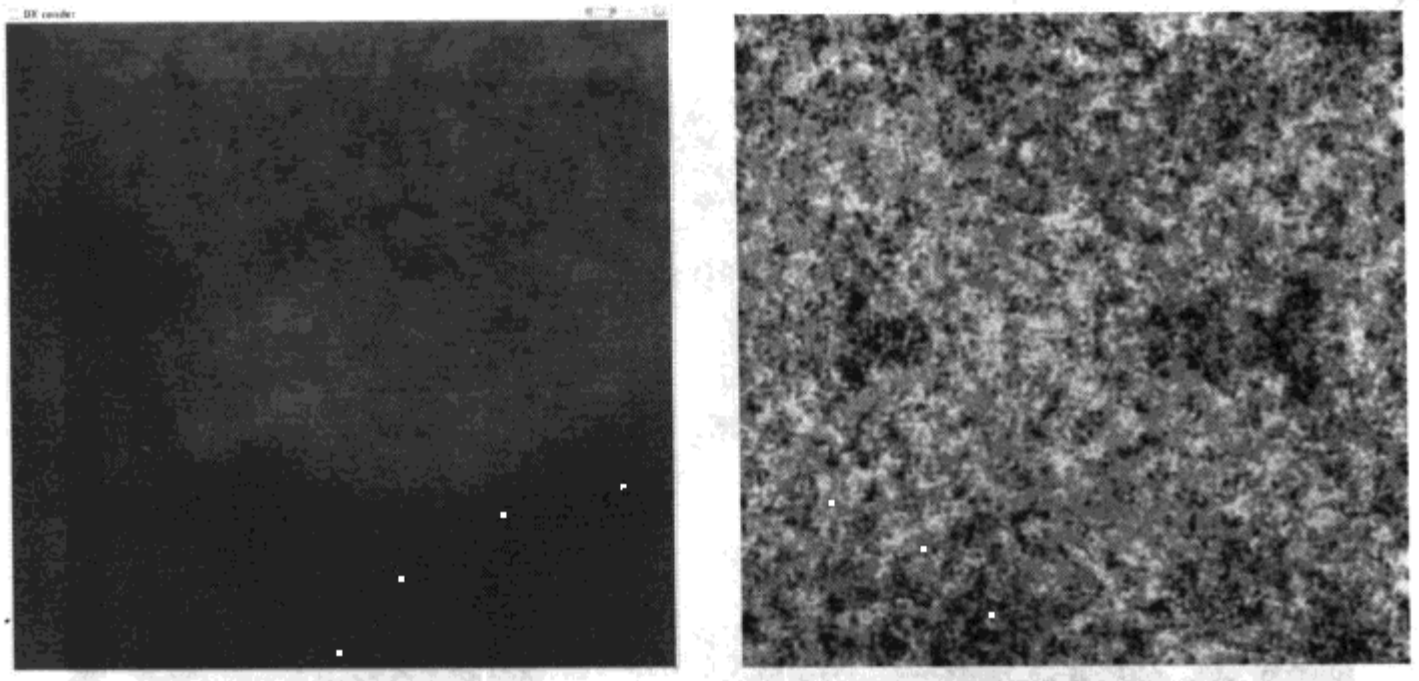
图 6-20 在各个渲染步骤所计算的图像

6.5.2 热微光

虽然我们为热微光写了所有的代码，但是最后还是决定不使用它。因为这种景物太暗了，

不值得为它花费那么多宝贵的资源。

热微光效果是以一种直接方式制作的：我们渲染一个在纹理目标中的“热”粒子的粒子系统。在最后的合成中，使用每个“热渲染目标”像素的值（红、绿），代替每个像素的 2D 纹理坐标 (u, v)，这是在取得所渲染场景的纹理目标的纹理点时做的。这就创建了噪声效果，它看起来像热粒子闪光一样，见图 6-21。



（a）热粒子渲染在纹理中　　（b）热微光噪波纹理

图 6-21　热微光

许多游戏开发者在每个顶点基础上使用热微光，我们也可以这样做。例如，可以从同样的“热渲染”读值，然后用这些值扰动细划分的 2D 网格，并使图像变形，见图 6-22。

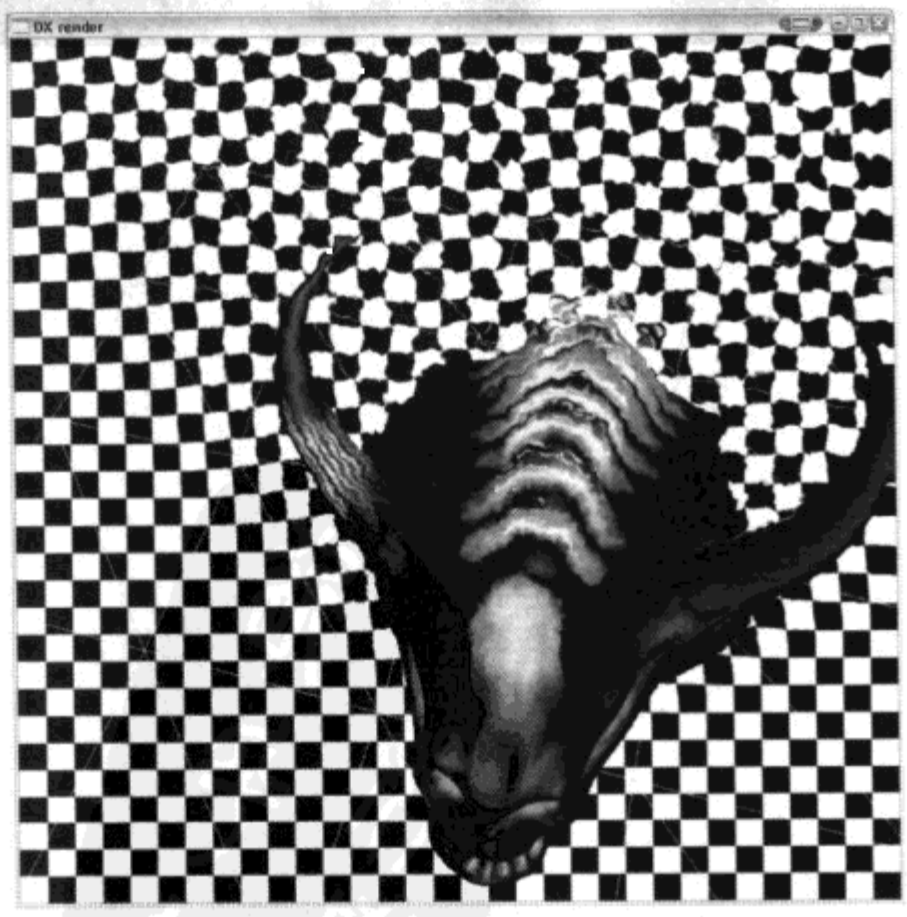


图 6-22　开发中每个像素的热微光

6.5.3 颗粒

电影业中时常用到颗粒，为的是让计算机所产生的效果有胶片样的颗粒。在我们的演示中，使用颗粒把渲染变得“朦胧”，它工作得不错。但是我们担心观众可能不喜欢颗粒，或更糟的是，把它当作“锯齿”，因此，我们没有用它，只在使用 Photoshop 噪波过滤时，创建过颗粒纹理，见图 6-23 和 6-24。

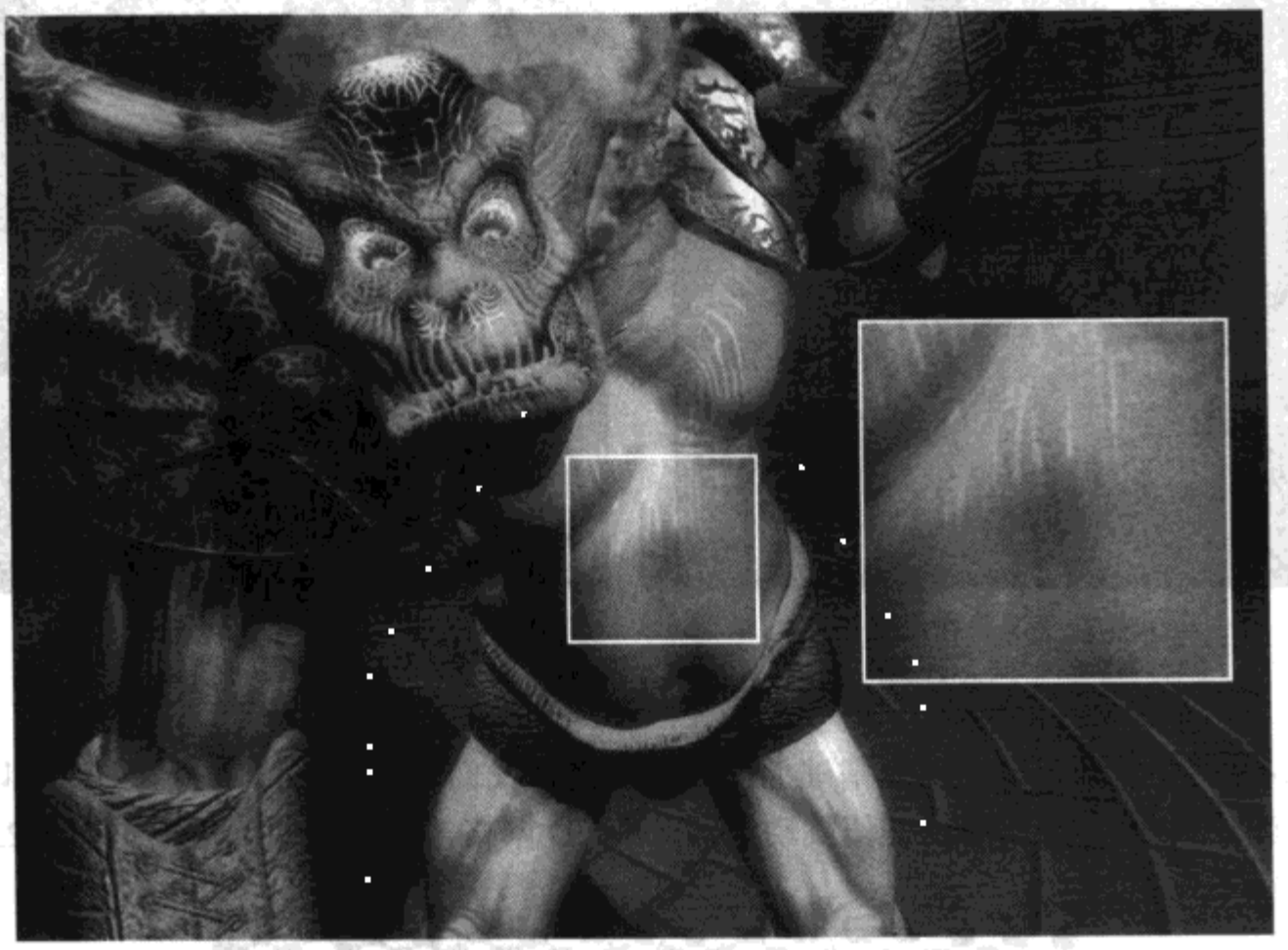


图 6-23 在晶粒上的放大

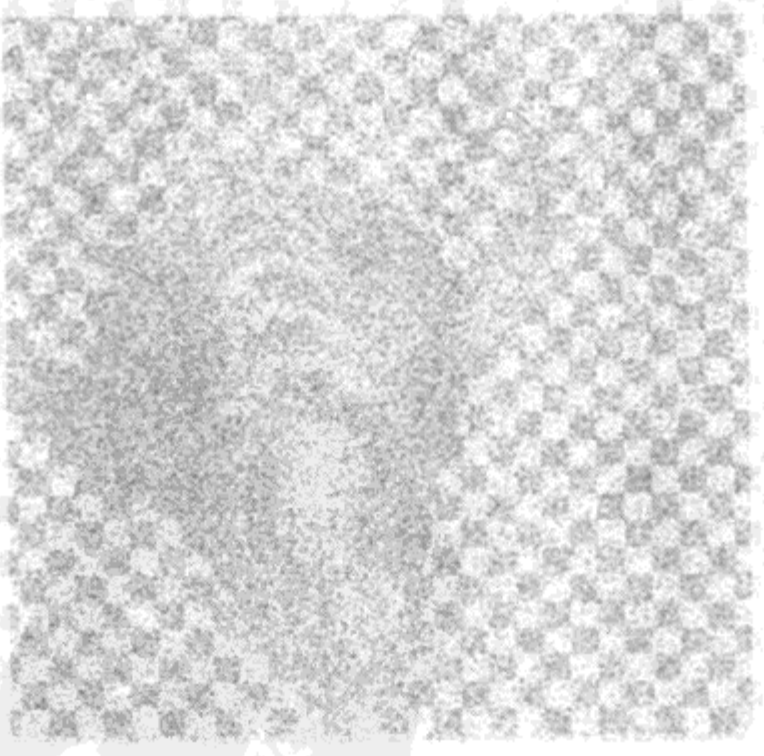


图 6-24 存储晶粒图案的纹理

6.5.4　最终的程序

列表 6-1 显示把所有不同层次放在一起，创建最后图像的 shader。

列表 6-1

```
struct v2fConnector {
  float2 tex1024x768      : TEXCOORD0;
  float2 colorTexGrain    : TEXCOORD1;
  float2 tex256x192       : TEXCOORD2;
  float2 tex512x384       : TEXCOORD3;
  float2 texNormalized    : TEXCOORD4;
};

struct f2fConnector {
  float4 COL;
};

f2fConnector main(v2fConnector v2f,
                  uniform texobjRECT renderedflames, // "Flames"
                  uniform texobjRECT renderedscene, // "Scene"
                  uniform texobj2D   grain, // "Grain"
                  uniform texobjRECT heat, // "Heat"
                  uniform texobjRECT blur) // "BlurY", "Blur Alpha"
{
  f2fConnector f2f;

  // Fetch heat distortion offset
  half4 HeatDistort = f4texRECT(heat, v2f.tex512x384) * float4(17, 23, 0, 1);

  // Fetch scene render using heat distortion
  half4 Scene = f4texRECT(renderedscene, v2f.tex1024x768 + HeatDistort.xy);

  // Fetch blurred version of the scene
  half4 BlurredScene = f4texRECT(blur, v2f.tex256x192);

  // Fetch flames and smoke
  half4 FlamesSmoke = f4texRECT(renderedflames, v2f.tex512x384);

  // Extract bright parts from blurred scene
  half4 Glow = BlurredScene * BlurredScene.a + Scene;

  // Compute final color output
  half4 Fcompoblur = lerp(Scene, BlurredScene, HeatDistort.a);
  half4 Fcomposited = lerp(Fcompoblur, FlamesSmoke, FlamesSmoke.a);

  f2f.COL = Fcomposited + Glow;

  return f2f;
}
```

6.6 小结

我们最后终于达到了目标，能够显示一些非常逼真的火。使用基于视频的方法是个好的解决方案。虽然这个技术占用相当多的内存，但它是可伸缩的，也能够用在内存很少的 GPU 上。火的效果一直是许多游戏都需要的，我们希望这章能激发你的创作灵感，使你在自己的应用中创建出更高级的火的效果。

第 7 章 无数波动草叶的渲染

Kurt Pelzer
Piranha Bytes

7.1 引言

为了以写实的风格模拟田园诗般的自然场景，除了精致的树林和灌木，复杂的水面和穹庐的天空，还需要高质量的草的效果。我们必须能够覆盖大面积的地形，还不能因此而独占 GPU。草不但应该看起来生长得自然，而且还要能够逼真地在风中舞动。

过去，人们认为草的高质量模拟对于实时应用来说太复杂了。Codecreatures 基准（Benchmark）（在 2002 年由 Codecult 出版）证明了这是过于悲观的假设，见图 7-1。在这章中，我们介绍一种灵活的、广泛应用的草模拟，基于基准中演示的草的效果。另外，本书附带的 CD 中包含一个特别版本的 Codecreatures 基准程序，它提供一个交互的演示模式。

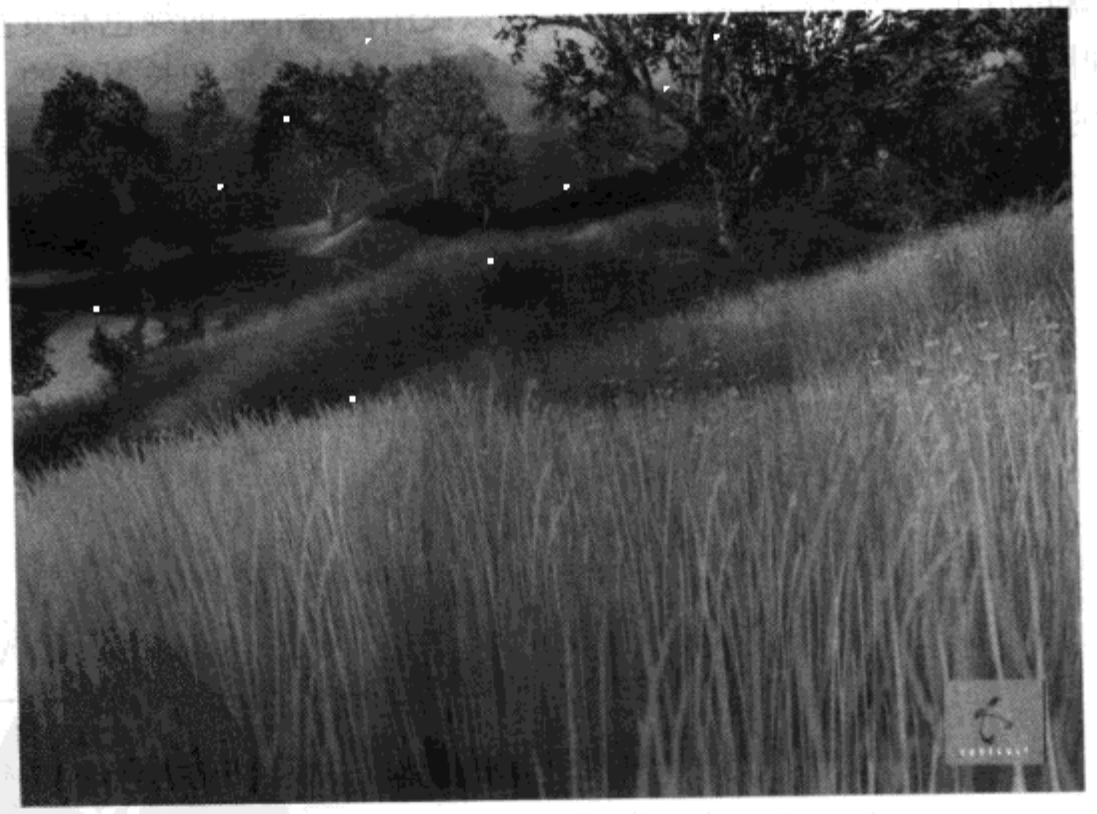


图 7-1 Codecreatures 基准程序的屏幕截图

7.2 概述

首先，我们应该认识到对单个草叶的细节建模意义不大，因为那样大

片的草地需要的多边形的数目会太多。对今天的图形硬件而言，要产生一个实时的、有无数多边形草叶的场景是不现实的。

因此我们必须提出一个简单而有用的替代方案，来满足以下条件：

- 许多草的叶片必须由少数多边形代表；
- 从不同的视线看起来草都必须显得密集。

在下一节中，我们将建立满足这些条件的草体。

另外，我们必须能够使草逼真地动画。在第 7.4 节中，我们将讨论 3 个不同的动画方法。

7.3 草体的准备

如上所述，许多草叶一定要由少数多边形来表现。我们从解决这个问题开始。让场景不依赖于摄像机的位置和方向，看起来像在空旷的乡间一样。所幸的是，这一点解决起来还不是太困难。在第 7.3.1 节中，我们把一些草叶组合起来，表示在一个纹理中。但是仅仅这样还是不够的：在用这个纹理表现几个多边形时，必须把它们组合起来，而且在结果中单个的多边形不应该引起注意（见 7.3.2 节）。当观者在附近活动的时候，我们可以通过进进出出地混合草体，在一定距离内增加或删去草体。这将保证整个草的效果具有稳定的视觉质量。

7.3.1 草的纹理

现在来看如何创建一个可以解决这个问题的纹理。所需要的纹理应该是一些丛生的草；否则，将会出现大片的透明区域。做到这一点很容易，只要在透明的 alpha 通道上画实体的草茎就可以了。在彩色通道中，我们应该使用深浅不同的绿色和黄色，来较好地区别各个单独的叶片。我们还应该模拟不同情况的草叶：长得好的和长得差的、老的和嫩的，甚至区别叶片的前面和后面。

一个草纹理的具体例子显示在图 7-2 中。

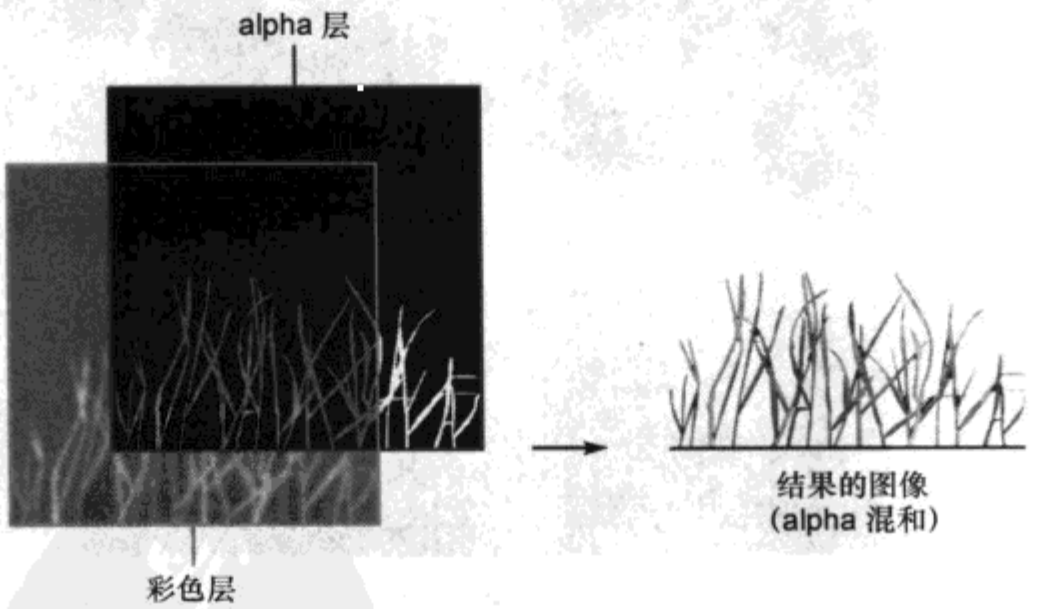


图 7-2 草纹理的示意图

7.3.2 草体

本节将解释如何组合一些多边形，并用上节提到的草纹理进行映射，组合结果应该使模

拟的草显得茂密，而不凸显个别的多边形。这个技术也保证看不到个别的多边形。

因为用户能够自由地在场景中游弋，图 7-3 中所显示的结构就不能产生令人信服的效果了。如果从垂直于多边形的方向观看场景，立刻就会看出草多边形的结构是线性排布的。另外，在这种情况下草地会看起来非常稀疏。只有在摄像机自动游弋，或无法到达远距离的草地时，才会考虑像这样的排布。

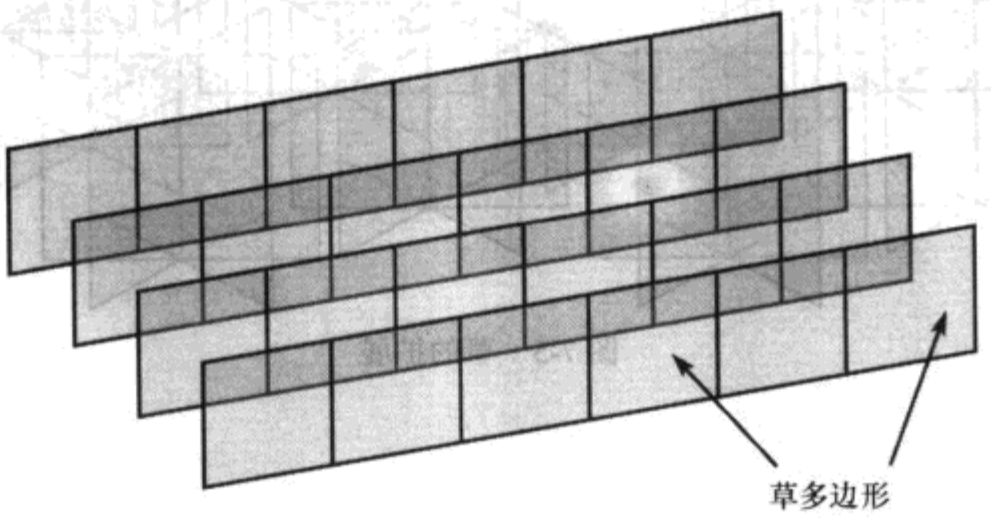


图 7-3　线性排布

为了保证好的视觉质量不受目前视线的约束，我们必须交叉草多边形。已证明使用星形结构是非常好的。图 7-4 给出“草体”的两个可能的变异，它们是由 3 个相交的方块构成的。我们必须禁用背面剔除来渲染多边形，以保证双面都可见。为了得到合适的照明度，应该让所有顶点的法线方向与多边形的垂直边平行。这保证了位于斜坡上的所有草体都可以得到正确的照明，不会由于地形的亮度而出现差异。

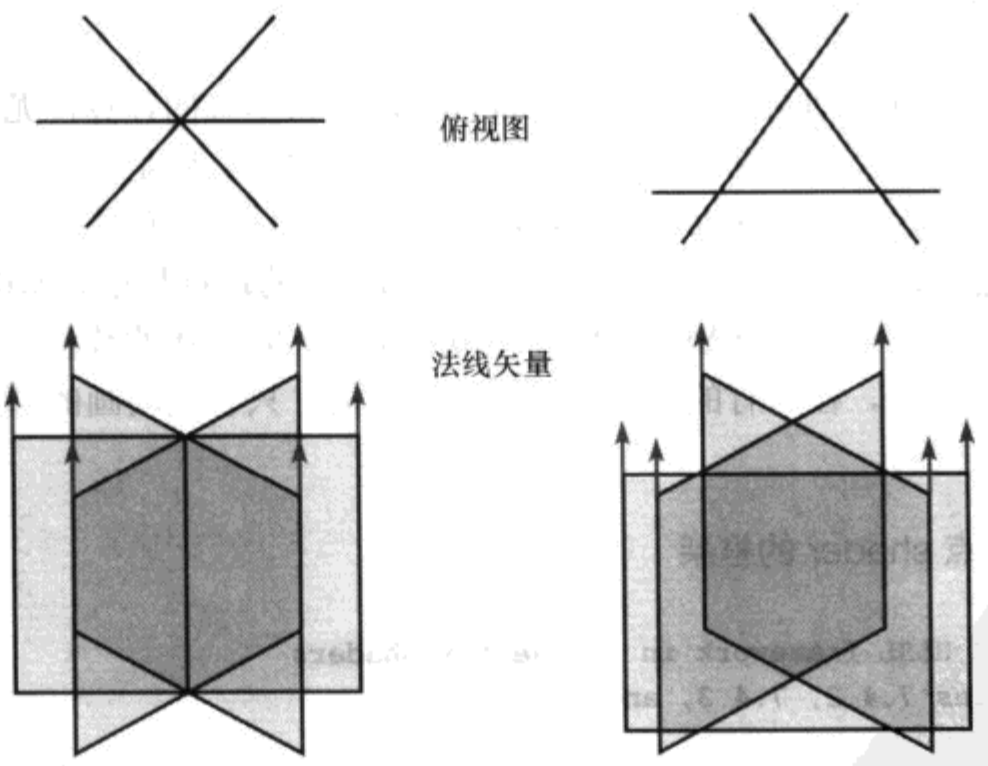


图 7-4　草体

如果把这些草体彼此相当靠近地设置在一个大的区域里，如图 7-5 所示，在运行期间把它们从后到前地排序，使用 alpha 混合，并启用绘制调用中的 z 测试/写入，那么就会得到自然而茂密的草地图像。

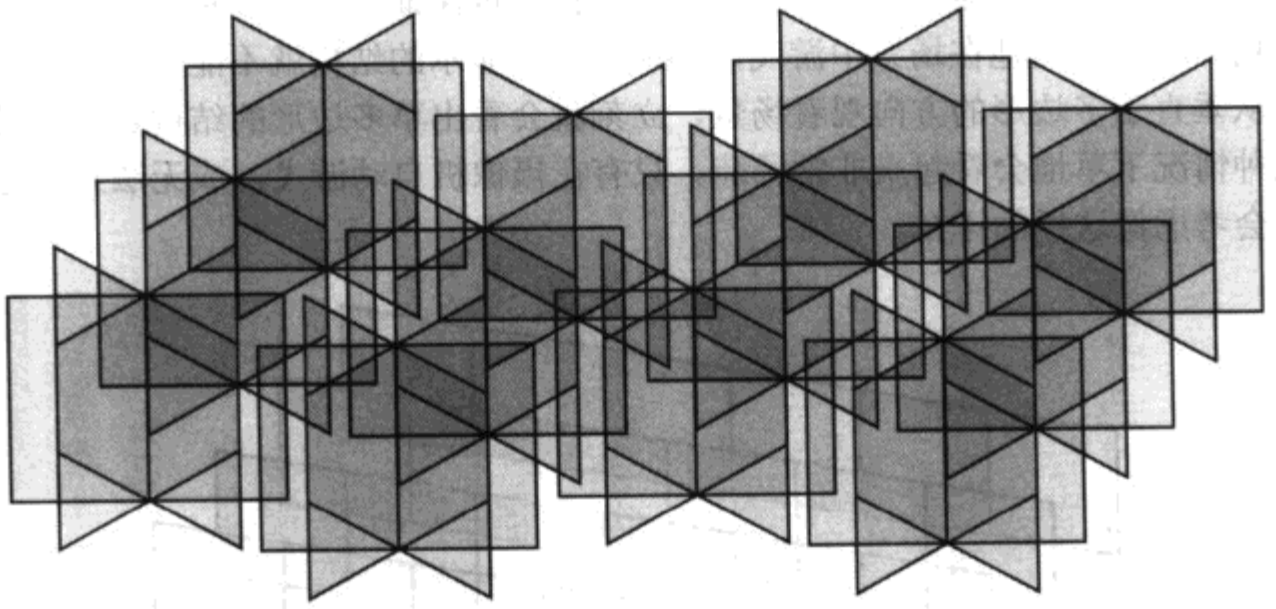


图 7-5 草的扩展

7.4 动画

下一步就是要使整个草地的草逼真地动画起来，草地是用图 7-5 中呈现的那种“草体”构造而成的。这节描述 3 种不同的动画。每个都有其优点和缺点。第 7.4.1 节提出动画方法的一般思路。在第 7.4.2 节中，聚集在一起的几个草体簇以相同的方法运动。在第 7.4.3 节中，每个顶点都有它自己的平移矢量。最后，在第 7.4.4 节中，我们试着寻找最好的方法，让每个草体具有一个不同的动画。

7.4.1 一般思路

为了得到高度逼真的动画，我们将进行以三角函数为基础的计算，尤其是正弦和余弦。计算应该考虑到必须移动的位置（无论它是草体或草丛的顶点还是中心）和当前的时间。同时，风的方向和强度也是考虑的因素。我们的每种技术都只移动草体较高的顶点。在一个顶点 shader 中，通过检查纹理坐标，区别较高的顶点和较低的顶点是很容易的。对草纹理而言，所有的上面顶点应该有相同的 v 坐标：例如为 0，或一个接近 0 的数值。如列表 7-1 所示的顶点 shader 代码的框架，在所有的 3 种技术中是相同的；只有纯动画的部分有差别。可以从以下部分找到动画代码。

列表 7-1 顶点 shader 的框架

```
//
// Equal Cg / HLSL framework in the vertex shaders
// for Sections 7.4.2, 7.4.3, and 7.4.4
//

struct VS_INPUT {
  float3 vPosition : POSITION;
  float3 vNormal   : NORMAL;
  float2 TexCoords : TEXCOORD0;
```

```
  // This member is needed in Section 7.4.4
  float3 vObjectPosition : TEXCOORD1;
};

struct VS_OUTPUT {
  float4 vPosition : POSITION;
  float4 vDiffuse  : COLOR;
  float2 TexCoords : TEXCOORD0;
};

struct VS_TEMP {
  float3 vPosition;
  float3 vNormal;
};

  float4x4 mWorldViewProjMatrix;
  float4 vLight;
  float fObjectHeight;

VS_OUTPUT main(const VS_INPUT v)
{
  VS_OUTPUT out;
  VS_TEMP temp;

  // Animate the upper vertices and normals only
  if (v.TexCoords.y <= 0.1) { // Or: if(v.TexCoords.y >= 0.9)
    // ANIMATION(to world space)
    // Insert the code for 7.4.2, 7.4.3, or 7.4.4
    . . . // <- Code for our different animation methods
  }

  // Output stuff
  out.vPosition = mul(float4(temp.vPosition, 1), mWorldViewProjMatrix);
  out.vDiffuse = dot(vLight, temp.vNormal);
  out.TexCoords = v.TexCoords;
  return out;
}
```

7.4.2　每丛草体的动画

下列的方法用于 Codecreatures 基准程序，而且能够随着阵风产生逼真的效果，风的强度和方向是在不变地改变着。这里，对一丛靠近的草体，上面的多边形顶点统一地发生移动。为了产生一个看起来自然的动画，我们选择的草丛尺寸不应该太大，见图 7-6。

动画的平移矢量是由 CPU 计算的，并且作为一个常量移交给顶点 shader。若在 CPU 上使用更费时的算法，还可以让我们利用非常复杂的风模拟。因为每丛草体由它自己的平移矢量来支持，所以必须针对每丛草体来改变这个常量。由此不得不经常地打断对整个草地的渲染，而且为每丛草体使用独立的绘图调用。

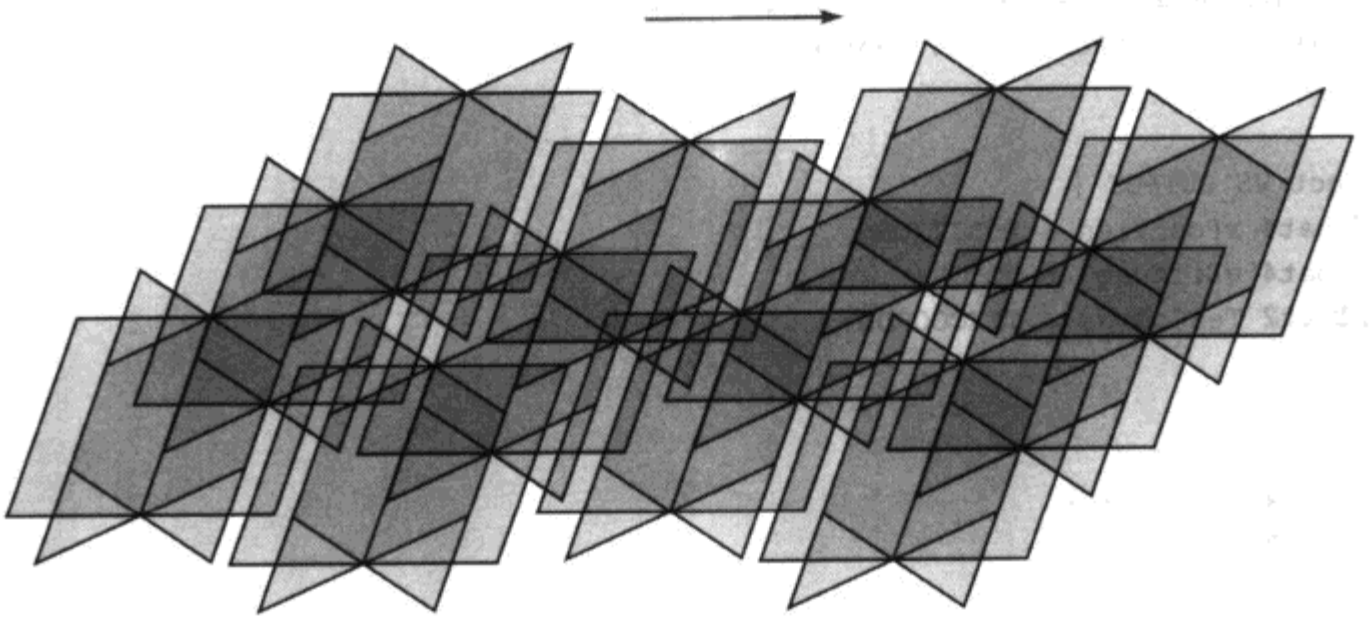


图 7-6 每丛草体的动画

优点

- 复杂的动画计算基于 CPU 的算法进行。
- 因为一个多边形上面顶点的距离是固定的，所以不发生变形。

缺点

- 为了显示一个完整的草地，需要许多的绘图调用。
- 由于整个草丛的所有顶点同步动画，可能出现聚集成簇的现象。

算法

1. 在 CPU 上通过草丛的中心位置，计算下一个草丛的当前平移矢量。
2. 为顶点 shader 把平移矢量设置为一个常量。
3. 为草丛执行一个绘图调用。
4. 在顶点 shader 中，把平移矢量加到上面顶点的位置上，见列表 7-2。

列表 7-2 每丛草体动画的代码

```
//
// Animation per Cluster of Grass Objects (7.4.2)
//

float3 vClusterTranslation; // Calculated on CPU

VS_OUTPUT main(const VS_INPUT v)
{
  . . .
  // ANIMATION (to world space)
  // Here comes the code for 7.4.2
  temp.vPosition = v.vPosition + vClusterTranslation;
  temp.vNormal = normalize(v.vNormal * fObjectHeight + vClusterTranslation);
  ...
}
```

7.4.3　每个顶点的动画

7.4.2 节所讨论的方法的主要问题之一是运行性能差，因为绘图调用的次数很多，而每次调用又只是个别地渲染少数的多边形。如果仅使用少得多的绘图调用，则渲染一个草覆盖的很大区域就能好得多。但是那样一来，我们必须把整个动画计算移进顶点 shader 内，shader 能够相对于它的位置分别地移动每个顶点。见图 7-7。

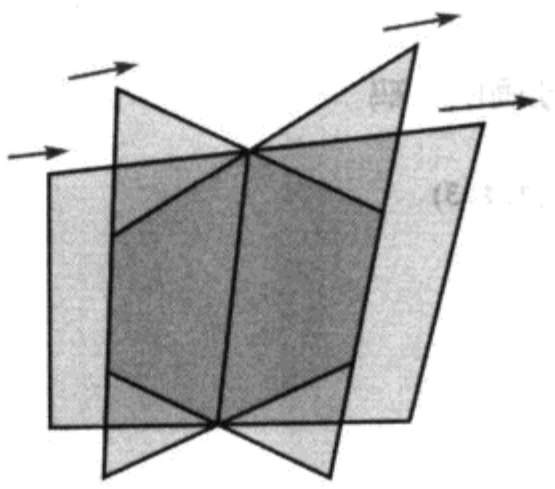


图 7-7　每个顶点的动画

因为对每个顶点的移动是分别计算的，草多边形的上面顶点之间的边长就不再是常数了，如图 7-8 所示。因此，因为每个草叶的长度和厚度不是常量，所以可见的变形就会出现，但是通常这些失真还不易觉察。

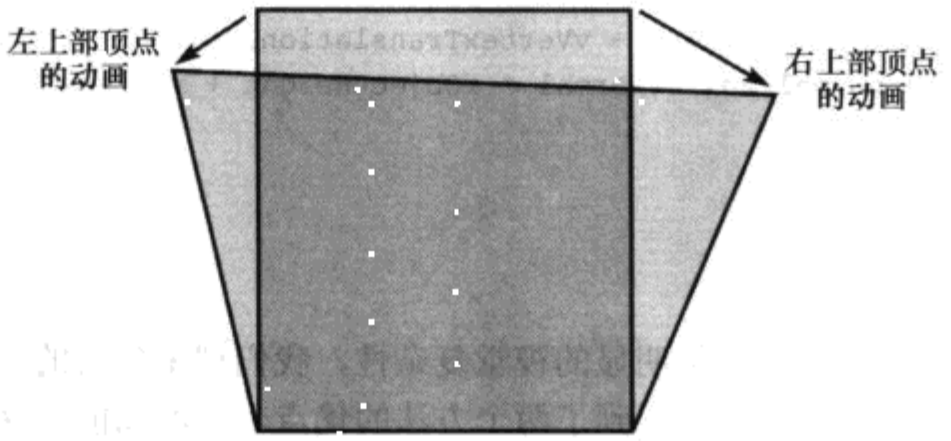


图 7-8　纹理变形

另外，这个总的效果似乎比前一个方法更不自然。因为在一个邻近的区域中，所有顶点的移动非常相似，所以缺少了自然状态的局部无序，产生的动画显得过于均匀。在顶点 shader 中使用一个伪随机函数，能够除去这个缺点，得到更加多变的结果。

优点

- 只用很少的绘图调用，甚至仅仅 1 个，就能显示一个完整的草地。
- 在顶点 shader 中改变顶点位置，产生连续的风吹草地引起的波动荡漾。
- 不会看出个别的草丛。

缺点

- 由于多边形上面顶点间的距离可变，出现变形。

- 由于缺少了局部的无序，动画可能显得过于均匀。
- 动画计算的复杂性受到限制。

算法

1. 为顶点 shader 设定常量，例如风的基本强度和方向，以及时间延续性。
2. 为完整的草地或大面积的草场，执行绘图调用。
3. 使用顶点 shader 计算基于顶点位置的动画。见列表 7-3。

列表 7-3 用于每个顶点动画的代码

```
//
// Animation per Vertex (7.4.3)
//

float fTimeStamp;
float3 vWindDirection;
float fWindStrength;

VS_OUTPUT main(const VS_INPUT v)
{
  . . .
  // ANIMATION (to world space)
  // Here comes the code for 7.4.3
  float3 vVertexTranslation = CalcTranslation(v.vPosition, fTimeStamp, vWindDirection,
                              fWindStrength);
  temp.vPosition = v.vPosition + vVertexTranslation;
  temp.vNormal = normalize(v.vNormal * fObjectHeight + vVertexTranslation);
  . . .
}
```

7.4.4 每个草体的动画

为了给前两节所述的动画增加明显的视觉复杂性，我们把未失真的草纹理和少量的绘图调用与局部的无序结合起来，从而得到了两个方法的优点。之所以能够这样做，是因为我们计算动画不是基于每个顶点的位置，而是基于草体的中心位置，而每个草体是由 3 个方块组成的（见 7.3.2 节）。

因为附近的草体现在有不同的动画，我们能表现那种要求的局部混乱，如图 7-9 所示。另外，每个草体的恒定动画避免了水平方向的纹理变形。

为了做到这一点，每个顶点也必须知道它的草体的中心位置，即它的相对位置或在世界空间中的绝对位置。因为顶点 shader 必须读这个数值，所以这个信息所需要的草体位置矢量必须在顶点格式中（即，存储在纹理坐标中）。

优点

- 只需要几个绘图调用，甚至仅仅 1 个就能显示一个完整的草地。
- 因为多边形的上面顶点的距离固定，所以没有变形。
- 局部多变，产生更自然的画面。

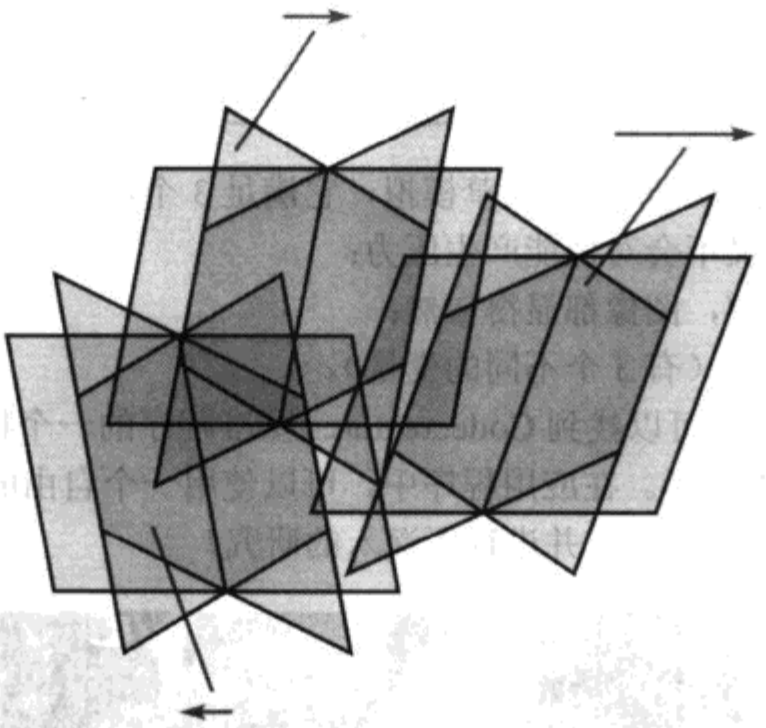


图 7-9　每个草体的动画

缺点

- 因为每个顶点也包含它的草体的中心位置值，所以在顶点格式中需要附加的数据。
- 为了缩小 shader 开销，动画计算的复杂性受到限制。

算法

1．为顶点 shader 设定常量，例如风的方向、基本强度和时间延续性。
2．为完整的草地或大面积的草场，执行绘图调用。
3．在顶点 shader 中，以草体的中心位置为基础计算动画，见列表 7-4。

列表 7-4　用于每个草体动画的代码

```
//
// Animation per Grass Object (7.4.4)
//

float fTimeStamp;
float3 vWindDirection;
float fWindStrength;

VS_OUTPUT main(const VS_INPUT v)
{
  . . .
  // ANIMATION (to world space)
  // Here comes the code for 7.4.4
  float3 vObjectTranslation = CalcTranslation(v.vObjectPosition, fTimeStamp,
                              vWindDirection, fWindStrength);
  temp.vPosition = v.vPosition + vObjectTranslation;
  temp.vNormal = normalize(v.vNormal * fObjectHeight + vObjectTranslation);
  . . .
}
```

7.5 小结

我们已经成功地构建了一个逼真的草模拟，它满足 3 个最重要的要求：

- 广泛的可用性，又不会对性能产生压力；
- 从所有的视线观察，图像都显得自然；
- 动画基于刮风条件（有 3 个不同的变量）。

在本书的 CD 或网站上可以找到 Codecreatures 基准程序的一个特别版本，它提供了如图 7-10 所显示的一个交互的演示。在应用程序中，可以使用一个自由的摄像机游弋，并转换渲染状态。欢迎检测这个应用程序，并进行更深入的研究！

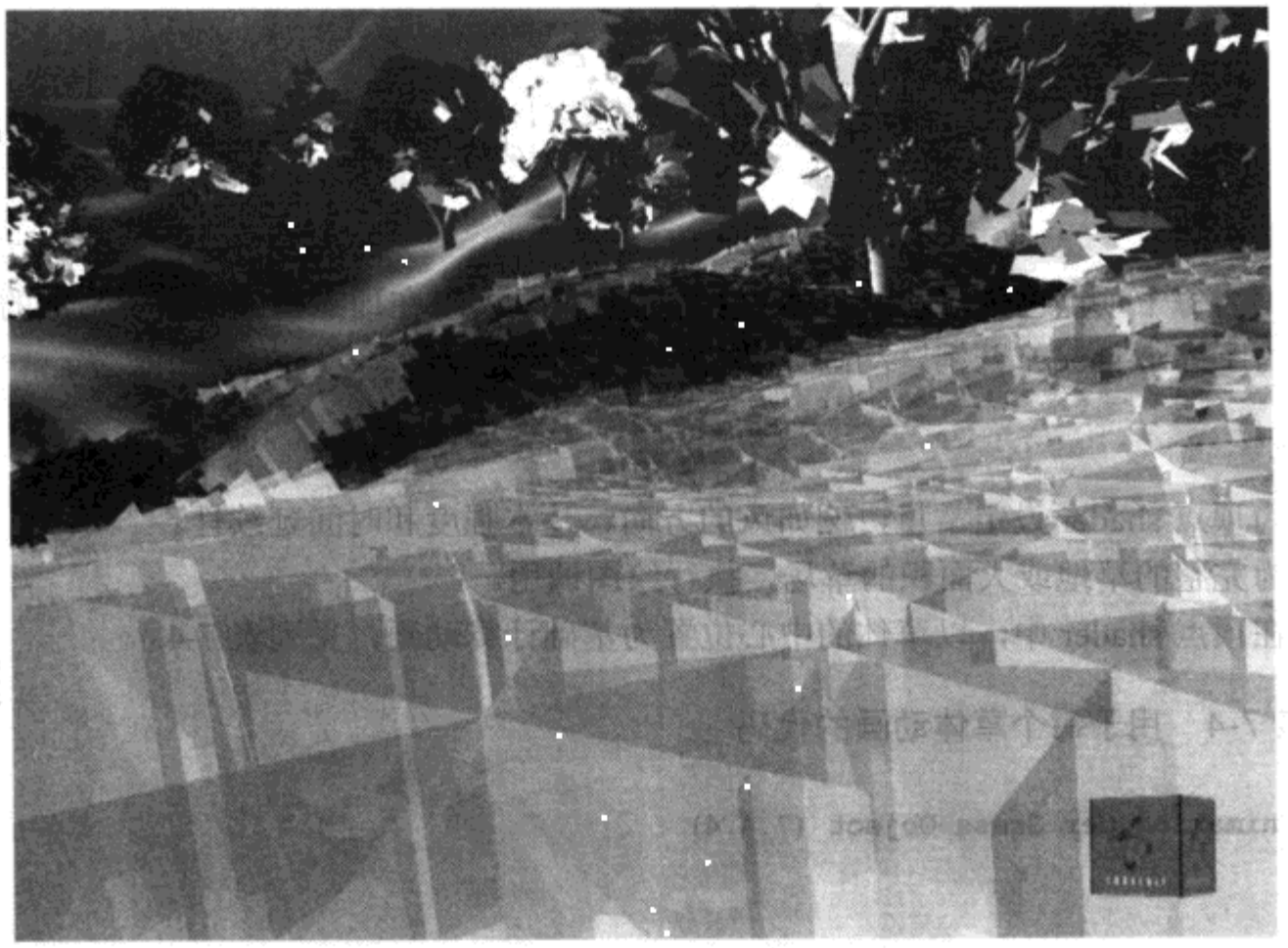


图 7-10 Codecreatures 基准程序的屏幕截图

7.6 参考文献

如果对进一步研究草的模拟动画感兴趣，可以参考如下提供的有关这个主题的资料。

可以在这里找到描述顶点 shader 的文章，它使用一个正弦函数做草的几何体的程序式动画。

NVIDIA Corporation. 2003. "Basic Profile Sample Shaders: Grass." In *The Cg Toolkit User's Manual*. Available online at **http://developer.nvidia.com/object/cg_users_manual.html**

Isidoro, J., and D. Card. 2002. "Animated Grass with Pixel and Vertex Shaders." In *Direct3D ShaderX*, edited by W. F. Engel. Wordware Publishing.

提出草体效果的其他样本发布在以下两个网站上。这两个样本也使用顶点 shader 计算草

的几何体的波动。

NVIDIA Web site: **http://developer.nvidia.com/view.asp?IO=demo_grass** ATI Web site: **http://www.ati.com/developer/Samples/Grass.html**

感谢 Piranha Bytes 和 Codecult 中我的同事对 Codecreatures 基准程序所做的贡献，特别是 Horst Dworczak（主美工），他提供了在基准程序中构建和动画草体的思路，还有 Oliver Hoeller（主程序员），他帮助把全部的效果整合到引擎中。

第 8 章 衍射的模拟

Jos Stam
Alias Systems

8.1 什么是衍射

计算机绘图的大多数表面反射模型都忽略自然光的波动效果。当表面的细节比光的波长（约 1μm）大许多时，不存在问题。但是对小尺度的细节，例如一个光盘的表面，波效应就不能忽略了。小尺度的表面细节引起反射波彼此干扰。这个现象就是衍射。

衍射使这些表面的反射光呈现五彩缤纷的图案，由光盘的精细反射可以看到这一现象。展现衍射的其他表面现在很普遍，而且正大量地生产，例如时髦的包装纸、五彩缤纷的玩具和奇妙的水印等。

在这章中我们介绍如何实时地为任意表面的衍射效果建模。这可能要多谢目前图形卡上的可编程硬件。我们特别提供了一个使用 Cg 程序语言的 shader 的完全实现。这个 shader 是 *SIGGRAPH 1999* 论文（Stam 1999）中描述的一般模型的简化版本。

8.1.1 波动光学

波动性是光的本性之一。实际上，用于计算机绘图的光线理论是这个波理论的近似。许多自然现象的物理学理论中都出现波。这不奇怪，因为自然界在时间上和空间上存在着大量重复的图案。

如图 8-1 所示的简单的一维波，完全由它的波长 λ 和振幅 A 表示。波长表示振荡性，而振幅决定波的强度。可见光由这些波叠加组成，波长范围从 0.5 个 μm（微米）（紫外线）到 1 个 μm（红外光）。光源的颜色由它

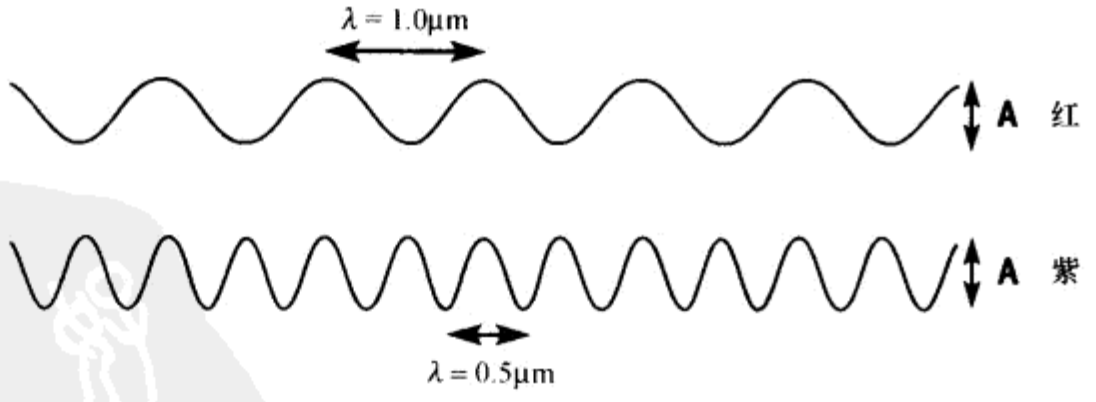


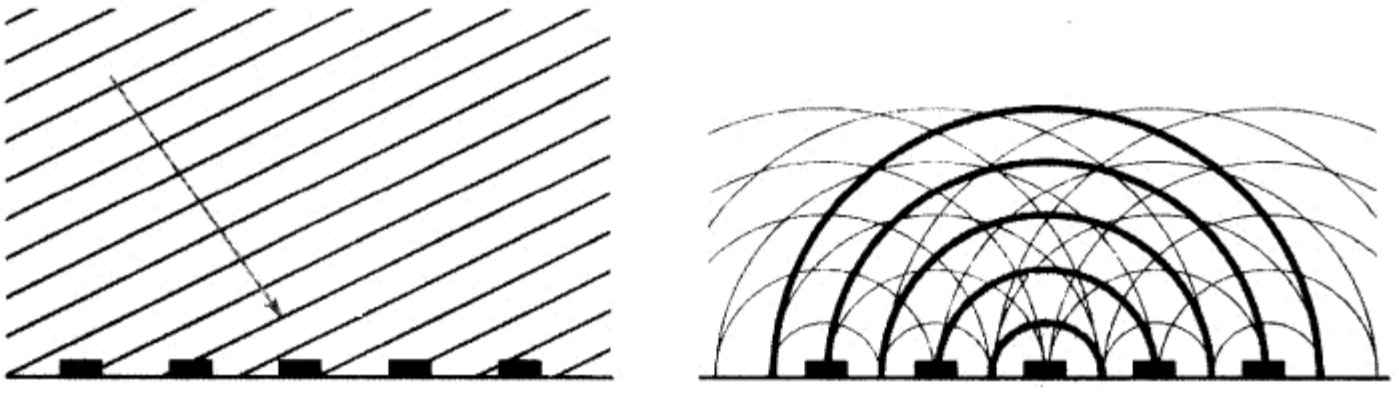
图 8-1 从紫外线到红外光的波长范围

发射的波的分布所决定。例如一个微红的光源主要由波长为 1μm 的光组成，而太阳光由振幅相同的各波长的光组成。

8.1.2　衍射的物理学

衍射 shader 对从衍射光栅表面反射的光建模。衍射光栅由一组平行的、间距为 d 的、狭窄的反光条带组成，图 8-2（a）表示这种表面的横断面。

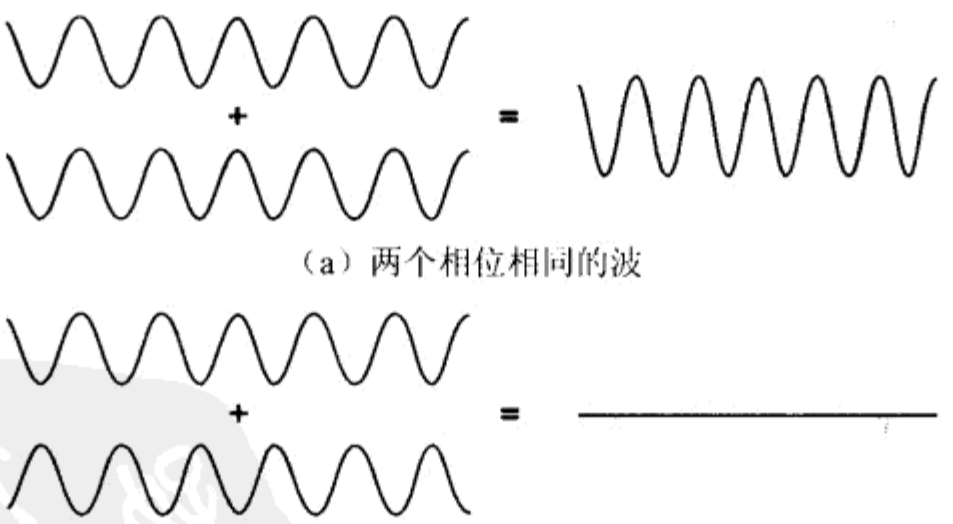
从一个光源发射的光波通常近似于平面波。这种波的横截面由许多沿着波峰所画的线表示。与简单的一维波不同，平面波除了波长和振幅之外，还需要指定方向。图 8-2（a）描绘的是平面波入射到衍射光栅上。线之间的间距对应于波长 λ。当平面波射到衍射光栅上的时候，它在每个带上产生一个球面波，如图 8-2（b）所示。球面波的波长与入射的平面波相同，波峰也同样表示。惟一的差别是波峰表示为同心圆而不是平行线。离开表面的任何接收点上的反射波，等于那个位置球面波的总和。



（a）一个平面波射到衍射光栅上　　（b）产生的球面波

图 8-2　一个衍射光栅

波动说与几何光学之间的主要差别，在于振幅不能简单地加合。波要干涉。我们用图 8-3 说明这个现象，在这里表示了两种极端的情况。在第一种情况（a）下，两个波“相位相同”，于是振幅相加。在第二种情况（b）下，波彼此抵消，产生的波振幅为 0。这两种情况说明了波的干涉可以是正的和负的。一般而言，干涉的结果处于这两种极端情况之间。然而，第一种情况是我们最感兴趣的：同相位的波干涉时，它们产生的强度最大。



（a）两个相位相同的波

（b）彼此抵消的两个波

图 8-3　波的干涉

现在来更详细地说明相位相同的波的性质。在这种情况下，较之表面细节的尺寸，我们假设光源和观察点离反射表面非常远。因为表面的细节是几个 μm 宽，而我们观察的距离是 1m 远。它们之间相差 6 个数量级。在这种情况下，我们假设所有发自表面、而到达接收点的波是彼此平行的。若要从不同条带发出、最后到达接收点的波相位相同，它们从光源到接收点的光程差只能是波长 λ 的整数倍。θ_1 为平面波的入射角度，而 θ_2 是对接收点的角度。

然后，如图 8-4 所示，使用一些基本三角方法，我们发现当下式成立时，波的相位相同：

$$|u|d = n\lambda$$

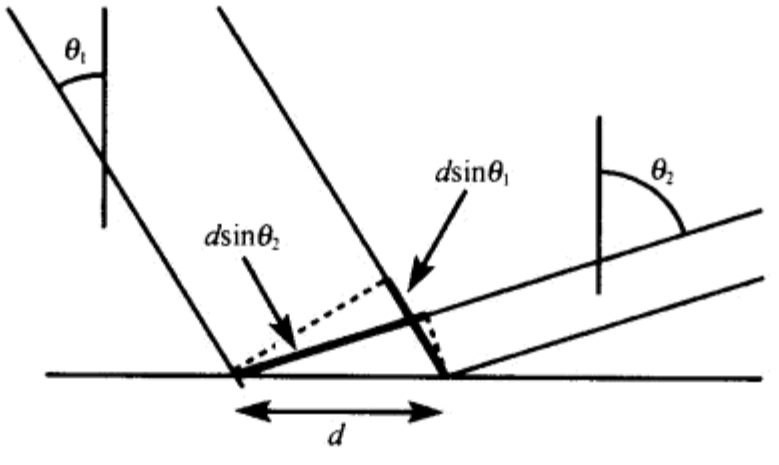


图 8-4 用于计算反射波之间相位差的角度

这里 n 是一个任意正整数，而 $u = \sin\theta_1 - \sin\theta_2$。这个关系给出相互干涉、在接收点得到最大强度的波长。这些波长是：

$$\lambda_n = \frac{|u|d}{n}$$

由于 n 是任意的，波长的数目也就是无穷的。然而，可见光的波长范围是[0.5, 1] μm。因此根据 $|u|$ 和间距 d，给出可能的 n 值范围：

$$|u|d < n < 2|u|d$$

结果，在接收点的光的颜色，是所有上式给出波长的光的颜色总和。

什么是对应于给定波长的颜色？颜色按从红到紫的彩虹形式排列。虽然准确的颜色能从理论上由一个特定的波长来确定，我们还是使用一个简单的近似。我们需要的是从紫到红排列的模拟的彩虹的映射。设 $C(\lambda) = (R(\lambda), G(\lambda), B(\lambda))$ 是这样一个映射，在范围[0.5, 1] μm 中的每个波长 λ 返回一个 RGB 值。到达观察点的光的颜色，通过加和有效 n 的颜色 $C(|u|\ d/n)$ 求出。当 $u = 0$（完全反射）时，所有波长的光都反射。我们将在 shader 的实现中单独处理这个情况。

8.2 实现

我们现在用 Cg 的顶点程序实现上面的理论。也可以使用片元程序来实现。实现可以在任何网格上进行，只要提供一个“切线向量”，和每个顶点的法线及位置。切线向量提供表面上窄条带的局部方向。对于一个光盘，它们是轨道的方向，如图 8-5 所示。

顶点程序的全部实现如程序列表 8-1 所示。

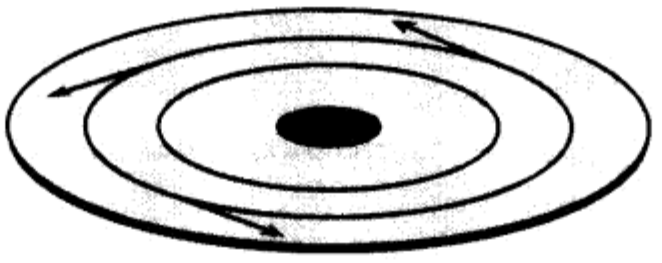


图 8-5　光盘的切线向量

列表 8-1　衍射 Shader 的顶点程序

```
float3 blend3 (float3 x)
{
  float3 y = 1 - x * x;
  y = max(y, float3(0, 0, 0));
  return (y);
}

void vp_Diffraction (
  in float4 position   : POSITION,
  in float3 normal     : NORMAL,
  in float3 tangent    : TEXCOORD0,
  out float4 position0 : POSITION,
  out float4 color0    : COLOR,
  uniform float4x4 ModelViewProjectionMatrix,
  uniform float4x4 ModelViewMatrix,
  uniform float4x4 ModelViewMatrixIT,
  uniform float r,
  uniform float d,
  uniform float4 hiliteColor,
  uniform float3 lightPosition,
  uniform float3 eyePosition
)
{
  float3 P = mul(ModelViewMatrix, position).xyz;
  float3 L = normalize(lightPosition - P);
  float3 V = normalize(eyePosition - P);
  float3 H = L + V;
  float3 N = mul((float3x3)ModelViewMatrixIT, normal);
  float3 T = mul((float3x3)ModelViewMatrixIT, tangent);
  float u = dot(T, H) * d;
  float w = dot(N, H);
  float e = r * u / w;
  float c = exp(-e * e);
  float4 anis = hiliteColor * float4(c.x, c.y, c.z, 1);

  if (u < 0) u = -u;

  float4 cdiff = float4(0, 0, 0, 1);
  for (int n = 1; n < 8; n++)
  {
  float y = 2 * u / n - 1;
```

```
    cdiff.xyz += blend3(float3(4 * (y - 0.75), 4 * (y - 0.5), 4 * (y - 0.25)));
    }

    position0 = mul(ModelViewProjectionMatrix, position);

    color0 = cdiff + anis;
}
```

代码计算了彩色的衍射图案，以及对应于 $u=0$ 的各向异性的高光。

首先介绍衍射图案的计算。取光源到接收点（未归一化）之间的中间向量，把它投影到局部空间的切线向量上来计算 u 值。根据这个数值和间距 d，计算相位相同的干涉波长。为了正确地计算波长，应该首先决定 n 值的有效范围，然后求相应颜色的总和。然而，目前的 Cg 编译器使用 for 循环求它的解，循环的大小受到顶点程序的限制。在实现中给定一个确定数作为 n 的允许值。这里用的数值是 8。在 shader 的以后版本中，还可以根据 $|u|$ 和 d 决定循环变量，如 8.1.2 节所解释的。

为了确定对应于给定波长的颜色，我们使用简单近似的彩虹贴图。贴图从紫到红排列，而且提供彩虹的大部分颜色。用 3 个理想凹凸函数（峰值分别在蓝、绿和红的区域）简单混合而成，如图 8-6 所示。

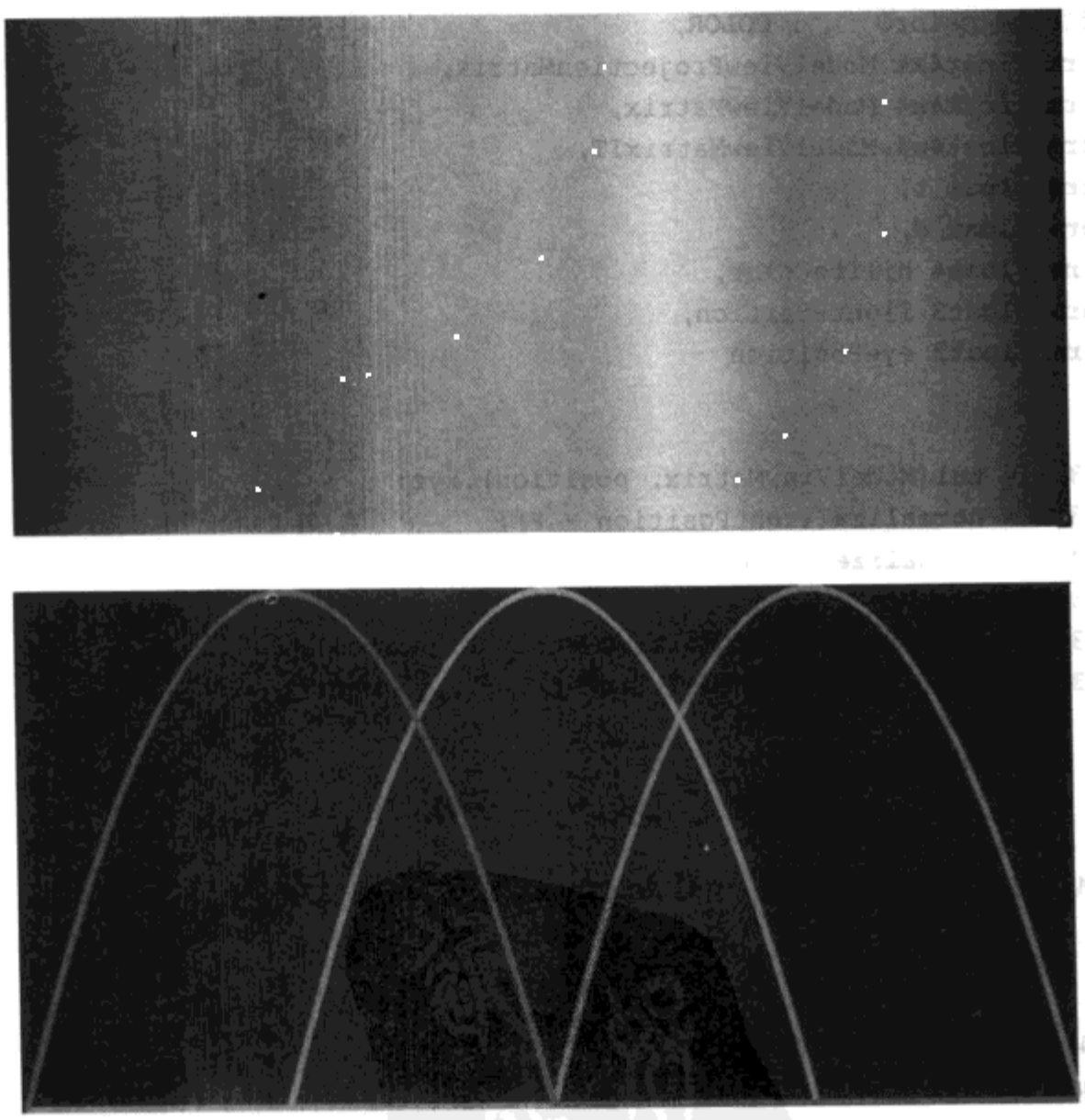


图 8-6 用于 shader 的彩虹彩色贴图

更精确地，我们的凹凸函数等于：

$$bump(x) = |x| > 1 ? 0 : 1 - x \times x$$

用这个函数，我们定义彩虹贴图 RGB 分量为：

$$R(\lambda) = bump\big(C \times (y - 0.75)\big)$$

$$G(\lambda) = bump\big(C \times (y - 0.75)\big)$$

$$B(\lambda) = bump\big(C \times (y - 0.75)\big)$$

这里 $y = 2\lambda - 1$ 把波长映射到[0, 1] μm 范围，C 是一个控制彩虹形状的参量。$C = 4.0$ 时工作得很好。这只是可能得到彩虹贴图的一种途径。事实上，使用一维的纹理贴图或一个查询表可能更好。

使用简单的各向异性的 shader 处理 $u = 0$ 的情况。理论上这应该对应于一个无限细的白色强光。然而，在衍射光栅（例如，在一个光盘上）每个条带上的不规则凹凸导致了强光的可见性展开。因此，我们用 Greg Ward（Ward 1992）的一个简单各向异性的 shader 对这个现象建模。使用粗略参量 r 和 highliteColo 参量给出的颜色，对强光的展开建模。结果的表达式是：

$$anis = hiliteColor \times \exp\big(-(ru / w)^2\big)$$

这里 w 是中间向量在法线方向上的分量。

最后的颜色是彩色的衍射图案和各向异性高光的简单相加的和。

8.3　结果

我们使用衍射顶点 shader 写一个程序，用来看光盘的反射。对光盘建模使用了一组精细的方块。这里的 shader 不限于光盘的几何形状：只有对每个顶点提供切线方向是必要条件。

此外，用一个类似 Fresnel 的反射 shader，在光盘上添加一个薄的透明层。这一层能使斜射角更强烈地映射环境。图 8-7 表现实时演示的 3 个快照及 3 个不同位置的 CD。图 8-8 表现衍射 shader 用于纹理映射了各向异性的主要方向的 1 个表面。

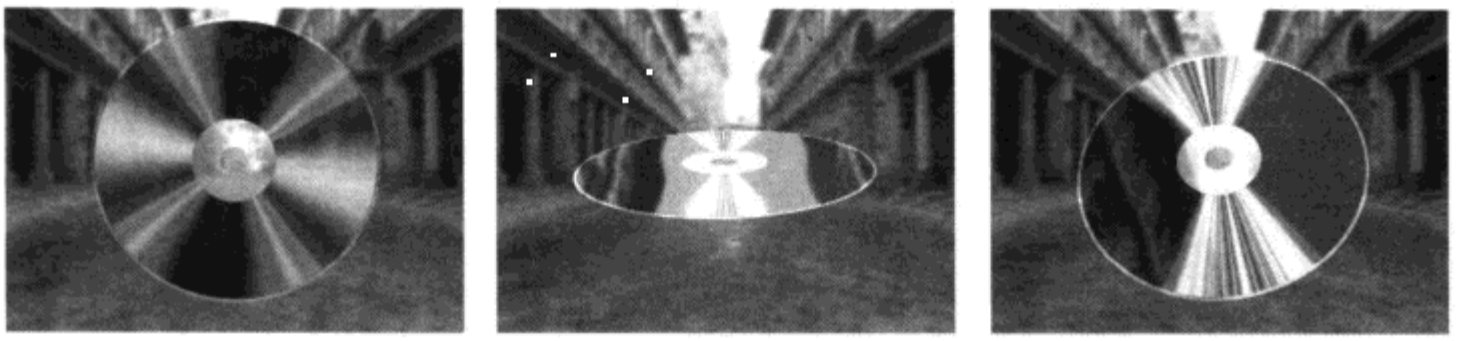


图 8-7　光盘实时演示的 3 个快照

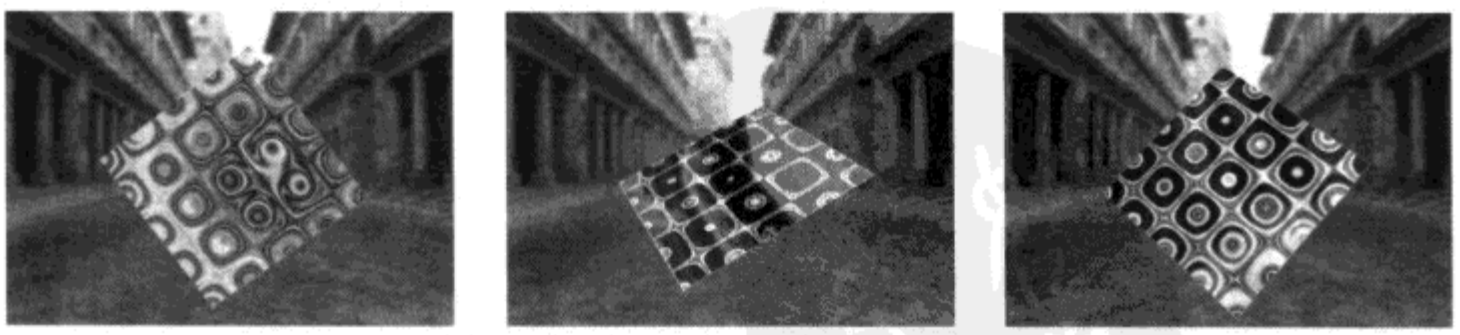


图 8-8　用纹理映射各向异性主要方向的表面的 3 个快照

8.4 小结

我们已经介绍了如何实现一个简单的衍射 shader，它表现自然光的波动性。更复杂的表面细节的推导，可参考 SIGGRAPH 1999 论文（Stam 1999）。对波动光学更感兴趣的读者可以参考经典的《光学原理》一书（Born and Wolf 1999），我们发现很有用。我们将来会研究更复杂的衍射光栅表面。举例来说，会试图对几个散射层组成的金属油漆的反射进行建模。在这些油漆中的细小色料引起许多可见波动效果。开发这样一个模型，在新油漆的制造中会有许多应用。

8.5 参考文献

Born, Max, and Emil Wolf. 1999. *Principles of Optics: Electromagnetic Theory of Propagation, Interference and Diffraction to Light*, 7th ed. Cambridge University Press.

Stam, Jos, 1999. "Diffraction Shaders." In *Proceedings of SIGGRAPH 99*, pp. 101～110.

Ward, Greg. 1992. "Measuring and Modeling Anisotropic Reflection." In *Proceedings of SIGGRAPH 92*, pp. 265～272.

第 2 部分

光照和阴影

简 介

无论在表面 shader 中（它决定如何结合表面和场景参量来产生颜色），还是有效组织场景物体的算法中，对光照和阴影算法的选择，是影响渲染引擎设计的最重要的因素。一个算法的选择不仅影响场景看起来怎么样，它还影响到内容创作的方式，以及场景的复杂度和互动方式。这篇中的章节介绍了几种光照和阴影的算法，以及使其更有效和更稳固的技术。

在**第 9 章“有效的阴影体渲染”**中。**Morgan McGuire** 全面地讲述了常见的模板阴影体技术，用于渲染实时阴影。初看模板阴影体很容易实现，但是实际上执行起来效率低、速度慢。这章重点是要得到正确的角度情况，减少几何图形和填充率的消耗。

在**第 10 章“电影级的光照”**中，**Fabio Pellacini** 和 **Kiril Vidimce** 提出了一个一般的光照 shader，它基于 Pixar 动画工作室所用的 shader，但是对实时光照进行了简化。写这个 uberlight shader 的基本目的，是对美工光照的场景提供更多光照参量的控制。

当今最流行的通用实时光照算法是阴影贴图。阴影贴图的主要问题是走样。在**第 11 章“阴影贴图反走样”**中，**Mike Bunnell** 和 **Fabio Pellacini** 介绍如何通过靠近百分比过滤法有效减少阴影贴图的走样。

在**第 12 章的“全方位的阴影贴图”**中，**Philipp S. Gerasimov** 把阴影贴图的思路扩展到正确处理全方位的（点）光源。其中包括实现细节，也涉及到基本硬件能力不足时的低效运行。

实时游戏中大多数阴影的边缘清晰而且走样，这是由于把光源近似为无面积的点而造成的。在现实世界中，所有的光源面积都不会是 0，因此所有的阴影都不同程度地模糊。在**第 13 章“使用遮挡区间映射产生模糊的阴影”**中，Will Donnelly 和 Joe Demers 介绍了一种新技术，能够正确渲染在静态场景中，光源沿着预定路径移动时产生的模糊阴影。这个技术用于 NVIDIA GeForce FX 5900 的上市演示，Last Chance Gas。

在**第 14 章“透视阴影贴图”**，**Simon Kozlov** 继续介绍反走样。他介绍一种优化透视阴影贴图的新思路，是在 SIGGRAPH 2002 被 Stamminger 和 Drettakis 介绍的一种新型阴影贴图。透视阴影贴图为了减少或消除阴影贴图的失真走样，对投射到大像素区域的物体，取最大的阴影贴图纹素密度。

最后，在**第 15 章“逐像素光照的可见性管理”**中，**John O'Rorke** 观

察到，增加视觉复杂性的技术，也会增加送到硬件的批数；如果想使现代 GPU 达到最好的性能，就必须将批数减到最小。这章使用若干可见度技术寻找最佳批组，使性能得到很大提高。这个技术还附带减少了 CPU 和 GPU 的负担。

Cem Cebenoyan, NVIDIA

第 9 章 有效的阴影体渲染

Margan McGuire
Brown university

9.1 引言

保安的影子映到屋顶，使小偷提前警觉而藏到天花板上。外星怪物贴着墙，隐藏在碉堡的阴影里，准备猛扑向一个麻痹大意的航空兵。骑士的火把和德鲁伊的法杖照亮坟墓里面的雕像，大理石古代神像的黄色和红色影子在墙上闪动。这些仅仅是一些例子，生动地说明实时阴影在今天游戏中的使用。

新的 3D 游戏需要实时阴影。逼真的光照带来的感性电影效果，已经为玩家所熟悉。与其他效果不同，阴影不是渲染的物体。相反，它们是比其他部分更暗的屏幕区域，原因是计算光照强度时，那里接收的光比较少。在渲染引擎中添加阴影的最大困难，是实时地发现那些阴影区域。这章描述如何使用“阴影体”方法（shadow volumes），即在模板（stencil）缓冲区标记阴影的像素；id 软件的游戏 *Doom3* 就是用这种方法制作阴影的，见图 9-1。简单地把像素分为阴影的或照明的，再调节负责光照的像素程序，使阴影像素的照明度贡献为 0。

图 9-1 来自 id 软件的 *Doom3* 的一个场景
Doom3 游戏图像版权归 id 软件所有，由 Activision 出版

阴影体技术为点光源、聚光灯和方向光源创建清晰的、逐像素精度的阴影。单个物体可以被多个光源照亮，而且光有任意颜色和衰减度。阴影从三角形网格投射到深度缓冲区上。这意味着被遮挡的物体，可以是带有深度缓冲区的网格、布告板、粒子系统或预先渲染的场景。

较之其他运算，阴影体能够更好地处理许多制作困难的阴影场景。图 9-2 就显示了一个这样棘手的场景。惟一的光源是插在万圣节灯笼里的点光源。除了透过南瓜的窟窿发出的光在地上照亮的三角形区域以外，整个场景处在阴影中。这种情况难以处理有以下几个原因：通常的情况是大部分场景被照亮，而阴影很小；现在的情况则相反，这种整个都是阴影的场景是很少制作的。被照亮的区域比产生它们的南瓜窟窿要大得多。虽然光线只从前面和底部射出，但光是全方位的，所以必须从所有的角度考虑阴影。最后，投射的阴影不是仅仅靠近光源，它包围着光源。

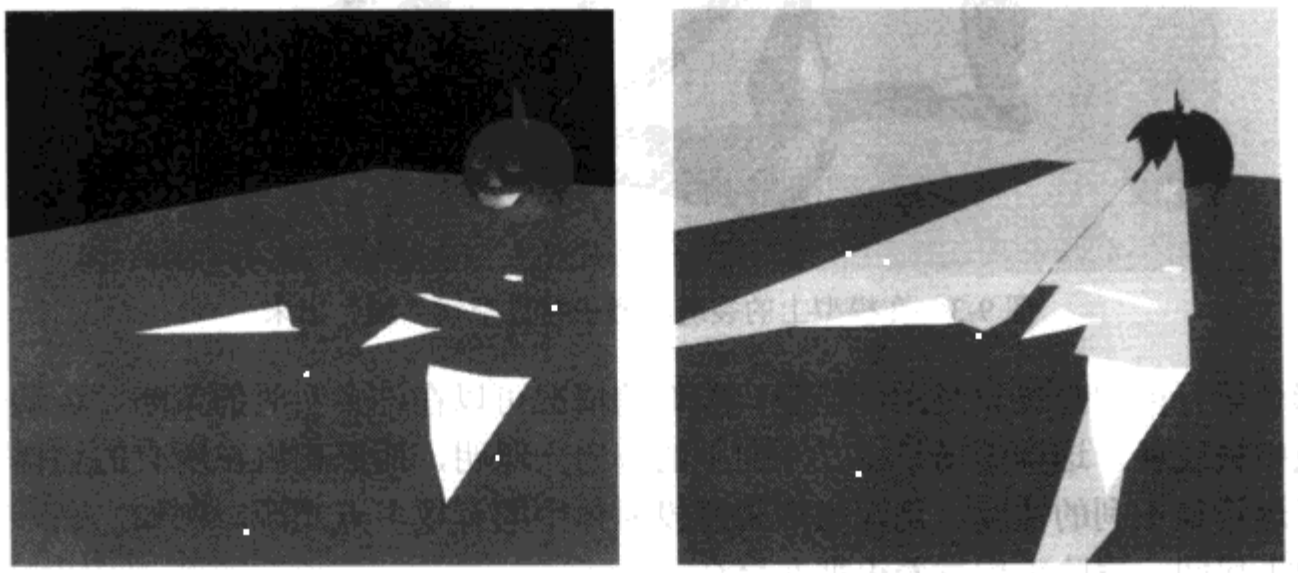


图 9-2 制作阴影困难的场景：一个插在万圣节灯笼里的光源
右：用于计算图像的阴影体

阴影体并不是对所有的场景都理想。这个技术涉及到构造一个 3D 体积，由一个物体投射的所有阴影都包围在其中。这个体积由阴影投射体的网格构成；但是有些阴影投射体，没有能够正确表达它们形状的网格。例如布告板、粒子系统，或一个有 alpha 粗糙度的纹理的网格（比如一片树叶）。这些投射体基于它们的真实网格产生阴影，阴影与物体的真实形状并不匹配。举例来说，布告板烟云就投射一个矩形的阴影。

另一个问题是，有时物体是含有边的网格但只有一个相连的面，通常称作一个裂缝。在现实世界中，如果观察裂缝，会看到物体的内部。但是在渲染引擎中，则会透过物体看到它的另一面，因为在渲染时内部与被剔除的背面多边形排成一条线。这个物体不是一个阴影投射体。结果从某些角度看，它投射一个实体的阴影；从其他的角度，光穿过洞照到另一面。更糟的是，使用这些物体时候，阴影体的优化被破坏，产生一种大的黑条带悬挂在空间，如图 9-3 所示。

这种方法另外的可能限制是，它要求场景中每样东西都投射影子。当角色的影子投到墙上的时候，它也要投到墙后面的每件东西上。之所以没有在墙的另一面看到角色的影子，是因为墙也投射它自己的影子，并与角色的影子重叠。如果只是从角色投射影子，而不从场景几何体投射，那么角色影子就要透过实体。

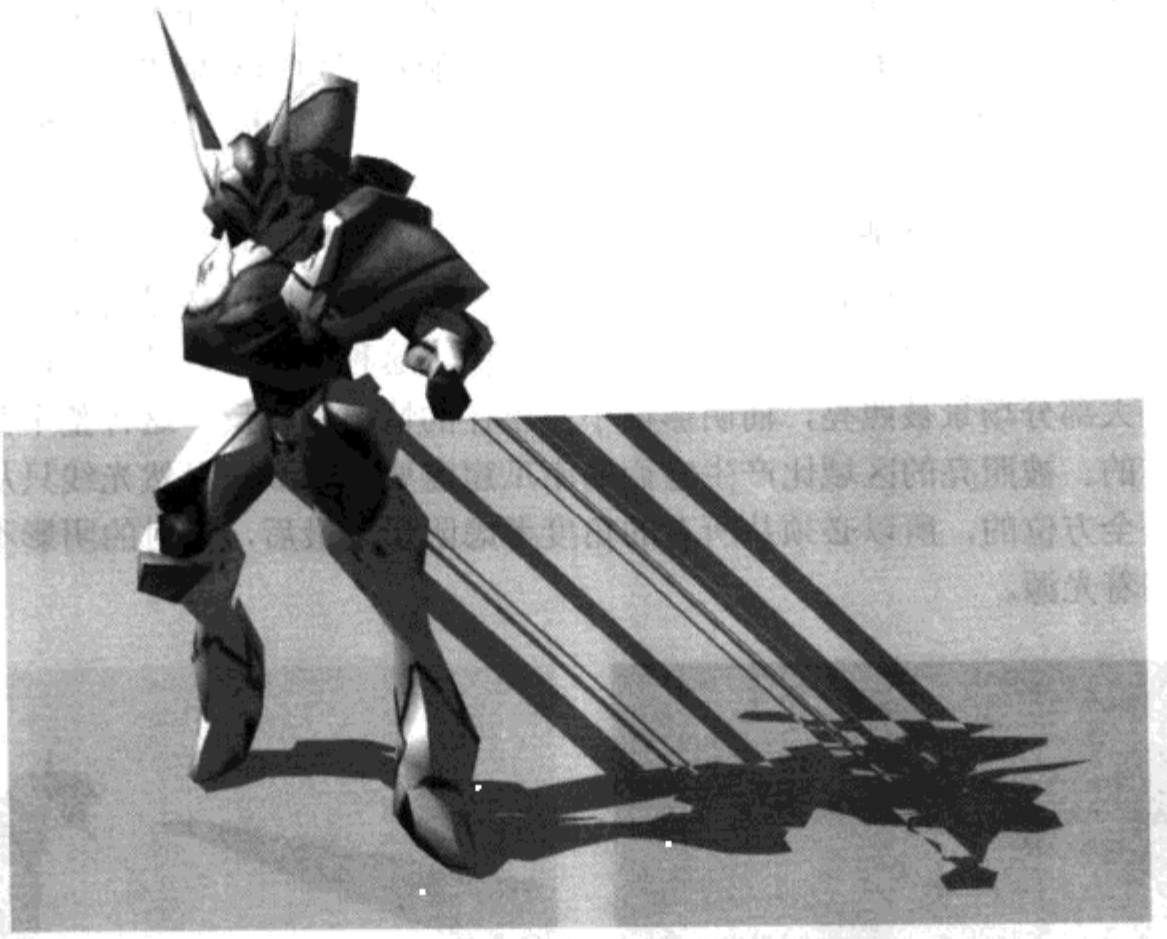


图 9-3 在模型上的裂缝让影子穿过空气“漏”出来

阴影体运行的理想场景是顶部俯视，类似的情况可以在许多实时的策略、运动和地上的冒险游戏中被发现。这样的场景由几个向下的方向光照明，而照相机在所有的物体上面俯瞰大地。对运行最不利的情况，是在很多阴影投射体中间有很多点光源。例如，在一个有许多柱子的地下房间，冒险家们举着火把在聚会。

9.2 程序结构

阴影体技术由两个部分组成：从轮廓边缘构成体，并把它们渲染进模板缓冲区内。对每个光源重复这两部分，并把产生的图像加在一起，创建一个最终的帧（称为多遍渲染的过程）。基本的算法是容易理解和实现的，但是对于大的场景来说太慢。为了实现它，进行了一系列的优化，以减少几何体的处理和对填充率的要求。

我们首先总览整个程序结构，然后对每个步骤的细节进行讨论，接着看一下优化方面的介绍。最后，我们考查阴影体上的一些研究项目，前瞻一下它的未来。

9.2.1 多遍渲染

从数学上说，一个点上的照明是几项的总和。我们在单个光源的 Phong 照明公式中看到环境、发射（内在发光）、漫反射和高光反射成分的总和。多个光源的场景，有一个环境项和一个发射项，但是它对每个光源都有一个漫反射项和一个高光反射项。当没有阴影渲染时，多个光源可以全部渲染在一个遍中。典型做法是启用多个硬件光源，或用每个光源的代码实现一个像素 shader。

当有阴影渲染时，由于有些点被遮挡，所以某个光源在这些点上的光强度是零。为了考

虑这一点，要对每个光源的漫反射和高光反射强度，在独立的渲染遍中计算。最终的图像是几个遍的总和，包括原先计算环境和发射强度的遍，以及各个照明遍。因为最初的遍把深度值写进了 z 缓冲，附加的遍就不必再浪费时间，实际上节省了填充率。在附加遍中渲染的物体也可能由遮挡剔除消去。

虽然阴影体不创建由面光源投射的模糊阴影，但是可以利用多遍创建类似效果，在面光源表面分布多个暗淡的聚光灯。但是对于复杂场景，虽然多个光源使它好看，但是实际应用起来太慢（2003 Assarsson 等人描述的新的研究技术，提出了用阴影体渲染模糊阴影的更有效方法）。

各个光照遍是用 alpha 混合结合起来的。为了做到这一点，用激活的深度写入把环境/发射的遍渲染到后台缓冲区去，把混合函数设为 glBlendFunc（GL_ONE，GL_ZERO）。这初始化了深度缓冲区并创建基础的照度。

然后对于光源遍，禁用深度写入并将混合函数换成 glBlendFunc（GL_ONE，GL_ONE）。这个混合模式将新的渲染像素加到原有的像素上。预先初始化的深度缓冲区避免了浪费。也就是说，保证将深度测试设定为 glDepthFunc（GL_LEQUAL），避免在后来的遍之间的 z 冲突。

利用这些设定，为每个光源制作 1 个遍。每个遍对模板缓冲区清 0，标记里面的遮挡区域，然后计算非遮挡区域中的光照强度，并把它们加入帧缓冲区。

程序中渲染部分的全部结构在图 9-4 中显示。这个过程的简化版本显示在列表 9-1 中。简化是“标记阴影”步骤被化简为最坏的情况，那时框图中的每个条件判断都返回真。在详细地研究过代码之后，我们将把较短路径放回原处进行优化。把优化中改变的代码区段加以标注，以便将来容易查找。

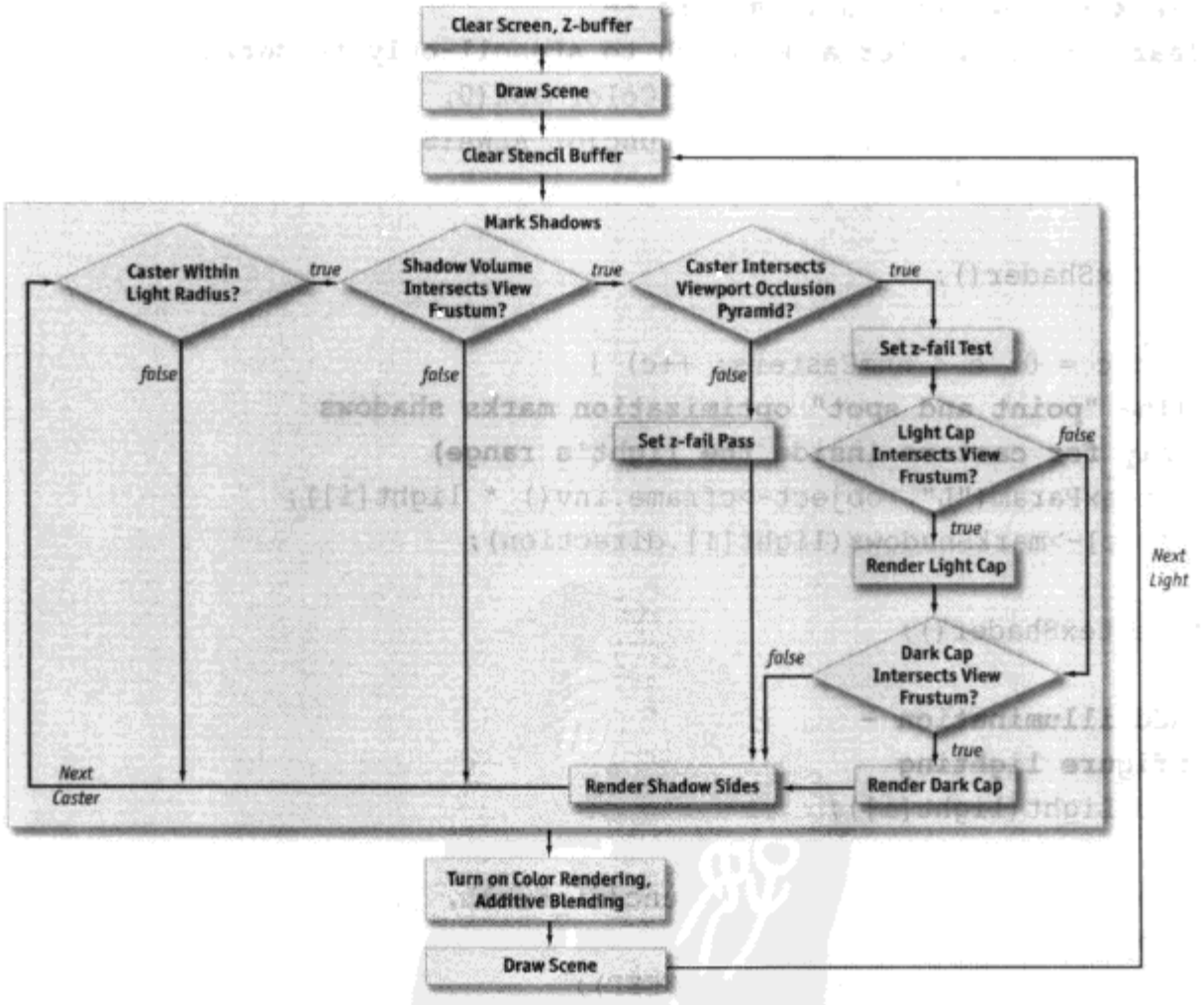


图 9-4　程序结构图

列表 9-1 程序结构伪代码

```
static const float black[] = {0.0f, 0.0f, 0.0f, 0.0f};

glPushAttrib(GL_ALL_ATTRIB_BITS);
setupCamera();

// -- Ambient + emissive pass --
// Clear depth and color buffers
glClear(GL_DEPTH_BUFFER_BIT | GL_COLOR_BUFFER_BIT);

glBlendFunc(GL_ONE, GL_ZERO); glEnable(GL_BLEND_FUNC);
glDepthMask(0xFF); glDepthFunc(GL_LEQUAL);
glEnable(GL_LIGHTING); glDisable(GL_LIGHT0);
glLightModelfv(LIGHT_MODEL_AMBIENT, globalAmbient);
drawScene();

// Light passes
glLightModelfv(LIGHT_MODEL_AMBIENT, black);
glEnable(GL_LIGHT0); glBlend(GL_ONE, GL_ZERO);
glDepthMask(0x00); glEnable(GL_LIGHT0); glEnable(GL_STENCIL_TEST);
glEnable(GL_STENCIL_TEST_TWO_SIDE_EXT);

for (int i = numLights - 1; i >= 0; --i) {
  // (The "XY" clipping optimizations set the scissor
  // region here.)

  //-- Mark shadows from all casters --
  // Clear stencil buffer and switch to stencil-only rendering
  glClear(GL_STENCIL_BUFFER_BIT); glColorMask(0, 0, 0, 0);
  glDisable(GL_LIGHTING); glStencilFunc(GL_ALWAYS, 0, ~0);
  glStencilMask(~0);

  loadVertexShader();

  for (int c = 0; c < numCasters; ++c) {
    // (The "point and spot" optimization marks shadows
    // only for casters inside the light's range)
    setVertexParam("L", object->cframe.inv() * light[i]);
    object[c]->markShadows(light[i].direction);
  }
  unloadVertexShader();

  //-- Add illumination -
  // Configure lighting
  configureLight(light[i]);

  glEnable(GL_LIGHTING); glStencilFunc(GL_EQUAL, 0, ~0);
  glActiveStencilFaceEXT(GL_FRONT);
  glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
  glActiveStencilFaceEXT(GL_BACK);
```

```
    glStencilOp(GL_KEEP, GL_KEEP, GL_KEEP);
    glDepthFunc(GL_EQUAL); glColorMask(1, 1, 1, 1);
    glCullFace(GL_BACK);

    // (The "point and spot" optimization adds illumination
    // only for objects inside the light's range)
    drawScene();
  }

  glPopAttrib();
```

9.2.2　顶点缓冲器结构

网格的阴影由它的轮廓投射。为了很快地找到轮廓边缘，把它们放进一个阴影体内，网格需要更多的信息，像传统渲染结构那样只使用面三角形是不够的。

在我们的系统中，网格的顶点缓冲区包含每个顶点的两个拷贝。若有 n 个原始的顶点。顶点缓冲区中的 0 到 $n-1$ 的元素包含典型的顶点（$x, y, z, 1$），是第 1 组；n 到 $2n-1$ 的元素是第 1 组的拷贝，形式为（$x, y, z, 0$）。第 1 个组用于普通渲染。两组都用于确定阴影，顶点 shader 将把第 2 组转换到无限远。

物体也必须有邻接信息和每个面的法线。对于每个面，我们需要知道 3 个反时针方向的顶点索引和表面法线。对于每条边，我们需要 2 个毗邻面的索引和 2 个顶点的索引。如前所述，模型一定要封闭没有裂缝。根据邻接信息，每条边有 2 个含有相同顶点的毗邻的面，但是顶点次序相反。我们规定，边的顶点按顺时针排列的面为第 1 个面，而顶点次序相反的面是第 2 个面。有的模型上的顶点可能不在任何边或面上。因为在创建 3D 模型时，为了用不同的纹理坐标安排顶点，经常这样做。对于邻接信息，我们只关心原始的几何图形，而忽略纹理坐标、法线和顶点颜色等，这些信息是为了渲染目的而与模型一起存储的。

9.2.3　在无限远处工作

不像其他的 OpenGL 程序，这里阴影体把坐标拉伸到无限远。阴影体本身由有限的几何体和无限远的几何体两者构成。这个算法是实现点光源的，方向光源可作为无限远的点光源来处理。远裁剪平面必定在无限远处，所以无穷远的阴影体不会被切断，构成透视的投影时必须考虑到这点。

OpenGL 完全支持使用齐次坐标系在无限远处工作。这节复习一下齐次顶点（对于几何体和光源）和介绍如何构造一个无限远透视的矩阵。

有限的齐次点表示为（$x, y, z, 1$），w 成分等于 1。这意味着点的 3D 位置为（$x/1, y/1, z/1$）。透视的投影矩阵使用无单位的 w 成分做除法，在除法之后，顶点（$x, y, z, -z$）变成（$x/-z, y/-z, -1$）。当 w 成分是 0 时，我们得到形式为（$x/0, y/0, z/0$）的点。这点“在无限远”处。当然，如果我们真的用 0 除以每个成分来计算点的位置，它会变成（∞，∞，∞），就丢掉了很重要的信息，即点走向无限远的方向。（$x, y, z, 0$）意味着，使用 $w=0$ 来标记点（x, y, z）处于“无限远”，但仍然保留方向信息。

直观来看，一个无限远的点的行为就如同它在非常远处一样，与场景的物理维数无关。正如夜晚天空的星星，是固定地保持在无限远的点，当观察者的位置变化时，它们跟着观察

者的方位运动。OpenGL 用 $w=0$ 正确地渲染点。像星星一样，它们好像在一个以观察者为球心的“无限大”的球面上渲染。注意，对于在无限远的点 (x, y, z)，重要的只是方向，各个成分的大小并不重要。因此 OpenGL 使用 $w=0$ 表示一个方向光，方向光是沿着特定方向移到无限远的点光源。

全章使用 $w=0$ 表示在无限远的点。不仅使用无限远的点光源，也将阴影体延伸到无限远处。在上面，我们以 $w=0$ 作为处于顶点缓冲第 2 部分的标记。这是因为那些顶点会被移到无限远（它们是阴影体的无穷远端点）。然而，在它们实际上被转换到无限远之前，顶点 shader 将会相对于光源移动它们。

当渲染所有这些在无限远的物体时，不可能用远剪裁面剪裁它们。因此，需要把远剪裁面移到无限远，可以通过计算标准投射矩阵的界限做到这一点。

$$\lim_{Far\to\infty} \boldsymbol{P} = \boldsymbol{P}_{\inf} = \begin{bmatrix} \frac{2\times Near}{Right-Left} & 0 & \frac{Right+Left}{Right-Left} & 0 \\ 0 & \frac{2\times Near}{Top-Bottom} & \frac{Top+Bottom}{Top-Bottom} & 0 \\ 0 & 0 & -1 & -2\times Near \\ 0 & 0 & -1 & 0 \end{bmatrix}$$

在代码中，对计算透视的投射矩阵来说，这只是一个较小的变化。创建如列表 9-2 所示的投射矩阵以代替使用 glFrustum。

列表 9-2 一个 glFrustum 风格的无限远投射矩阵

```
void perspectiveProjectionMatrix(double left,
                                 double right,
                                 double bottom,
                                 double top,
                                 double nearval,
                                 double farval)
{
  double x, y, a, b, c, d;
  x = (2.0 * nearval) / (right - left);
  y = (2.0 * nearval) / (top - bottom);
  a = (right + left) / (right - left);
  b = (top + bottom) / (top - bottom);

  if ((float)farval >= (float)inf) {
    // Infinite view frustum
    c = -1.0;
    d = -2.0 * nearval;
  } else {
     c = -(farval + nearval) / (farval - nearval);
     d = -(2.0 * farval * nearval) / (farval - nearval);
    }

  double m[] = {x, 0, 0, 0,
                0, y, 0, 0,
                a, b, c, -1,
```

```
                0, 0, d, 0};

  glLoadMatrixd(m);
}
```

列表 9-3 中，Cg 顶点 shader 用 $w = 1$ 正常地变换点，用 $w = 0$ 把点发送到离光源无限远的地方。

列表 9-3　为把 $w = 0$ 的顶点搬到远离光源的一个顶点 shader

```
VOut main(const float4x4 uniform in MVP,
          const float4 uniform in L,
          const VIn in vin)
{
  VOut vout;
  // (The "directional" optimization eliminates the vertex shader
  // by using different rendering loops for point and directional
  // lights.)
  vout.pos = MVP * (vin.pos.w == 0 ? float4(vin.pos.xyz * L.w - L.xyz, 0) : vin.posvin.pos);

  return vout;
}
```

由于旧的图形卡不支持顶点 shader 中的分支运算(?)，可以调用 *lerp* 函数代替分支运算。注意用在中线上的 **L**.*w* 乘以点的位置，使得那个点的位置与方向光无关。这是因为从光到点的矢量与方向光的点的位置无关。在列表 9-1 中，调用 setVertexParam 设定物体空间的光矢量。loadVertexProgram、unloadVertexProgram 和 setVertexParam 的实现取决于使用的顶点 shader 运行库。

9.3　详细的讨论

markShadows 的目的是设定模板缓冲，让照明的像素为 0，而阴影的像素为一个非 0 值。做法是构造一个包容阴影区的阴影体，并把它渲染入模板缓冲区内。这里我们看一下数学证明，然后详细地讨论实施。

9.3.1　数学

图 9-5 展示了一个简单的场景，有一个点光源（灯泡图标）、一个投射阴影的盒子、一个接收阴影的地平面和一个在左边的观察者。在观察者前面的线表示像平面，它对 9.5 节的讨论很重要。蓝色的箭头表示来自光源的光线（为了清楚起见，只显示了一些）。最左边的和最右边的光线照到的地面是明亮的。中间的光线射到阴影投射盒上并且被阻挡。在盒子下面的地面是暗的，因为光线被盒子阻挡。在图中外部和内部之间的光线是边界线，用虚线表示。这些虚线标记阴影的边缘。它们通过光源的中心和阴影投射体的边缘。图是二维的；在三维图中，线变成四边形。这些线是阴影体的边。比盒子更远的每点都在阴影中。其他的各点都被照明。

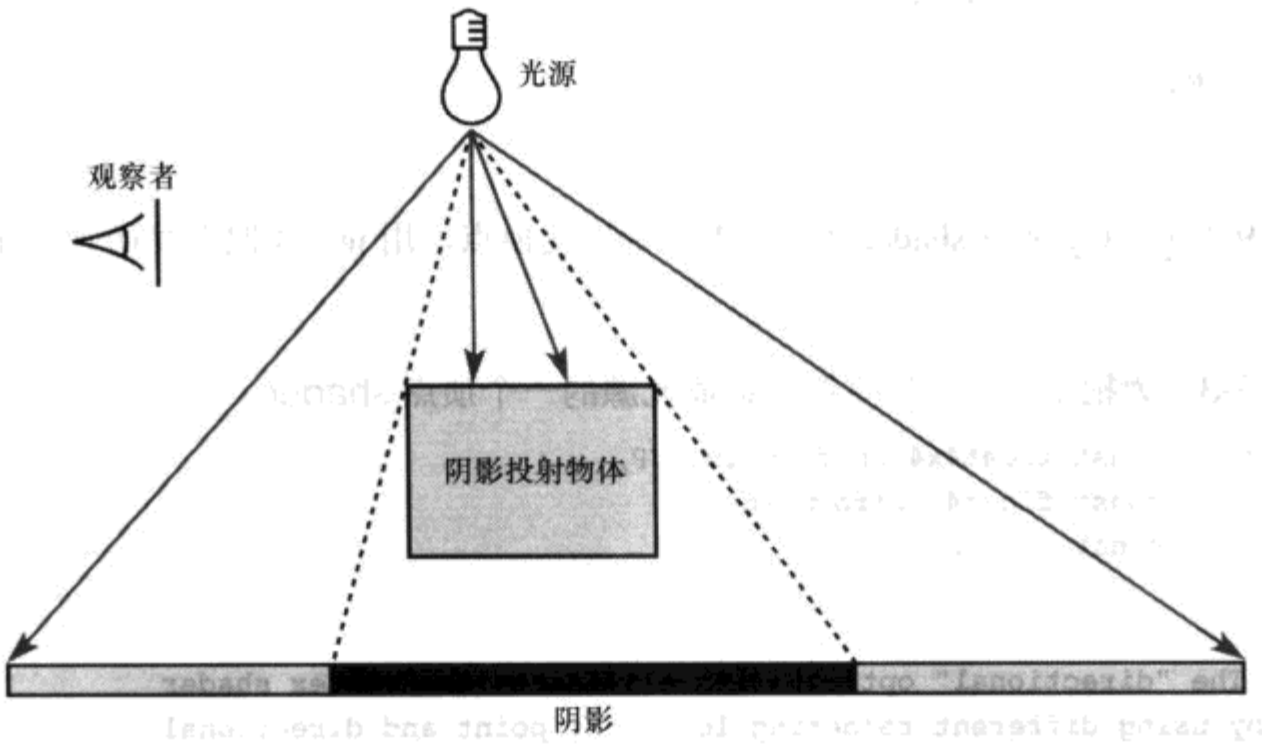


图 9-5 一个简单的场景

图 9-6 清楚地表示了阴影体。阴影体有 3 片。

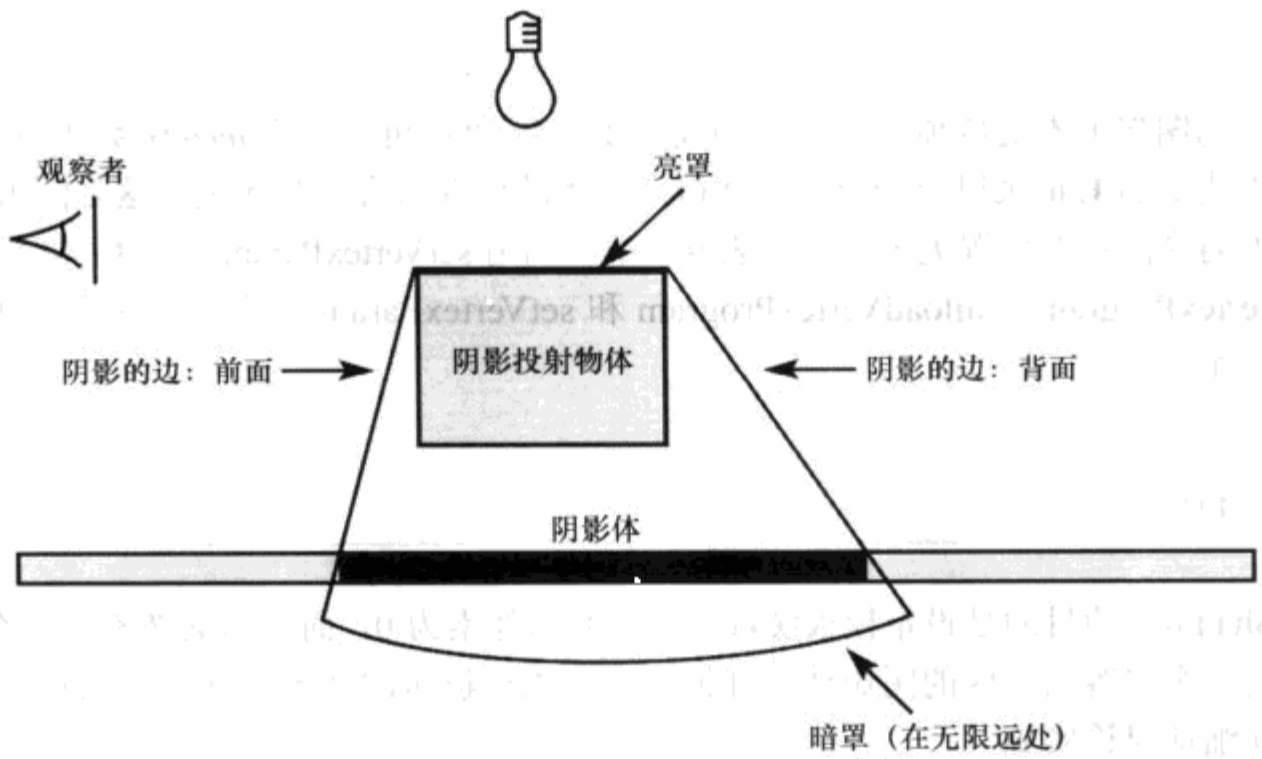


图 9-6 简单场景的阴影体

- 阴影的侧面由投射体的边延伸到无限远构成，形状是方块。在投射体和光源之间的物体不应该被遮挡，因此，必须在顶部封住阴影体。因为物体投射的阴影与它的轮廓相同，所以我们只需要延伸轮廓的边。从光的视角出发，一个面是背面（$\boldsymbol{N}\cdot\boldsymbol{L}<0$），而另一个面是前面（$\boldsymbol{N}\cdot\boldsymbol{L}>0$）。
- 亮罩是从近光源面封住阴影体的几何图形。它由对着光源的投影体的多边形组成（对光源而言是前面的多边形）。阴影延伸到离光源无限远，但是从数学上说，我们必须在后端封闭几何体使它成为一个体积。
- 暗罩由背对着光源的投影体的多边形扩展到无限远组成（即，对光源而言是背面的多

边形）。在图 9-6 中，它们被显示为一个曲线，因为在无限远的多边形，可以看作放在包围整个场景的非常大的球体上。

虽然示意图表示为二维的简单场景，但是应该记住阴影体在三维中，假如投射物体的形状复杂，阴影体的几何形状也可能很复杂。为了进行比较，真正三维的阴影体几何图形显示在图 9-2、9-7 和 9-10 中。如果有多个阴影投射体（经常如此），阴影体将会有许多独立的部分。这些部分可能重叠。但这不是问题；无须任何特别处理，算法对三角形或完全的场景同样很适用。

这里是使用阴影体进行阴影检测的数学方法。当渲染时，每个像素对应于 3D 空间中的一个点。如果点在阴影体内，把它的像素在模板缓冲中设定为非 0 值（遮挡）；否则，设它为 0（照亮）。称查询的点为 P。考虑阴影体和光线的相交，光线从 P 开始，沿着负视线矢量 $-\boldsymbol{V}$ 射向无限远。有两种相交的情况。一种是“进入”相交，发生于光线从外面进入阴影体之内时。设 $\boldsymbol{M}$ 是相交的阴影面的表面法线。在进入相交时，$\boldsymbol{M}\cdot\boldsymbol{V}>0$。一种是“离开”相交，发生于光线离开阴影体时，有 $\boldsymbol{M}\cdot\boldsymbol{V}<0$（$\boldsymbol{M}\cdot\boldsymbol{V}=0$ 时，忽略相交）。主要思路是计算每种相交发生的数目：

当一道光线射向无限远时，只有进入相交的数目超过离开相交的数目时，才能说点 P 在阴影内。

当光线沿着负视线矢量传播时，它完全位于在透视投射下的一个像素之内。我们利用这个事实，更快地在使用模板缓冲的硬件中进行相交计算。

9.3.2　代码

如何在硬件中有效地进行观测？如果图形卡支持，则设定模板缓冲初始值为 0，并且启用环绕（wrapping）增量和减量操作（如果图形卡不支持环绕，把所有的模板缓冲初始值设为 128 或其他值，以避免下溢）。禁用彩色渲染及把阴影体几何图形渲染到模板缓冲区。计算相交的光线是从每个可见点开始，向远离观察者的方向传播，在深度测试失败的时候，让硬件改变模板值。让每个背面像素的模板缓冲增加，而每个前面像素的模板缓冲减少。

在禁用颜色渲染后，马上渲染阴影体，而在环境照明通过之后，禁用刚才的深度写入。因为颜色和深度两个写入都禁用，阴影体的渲染只影响模板缓冲。必须禁用颜色写入，是因为我们只想在最后的图像中见到阴影（以模板计数为基础），不想见到阴影体。必须禁用深度写入，是因为我们假定在 z 缓冲中深度值代表可见面深度（而不是阴影体）。由于禁用深度写入，阴影面不互相干扰，因此不用考虑渲染它们的次序。

在渲染之后，如果所渲染的前面和后面的数目相同，在一个像素上的模板值将会是 0，如果计数有差别，模板值将是非 0。“进入”相交总是由前面产生，而“离开”相交总是由背面产生。因此在渲染后的模板计数，是“进入”相交数减“离开”相交数，为了阴影检测，需要精确的结果。

9.3.3　markShadows 方法

在对象类中的 markShadows 方法的代码显示在列表 9-4 中。

首先，我们把光矢量从世界空间转换到物体空间。对于点光源或聚光灯，这个矢量是位置 $(x, y, z, 1)$。对于方向光，形式为 $(x, y, z, 0)$，这里 (x, y, z) 是对光源的矢量。一般而言，$w=0$ 的一个齐次矢量，可以看作在无限远处一个球体上的点。因此方向光与无限远处的点光

源相同。

列表 9-4 markShadows 方法

```
// isBackface[f] = true if face f faces away from the light
std::vector<bool> backface;

void Object::markShadows(const Vector4& wsL)
{

  // (When the viewport is not shadowed by this object, this
  // section is changed by the "uncapping" optimization.)
  // Decrement on front faces; increment on back faces
  // (a.k.a. z-fail rendering)
  glActiveStencilFaceEXT(GL_FRONT);
  glStencilOp(GL_KEEP, GL_DECR_WRAP_EXT, GL_KEEP);
  glActiveStencilFaceEXT(GL_BACK);
  glStencilOp(GL_KEEP, GL_INCR_WRAP_EXT, GL_KEEP);
  glCullFace(GL_NONE);

  // (The "Z bounds" optimization sets the depth bounds here.)
  // Take light to object space and compute light back faces
  obj->findBackfaces(cframe.inv() * wsL);

  // Set up for vertex buffer rendering
  glVertexBuffer(vertexBuffer);

  renderShadowCaps();

  renderShadowSides();

  glVertexBuffer(NULL);
}
```

用这个物体空间的光矢量，计算光源的前面和光源的背面。面对方向只是暂时需要，并且存储在一个全局数组中。然后选择（双倍长）顶点缓冲，把阴影体的亮罩和暗罩渲染为三角形。最后，把阴影体的侧面渲染为方块。

9.3.4 findBackfaces 方法

findBackfaces 方法在每个面上迭代和计算 $\boldsymbol{N} \cdot \boldsymbol{L}$，显示在列表 9-5 中。

列表 9-5 findBackfaces 方法

```
void Object::findBackfaces(const Vector4& osL)   // 物体空间的光矢量

{
  backface.resize(face.size());
  for (int f = 0; f < face.size(); ++f) {
    Vector3 L = L.xyz() - vertex[face[f].vertex[0]] * L.w;
    backface[f] = dot(face[f].normal, L) < 0;
  }
}
```

对于一个有限距离的点光源，需要指向特定多边形的矢量，因此，我们从光源的位置减去一个面顶点的位置。对于方向光，光的方向是不变的。具体执行时，这些情形能在独立的回路中处理；只是为了简便在这个例子中把它们组合在一起。因为我们只对 $\boldsymbol{N}\cdot\boldsymbol{L}$ 的符号感兴趣，不关心它的大小，所以矢量不需要有单位长度。

如果模型是动画的，对每帧必须从动画的顶点重新计算面法线。因为调整过的几何图形不适于在 CPU 上运行，所以不能在硬件上使用矩阵蒙皮或顶点混合。在本章结尾，我们将讨论把阴影体与硬件顶点置换相结合的技术。

9.3.5　亮罩和暗罩

给定背面的数组，我们可以计算罩和阴影体的侧面。在计算时，我们将会累积一连串的顶点索引，然后用 glDrawElements 渲染来自顶点缓冲的索引。索引暂时存储在其他的全局数组中，叫做 index。

亮罩和暗罩的代码列在程序列表 9-6 中。

列表 9-6　renderShadowCaps 方法

```
// Indices into vertex buffer
std::vector<unsigned int> index;

void Object::renderShadowCaps()
{
  // (The "Culling" optimization changes this method
  // to try to cull the light and dark caps separately.)
  index.resize(0);
  for (int f = face.size() - 1; f >= 0; --f) {
    if (backface[f]) {
      // Dark cap (same vertices but at infinity)
      for (int v = 0; v < 3; ++v) {
        index.pushBack(face[f].vertex[v] + n);
      }
    } else {
      // Light cap
      for (int v = 0; v < 3; ++v) {
        index.pushBack(face[f].vertex[v]);
      }
    }
  }

  glDrawElements(GL_TRIANGLES, index.size(), GL_UNSIGNED_INT, index.begin());
}
```

亮罩是面向光源的简单多边形。为了创建暗罩，我们取光的背面，把它们发送到离光源无限远的地方。为了这样做，我们从第 2 组顶点渲染，顶点 shader 把它们发送到无限远处。

图 9-7 表示动画 *Quake 3* 中的人物站立在白色的地面上，被一个白色的点光源照亮。人物的阴影体用身体右边的黄色显示。暗罩是离人物很远的阴影体的一部分，暗罩的形状与人物的相同，但是被放大了。因为亮罩在人物中，所以是看不见的。在亮罩和暗罩之间拉伸的

多边形是阴影的侧面，由轮廓边缘构成。

（a）单个点光源　　（b）可见的阴影体

图 9-7　一个动画 *Quake 3* 中的阴影人物

9.3.6　侧面

阴影体的侧面是在第 1 组和第 2 组顶点之间的四边形，处于物体和无限远之间。我们在网格的边上迭代。只有在轮廓上的边才需要拉伸成方块；另一些边不影响阴影体。

在物体的光背面碰到它的光前面的地方产生轮廓的边缘。所有信息可以用来进行一个分类。边存储两个毗邻面的索引，而且背面的数组告诉我们哪个面的索引对应于光背面，见列表 9-7。

用一致的边缘方向构造网格的边缘数据很重要，这样可使产生的阴影面块有正确的发散方向。在阴影面上，顶点必须是反时针顺序，使表面法线指向阴影体外。为了保证这一点，我们约定，从 *v*0 = edge [e].vertex [0]到 *v*1 = edge [e].vertex [1]的边的方向，在索引为 edge [e].face [0]的网格面上是反时针的，而在索引为 edge [e].face [1]的网格面上是顺时针的（背向的）。

阴影方块必然包含与背面同样定向的边。因此，如果面（edge[e].face[0]）是背面，阴影面包含从 *v*0 到 *v*1 的边，否则，它包含从 *v*1 到 *v*0 的边。图 9-8 显示面向光的面和阴影方块在从 *v*0 到 *v*1 的方向边上转动的方向。

列表 9-7　renderShadowSides 方法

```
void Object::renderShadowSides()
{
  index.resize(0);

  for (int e = edges.size() - 1; e >= 0; --e) {
    if (backface[edge[e].face[0]] != backface[edge[e].face[1]]) {
      // This is a silhouette edge
      int v0, v1;
      if (backface[edge[e].face[0]]) {
        // Wind the same way as face 0
        v0 = edge[e].vertex[0];
        v1 = edge[e].vertex[1];
      } else {
        // Wind the same way as face 1
```

```
            v1 = edge[e].vertex[0];
            v0 = edge[e].vertex[1];
          }

          // (The "directional" optimization changes this code.)
          index.pushBack(v0);
          index.pushBack(v1);
          index.pushBack(v1 + n);
          index.pushBack(v0 + n);
        }
      }

      // (The "directional" optimization changes this to use
      // GL_TRIANGLES instead of GL_QUADS.)
    glDrawElements(GL_QUADS, index.size(), GL_UNSIGNED_INT, index.begin());
  }
```

我们现在了解了整个阴影渲染的程序。我们建造了一个系统，它把在模板缓冲中的像素分为遮挡的或未遮挡的，然后给未遮挡的像素加入照明。这个系统能处理许多不同类型的光源和复杂的阴影投射几何体。它也可以和其他的阴影算法合并，例如投射阴影和阴影贴图。也可以改变程序，只对那些通过了所有阴影测试的区域添加照明。

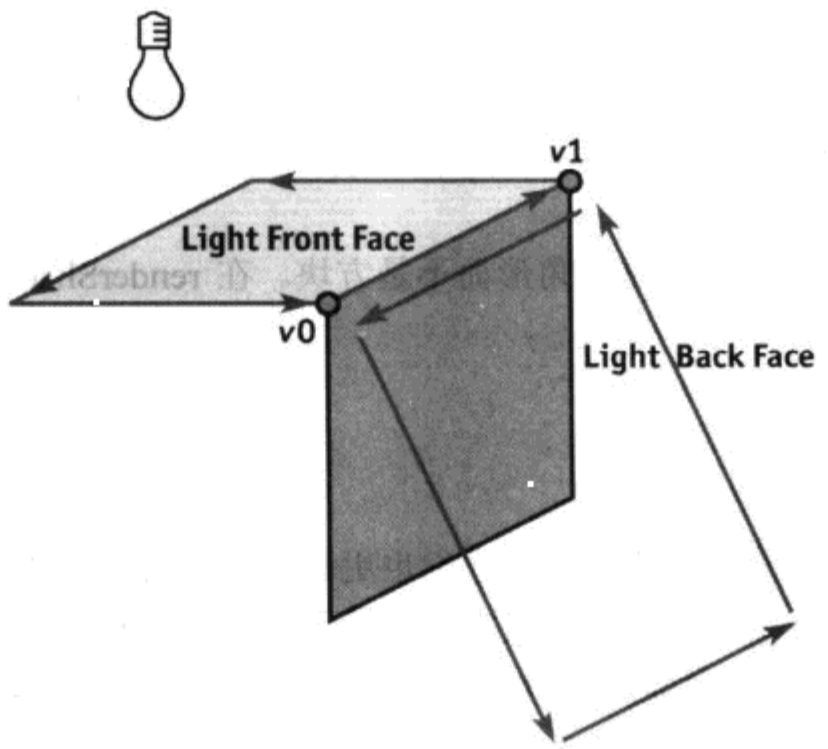


图 9-8　转动方向
由从 v_0 到 v_1 的轮廓边产生的阴影侧面。箭头表示在前面和阴影面的顶点的反时针方向

利用通常的阴影体算法的简化，可以明显地加速运算。这章的其余部分描述加速阴影体创建和渲染的方法。实际上下列方法能使基本的运算加速 4 倍。

9.4　调试

现在试验是否可以正确地制作阴影体，暂时地激活彩色渲染，然后用附加的 alpha 混合画阴影体。开启面裁剪，对前面使用一种颜色，对背面使用另一种颜色。这些方式除了调试

还有其他的用途，例如在玩游戏时渲染可见的阴影体，得到让光线穿过云或树的效果。

请记住，OpenGL 要求激活模板测试，甚至它设定为 GL_ALWAYS_PASS 时也是如此。也不要忘记模板掩码 glStencilMask（~0）。如果忘记这两个先决条件，写入操作将不工作。

使用断言（assertions）去检查每条边是否确实有两个毗邻的面。如果在模型上有一个裂缝，就会得到图 9-3 中那样的空中阴影条带。可以使用 3ds max 那样的软件工具自动地修补裂缝（叫做焊接顶点）。

9.5 几何优化

为了清楚和简单，在本章的前半部分以一般的形式描述了基本的阴影体算法，方向光、点光源和聚光灯的处理方法相同。在列表 9-5 中用了数学技巧 ***L***.*xyz* () − V * ***L***.*w*，与列表 9-3 中的顶点 shader 类似。这些列表用一个表达式计算两种光源的光矢量。产生阴影体的整个过程中，在顶点 shader 中分别处理它们，还可以进一步改进它们。由方向光产生阴影体比点光源更简单，所以可以方便很多（但代码的复杂性会变大）。

我们也可以使用传统的包围盒技术剔除阴影几何体，从而改善几何处理过程。这节讲述了这些优化方法。

9.5.1 方向光

对于方向光，光矢量是 ***L***.*xyz*。因为光矢量在所有的顶点都是相同的，暗罩上的所有顶点都落在同一点上，即−***L***。这意味着不存在暗罩：由于阴影体的平行侧面会聚于无限远的单个点，因此罩是不必要的。

因为它们会聚于一点，侧面是三角形而不是方块。在 renderShadowSides（列表 9-7）中的 push 语句变成：

```
index.pushBack(v0);
index.pushBack(v1);
index.pushBack(v2);
```

这些语句不仅有比较少的索引，而且它们也更利于顶点高速缓冲存储。那是因为相同的顶点数 *n* 被转换多次（我们可以转移任何大于或等于 *n* 的索引的顶点，因为它们全都转换到同一点）。我们可以选择除去全部顶点 shader，并加入另一个索引为 2*n* 的顶点，它在每个阴影遍之前被设定成负的光矢量。

9.5.2 点光源和聚光灯

点光源随距离而衰减。在一定的距离之外，点光源发出的光可以忽略（当它降低到 1/255 以下，我们就不能在一个 8 位的帧缓冲中见到结果）。聚光灯随角度，有时也随距离衰减。聚光灯的圆锥体之外没有照明。

如果一个物体在这两类光源的有效范围之外，那么在它后面的任何物体也在范围之外，所以它不需要投射阴影。这时针对随距离衰减的光包围球，或者随角度衰减的光圆锥体，检测阴影投射体的包围盒。若物体在光照范围之外，不需要为它标记阴影。同样，它也不需要照明遍。

9.5.3　剔除阴影体

正像处理一般的几何体一样，可以通过剔除视域棱锥之外的阴影几何体，提高处理阴影体顶点的速率。要注意，在视域棱锥之外的投射体，仍然可能投射阴影到可见物体上，所以剔除投射体和阴影几何体应该是完全独立的。

侧面和罩要分别地进行剔除。对每个面，用几何原形近似阴影几何体，并用视域棱锥对那个原形进行剔除。因为明亮罩在投射体里，所以对明亮罩能与投射体使用同样的几何体。对暗罩也使用相同的几何体，但是发送到离光源无限远处。

举例来说，一个包围盒可适用于投射体。使用公式 $\boldsymbol{v}' = \boldsymbol{MV} * (\boldsymbol{v} * \boldsymbol{L}_w - \boldsymbol{L}_{xyz})$ 把包围盒的每个顶点 $\boldsymbol{v}$ 转换到无限远，$\boldsymbol{L}$ 是物体空间光矢量，$\boldsymbol{MV}$ 是 modelview 矩阵。然后针对视域棱锥测试转换过的包围盒。如果包围盒被剔除，暗罩也被剔除。阴影体侧面最容易被方向光的圆柱体包围，也容易被点光源圆锥台包围。

虽然任何的剔除都是有益的，但因为罩的顶点比侧面多，所以剔除罩特别有助于加速顶点处理。对于点光源而言，暗罩可能极大；剔除它能降低许多填充率。在卡通中可以见到这种效果：小猫站在一个手电筒前可以投射出狮子的阴影。图 9-7 示例了这种放大效果，那里模型的暗罩比模型本身大了好几倍。

9.5.4　除罩操作

虽然可以不调用罩，但在视野不遮挡的特殊情况下，可以用另外的技术除去罩。

我们在数学公式中，使用了远离视点的从一点到无限远的光线。在实施中通过渲染对模板缓冲的多边形来模拟这些光线。我们把远剪裁面移动到无限远，并发送远离视点的光线，这样就不会因为裁剪而错过点到无限远之间的任何相交。

也可以朝另一个方向进行计算。为了计算远离视点的情况，当深度测试失败时，增加或减少模板缓冲。反之，要计算朝向视点的情况，当深度测试通过时，增加或减少模板缓冲。当视口不在阴影体中的时候，沿着一个线段从不遮挡点到图像面的相交数是 0。这是因为要一条线从不遮挡的点达到一个不遮挡的视口，它经过的进入和离开相交的数目必须完全相同。如果点被遮挡，相交数将不会是 0。当然，只能在视口不在阴影中的方向上计数；否则，由于阴影体遮挡视口像素而中断计数。图 9-7 表示了能使用这种优化的情形，那里延伸回场景的阴影不遮挡视口。图 9-2 表示了不能够使用的例子，因为视口在南瓜投射的阴影中，实际上，在场景中的每个东西都是在阴影中，除了从窟窿射出的光照到地面上的三角形。

朝向视点计算的好处是不需要渲染亮罩和暗罩。亮罩总是让深度测试失败，因为它在阴影投射体里，所以没有必要渲染它。因为正从可见点到视点进行计算，没有方法让暗罩（它在无限远）产生相交，它不会改变结果，所以也不需要渲染它。

优化要求根据以下两点改变代码。

1．需要测试视口是否是（保守地）在一个阴影体中。要分别地对每个阴影投射体进行测试；为每个投射体可以独立地选择计算方向，并得到一个正确的结果。

2．如果视口不在阴影体中，需要使模板操作的增加/减少（只是对那个投射体）状态变反。

图 9-9 显示了视口的遮挡棱锥。顶端在光源（如果它是一个方向光，在无限远），而底面

是视口。如果阴影的包围盒、投射体与这个棱锥相交，视口可能被遮挡，就不能够进行优化。在这样的情况下，我们必须以正常的深度失败的操作渲染并画两种罩（如果可见）。如果包围盒不与这个棱锥相交，可以改变模板操作。

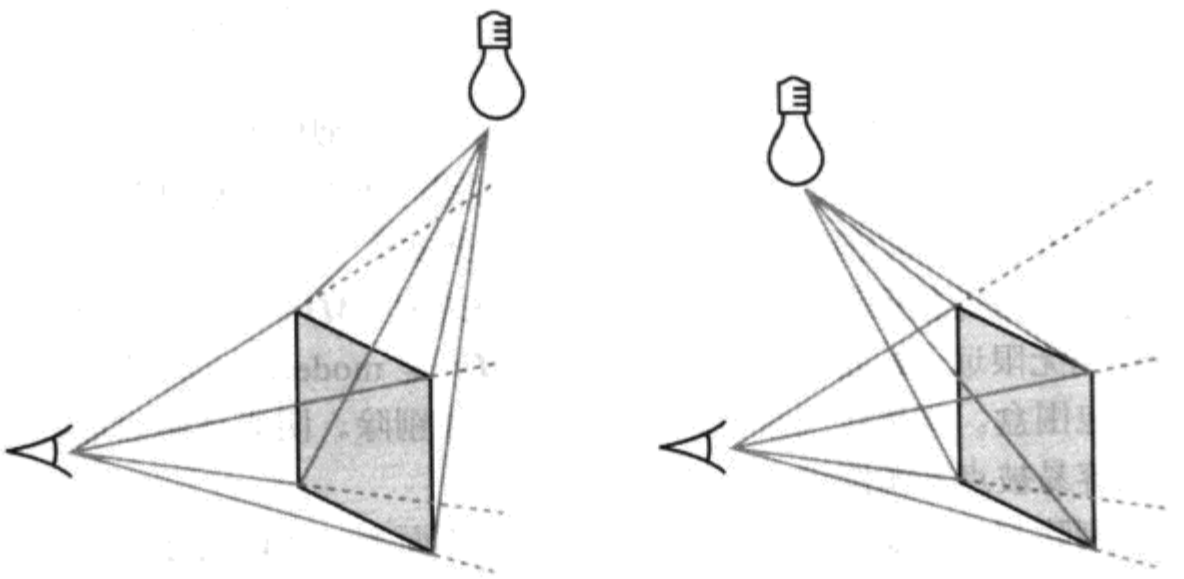


图 9-9 从视口伸展到光源（可能在无限远）的遮挡棱锥

遮挡棱锥可能在视口的两边。如果阴影投射体与绿色的棱锥相交，不能够使用“除罩”优化。

为朝向视点计算，模板操作设定如下：

```
// Increment on front faces, decrement
// on back faces (a.k.a. z-pass rendering)
glActiveStencilFaceEXT(GL_FRONT) ;
glStencilOp(GL_KEEP, GL_KEEP, GL_INCR_WRAP_EXT);
glActiveStencilFaceEXT(GL_BACK);
glStencilOp(GL_KEEP, GL_KEEP, GL_DECR_WRAP_EXT);
```

因为这是除罩渲染，在这个情形下完全地省略渲染阴影体罩的代码。

9.6 填充率的优化

填充率是阴影体的致命弱点。阴影体包括许多像素而且很费时。投射体产生更大更远的阴影，这对点光源是个特别的麻烦。幸好点光源实际照明范围之外的光强度为 0，所以它可能很好地优化。我们已经讨论过，在光照范围之外不为投射体制作阴影，也不渲染在物体上的照明。为了降低在范围内的投射体的填充率，我们研究以下 3 个方法：有限的体积、XY 裁剪和 z-范围。

9.6.1 有限的体积

点光源的范围是球体。在这个球体之外的物体不接收照明，因此不需要投射阴影到球体之外。不再把阴影体延伸到无限远，而是把它们扩展到球体半径，从而降低了填充率。只要简单改变顶点 shader 就可以做到这一点。因为这个方法中暗罩更可能出现在屏幕上，不大可能被剔除，所以有可能增加几何处理。

另一个选择是仍然创建伸展到无限远的多边形，但是剪裁它们到二维的光半径，在下一

个优化法中描述。

9.6.2　XY 裁剪

范围球体投影为荧屏上的椭圆。只有在椭圆里面的像素能被照明。我们不需要渲染这个椭圆外面的阴影多边形或照明。但是硬件支持的裁剪区域是矩形，而不是椭圆。我们可以计算投射的椭圆包围盒，但是使用光源范围内所投射的 3D 包围盒的 2D 包围盒更方便。虽然固定函数的传递途径只支持半径衰减，但是指定光源的立方体上的任意衰减函数，美工能够得到更加受控的效果，如在 *Doom 3* 中所做的。在盒里能任意地衰减，因此，我们仅仅以那个盒作为光范围。对视域棱锥裁剪光源的包围盒。如果它没有完全被裁剪，把剩余多面体的所有顶点投射到视口上，并界定它们。把最终的 2D 边界当作矩形的裁剪区域。用 glScissor 指令设定裁剪区域：

```
glScissor(left, top, width, height);
glEnable(GL_SCISSOR_TEST) ;
```

图 9-10 表现一个 *Quake 3* 人物站在一栋建筑物的外面。这是来自图 9-7 的场景，但是现在照相机已经向后移动。单个点光源创建人物和建筑物的阴影体，假如它们没有被光源的有效范围裁剪掉，就可能填充到屏幕上了。在右边图中，裁剪区域显示为一个白色的方块。左边图显示可见的场景，那里裁剪的效果不明显，因为光没有照亮离建筑物远的地方。对于这个场景，只渲染裁剪区域内的阴影体像素，填充花销减少一半。

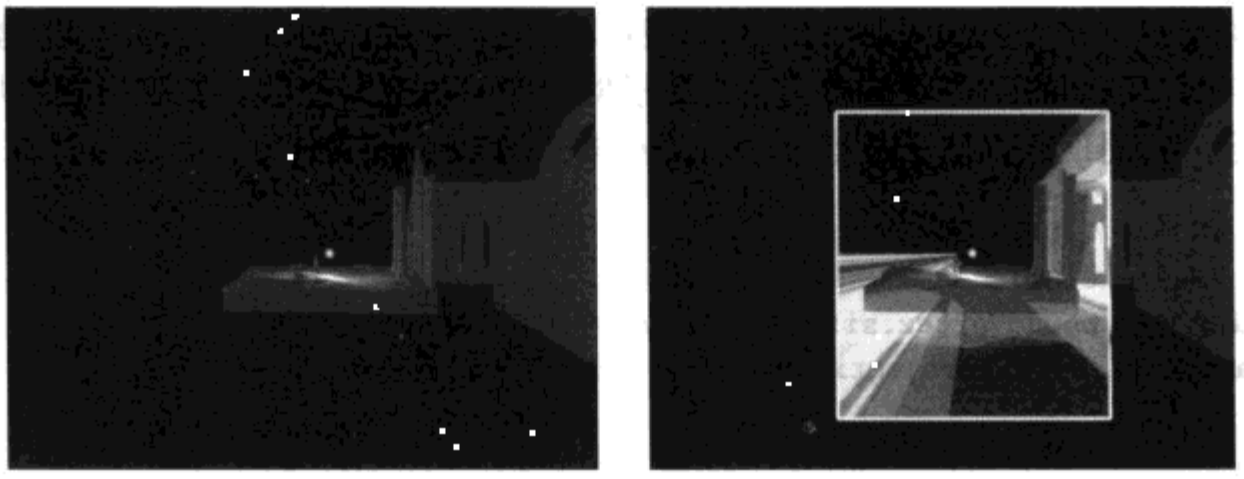


图 9-10　*Quake 3* 的裁剪
左边：最后的场景。右边：裁剪阴影体的区域（表示为白色的方块）

9.6.3　Z-边界

如果给定像素上的点在光范围之外（它或是离视点太近，或是离视点太远而超出范围边界，致使那点不能够被照明），就不需要在那个像素上制造阴影标记或照明遍。把遍限制在特定的深度范围内，只填充实际上受光影响的像素（可能比光的 2D 范围中的像素更少）。

glDepthBoundsEXT 函数让我们可以这样设置：

```
glEnable(GL_DEPTH_BOUNDS_TEST_EXT);
glDepthBoundsEXT(zMin, zMax);
```

这样设置避免渲染那种像素（在环境遍中渲染的）：它上面的可见点在深度缓冲范围[zMin, zMax]之外。剪裁面避免渲染边界之外的多边形上的新像素，两者是不同的。

图 9-11 显示了视点和由点光源照明的物体。投影体向下投射阴影到高低不平的斜坡上。地面的绿色粗线部分不可能被投射体遮挡。深度边界测试可以省去桔色部分阴影体的填充，因为在绿色粗线后面的可见像素在限度之外。阴影体本身在边界内，但是这没有关系，深度边界只应用于在环境遍中所渲染的像素，不应用于阴影几何体。

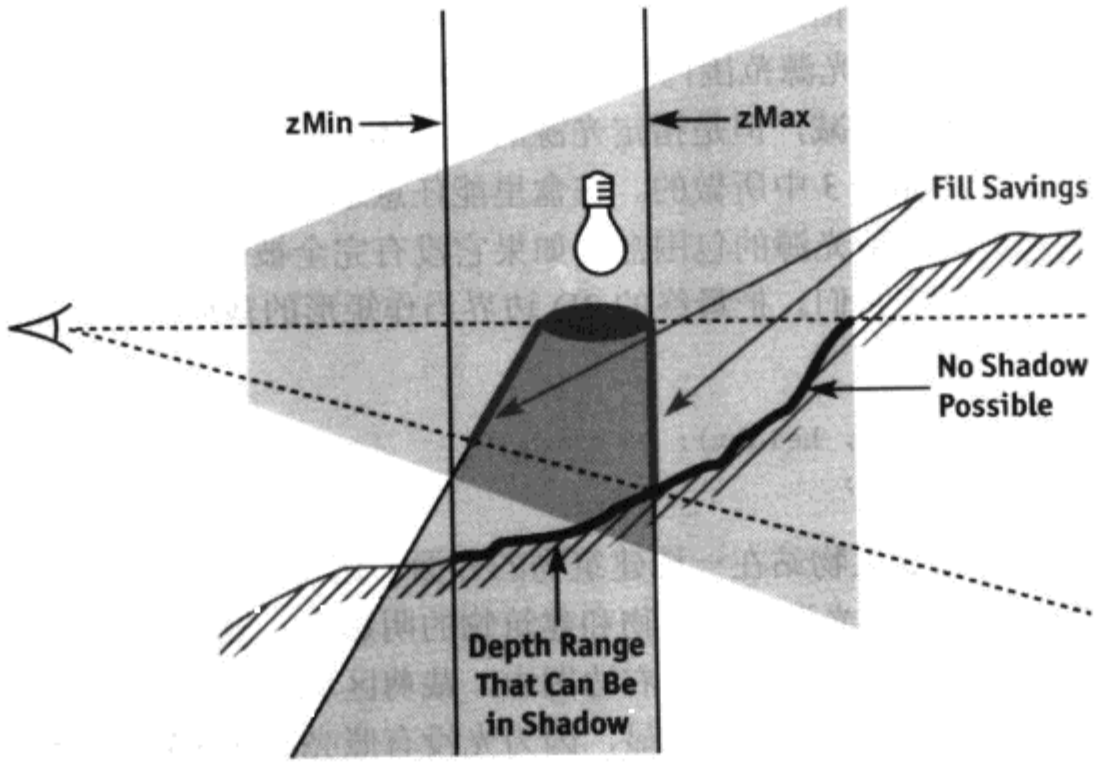


图 9-11 深度边界

注意深度限度比光范围限制得更严格。深度范围是由视域棱锥、光范围和阴影体边界的相交点决定的。OpenGL 函数的自变量是后投射的照相机空间的值。如果相交的几何图形是一个多边形，它的顶点存储在数组 std::vector<Vector4> boundVert 中，自变量计算如下。

```
float zMin = 1.0f;
float zMax = 0.0f;
for (int v = boundVertex.size() - 1; v >= 0; --v) {
  float z = 1.0f / (projectionMatrix * boundVert[v]).w;

  zMin = min(zMin, z);
  zMax = max(zMax, z);
}
```

9.7 将来的阴影

目前高度优化的阴影体方法是几十年来工业界和学术界的贡献。在这章中介绍的 Frank Crow 在 SIGGRAPH 1977 的基本方法已经成熟。阴影体的历史和一些研究、开发者的个人贡献，概述在 NVIDIA 开发者网站上的技术报告中（Everitt and Kilgard 2002，McGuire et al. 2003）。本章对 McGuire 等人 2003 年提供的方法进行了正式的描述和分析。

新的改进阴影体的优化方法，仍是活跃的研究领域。轮廓检测的主要限制，是它总在 CPU 上运行。它排斥使用顶点 shader 中的矩阵蒙皮或其他变形，而是连续在 CPU 上进行渲染。

已经提出几种直接在可编程图形硬件上进行轮廓检测的方法。Michael McCool（2001）

提出通过阴影贴图计算投射体轮廓。Brabec 和 Seidel（2003）通过像素处理器把几何图形编码为颜色，在那里计算轮廓；然后重读帧缓冲，并在阴影渲染中把它作为顶点缓冲。John Hughes 和我最近介绍了一种方法，使用专门预先计算的网格，完全是在顶点 shader 中找到轮廓，并拉伸阴影体侧面（McGuire and Hughes 2003）。

另一个公开研究的主题，是用阴影体从面光源得到好看而高性能的模糊阴影。Ulf Assarsson 和 Tomas Akenine-Möller 已经研究这个问题一段时间了。他们最近与 Michael Dougherty 和 Michael Mounier 发表的论文（Assarsson et al. 2003），介绍如何使用可编程的硬件，为半影（模糊阴影区域）的内、外边构造清晰的几何图形。

有人已经提出把个别的轮廓边缘加进连接的条带，从而可以用方形条带（点光源）和扇形条带（方向光）来渲染阴影体侧面。Alex Vlachos 和 Drew Card（2002）已经提出另外的简化思路：剔除和裁剪嵌套的阴影体，因为它们不影响最后的结果。

所有这些方法正在实验，尚待真实的游戏引擎予以修改和精化。对于阴影体方法的发展，这是一个良好开端。希望将来的研究和图形硬件能够改进和加速这些方法。

9.8　参考文献

Assarsson, U., M. Dougherty, M. Mounier, andT. Akenine-Möller. 2003. "An Optimized Soft Shadow Volume Algorithm with Real-Time Performance." In *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003.*

Brabec, S., and H. Seidel. 2003. "Shadow Volumes on Programmable Graphics Hardware." Eurographics 2003 (Computer Graphics Forum).

Everitt, Cass, and Mark Kilgard. 2002. "Practical and Robust Stenciled Shadow Volumes for Hardware-Accelerated Rendering." NVIDIA Corporation. Available online at **http://developer.nvidia.com/object/robust_shadow_volumes.html**

McCool, Michael. 2001. "Shadow Volume Reconstruction from Depth Maps." *ACM Transactions on Graphics,* January 2001, pp. 1～25.

McGuire, Morgan, and John F. Hughes. 2003. "NPR on Programmable Hardware." To appear in *Proceedings of NPAR 2004,* June 7～9, Annecy, France.

McGuire, Morgan, John F. Hughes, Kevin Egan, Mark Kilgard, and Cass Everitt. 2003. "Fast, Practical and Robust Shadows." Available online at **http://developer.nvidia.com/object/fast_shadow_volumes.html.** 早期刊物见 *Brown Univ. Tech. Report* CS03-19.

Vlachos, Alex, and Drew Card. 2002. "Computing Optimized Shadow Volumes." In *Game Programming Gems 3,* edited by Dante Treglia. Charles River Media.

Tekkaman Blade robot model by Michael Mellor (mellor@iaccess.com.au); Tick model by Carl Schell (carl@cschell.com). Both available for download at http://www.polycount.com. Cathedral model by Sam Howell (sam@themightyradish.com), courtesy Sam Howell and Morgan McGuire. "The Tick" character is a trademark of New England Comics. Quake 2, Quake 3, and Doom 3 are trademarks of id Software.

第 10 章 电影级的光照

Fabio Pellacini
Pixar Animation Studios
Kiril Vidimce
Pixar Animation Studios

在这章中，我们介绍一个简化的泛光（uberlight）实现，光照 shader 是根据 Ronen Barzel（1997，1999）提出的照明模型编写的。这个模型的超集由 Pixar 动画工作室在几年内开发出来，并且已用来生产动画电影，例如 Walt Disney（迪斯尼）的电影 *Toy Story*（玩具总动员）、*A Bug's Life*（蚁哥传奇）、*Monsters, Inc.*（怪物公司）和 *Finding Nemo*（海底总动员）。

我们的 Cg 实现基于 Barzel 提出的方法和 Larry Gritz（Barzel 1999）用 RenderMan 着色语言所写的实现。这种光照方法的细节和它在电影制作中的应用，可以在 Apodaca and Gritz 1999 以及 Birn 2000 中查到。

10.1 引言

光照是计算机电影技术的重要组成部分，它用光和影来营造气氛和支持情节（Calahan 1999）。尽管真实性仍然是计算机产生影像的一个重要方面，但是灯光设计师一直在尝试非物理光照，以支持动画电影的艺术描写。在真实世界里实施这些技巧是艰苦的，常常需要数小时进行设置。

抛开真实物理的限制，计算机电影技术的开发者设计出的光照模型使美工可以直观地照明场景。这章中介绍的光照模型，是 Pixar 动画工作室近 10 年发展的改进模型，已经应用于许多电影的制作，见图 10-1、10-2、和 10-3。

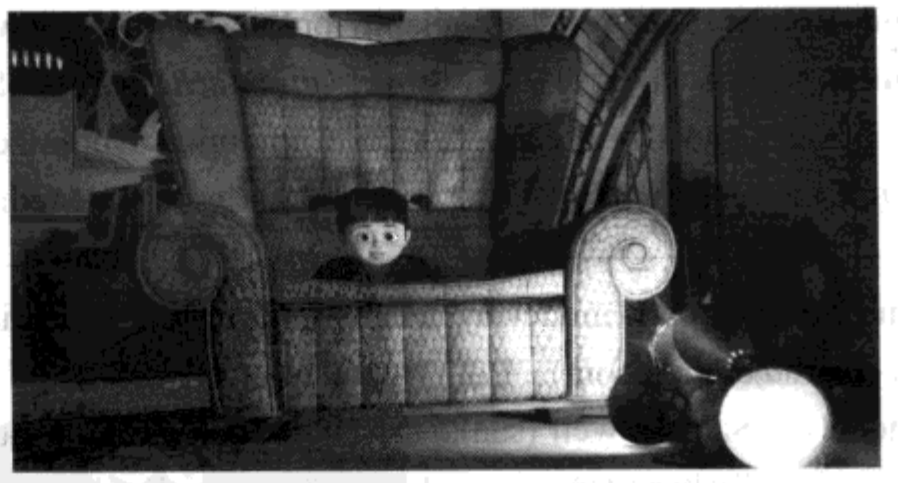


图 10-1 在 *Monsters, Inc* 中的谷仓照明
版权属于 2001 Disney 影业公司/Pixar 动画工作室

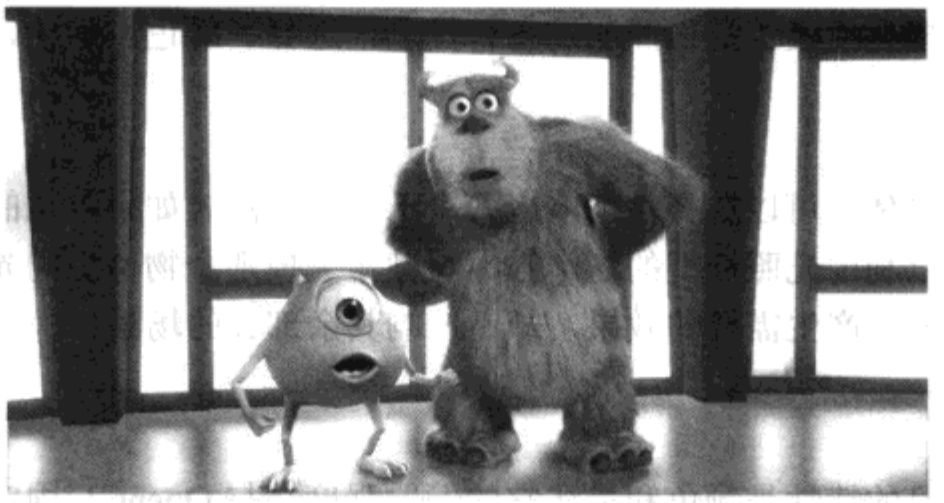


图 10-2　在 *Monsters, Inc* 中 cookies 对窗户效果的贡献
版权属于 2001 Disney 影业公司/Pixar 动画工作室

图 10-3　在 *Monsters, Inc* 中用光照表达气氛
版权属于 2001 Disney 影业公司/Pixar 动画工作室

10.2　直射光照明模型

我们在这章提供的 shader 模型，只对照明场景的光源塑形和控制进行建模，不对表面细节和光反射现象建模（一些有趣的表面现象例子可以在 Apodaca 和 Gritz 1999 中查到）。

大体上，用于电影产品的照明模型进行两种操作，类似于显示在这里的伪代码。

```
color illuminationModel()
{
  Compute the surface characteristic
  For each light {
    Evaluate the light source
    Compute the surface response
  }
}
```

首先，通过这些方式计算表面着色信息：运行各种纹理查询，在网格上插值，计算程序模式等。然后在照明物体的每个光源上循环，计算出它的贡献。我们通过对每个光先计算光的颜色，然后计算表面对照明的响应来进行这个操作。

在这章中，我们介绍一个简单的 shader，它只计算一个光源对塑料反射建模的贡献。作为练习留给读者的是，将其扩展到多个光源和更好看的表面，求出更一般的解。

我们的光照明模型给美工提供照明的各种控制：选择、颜色、形状、阴影和纹理。

10.2.1 选择

场景中的每个物体，可以选择性地响应各个不同的光源。如果附加的光源创建出了不希望的效果，美工可以利用光照模型的这个强大特性，关掉那个物体上的光。也可以增加额外的光源，在特定位置上产生需要的效果，却不影响其余部分的场景。

10.2.2 颜色

光源最引人注目的特性是颜色和描述它的发射强度。与 OpenGL 固定函数的光照类似，我们的实现为环境光、漫反射和高光反射效果提供了各自的权重，美工可以分别调节这些效果。我们光照模型最重要的特点是对每个物体可以自由地改变这些项，从而使美工可以用少量的光源进行全面的设置。

10.2.3 造型

为了控制由光源照明的场景区域，真实世界的电影摄影师普遍采用聚光灯和矩形灯光（也叫谷仓门）对光的分布塑形。我们的光照模型通过提供两种形状来概括这些概念。

- 泛光灯（Omni）：这个光源有近和远的两个截断距离（两者之间以最大强度照明），在每个截断边附近有一个可调节的、平滑衰减的区域。
- 谷仓形状：谷仓形状是一个截头锥体（锥台，译者注），它的横截面是一个有可调模糊边的超级椭圆。为了拓宽形状范围，超级椭圆的宽度和高度能分别地控制。超级椭圆横截面的使用，使我们能不断地改变光源的形状，从圆到正方形（甚至更奇异的像星一样的形状）。此外，锥体可以自由地修剪，以产生窗户和门的效果。锥体横截面的形状变化的例子见图 10-4 和 10-5。

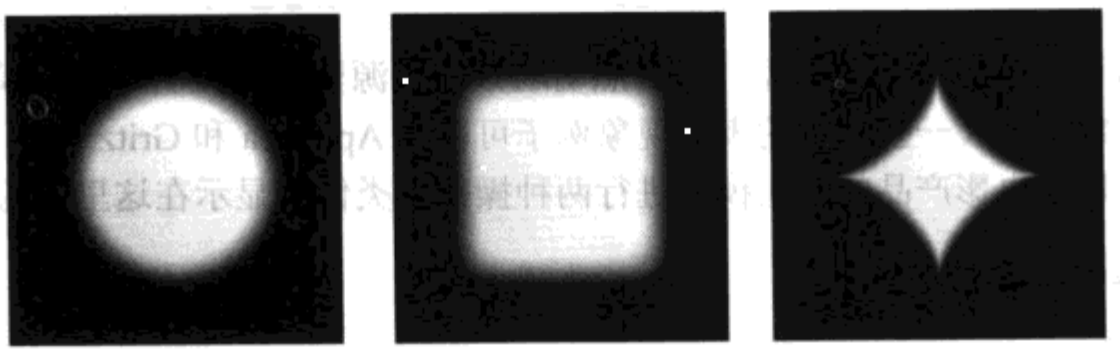


图 10-4 谷仓门的形状
版权属于 2003 Pixar 动画工作室

图 10-5 谷仓门边缘的尺寸
版权属于 2003 Pixar 动画工作室

10.2.4　阴影

阴影是我们光照明模型的一个重要方面；它也许是美工最经常使用的一种技巧。调节阴影不仅是从速度方面考虑，而且由于它对电影的整体气氛非常重要，因此应该控制阴影的每个小方面。例如，黑帮电影中非常浓重的阴影，在音乐喜剧片中几乎见不到。

像照明强度一样，美工决定哪个物体投射阴影，哪个物体接收阴影。灯光设计师也可以通过改变相对于光源的关系来调节阴影位置。例如，可以让角色眼中有明亮高光的同时保证阴影不穿过角色的脸。

黑暗

直接的光照明模型最大的问题之一是阴影容易太暗。这是因为在环境中自然发生的大部分间接照明没有计算在内。为了使模型逼真，我们创建了能调节的光来改变阴影的密度，即让一些光通过在场景中的物体进行传递。在实现中，我们使用表面的漫反射给阴影区域着色。这种方法保持了阴影很好的梯度，使阴影变得更可信，所以比使用一个环境光更好。在本章的后面，我们将详述这个问题。

色调

在真实世界中，影子区的色调与附近的未遮挡区略有不同，举例来说，在一个明亮的阳光普照的天气里，可注意到户外的影子带有微微的蓝色调。为模仿这个效果，美工可以改变影子的颜色。色调的微小变化可以区分出好看的阴影和虚假的阴影。实际上，可以把它想像为阴影投射体把颜色“喷洒”到阴影接收体上，颜色一般为黑色。图 10-6 为不同阴影颜色的图像。

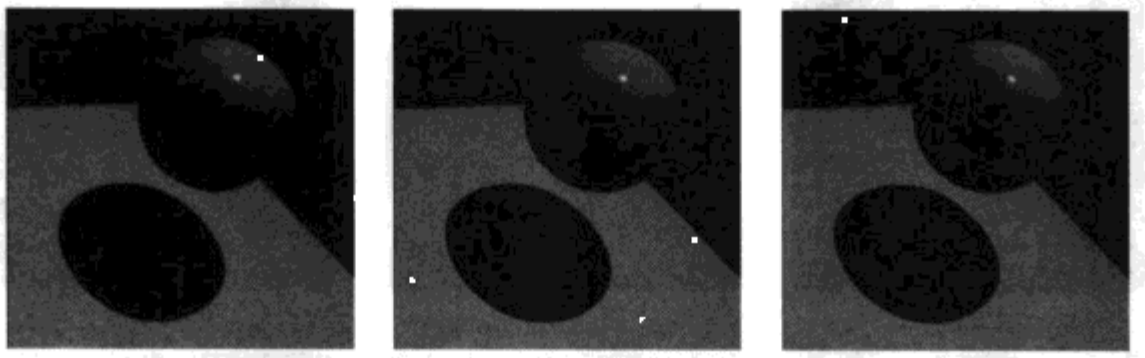


图 10-6　阴影颜色的变化
版权属于 2003 Pixar 动画工作室

反射

对于高光有一种特别的限制。计算阴影区中表面的贡献时，为了得到室外环境精致的阴影梯度，只使用漫反射响应，而不想在阴影区见到高光，因为要避免漫反射的相互反射（interreflection）。为了达到这个效果，我们简单地在阴影区关掉高光反射。这个小调整只是一个例子，说明可以怎样改变表面的光反射现象。像这样的调节广泛地应用于电影制作中，以实现我们希望观众喜欢的特殊效果。

阴影贴图

有很多技术可以用来实现阴影，我们在模型中选择了阴影贴图，它们简单而灵活，在电影中经常使用。虽然本章中的 shader 基于这样的算法，但是我们鼓励读者实验其他的阴影算

法。在 shader 中，重要的不在于如何判别一个像素是否在阴影中，而在于如何使用这个信息。

阴影的模糊

我们的实现中最缺少的是阴影的模糊。为了要调节区域光，或者只是要满足导演的特殊要求，美工时常需要调节阴影边缘的模糊度。有效地模糊阴影边缘特别困难。有各种可用的技术，不过对它们的介绍超出了本章的范围。

10.2.5 纹理

最后，我们添加投射的纹理来支持各式各样的效果，例如滑动放映机和来自屏幕之外物体的假阴影。这些技巧在电影业中称为 cookies，它们有时也用于游戏制作。游戏开发者为形状、上色和阴影效果使用纹理投射，而电影制作者倾向于使用软的 cookies 丰富视觉细节和添加特效，例如摄相机之外的阴影投射体或形状奇怪的灯光等。

10.2.6 结果

图 10-7 和 10-8 举例说明泛光（uberlight）的使用，照明来自 Pixar 短片“Geri 的游戏”中人物的头。模型的表面是平坦的，像塑料一样，没有在它上面应用任何材质的纹理。使用这种简单的表面模型，可以更好地强调不同参数的光 shader 的各种效果。如在原始的 Pixar 短片中一样，光和表面建模的适当组合使这个人物活起来。

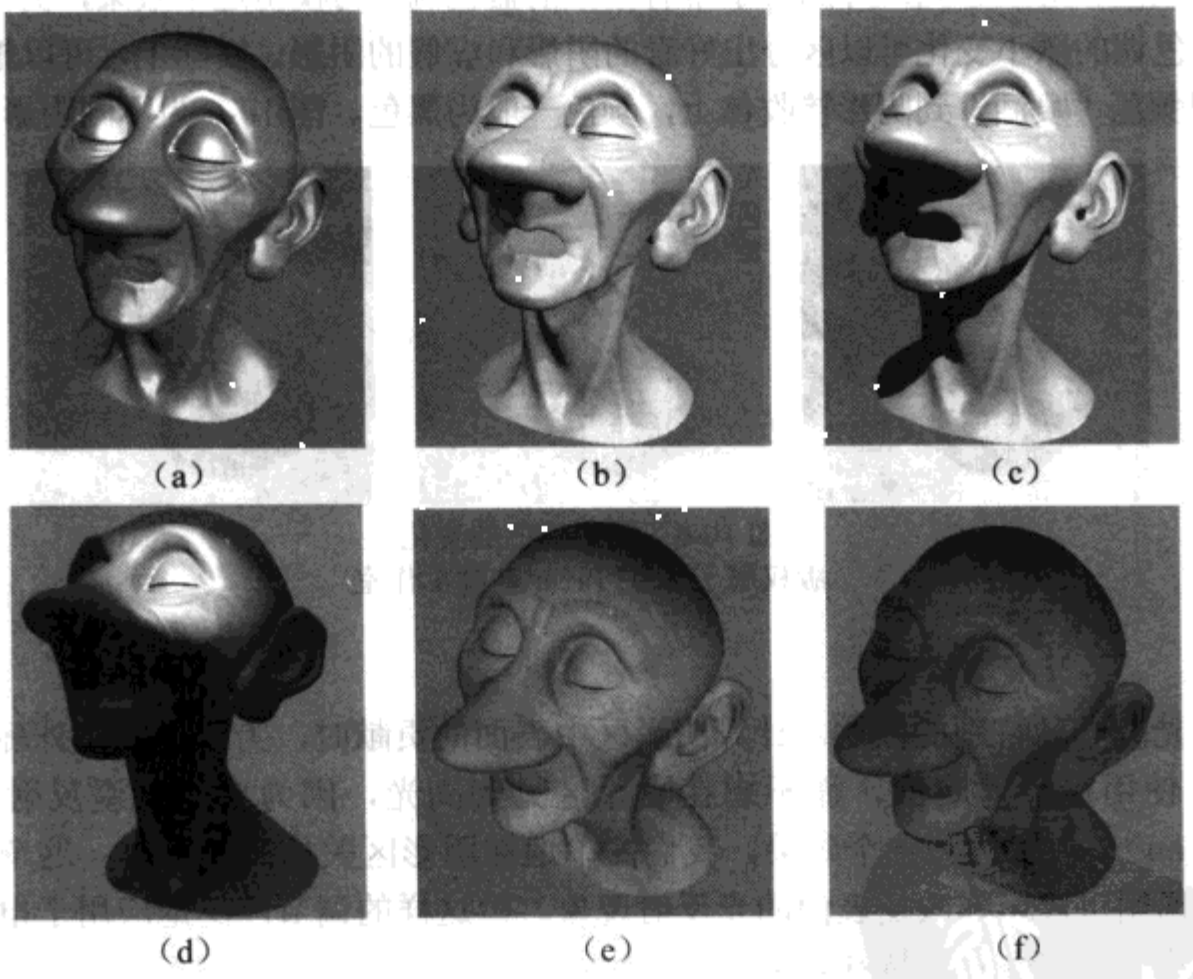


图 10-7 Geri 的光照明

（a）Geri 由一个光源照明；（b）改变光的权重，修改反射高光对比度；（c）改变阴影颜色，加强阴影；（d）改变谷仓形状，创建更戏剧化的姿态；（e）使用一块模糊的纹理 cookie，丰富图像；（f）夸大投射的 cookie 的对比，创建像外星人一样的效果

版权属于 1988 Pixar 动画工作室

（a）　　（b）　　（c）

图 10-8　照明风格

（a）常态；（b）黑帮（noir）片的高反差；（c）柔和的神情（以两个光获得）

版权属于 1988 Pixar 动画工作室

10.3　泛光 Shader

列表 10-1 和 10-2 显示泛光 shader 的源代码，基于 Larry Gritz 在 Barzel 1999 上提出的程序。

列表 10-1　一个类泛光 Shader 的顶点程序

```
void uberlight_vp(
  varying float4 Pobject : POSITION,          // Vertex position in object space
  varying float3 Nobject : NORMAL,            // Vertex normal in object space
  varying float3 VertexColor : COLOR0,        // Vertex color
  uniform float4x4 ModelViewProj,             // ModelViewProj matrix
  uniform float4x4 ObjectToWorld,             // ObjectToWorld matrix
  uniform float4x4 ObjectToWorldIT,           // Inverse transpose of the
                                              // ObjectToWorld matrix
  uniform float4x4 WorldToLight,              // Light space
  uniform float4x4 WorldToLightIT,            // Inverse transpose of light
                                              // space to transform normals
  uniform float4x4 WorldToShadowProj,         // Light space concatenated with
                                              // the projection matrix used for
                                              // the shadow. This defines
                                              // shadow space
  uniform float3 CameraPosInWorld,            // Camera position
                                              // in world space
  uniform float ShadowBias,                   // Shadow bias
  out float4 HPosition : POSITION,            // Rasterizer position
  out float3 CameraPosInLight : TEXCOORD0,    // Camera position
                                              // in light space
  out float3 Plight : TEXCOORD1,              // Interpolated position
                                              // in light space
  out float3 Nlight : TEXCOORD2,              // Interpolated normal
                                              // in light space
  out float4 ShadowUV : TEXCOORD3,            // Shadow UV
```

```
    out float3 Color : COLOR0)                    // Surface color
{
  // Compute coordinates for the rasterizer
  HPosition = mul(ModelViewProj, Pobject);

  // Compute world space pos and normal
  float4 Pworld = mul(ObjectToWorld, Pobject);
  float3 Nworld = mul(ObjectToWorldIT, float4(Nobject, 0)).xyz;

  // Compute the position of the point in light space
  CameraPosInLight = mul(WorldToLight, float4(CameraPosInWorld, 1)).xyz;
  Plight = mul(WorldToLight, Pworld).xyz;
  Nlight = mul(WorldToLightIT, float4(Nworld, 0)).xyz;

  // Compute the U-V for the shadow and texture projection
  float4 shadowProj = mul(WorldToShadowProj, Pworld);
  // Rescale x, y to the range 0..1
  ShadowUV.xy = 0.5 * (shadowProj.xy + shadowProj.ww);
  // When transforming z, remember to apply the bias
  ShadowUV.z = 0.5*(shadowProj.z + shadowProj.w - ShadowBias);
  ShadowUV.w = shadowProj.w;

  // Pass the color as is
  Color = VertexColor;
}
```

列表 10-2 一个类泛光 Shader 的片元程序

```
// SHADER PARAMETERS ==========================================================
// Superellipse params
struct SuperellipseShapingParams {
  float width, height;
  float widthEdge, heightEdge;
  float round;
};

// Distance shaping params
struct DistanceShapingParams {
  float near, far;
  float nearEdge, farEdge;
};

// Light params
struct LightParams {
  float3 color;    // light color
  float3 weights; // light weights (ambient, diffuse, specular)
};

struct SurfaceParams {
  float3 weights;  // surface weights (ambient, diffuse, specular)
  float roughness; // roughness
```

```
};

// BRDF/LIGHT INTERACTION ==========================================
// Compute the light direction
float3 computeLightDir(float3 Plight)
{
  // Spot only
  return -normalize(Plight);
}

// Ambient contribution of lit
float ambient(float3 litResult)
{
  return litResult.x;
}

// Diffuse contribution of lit
float diffuse(float3 litResult)
{
  return litResult.y;
}

// Specular contribution of lit
float specular(float3 litResult)
{
  return litResult.z;
}

// SUPERELLIPSE ====================================================
float computeSuperellipseShaping(
  float3 Plight, // Point in light space
  bool barnShaping, // Barn shaping
  SuperellipseShapingParams params) // Superellipse shaping params
{
  if(!barnShaping) {
    return 1;
  } else {
    // Project the point onto the z == 1 plane
    float2 Pproj = Plight.xy/Plight.z;
    // Because we want to evaluate the superellipse
    // in the first quadrant, for simplicity, get the right values
    float a = params.width;
    float A = params.width + params.widthEdge;
    float b = params.height;
    float B = params.height + params.heightEdge;

    float2 pos = abs(Pproj);

    // Evaluate the superellipse in the first quadrant
    float exp1 = 2.0 / params.round;
    float exp2 = -params.round / 2.0;
```

```
    float inner = a * b * pow(pow(b * pos.x, exp1) + pow(a * pos.y, exp1), exp2);
    float outer = A * B * pow(pow(B * pos.x, exp1) + pow(A * pos.y, exp1), exp2);
    return 1 - smoothstep(inner, outer, 1);
    }
}

// DISTANCE SHAPING ============================================================
float computeDistanceShaping(
  float3 Plight, // Point in light space
  bool barnShaping, // Barn shaping
  DistanceShapingParams params) // Distance shaping params
{
  float depth;
  if(barnShaping) {
    depth = -Plight.z;
  } else {
    depth = length(Plight.z);
    }

  return smoothstep(params.near - params.nearEdge, params.near, depth) *
                    (1 - smoothstep(params.far, params.far + params.farEdge, depth));
}

// MAIN ========================================================================
float4 uberlight_fp(
  float3 CameraPosInLight : TEXCOORD0, // Camera position in light space
  float3 Plight : TEXCOORD1,    // Interpolated position in light space
  float3 Nlight : TEXCOORD2,    // Interpolated normal in light space
  float4 ShadowUV : TEXCOORD3, // Shadow UV

  // SURFACE PROPERTIES ----------------------
  float3 SurfaceColor : COLOR0,   // Surface color
  uniform SurfaceParams Surface, // Other surface params
                                 // (weights, roughness)

  // LIGHT PROPERTIES ------------------------
  uniform LightParams Light, // Light properties

  // SHAPING ---------------------------------
  // Choose between barn shaping (superelliptic pyramid)
  // and omni shaping
  uniform bool BarnShaping,
  uniform SuperellipseShapingParams SuperellipseShaping, // Superellipse
                                                         // shaping
  uniform DistanceShapingParams DistanceShaping,         // Distance
                                                         // shaping

  // DISTANCE FALLOFF ------------------------
  // COOKIES AND SHADOWS ---------------------
  uniform sampler2D Shadow,      // Shadow texture
  uniform float3 ShadowColor,    // Shadow color
```

```
    uniform sampler2D Cookie,          // Cookie texture
    uniform float CookieDensity)       // Cookie density
{
    // TRANSFORM VECTORS TO LIGHT SPACE --------------------
    // Compute the normal in light space (normalize after vertex
    // interpolation)
    float3 N = normalize(Nlight);
    // Compute the light direction
    float3 L = computeLightDir(Plight);
    // Compute the view direction (vector from the point to the eye)
    float3 V = normalize(CameraPosInLight - Plight);
    // Compute the half-angle for the specular term
    float3 H = normalize(L + V);

    // COMPUTE THE TEXTURE PROJECTION - COOKIE
    float3 cookieColor = tex2Dproj(Cookie, ShadowUV).xyz;
    Light.color = lerp(Light.color, cookieColor, CookieDensity);

    // COMPUTE THE SHADOW EFFECT ---------------------------
    // Get the amount of shadow
    float shadow = tex2Dproj(Shadow, ShadowUV).x;
    // Modify the light color so that it blends with the shadow color
    // in the shadow areas
    float3 mixedLightColor = lerp(ShadowColor, Light.color, shadow);

    // COMPUTE THE ATTENUATION DUE TO SHAPING --------------
    float attenuation = 1;
    // Contribution from the superellipse shaping
    attenuation *= computeSuperellipseShaping(Plight, BarnShaping, SuperellipseShaping);
    // Contribution from the distance shaping
    attenuation *= computeDistanceShaping(Plight, BarnShaping, DistanceShaping);

    // APPLY TO SURFACE ------------------------------------
    // Here you should substitute other code for different
    // surface reflection models. This ccde computes the lighting
    // for a plastic-like surface.

    // Lighting computation
    float3 litResult = lit(dot(N, L), dot(N, H), Surface.roughness).xyz;

    // Multiply by the surface and light weights
    litResult *= Surface.weights * Light.weights;

    // Compute the ambient, diffuse, and specular final colors.
    // For the ambient term, use the color of the light as is
    float3 ambientColor = Light.color * SurfaceColor * ambient(litResult);
    // For the diffuse term, use the color of the light
    // mixed with the color in the shadow
    float3 diffuseColor = mixedLightColor * SurfaceColor * diffuse(litResult);
    // The specular color is simply the light color times the specular
    // term, because we want to obtain white highlights regardless of the
```

```
    // surface color. Our shadows won't be fully black, so we want to
    // make sure that the highlights do not appear in shadow.
    float3 specularColor = mixedLightColor * shadow * specular(litResult);

    // Compute the final diffuse color
    float3 color = attenuation * (ambientColor + diffuseColor + specularColor);

    // Compute the diffuse color
    return float4(color, 1);
}
```

10.4 性能分析

10.4.1 速度

用纹理查询代替某些分析计算，加速泛光 shader 很容易。纹理是通过我们想要避免的断续取样函数计算生成的。但只要纹理有足够高的分辨率（有可能基于场景和照相机参数重新创建），图像的质量还是非常好的。因为产品和质量要求离线渲染，所以很少实行这种优化。

10.4.2 开销

在这个 shader 中最费时的程序，是计算灯光的基于形状的衰减。我们可以构造一张纹理贴图，对一组给定的谷仓参数计算超级椭圆形；然后使用阴影纹理坐标，查找谷仓贴图在光的空间中作为一个投射纹理的贡献。谷仓贴图的使用大大地减少了 shader 的指令数。它能够使着色法的速度加快 2 倍（根据 NVIDIA 的 Quadro FX 2000 的测量）。

10.4.3 优化

没有摄像机和物体进入场景的时候，我们也可以优化与摄像机和场景相关的 shader 的着色部分（例如根据距离改变阴影形状）。美工常用这种模式调节光照参数以照亮一帧给定的场景。如果美工以 2 个纹理查询表代替超级椭圆和基于距离的变形，修改后的 shader 运行速度比最初的快 3 倍。

如果选择这种方式优化 shader，可以考虑使用一种高级语言在线地创建这些贴图。在拥有的多遍交互渲染器中，我们决定分遍地创建这些贴图。贴图一经创建，结果就被高速缓存并重复使用。只有当影响这些贴图的参数改变的时候，通道才被标记为脏的，要为重新估值而排队，并再次高速缓存。

10.5 小结

我们的光照明模型力图提供一组全面的照明控制，包括灯光美工日常所用的大部分效果。虽然我们的实现包括了各式各样的效果，但还可以增加（而且确实在一天天增加）更多可以给美工增加灵活性和表现力的效果。这种控制的例子可在 Barzel 1999 中发现。读者也可以扩展我们的源代码例子，以包括要用的算法。

这个光照控制只包括部分光源影响的效果。当开发一个完全的照明模型时，表面反射特性也很重要，这种属性使人可以分辨出场景中不同材质的外表。

10.6　参考文献

Apodaca, Anthony A., and Larry Gritz, eds. 1999. *Advanced RenderMan: Creating CGI for Motion Pictures*. Morgan Kaufmann.

Barzel, Ronen. 1997. "Lighting Controls for Computer Cinematography." *Journal of Graphics Tools* 2(1), pp. 1～20. Available online at **http://www.acm.org/jgt/papers/Barzel97**

Barzel, Ronen. 1999. "Lighting Controls for Computer Cinematography." In *Advanced RenderMan: Creating CGI for Motion Pictures*, edited by Anthony A. Apodaca and Larry Gritz. Morgan Kaufmann. Code for the chapter was provided by Larry Gritz.

Birn, Jeremy. 2000. *Digital Lighting and Rendering*. New Riders Publishing.

Calahan, Sharon. 1999. "Storytelling through Lighting: A Computer Graphics Perspective." In *Advanced RenderMan: Creating CGI for Motion Pictures*, edited by Anthony A. Apodaca and Larry Gritz. Morgan Kaufmann.

第11章 阴影贴图反走样

Michael Bunnell
NVIDIA
Fabio Pellacini
Pixar Animation Studios

11.1 引言

对电影和电视进行高端渲染时，阴影映射是创建阴影的一种方法。然而，在实时程序中（例如视频游戏），阴影映射存在放大的锯齿状走样问题。这章介绍如何在 shader 中明显地减少阴影贴图的走样。描述如何实现一个“靠近的百分比过滤（percentage-closer filtering）”的简化版本，利用 GPU 的阴影映射硬件，以实时的速率来渲染边缘模糊的反走样阴影。

阴影映射包括把阴影贴图投射到几何体上，然后在每个像素上将阴影贴图的值与从光源察看的深度值进行比较。如果投射使得阴影贴图放大，难看的锯齿状走样会出现在阴影边缘上。通常使用高分辨率的阴影贴图和增加阴影贴图的分辨率来反走样，也就是使用 Stamminger 和 Drettakis 2002 提出的“透视阴影贴图”技术。但是，当光与表面接近于平行的时候，使用“透视阴影贴图”技术和增加阴影贴图分辨率就不起作用了，因为放大倍数接近于无穷大。

高端渲染软件使用“靠近的百分比过滤”技术解决走样问题。

11.2 靠近的百分比过滤

不像正常的纹理，阴影贴图纹理不能通过预过滤来消除走样。取而代之的是多次比较阴影贴图的每个像素，并求其平均值。这个技术叫做“靠近的百分比过滤”（PCF），因为它计算的是靠近光源的表面的百分比，而不是在阴影中的表面的百分比。

最初的 PCF 算法由 Reeves 等人 1987 年给出，要求把遮挡的区域映射到阴影贴图空间中去，并在那个区域随机取样（任意地）。这个算法首先用 REYES 渲染引擎实现了，因此被遮挡的区域是个小四边形。图 11-1 显示那个实现的例子。

在实现中，为了使其更容易和有效地应用，我们对 PCF 算法做了一些细微的改变。不再计算阴影贴图空间中被遮挡的区域，只是简单地在各处

使用一个 4 × 4 个 texel（纹素）的样本块。这个块应该大到能够有效地减少走样，但是不能大到要求大量样本和随机取样的程度。注意取样区域不能对齐于纹理边界。那种对齐不能达到我们想要的反走样效果。

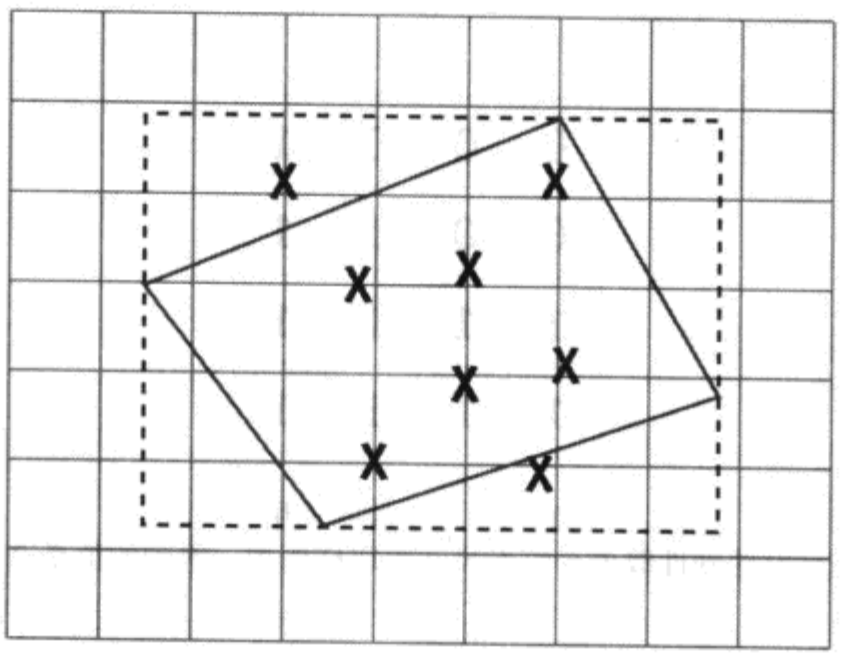


图 11-1　靠近的百分比过滤

第 1 个靠近的百分比过滤的实现要求将微多边形转换到纹理空间来计算采样区域

硬件 shader 是在像素上，而不是在微多边形上工作的，因此，开始时应该把一个代表荧屏像素的四边形转换到阴影贴图空间，以计算采样区域。我们的实现改为使用固定尺寸的样本块。固定尺寸的样本块可以使我们跳过复杂的转换，而且可以计算出精确的阴影百分比，不用随机取样得到近似的结果，见图 11-2。

0	0	0	0	0	0	0	0	1
0	0	0	0	0	0	1	1	1
0	0	0	0	0	1	1	1	1
0	0	0	0	0	1	1	1	1
0	0	0	0	1	1	1	1	1
0	0	0	0	1	1	1	1	1
1	1	1	1	1	1	1	1	1

图 11-2　4 × 4 纹素区域的采样

11.3　平滑滤波的实现

NVIDIA 图形处理器内建用于阴影贴图取样的靠近的百分比过滤器。硬件做 4 个深度比

较，用纹理坐标的小数部分双线性差值求阴影值。阴影结果是一个纹素尺寸样本块在阴影中所占的百分比，见图11-3。单个纹素尺寸的样本块太小，不能有效地消除走样，但是可以平均16个阴影比较值，把块增大为4×4纹素。为了使采样位置分开一个texel单位，x和y的偏移为−1.5、−0.5、0.5和1.5。

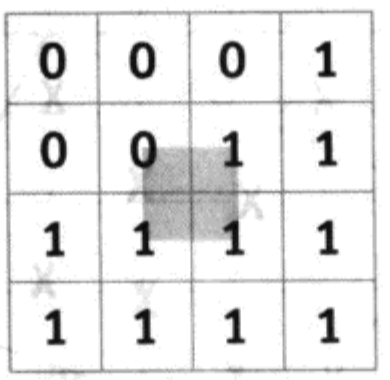


图11-3 硬件的使用
阴影贴图硬件取单个纹素尺寸为取样面积，得到的值是0.75

下面的函数能用来读取一个投影的纹理贴图，所给的偏移以纹素为单位。其中变量texmapscale是一个float2，包含阴影贴图的1/width和1/height。

```
float3 offset_lookup(sampler2D map,
                     float4 loc,
                     float2 offset)
{
  return tex2Dproj(map, float4(loc.xy + offset * texmapscale * loc.w, loc.z, loc.w));
}
```

片元程序的16样本版本的实现如下。

```
float sum = 0;
float x, y;

for (y = -1.5; y <= 1.5; y += 1.0)
  for (x = -1.5; x <= 1.5; x += 1.0)
    sum += offset_lookup(shadowmap, shadowCoord, float2(x, y));

shadowCoeff = sum / 16.0;
```

11.4 较少地取样

强制方法的运行比预期的要好。许多纹理是从纹理高速缓冲区中取得的，因为在那里它们保证是彼此靠近的。但是，如果我们改变每个像素采样的模式，在每个像素中只取4个样就可以得到类似的结果。

4样本技术的结果类似于混合黑白数据渲染灰度图像。保持相同大小的取样区域，但是只在每个像素的16个样本中取4个。4样本组改变了对荧屏位置的依赖。图11-4显示了用

于 shader 的 4 样本取样方式，从每个像素的 16 个可能的样本位置中选择 4 个。

0	3	0	3
2	1	2	1
0	3	0	3
2	1	2	1

图 11-4　shader 的 4 样本取样方式

4 样本的版本可以实现如下。

```
offset = (float)(frac(position.xy * 0.5) > 0.25); // mod
offset.y += offset.x; // y ^= x in floating point
if (offset.y > 1.1)
  offset.y = 0;
shadowCoeff = (offset_lookup(shadowmap, sCoord, offset + float2(-1.5, 0.5)) +
               offset_lookup(shadowmap, sCoord, offset + float2(0.5, 0.5)) +
               offset_lookup(shadowmap, sCoord, offset + float2(-1.5, -1.5)) +
               offset_lookup(shadowmap, sCoord, offset + float2(0.5, -1.5)) ) * 0.25;
```

11.5　工作原理

为什么固定尺寸的样本块能够对阴影反走样？甚至把阴影贴图投射放大到极大时仍然起作用？答案很简单：当放大纹理时，那些纹理贴图样本对于相邻像素也是很近的。如果样本彼此接近，而且采样的区域相对大，就会造成采样区域的很大重叠，那么阴影值之间只可能有非常小的不同。图 11-5 显示了当阴影贴图放大时，相邻像素的采样区之间是如何重叠的。

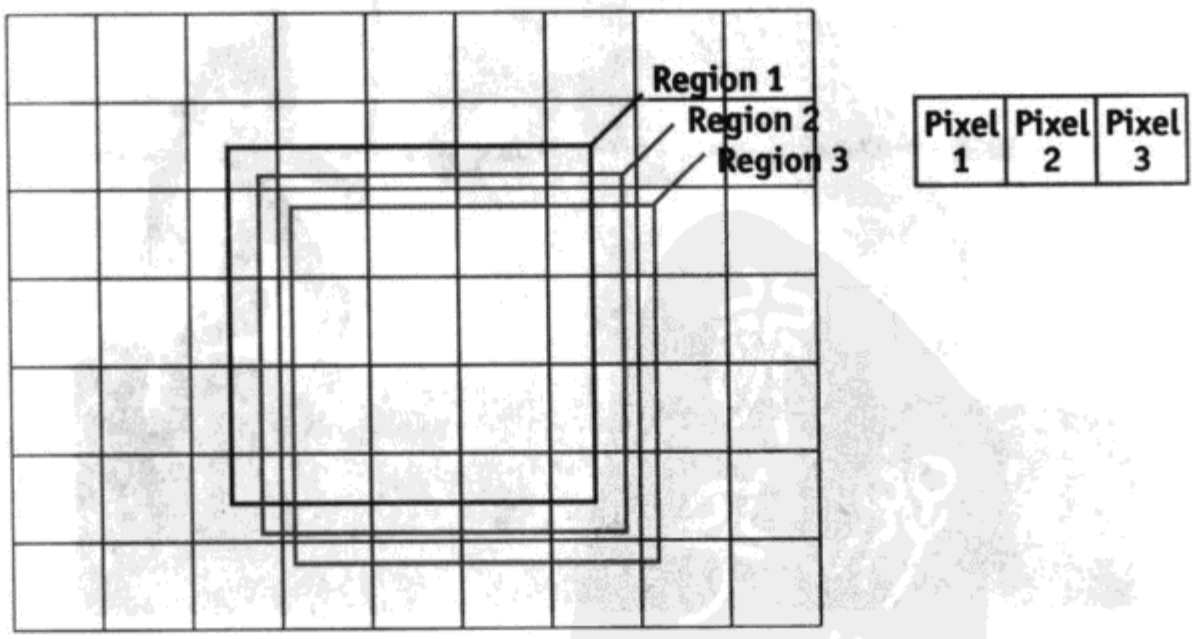


图 11-5　相邻像素的取样区域重叠

阴影贴图放得越大，相邻的像素之间的差异就越小，在阴影区和非阴影区之间的变换就越平滑。硬件用 8bits 的精度计算阴影百分比，甚至在极端放大和高反差的情况下，也总是得到无镶边的平滑阴影变换。如果阴影区彼此非常接近，在每个成分的 8bits 的输出中，阴影值只会相差很少 bit（位）。这在图 11-6、图 11-7 和图 11-8 中得以说明，它们分别以每像素 1、4 和 16 个样本来表示一个武士模型的阴影。图 11-9 表示在 3 种情形下，武士拇指阴影的放大图。注意在 16 个样本的情况下阴影质量得到极大的改善。

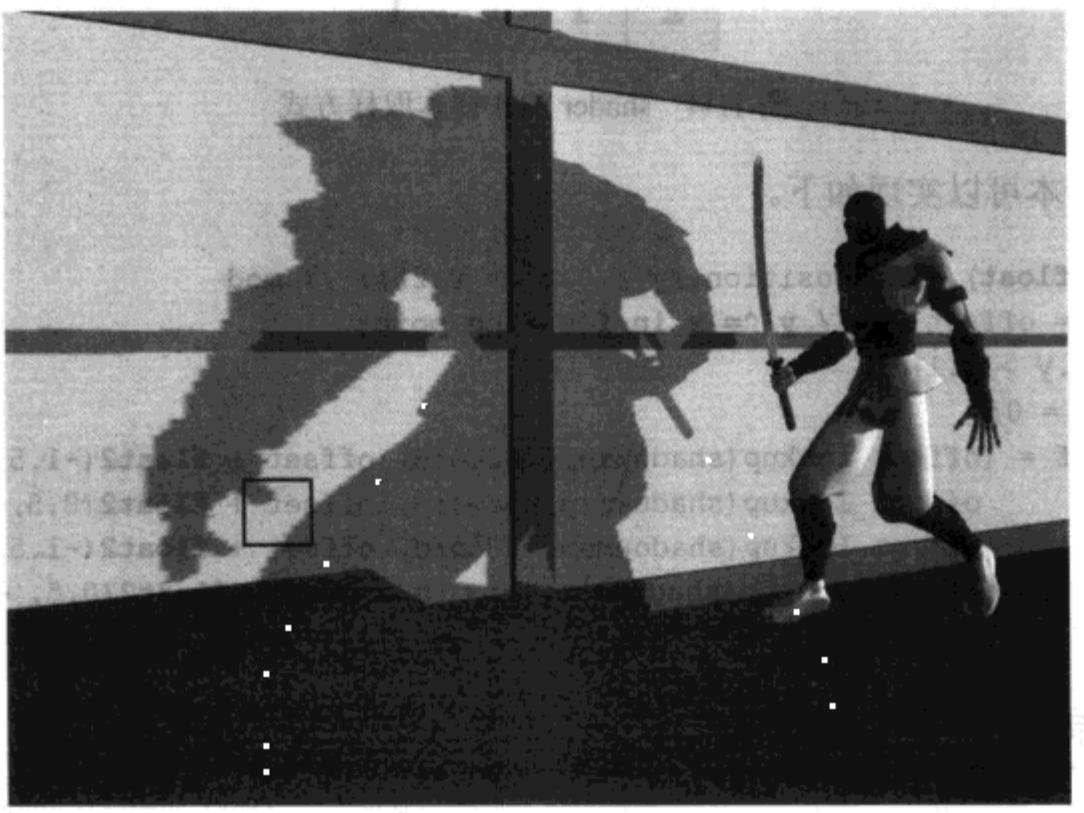


图 11-6 每个像素取 1 个样本的武士影子

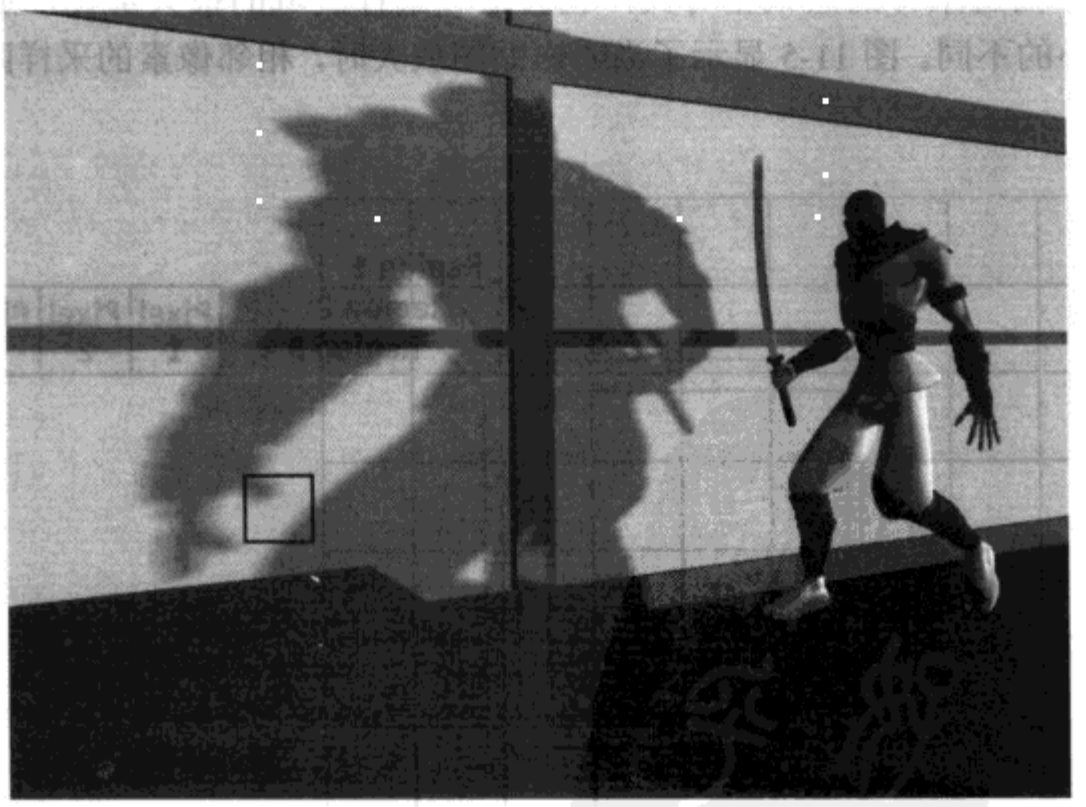


图 11-7 每个像素取 4 个样本的武士影子

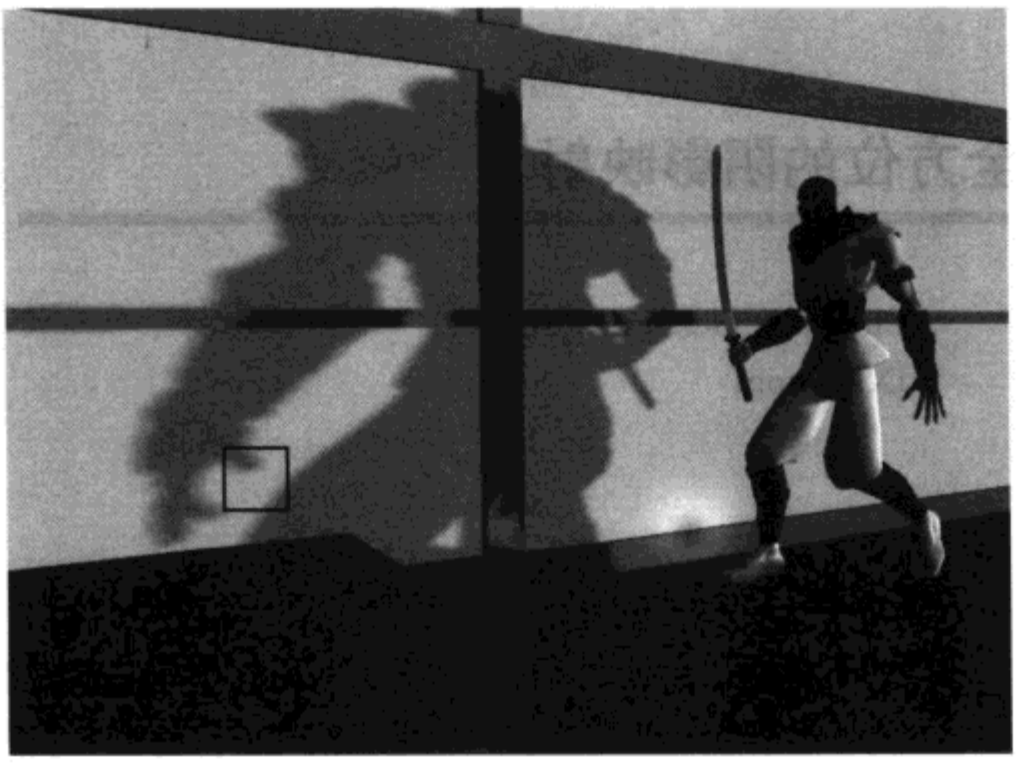
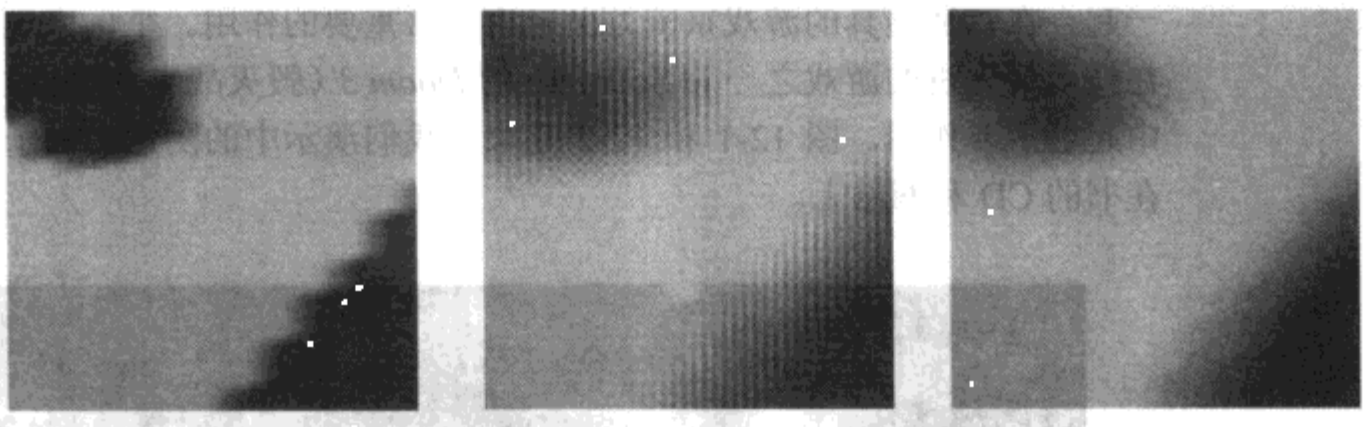


图 11-8　每个像素取 16 个样本的武士影子



（a）1 个样品的图像　　（b）4 个样品的图像　　（c）16 个样品的图像

图 11-9　放大的阴影

当图 11-6、11-7 和 11-8 的武士拇指的影子放大时，其质量的差异很明显

11.6　小结

阴影映射是渲染阴影的常见方法，但是它存在走样问题。我们能够通过平均多个阴影贴图的值，大大减少阴影贴图的走样。如果利用图形处理器的阴影映射硬件，而且使用有效的采样技术，我们就能以很高的帧速率渲染出边界模糊、反走样的阴影。

11.7　参考文献

Fernando, Randima, and Mark Kilgard. 2003. *The Cg Tutorial.* Addison-Wesley. This introduction to the Cg language has a good section on shadow mapping.

Reeves, W. T, D. H. Salesin, and P. L. Cook. 1987. "Rendering Antialiased Shadows with Depth Maps. " *Computer Graphics* 21(4) (Proceedings of SIGGRAPH 87).

Stamminger, Marc, and George Drettakis. 2002. "Perspective Shadow Maps." In *Proceedings of SIGGRAPH 2002,* pp. 557～562.

第 12 章 全方位的阴影映射

Philipp S. Gerasimov
iXBT.com

12.1 引言

实时计算机图形最困难的问题之一是产生高质量阴影。而阴影是得到逼真图形的最重要因素之一。在计算机产生的场景中，阴影增强了我们对物体的感觉及物体之间的关系。在电脑游戏中，阴影与光照、音乐和特效一起，在营造逼真的游戏氛围方面起着非常重要的作用。举例来说，阴影在技术最先进的游戏之一 id Software 的 *Doom 3*（毁灭战士 3）的故事主线中起着主要作用。图 12-1 和 12-2 是来自我们演示中的阴影的例子，它附在书的 CD 和网站上。

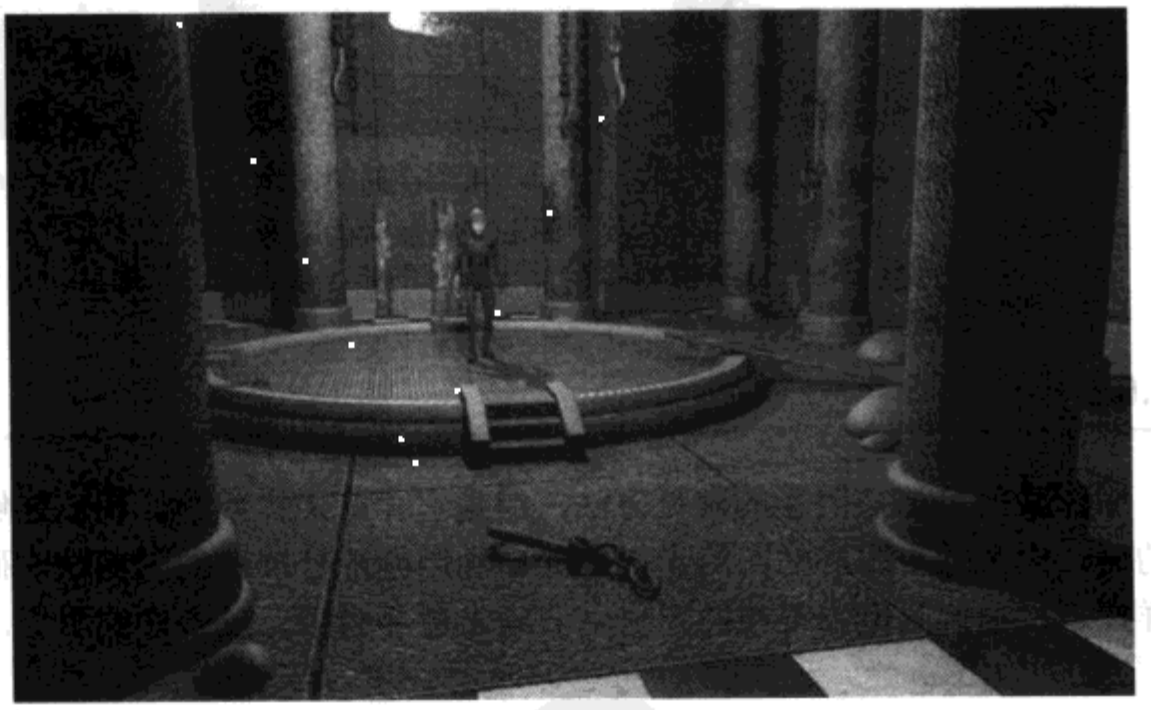


图 12-1 演示游戏的截图，展示光源从人物头顶飞过的效果

现在 GPU 所创建的图像以前只有专业的 3D 离线渲染程序才能产生。现代 GPU 的几何处理器每帧可以处理几百万个图元（primitive），满足了设计复杂世界的需要。随着逐像素着色的出现，使用复杂的数学和物理学的光照模型，可以渲染出逼真的材质来。

在实时计算机图形学中产生可见阴影的两个流行方法是：模板阴影和阴影映射。

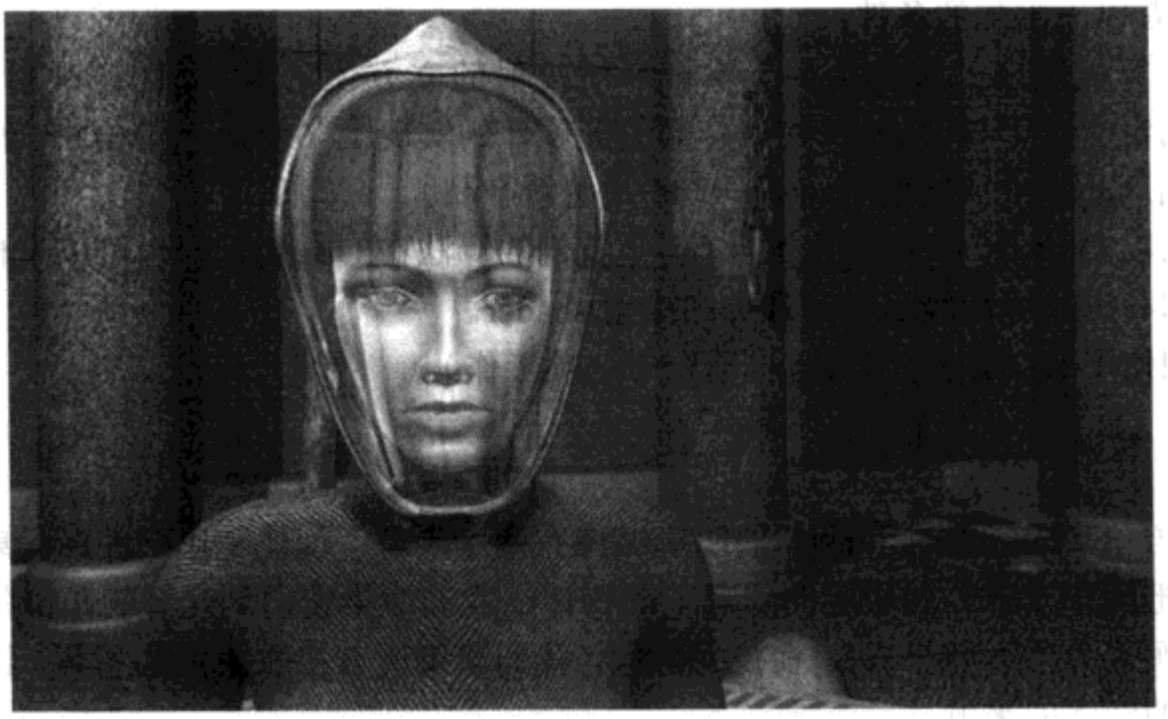


图 12-2　人物特写镜头

12.1.1　模板阴影

在 *Doom 3* 中所演示的模板阴影方法广泛地用于游戏开发。该方法的优点是它得到大量 GPU 的支持（它对硬件的要求仅仅是支持 8 位的模板缓存）、独立于光源的种类、产生的阴影质量很高。然而，模板阴影方法也有一些严重的缺点：它严重依赖于 CPU、只能产生清晰的影子、需要很高的填充率（这意味着，即使图形处理器支持这个技术，也运行得不好），而且它不能够与硬件镶嵌（hardware-tessellated）的表面一起使用。

12.1.2　阴影映射

1978 年 Lance Williams（Williams 1978）把阴影映射的算法引入计算机图形学。今天多数好莱坞电影都在使用这个方法，包括计算机绘图和特效。为了计算阴影，阴影映射在场景几何体上投射特殊的动态创建的纹理。它可以渲染清晰的和模糊的影子，以及由不同类型的光源产生的阴影，它还可以与硬件镶嵌的表面以及 GPU 动画的网格（例如蒙皮网格）一起使用。

有不少的图形处理器制造商（包括 NVIDIA）的硬件直接地支持阴影映射，并且他们许诺将来还要增加对这方面的支持。NVIDIA 的 GeForce3、GeForce4 Ti 和所有的 GeForce FX（以及更新的）GPU，都通过 DirectX 和 OpenGL 支持硬件阴影映射。由于 NVIDIA CineFX 结构支持真实浮点精度和浮点纹理格式的长片元程序，阴影渲染的质量有可能提高到新的水平。

12.2　阴影映射的算法

12.2.1　条件

阴影映射使我们看到由不同类型的光源产生的阴影，例如方向光、点光源和聚光灯。光源的类型规定了需要使用的技术。这章把重点集中于由点光源产生的可见阴影。点光源广泛地用于电脑游戏，由这种光源照明的物体所投射的阴影质量非常重要。

我们还有以下附加的条件。

- 在计算机图形的传统上用于实现环境反射的立方体贴图，是阴影映射算法的积木。
- 我们将专注于与 DirectX 9 兼容的图形硬件的实现上（例如，NVIDIA 的 GeForce FX 系列产品，或 ATI 的 Radeon 9500 及更新产品）。
- 我们以 DirectX 作为图形 API（但是我们的算法在 OpenGL 中也可以工作）。
- 我们利用一种高级语言写顶点和像素 shader。
- 我们使用 HLSL（但是 Cg 也适合于这项工作）。

12.2.2 算法

使用全方位（omnidirectional）的阴影贴图有两个步骤：创建阴影贴图和投射。在创建阶段，对所有把阴影投射到阴影贴图纹理上的物体，我们渲染它们到光源的距离的平方（在后面将会看到距离为什么要平方）。在投射阶段，我们渲染所有接收阴影的物体，比较所渲染的像素到光源的距离的平方。

下面的技术填充立方体贴图所有的 6 个面，全部方向为：$+x$、$-x$、$+y$、$-y$、$+z$、$-z$。阴影贴图或是预先计算好（对于静态的场景），或是每帧重新渲染。我们专注于介绍每帧重新渲染的完全动态的阴影贴图。对于每个光源各个物体都投射一个影子并接收一个阴影。各物体都有自阴影（self-shadow）。对所有的光源使用一个阴影贴图，为每个光源执行一个遍，用多遍渲染创建一个图像。

列表 12-1 是这个算法的伪代码例子。

列表 12-1 全方位的阴影映射算法的伪代码

```
for (iLight = 0; iLight < NumberOfLights; iLight++) {
  // 填充阴影映影
  for (iObject = 0; iObject < NumberOfObjects; iObject++) {
    RenderObjectToShadowMap(iLight, iObject);
  }

  // 光照和阴影映射
  for (iObject = 0; iObject < NumberOfObjects; iObject++) {
    LightAndShadeObject (iLight, iObject);
  }
}
```

因为我们使用多遍运算法则（即，为每个光源做一个通道），所有的物体必须复合进帧缓存内。为了减少浪费和改善性能，我们先只渲染一遍深度。这个标准的技术保证后来的所有光照遍只在可见的像素上运行。对只渲染深度非常快（许多 GeForce FX GPU 在只渲染深度时可以达到双倍速度），因此它只需要很小的开销。因为透明物体不更新深度缓冲，所以只渲染深度的遍中不渲染它。见列表 12-2。

列表 12-2 只渲染深度

```
// 清除颜色和深度缓冲
ClearAllBuffers();

// 填充 z-缓冲
```

```
for (iObject = 0; iObject < NumberOfObjects; iObject++) {
  RenderObjectToZBufferOnly (iObject);
}
```

12.2.3　纹理格式

纹理格式的类型是这个算法的重要因素。我们考虑两个格式：把深度值紧缩/释放进彩色通道的浮点纹理及 32 位整数 RGBA 纹理。

浮点纹理格式对阴影映射是理想的，因为它允许使用高精度深度值。但是这种纹理比整数 RGBA 纹理慢得多，而且只有少数 GPU 支持它。与之相反，32-bit 整数 RGBA 纹理运行快，而且大多数 3D 硬件支持它。

但是，为了保证计算的高精度，必须把深度值紧缩进纹理的彩色通道，而为阴影映射进行深度比较时，又必须释放出每个值。下面介绍两种方法，可以选择比较方便的一个方法使用。

12.2.4　阴影贴图的尺寸

阴影贴图的尺寸影响阴影质量和渲染速度。尺寸取决于目标硬件的能力、所需要的质量和影子相对于摄像机的位置。当然，一般阴影贴图越大，产生的结果越好。

因为我们使用立方体贴图纹理，所以必须记住有 6 个彩色表面和额外的 z-缓冲。对于 32-bit 的纹理和 1024 × 1024 分辨率，我们将需要 4 × (6 + 1) × 1024 × 1024 字节的图像存储器，即 28MB! 这说明对所有的光源使用单个阴影贴图是多么重要。

12.3 节将考查算法的每个步骤。

12.2.5　几何体的数值范围

为了将渲染的失真减到最小，把所有的几何体放入一个 [−0.5, +0.5] 范围（或 [0, 1]）。这增加了计算的准确度，特别在使用 16bit 的精度和整数纹理时尤为如此。我们可以在载入时或在顶点 shader 中缩放几何图形，比如说使用下面的顶点 shader 编码：

```
o.vPositionWorld = mul(vPosition, matWorld) * fGeometryScale;
```

12.3　实现

12.3.1　系统需求

以下是我们的系统需求：

- 硬件支持顶点 shader（vs.1.1+）和像素 shader（ps.2.0+）；
- 硬件能够渲染到立方体贴图；
- 如果使用浮点纹理，则支持单一成分的立方体贴图的浮点纹理格式，例如 D3DFMT_R16F 或 D3DFMT_R32F，带有 D3DUSAGE_RENDERTARGET 标记；
- 如果使用整数纹理，则支持 D3DFMT_A8R8G8B8 立方体纹理格式，带有 D3DUSAGE_RENDERTARGET 标记。

12.3.2 资源创建

使用以下一些有用的 Direct3D 程序库函数，我们可以创建所有需要的物体和纹理（阴影贴图纹理、深度缓冲区和 shader）。

```
D3DXCreateCubeTexture()
D3DXCreateRenderToEnvMap()
D3DXCreateEffectFromFile()
```

12.3.3 渲染阶段 1：渲染到阴影贴图

下面我们进入阴影贴图的渲染。按照以下要求，我们将会从光源的观点出发，把物体渲染进立方体贴图的每个面：

- 视域必须是 90 度。
- 必须为每个面适当地创建观察矩阵。我们可以从光源视点的上、下、东、西、南、北的矢量创建这些矩阵。

顶点 Shader

在顶点 shader 中，可以写出经过缩放的顶点的世界空间位置，以供像素 shader 使用。或者写出光的方向以节省一条用于计算世界空间的光矢量的像素 shader 指令。

像素 shader

我们既可以使用浮点纹理，也可以使用整数纹理。

- 当使用浮点纹理的时候，我们把从像素到光源的距离的平方，写入以下的像素 shader 代码中：

```
return dot(vLight, vLight)
```

为什么使用距离的平方？因为平方是比较快的运算，这样只是做乘法，而免去了计算平方根。在填充阴影贴图的时候，我们使用距离的平方；而在基本渲染通道中访问阴影贴图的时候，也使用距离的平方。这样两次节省像素 shader 指令。但是使用距离的平方能引起一些精度问题，因此使用时要小心。

- 当使用整数纹理的时候，我们需要紧缩距离的平方值，并且把它写进颜色通道。有两个方法将浮点数填进整数纹理：

1.
```
Out.r = SquaredDistance * 2^0
Out.g = SquaredDistance * 2^8
Out.b = SquaredDistance * 2^16
Out.a = SquaredDistance * 2^24
float4 vPack = {1.0f, 256.0f, 65536.0, 16777216.0f};
return vPack * dot(vLight, vLight);
```

2.
```
Out.r = floor(fDepth) / 256.0;
Out.g = frac(fDepth);
float fDepth = dot(vLight, vLight);
```

```
return float(floor(fDepth) / 256.0, frac(fDepth), frac(fDepth), frac(fDepth));
```

通过把 frac（fDepth）写进绿色和 alpha 通道，我们可以节省一条像素 shader 指令（否则，需要另一个指令填充这些通道）：

```
mov r2.gba, r0.g // r0.g 包含 frac(fDepth)
```

方法 1 计算比较省时，但是第 2 个方法精度更高。

12.3.4　渲染阶段 2：基本渲染

基本渲染阶段分为两个主要部分。

1. 只将物体渲染进 z-缓冲（只有 z 的遍），需要完成以下步骤：
   a. 禁止渲染颜色通道；
   b. 启用渲染 z-缓冲区；
   c. 把所有的物体渲染进 z-缓冲区内（仅此而已）。
2. 为每个光源制作一个着色遍（lighting times shadow）。

12.3.5　光照计算

我们可以使用任何光照模型（例如逐像素的 Phong、Blinn 或 Oren-Nayar），为每个像素计算由光源产生的光照。

12.3.6　阴影的计算

通过以下步骤计算阴影。

1. 计算从当前的像素到光源的距离的平方。
2. 把阴影贴图纹理投射到当前的像素上。
3. 从当前的像素取出阴影贴图纹理值。
4. 把计算的距离值与取出的阴影贴图值作比较，决定该像素是否在阴影中。

对于浮点纹理，我们使用所取出的纹理样品的 x 部分。以下是像素 shader 的代码：

```
float fDepth = fDistSquared - fDepthBias;
float3 vShadowSample = texCUBE(ShadowMapSampler, -vLight.xyz);
float fShadow = (fDepth - vShadowSample.x < 0.0f) ? 1.0f : 0.0f;
```

fDistSquared 是以前在像素 shader 中计算的。为了渲染整数纹理，我们必须从所取的纹理样品的颜色通道取值。

1. `DepthValue = ShadowSample.r / 1 + ShadowSample.g / 256 + ShadowSample.b / 65536 + ShadowSample.a / 16777216`
2. `DepthValue = ShadowSample.r * 256 + ShadowSample.g`

以下是像素 shader 代码：

```
float fDepth = fDistSquared - fDepthBias;
float4 vShadowSample = texCUBE(ShadowMapSampler, -vLight.xyz);
float fShadow = (fDepth - dot(vShadowSample, vUnpack) < 0.0f) ? 1.0f : 0.0f;
```

12.3.7 技巧和窍门

1．有几个不同的方法能用来计算深度偏差：

- fDistSquared – vShadowSample.x ——很可能出现走样。
- (fDistSquared – DepthBias) – vShadowSample.x ——距离的平方是非线性的。
- (fDistSquared * DepthBias) – vShadowSample.x ——这种方法实际上工作得最好。

2．光方向：把光方向的计算挪入顶点 shader 内。光方向是线性的，并且能容易地逐个顶点地计算。

3．相反的光方向：我们需要从阴影贴图中取出相反的光方向。但是 texld 像素 shader 指令不支持“求负”修饰语，因此如果我们使用 texCUBE（ShadowMapSampler，–vLight.xyz），会随着每次取出而需要一条额外的 add 指令。所以我们把这个计算移进顶点 shader 内，并且插值求的是–vLight.xyz，而不是 vLight.xyz。

4．HLSL 和 Cg shader 的预处理指示：使用预处理指示不同的选择，例如浮点/整数纹理、清晰的/模糊的阴影，以及全部/一半/固定的精度，从而减少需要写的 shader 数量。

5．像素 shader 的精度：对于大部分阴影计算使用一半的精度就够了，而且在一些硬件上还会得到额外的速度。如果发现失真，则使用完全的精度。

12.3.8 最终的着色遍（Lighting × Shadow）

最后一步是基于所计算的光照和阴影写像素颜色值。对于每个光源，通过对所有物体的阴影的写入和着色遍，把计算的光照加到背面缓冲中。当处理完所有的光源时，我们就得到动态光照和阴影的场景。

12.4 添加模糊的阴影

看看我们的场景，可以注意到影子的边缘是失真的和很“清晰”的。失真的程度取决于投射时阴影贴图的大小和放大的倍数。为了减少这些失真，我们从阴影贴图中取多个样品将其平均，从而创建一个比较“模糊”的阴影。因为真实的光源并非理想点光源，这样做也可以提供比较真实的阴影。

列表 12-3 列出了一些样品代码。

列表 12-3 它产生一个较模糊的阴影

```
float fShadow = 0;

for (int i = 0; i < 4; i++) {
  float3 vLightDirection = -vLight.xyz + vFilter[i];
  float4 vShadowSample = texCUBE(ShadowMapSampler, vLightDirection);
  fShadow += (fDepth - vShadowSample.x < 0.0f) ? 0.25f : 0.0f;
}
```

注意我们先比较距离的平方，然后平均比较的结果。这叫做“靠近的百分比过滤”，而且是对多个阴影贴图检测进行平均的正确方法（见本书的第 11 章“阴影贴图的反走样”对这个技术的详细讨论）。

当计算-*vLight. xyz* + *vFilter* [*i*]的值时，如果把它移进顶点 shader 之内，就能节省一些像素 shader 的指令。

如果给 *vFilter*[*i*]选择不同范围的值，我们将得到模糊程度不同的阴影，从轻微反走样效果的阴影到非常模糊的阴影。我们定义的过滤器核心越大，避免带状走样所需要的样本数就越多。显然，更多采样等于处理更多的指令和获取更多的纹理，这会减小受限于 shader 情况下的运行性能。虽然这个技术可以产生“更模糊的”阴影效果，但由于没有考虑遮光板、接收器和光源尺寸之间的相对关系，阴影当然不是准确的模糊阴影（关于更多模糊阴影信息，见第 13 章“使用遮挡间隔贴图产生模糊阴影”）。

12.5　小结

DirectX 9 级别硬件的新能力，使得改善视觉质量的新算法变得可能和容易实现。使用硬件 shader，我们可以创建逼真的、动态的、任意数量的点光源所产生的阴影，甚至能实现基本上堪称“模糊的”阴影。

使用目前的第一代 DirectX 9 级硬件，运算起来实际上还不够快（虽然它是实时的）。那是因为从光源观点而来的大量渲染，以及“模糊的”阴影效果所必需的长像素 shader 造成的。但是快得多的图形硬件马上就会出现，而且它的先进性能会使这些运算法则在实时的游戏中实现。

12.6　参考文献

Williams, Lance. 1978. “Casting Curved Shadows on Curved Surfaces.” In *Proceedings of the 5th Annual Conference on Computer Graphics and Interactive Techniques*, pp.270～274.

非常感谢 NVIDIA 的 Chris Wynn 和 John Spitzer 以及 ATI 的 Guennadi Riguer 给予的帮助。

第 13 章 使用遮挡区间映射产生模糊的阴影

William Donnelly
University of Waterloo
Joe Demers
NVIDIA

这章将介绍一种渲染模糊阴影的技术，我们称之为遮挡区间映射。NVIDIA GeForce FX 5900 的演示 Last Chance Gas 中的模糊阴影就是用遮挡区间映射制作的。见图 13-1。我们之所以称这种技术为遮挡区间映射，是因为使用纹理贴图存储这种光源可见、而本身被遮挡的区间。在满足算法所要求的条件时，遮挡区间映射以很高的帧速率达到令人印象深刻的视觉结果。

图 13-1 Last Chance Gas 演示

13.1 加油站

GeForce FX 5900 演示 Last Chance Gas 的目标之一是用正确的户外光照创建场景。户外光照的一个重要方面是来自太阳的模糊阴影。不像由阴影贴图或模板阴影体产生的清晰的影子，模糊的阴影有一个半影区域，它在阴影区和非阴影区之间给出一个平滑的过渡。一个正确的半影能够制作更加逼真的阴影，并且给予用户更好的空间感，使他们感到场景更加逼真，易于产生身临其境的感觉。

当我们开始写样本的时候，没有找到任何适当的实时模糊阴影的算法，

虽然在这方面已经做了大量研究。

模糊阴影的问题非常困难，因此需要简化问题，使它可以实时地运行。

给出的加油站场景是静态的，我们本打算使用一种预计算可见度技术，例如球形调和光照模型（Sloan et al. 2002）。可惜它不适于我们的目的，球形调和光照适用的是非常低频的光照，不适用于像太阳那样小面积的光源。所以我们才开发遮挡区间映射这种新的预计算可见度的技术，它能够实时地排序太阳所产生的阴影。我们通过把问题简化为在固定轨道上的线性光源来达到目的。

这个算法有些类似于水平映射（Max 1988）。但是水平映射覆盖整个可见的半球，而遮挡区间映射则不同，它只对可见半球上沿着一条路径的光源起作用。遮挡区间映射也能处理任意的几何体，不只是高度场。

遮挡区间映射并不是对模糊阴影的一般解决办法，但是它对静态的户外环境的阴影是有用的。正如其他预计算可见度的技术一样，遮挡区间映射也是离线存储所有可见度信息，所以不适用于移动物体或任意光源。

13.2　算法

假设光源是具有固定轨道的太阳，假设要为这个光源预计算清晰的影子。我们可以将那个影子表达为一个可见性函数，当点在影子中时，它的值是 0，当它被照明的时候，它的值是 1。在渲染的时候，计算这个可见度函数，再将结果乘以一个着色计算，得出最终的颜色值。

可见度函数是一个 3 个变量的函数：2 个物体表面的空间变量，1 个时间变量。虽然可以用 3D 纹理存储这个函数，但是需要极大的存储空间。由于可见度函数的所有值是 0 或 1，因此我们使用一个类似于游程编码的方法来存储这个函数。对于每个点，我们找到它在时间域的上升边和下降边。它们分别对应于太阳出现和消失的时间。我们定义“上升”矢量为所有上升的边的矢量，而“下降”为所有下降的边的矢量，见图 13-2。

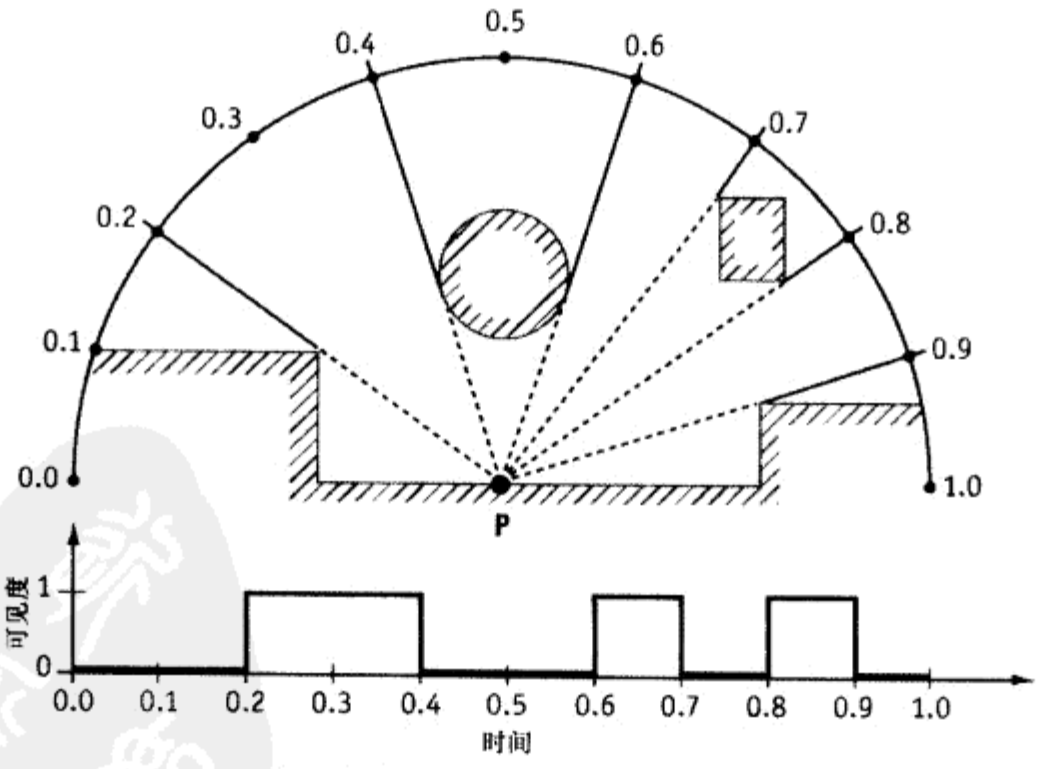


图 13-2　在一个场景上的单个点 **P** 及它的可见度函数
这个点的上升矢量是 $\boldsymbol{R}$ = (0.2, 0.6, 0.8)，而对应的下降矢量是 $\boldsymbol{F}$ = (0.4, 0.7, 0.9)

现在有了预计算清晰影子的可见度所需要的一切：给定一个上升矢量、一个下降矢量和时间，我们就可以计算可见度函数来确定点是否在影子中。

为了将以上计算变成一个模糊阴影的算法，沿着光源的轨道扩展它。现在不再计算点光源的影子，而是计算线光源产生的影子。假设在一个时间段 $[t-\frac{1}{2}dt,\ t+\frac{1}{2}dt]$ 中，光源在空间掠过一段曲线。如果求出在时段 $[t-\frac{1}{2}dt,\ t+\frac{1}{2}dt]$ 上的平均照明，我们就计算了线光源产生的正确的影子。这意味着可以用渲染清晰影子的相同信息，来渲染模糊的阴影，见图 13-3。

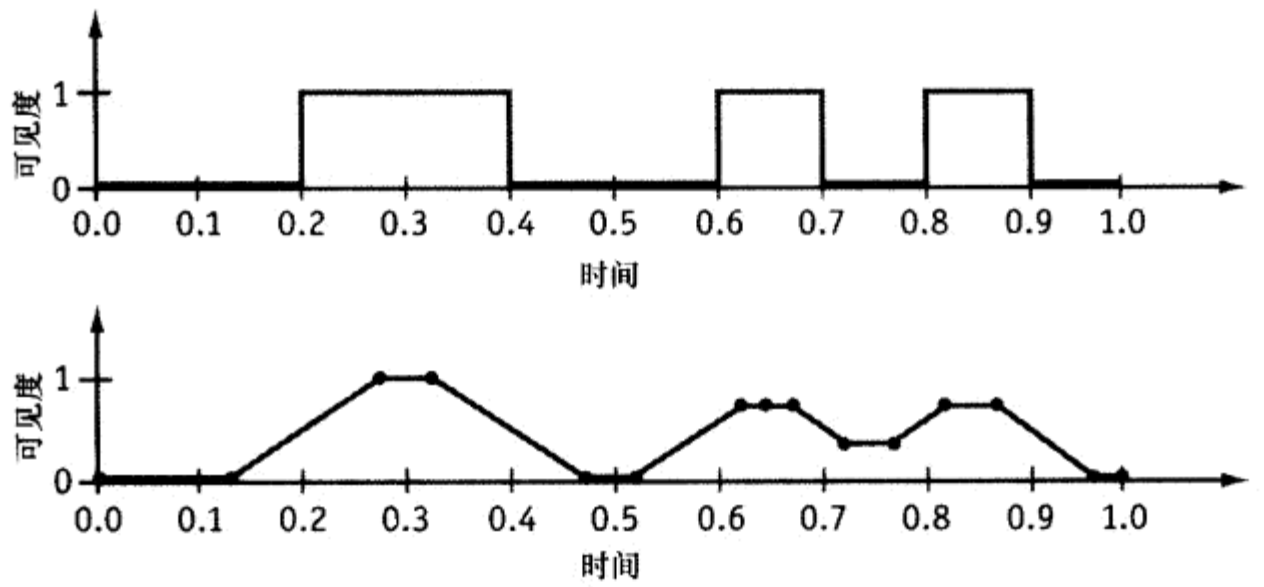


图 13-3 点光源可见度函数和对应的线性光可见度函数
下面的图像是通过模糊上面的图像产生的

13.3 创建映射

为了要产生遮挡区间映射，应该从光源轨道上的每个点，到遮挡区间映射的每个像素计算可见度函数。具体做法是按照曲线上各点的顺序，跟踪来自遮挡区间映射像素的每道光线到每个点，并求出每个可见度函数的上升边和下降边。我们以 8 个 bit 存储上升边和下降边的值。这样，256 道光线对区间中的所有像素就完全足够了。为了减少存储的信息量，当点的法线背离光源时，不存储上升边和下降边的值。因为背面的像素总是黑的，这会节省空间，又不影响渲染的图像。

任何可编程的光线跟踪引擎都能计算可见度函数。我们使用 mental ray 软件包中的自定义 shader 计算所有的场景数据。因为必须为遮挡区间映射的每个像素投射 256 道光线，所以计算这些纹理可能是极大的消耗。对于 Last Chance Gas，计算整个场景的阴影需要好几个小时。

我们把“上升”矢量和“下降”矢量分别存储在两组彩色纹理中，每个纹理有 4 个通道。“上升”纹理存储每个通道中的光区间的开始，而“下降”纹理存储光区间的结束。在介绍以遮挡区间映射渲染的算法时，就会明白这样区分纹理的原因。

为了减轻遮挡区间映射对存储器的额外需求，我们以一半大小的彩色纹理计算映射，这减少了 4 倍的存储需求。因为阴影的模糊有一种模糊效果可以补偿较低的分辨率，所以可以减小遮挡区间映射的分辨率。有时，即使映射的分辨率小于彩色纹理的一半，也不会引起明显的走样。

为了参数化物体，我们使用物体的纹理坐标。因为遮挡区间映射的像素存储的信息与位置相关，所以物体的纹理坐标不能够重叠。对于有重复排列纹理的物体，这意味着必须计算一组新的惟一的纹理坐标。

13.4　渲染

渲染时，必须在参数值的区间上求可见度函数的平均值。用数学术语来说，这意味着求宽度为 dt 的窗户函数的卷积。计算的公式是：

$$V_{LinearLight}(t)=\int_0^1 V_{PointLight}(u)W_{dt}(t-u)\mathrm{d}u$$

这里 $V_{PointLight}$ 是可见度函数，而 W_{dt} 是一个宽度为 dt 的盒过滤器，$-dt/2<t<dt/2$ 时，$W_{dt}(t)=1/dt$，其他情况下，$W(t)=0$。给定“上升”矢量 $\boldsymbol{R}=(R_1, R_2, \cdots, R_n)$，而“下降”矢量 $\boldsymbol{F}=(F_1, F_2, \cdots, F_n)$，我们按下式扩展 $V_{PointLight}$ 为：

$$V_{PointLight}(t)=\sum_{i=1}^{n} B(R_i, F_i, t)$$

式中 $B(a, b, t)$是箱函数，$a<t<b$ 时，定义 $B(a, b, t)=1$，其他情况下，$B(a, b, t)=0$。我们现在能按下式计算 $V_{LinearLight}$：

$$\begin{aligned}
V_{LinearLight}(t) &= \int_0^1 \sum_{i=1}^{n} B(R_i, F_i, u)W_{dt}(t-u)\mathrm{d}u \\
&= \sum_{i=1}^{n} \int_0^1 B(R_i, F_i, u)W_{dt}(t-u)\mathrm{d}u \\
&= \sum_{i=1}^{n} \int_{\max\left(t-\frac{1}{2}dt, R_i\right)}^{\min\left(t+\frac{1}{2}dt, F_i\right)} \frac{1}{dt}\mathrm{d}u \\
&= \sum_{i=1}^{n} \frac{\max\left(0, \min\left(t+\frac{1}{2}dt, F_i\right)-\max\left(t-\frac{1}{2}dt, R_i\right)\right)}{dt}
\end{aligned}$$

利用上述的等式，我们可以只通过求最小、最大，并将结果相减，就能很容易地为一对“上升边”和“下降边”计算出模糊的阴影。我们还能进一步优化它。因为 shader 指令在四元分矢量上操作，可以同时实施 4 个运算，只消耗一个运算开销。这就是把“上升”和“下

降”装填到不同纹理的原因。最后的 Cg 代码显示在列表 13-1 中。

列表 13-1 使用遮挡区间映射计算模糊阴影的函数

```
half softshadow(sampler2D riseTexture,
                sampler2D fallTexture,
                float2 texCoord,
                half intervalStart,
                half intervalEnd,
                half intervalInverseWidth)
{
  half4 rise = h4tex2D(riseTexture, texCoord);
  half4 fall = h4tex2D(fallTexture, texCoord);
  half4 minTerm = min(fall, intervalEnd);
  half4 maxTerm = max(rise, intervalStart);
  return dot(intervalInverseWidth, saturate(minTerm - maxTerm));
}
```

用 *saturate*(*x*)代替 max(0, *x*)。这两个操作是等价的，原因是可见光源区间的宽度总是小于 1。我们选择 *saturate*(*x*)代替 max(0, *x*)，是因为它能被应用为一个输出修饰语，节省一个指令。我们的计算超出固定精度范围，但是又不需要完全的 32 位浮点数，因此使用 16 位的半数据类型，它完全可以满足要求。

在列表 13-1 中最后一行的点积不是用于平常的几何目的，使用它是为了能够同时进行光源宽度的除法，并把平行计算的阴影值加在一起。我们把 intervalStart = $t - 1/2dt$、intervalEnd = $t + 1/2dt$ 和 intervalInverseWidth = $1/dt$ 的值带入 shader。

这个 shader 在 GeForce FX 上编译为只有 6 个汇编代码的指令：2 个纹理查询、1 个最小、1 个最大、1 个带有饱和修饰语的减法和 1 个点积。函数计算阴影的 4 个区间的宽度。如果有多个“上升”和“下降”纹理，就多次调用函数，并且把结果加合起来。

13.5 局限性

正如先前讨论的，这个技术只对沿固定轨道传播的单条光线的静态场景适用。这意味着它对人物和其他动态物体的阴影无效，但是它适用于静态户外环境中的阴影。

因为遮挡区间映射对每个通道需要 8 位的精度，纹理压缩将导致视觉效果失真。因此，必须禁用纹理压缩，这就增加了纹理用量。这种增加使遮挡区间映射的分辨率不得不降低。遮挡区间映射上的间断意味着，即使双线过滤也会产生失真。结果，在遮挡区间映射上不得不禁用任何类型的纹理过滤。这使得阴影显出斑驳的花纹。所幸因为阴影的模糊（smoothness），这种效果不像在细致的彩色纹理上那样引人注意。

用线光源近似太阳也会导致视觉失真。如果靠近地看阴影边界，将会发现，在平行于光源路径方向上的阴影较模糊，而在垂直方向上的阴影则比较清晰。所幸的是，除非光源非常大，否则这个效果很细微。对于我们所使用的宽度范围，这种效果还不引人注意。Heidrich 等（2000）使用线光源近似面积光源，发现线光源的阴影与真实的面积光源的阴影非常像。见图 13-4。

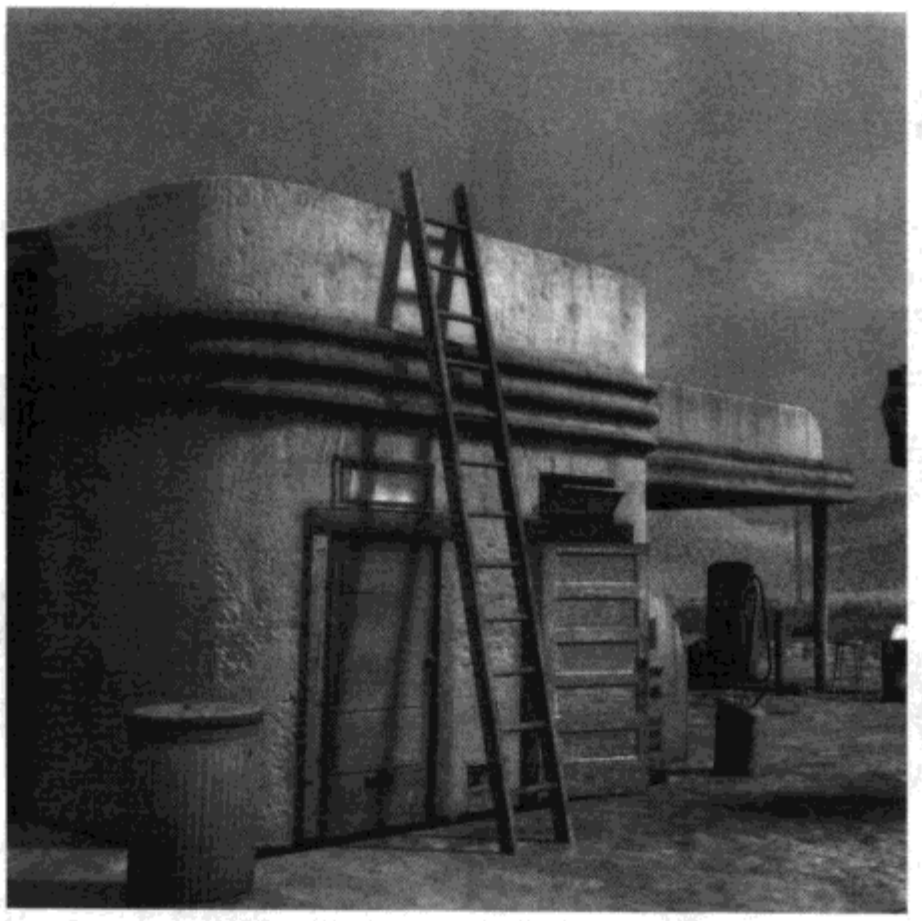


图 13-4　照明梯子
梯子影子的垂直方面比水平方面更模糊，在极端情况下，这种效果没有减弱影子的模糊性

13.6　小结

实时模糊阴影的渲染是非常困难的问题。图 13-5 和 13-6 显示了两个相关细节的例子。预计算可见度技术利用加在场景和光源的假设产生模糊阴影。在遮挡区间映射的情况下，损失运行性能而获得模糊阴影的算法可以在静态场景上实时运行。用遮挡区间映射代替静态光源映射，可以得到从日出到日落光照明变化的动态效果，如 Last Chance Gas 中所示。

图 13-5　加油站入口
注意加油站墙上的阴影在图像的左上方有清楚的边界，但是它朝着右下方变得模糊。这种相联系的清晰和模糊的变化是真实模糊阴影的重要性质，这是由遮挡区间映射正确地得到的

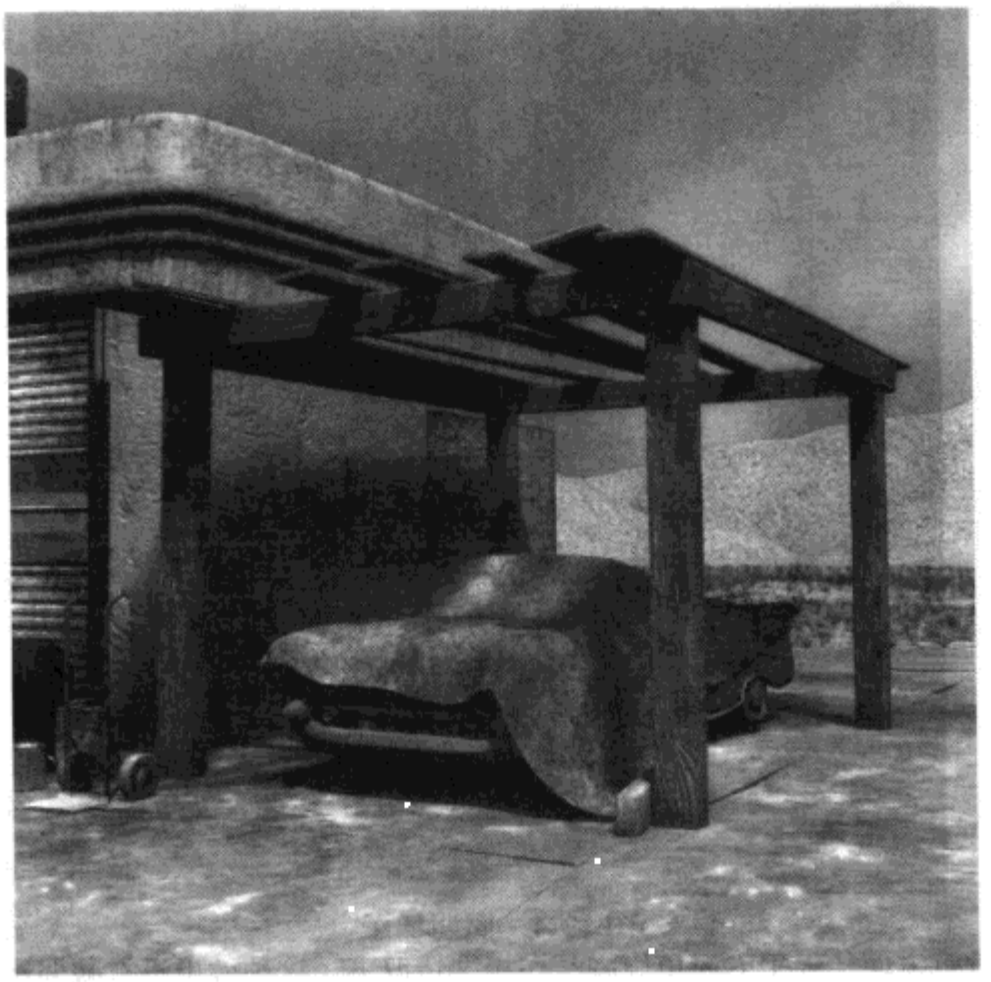


图 13-6 在汽车苫布上的阴影

在苫布上面的木头条形成了复杂的阴影，这对算法来说是最坏的情况。这些木头条使苫布上的遮挡区间映射必须存储在 5 个不同的纹理中。对于场景中的大多数物体，4 个纹理就足以取得所有的阴影

13.7 参考文献

我们也考虑过使用 Assarsson 等人提出的模糊阴影体积技术，但是由于实时使用该技术，就不得不在场景中使用太多的多边形，结果只好作罢。

Assarsson, Ulf, Michael Dougherty, Michael Mounier, and Tomas Akenine-Möller. 2003. "An Optimized Soft Shadow Volume Algorithm with Real-Time Performance." *Graphics Hardware,* pp. 33～40.

Heidrich, Wolfgang, Stefan Brabec, and Hans-Peter Seidel. 2000. "Soft Shadow Maps for Linear Lights." In 11*th Eurographics Workshop on Rendering,* pp. 269～280.

Max, N. L. 1988. "Horizon Mapping: Shadows for Bump-Mapped Surfaces. *The Visual Computer* 4(2), pp. 109～117.

Sloan, Peter-Pike, Jan Kautz, and John Snyder. 2002. "Precomputed Radiance Transfer for Real-Time Rendering in Dynamic, Low-Frequency Lighting Environments."

ACM Transactions on Graphics 21, pp. 527～536.

第 14 章　透视阴影贴图

Simon Kozlov
SoftLab-NSK

14.1　引言

阴影的生成一直是实时 3D 图形的大问题。对现代 GPU 而言，确定一个点是否在阴影中并不容易，特别是因为 GPU 是以光栅化多边形而不是光线跟踪的方式工作的。

今天的阴影应该是完全动态的。在场景中的每个物体都应该投射和接收影子，每个物体都应该有自阴影，还应该有模糊的阴影。能够满足这些要求的只有两个算法：阴影体（或模板阴影）和阴影映射。

阴影体和阴影映射算法之间的不同在于，是涉及物体空间还是图像空间。

- 阴影体是物体空间的阴影算法，通过创建代表阴影遮挡的多边形结构来工作，这样总是能得到像素精确但清晰的影子。这个方法不能够处理没有多边形结构的物体，例如 alpha 检测修正的几何体或位移映射的几何体。同时，由于画阴影体需要大量的填充率，该方法很难应用在密集场景中的每个物体上，尤其是有多个光源时。
- 阴影映射是图像空间的阴影算法，它可以处理任何物体的修改（如果能够渲染一个物体，就能得到它的阴影），但是存在走样问题。走样时常发生在有较宽的或全方向光源的大场景中。但是在阴影映射中，投射变换改变了阴影贴图纹素的屏幕尺寸，使得靠近摄像机的纹素变大。结果，为了得到好的质量，不得不使用极大的阴影贴图（4 倍于屏幕分辨率甚至更大）。而且，在复杂场景中，阴影映射比阴影体快得多。

Stamminger 和 Drettakis（2002）在 SIGGRAPH 2002 提出透视阴影映射（PSMs），通过使用在投射后空间中的阴影贴图来除去其中的走样，在投射后空间中，所有近处的物体都变得比远处的大。不幸的是，它只在某些情况下工作良好，所以原始的算法很难使用。

透视阴影贴图（PSM）算法有以下 3 个严重问题：

- 当光源在摄像机后面的时候，有一个虚拟的摄像机锥体。若在锥体内保持所有潜在的阴影投射体，阴影质量就会变得很差。
- 光源在摄像机空间中的位置对阴影质量影响很大。

- 最初的文章没有讨论过偏置（bias）问题。偏置是随 PSM 而来的问题，因为纹素的面积以不均匀方式分布，这意味着偏置不再是常量，应该与纹素的位置有关。

以上问题将在下节讨论。这章的焦点是方向光源（因为它们的走样问题较大），但是所有的思路和算法都可以容易地应用到其他类型的光源（将在适当的地方提供细节）。另外，我们还讨论用过滤和模糊改善阴影贴图质量的技巧。

大体上，这章描述的技术和方法能增加 PSM 的效果。当然，这些方法应该结合具体需要使用。

14.2 PSM 算法的问题

14.2.1 虚拟摄像机

首先，考查这个问题的实质。通常的投射变换把摄像机后面的物体移到投射后空间另一边的无限远平面。但是如果光源也在摄像机后面，这个物体就是潜在的阴影投射体，应该画进阴影贴图。

在图 14-1 的透视变换中，在光线上点的次序发生了变化。最初 PSM 文章的作者提出把观察摄像机“虚拟地”后退，将潜在的阴影投射体保持在视锥中，如图 14-2 所示，这样就能以正常方式使用 PSM。

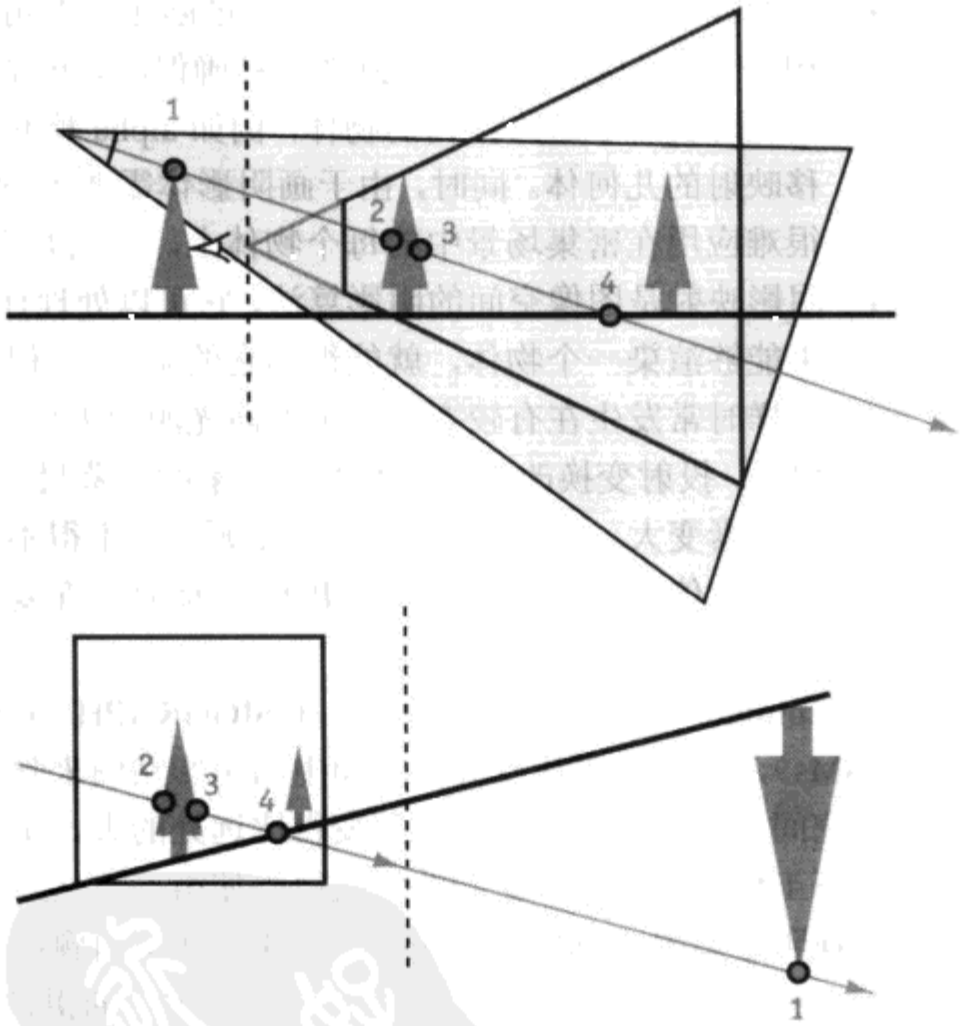


图 14-1 在投射后空间中摄像机后的物体
根据 Stamminger 和 Drettakis 2002 年的论文

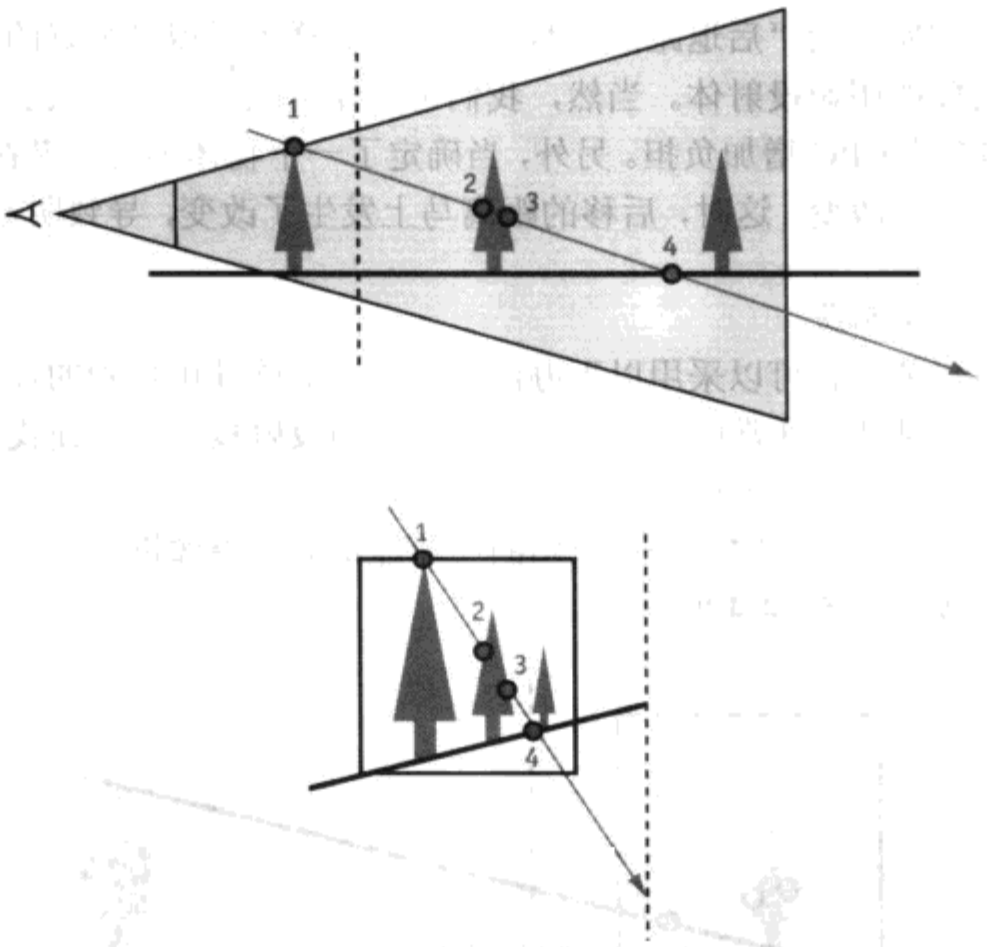


图 14-2　虚拟摄像机的使用
根据 Stamminger 和 Drettakis 2002 年的论文

虚拟摄相机

实际上虚拟摄像机的使用导致了很差的阴影质量。“虚拟的”滑动极大地减小了有效的阴影贴图的分辨率，使得靠近真正摄像机的物体变得更小，结果在阴影贴图中许多空间都没有用。另外，对摄像机后大的阴影投射体，我们不得不明显地把摄像机后移。图 14-3 显示即使摄像机有一点小的移动，质量也会发生非常明显的变化。

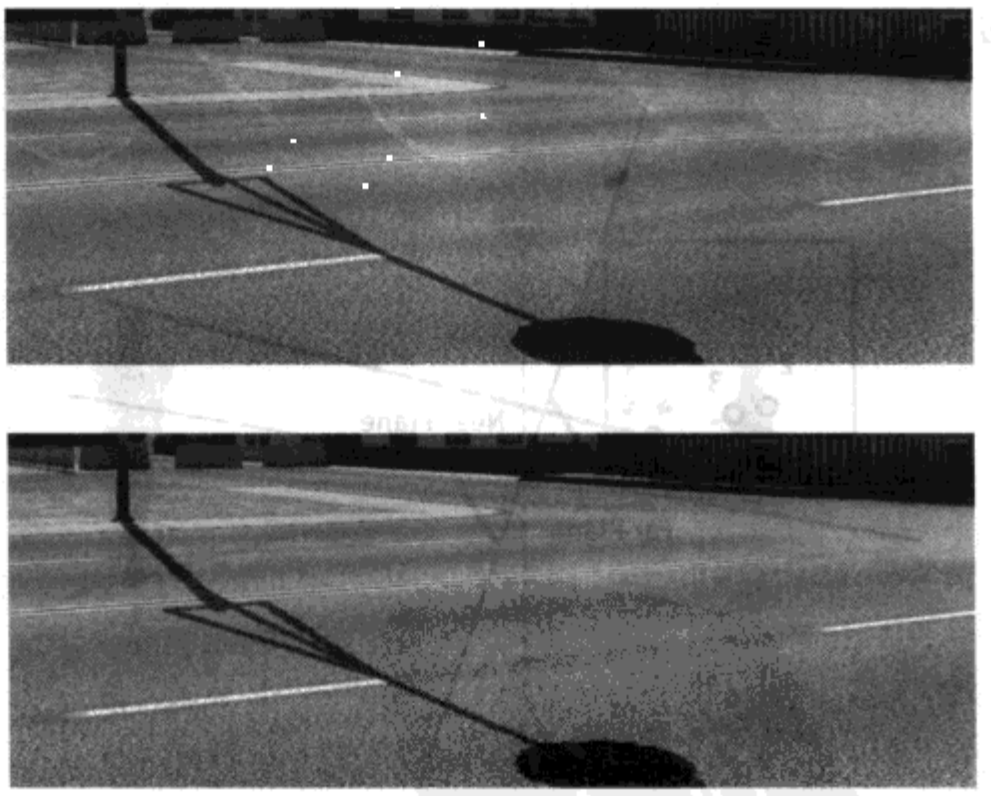


图 14-3　虚拟移动对阴影质量的影响
顶部：有虚拟的移动；底部：没有虚拟的移动

另一个问题是将实际的“后退距离”减到最小，那样可以得到最好的图像质量。这要求分析场景、发现可能的阴影投射体。当然，我们可以使用包围体、场景层次组织，以及类似的技术，但是它们会给 CPU 增加负担。另外，当确定了一个物体不再是潜在的阴影投射体时，阴影质量总是有突然的改变。这时，后移的距离马上发生了改变，导致阴影质量也突然变化。

虚拟照相机问题的解决

解决虚拟摄像机的问题可以采用以下办法：对光源矩阵使用特别的投射变换。事实上，投射后空间可以使用某些在通常的世界空间中不能做的投射技巧。它使我们可以建立特殊的投射矩阵，可以看作“比无限远更远”。

让我们考查投射后空间，它由一个原始的（非虚拟的）摄像机、一个“逆向”光源和摄像机后面的物体组成，如图 14-4 所示。

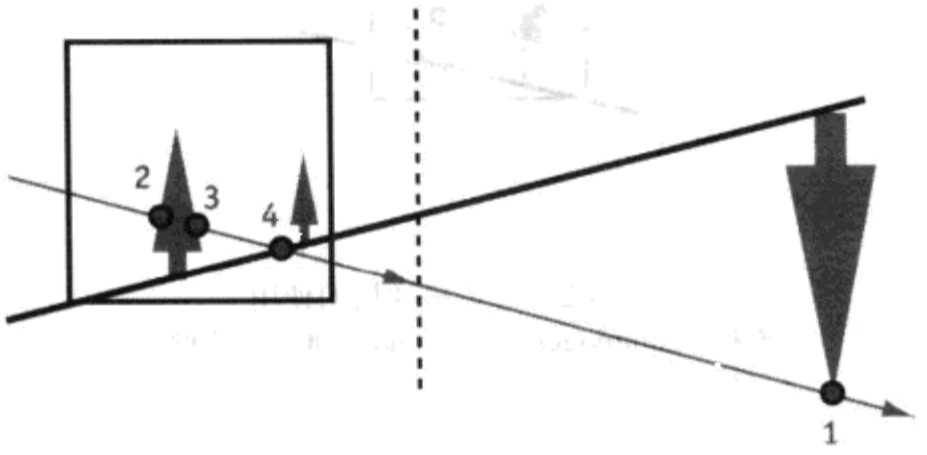


图 14-4 具有逆向光源的投射后空间
根据 Stamminger 和 Drettakis 2002 年的论文

这个解法的缺点是，光线应该（实际上不是）从光源出发，经过点 1，射向负的无穷远，然后经过正无穷远，回到光源，在点 2、3 和 4 取得信息。幸运的是，有一个投射矩阵能符合与这道“不可能的”光线，在那里可以把近平面设定为负值，把远平面设定为正值，见图 14-5。

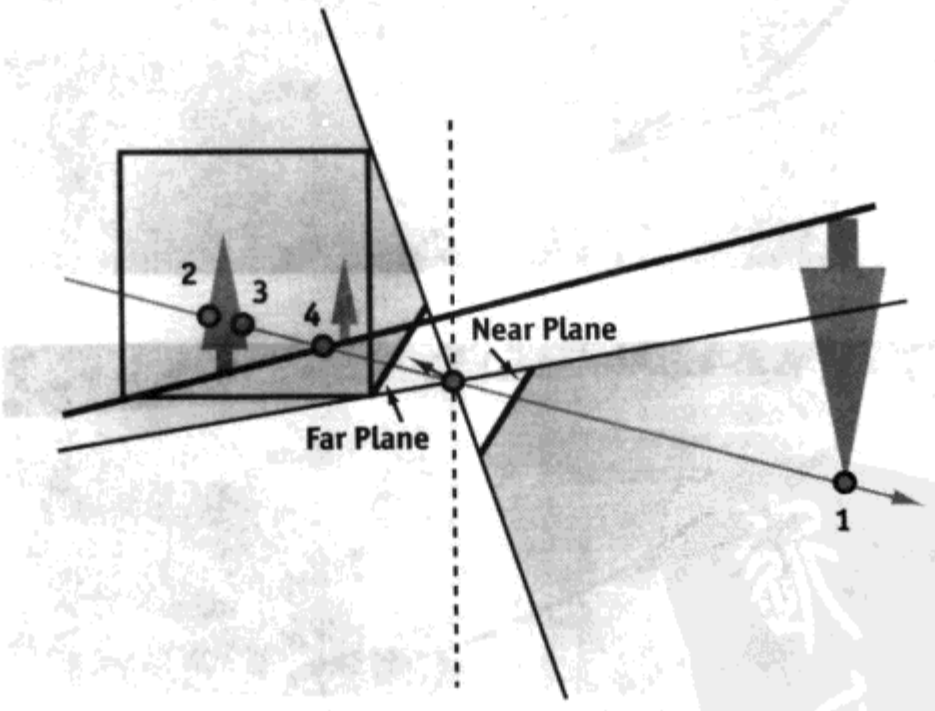


图 14-5 一个逆投射矩阵
红色箭头是光源摄像机方向

在最简单的情况下，

$$|Z_n| = |Z_f| = a$$

这里 a 小到适于整个单元立方体。然后我们把这个逆投射构建为平常的投射矩阵，正如在这里显示的，这里矩阵被写成行优先的矩阵：

$$\begin{pmatrix} c & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & Q & 1 \\ 0 & 0 & -QZ_n & 0 \end{pmatrix} \rightarrow \begin{pmatrix} c & 0 & 0 & 0 \\ 0 & d & 0 & 0 \\ 0 & 0 & \frac{1}{2} & 1 \\ 0 & 0 & \frac{1}{2}a & 0 \end{pmatrix}$$

其中：

$$Q = \frac{Z_f}{Z_f - Z_n} = \frac{a}{a-(-a)} = \frac{1}{2}$$

于是进入阴影贴图的最后转换的 z 坐标是：

$$Z_{psm} = Q \times \left(1 - \frac{Z_n}{Z}\right) = \frac{1}{2} \times \left(1 + \frac{a}{Z}\right)$$

这里 $Z_{psm}(-a) = 0$，如果我们持续减少 z 值到负无限远，Z_{psm} 驱向于 1/2。同样地，$Z_{psm} = 1/2$ 对应于正无限远，而从正无限远到远平面的移动，使 Z_{psm} 在远平面上增加到 1。这就是光线以正确次序经过所有点的原因，以及创建投射后空间时不需要使用“虚拟移动”的原因。

这个技巧只能在投射后空间中起作用，因为在正常情况下，无限远平面后的所有点 $w < 0$，它们不可能光栅化。但是由光源摄像机引起的其他的投射转换，这些点位于摄像机后，因此，w 坐标再次反转，变成正的。

使用这个逆的投射矩阵，就不一定使用虚拟摄像机。结果，我们没有做任何 CPU 场景分析，就得到了更好的阴影质量，也没有相关的走样。

逆投射矩阵的惟一缺点是由于阴影贴图深度值的范围很大，需要更好的 z 值精度。24 位的定点深度值可以满足正常情况的要求。

因为阴影质量取决于摄像机近平面的位置，所以虚拟摄像机还是有用的。投射后的 z 的公式是：

$$Z_{pp} = Q\left(1 - \frac{Z_n}{Z_{world}}\right) \qquad \text{其中：} \quad Q = \frac{Z_f}{Z_f - Z_n}$$

如公式所示，Q 非常接近于 1，只要 Z_n 比 Z_f 小许多，Q 就不会明显变化。所以近平面和远平面必须改变很大，才能影响 Q 值，而这通常是不可能的。同时，近平面值极大地影响投射后空间。例如，Z_n= 1m 时，世界空间中近平面后的第 1m 将占有投射后空间中的半个单位立方体。如果我们把 Z_x 变为 2m，z 值分辨率会变为原来的两倍，而且也改善了阴影质量。这意味着应该取 Z_n 的最大值。

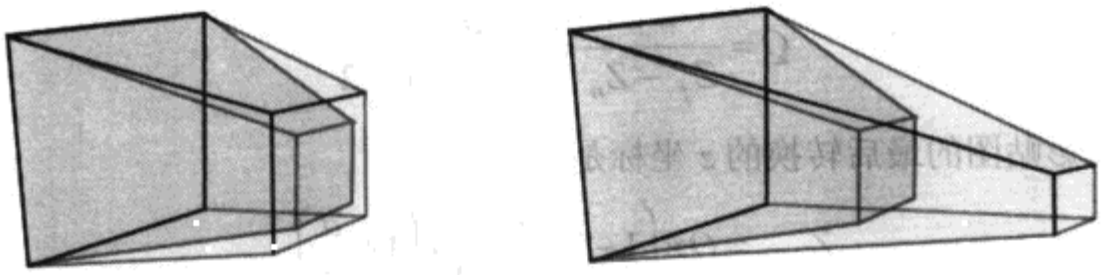
最初的 PSM 文章提出的方法看起来很完美：读回深度缓存，扫描每个像素，并为每帧找到 Z_n 的最大值。可惜这个方法太费时：它需要读回大量的显存，这会引起额外 CPU/GPU 的延迟，而且在被混合的和分层次的深度缓冲区不能很好地工作。因此我们应该使用其他方

法，为 PSM 渲染找到适当的近平面值。

其他寻找近平面值的方法，包括以下各种 CPU 场景分析方法：

- 基于计算包围体积的方法（稍后在“光源摄像机”中简介）。
- 用于估计最近物体距离的碰撞检测系统。
- 以低多边形的细节等级渲染的附加软件场景，它也可能用于遮挡剔除。
- 基于特殊应用的场景结构特性的复杂分析。例如，使用固定摄像机路径处理关卡的时候，可以预先计算每帧的近平面值。

这些方法力图增加实际的近平面值，但是也可以“虚拟地”增加数值。思路与原先的虚拟摄像机基本相同，只有一点不同。即把摄像机后移时，增加近平面的值，以便原先的近平面方形与虚拟摄像机保持在相同的平面上，见图 14-6。



建议的虚拟照相机　　最初文章提出的虚拟照相机

图 14-6　虚拟照相机之间的不同

当把虚拟摄像机后移时，z 值的分辨率得以改善。然而，却使得近处物体的 x 和 y 值的分布变得更坏，结果使离摄像机近的和远的阴影质量得以平衡。由于在投射后空间中 z 值的分布很不规律，而且近平面值的影响很大，所以如果“虚拟的”摄像机不后移，这个平衡就无法达到。离摄像机近的阴影看起来很好，而远处的阴影质量却很差，这是由阴影贴图纹素面积分布不平衡造成的。

14.2.2　光源摄像机

PSMs 的另一个问题是阴影质量依赖光源和摄像机位置之间的关系。对于垂直的方向光，完全没有走样问题，但是当光源朝向摄像机并迎面靠近它时，阴影映射的走样就很严重。

我们试着把整个单位立方体保持在一个阴影贴图纹理中，因此，必须使光源的视域大到能够容纳整个的立方体。这也就意味着与近平面靠近的物体不可能有足够的纹理样本，见图 14-7。

光源越靠近单位立方体，质量越差。如我们所知，

$$Z_{\infty} = Q = \frac{Z_f}{Z_f - Z_n}$$

所以对于具有 $Z_n = 1$，$Z_f = 4000$ 和 $Q = 1.0002$ 的很大的户外场景，意味着光源非常接近单位立方体。Z_f/Z_n 通常是大大地超过 50，它对应于 $Q = 1.02$，这样的接近就会产生问题了。

把整个单位立方体安放在阴影贴图纹理中总是存在问题。有两个办法，每个只解决问题的一部分：单位立方体裁剪法，把光源摄像机对准单位立方体的必要部分；立方体映射法，使用多个纹理来存储深度信息。

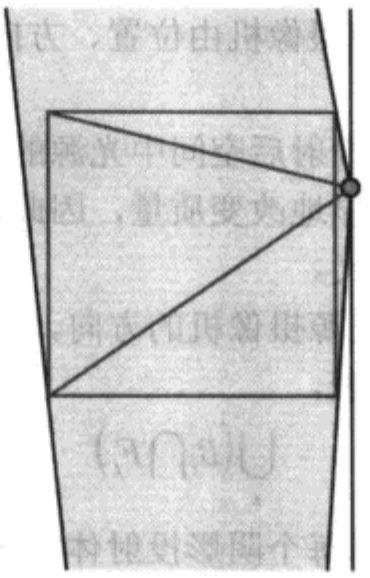


图 14-7　具有低照明角的光源摄像机

单位立方体裁剪法

由于实际物体的体积通常比整个视锥体小得多（尤其在接近远平面时），而只有这些物体上的阴影映射信息才是有用的，因此可以进行优化。如果调节光源摄像机，使其只对准实际的物体（而不是整个的单位立方体），我们将会得到更好的质量。调节摄像机时应该使用简化的场景结构，如包围体积。

最初 PSM 文章中提到了立方体裁剪，但是它考虑场景中的全部物体，包括在视域锥中的阴影投射体和用于构造虚拟摄像机的锥体之外的潜在阴影投射体。因为不需要虚拟摄像机，我们可以把光源摄像机只集中在阴影接收体上，这样更高效，见图 14-8。对于投射后空间中的光源摄像机，还应该选择近和远裁剪平面值，以便将所有阴影投射体容纳在阴影映射中。由于这样做不改变纹素面积的分布，因此不影响阴影质量。

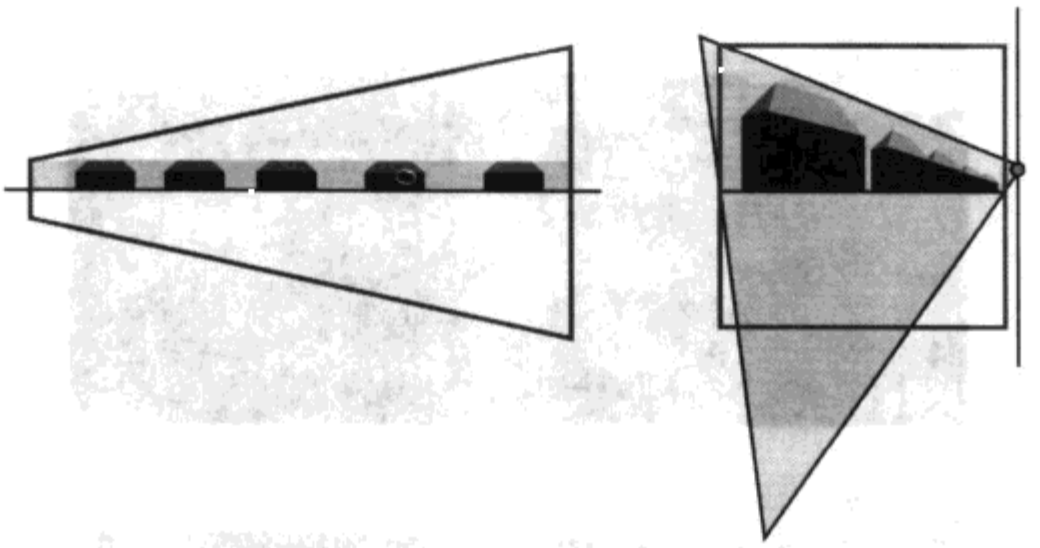


图 14-8　基于阴影接收体的包围体来调整光源摄像机的聚焦

因为这些包围体的远处部分在投射后空间中大大地压缩了，光源摄像机的视域并没有变得很大，即使光源接近其余场景时也是如此。

在实践中，使用大致的包围体积仍然可以得到足够好的质量，一般只需要指出感兴趣的是哪部分场景。在户外场景中，它是风景中物体的近似高度；在室内场景中，它是当前房间的包围体，等等。

在构建大致描述场景的包围体集之后，我们要写计算光源摄像机的算法，摄像机聚焦在

场景中阴影接收体上。事实上，光源摄像机由位置、方向、向上矢量和投射参数给定，其中大部分是预先定义的。

- 我们不可能改变位置：它是投射后空间中光源的位置。
- 在实际中，向上矢量并不明显地改变质量，因此，我们能选择任何合理的量。
- 投射参数完全由观察矩阵定义。

最有趣的事是基于包围体选择光源摄像机的方向。建议的算法如下。

1．计算几何实体结构的顶点列表，

$$\bigcup_i (B_i \bigcap F_i)$$

这里 B_i 是第 i 个包围体，F 表示每个阴影投射体的包围体的锥体，包围体积是在当前帧中所见的，所有这些运算先在观察摄像机空间中执行。然后把所有的顶点转换到投射后空间。

完成这步后，我们就有了光源摄像机应该“看到”的所有点（顺便提一下，因为读回深度缓存不是好办法，我们应该基于这些点找到较好的近平面值）。

2．求光源摄像机。也就是求最好的光源摄像机方向，因为如果给定方向，很容易计算其他所有参数。建议使用最小圆锥体的轴来近似得到最佳方向，圆锥体以光源为中心而且包括列表中的所有点。为一组点求最佳圆锥体的算法复杂度是线性时间，类似于在线性时间对一组点求最小包围球的算法（Gamier 1999）。

用这种方式，我们可以在线性时间求出最佳的光源摄像机，它依赖的包围体数目不是很大，因为我们只需要大致的场景结构信息。

这个算法在大的户外场景中对方向光源是有效的。光源角度对阴影质量的影响不大，仅在光源直接指向摄像机时，质量会略微变差。图 14-9 显示使用单位立方体剪切和不使用两者之间的差别。

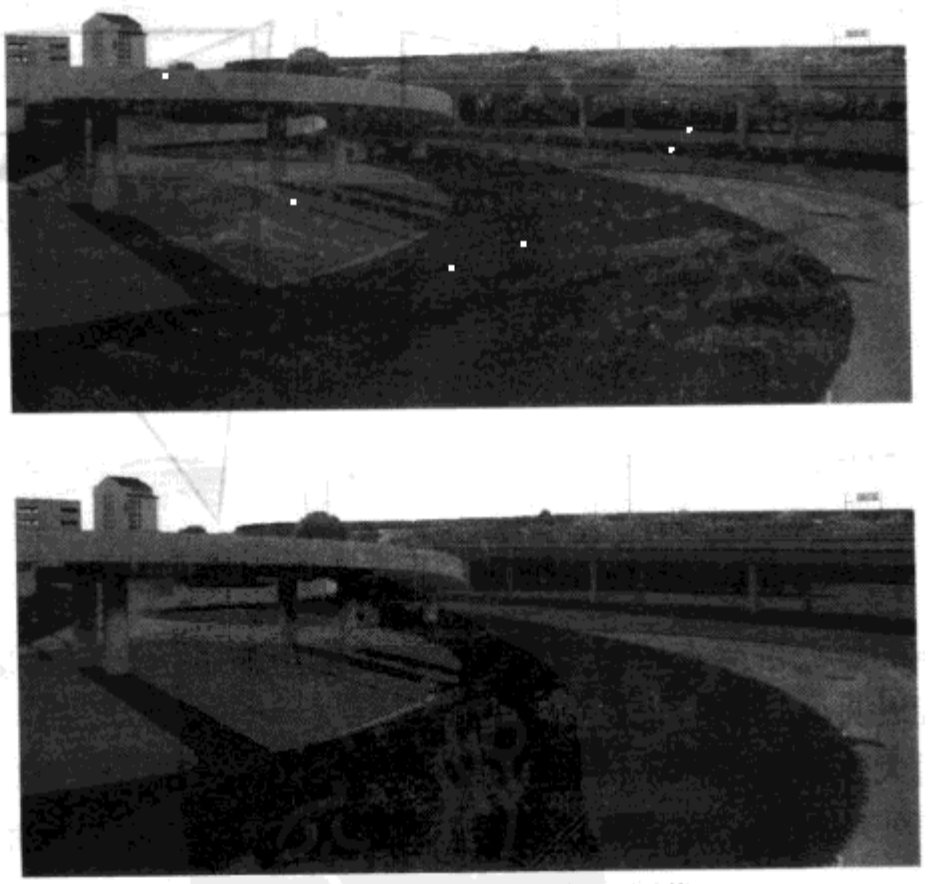


图 14-9 单位立方体剪切图像
使用单位立方体剪切（顶部）和不使用单位立方体剪切（底部）产生的图像

使用立方体映射

虽然立方体剪切在某些情况下是有效的，但在另外的时候却难以使用。举例来说，我们可能有一个填充很密的单位立方体（这是经常的），或者我们完全不想使用包围体。再说立方体剪切也不能用于点光源。

更一般的方法是使用立方体映射纹理作为阴影贴图。大多数光源在投射后空间中变成点光源，使用立方体映射作为点光源的阴影贴图是很自然的。但是在投射面后空间中，因为只需要存储关于单位立方体的信息，所以使用立方体映射应该也有些不同。

建议的解决办法是使用单位立方体上背对光源的面，作为立方体映射面纹理的平台。

对于投射面后空间中的一个直射光源，立方体映射看起来像图 14-10 一样。

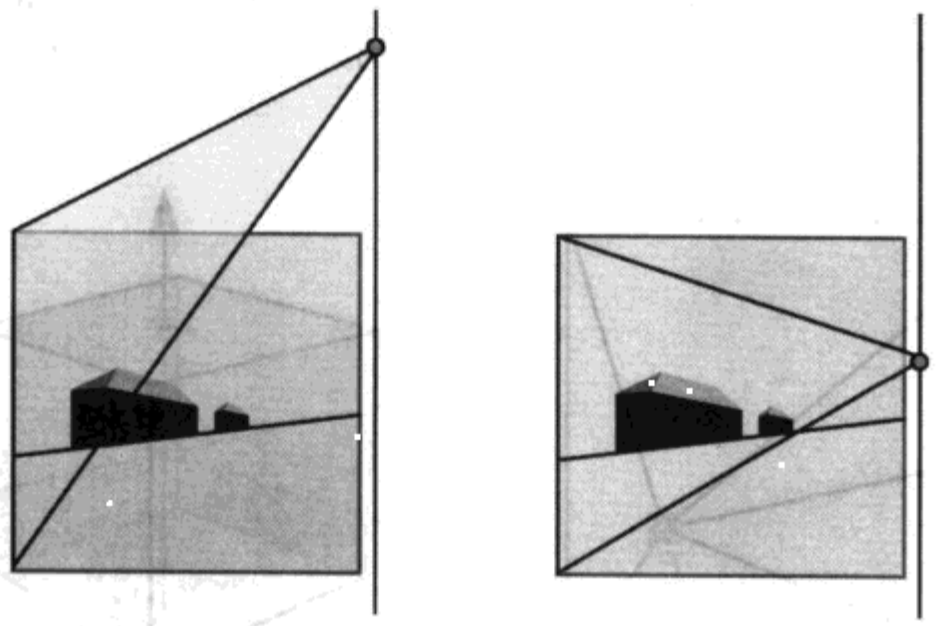


图 14-10　为方向光源使用立方体映射

所使用的立方体映射的面数（从 3 到 5）依赖于光源的位置。当光源接近其余场景和指向摄像机时，额外的纹理资源是必需的，所以我们使用最多的面。位于单位立方体外面的其他类型的光源，图像是相似的。

位于单位立方体里的点光源，应该使用立方体映射的 6 个面，但是它们还是被聚焦在单位立方体的面上，见图 14-11。

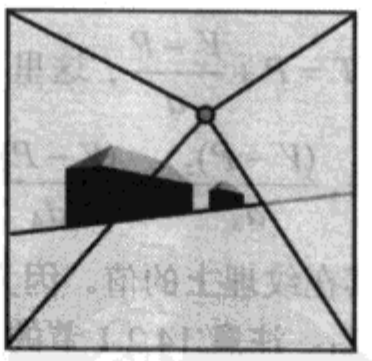


图 14-11　为点光源使用立方体映射

我们形成一个“有偏移中心的立方体映射”，它类似于正常的立方体映射，但是它的纹理坐标上加有一个固定的矢量。换言之，立方体映射的纹理坐标是投射后空间中的顶点位置，并随着光源位置而偏移。

$$\text{纹理坐标} = \text{顶点位置} - \text{光源位置}$$

在选择作为立方体映射平台的单位立方体面时，我们可以根据屏幕大小按比例地安排纹理面积，保证阴影质量与光源和摄像机位置无关。事实上，在投射后空间中纹素的尺寸处于规定的范围内，于是它在屏幕上的投射只与所投射的面有关。这个投射不会很大地放大纹素，所以在荧屏上的纹素的尺寸也限于规定的范围。

因为渲染阴影映射时顶点和像素 shader 相对很短，所以问题在于背面缓冲区和阴影映射深度缓冲区的纯填充率。画单个的阴影映射或立方体映射时，如果总的纹理尺寸相同，两者几乎没有差别（尽管使用了很好的遮挡剔除）。只是立方体映射的质量更好。不同之处是渲染目标转换的消耗，同时在顶点和像素 shader 中计算立方体映射的纹理坐标需要另外的指令。

下面计算这些纹理坐标。首先，考虑在图 14-12 中显示的图片。蓝色的正方形是单位立方体，P 表示光源的点，V 是生成的纹理坐标的点。我们在分别的遍中渲染 6 个立方体映射的面；每个遍的近平面显示为绿色。它们正在形成另外的小立方体，$Z_1 = Z_n / Z_f$ 对每个遍是常数。

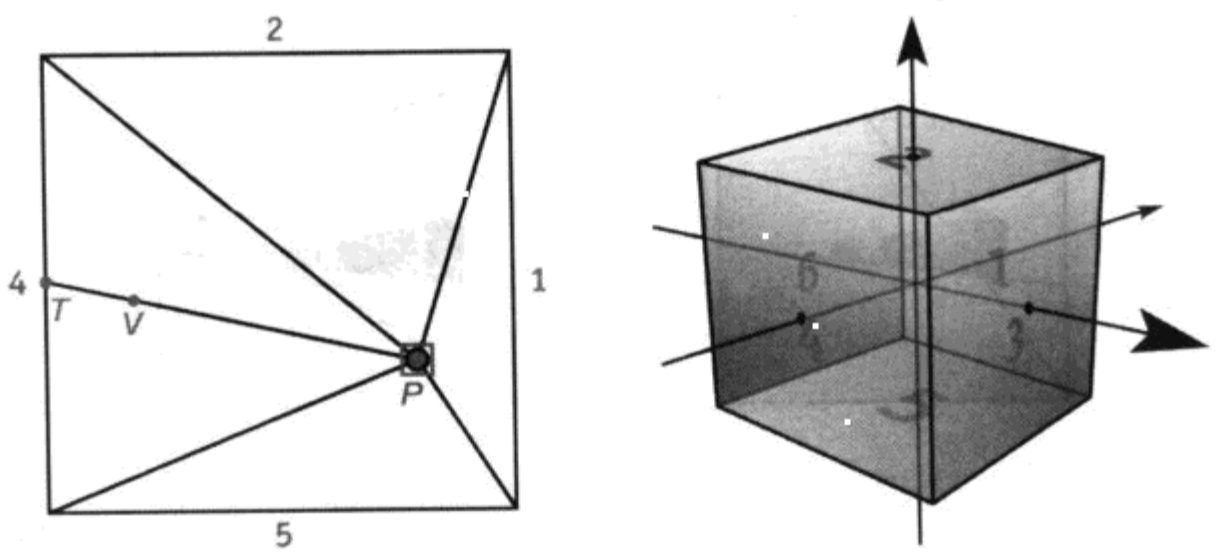


图 14-12 投射后空间中的立方体映射

现在计算纹理坐标和用于比较 V 点的深度值。应该在（$V-P$）方向移动这个点，直到与立方体相交。以 d_1、d_2、d_3、d_4、d_5 和 d_6 作为 P 到每个立方体映射面的距离（面的号码见图 14-12）。

我们正在寻找的立方体面上的点（也即立方体映射的纹理坐标）是：

$$T = P + \frac{V-P}{a}, \text{ 这里}$$

$$a = \max\left(\frac{(V-P)_x}{d_1}, \frac{(V-P)_y}{d_2}, \frac{(V-P)_z}{d_3}, -\frac{(V-P)_x}{d_4}, -\frac{(V-P)_y}{d_5}, -\frac{(V-P)_z}{d_6}\right)$$

针对 a 值的投射变换结果比较其在纹理上的值。因为用相应的 d 值除它，必须应用那个投射变换，有效地使 $Z_f = 1$ 和 $Z_n = Z_1$。注意 14.2.1 节的倒转摄像机投射的情况中，$Z_n = -Z_1$ 和 $Z_f = Z_1$。

$$Z_{out} = Q\left(1 - \frac{Z_n}{a}\right) = \frac{Z_f}{Z_f - Z_n}\left(1 - \frac{Z_n}{a}\right)$$

（所有这些计算是在 OpenGL 那样的坐标系中做的，在那里单位立方体是实际上的单位立方体。在 Direct3D 中，因为 z 坐标范围是[0..1]，所以单位立方体的大小是一半）。

列表 14-1 是一个 shader 代码的例子。

列表 14-1　计算立方体映射纹理坐标的 shader 代码

```
// c[d1] = 1/d1, 1/d2, 1/d3, 0
// c[d2] = -1/d4, -1/d5, -1/d6, 0
// c[z] = Q, -Q * Zn, 0, 0
// c[P] = P
// r[V] = V
// cbmcoord - output cube map texture coordinates
// depth - depth to compare with shadow map values

//Per-vertex level
sub r[VP], r[V], c[P]
mul r1, r[VP], c[d1]
mul r2, r[VP], c[d2]

//Per-pixel level
max r3, r1, r2
max r3.x, r3.x, r3.y
max r3.x, r3.x, r3.z
rcp r3.w, r3.x
mad cbmcoord, r[VP], r3.w, c[P]

rcp r3.x, r3.w
mad depth, r3.x, c[z].x, c[z].y
```

因为深度纹理不能是立方体映射，我们可以使用彩色纹理，把深度值放进彩色通道。有许多方法可以做到，也有许多实现相关的技巧，但是对它们的描述不在本章的范围内。

另一种可能是用多纹理模拟立方体映射的方式，这时每个立方体面变成一个独立的纹理（这时深度纹理是很好的）。我们在顶点 shader 中形成几个纹理坐标组，并且与像素 shader 中的阴影结果相乘。技巧是在场景中的物体上处理这些纹理，因为很少有每个物体需要所有 6 个面的情况。

14.2.3　偏置

如前所述，在均匀阴影贴图中常用的固定的偏置不能用于 PSM，因为随着场景中光源位置和点的变化，z 值和纹素面积分布非常不同。

如果使用深度纹理，把缩放的 z 斜率用于偏置。这可以消除失真，尤其对非常远的不在摄像机内的物体。但是有些卡不支持深度纹理（在 DirectX 中，只有 NVIDIA 卡支持深度纹理），而且深度纹理不可能是立方体映射。在这些情况下，需要不同的更一般的算法来计算偏置。另外的困难是，它很难模拟和调节缩放的 z 斜率的偏置，因为它需要把另外的数据代入像素 shader 中（例如当前三角形的顶点坐标），再加上一些不稳健的计算。

现在考察不能再使用固定偏置的原因。考虑光源靠近和远离单位立方体的两种情况，见图 14-13。

Z_f/Z_n 的相互关系决定 Z 值在阴影映射内的分布，它在这两种情况中变化很大。固定的偏置意味着在世界空间和投射后空间中完全不同的实际偏置：适用于第一个光源位置的固定偏

置对第二个光源是不正确的，反之亦然。同时因为光源可能离单位立方体很近，也可能离得很远（甚至在无限远），所以 Z_f / Z_n 改变很大，这取决于世界空间中光源和摄像机的相对位置。

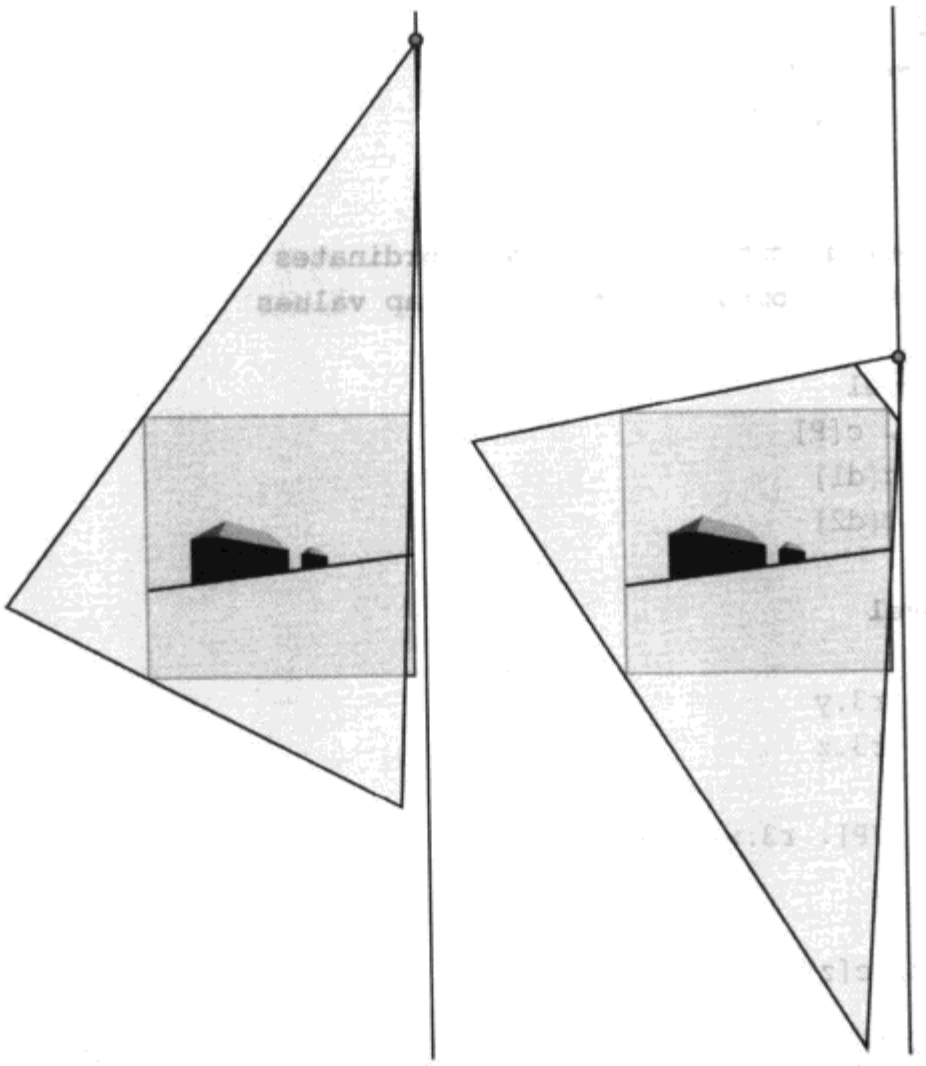


图 14-13 光源离单位立方体很近和很远

对某个固定的光源位置，有时我们甚至不能够找到适当的常数作为偏置。因为投射变换放大近的物体和缩小远的物体，所以偏置应该取决于点位置，对离摄像机近的物体应该是比较小的，对远的物体是比较大的。图 14-14 表现使用常数偏差的典型失真。

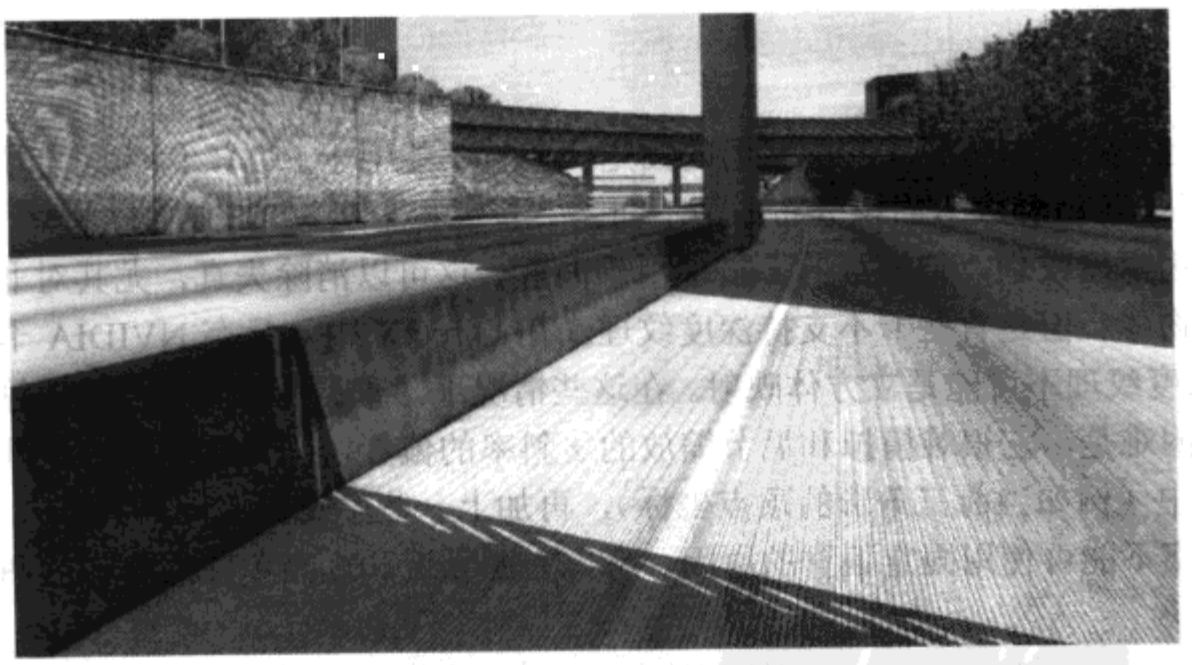


图 14-14 固定偏置的失真
注意靠近垂直表面（例如左边远处墙壁和桥）充满失真

简而言之，解决方法是使用在世界空间中的偏置（而且不再分析双投射矩阵的结果），然后把这个世界空间偏置转换到投射后空间。所计算的值依赖于双投射，它对任何光源和摄像机位置都是正确的。这些可以容易地在顶点 shader 中操作。此外，这个世界空间偏差值应该由世界空间的 texel 尺寸缩放，来处理由 texel 面积的不均匀分布所引起的失真。

$$P_{biased} = \left(P_{orig} + L(a + bL_{texel})\right)M$$

这里 P_{orig} 是最初点的位置，L 是世界空间的光源矢量方向，L_{texel} 是世界空间中 texel 尺寸，M 是最后的阴影映照矩阵，a 和 b 是偏置系数。

世界空间中 texel 的大小可以用简单矩阵近似计算。首先，把点转换到阴影映照空间，然后不变更深度而通过 texel 的大小改变这点。下一步，把它转换回到世界空间，并且求这点和最初点的长度差的平方。这给我们 L_{texel}：

$$L_{texel} = \left|P_{orig} - (P_{orig}M + c)M^{-1}\right|^2$$

这里 $c = \left(\frac{2}{S_x}, \frac{2}{S_y}, 0\right)$，$S_x$ 和 S_y，是阴影映照分辨率。

明显地，我们能建立一个矩阵，运行所有的变形（当然，除了坐标乘法之外）：

$$L_{texel} = \left|P_{orig}M'\right|^2$$

这里 M' 包括变换、移动、转置，和减法。

这成为经验的解法，但是还是应该根据你的特别需要进行调节，见图 14-15。

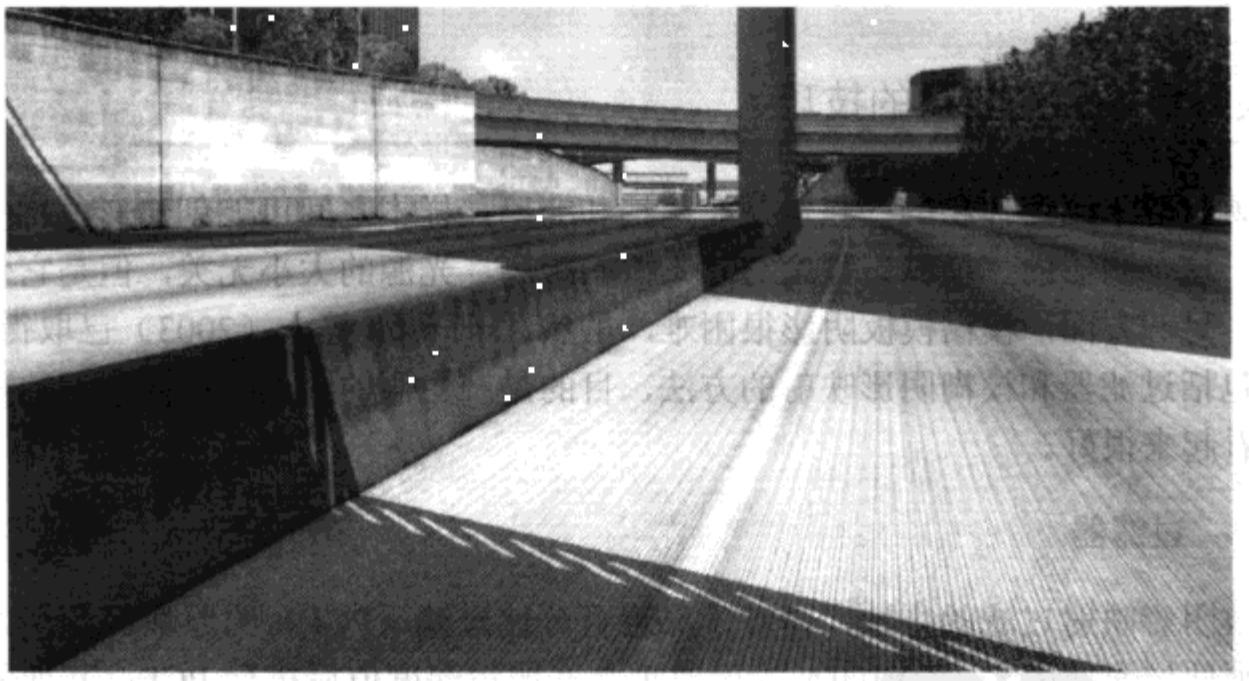


图 14-15　在顶点 shader 中所计算的偏差

列表 14-2 是执行这些计算的顶点 shader 代码。

列表 14-2　计算偏差的顶点 shader

```
def c0, a, b, 0 ,0

// Calculating Ltexel
dp4 r1.x, v0, c[LtexelMatrix_0]
```

```
dp4 r1.y, v0, c[LtexelMatrix_1]
dp4 r1.z, v0, c[LtexelMatrix_2]
dp4 r1.w, v0, c[LtexelMatrix_3]

// Transforming homogeneous coordinates
// (in fact, we often can skip this step)
rcp r1.w, r1.w
mul r1.xy, r1.w, r1.xy

// Now r1.x is an Ltexel
mad r1.x, r1.x, c0.x, c0.y

dp3 r1.x, r1, r1

// Move vertex in world space
mad r1, v0, c[Lightdir], r1.x

// Transform vertex into post-projective space
// (we need z and w only)
dp4 r[out].z, r1, c[M_2]
dp4 r[out].w, r1, c[M_3]
```

r[out]寄存器保存偏置的结果：深度值和经过三角形插值所对应的 w。注意这个插值应该与纹理坐标的插值分开（x、y 和对应的 w），因为它们的 w 坐标是不同的。当比较阴影映射值，或渲染真实的阴影映射时（阴影映射保存了偏置值），可能用到这些偏置值。

14.3 获得更好阴影映射的技巧

阴影映射优于阴影体积的一点，是在遮挡和非遮挡样本之间可能创建彩色梯度，这很像模糊阴影。这个阴影的“模糊”程度与遮光板的距离、光源的大小无关，但是它仍然能用于世界空间。另一方面，模糊模板阴影很困难，虽然 Assarsson 等人（2003）已取得显著进展。

这节包括过滤器和模糊阴影映射的方法，目的是产生假的阴影模糊度，它的模糊程度固定，但是看起来很好。

14.3.1 过滤器

大多数阴影映射过滤的方法，是基于靠近百分比过滤（PCF）的原理。各个方法的差别在于不同硬件如何使用它。NVIDIA 深度纹理是在比较深度值后执行 PCF；其他的硬件，必须从最接近的纹素取几个样本，然后平均它们的结果（对于真实的 PCF）。大体上，深度纹理滤波器比人工取 4 个样本的 PCF 技术更有效（PCF 大约需要 8 个样本才能产生可比较的质量）。另外，使用深度纹理滤波器并不禁用 PCF，因此，我们能取几个过滤的样本，来更进一步提高阴影质量。

在 PSM 中使用的 PCF（靠近百分比过滤）与标准的阴影映射中的 PCF 没有差别：两者都是用毗邻的纹素样本进行过滤。在 GPU 上就是在各个方向上把纹理坐标移动一个纹素。关于 PCF 的详细讨论，见第 11 章“阴影映射反走样”。

列表 14-3 和 14-4 显示的是 4 个样本的 PCF 的 shader 伪代码。

列表 14-3　用于 PCF 的顶点 shader 伪代码

```
def c0, sample1x, sample1Y, 0, 0
def c1, sample2x, sample2Y, 0, 0
def c2, sample3x, sample3Y, 0, 0
def c3, sample4x, sample4Y, 0, 0
// The simplest case:
// def c0, 1 / shadowmapsizeX, 1 / shadowmapsizeY, 0, 0
// def c1, -1 / shadowmapsizeX, -1 / shadowmapsizeY, 0, 0
// def c2, -1 / shadowmapsizeX, 1/ shadowmapsizeY, 0, 0
// def c3, 1 / shadowmapsizeX, -1 / shadowmapsizeY, 0, 0

. . .

// Point - vertex position in light space
mad oT0, point.w, c0, point
mad oT1, point.w, c1, point
mad oT2, point.w, c2, point
mad oT3, point.w, c3, point
```

列表 14-4　用于 PCF 的像素 shader 伪代码

```
def c0, 0.25, 0.25, 0.25, 0.25
tex t0
tex t1
tex t2
tex t3

. . .

// 深度比较后
mul r0, t0, c0
mad r0, t1, c0, r0
mad r0, t2, c0, r0
mad r0, t3, c0, r0
```

这些技巧改善了阴影质量，但是未解决严重的走样问题。举例来说，如果许多屏幕像素映射为一个阴影映射纹素，就会看到大的台阶走样，即使把它们略微模糊也还是能看到。图 14-16 显示的是未经过任何过滤的走样的阴影，图 14-17 显示 PCF 帮助改善阴影质量，但是不能够完全消除走样。

14.3.2　模糊

从投射阴影可知，最好的模糊结果时常是这样得到的：先用模糊像素 shader 来渲染一个分辨率较小的纹理，然后再通过模糊像素 shader 将这个产生的纹理反馈回去几次（称为乒乓渲染法）。阴影映射和投射阴影是相似的技术，不使用这个方法的原因是：阴影映射不是一个黑白图片；它是一个深度值的集合体，而模糊深度映射是没有意义的。

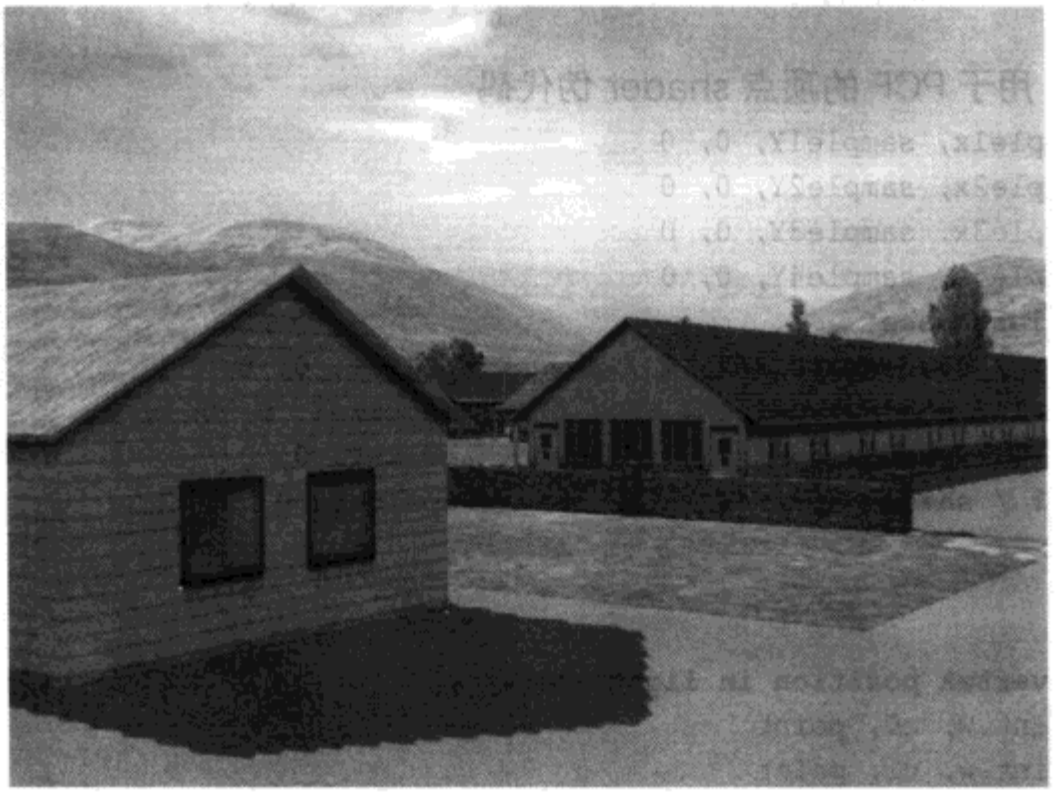


图 14-16 严重走样

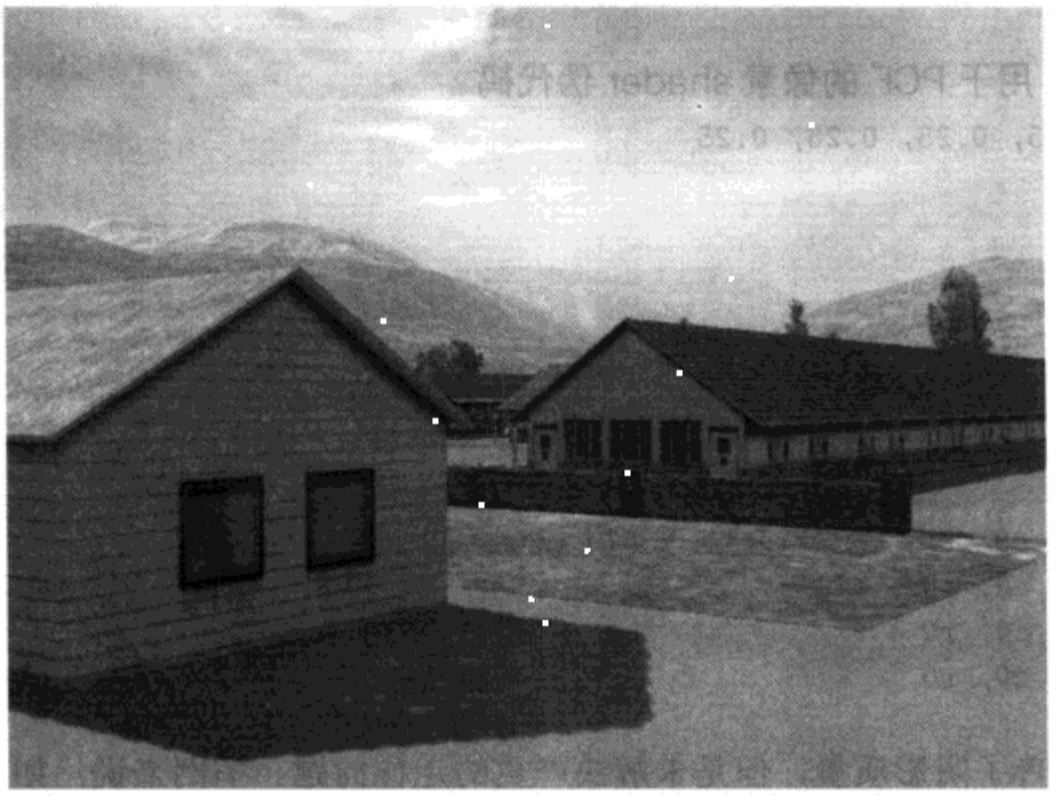


图 14-17 用过滤反走样

建议使用阴影映射渲染的彩色部分作为一些物体的投射纹理（那几乎不花费什么）。例如，假定我们有一个户外的风景场景，而且想要让地上有高质量的模糊阴影，因为地面的阴影是最引人注目的。

1．在渲染深度阴影映照之前，用 1 清除彩色缓冲区。在渲染的时候，在除风景之外的每个物体的彩色缓冲区内填 0；在风景颜色缓冲区内填 1。在渲染整个阴影映射之后，未遮挡的风景彩色值为 1，而被遮挡的地方为 0。见图 14-18。

2．用多遍的简单模糊像素 shader，极大地模糊图像（图 14-18）。例如，使用简单的两遍

的 Gaussian 模糊，得到好的结果（还可能想要调整远处物体的模糊半径）。在这个步骤之后，我们将会得到一个高质量的模糊纹理，如图 14-19 所示。

图 14-18　一个小测试场景的最初的彩色部分

图 14-19　使一个小测试场景的彩色部分模糊

3．用阴影渲染场景的时候，用模糊纹理代替阴影映射渲染风景，而用阴影映射的深度部分渲染其他的所有物体。见图 14-20。

图 14-20　对真实场景应用模糊

质量的差异是很明显的。

当然，这个方法不仅可以用于风景，也可以用于任何无需自阴影的物体（像地板、墙壁、地平面等）。所幸的是，在这些面积中阴影是最引人注目的，走样问题也是最明显的。因为有几个彩色通道，我们可以同时模糊几个物体上的阴影。

- 使用深度纹理，彩色缓冲区完全自由，因此我们能对 4 个物体使用 4 个通道。

- 对于浮点纹理，1 个通道存储深度信息，3 个通道用于模糊。
- 对于定点纹理，深度通常存储在红色的和绿色的通道中，因此，只有 2 个自由通道。

这样我们将会在地上、地板上和墙壁上得到很好的模糊阴影，而同时在其他的物体（有适当的自阴影）上也保持各自所有的阴影（用 PCF 模糊的）。

14.4 结果

图 14-21、14-22、14-23 和 14-24 的荧屏截图是在 NVIDIA GeForce4 Ti 4600 上取得的，荧屏分辨率为 1600 × 1200，可见的多边形有 100 000 到 500 000 个。所有的物体都以实时帧速率接收和投射阴影（多于 30 帧）。

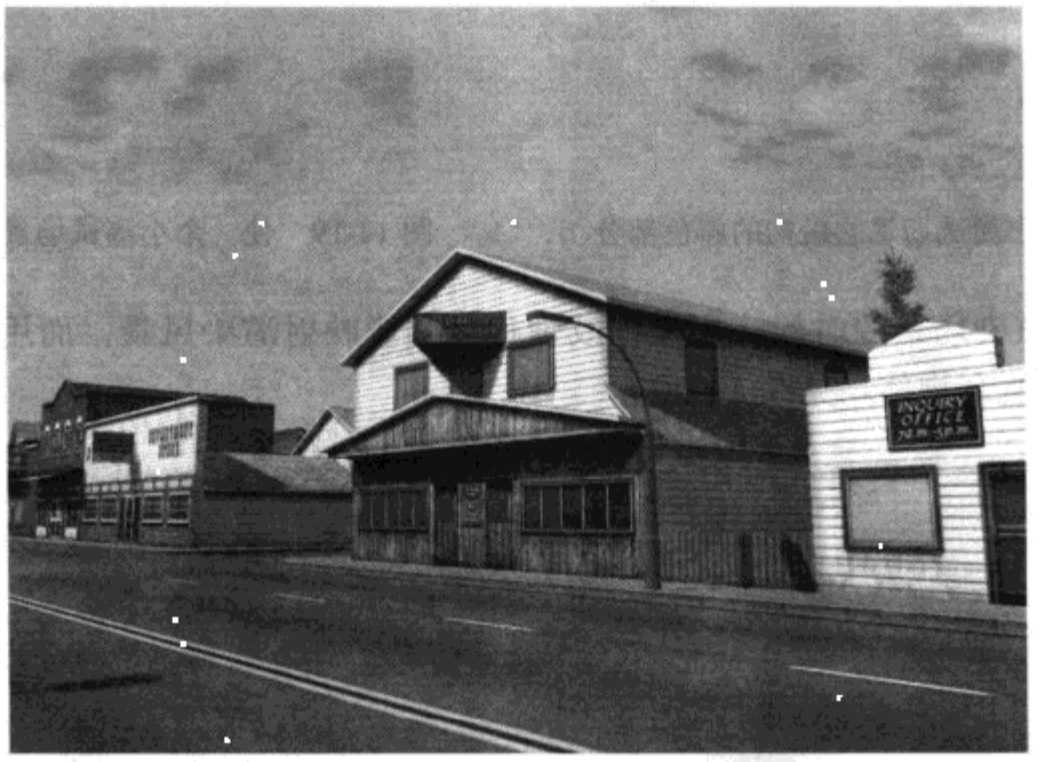


图 14-21

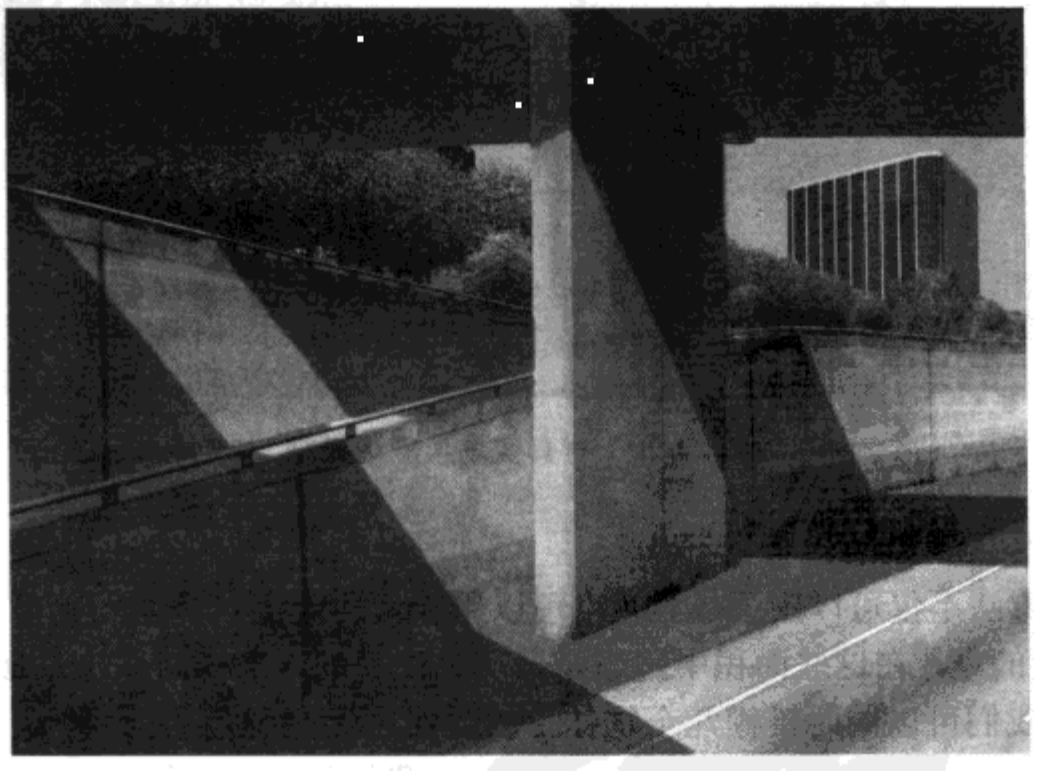


图 14-22

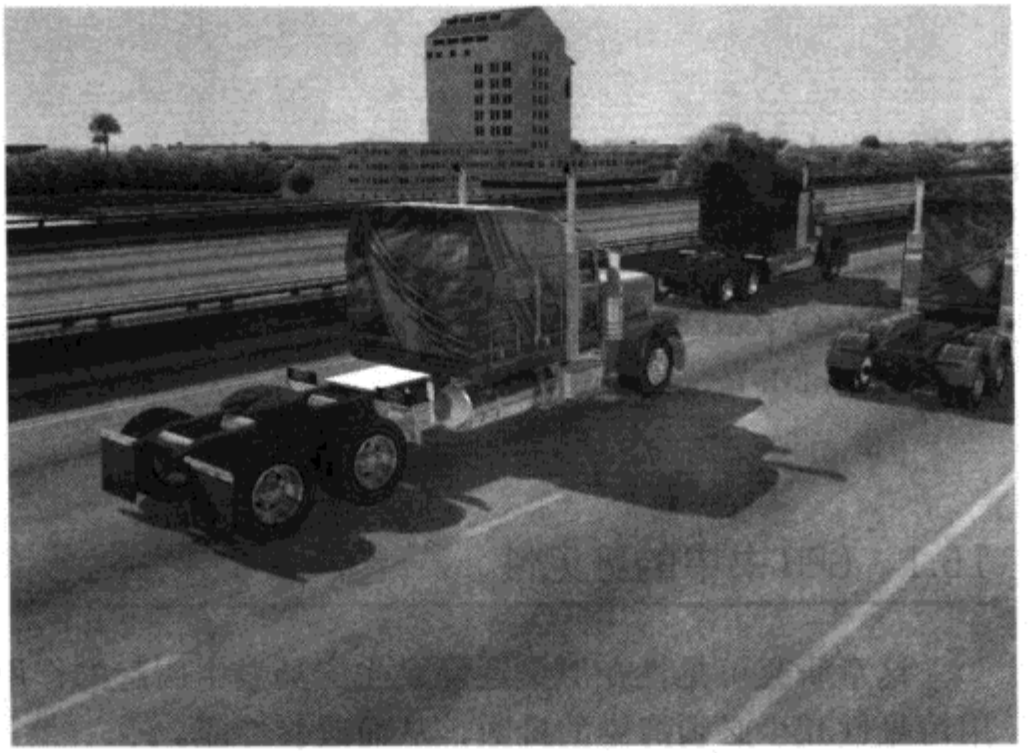


图 14-23

图 14-24

14.5　参考文献

Assarsson, U., M. Dougherty, M. Mounier, and T. Akenine-Möller. 2003. "An Optimized Soft Shadow Volume Algorithm with Real-Time Performance." In *Proceedings of the SIGGRAPH/ Eurographics Workshop on Graphics Hardware 2003.*

Gartner, Bernd. 1999. "Smallest Enclosing Balls: Fast and Robust in C++." Web page. http://www.inf.ethz. ch/personal/gaertner/texts/own_work/esa99_final.pdf

Stamminger, Marc, and George Drettakis. 2002. "Perspective Shadow Maps." In *Proceedings of ACM SIGGRAPH 2002.*

感谢 Peter Popov 给予的大量帮助和建议。

第15章 逐像素光照的可见性管理

John O'Rorke
Monolith Productions

15.1 GPU书中的可见性

这章研究可见性在渲染逐像素光照场景中所起的作用。也考虑如何使用可见性减少必须渲染的批数，从而改善性能。

乍看起来，可能会认为在图形硬件高级应用的书中没有可见性的地位。不管在GPU上使用多少技巧和优化，最快的多边形仍然是没有渲染的那个。这意味着如果减少所渲染的批数，就可以在场景中加入更多的物体，或对物体使用更复杂的几何体和技术。

无数的文章和介绍都宣扬可以减少进入图形卡的批数，以避免变成受限于CPU的应用程序。为使术语清楚，本章中"批"是指任何一组多边形，把它送到图形卡时未通过状态变化打碎过它。例如，在Direct3D中一个DrawPrimitive调用就代表一个批。从前批的减少就很重要，而对最新的GPU就更加重要了。GPU处理批越来越快，即在一个批里能够更快地处理多边形。但是把批送到图形处理器的速率却增加得非常缓慢。目前逐像素光照的趋势更使问题加剧，因为这会大量地增加渲染场景所要求的批数。

15.2 批和逐像素光照

使用逐像素光照时批数明显增多，这是由这个技术的实现方式引起的。这个算法要渲染一遍环境，目的是为这帧应用全局环境项和建立深度缓冲区。还必须对每个光源求出两个值：阴影项和光照贡献值。

阴影项时常用延伸模板体或阴影映射来计算，这两者对每个物体至少要渲染一个批。阴影项求出后，为了应用光照的贡献，必须再次渲染光源影响到的各个物体，也包含某些在阴影中已经被适当地标出的物体。

许多逐像素光照的开发者，力图批化场景并有效地渲染。结果，由于场景中的光源数，这个新技术所需的批数使早先的问题更加严重，更加难以有效渲染。

15.2.1 逐像素光照的例子

为了说明渲染逐像素光照的场景需要多少批，让我们考查一个非常简

单的情景。我们有一个房间，因为房间的不同部位是不同的材质，所以将其分为 8 个分别的批。另外，在房间中有 3 个不同的模型，每个模型又被分成 2 个批，为的是进行矩阵调色板蒙皮。这个房间里还有 3 个光源。

现在来决定需要多少批。在环境遍中，为了建立场景的深度值和应用环境或发射光照，必须将所有的批渲染一次。这样就有 14 个批需要启动。为了把光源累积进帧缓冲区内，针对每个光源，为阴影要对所有的批渲染一次，为真实的光照遍再渲染一次。于是这个简单场景已经达到了 98 个批。大约一秒钟可以传送 10 000 到 40 000 个批到图形硬件，这要消耗 1GHz 处理器的全部 CPU 时间（Wloka 2003）。如果那个 CPU 速率是我们的最低要求，而且只有 10%能用于批提交，那么每秒钟只可能提交 1 000 到 4 000 个批！因此，在这个简单的场景中，批提交将帧速率限制为每秒钟 10 到 40 帧。真实场景所要求的批，往往比这个例子的多得多，要使批数合理必须付出许多努力。

这章的意图不是考查各种可见性算法，或实现可见性系统（可见性算法的讨论在 Akenine-Möller and Haines 2002 中）。而是举例说明，应该如何使现有的可见性算法适合于逐像素照明的独特需求，将发送到硬件的批数减到最小。

15.2.2　究竟需要多少批

下列的伪代码举例说明在一个场景中必须渲染的批数。

```
For each visible object
  For each pass in the ambient shader
    For each visible batch in the object
      Render batch
For each visible light
  For each visible shadow caster
    For each pass in the shadow shader
      For each shadow batch in the object
        Render batch
For each lit visible object
  For each pass in the light shader
    For each visible batch in the object
      Render batch
```

正如伪代码表示的，为了减少批数可以进行一些与非可见性相关的优化。最应该优化的是渲染每个光照所必需的遍数。批数随遍数线性地增加，因此，我们应该最小化受限于 CPU 的游戏的遍数。

伪代码也显示了用于阴影渲染的不同批。虽然延伸阴影体总是使用分别的批，但阴影贴图实现却很少这样做。因为执行阴影渲染时，时常移动某些批的边界，所以渲染正常区域和阴影区域使用不同的批是有益的。例如，使用两个独立的部分画一个有两种不同材质的网格。只有当我们执行光照操作，而且它们与阴影运算不相关的时候，材质才看得见。因此，在预处理一个模型时，可能产生两组批：一组是用来渲染光照明的，一组是用来渲染阴影的。减少批和减少遍这两个技术对减少批数都是必不可少的，但是单独使用任何一个都是不够的。通过可见性测试，可以在场景中剔除相当多的批，而且获得好得多的性能。

15.3 作为集合的可见性

为了了解如何使用可见性减少批数，我们先宏观地看可见性的使用和借助组的逻辑来定义运算。然后将介绍如何计算这些组。可见性不只是考虑场景的观察者，而且也考虑场景的每个光源，因为如果光源不能够见到一个物体，那个物体就不需要在光照遍中渲染。

15.3.1 可见集合

第一个定义的是可见集合，由从摄像机的视点可见的所有物体组成。几乎每个渲染应用都可以决定可见集合，我们定义为 V。

15.3.2 光源集合

除了在场景中发现哪个物体是可见的，还需要在可见物体集合中确定光源集合。

对于每个可见光源，必须从光源的视点创建另一个可见集合。让 L 表示从光源可见的物体集合。大多数逐像素光照的解每次只用一个光源，简单地在帧缓冲区累积结果。于是光源之间常常不存在相关性，所以对当前正在渲染的光源，每次只存在一个光源可见集合。这个技术避免每次把所有的光源集合都存储在存储器中；但是，通过简单地把结果累积进单个的照明集合内的方法，这个概念扩展为在单个遍中渲染多个光源。

既然已经为观察者和光源定义了可见性集合，我们能够建立几个集合，它们将减少在渲染中所画的物体数。

15.3.3 照明集合

决定照明集合的第一个规定是，照明遍发生在既包含于 V 集合又包含于 L 集合中的物体集合上，或使用集合符号：$V \cap L$。因为只有光源能见到的物体才需要为提供照明对它再次进行渲染；同时只有在可见集合中的物体，才能在屏幕上看得到。为了照明某物体集合需要再次对它进行渲染，指定该物体集合为 I。如果 I 是空的，我们就略去那个光源的渲染。这个规定一般可以很快而且很简单地应用，对处于视锥边界上的光源、半径非常大的光源，或遮挡在视域锥中的光源，它能够很好地进行优化。

15.3.4 阴影集合

对于光照遍已经把物体集合减小到合理的范围，现在来看阴影遍的物体集合。阴影集合的确定更加困难，需要在全面剔除的开销和渲染的物体数之间求得平衡。

一开始常见的错误是使用 I 集合作为阴影遍。但是，如图 15-1 所示，有时在锥体之外的物体把影子投进锥体之内，这可能影响最终的渲染图像。因此我们必须产生一个不同的组，叫做 S 组，它是一个 L 组的子集，它包括向可见区投射阴影的所有物体。

现在，我们已经定义了用逐像素光照进行渲染所需要的所有集合。首先，从摄像机产生 V 组，为环境遍而渲染它。接着是每个光源的渲染。对每个光源确定 L 集合，并且由此产生集合 I 和集合 S。集合 S 决定了光源的阴影项，然后对 I 集合内的每个物体应用光照渲染。

现在我们讨论有效地产生每个集合的细节。

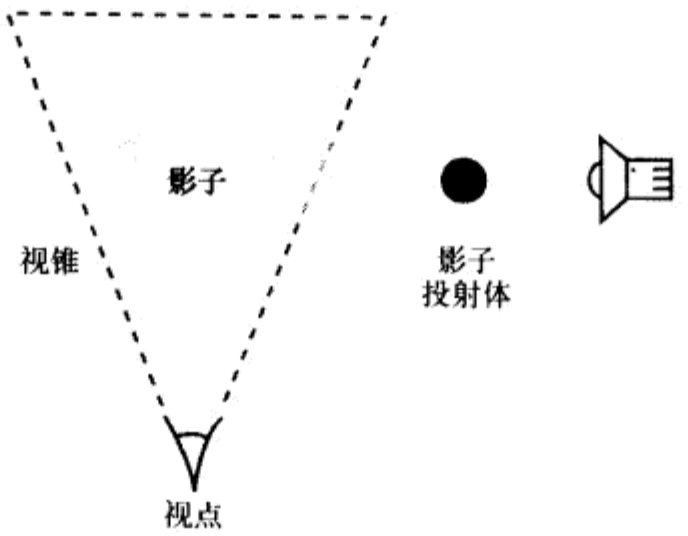


图 15-1　不在可见组中的物体可以影响所渲染的场景

15.4　各集合的生成

理论很伟大，但是微小的差别可以造成高压力或高帧速的不同结果。在这节中，我们概述前面所介绍的每个集合的产生细节，同时注意其实际应用。

15.4.1　可见集合的生成

需要光源和视口才能产生一个可见集合。但是可见集合要紧密到什么程度？确定这些集合应该花费多少处理时间？这取决于所开发的应用程序类型。至少确定可见性的算法应该执行锥体级的裁剪，和一定量物体级的遮挡。这种要求只是尺度问题。如果标准的可见集合包含锥体中的 10 个物体，而 30%的物体被遮挡，那么可以剔除 3 个物体。

因为是针对场景中的光源使用可见性，所以让我们看与光源相关的因素。如果每个光源的遮挡统计数相似，那么在真实场景中，就能够每帧少渲染数十个或甚至数百个物体。因此确定逐像素光照渲染的可见性系统时，应该小心地考虑在锥体里的遮挡程度。由此可见，决定最佳的 CPU 时间对遮挡之比是一个简单的平衡问题。

15.4.2　光源集合的生成

对光源集合和可见集合的决定几乎是相同的。但是点光源可能引起可见性算法（对平面进行投射）的问题。点光源有一个完整的球形视域，因此不能简单地映射到平面上，而许多可见性算法是依赖对平面的投射的。一个解决方案是把一个立方体放在光源周围，然后从光源的视点开始，对于立方体的每个面都执行可见性测试。因为需要为每个面做一次可见性检测，共做 6 次，所以这个方法十分费时；如果一个物体从多个面可见，还要小心不要把它重复地加入可见集合。一般来说，要考查可见性系统，如果它使用投射到平面上的形式，就要把它转换为没有任何投射的可见性系统，或实现一个专门查询点光源可见性的系统。

15.4.3　照明集合的生成

快速集合运算对有效地确定照度集合是至关紧要的。有两个方法可以实现必需的集合运算：（1）让集合知道它包含什么物体；（2）让物体知道它属于哪个集合。

第一个方法使用排序列表，它包含那个集合里的所有物体的引用。为了在线性时间内经

过合并检测交集和并集，列表必须是有序的，但是排序也可以基于任何合适的标准。因为包括排序、合并和偶尔的搜索，所以这个实现难以有效地执行，但是对于某些应用，如果其他操作需要这些集合，它也工作得很好。使用这个方法确定照度集合只是简单地检测组 V 和 L，然后求都包含在这两组里的元素。

第二个方法对于逐像素光照工作得特别好，物体所属的固定集合数可以作为物体的标记来存储。我们不是构建一个组，而是在物体的列表上执行操作。为了构建可见组，找到视域中所有的物体，把它们标记为可见集合。为了确定 L 集合，我们执行相似的过程，但是标记物体为光照组。然后通过查询光源可见的物体列表，并找到已经标记为可见集合的物体，于是 I 组就能很容易地确定。第二个方法的实现比第一个简单，但是在操作完成后，标记需要复位。

15.4.4 阴影集合的生成

阴影计算比其他运算更困难，因为涉及到光源所延伸的体积上的操作。物体常用包围盒或包围球代表（即围绕着可见几何体的立方体或球）。沿着图元的每点到光源的光矢量进行延伸，延伸的距离等于光源半径，物体的影子所占的空间体积即可确定。因此，当物体的光矢量从光源延伸到有效光源半径时，所有与视域锥体相交的物体，必须包含在阴影集合内。

在视锥内的光源的阴影

光源总是从光源位置向外投射阴影。因此，如果光源位于视锥内，从阴影集合剔除任何锥体外的物体是没有问题的。这是因为如果光源在一个凸的体内，而一个物体在体外，那么物体的影子总是向远离体积的方向延伸，因此不可能与视锥相交。

要确定哪个物体投射影子，需要使用视锥内的所有物体集合，而不是可见组 V，它只是视锥内物体的子集。组 V 可能遮挡不直接可见、但仍能向可见区投射影子的物体，如图 15-2 所举的例子。因此，定义一个新的集合 F，表示视域锥体内的没被遮挡的所有物体。如果光源在锥体内，那么阴影集合 S 由集合 F 和 L 的交集决定。

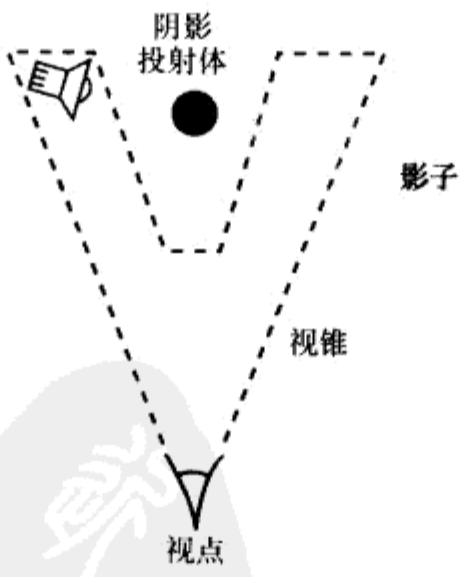


图 15-2 一个不在可见组中、但在锥体内的物体投射影子

在视锥外的光源的阴影

不幸的是，光源在锥体外的情况并不简单，因为光源能把锥体外物体的影子投射到视锥

之内。但是这只发生在特定的空间区域里，即围绕着视域锥体和光源的凸包（convex hull）（Everitt and Kilgard 2003）。一旦确定了这个凸包，判断一个物体是否投射影子就简单了，只要看它是否与这个凸包重叠即可。

这个方法的技巧是很快地决定包容光源和视域锥体的凸包。把一个点加到已经存在的凸包中去是一个棘手的情况，因此，我们使用一个简单的解法。把 6 个平面的锥体定义为空间的凸域，锥体所在的一边为面的内部空间，另一边为面的外部空间。我们的目标是创建一个体，它包含光源到视域锥体内每个点的连线。图 15-3 举例说明这个过程。

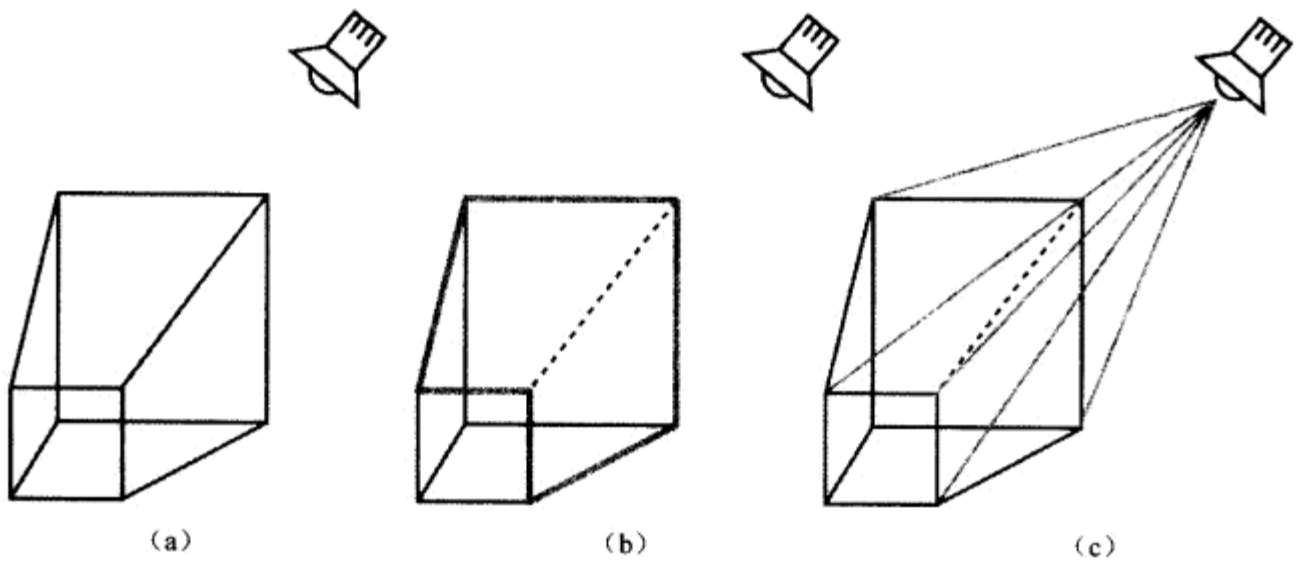


图 15-3　创建包含光源和视锥的凸包

（a）针对锥体的各个面检查光源，（b）把包容光源位置的面加到凸包上，并确定轮廓的边（以粗线表示），（c）在每个轮廓的边和光源之间加上面，形成凸包的其他部分

最后的凸包由两种平面组成，一种是原始视锥上的平面，这些平面把光源包在内部空间中。另一种是连接光源和每个轮廓边的平面。所谓轮廓边是光源包在内部空间和外部空间的两个平面的相交线。以上两种平面能简单而有效地求出，但是除了平面之外，视锥还需要另外的数据。这些数据列表如下：

- 视锥的边
- 视锥边上的两个点
- 一个边计数，用于检测与光源有关的轮廓边
- 一个映射，指出每个锥体平面定义了哪些轮廓边
- 每条边关于各平面的旋转次序

对视域锥体的每个平面，检查光源的位置。如果光源位置与视域锥体在平面的同一边，那么这个平面能直接地用于构造凸包。但是如果光源位置与视域锥体在平面的不同边，就要剔除这个平面。同时，受平面影响的各个边的边计数一定要增加。一旦检查过视域锥体的所有平面，就得到平面的列表和视域锥体的各个边的计数。

边计数是用来求轮廓边的。如果某条边的计数是零，光源位于构成边的两个平面的内部空间，那条边就不参与形成轮廓。如果某条边的计数超过 1，光源处于构成边的两个平面的外部空间，该边也不是轮廓边。如果某条边的计数为 1，这条边是光源投射的轮廓上的一条边，也就是说，由光源位置和这条边的两个点形成一个平面。生成这个平面的同时，要考虑边的旋转次序。只要光源处于那个平面的外部空间，就应该把边和边的旋转次序同时存储。标记边时应该朝向平面，标记为顺时针或反时针方向，旋转次序能够用来保证产生的平面朝

向正确的方向。这样做是有必要的，因为一条边是由两个平面定义的，不可能所有边的旋转次序相对于所有定义它的平面都一致。

轮廓边的算法求出一个锥体，它从光源的视点延伸出来，是相对于光源勾画的锥体。这个体应该是无限伸展的，但是它被把光源包含在内部半空间的平面所截断，这样有效地形成一个罩子，产成一个包围锥体和光源的凸包。

如下伪代码能很快地产生凸包。

```
Initialize all edge counts to zero

For each plane
  If the light is on the inside
    Add the plane to the final plane list
  Else
    For each edge the plane partially defines
      Increment the edge count
      Store the winding for this edge

For each edge
  If the edge count is equal to one
    Add a plane that includes the edge and the light point,
      flipping the plane normal if the winding is reversed
```

15.5 可见性改善填充率

可见性不仅能有效地改善 CPU 的操作性能，对 GPU 也是如此。当我们对模板体执行逐像素光照时，填充率的消耗（对模板体的填充或多次渲染大的物体）很快就变成瓶颈。克服这个瓶颈的最好办法，是使用剪切矩形（scissor rectangle）限制图形卡渲染的面积。最近 NVIDIA 的大部分卡上，剪切矩形的概念更进一步扩展了，操作起来像 z 剪切范围的深度测试，而且有效地模拟了空间的剪切锥体。

为了创建一个剪切矩形，在屏幕上投射光源的尺寸，并限制为只渲染那个区域。然而，因为光源覆盖了屏幕的大部分面积，所以即使对于中等大小的光源，这个方法的优势也会很快丧失。特别对大光源下的小面积尤其无效。在这种情形下，光源并不影响大部分荧屏；但是，因为光源半径，自然的实现有一个超出需要的非常大的剪切矩形。要画一个光源向下照亮大部分体积的通风井。为了照亮这个大的面积，光源必须有大半径，这导致了一个大的剪切矩形，即使它只是照亮这个占小部分屏幕面积的通风井。

为了保证剪切矩形是尽可能包紧，可以使用前面概述过的组。我们有在渲染时光源将要影响的物体的列表，而且能够确定图元投射在屏幕上的包围矩形。因此通过这个方法可以找到非常紧的剪切矩形，把光源影响的每个物体投射到屏幕上，找到那些物体的总包围盒，对投影的包围盒与光源本身执行一个交集。最好使用低级别的图元形状（比如包围球或轴对齐包围盒）进行投射，因为它们可能很耗时，而且不能有效地增加包围图元的紧密度。

通过执行这些操作（即使光源有极大的半径），我们可以创建一个剪切矩形，它不比投影在屏幕上的光源影响的物体的面积大。这个方法能明显地减少填充率，而且非常有助于使

帧速率与场景保持一致。

15.6　实际的应用

这章中提到的所有技术都已经在一个现有系统中实现，运行性能有了显著的改进。可见性解法是对视点和光源使用相同可见性系统的入门方法。

表 15-1 对启用的不同成分，以归一化的形式显示了每秒的帧数及各种场景所需的批数。对每个场景，帧速率是归一化的并且显示在以下情况中：

（a）没有使用可见性；（b）只对摄像机使用可见性；（c）对光源和摄像机使用可见性；（d）对光源、摄像机和阴影使用可见性；（e）加上剪切长方形。

表 15-1　不同可见性技术对性能的影响

场景 1	归一化的帧速率	批数	场景 2	归一化的帧速率	批数
基于视域锥体的可见性	0.87	1171	基于视域锥体的可见性	0.56	1414
可见性遮挡	0.92	492	可见性遮挡	0.80	521
光照明遮挡	0.99	468	光照明遮挡	0.98	438
阴影遮挡	0.99	460	阴影遮挡	0.98	437
剪切长方形	1.00	460	剪切长方形	1.00	437

正如表 15-1 显示的，把可见性引入场景可以显著性地改善帧速。场景的进一步测试表明，应用程序会在某点达到极限填充率，而由可见性得到的性能改善，是由于遮挡减少了在图形卡中光栅化的物体。

15.7　小结

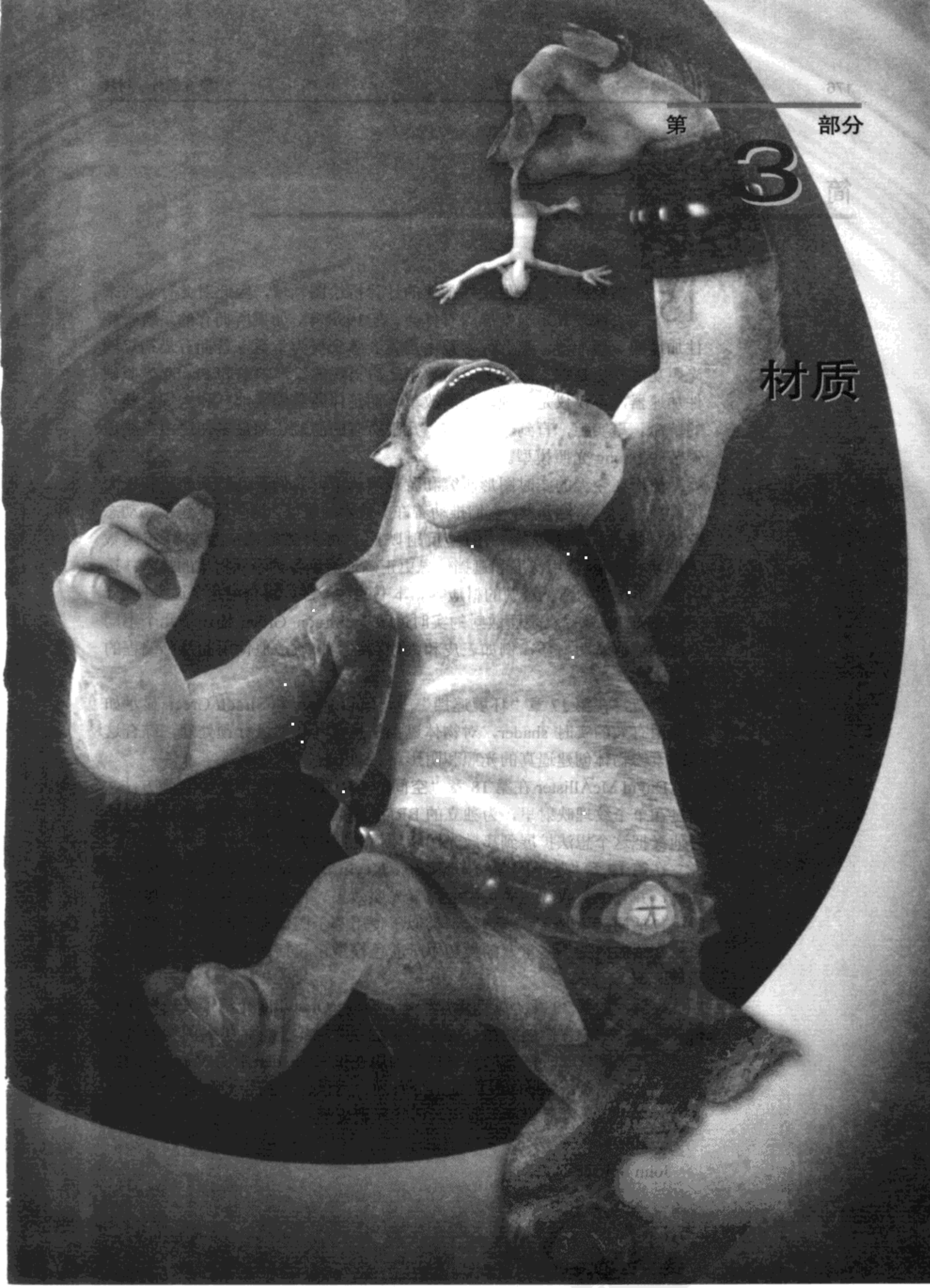
逐像素的照明需要大量的批数和极高的填充率，所以要减少渲染的物体数和它们影响的屏幕面积。使用本章所述的标准可见性算法和技术，可以充分地改善运行性能。

15.8　参考文献

Akenine-Möller, Tomas, and Eric Haines. 2002. *Real-Time Rendering,* 2nd ed. A. K. Peters. See the discussion of visibility algorithms on pp. 345～389.

Everitt, Cass, and Mark J. Kilgard. 2003. "Optimized Stencil Shadow Volumes." Presentation at Game Developers Conference 2003. Available online at **http://developer.nvidia.com/docs/IO/ 8230/GDC2003_ShadowVolumes.pdf**

Wloka, Matthias. 2003. "Batch, Batch, Batch: What Does It Really Mean?" Presentation at Game Developers Conference 2003.
Available online at **http://developer.nvidia.com/docs/IO/ 8230/BatchBatchBatch.pdf**



第 3 部分

材质

简 介

15年前我就开始涉足硬件加速的计算机绘图领域，因此对此行业的来龙去脉及其迅猛发展深有体会。在1988年，如果所拥有的系统带硬件加速的z缓冲器，那么大多数用户就会感到极为幸运，若拥有最初等的光照功能就更是锦上添花了。而数年之后出现的纹理映射和Phong光照硬件加速器，不仅仅是一项功能的改革，而是伴随着缩减消耗和大规模整合的革命。事实上，直到一两年前，几乎所有的游戏还都最多使用由多纹理调节的Phong光照模型。

然而在今天的实时图形研究和游戏演示中，我们看到复杂光照和表面反射模型的爆炸性开发，它们推动着许多技术的进展。这些技术包括高动态范围环境光照和预先卷积的漫反射映照、可分离的双向反射分布函数(BRDF)，以及使用深度和遮挡信息更正确地渲染各种不同材质的技术：透明的、半透明的或不透明的材质等。本书的这一部分将介绍几个这类技术。

在**第16章“次表面散射的实时近似”**，Simon Green描述了几个不同方法来近似透明材质，例如表皮和大理石这类材质，他使用的是可编程的图形硬件。

接着，在**第17章“环境遮挡”**中，Matt Pharr和Simon Green显示如何使用有效的实时shader，对物体遮挡信息及其环境进行预处理，综合这些因素给物体创建逼真的光照和阴影。

David McAllister在**第18章“空间的BRDFs”**中提出一个有趣方法，通过在单个纹理映射里，为独立的BRDF编码，表现多个复杂的材质。接着他又把这个想法扩展到离散光源和环境光照中去。

在**第19章“基于图像的光照”**中，**Kevin Bjorke**打破标准立方体映射环境，并且深入研究更多可能的逼真光照效果。首先他研究基于图像的光照，包括局部化的立方体映射，类似于使用基于图像渲染的局部光照，然后他介绍如何把那些重要的技巧用于着色模型，包括逼真的反射、阴影和漫反射/环境项。

最后，在**第20章“纹理爆炸”**中，Steve Glanville介绍了纹理爆炸和相关细胞技术，它们能够在shader中增加视觉的丰富性，扩大图像的可变性，并减少大块纹理图案的重复性。他用在像素shader中的Voronoi图案的例子做了总结，Voronoi图案十分有趣，可以作为描绘多种自然现象的shader的核心，包括鳞片、泡沫、树叶和表皮等。

John Spitzer, NVIDIA

第 16 章　次表面散射的实时近似

Simon Green
NVIDIA

今天，实时图形中的大多数着色模型只考虑光在物体表面的相互作用。但是，在真实的世界中许多物体是有点半透明的：光射进表面，在材质里散射，然后从与射入点不同的地方退出表面。

对有效而正确的次表面光传输的模型已经有了很多研究。虽然在目前图形硬件上还达不到完全正确的物理模拟，但有可能得到实时的近似视觉效果。这章介绍几种方法，用可编程的图形硬件，近似表皮和大理石等半透明材料。

16.1　次表面散射的视觉效果

要再生任何视觉效果，经常的做法是考查这种效果的图像，并把可视外表分解为它的组成要素。

在观察半透明物体的相片和图像时，能够注意到几点。首先，次表面散射往往使照明的整体效果变得柔和。一个区域的光线往往渗透到表面的周围区域，而小的表面细节变得看不清了。光线穿入物体越深，就衰减和散射得越严重。对于皮肤来说，在照亮区到阴影区的衔接处，散射也往往引起微弱的倾向红色的颜色偏移。这是由于光线照亮表皮并进入皮肤，接着被皮下血管和组织散射和吸收，然后从阴影部分离开。散射效果在皮肤薄的部位更加明显，比如鼻孔和耳朵周围。

16.2　简单的散射近似

近似散射的简单技巧是环绕光照（wrap lighting）。正常情况下，当表面的法线对于光源方向垂直的时候，漫射（Lambert）提供的照明度是 0。环绕光照修改漫射函数，使得光照环绕在物体的周围，越过那些正常时会变黑暗的点。这减少漫射光照明的对比度，从而减少了环境光和所要求的填充光的量。环绕光照是对 Oren-Nayar 照明模型的一个粗糙的近似，那个模型力图更精确地模拟粗糙的不光滑表面（Nayar and Oren 1995）。

显示在这里的代码和图 16-1，说明了如何改变漫射光照函数，使它包含环绕效果。环绕值是范围在 0 和 1 之间的浮点数，范围值控制光照环绕

物体周围的距离。

```
float diffuse = max(0, dot(L, N)) ;
float wrap_diffuse = max(0, (dot(L, N) + wrap) / (1 + wrap));
```

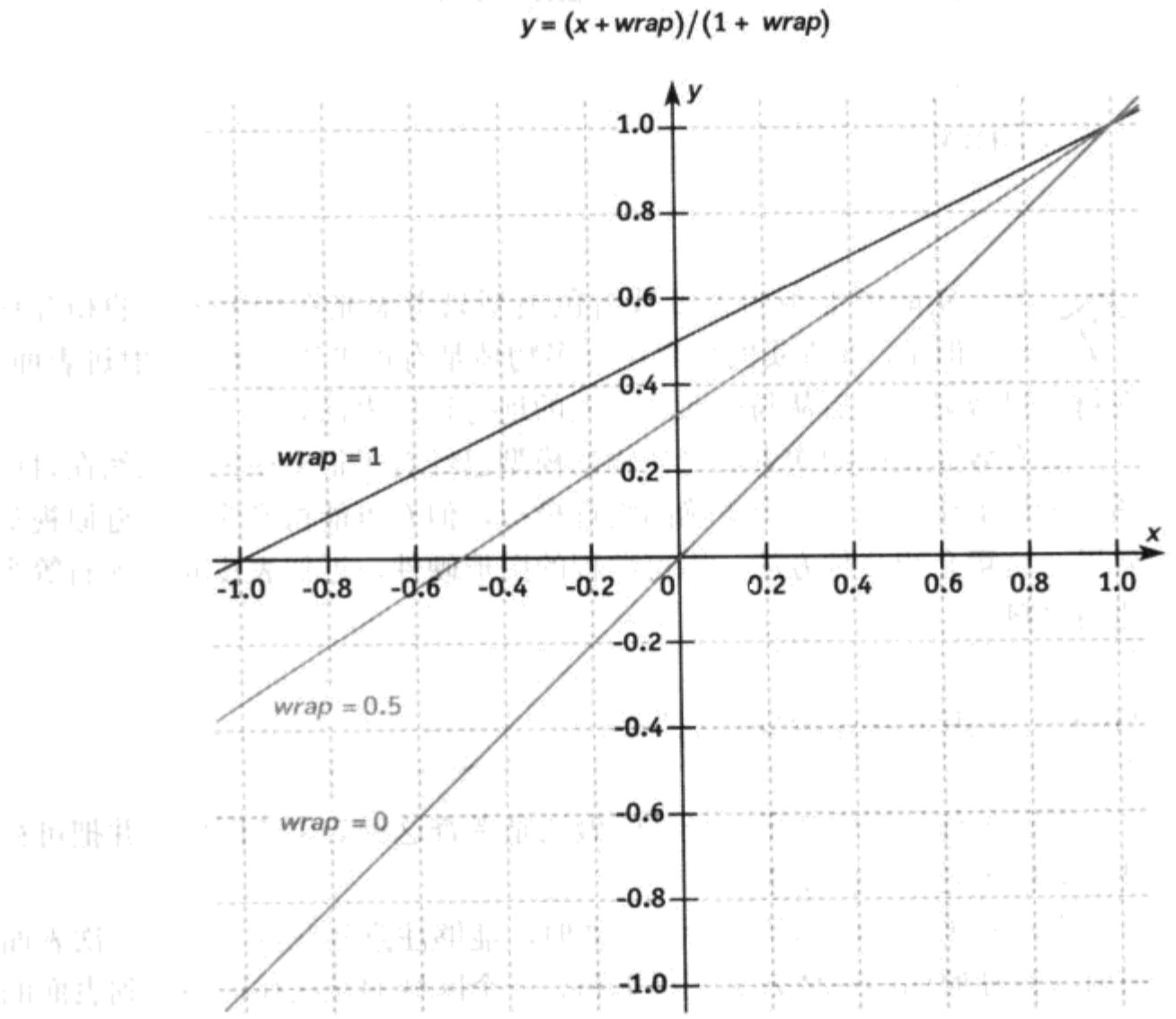


图 16-1 环绕光照函数的图表

为了在片元程序中有效地计算，函数可以编码到纹理中，用光矢量和法线的点积为索引。这个纹理也可以创建为，在照明度接近 0 时，表达那种倾向红色的微小颜色漂移。这是模拟皮肤散射的一种廉价方法。同样的纹理的 alpha 通道也包含了的高光镜面照明的幂函数。列表 16-1 的 FX 代码示范了如何使用这种技术，见图 16-2 的例子。

（a）没有环绕光照明的球体　（b）有环绕光照明的球体　（c）有环绕光照明和颜色漂移的球体

图 16-2 把环绕光照明应用于球体

列表 16-1　合并环绕照明的皮肤 shader 效果的摘录

```
// 为皮肤着色产生 2D 查找表
float4 GenerateSkinLUT(float2 P : POSITION) : COLOR
{
  float wrap = 0.2;
  float scatterWidth = 0.3;
  float4 scatterColor = float4(0.15, 0.0, 0.0, 1.0);
  float shininess = 40.0;

  float NdotL = P.x * 2 - 1; // 把[0, 1]重映射到[-1, 1]
  float NdotH = P.y * 2 - 1;

  float NdotL_wrap = (NdotL + wrap) / (1 + wrap); // 环绕光照
  float diffuse = max(NdotL_wrap, 0.0);

  // 在从明到暗的转变中添加色调
  float scatter = smoothstep(0.0, scatterWidth, NdotL_wrap) * smoothstep(scatterWidth *
                  2.0, scatterWidth, NdotL_wrap);

  float specular = pow(NdotH, shininess);
  if (NdotL_wrap <= 0) specular = 0;
  float4 C;
  C.rgb = diffuse + scatter * scatterColor;
  C.a = specular;
  return C;
}
```

```
// 使用查找表着色皮肤
half3 ShadeSkin(sampler2D skinLUT,
                half3 N,
                half3 L,
                half3 H,
                half3 diffuseColor,
                half3 specularColor) : COLOR
{
  half2 s;
  s.x = dot(N, L);
  s.y = dot(N, H);
  half4 light = tex2D(skinLUT, s * 0.5 + 0.5);
  return diffuseColor * light.rgb + specularColor * light.a;
}
```

16.3　用深度映射模拟吸收

吸收是模拟半透明材质最重要的因素之一。光线在物质中传播得越远，它被散射和吸收得就越厉害。为了模拟这种效果，我们需要测量光在物质中传播的距离。

估算这个距离的方法是使用深度映射（Hery 2002）。这个技术非常类似于阴影映射，而且可用于实时渲染。在第一遍中，我们从光源的视点渲染场景，存储从光源到某个纹理的距

离。然后使用标准的投射纹理映射，再把这个图像投射回场景。在渲染遍中，给定一个要着色的点，查询这个纹理，可以得到从光源到表面上入射点之间的距离（d_i），和从光源到射出点之间的距离（d_o），后者减去前者，就获得光线在物体中传播的距离（s），见图 16-3。

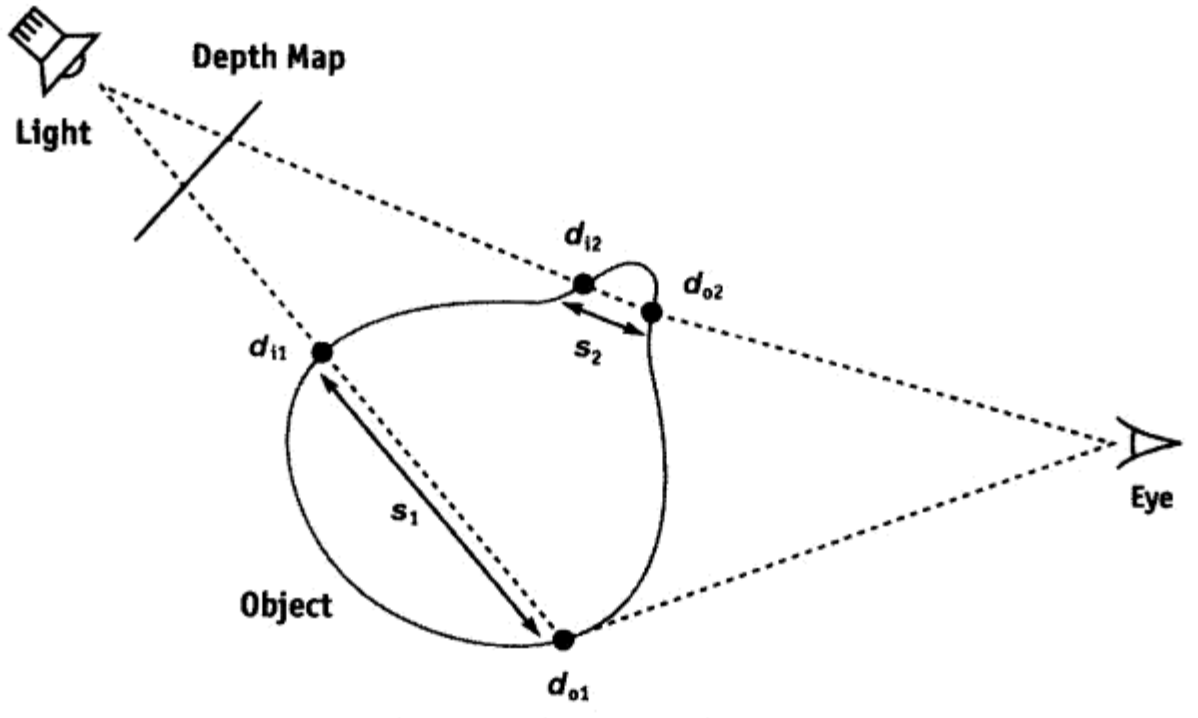


图 16-3 使用深度映射计算光在物体中传播的距离

这个技术的问题是它只对凸的物体有效，不能正确描绘物体上的洞。在实际中，这不是大问题，但是可能造成使用深度剥离相关的问题，它会一层又一层地移除物体（Everitt 2003）。

对于静态的物体，也许可以绘制或预计算一个映射，以表现表面每个点的大约厚度。之所以使用深度映射，好处在于考虑到了光的入射方向，而且也可以用于动画模型（假定要每帧再产生深度映射）。

列表 16-2 和 16-3 的程序示范了如何渲染从光源到纹理的距离。假定通过应用程序为光源视点建立了 modelView 和 modelViewProj 矩阵。

列表 16-2 深度遍的顶点程序

```
struct a2v {
  float4 pos    : POSITION;
  float3 normal : NORMAL;
};
struct v2f {
  float4 hpos : POSITION;
  float dist  : TEXCOORD0; // distance from light
};

v2f main(a2v IN,
         uniform float4x4 modelViewProj,
         uniform float4x4 modelView,
         uniform float    grow)
{
  v2f OUT;
  float4 P = IN.pos;
  P.xyz += IN.normal * grow; // scale vertex along normal
```

```
  OUT.hpos = mul(modelViewProj, P);
  OUT.dist = length(mul(modelView, IN.pos));
  return OUT;
}
```

列表 16-3　深度遍的片元程序

```
float4 main(float dist : TEX0) : COLOR
{
  return dist; // return distance
}
```

在列表 16-4 中的片元程序显示了如何查询光线距离纹理以计算深度。为了灵活性，这段代码在片元程序中进行投影，但是如果只取少量样本，在顶点程序中计算这些变换会更有效。

列表 16-4　使用深度映射，计算渗透深度的片元程序函数

```
// 给定物体空间的一点，进入深度纹理内查询
//返回深度
float trace(float3 P,
            uniform float4x4 lightTexMatrix, // 到光源纹理空间
            uniform float4x4 lightMatrix,    // 到光源空间
            uniform sampler2D lightDepthTex,
            )
{
  // 把点转变到光源纹理空间
  float4 texCoord = mul(lightTexMatrix, float4(P, 1.0));

  // 在进入点从光源得到距离
  float d_i = tex2Dproj(lightDepthTex, texCoord.xyw);

  // 把位置转变到光源空间
  float4 Plight = mul(lightMatrix, float4(P, 1.0));

  // 从光源（退出）到这个像素的距离
  float d_o = length(Plight);

  // 计算深度
  float s = d_o - d_i;
  return s;
}
```

一旦计算出光在物质中传播的距离，就可以有几个方法使用它。一个简单的方法是直接用它索引美工所创建的一维纹理，纹理把距离映射为颜色。颜色应该随着距离呈指数衰减。通过改变这个颜色映射，并与其他更传统的光照明模型相结合，就可以生成不同材质的图像，例如大理石或玉。

```
float si = trace(IN.objCoord, lightTexMatrix, lightMatrix, lightDepthTex);
return tex1D(scatterTex, si);
```

我们也可以选择直接计算指数函数：

```
return exp(-si * sigma_t) * lightColor;
```

这个技术的问题是没有模拟光通过物体传播所造成的漫反散。当光源在物体后面的时候，会清楚地看到物体背面的特征显示到前面。解决的办法是在表面的不同点多次取样，或如下节所讨论的那样使用不同的近似漫反射。

16.3.1 实现细节

在 GeForce FX 硬件上，当读取深度纹理时，只能得到 8 位的深度值。这种精度是不够的。可以使用浮点纹理改善它，或使用 NVIDIA 的片元程序扩展里的打包和解包指令，把 32 位的浮点数存储在普通的 8 位 RGBA 纹理中。浮点纹理目前不支持过滤，因此当所投射的纹理放大时，有时会看见失真。如果必要的话，可以在 shader 中执行双线过滤，尽管这会降低一些性能。

投射深度映射的另一个问题是时常在投射的边缘周围出现走样。这与阴影映射时见到的自阴影走样类似。主要是由于纹理映射的分辨率有限，结果把背景的像素投射到物体的边缘上。示范的代码在深度映射的遍中，沿着顶点法线稍微放大了物体，从而避免了这个问题。

为了更准确地模拟，需要知道光在物体上入射点的法线，及潜在的表面颜色。可以通过在另外的遍渲染额外的信息到纹理的方法来办到这点。我们以类似于深度纹理的方法查询这些纹理。在支持多重渲染目标的系统上，可以将深度、法线和其他的遍整合进一个多值输出遍，见图 16-4。

图 16-4 使用深度映射去近似散射
物体上薄的部位传输更多的光

16.3.2　更精密的散射模型

更精密的模型试图精确地模拟在介质里散射的累积效应。

一种模型是单次散射的近似，它假设光在材质中只反弹一次。沿着材质内的折射光线，可以计算有多少光子会朝向摄像机散射。当光击中一个粒子的时候，光散射方向的分布用相位函数来描述。入射点和出射点的 Fresnel 效应也很重要。

另一个模型，近似漫反射，模拟高散射介质（例如皮肤）的多次散射效果。

但是这些技术超出了本章范围。

SIGGRAPH 2003 RenderMan 课程（Hery 2003）的 Christophe Hery 章节，讨论了皮肤 shader 的单次和漫反射散射的细节。

16.4　纹理空间的漫反散

次表面散射最明显的视觉特征之一是一般模糊的光照效果。事实上，3D 美工时常在屏幕空间中仿效这个现象，他们在 Adobe Photoshop 上执行 Gaussian 模糊，然后把模糊图像少量地覆盖在原始的图像上。这种“辉光”技术使照明变柔和，而且使图像看起来不太像由计算机产生的。

在纹理空间中模拟漫反射是可能的（Borshukov and Lewis 2003）。我们可以用顶点程序展开物体的网格，程序使用纹理坐标 UV 作为顶点的屏幕位置。程序简单地把[0, 1]范围的纹理坐标重映射为[−1, 1]的规范化的坐标。我们必须使物体有好的 UV 映射；也就是说，在纹理上的每个点必须映射为物体上的惟一点，不能重叠。通过以正常的方式照明这个展开的网格，我们就获得了代表物体光照明的 2D 图像。然后可以处理这个图像，并且再度把它像正常的纹理一样用于 3D 模型。

列表 16-5 的顶点程序示范如何渲染 UV 空间的模型，并执行漫反射照明。

列表 16-5　用于展开模型和执行漫反射光照的顶点程序

```
struct a2v {
  float4 pos      : POSITION;
  float3 normal   : NORMAL;
  float2 texture : TEXCOORD0;
};

struct v2f {
  float4 hpos      : POSITION;
  float2 texcoord : TEXCOORD0;
  float4 col       : COLOR0;
};

v2f main(a2v IN,
         uniform float4x4 lightMatrix)
{
  v2f OUT;

  // 将纹理坐标重映射为 [-1, 1] 的规范化的坐标位置
```

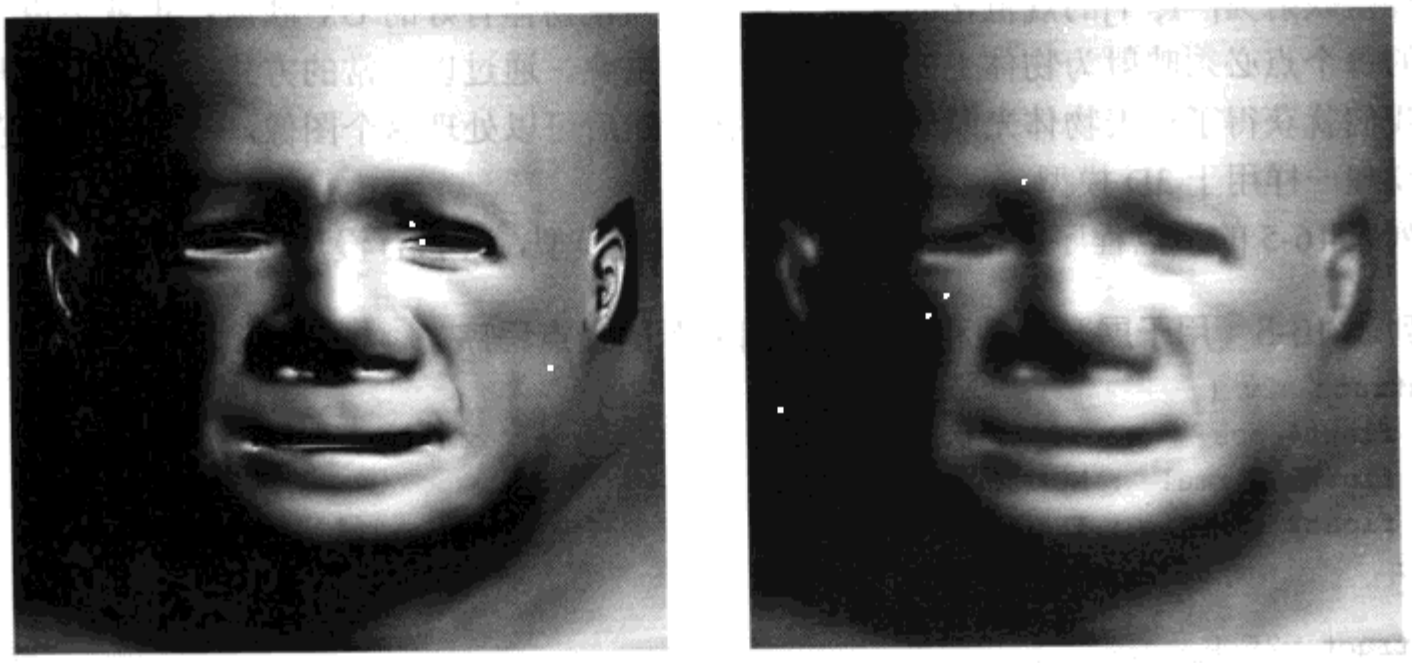
```
    OUT.hpos.xy = IN.texture * 2 - 1;
    OUT.hpos.z = 0.0;
    OUT.hpos.w = 1.0;

    // 漫反射光
    float3 N = normalize(mul((float3x3) lightMatrix, IN.normal));
    float3 L = normalize(-mul(lightMatrix, IN.pos).xyz);
    float diffuse = max(dot(N, L), 0);
    OUT.col = diffuse;

    OUT.texcoord = IN.texture;
    return OUT;
}
```

这个技术可以用于其他应用，因为它减轻屏幕分辨率造成的着色复杂性：现在着色只是在纹理映射的每个纹素上执行，而不是在物体上的每个像素上执行。比如卷积那样的许多运算，在图像空间比在 3D 表面上执行起来更有效。因为在世界空间中靠近的点，映射到纹理空间中也是靠近的点，所以如果表面的 UV 参数化是相对均匀的，近似起来也会很好。

为了模拟图像空间的漫反射，可以简单地模糊光照映射纹理。我们可以利用一切通常的 GPU 图像处理技巧，例如使用可分离的滤波器，以及双线性过滤硬件。对相对低分辨率的纹理渲染照明，已经提供了某种程度的模糊。图 16-5 显示了展开的头部网格，以及模糊光照映射纹理的结果。



（a）模糊前　　（b）模糊后的网格

图 16-5　展开的头部网格

漫射彩色映射也能包含在光照映射纹理中，然后彩色映射的细节也会被漫反射。如果阴影包含在纹理中，模糊的过程将造成模糊的阴影。

为了模拟吸收和散射与波长相关的事实，我们对每个彩色通道分别地改变滤波权重。在列表 16-6 和 16-7 中显示了 shader 样本，目的是模拟皮肤。它是有 Gaussian 权重的 7 个纹理样本。红色通道的滤波器宽度比绿色和蓝色通道的更宽，所以红色比其他通道漫反射得更厉害。列表 16-6 的顶点程序为 x 方向的模糊计算样本位置；y 方向的程序相同。为了利用硬件

的双线性滤波能力，在分开的两个纹素中取样。

列表 16-6　用于漫反射模糊的顶点程序

```
v2f main(float2 tex : TEXCOORD0)
{
  v2f OUT;
  // 7 samples, 2 texel spacing
  OUT.tex0 = tex + float2(-5.5, 0);
  OUT.tex1 = tex + float2(-3.5, 0);
  OUT.tex2 = tex + float2(-1.5, 0);
  OUT.tex3 = tex + float2(0, 0);
  OUT.tex4 = tex + float2(1.5, 0);
  OUT.tex5 = tex + float2(3.5, 0);
  OUT.tex6 = tex + float2(5.5, 0);
  return OUT;
}
```

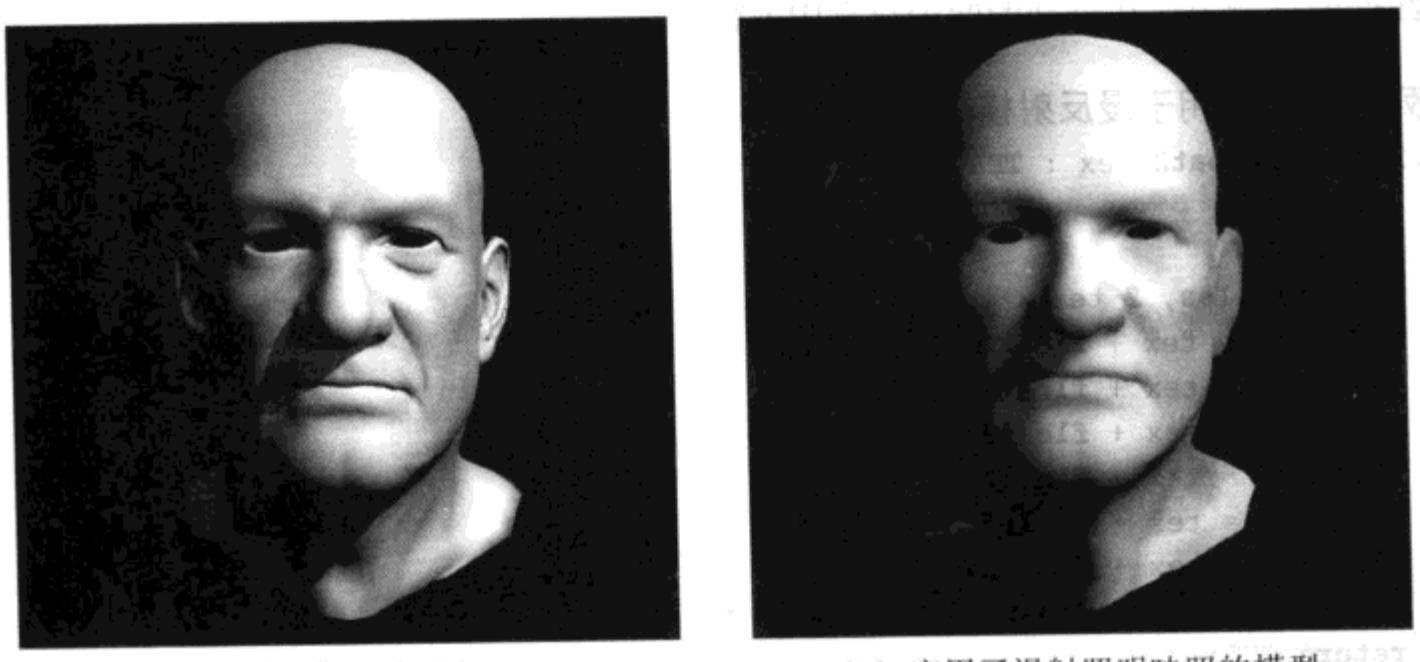
列表 16-7　用于漫反射模糊的片元程序

```
half4 main(v2fConnector v2f,
           uniform sampler2D lightTex
           ) : COLOR
{
  // 使红色通道比绿色和蓝色通道更加模糊的加权
  const float4 weight[7] = {
    { 0.006, 0.0, 0.0, 0.0 },
    { 0.061, 0.0, 0.0, 0.0 },
    { 0.242, 0.25, 0.25, 0.0 },
    { 0.383, 0.5, 0.5, 0.0 },
    { 0.242, 0.25, 0.20, 0.0 },
    { 0.061, 0.0, 0.0, 0.0 },
    { 0.006, 0.0, 0.0, 0.0 },
  };

  half4 a;
  a = tex2D(lightTex, v2f.tex0) * weight[0];
  a += tex2D(lightTex, v2f.tex1) * weight[1];
  a += tex2D(lightTex, v2f.tex2) * weight[2];
  a += tex2D(lightTex, v2f.tex3) * weight[3];
  a += tex2D(lightTex, v2f.tex4) * weight[4];
  a += tex2D(lightTex, v2f.tex5) * weight[5];
  a += tex2D(lightTex, v2f.tex6) * weight[6];
  return a;
}
```

为了得到更大的模糊，可以多次重复应用模糊 shader，或写一个 shader，通过在片元程序中计算样本位置取更多样本。图 16-6 显示了把模糊光照映射纹理又用到 3D 头部模型。

最后的 shader 用最初的高分辨率彩色映射，混合被漫反射的照明纹理，获得最终的效果，如图 16-7 所示。



（a）原始的模型 （b）应用了漫射照明映照的模型

图 16-6 在头部模型上的纹理空间的漫反射

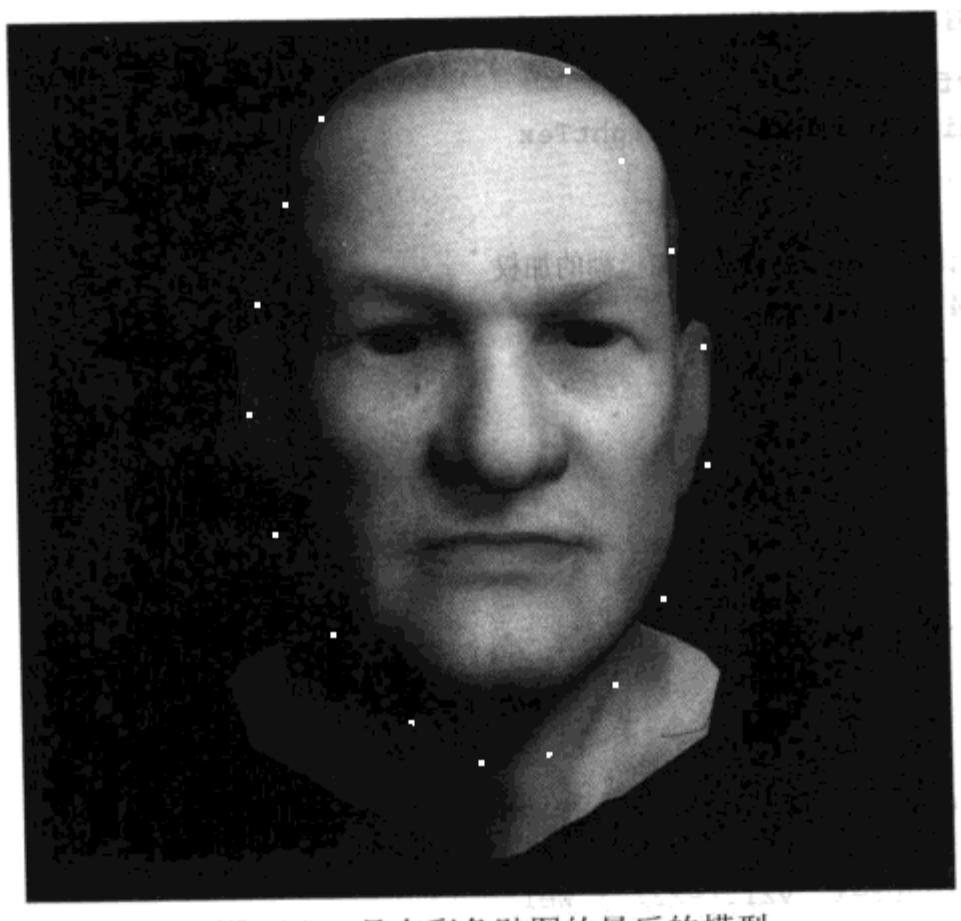


图 16-7 具有彩色贴图的最后的模型

为描绘物体中更密集的物体（例如身体里的骨头），渲染额外的深度遍是深度映射技术可能的扩展。但是使用基于表面的方法去考虑体效果，会出现问题。体渲染没有这个限制，而且当物体的密度变化时，它能产生更加正确的渲染。关于更多的体渲染，见本书的第 39 章“体渲染技术”。

这个技术另外可能的扩展是提供几个彩色贴图，每个代表皮肤的不同层次的颜色。举例来说，可以给表皮的颜色提供一个贴图，而给皮肤下的血管和毛细管提供另一个贴图。

Greg James（2003）介绍了一种技术：处理任意多边形的物体时，首先加上所有背表面的距离，然后减去所有前表面的距离。应用程序计算屏幕空间的距离，用于产生体雾效果，

还有可能扩展到更一般的情形。

未来研究的有趣方向，是结合深度贴图和纹理空间技术，以获得两者最好的效果。

16.5　小结

要产生使人信服的皮肤和其他半透明材质的图像，次表面散射的效果是十分重要的因素。使用几个不同的近似，我们已经得到实时的次表面散射的效果。随着图形硬件变得更为强大，可能会出现更为精确的次表面光传播模型。

我们希望这章中介绍的技术有助于改善实时游戏中人物的逼真度。但是要记住的是，要有好的艺术、好的着色法才能给予帮助!

16.6　参考文献

Borshukov, George, and J. P. Lewis. 2003. "Realistic Human Face Rendering for 'The Matrix Reloaded.' " SIGGRAPH 2003. Available online at

http://www.virtualcinematography.org/

Everitt, Cass. 2003. "Order-Independent Transparency." Available online at

http://developer.nvidia.com/view.asp?IO=order_independent_transparency

Hery, Christophe. 2002. "On Shadow Buffers." Presentation available online at

http://www.renderman.org/RMR/Examples/srt2002/PrmanUserGroup2002.ppt

Hery, Christophe. 2003. "Implementing a Skin BSSRDF." RenderMan course notes, SIGGRAPH 2003. Available online at

http://www.renderman.org/RMR/Books/sig03.course09.pdf.gz

James, Greg. 2003. "Rendering Objects as Thick Volumes." In *ShaderX2: Shader Programming Tips & Tricks With DirectX 9,* edited by Wolfgang F.Engel. Wordware. More information available online at **http://www.shaderx2.com**

Jensen, Henrik Wann, Stephen R. Marschner, Marc Levoy, and Pat Hanrahan. 2001. "A Practical Model for Subsurface Light Transport." In *Proceedings of SIGGRAPH 2001.*

Nayar, S. K., and M. Oren. 1995. "Generalization of the Lambertian Model and Implications for Machine Vision." *International Journal of Computer Vision* 14, pp. 227～251.

Pharr, Matt. 2001. "Layer Media for Surface Shaders." Advanced RenderMan course notes, SIGGRAPH 2001. Available online at

http://www.renderman.org/RMR/Books/sig01.course48.pdf.gz

Statue model courtesy of De Espona Infographica (**http://www.deespona.com**). Head model courtesy of Steven Giesler (**http://www.stevengiesler.com**).

第 17 章 环境遮挡

Matt Pharr
NVIDIA
Simon Green
NVIDIA

最近这种方法开始得到实时计算机图形业的赏识：它不是使用少数简单光源，而是用环境贴图的复杂光分布照明物体，从而增加逼真度。在真实世界中，光不只是来自点光源或方向光源，而是从所有的方向到达物体表面，这明显地影响着物体的外观。最近已经开发多种技术，以求得真实世界的照明（比如在电影中的情况），并且使用它渲染物体，使物体好像由天然环境光照明一样，让计算机图形与真实场景惟妙惟肖。对于完全人造的场景，可以用这些技术渲染场景的环境贴图，并用它照亮场景里的人物和其他物体，从而改善渲染图像的逼真度。使用贴图不仅能得到完美的高光反射，也可以用它计算光滑的和漫反射的表面。

这章介绍用于实时环境照明的简单技术。虽然仅限于漫反射表面，但是足以实时使用。此外，此方法还能正确求出着色点上由于环境中几何体遮挡而产生的阴影。与最近文献中更复杂的技术比较，这个技术计算的着色值可能存在若干误差，但是它相对容易实现（不一定要了解和实现了球面调和库才能使它工作）。这个方法在许多情况下可以给出极好的结果，而且它在现代硬件上可以交互式运行。

这个方法基于与视点无关的预处理，用光线跟踪计算遮挡信息，然后使用这个信息在运行期快速计算环境的漫反射着色近似值。这个技术最初由 Industrial Light & Magic 的 Hayden Landis（2002）和同事开发，它已经用在几个 ILM 的产品上（非实时的渲染）。

17.1 概述

我们描述的环境照明技术已经命名为环境遮挡照明。可以将这个方法看作“智能”的环境项，它在模型表面的变化取决于在每点可见多少外部环境。也可以把它看作漫反射项，它支持入射光的复杂分布。在这章中我们考虑第 2 种解释。

这个技术的基本思路是，假如预处理一个模型，计算它上面每个点可以看到多少外部环境，可以相反地计算有多少环境被模型的其他部分遮挡，

然后在渲染时使用这个信息计算漫反射阴影项的值。结果模型上的裂缝变暗，而暴露部分接收更多的光，因此更亮。这种效果实质上比使用标准的着色模型更逼真。

这个方法可以扩展为使用环境光照作为照明源，用代表各个方向入射光的环境贴图，来决定物体上每点光的颜色。为了这个特性，除了记录在点上可以看到多少环境，也记录大部分可见光从哪个方向到达。这两个量有效地定义了从外面进入场景的未被遮挡的方向圆锥体，可以一起用来做来自环境贴图的极端模糊的查询，模拟着色点上从感兴趣的方向圆锥体来的全部入射照度。

17.2　预处理步骤

这个技术对要着色的模型，需要知道模型上每点的两个信息：（1）该点的“可到达性”，即点上半球的多大部分不被模型的其他部分所遮挡；（2）未被遮挡的入射光的平均方向。图 17-1 说明在平面上的这两个概念。给定在表面上的点 P，法线为 $\boldsymbol{N}$，这里 P 上半球的 2/3 被场景中其他几何体遮挡，半球的 1/3 不被遮挡。入射光的平均方向用 $\boldsymbol{B}$ 指示，它在法线 $\boldsymbol{N}$ 的右边。大致地说，在 P 的入射光的平均颜色，可以通过求围绕 $\boldsymbol{B}$ 矢量的未遮挡入射光的圆锥体的平均值得到。

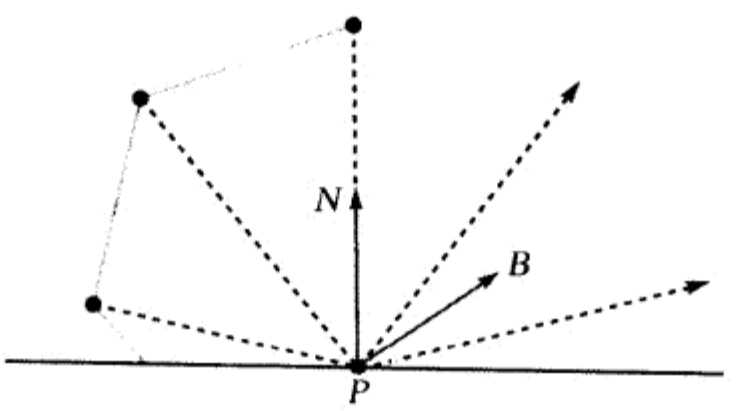


图 17-1　可到达度和平均方向的计算

这个 $\boldsymbol{B}$ 矢量和可到达值，只与模型相关而与照明无关，可以用光线跟踪进行离线预处理。在本章的例子中，使用的模型是精细的棋盘格，因此我们在每个三角形的中心计算这些数值，然后把数值存储在每个顶点，由它来保存相邻面数值的平均值。然后通过顶点 shader 传递每个顶点值，以便用片元 shader 在每个像素上内插。也可以把它们存储在纹理贴图中；假如模型是粗糙棋盘格，结果会更好。

列表 17-1 中的伪代码显示了我们的基本方法。在每个三角形的中心，我们产生一组以表面法线为中心的半球形光线。我们跟踪每道光线进入场景，记录哪些光线与模型相交，标志不能从环境接收的光线，以及不被遮挡的光线。我们计算不被遮挡光线的平均方向，它是入射光平均方向的近似值（当然，计算出的方向本身就有可能是被遮挡的，我们忽略这个问题）。

列表 17-1　计算环境遮挡量的基本算法

```
For each triangle {
  Compute center of triangle
  Generate set of rays over the hemisphere there
```

```
Vector avgUnoccluded = Vector(0, 0, 0);
int numUnoccluded = 0;
For each ray {
  If (ray doesn't intersect anything) {
    avgUnoccluded += ray.direction;
    ++numUnoccluded;
  }
}
avgUnoccluded = normalize(avgUnoccluded);
accessibility = numUnoccluded / numRays;
}
```

采用拒绝取样法来产生这些光线：检测在 x、y 和 z 为-1 到 1 的三维立方体中的任意光线，拒绝不在围绕法线的单位半球中的光线。通过检测的光线属于所需要的分布。列表 17-2 的伪代码显示了这个方法。也可以用更复杂的 Monte Carlo 取样法则来得到样本方向的更好分布。

列表 17-2 用拒绝取样法计算随机方向的算法

```
while (true) {
  x = RandomFloat(-1, 1); // random float between -1 and 1
  y = RandomFloat(-1, 1);
  z = RandomFloat(-1, 1);
  if (x * x + y * y + z * z > 1) continue; // ignore ones outside unit
                                           // sphere
  if (dot(Vector(x, y, z), N) < 0) continue; // ignore "down" dirs
  return normalize(Vector(x, y, z)); // success!
}
```

17.3 硬件加速计算遮挡

用图形硬件代替光线跟踪软件有可能加速遮挡信息的计算。阴影贴图提供快速图像空间法，确定一个点是否在阴影中（GeForce FX 具有特别的硬件支持高速阴影映射所需要的只有深度图像的光栅化）。我们把问题倒过来，不是从表面的每个点发射光线，而是用投射阴影的光线球包围物体。表面上点的遮挡量是各个光线对阴影贡献的平均值。可以使用浮点累积缓冲区来计算这个平均值。对于 n 条光线，我们渲染场景 n 次，每次把投射阴影的光线移动到球体的不同位置。累积这些黑白图像形成最后的遮挡图像。在极限情况下，这模拟了一个包含天空的大面积光源。

影响最终结果质量的另一个问题是阴影映射的走样。这时常表现为在表面上从明亮区变化到阴影区时出现条纹。遮挡项乘以法线和光线方向的函数可以减轻这个问题，这样可以保证背对光源的表面总是黑色的。

这个方法的缺点是每条光线需要两个通道：一个产生阴影贴图，一个渲染附有阴影的场景，还要加上累积图像的开销。这意味着 n 条光线需要 $2n$ 个通道。把多个阴影贴图的效果累积在一个通道中，有可能加快速度。因为目前的硬件支持 8 组纹理坐标，可能在 shader 中一次累积 8 个阴影。这意味着总共需要渲染场景 $n + n/8$ 次。

也能扩展这个方法产生未遮挡方向的平均值，或变化的法线。可以用 shader 计算阴影值乘以光的方向，然后复制结果到 RGB 输出颜色。可以把遮挡信息存储在 alpha 通道中。以和遮挡值相同的方式累积这些 RGB 法线值，然后规范化结果，得到未遮挡法线的平均值。要正确地表示这些矢量之和，half 浮点（16 位）累积缓冲区的精度可能是不够的。

已经介绍了在摄像机空间产生遮挡图像的技术，但是还时常产生纹理来存储遮挡信息。如果我们正在使用的物体有惟一的纹理坐标（即，纹理不重叠地覆盖物体的整个表面），这还是相对容易的。为了展开模型，可以使用一个顶点程序，它以纹理坐标 UV 作为荧屏空间的位置。正常地进行计算是可以的，但是我们不按正常方式渲染物体，而是渲染展开网格的矩形。这个遮挡纹理一旦生成，它就能用于实时渲染，如本章其余部分所述。

17.4　用环境遮挡贴图来渲染

我们把这个方法应用到平面上的复杂模型组作为例子（在这个场景中没有纹理贴图，所以能更清楚地看到方法的效果。不过这个技术与纹理结合是很容易的）。我们对每个三角形用了 512 道光线计算可到达值，对 150 000 个三角形中的每一个平均未被遮挡方向；在 AMD Athlon 1800 XP CPU 的光线跟踪软件上预处理过程大约运行 4 分钟。然后把这个信息写到磁盘文件中用于样本。图 17-2 显示了点光源照明的简单漫反射着色模型图像，它没有影子。这是经典的不真实的计算机图形，也缺乏着色的复杂性。例如，袋鼠的下部和它的两后腿的内部都太亮。

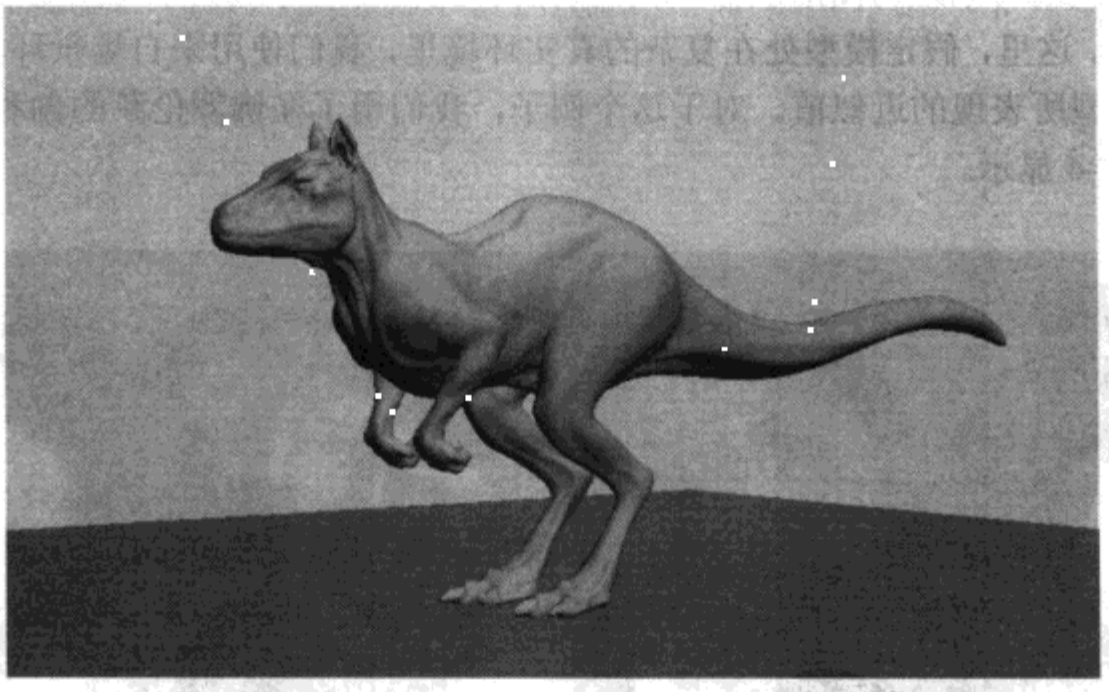


图 17-2　用简单的漫反射着色模型描画的场景
3D 扫描和画像由 headus 制作；设计和泥雕由 Martin Rezard 完成

在图 17-3 中，我们用光线跟踪预先计算在每点的可到达度，将其设定为颜色，来描画模型。可以看到模型两腿下面的凹处和胸前比暴露的区域（比如它的背部）更暗。图 17-2 中从亮到暗的刺眼过渡，是由于表面法线的方向相对于点光源位置发生变化的结果，在图 17-3 中这种过渡已经变平滑了，给出逼真的“阴暗的天窗”效果。在袋鼠下面的地面上出现非常模糊的阴影，因为到达身体下面的光线数量减少了。

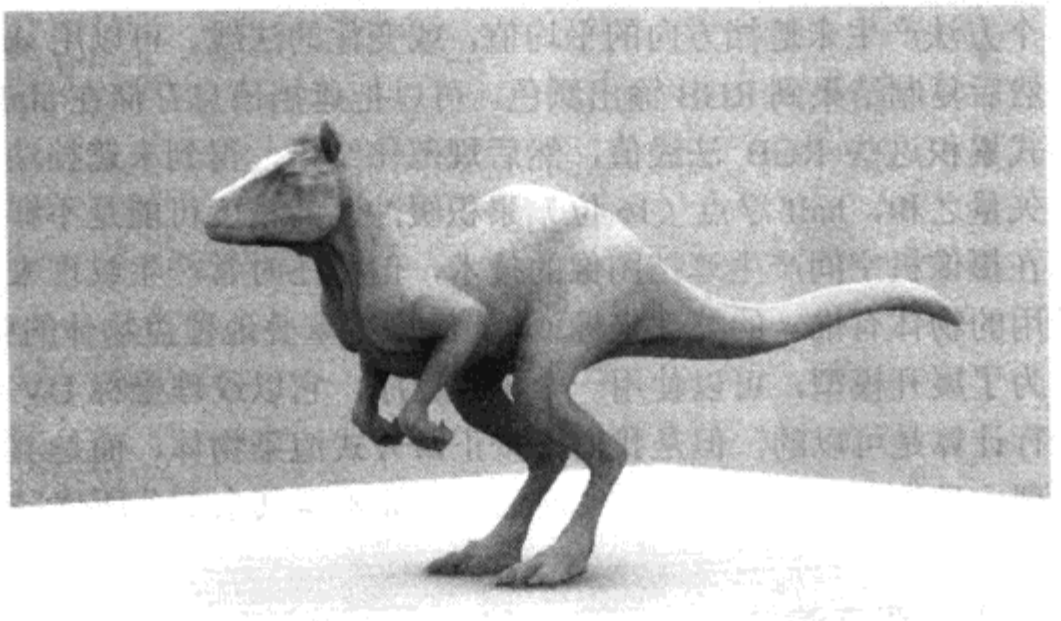


图 17-3 带有可到达信息的场景
使用可到达值制作模型给出更逼真的形象，而且有自阴影效果
3D 扫描和画像由 headus 制作；设计和泥雕由 Martin Rezard 完成

这个阴影有助于模型立在地上，显示它事实上是在地面上，而不是像图 17-2 那种情况漂浮在地上部。

用可到达值着色最大的优点是不需要高级的可编程图形硬件；它也可以工作于旧的图形硬件，只要将模型的每个顶点带入适当的彩色值即可。这样，它能很好地在多种 GPU 结构上运行。如果模型有漫反射纹理贴图，只要把纹理值乘以可到达值就可以着色了。

如果能够使用可编程的图形硬件，我们就能以更精密的方式使用可到达信息，产生更复杂的着色效果。这里，假定模型处在复杂的真实环境里，我们使用来自场景环境贴图的照度，来计算这个模型所表现的近似值。对于这个例子，我们用了在佛罗伦萨的伽利略的坟墓环境贴图，如图 17-4 显示。.

图 17-4 经纬度格式的伽利略坟墓的环境贴图，由 Paul Debevec 提供

较之于目前在交互式图形学中广泛应用的立方体贴图和球形贴图，这个环境贴图有着不同的参照系。它是一个经纬（lat-long）贴图，即经度、纬度，因为参数与用来描述地球上位置的经度和纬度类似。它使用球形坐标系把方向映射为贴图上的点，垂直方向 θ 角的范围从

0 到π，水平角度ϕ的范围从 0 到 2π，$x = \sin\theta\cos\phi$，$y = \sin\theta\sin\phi$ 和 $z = \cos\theta$。给定方向（x, y, z），逆运算这些方程，给出（ϕ, θ）数值，用（2π, π）除以这个值，把它映射为贴图中的纹理坐标。使用这种形态的环境贴图，是因为它能够无走样地做极其模糊的查询；不幸的是，目前的硬件不能正确地处理横跨多面的模糊立方体贴图的查询。

用这种环境贴图来着色，它的程序的基本思路是：我们对有多少光到达表面上的点做了很好的近似计算，现在要使用这个信息。影响这个数值的两个因素是，（1）在那个点上半球的哪部分不被点和环境贴图之间的几何体遮挡；（2）沿着这些方向的入射光是什么。图 17-5 显示两个不同的情况。在左图中，只能看到着色点上面的一小部分方向，由方向矢量 ***B*** 和围绕它的方向圆锥体所表示，该点的可到达度非常低。在右图中，沿着更大范围的方向有更多的光线到达给定点。

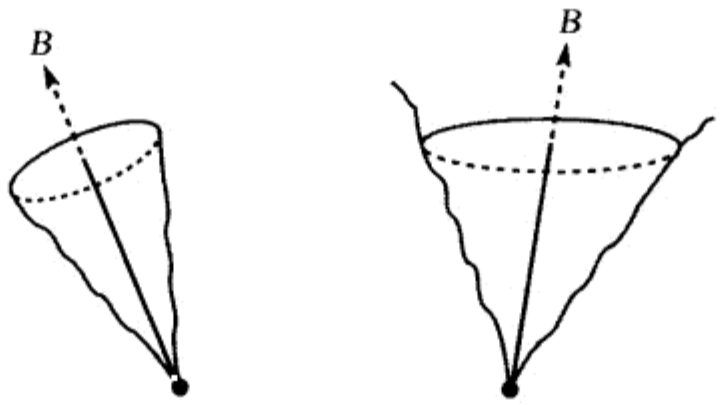


图 17-5　不同量的可见度的近似

左边：由于附近几何体的遮挡比较严重，这点得到的照度较小
右边：沿着更宽的方向圆锥体，更大量的光到达这点

预计算的可到达值告诉我们，能见到环境贴图的是半球的哪部分，而可见方向的平均值给出一个近似方向，围绕它计算入射光。虽然这个方向可能指向实际上被遮挡的方向（例如，如果半球上的两个分开区域不被遮挡，而其余部分被遮挡，平均方向可能在这两者之间），但是在实际中它通常工作得很好。

我们使用的基本片元 shader 显示在列表 17-3 中。入射光的平均方向由 ***B*** 变量引入，而未被遮挡的半球部分由 accessibility 引入；envlatlong sampler 包容了图 17-4 的环境贴图。

列表 17-3　用可达到度和环境贴图着色的碎片 shader

```
half4 main(half3 B : TEXCOORD0,
           half accessibility : TEXCOORD1,
           uniform sampler2D envlatlong) : COLOR
{
  half2 uv = latlong(B);
  half2 blurx, blury;
  computeBlur(uv, accessibility, blurx, blury);
  half3 Cenv = tex2D(envlatlong, uv, blurx, blury).xyz;
  return half4(accessibility * Cenv, 1);
}
```

这个 shader 有 3 个主要步骤。首先，我们调用在列表 17-4 中的 latlong()的函数，计算在经纬环境贴图中的纹理坐标（u, v）。其次，调用显示在列表 17-5 中的 computeBlur()函数，确

定在着色的点上要模糊的贴图的面积。最后，通过对 tex2D()函数的变量求导，做模糊的环境贴图查询。导数指定做纹理过滤的贴图面积，给出在圆锥体里平均光的颜色。通过可到达度缩放数值并返回。由于 mipmapping，可以在图形硬件中有效地计算环境贴图的大片模糊区域。

列表 17-4 latlong()函数定义

```
#define PI 3.1415926

half2 latlong(half3 v) {
  v = normalize(v);
  half theta = acos(v.z); // +z 上升
  half phi = atan2(v.y, v.x) + PI;
  return half2(phi, theta) * half2(.1591549, .6366198);
}
```

列表 17-5 computeBlur()函数定义

```
void computeBlur(half2 uv,
                 half accessibility,
                 out half2 blurx,
                 out half2 blury)
{
  half width = sqrt(accessibility);
  blurx = half2(width, 0);
  blury = half2(0, width);
}
```

为了清楚起见，我们分别列出片元 shader 的两个子程序。第一个是 latlong()，它取出 3D 方向并求出 2D 纹理位置，即为经纬贴图应该映射的方向。因为硬件不直接支持经纬映射，需要我们自己进行计算。

因为逆三角函数的运算耗时太多，在实现中使用有符号的 8 位成分，把 theta 和 phi 值存储在 256 × 256 的立方体贴图的纹理中；并且用单个的立方体贴图查询，代替整个的 latlong()函数。这个修改在汇编中节省大约 50 个片元程序指令，而且最终图像质量不受影响。

这个 shader 惟一的其他技巧，是计算有多少环境贴图进行过滤（回想图 17-5，较之右边的点，左边的点从更小的方向圆锥体接收光）。在 computeBlur()函数中计算过滤面积的非常粗糙的近似值；它仅取由于增加可达到度而增加滤波面积的一阶效果。

对模型应用这个 shader 的结果显示在图 17-6 中。这个 shader 编译大约 10 条 GPU 指令，而场景在现代 GPU 上以实时速率渲染。

17.5 小结

这里介绍的环境照明技术使用许多近似值，已经在电影的渲染中得到很好的效果。该方法把问题分解为相对昂贵的预处理，它只在渲染时计算快速着色所需要的信息。预处理与光照明环境贴图无关，所以场景的动态照度能很容易地使用。因为使用非常模糊的场景环境贴图，不一定需要再渲染每帧。这样着色的物体视觉形象比使用标准光照明模型更逼真。

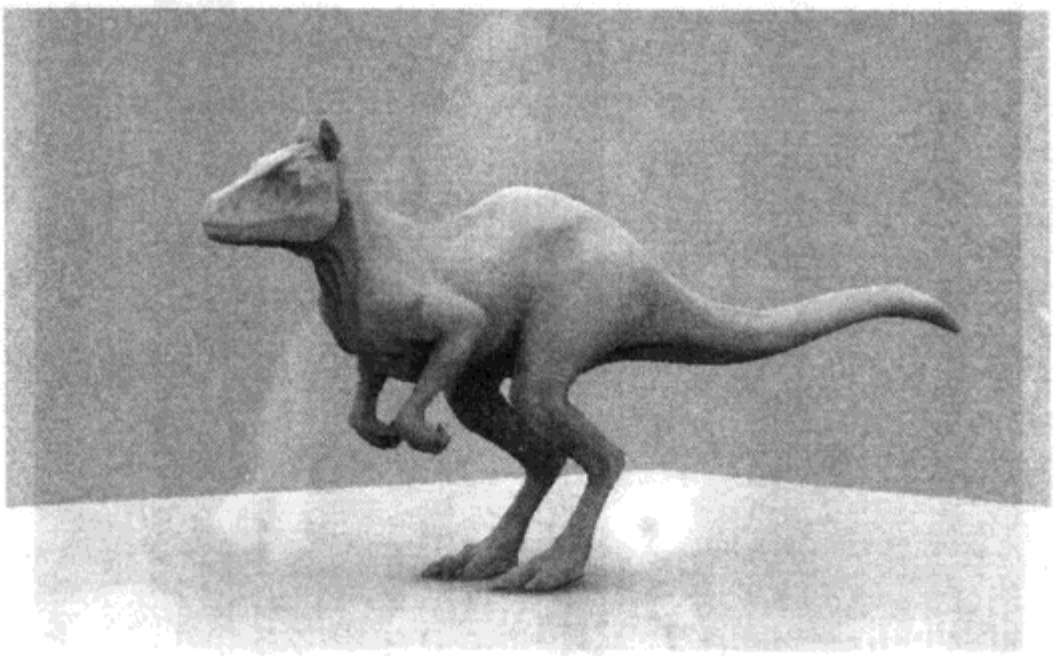


图 17-6　用可到达度信息和环境贴图的照度着色的场景

使用变化的法线和环境贴图的模糊照度着色的模型，有效地近似被环境贴图包围的模型。3D 扫描和画像由 headus 制作；设计和泥雕由 Martin Rezard 完成。

在这个方法中，可以容易地把纹理映射的表面和标准的光源结合。用少数的标准点光源或方向光源投射清晰的影子，提供“关键”照明，并产生镜面反射的高光，得到了更好的结果。

许多环境照明的应用集中于利用“高动态范围”的环境贴图（即，有浮点纹素值的贴图，这样可以在宽的亮度范围里编码），这个方法对每通道 8 位的标准纹理贴图也工作得很好。因为它只从漫反射表面模拟反射，平均许多方向上的照度，这时精确地表现少量方向的明亮光并不重要，而表现更光滑的反射却是必须的。

这章没有提到动画。当动画的人物模型改变姿势时，在整个模型上的遮挡显然会发生变化。我们相信，计算一系列相关姿势的遮挡信息，以及在人物姿势之间进行插值，在实际中会工作得很好，但是我们还没有实施。而 NVIDIA 的“食人恶魔（Ogre）”演示已成功地使用了这个方法，见图 17-7。

17.6　参考文献

这节以更一般的方式介绍其他可能的环境照明方法。尤其可以在球面调和照明技术中理解环境遮挡。只使用一阶球面调和就可以极大地简化反射的表现。

Landis, Hayden. 2002. “Production-Ready Global Illumination.” Course 16 notes, SIGGRAPH 2002. Available online at **http://www.renderman.org/RMR/Books/ sig02.course 16.pdf.gz.**

Hayden Landis 和 ILM 的其他人开发了在这章中以 Cg 描述和实施的思路。Hayden 证明了这些方法，而且在最后列出这个着色技术的完全列表。

Zhukov, S., A. Iones, and G. Kronin. 1998. “An Ambient Light Illumination Model.” In *Proceedings of Eurographics Rendering Workshop '98*, pp. 45～56. Zhukov 等人介绍了一个“智能环境”方法，基于该文所述的光线追踪的预处理，但是没有使用它作为环境照明。

Blinn, J. F, and Newell, M. E. 1976. “Texture and Reflection in Computer Generated Images.” *Communications of'the ACM*19(10)(October 1976), pp. 542～547.

图 17-7 食人恶魔（Ogre）人物
环境遮挡（上图）和完整的着色（下图）
食人恶魔人物由 Spellcraft 工作场所提供

使用环境贴图来照明镜面高光物体的思路首次在该文中介绍。

Miller, Gene S., and C. Robert Hoffman. 1984. "Illumination and Reflection Maps:

Simulated Objects in Simulated and Real Environments." Course notes for Advanced Computer Graphics Animation, SIGGRAPH 84.

照明非镜面高光物体的应用首次在该文中介绍。

Debevec, Paul. 1998. "Rendering Synthetic Objects into Real Scenes: Bridging Traditional and Image-Based Graphics with Global Illumination and High Dynamic Range Photography." In *Proceedings of SIGGRAPH 98*, pp. 189～198.

这篇论文发表之后，对环境照明的兴趣发生实质性增长。

Cohen, Michael, and Donald P. Greenberg. 1985. "The Hemi-Cube: A Radiosity Solution for

Complex Environments."

首次介绍为辐射度进行半立方体算法的文章，另一种遮挡预处理方法，使得那个步骤能够容易地利用图形硬件。

Ramamoorthi、Hanrahan 和 Sloan 等人最近发表的一系列 SIGGRAPH 论文，为快速环境照明建立了基于球面调和函数的主要数学原理和算法。

第18章 空间的BRDFs

David McAllister
NVIDIA

空间的双向反射分布函数（SBRDF）表示了表面上每点的反射，包括每个入射角和每个出射角。这个函数对表面外观的表达既通用又详细。这章讨论压缩的SBRDF表达式，以及由离散光或环境贴图所照明的SBRDF的渲染方法。

18.1 什么是SBRDF

SBRDF是纹理贴图和双向反射分布函数（BRDF）的组合。纹理贴图存储了反射或其他的属性，它们沿着2D表面上的空间变化。BRDF存储的是表面上单个点的反射，包括从入射角到出射角的全部分布，见图18-1。

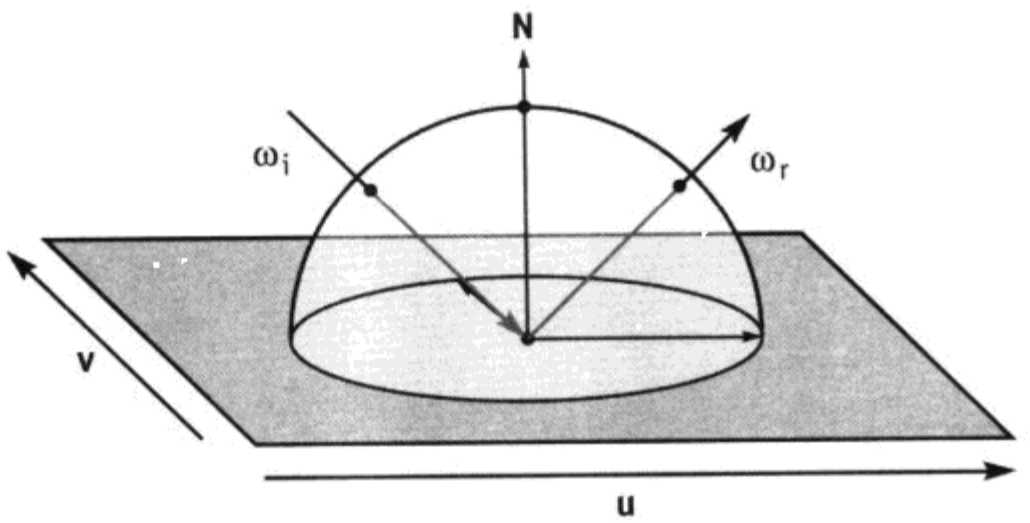


图18-1 SBRDF的定义域

真实表面通常有重要的变化，既有空间的变化，又有光线角度和观察方向的变化。因此，为了更逼真地表现大部分表面，我们应该把BRDF和纹理映射结合起来。这个结合就是SBRDF。更详细的内容请见McAllister 2002。

18.2 表达式的详述

最直接的SBRDF表达式，是在纹理贴图的每个纹素上存储那点的BRDF的所有参数。可以使用任何BRDF表达式，但最紧凑与适合硬件渲染

的是 Lafortune 表达式（Lafortune 1997）。这个表达式由几项加和组成：

$$f_r(\omega_i \to \omega_r) = \rho_d + \sum_j \rho_{s,j} \cdot s_j(\omega_i, \omega_r)$$

这里，ρ_d 是漫反射。加和中的项是高光镜面反射的圆凸起形状。对每个圆凸起形状 j 有一个反照率 $\rho_{s,j}$（反射换算系数）和凸起形状 s_j，入射和出射方向是 ω_i 和 ω_r。与标准的 Phong 凸起形状类似，这里凸起形状定义如下：

$$s(\omega_i, \omega_r) = \left(\begin{bmatrix} \omega_{r,x} \\ \omega_{r,y} \\ \omega_{r,z} \end{bmatrix}^T \cdot \begin{bmatrix} C_x & & \\ & C_y & \\ & & C_z \end{bmatrix} \cdot \begin{bmatrix} \omega_{i,x} \\ \omega_{i,y} \\ \omega_{i,z} \end{bmatrix} \right)^n$$

这个方程可以被看成一般的点积，其中的 3 项可以任意地缩放：

$$s(\omega_i, \omega_r) = \left(C_x \omega_{i,x} \omega_{r,x} + C_y \omega_{i,y} \omega_{r,y} + C_z \omega_{i,z} \omega_{r,z} \right)^n$$

见图 18-2。

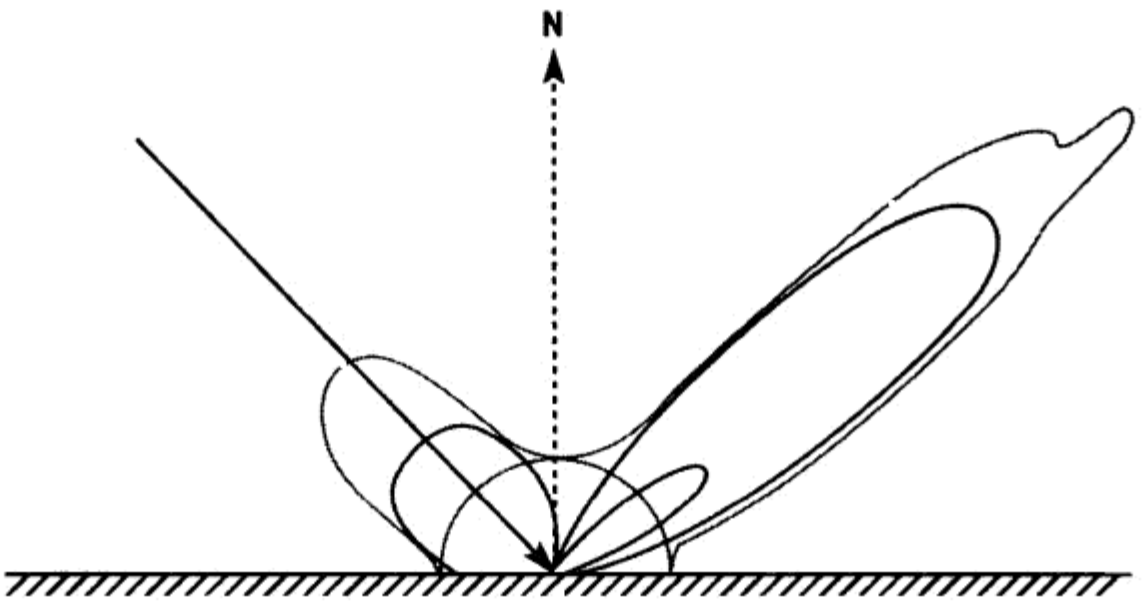


图 18-2　3 个圆凸起形状和一个漫反射组分组成一个 BRDF

Lafortune 表达式在局部表面坐标中计算。x 轴和 y 轴方向是各向异性的（scratch or thread 方向），z 轴是法线。矩阵 C 定义为 $C_x = -1$，$C_y = -1$，$C_z = 1$，这使得 ω_i 围绕法线反射，产生一个标准的 Phong 圆凸起形状。但是为了表现真正的表面散射行为，有时需要剪切镜面圆凸起形状，这时 C 系数也可能取其他值。叶片的尖峰是在 $C \cdot \omega_i$ 方向。对于各向同性的 BRDF，$C_x = C_y$。没有镜面反射时，$|C_z| < |C_x|$，把叶片拉向切线平面。对于回射（retroreflection）情况，$C_x > 0$ 和 $C_y > 0$。当 C_x 和 C_y 符号相反的时候，一个叶片向前散射平行光，但是向后散射垂直光，这模拟了 threads and scratches 的行为。

Lafortune 表达式的灵活性对瞄准和缩放每个散射叶片是仅使用几个叶片就成功地近似 BRDF 的关键。这个特性也可以实现 18.4 节所描述的光滑的环境映射技术。Lafortune 表达式很好地适应典型的 BRDF 形状。它是压缩的，并且能够表现有趣的 BRDF 特性，例如逐步增加的 Fresnel 反射、避开镜面反射的尖峰和回射等。

反照率 ρ 的值是 3 个一组的 RGB，但是 C_x、C_y、C_z 和 n 的值在通道之间共享，每个镜面的叶片总共有 7 个系数。1、2 或 3 个镜面项通常足够了，取决于表面性质和质量与速度的

折衷情况。由于仅用 10 个系数（3 个漫反射加 7 个镜面的），SBRDF 表达式只比 RGB 纹理贴图大 3 倍，但是它达到非常逼真的效果。

可以从真正的表面测量 SBRDF 数据（McAllister 2002），或者由美工使用专门的程序画出。最简单的绘画方式是使用现成的具有 BRDF 的调色板，并把它们画进一个 SBRDF 纹理贴图。调色板的输入可以混合现有的 BRDF，也可以从 SBRDF 数据库取样（McAllister 2003），或使用设置 ρ、C_x、C_y、C_z 和 n 值的对话框来定义。

18.3 使用离散光的渲染

对于以上两种渲染方式，SBRDF 以相同的方法载入纹理存储器。漫反射颜色 ρ_d 存储在纹理贴图中（它的 alpha 通道可用于 alpha 终止（kill）或其他用途）。每个叶片 ρ 被存储在一个纹理贴图中（用它的 alpha 通道表示是否用于各向异性的方向），而 C_x、C_y、C_z 和 n 被存储在第 2 个贴图中。使用 8 位的纹理贴图时，必须应用一个缩放比例和偏移，因为 C_x 和 C_y 可用的范围是-1.1 到 1.1。缩放比例 96，偏移 128，可以工作得很好。根据可用的存储器带宽和片元 shader 的性能，把 SBRDF 数据存储在浮点纹理，而不是 8 位的纹理可能更有效，这样可以避免缩放和改善准确度。

由标准的点光源或方向光源照明的 SBRDF 表面的 Cg shader 很简单，如列表 18-1 所示。它类似于标准 Phong 模型和硬件光源的 shader。一点不同是所有的 BRDF 参数全都是从纹理贴图中提取的，而不只是漫反射颜色。另一个差别是在局部表面坐标系中计算点积，所以它的成分能被 C_x、C_y 和 C_z 缩放。这要求光矢量从局部表面坐标系的顶点程序而来。样例结果见图 18-3。

列表 18-1 离散光源的一个 SBRDF 片元 shader

```
#define NUM_LOBES 3
#define NUM_LIGHTS 2

// Store Cx, Cy, Cz on range -1.3333 .. 1.3333.
#define SCL (1.0 / (96.0 / 256.0))
#define BIAS (SCL / 2.0)
#define EXSCL 255.0 // Scale exponent to be on 0 .. 255.

// Rasterize the view vector and all the light vectors.
// Pass the light colors as uniform parameters.
struct fromrast
{
  float2 TexUV : TEXCOORD0; // Surface texcoords
  float3 EyeVec : TEXCOORD1; // Vector to eye (local space)
  float3 LightVec[NUM_LIGHTS] : TEXCOORD2; // Lights (local space)
};

float4 main(fromrast I,
            uniform sampler2D tex_dif,                // Diffuse
            uniform sampler2D tex_lshp[NUM_LOBES], // Lobe shape
            uniform sampler2D tex_lalb[NUM_LOBES], // Lobe albedo
```

```
            uniform float4 Expos,
            uniform float3 LightCol[NUM_LIGHTS]
            ) : COLOR
{
  // Load the BRDF parameters from the textures
  float4 lobe_shape[NUM_LOBES];
  float4 lobe_albedo[NUM_LOBES];

  for(float p = 0; p < NUM_LOBES; p++)
{
    lobe_shape[p] = f4tex2D(tex_lshp[p], I.TexUV.xy) * float4(SCL, SCL, SCL, EXSCL) -
                    float4(BIAS, BIAS, BIAS, 0.0);
    lobe_albedo[p] = f4tex2D(tex_lalb[p], I.TexUV.xy);
}

float4 dif_albedo = f4tex2D(tex_dif, I.TexUV.xy);

// Vector to eye in local space.
float3 toeye = normalize(I.EyeVec.xyz);

// Accumulate exitant radiance off surface from each light
float3 exrad = float3(0, 0, 0);

for(int l = 0; l < NUM_LIGHTS; l++)
{
  // Vector to light in local space.
  float3 tolight = normalize(I.LightVec[l].xyz);

  // Evaluate the SBRDF for this point and direction pair
  float3 refl = dif_albedo.xyz;

  for(float p = 0; p < NUM_LOBES; p++)
  {
    // Shear eye vector
    float3 Cwr_local = lobe_shape[p].xyz * toeye;
    float thedot = dot(Cwr_local, tolight);
    refl = refl + lit(thedot, thedot, lobe_shape[p].w).z * lobe_albedo[p].xyz;
  }

  // Irradiance for this light (incident radiance times NdotL).
  float NdotL = max(0, tolight.z);
  float3 irrad = LightCol[l] * NdotL;
    // Reflectance times irradiance is exitant radiance.
    exrad += refl * irrad;
  }

  float4 final_col = exrad.xyzz * Expos.xxxx; // Set HDR exposure.
  final_col.w = dif_albedo.w; // Put alpha from the map into pixel.

  return final_col;
}
```

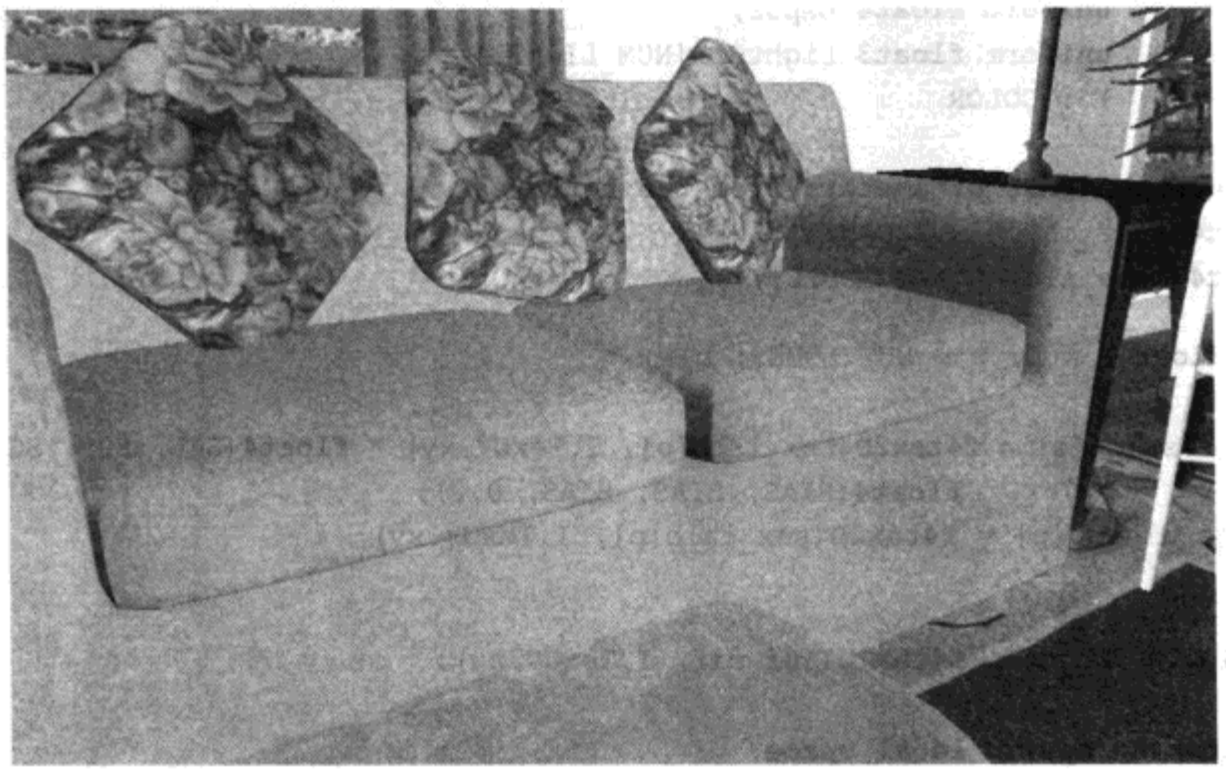


图 18-3 用离散的光渲染

白色的室内装潢织物、窗帘、灯和植物叶子使用 SBRDF。注意织物前景如何从暗变到明，而背景从明变到暗。这是在前景和背景之间不同的各向异性 BRDF 所致

C_x、C_y、C_z和 n 是非线性函数的参数，因此，应用双线性或双线性内插到这些纹理贴图会引起明显的走样。最简单的解决办法是对这个贴图使用点取样。

18.4 使用环境贴图的渲染

SBRDF 除了可用点光源或方向光源照明外，还可以用环境贴图所有方向的入射光照明。关键是在渲染前用 BRDF 的一部分卷积环境贴图。对于大多数的 BRDF 表达式，必须分别处理各个不同的 BRDF。但是因为一个 SBRDF 可能有上百万个不同的 BRDF，这么做是不可能的。

我们采取另外的做法，简单地用一个 Phong 叶片卷积环境贴图，叶片可以选择不同镜面指数，例如，$n=0$、1、4、16、64 和 256。这些贴图能存储在不同级别的立方体 mipmap 中，$n=0$ 是最粗级别的 mipmap，$n=256$ 为最细级别的 mipmap。随后，SBRDF 纹素的 n 值就指示细节级别（LOD），用于在立方体贴图中采样适当 mipmap 级别。

卷积的环境贴图组的另一个表达是单个的 3D 纹理，维数为 s 和 t 的贴图映射为抛物线贴图（Heidrich 和 Seidel 1999），而维数 r 映射为 n。

18.4.1 算法

推导从表达着色像素 $L(\omega_r)$的光开始，表达为 Lafonune BRDF 的漫反射项和镜面高光项：

$$L(\omega_r)=\rho_d\int_{\Omega_i}L_i(\omega_i)(\mathbf{N}\cdot\omega_i)d\omega_i+\sum_j\rho_{s,j}\int_{\Omega_i}s_j(\omega_i,\omega_r)L_i(\omega_i)(\mathbf{N}\cdot\omega_i)d\omega_i$$

Ω_i 表示在入射半球上的积分。入射光 $L_i(\omega_r)$存储在环境贴图中，按入射方向编入索引。漫反射项可以容易地在用 **N** 索引的环境贴图中编码：

$$D(\mathbf{N}) = \int_{\Omega_i} (\mathbf{N} \cdot \omega_i) L_i(\omega_i) d\omega_i$$

$D(\mathbf{N})$不受 BRDF 的约束，因此对反映环境贴图 $L_i(\omega_r)$的所有物体预先计算 $D(\mathbf{N})$一次。镜面高光项也利用预先计算的贴图。正像 **N** 索引预先卷积的漫反散贴图一样，观察方向的函数也索引一个预先卷积的高光镜面环境贴图：

$$p_j(\omega_r) = \begin{bmatrix} C_x & & \\ & C_y & \\ & & C_z \end{bmatrix} \cdot \begin{bmatrix} \omega_{r,x} \\ \omega_{r,y} \\ \omega_{r,z} \end{bmatrix}$$

$p_j(\omega_r)$是叶片形状的取样核心的峰矢量，即在叶片 j 中，使得出射光朝向 ω_r 方向的影响最大的入射方向。表达方程是：

$$L(\omega_r) = \rho_d D(\mathbf{N}) + \sum_j \rho_{s,j} \int_{\Omega_i} \left(p(\omega_r) \cdot \omega_i\right)^{n_j} L_i(\omega_i)(\mathbf{N} \cdot \omega_i) d\omega_i$$

由镜面凸起卷积的环境贴图是：

$$S(\omega_p, n) = \int_{\Omega_i} \left(\frac{\omega_p}{\|\omega_p\|} \cdot \omega_i \right)^n L_i(\omega_i) \mathrm{d}\omega_i$$

这个方程从最初的环境贴图创建卷积的环境贴图。这个贴图是参数化的，指数为 n 时，入射核心的峰方向为 ω_p。列表 18-2 的伪代码表现一个平滑滤波（brute-force）方法，计算一个指数为 n 的 Phong 叶片所缠绕的环境贴图。

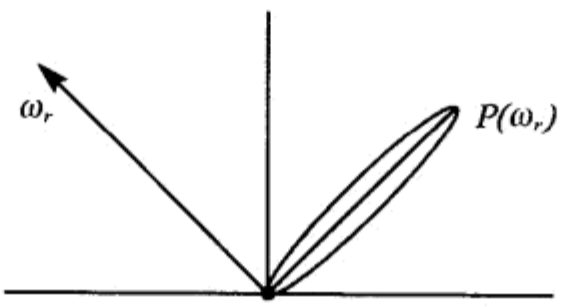


图 18-4　造成出射方向为 ω_r 的入射叶片尖峰

列表 18-2　用 Phong 叶片卷积环境贴图的伪代码

```
float3 S(float3 Wp, float n)
{
  float3 Sum = 0.xyz;
  float WgtSum = 0;

  for(int si = 0; si < 6; si++) {
    for(int yi = 0; yi < YMAX; yi++) {
      for(int xi = 0; xi < XMAX; xi++) {
        float3 Wi = VecFromCubeMap(xi, yi, si);
        float dp = dot(Wp, Wi);
        if(dp < 0) continue;

        float lobe_shape = pow(dp, n);
        float3 FN = FaceNormal(si);
```

```
        // Scale irradiance by length of cube vector to
        // compensate for irregular cube map distribution.
        float3 Irrad = Li(Wi) * dp / length2(Wi);

        Sum += Irrad * lobe_shape;
        WgtSum += lobe_shape; // Compute volume of lobe
      }
    }
  }
  return Sum / WgtSum;
}

// Convolve the cube map Li with a Phong lobe of exponent n
CubeMap PrecomputeEnvMap(float n)
{
  CubeMap Smap;

  // Loop over all the pixels of the cube map
  for(int s = 0; s < 6; s++) {
    for(int y = 0; y < YMAX; y++) {
      for(int x = 0; x < XMAX; x++) {
        float3 Wp = VecFromCubeMap(x, y, s);
        Smap(x, y, s) = S(Wp, n);
      }
    }
  }
  return Smap;
}
```

用于硬件渲染的最后表达式是：

$$L(\omega_r) \approx \rho_d D(\mathbf{N}) + \sum_j \rho_{s,j} S\left(\boldsymbol{p}(\omega_r), n_j\right) \left\|\boldsymbol{p}(\omega_r)\right\|^{n_j} \left(\mathbf{N} \cdot \boldsymbol{p}(\omega_r)\right)$$

$\left\|\boldsymbol{p}(\omega_r)\right\|^{n_j}$ 因子的产生是因为用规范化的 ω_p 计算 S，因此，入射光必须由凸起的量来缩放。这个方程只是近似值，因为分散的出射光 $\mathbf{N} \cdot \omega_i$ 也必须在 ω_i 的积分里计算。但是它不能存储在环境贴图中，因为它以后会被 $\boldsymbol{p}(\omega_r)$和 $\mathbf{N}$ 参数化。表达式不是将积分的所有方向相等地加权，而是以 $\mathbf{N} \cdot \boldsymbol{p}(\omega_r)$ 加权 $S(\omega_p, n)$。这个近似值在实际应用中没有问题。

以一个环境贴图查询和每个叶片的几个乘法，可以渲染任何数量的独立 BRDF，这些 BRDF 由同组的环境贴图来照明。

18.4.2 shader 代码

列表 18-3 的范例代码是用预先卷积的环境贴图进行渲染。它调用函数 f3texCUBE_RGBE_Conv(*env_rex*, *dir*, *n*)，用一个基于 *n* 计算的 LOD，在给定方向上取预先卷积环境贴图样本。结果见图 18-5。

为了简捷地表示大于 1.0 的入射光，可以使用 RGBE 表达式，对 float3 的解码也在这个函数里执行。

图 18-5　用环境贴图渲染

用蓝色的油漆和铝的 BRDF 制造的 SBRDF，使用合成的蓝笔和用预先卷积的环境贴图照明。这个 BRDF 使用 3 个叶片

列表 18-3　用于环境贴图的 SBRDF 片元 shader

```
#define NUM_LOBES 2

// Store Cx, Cy, Cz on range -1.3333 .. 1.3333.
#define SCL (1.0 / (96.0 / 256.0))
#define BIAS (SCL / 2.0)
#define EXSCL 255.0 // Scale exponent to be on 0 .. 255.

float3 ApplyLobe(float3 toeye,
                 float4 lobe_shape,
                 float3 lobe_albedo,
                 float3 Tan,
                 float3 BiN,
                 float3 Nrm,
                 uniform samplerCUBE tex_s)
{
  // Reflect the eye vector in local surface space to get p(wr)
  float3 Cwr_local = lobe_shape.xyz * toeye;

  // Transform lobe peak to world space before env. lookup
  float3 Cwr_world = ToWorld(Cwr_local, Tan, BiN, Nrm);

  // Sample the cube map at the lobe peak direction.
  float sharpness = lobe_shape.w; // This is n.
  float3 incrad = f3texCUBE_RGBE_Conv(tex_s, Cwr_world, sharpness);

  // (length^2)^(n*0.5) = length^n
  float lobelen = pow(dot(Cwr_world, Cwr_world), sharpness * 0.5);
```

```
  // Approximate the irradiance falloff at all points
  // by that of the peak dir.
  // This is N dot Cwr in local space.
  float3 irrad = incrad * (Cwr_local.z * lobelen);
  float3 radiance = irrad * lobe_albedo;

  return radiance;
}

struct fromrast
{
  float2 TexUV : TEXCOORD0;    // Surface texcoords
  float3 EyeVec : TEXCOORD1;   // Vector to eye (local space)
                               // - needs normalization
  float3 NrmVec : TEXCOORD2;   // Normal (world space)
                               // - needs normalization
  float3 TanVec : TEXCOORD3;   // Tangent (world space)
                               // - needs normalization
};

float4 main(fromrast I,
            uniform sampler2D tex_dif,      // Diffuse
            uniform sampler2D tex_lshp0,    // Lobe shape
            uniform sampler2D tex_lalb0,    // Lobe albedo
            uniform sampler2D tex_lshp1,    // Lobe shape
            uniform sampler2D tex_lalb1,    // Lobe albedo
            uniform samplerCUBE tex_envd,   // Cube diffuse env map
            uniform samplerCUBE tex_envs,   // Cube specular env map
            uniform float4 Expos) : COLOR
{
  // Preload all the BRDF parameters.
  float4 lobe_shape[NUM_LOBES];
  float4 lobe_albedo[NUM_LOBES];
  for(float p = 0; p < NUM_LOBES; p++)
{
  lobe_shape[p] = f4tex2D(tex_lshp[p], I.TexUV.xy) * float4(SCL, SCL, SCL, EXSCL) -
                  float4(BIAS, BIAS, BIAS, 0.0);
  lobe_albedo[p] = f4tex2D(tex_lalb[p], I.TexUV.xy);
}

float4 dif_albedo = f4tex2D(tex_dif, I.TexUV.xy);

float3 Nrm = normalize(I.NrmVec.xyz);
float3 Tan = normalize(I.TanVec.xyz);
float3 BiN = cross(Nrm, Tan);

// Vector to eye in local space.
float3 toeye = normalize(I.EyeVec.xyz);

// Accumulate exitant radiance off surface due to each lobe
```

```
float3 exrad = float3(0, 0, 0);
for(int l = 0; l < NUM_LOBES; l++)
  exrad += ApplyLobe(toeye, lobe_shape[l], lobe_albedo[l].xyz,
                     Tan, BiN, Nrm, tex_envs);

  // Add the diffuse term. tex_envd contains the irradiance
  // for a surface with normal direction Nrm.
  float3 irrad = f3texCUBE_RGBE(tex_envd, Nrm);
  exrad = exrad + dif_albedo.xyz * irrad;

  float4 final_col = exrad.xyzz * Expos.xxxx; // Set HDR exposure.
  final_col.w = dif_albedo.w; // Put alpha from map into the pixel.

  return final_col;
}
```

18.5　小结

考虑到 SBRDF 具有极大的一般性，离散光的 shader 性能是相当好的。它主要针对每个叶片和每个光源，计算点积和 lit()。

同样，环境贴图 shader 的性能也很好，它允许每个纹素有完全不同的 BRDF，但是仍由环境贴图照明。这个 shader 的核心是简单的环境贴图纹理查询，麻烦的是查询坐标的计算需要矩阵变换。这两个 shader 的大部分可以在半精度下执行，而视觉效果不变。

为了简单，这章省略了空间各向异性方向的变化。在 McAllister 2002 中可以发现有关细节，包括卷积的环境贴图、对应于片元 shader 的顶点 shader，以及更多样本的编码。

18.6　参考文献

Heidrich, Wolfgang, and H.-P. Seidel. 1999. "Realistic, Hardware-Accelerated Shading and Lighting." In *Proceeding of SIGGRAPH 99*.

Lafortune, E. P. F., S.-C. Foo, et al. 1997. "Non-Linear Approximation of Reflectance Functions." In *Proceedings of SIGGRAPH97*, pp. 117～126.

McAllister, David K. 2002. "A Generalized Surface Appearance Representation for Computer Graphics." Department of Computer Science, University of North Carolina, Chapel Hill. Available online at **http://www.cs. unc.edu/~davemc/Pubs.html.**

McAllister, David K., and Anselrno A. Lastra. 2003. "The SBRDF Home Page." Web site. **http://www.cs.unc.edu/~davemc/SBRDF.**

感谢 Ben Cloward 对旅馆前厅场景建模和纹理化，并且制作旅馆前厅的 HDR 辐射贴图，也感谢 Anselmo Lastra 博士，他是作者论文的导师并且为本文做了多方面的贡献。

第19章 基于图像的光照

Kevin Bjorke
NVIDIA

立方体贴图通常用来创建无限远处环境的反射。但是用少量 shader 算法，可以把物体放在特定大小和位置的反射环境中，给出高质量的基于图像的光照（IBL）。

19.1 基于图像光照的局部化

映射立方体的反射现在是实时图形的标准部分，它们是表现许多模型外观的关键。这种反射在某方面与现实主义不符：立方体贴图的反射总是使它好像在无限远处。这限制了它对小的封闭环境的应用：如果用于这种情况，每次模型彼此相互移动时，需要再产生立方体贴图，这会额外消耗资源，见图 19-1。

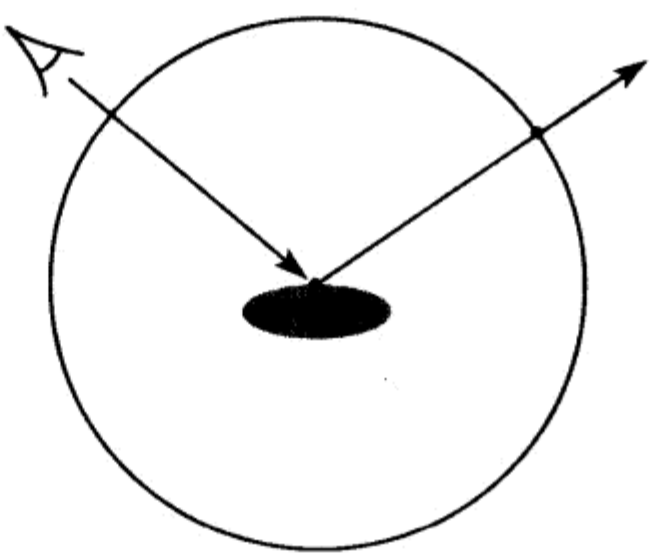


图 19-1 典型的“无限远”反射

简单的反射映射从立方体贴图中获取颜色，立方体贴图表示距离无限远的球体。反射矢量在任何地方都一样，只与反射方向有关

在室内环境移动模型时，环境最好是近距离的立方体贴图，距离的大小与当前的房间类似。当模型在那种房间移动时，根据模型在房间中的位置，适当地放大或缩小反射。这种方法使人感到模拟可靠而坚实，尤其在包含窗户、视频监视器和其他可识别光源的环境中，见图 19-2。

只要加很少的 shader 数学就能将反射局部化。事实上，最近游戏开发者已经用这种方法代替许多局部光照。

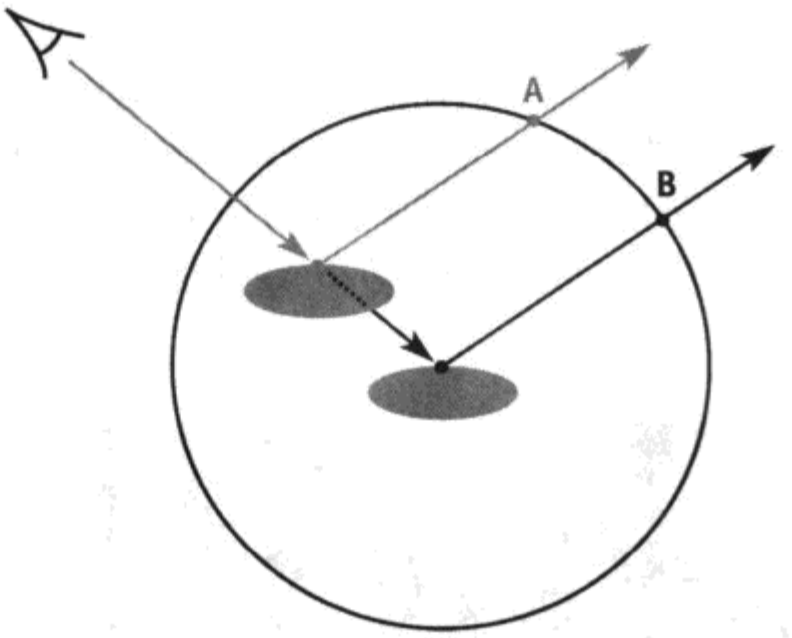


图 19-2　局部化的反射

选择反射环境的中心位置和尺寸，让我们创建一个较大的局部的幻影
无限远的距离贴图在点 B 取样，但是有限球体上的局部反射在点 A 上取样
畸变创建了局部反射的幻影

在图 19-3 中，我们在典型映射的反射环境中看到一个反射物体（大的黄金面具）。图 19-4 是同样的短动画中的另一帧。贴图没有改变，但是反射不同！先前窗户反射小，现

图 19-3　在局部反射中的反射物体

图 19-4　在不同位置上的局部反射

在变大了，并与物体排成一行。而且面具比窗户表面稍微地凸出来一些，先前正好在面具后面的窗户上映射的纹理看不到了。而所反射的画框现在赫然地显现在新的图像中。

同时，在第一帧中很明显的绿色天花板嵌镶板（这个照片的立方体贴图显示 NVIDIA 建筑物的大厅），现在已经退到远处，并且只覆盖小部分的反射。

这个反射也可以是凹凸映射，如图 19-5 所示（只增加了凹凸）。图 19-6 是图 19-5 的特写。

图 19-5 把凹凸应用于局部反射

图 19-6 图 19-5 的特写，显示反射平面图

无着色的、简单的几何体显示在图 19-7 中。

图 19-8 表示最简单的场景。大的立方体是房间模型（将在后面介绍着色）。房间体的 3D 变换传递给反射物体的 shader，利用这点可以在像素 shader 中创建反射的正确变形。

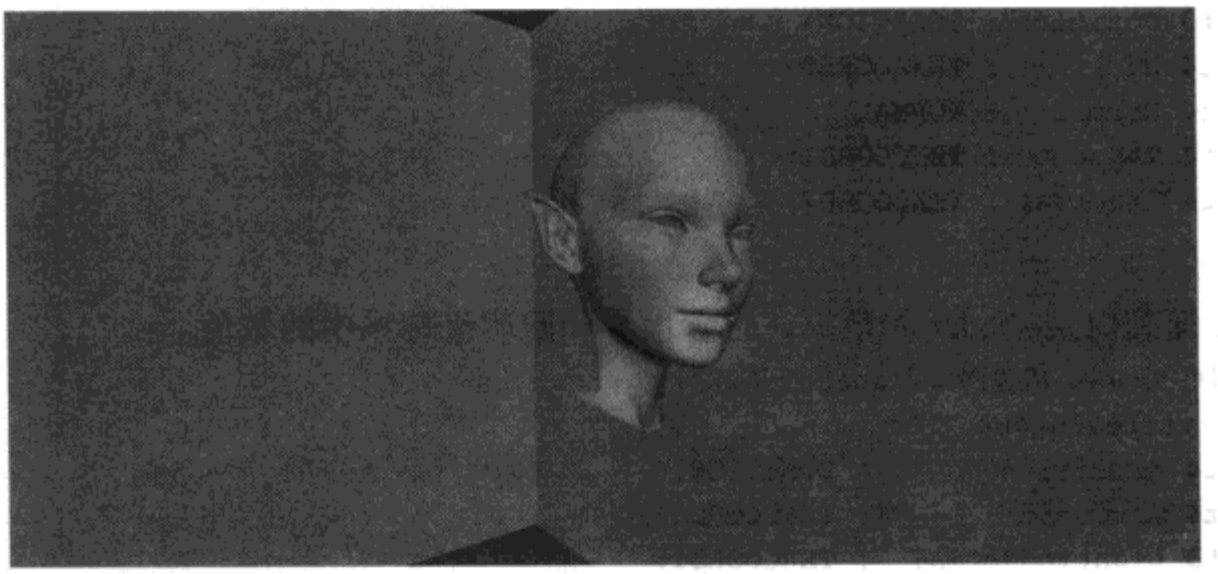


图 19-7　来自样历场景的无光泽着色的几何体

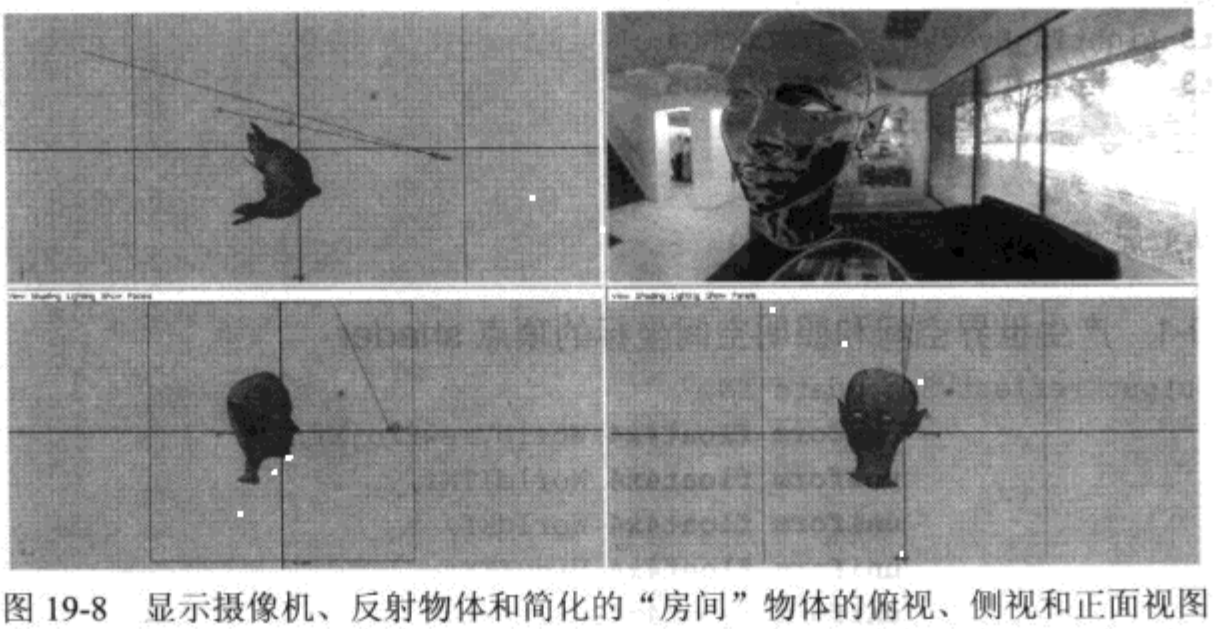


图 19-8　显示摄像机、反射物体和简化的“房间”物体的俯视、侧视和正面视图

19.2　顶点 Shader

要为光照创建局部的参照系，就需要创建新的坐标系。除了标准的坐标空间，比如视点空间和物体空间，还需要创建光照空间，即与立方体贴图本身相关的地点和方位。利用这个新坐标系计算与立方体贴图相关的物体位置。

为了简化算法，假定立方体贴图的固定“半径”是 1.0，即，立方体在每个方向上的变化范围是−1.0 到 1.0（立方体形状对于硬件纹理化是方便的；我们把它的角投射到 3D 方向矢量的球面上）。这使动画和照明/场景设计师能够容易地安排立方体贴图的位置和尺寸，通过使用 3ds max nulls、Maya place3DTexture 节点，或类似的“虚拟”物体。

在我们的例子中，把如下两个 4 × 4 浮点变换代到顶点 shader：光照空间的矩阵（相对于世界坐标）和它的逆转置。与世界空间和视点空间的变换相结合，在光照空间表达表面的坐标。

我们也代入来自 CPU 应用程序的每个顶点的法线、切线和副法线值，于是可以凹凸映射局部的反射。

```
// 来自应用程序顶点缓冲区的数据
struct appdata {
```

```
  float3 Position : POSITION;
  float4 UV       : TEXCOORD0;
  float4 Normal   : NORMAL;
  float4 Tangent  : TEXCOORD1;
  float4 Binormal : TEXCOORD2;
};
```

我们传递到像素 shader 的数据，包含世界和照明坐标系中的数值。

```
// 从顶点 shader 传递到像素 shader 的数据
struct vertexOutput {
  float4 HPosition       : POSITION;
  float4 TexCoord        : TEXCOORD0;
  float3 LightingNormal  : TEXCOORD1;
  float3 LightingTangent : TEXCOORD2;
  float3 LightingBinorm  : TEXCOORD3;
  float3 LightingEyeVec  : TEXCOORD4;
  float3 LightingPos     : TEXCOORD5;
};
```

列表 19-1 显示了顶点 shader。

列表 19-1 产生世界空间和照明空间坐标的顶点 shader

```
vertexOutput reflectVS(appdata IN,
                       uniform float4x4 WorldViewProjXf,
                       uniform float4x4 WorldITXf,
                       uniform float4x4 WorldXf,
                       uniform float4x4 ViewITXf,
                       uniform float4x4 LightingXf,
                       uniform float4x4 LightingITXf)
{
  vertexOutput OUT;
  OUT.TexCoord = IN.UV;
  float4 Po = float4(IN.Position.xyz,1.0); // 填上 "float4"
  OUT.HPosition = mul(WorldViewProjXf, Po);
  float4 Pw = mul(WorldXf, Po); // 世界坐标
  float3 WorldEyePos = ViewITXf [3].xyz;
  float4 LightingEyePos = mul(LightingXf, float4(WorldEyePos, 1.0));
  float4 Pu = mul(LightingXf, Pw);
  float4 Nw = mul(WorldITXf, IN.Normal);
  float4 Tw = mul(WorldITXf, IN.Tangent);
  float4 Bw = mul(WorldITXf, IN.Binormal);
  OUT.LightingEyeVec = (LightingEyePos - Pu).xyz;
  OUT.LightingNormal = mul(LightingITXf, Nw).xyz;
  OUT.LightingTangent = mul(LightingITXf, Tw).xyz;
  OUT.LightingBinorm = mul(LightingITXf, Bw).xyz;
  OUT.LightingPos = mul(LightingXf, Pw).xyz;
  return OUT;
}
```

在这个例子中，点和矢量值被转换两次：先进入世界空间，然后从世界空间进入光照空

间。可以用 CPU 预先连接这些矩阵，而且只用一次乘法运算就能转换位置、法线、切线和副法线矢量。所显示的方法使用 CgFX，World 和 WorldIT 的变换由 CgFX 语法分析器自动地跟踪和支持，而光照空间的变换由用户定义的值支持（即应用 DCC）。

19.3　片元 Shader

给定着色点的位置和矢量，相对于光照空间，像素部分是相当直接的。在光照空间表示反射矢量，它的起点是光照空间的表面，通过解球的二次方程，让矢量与那个球面相交，球面中心在照明空间的原点，半径=1.0。

把红色（float4(1, 0, 0, 0)）定为“安全警示”的默认色：如果一个点在球外边被遮挡（没有反射），那个点将会显示为红色，以此标明开发中的任何错误。片元 shader 显示在列表 19-2 中。

列表 19-2　局部反射的像素 shader

```
float4 reflectPS(vertexOutput IN,
                 uniform samplerCUBE EnvMap,
                 uniform sampler2D NormalMap,
                 uniform float4 SurfColor,
                 uniform float Kr, // intensity of reflection
                 uniform float KrMin, // typical: 0.05 * Kr
                 uniform float FresExp, // typical: 5.0
                 uniform float Bumpiness // amount of bump
                 ) : COLOR
{
  float3 Nu = normalize(IN.LightingNormal);
  // for bump mapping, we will alter "Nu" to get "Nb"
  float3 Tu = normalize(IN.LightingTangent);
  float3 Bu = normalize(IN.LightingBinorm);
  float3 bumps = Bumpiness *(tex2D(NormalMap, IN.TexCoord.xy).xyz - (0.5).xxx);
  float3 Nb = Nu + (bumps.x * Tu + bumps.y * Bu);
  Nb = normalize(Nb); // expressed in user-coord space
  float3 Vu = normalize(IN.LightingEyeVec);
  float vdn = dot(Vu, Nb); // or "Nu" if unbumped - see text
  // "fres" attenuates the strength of the reflection
  // according to Fresnel's law
  float fres = KrMin + (Kr - KrMin) * pow((1.0 - abs(vdn)), FresExp);
  float3 reflVect = normalize(reflect(Vu, Nb)); // yes, normalize
  // now we intersect "reflVect" with a sphere of radius 1.0
  float b = -2.0 * dot(reflVect, IN.LightingPos);
  float c = dot(IN.LightingPos, IN.LightingPos) - 1.0;
  float discrim = b * b - 4.0 * c;
  bool hasIntersects = false;
  float4 reflColor = float4(1, 0, 0, 0);
  if (discrim > 0) {
    // pick a small error value very close to zero as "epsilon"
    hasIntersects = ((abs(sqrt(discrim) - b) / 2.0) > 0.00001);
  }
```

```
    if (hasIntersects) {
      // determine where on the unit sphere reflVect intersects
      reflVect = nearT * reflVect - IN.LightingPos;
      // reflVect.y = -reflVect.y; // optional - see text
      // now use the new intersection location as the 3D direction
      reflColor = fres * texCUBE(EnvMap, reflVect);
    }
    float4 result = SurfColor * reflColor;
    return result;
}
```

我们提供几个另外的可选择项，以增加 shader 的逼真性。

第一项是表面颜色：因为来自金属的反射将带有它的颜色，所以这是支持金属表面的。而对于塑料或水等非金属材料，可以删去这项或安排为白色。

第二项是提供 Fresnel 类的反射衰减。对于纯金属表面这些项可以不考虑，但是它们对塑料和其他非金属材料的真实性是十分重要的。这里使用一个幂函数：如果不需要用户控制 Fresnel 近似值，衰减可以编码为一个 1D 纹理，并且针对 abs(vdn)索引。

针对没有凹凸的法线来衰减 Fresnel 项，某些模型可能会变得更好。这有助于抑制物体边缘的高频“火花”走样。那时用 Nu 代替 Nb 计算 vdn。

对于光滑的纯金属，Fresnel 衰减是 0：不再计算 fres，而代之计算 Kr。但是在现实世界中，很少有东西是真正纯的；甚至非常干净的金属表面，反射率都时常有微小的衰减，而肮脏的表面衰减就更明显。同样，比起“纯”金属反射的饱和色彩来，肮脏金属的色彩也时常会减弱。欲使实施性能和所需的复杂性达到平衡，需要最好的判断力。

可以用 FresExp 指数的值做实验，见图 19-9。这个近似法的创始人指定的指数是 5.0（Christophe Schlick 1994），如果使用更低的值能产生更有层次，或更有光泽的外表。指数 4.0 能通过 2 次乘法很快地计算，不需要运算昂贵的 pow()函数。

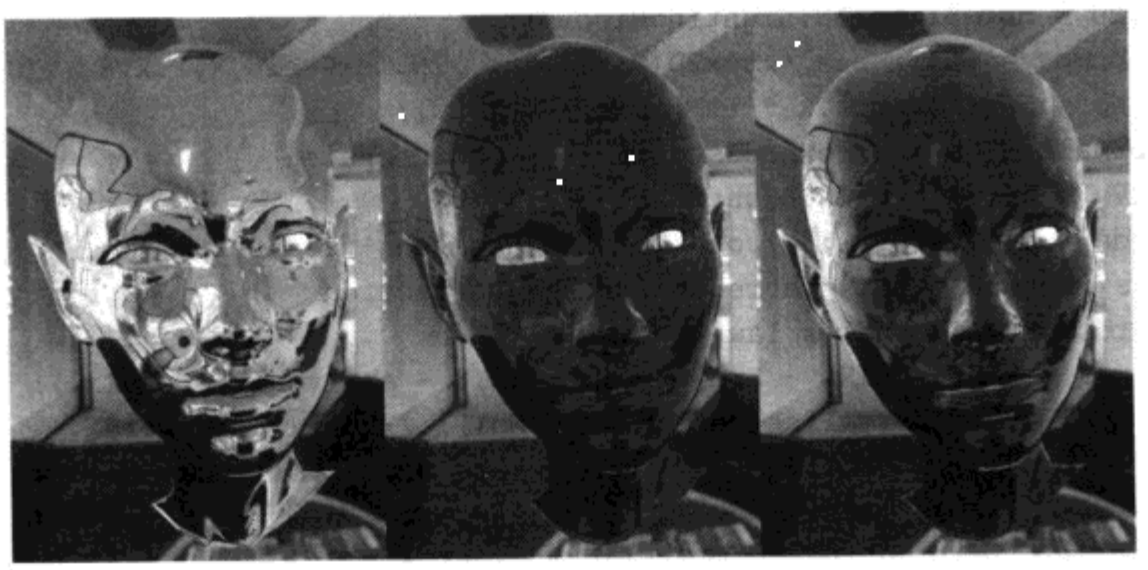


图 19-9 Fresnel 项衰减的效果

（a）这个图像没有衰减；（b）通过小的 reflMin 和一个 5.0 的 Fresnel 指数，这个图像有典型的非金属衰减；（c）这个图像指数减为 2.0，显示有光泽的外表

列表 19-2 的 shader 可以选择性地反转反射矢量的 y 成分。增加这个选择性的步骤是为了适应不同种类的发展环境，可以混合 OpenGL 和 DirectX 创建的立方体贴图（这些 API 的立

方体贴图规格的差别在于它们的 up 处理）。例如一个为 DirectX 的游戏引擎开发的场景也能用 Maya（OpenGL）开发。

19.4　漫反射 IBL

立方体贴图也能用来决定漫反射照明。Debevec 的 HDRShop 程序能够从映射立方体光照环境积分出全部 Lambernian 贡献，所以把表面法线代入预先卷积的立方体贴图，能够简单地查询漫反射贡献（这与反射照明相反，那时需要代入基于表面法线和视点位置的反射矢量）。

因为照明漫反射的贴图借助于点的“可见半球”编码，所以局部漫射矢量不如局部反射的结果令人满意。对于远离球心的数值，这些积分是不正确的。误差能否可以被接受，取决于应用的需要。某些情况下，在多个漫射贴图之间线性内插提供局部位置。这样的贴图倾向于低频。误差不大时不很引人注目，这是简单照明的好处。因此，使用漫反射和镜面反射的立方体贴图，有些应用能够执行所有的照明计算。

把漫射和镜面反射光照结合进立方体贴图，对有些应用而言，不再需要任何另外的光照信息。

19.5　影子

和 IBL 一起使用影子会将问题复杂化，但是有时仍然这样使用。像阴影贴图一样，模板阴影域技术可以用在这里。在这两种情况下，比较好的做法是提供一个小的环境照明项（当使用模板阴影体的时候，在另一遍中应用它），以避免物体完全地消失在黑暗中（除非那是所希望的）。

使用基于图像的光照自然要产生的问题是：阴影由何而来？即使来源不完全有根据，阴影仍然提供强有力的视觉信息。也就是说，阴影的真实来源可能不完全地对应于光源。在 IBL 的情况下这是真的，IBL 的阴影需要匹配很多光的方向，时常造成非常模糊的阴影。但是阴影映射和遮挡模板之类的技术，只能产生边缘清晰的影子，或很少模糊的阴影。

如果聪明地选择阴影来源的方向，这个问题就不存在。因为阴影的空间和图形通常比光照的“根源”更重要，所以观察者时常会接受高度人造的阴影（事实上，大多数的电视和电影更倾向于“无光源”的光照）。当把阴影加入任意 IBL 场景时，把最亮的面积取作立方体贴图的方向最好。此外，应该集中注意可能对模型幅度影响最大的阴影。

动画中的影子是十分重要的，因为它们关系到人物和模型与其周围的联系。人物在地上的影子会告诉人们，他相对于地面是站着、跑动还是跳跃。如果人物的脚与影子相接触，人是在地上（为这个目的而画的影子称为接触影子）。如果脚与影子不接触，人是在空中。

卡通漫画家已经利用这个影子的特性许多年了，由此可以想到，在 IBL 场景中只关心接触影子可能是有利的。如果我们所关心的是人物在地上的影子，那就可以做一个简单的假设，渲染影子时不需要计算深度，只需要颜色。这意味着，我们可以创建黑白或全色的投射影子，还可以使其模糊，而且假定它总是接触产生影子的物体。这避免了深度比较，而且增加有效纹理带宽（因为可以使用简单的 8 位纹理）。

这样的情节中，人物表面不包含他们自己阴影的贴图；也就是说，他们不是自阴影的。

他们的照明可能改为不受 IBL 的影响。而游戏玩家仍然能看到环境中的人物影子，给出影子原来的好处：把人物和 3D 游戏环境牢固地连接起来。

19.6 使用局部立方体贴图作背景

在这章的例子中，我们可以看到与背景互动的反射物体。没有背景可能注意不到效果。

许多情况下，我们可以用 3D 几何体做成立方体贴图，并且在正常地描绘环境的时候，把贴图应用到那个环境的物体上。我们也可以使用贴图作环境，把它投射到较简单的几何体上，如图 19-10 所示。

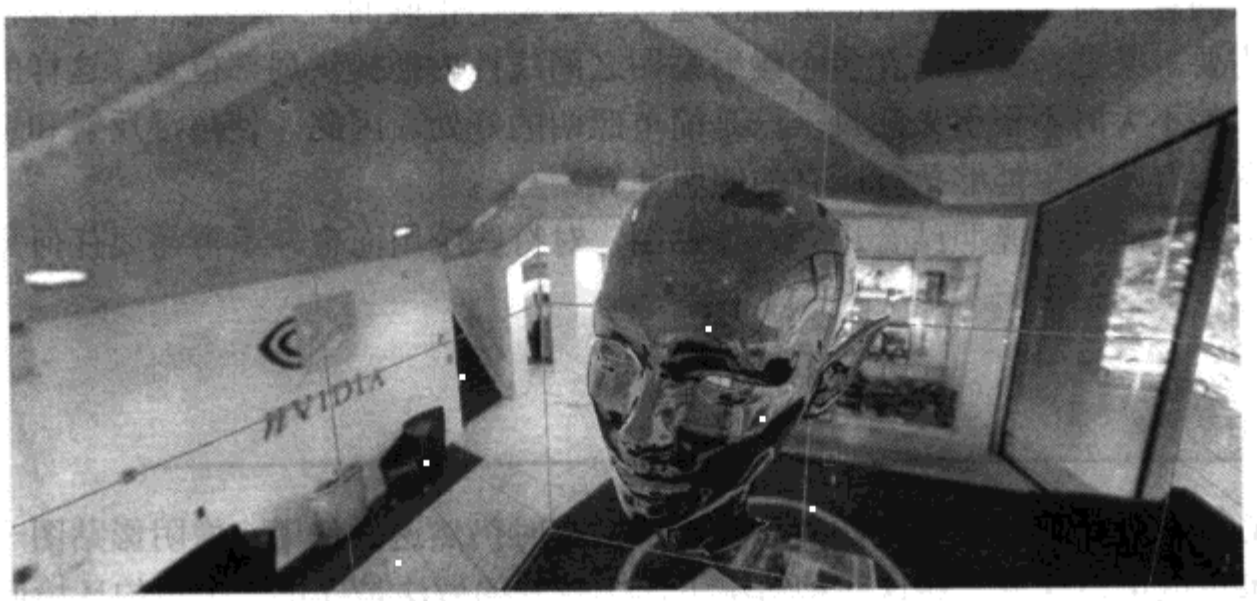


图 19-10 表示房间立方体边缘的线

用这个单位尺寸立方体的变换，来计算反射坐标系统和给背景本身涂颜色

对于背景立方体，我们也对单位立方体房间使用相同的变换。事实上，对于样例场景，我们只是对房间 shader 使用它自己的变换。简单的几何体就是几何体，不需要 UV 映射坐标或表面法线。

从图 19-10 可见，使用简单的立方体代替完整的场景几何体有一定的限度！注意左边的“弯曲”天花板。当摄像机接近立方体中心时，使用代用几何体通常工作得最好。组合的环境（例如这个相片）也能通过把平面排成行得到好处，例如正好是照明空间的分界线的墙壁和天花板。

```
// data from application vertex buffer
struct appdataB {
  float3 Position : POSITION;
};
```

顶点 shader 将传递观察矢量和通常所必需的剪裁空间位置。

```
// data passed from vertex shader to pixel shader
struct vertexOutputB {
  float4 HPosition : POSITION;
  float3 UserPos   : TEXCOORD0;
};
```

列表 19-3 表示了顶点 shader 本身。

列表 19-3　用于背景立方体物体的顶点 shader

```
vertexOutput xfBoxVS(appdataB IN,
                     uniform float4x4 WorldViewProj,
                     uniform float4x4 WorldIT,
                     uniform float4x4 World,
                     uniform float4x4 ViewIT,
                     uniform float4x4 UserXf)
{
  vertexOutputB OUT;
  float4 Po = float4(IN.Position.xyz, 1.0);
  OUT.HPosition = mul(WorldViewProj, Po);
  float4 Pw = mul(World, Po);
  OUT.UserPos = mul(UserXf, Pw).xyz;
  return OUT;
}
```

像素 shader 使用片元 shader，在立方体贴图之内，连同一种可选择的颜色，得到一个直接的纹理查询，见列表 19-4。

列表 19-4　用于背景立方体的像素 shader

```
float4 xfBoxPS(vertexOutput IN,
               uniform samplerCUBE EnvMap,
               uniform float4 SurfColor) : COLOR
{
  float4 reflColor = SurfColor * texCUBE(EnvMap, -IN.UserPos);
  return reflColor;
}
```

这个 shader 是特别设计的，当投射到立方体时（可能扭曲）工作得很好。也可以用于其他简单几何体，例如球体、圆柱体，或平板。

19.7　小结

基于图像的照明为麻烦的照明计算提供了综合而便宜的替代品。把一点数学加入纹理化方法，可以大大拓宽“简单的”IBL 的效果，给 3D 图像提供更强的方位感。

19.8　参考文献

Schlick, Christophe. 1994. “An Inexpensive BRDF Model for Physically-Based Rendering.” Computer Graphics Forum 13(3), pp. 233～246.

这篇文章介绍在图形业使用的 Fresnel 方程近似式，它应用十分广泛，以至于有些程序设计者把这个近似式误认为是 Fresnel 方程。

Paul Debevec 提供许多有用的 IBL 工具和文章，网址是 **http://www.debevec.org**

Ramamoorthi, Ravi, and Pat Hanrahan. 2001. "An Efficient Representation for Irradiance Environment Maps." In *Proceedings of SIGGRAPH 2001*。

这篇使用球面调和求漫反射卷积的文章可在 **http:// graphics.stanford.edu/papers/envmap** 上得到。

Apodaca, Anthony A., and Larry Gritz, eds. 1999. *Advanced RenderMan: Creating CGI for Motion Pictures*. Morgan Kaufmann.

在这本书中，Larry Gritz 介绍一个使用 RenderMan 着色语言的类似反射算法。

第 20 章　纹理爆炸

R. Steven Glanville
NVIDIA

可以用纹理给几何体加入视觉细节，但是它们不能延展覆盖像花园那样大的面积，或覆盖许多相似的物体，例如充满许多建筑物的城市。这种用途需要大量的纹理数据，或者重复相同的图案，结果会造成不好看的单调图样。纹理爆炸是一种程序化技术，它把小块图像以不规则的间隔放置，有助于减少图案的失真。

20.1　纹理爆炸 101

纹理爆炸的基本思路是把 UV 空间分为规则的单元栅格。然后使用噪声或伪随机数函数，把一个图像放在任意位置上的各个单元中。最终的结果是在背景上对这些图像的合成。

由于要组合数以百计的图像，因此实际上这种合成图像的方法效率不高。有步骤地计算图像虽然好，但是又不适于合成。在这章要讨论这两种技术。我们从像素的投射开始，并试着简化步骤。第一步是找到包含像素的单元。我们需要一个方法计算每个单元的坐标，很快地确定相邻单元的位置。使用这些坐标存取单元的独特参数，例如在单元中的图像位置。最后，从单元中图像的位置上采样，并把它和一种默认的背景颜色相结合。在这里我们假定，一个样本是 100%不透明的或透明的，这样可以简化合成步骤。

然而这样做是不简单的。如果单元图像的位置接近单元的边缘，或它比一个单元更大，它就要超出到相邻的单元。因此还需要考虑相邻单元的图像。大的单元图像要求从许多相邻的图像采样。如果限制图像尺寸不超过一个单元，那么为一个图像就需要对 9 个单元采样：2 个侧面的、1 个上面的、1 个下面的，4 个角上的。对图像规定几个限制，可以把采样数减少为 4 个。

20.1.1　求单元

使用像素 UV 参数的 floor 函数，可以简单计算单元的坐标：

```
float2 scaledUV = UV * scale;
int2 cell = floor(scaledUV);
```

注意单元的坐标是整数。相连的 x 单元或 y 单元之间相差 1。通过缩放因子可以根据需要改变单元尺寸。求单元中的位置使用减法：

```
float2 offset = scaledUV - cell;
```

20.1.2 对图像采样

在对图像采样前，需要确定它的位置。使用伪随机数所填充的二维纹理完成这个任务。用小数缩小单元坐标，推导出样本的坐标。

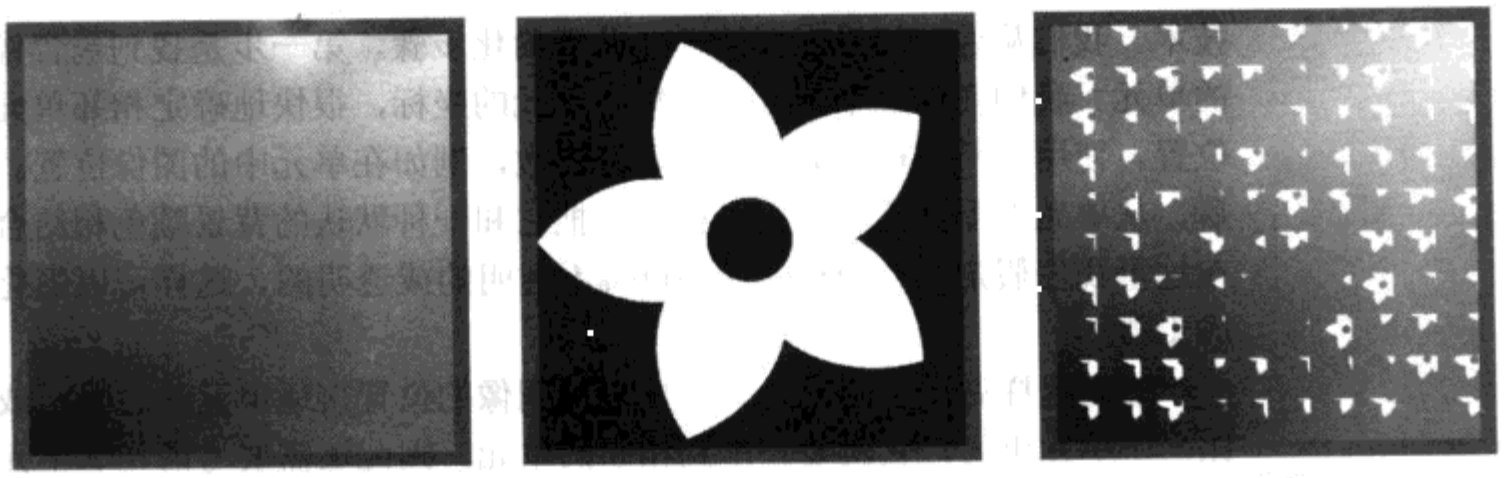
```
float2 randomUV = cell * float2(0.037, 0.119);
fixed4 random = tex2D(randomTex, randomUV);
```

用于计算随机 UV 的缩放因子并不神秘。它们把整个的单元映射到任意纹理里的某处，而且把邻近的单元映射为不相关的数值。那些数值应该大到足以把相连的单元分开，而分开的距离要超过一个 texel。

我们使用纹理的前两个成分作为图像在单元中的位置，必须把它们从单元中像素的偏移量中减去。如果图像的 alpha 成分不是 0，就用图像的颜色；否则就用原先的颜色。

```
fixed4 image = tex2D(imageTex, offset.xy - random.xy);
if (image.w > 0.0)
  color.xyz = image.xyz;
```

把这些一起放进样本程序 texbomb_1.cg，并使用图像 20-1（a）和 20-1（b）的背景和图像花纹，产生的结果显示在图 20-1（c）中（所有在这章中显示的样例程序可在书的 CD 和书的网站上找到）。



（a） （b） （c）

图 20-1 纹理爆炸的第一个尝试

（a）背景；（b）例子中使用的花朵图案；（c）texbomb_1.cg 输出的图案

还要用单元的边裁剪图像。只有图像偏移量是 0 的时候，才能看见完整的图像。我们也必须在相邻的单元里寻找图像。

20.1.3 相邻单元中的图像

已经讨论了对当前单元的图像采样。现在考虑重叠在相邻单元中的图像。因为图像的偏移量与一个单元的比为 0 到 1，一个图像只能重叠在某单元的上部和右部。这样我们只需要

对 4 个单元采样：当前的单元加上它下面和它左边的那些单元，见列表 20-1。

列表 20-1　把采样扩展到 4 个单元

```
for (i = -1; i <= 0; i++) {
  for (j = -1; j <= 0; j++) {
    float2 cell_t = cell + float2(i, j);
    float2 offset_t = offset - float2(i, j);
    randomUV = cell_t.xy * float2(0.037, 0.119);
    random = tex2D(randomTex, randomUV);
    image = tex2D(imageTex, offset_t - random.xy);
    if (image.w > 0.0)
      color = image;
    }
  }
}
```

20.1.4　图像优先级

还有一点问题。当两个图像重叠的时候，因为右上角的那个是最后一个被测试的，所以它总是在顶部。这对鱼鳞或条状花纹尚可接受，但是最好避免这种效果。为了解决这个问题，对每个单元规定一个优先级，优先级最高的图像优先处理。使用单元的任意参数 *w* 分量作为优先级，并把列表 20-1 所示的 if 检测，改变为列表 20-2 所示。

列表 20-2　加入图像优先级

```
fixed priority = -1;
. . .
if (random.w > priority && image.w > 0.0) {
  color = image;
  priority = random.w;
}
```

图 20-2 显示由这种修正的程序 texbomb_2.cg 制作的图像。

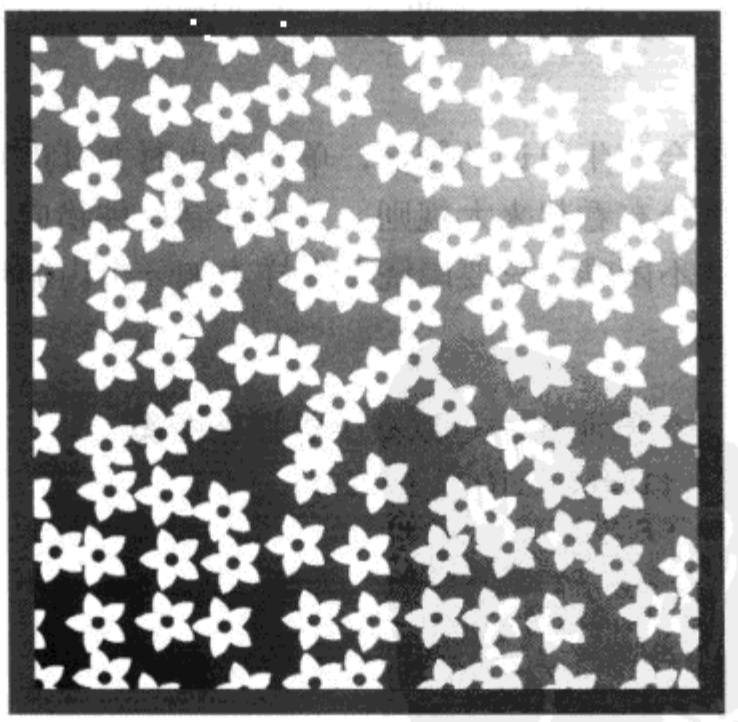


图 20-2　texbomb_2.cg 的输出

20.1.5 程序化图像

图像不一定由纹理产生。程序化图像工作得很好。在列表 20-3 中，用程序化产生的半径 0.5 的圆圈代替图像样本。

列表 20-3 程序化产生的圆圈的使用

```
offset_t -= float2(0.5, 0.5) + (float2) random;
fixed radius2 = dot(offset_t, offset_t);
if (random.w > priority && radius2 < 0.25) {
  color = tex2D(randomTex, randomUV + float2(0.13, 0.4));
  priority = random.w;
}
```

任意纹理的第 2 个例子给出了圆圈的颜色，由程序 gem_proc_2.cg 产生，如图 20-3 所示。

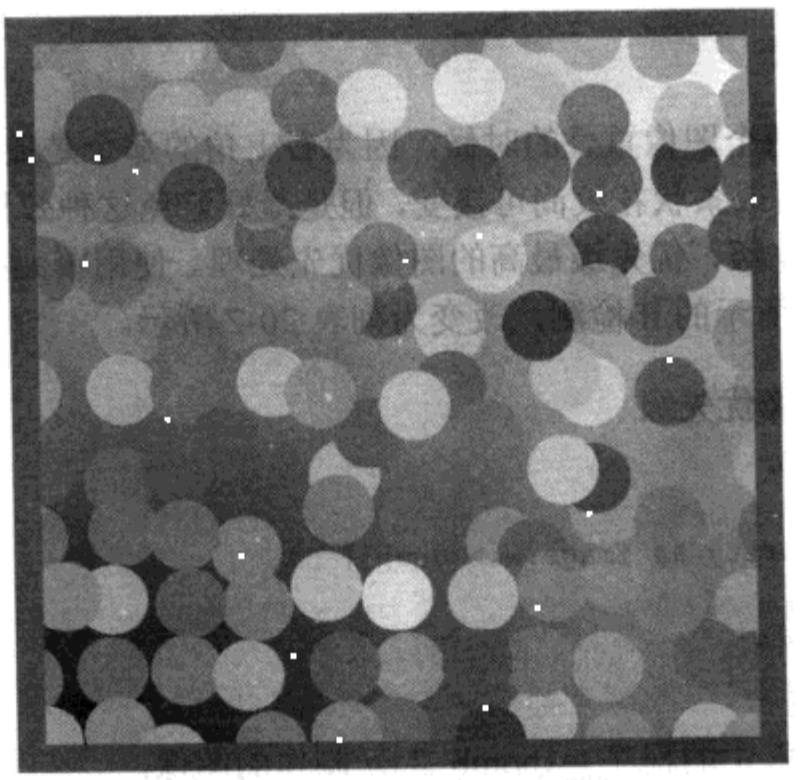


图 20-3 gem_proc_2.cg 的输出

每个单元多个图像

每单元只使用一个图像会产生单调的图案，单元很小时尤其严重。这是因为没有足够的变化隐藏计算的栅格性，使分布看起来太规则。为了抑制这种趋向，我们可以在每个单元中增加图像的数目。这样做并不困难，只要围绕图像样本加一个内循环，并偏移用于查询随机数的坐标即可，见列表 20-4。

列表 20-4 每单元取多个圆圈样本可以减少类栅格图案

```
randomUV = cell_t.xy * float2(0.037, 0.119);
for (k = 0; k < NUMBER_OF_SAMPLES; k++) {
  random = tex2D(randomTex, randomUV);
  randomUV += float2(0.03, 0.17);
  image = tex2D(imageTex, offset_t - random.xy);
  . . .
}
```

我们在内循环中对每个通道偏移坐标，以避免每次采样的数值相同，如图 20-4 所示。

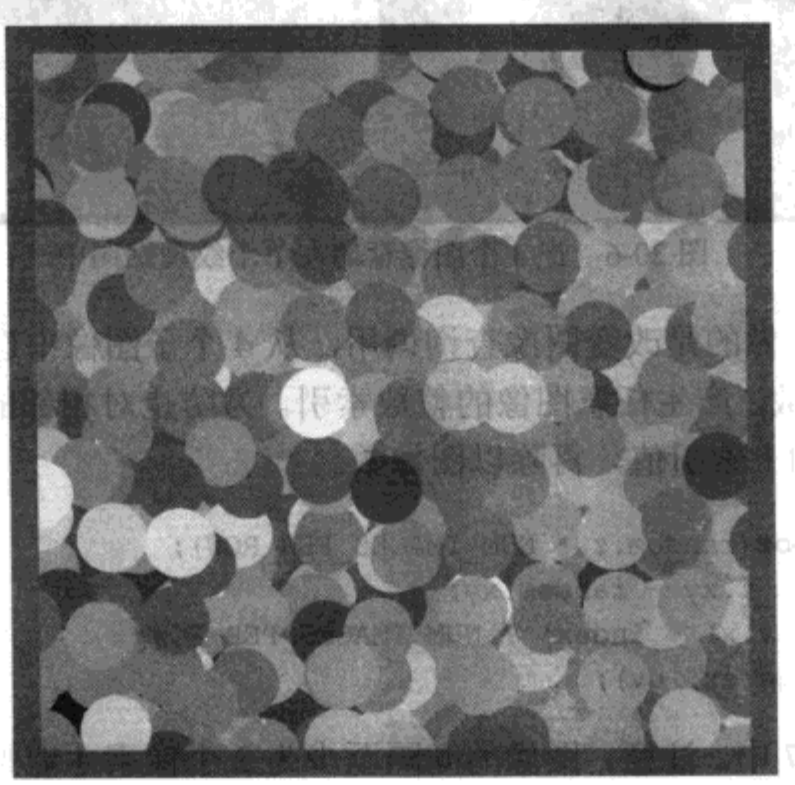


图 20-4　每单元采样 3 个圆圈

我们也可以使用另外的随机值，或只是设定初始优先值比 0 大，例如是 0.5，来改变每单元图像的数目。于是每个图像被拒绝的概率是 0.5。这种变化进一步明显增加结果的任意性，如图 20-5 所示。为了表示一个较大的数，这里单元的尺寸已经被减小了。

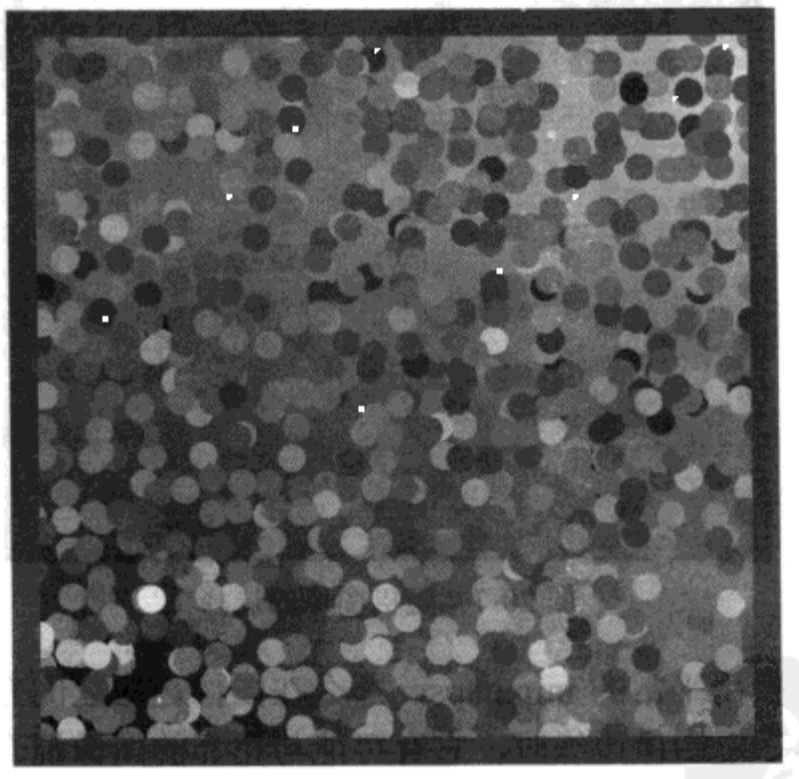


图 20-5　改变每单元圆圈的数目

20.1.6　图像的随机选择

另一个增加变化的办法是每次选择多个图像中的一个。一种方法是把图像重复排列在单个纹理之内，而且以任意纹理的 z 成分作为这些图像的索引。更简单的是把图像安排为一个水平行，如图 20-6 所示。黑面积对应于透明的区域，alpha 成分为 0。

图 20-6 把 4 个图像存储在单个纹理贴图中

然后，我们所需要做的是改变图像查询调用，从 4 个子图像中选择 1 个。缩放任意矢量的 z 成分并计算 floor 数，产生任意图像的整数索引。为防止对相邻的图像采样，我们需要钳制 x 坐标，然后将它加上索引值，再除以图像数。

```
float index = floor(random.z * NUM_IMAGES_PER_ROW);
float2 uv = offset_t.xy - random.xy;
uv.x = (saturate(uv.x) + index) / NUM_IMAGES_PER_ROW;
image = tex2D(imageTex, uv);
```

结果显示在图 20-7 中。注意，图像的间隔至少为 2 个像素才能把它们的边分开，否则另外的纹理采样将会引起失真。

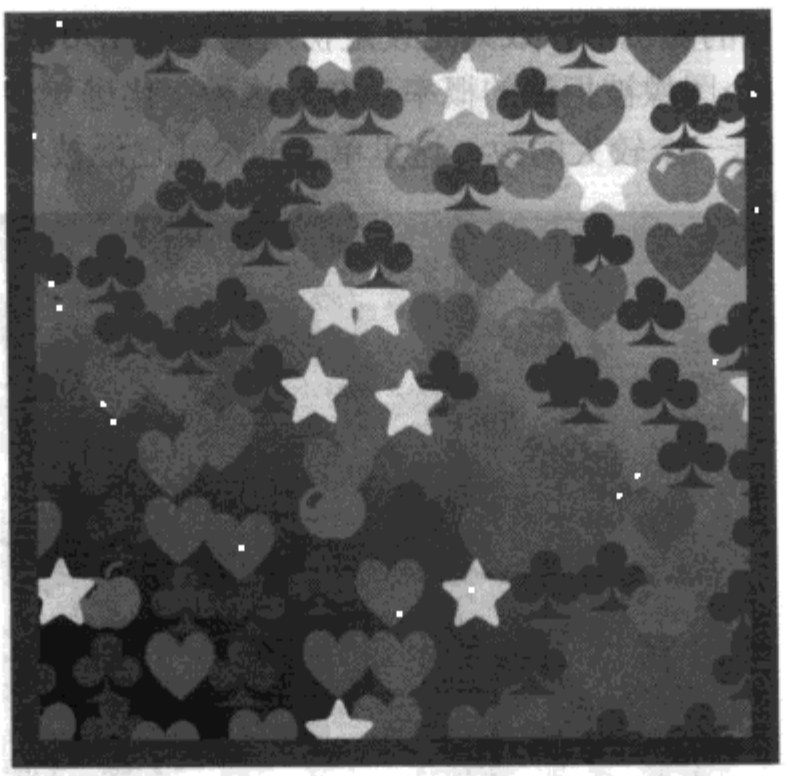


图 20-7 随机图像的选择

如果对多图像纹理使用平常的 mipmaps，那么在图像边缘之间有两个像素是不够的。从高级别贴图采样的样本靠近图像边缘时，它有可能与相邻的图像混合。要减小这个问题，可以增加边的宽度，或用每个级别都相似的边构造特殊的 mipmaps。为避免不好的情况，可以限制 mipmap 的最大级别，也可以把图像尺寸减小为 0。

20.2 技术上的考虑

下面讨论一些实施细节。

对随机数纹理选择正确的缩小滤波是很重要的。当启用 mipmapping 时，基于当前像素与相邻像素 UV 值之差的细节级别（LOD），由求得伪随机数的相关纹理读数生成。在单元内，随机 UV 对所有的像素是相同的。因此差别是 0，以级别（LOD）0 采样，产生一个从最初的随机数来的值（可能是双线性的插值）。然而，在两个单元之间的边界，随机 UV 的值会突然地改变。因此，使用更高的 mipmap 级别，也就引起了失真。

为了除去随机纹理的失真，指定一个 LINEAR 或 NEAREST 缩小滤波，因为他们强迫所有的样本为 LOD 0。对静态纹理最好使用 NEAREST 滤波。LINEAR 滤波减少了随机数的变化，而且使图像靠近单元的中心。但是如果移动图像的位置，那么使用 LINEAR 更利于平滑的运动。

纹理图像的包裹模式应该设定为 CLAMP。否则，会出现不该看到的“鬼”样。我们也可以把坐标钳制在[0, 1]范围。大多数的 GPU 可以无开销地完成它。

减少运行时间对像素 shader 永远是重要的，对纹理爆炸也不例外。相关的纹理采样、对多个单元重复地计算基本的着色，以及每个单元有多个样本，这些联合妨碍了实现性能。产生的 shader 指令可能长达百条。

举例来说，在图 20-2 中显示的来自 texbomb_2.cg 的雏菊图像，要求 8 个纹理样本和 44 个指令。在图 20-3 中显示的来自 gem_proc_2.cg 的第一个程序化图像，需要 8 个纹理样本和 62 个指令。

一些主要的优化法可以帮助提高实施性能。在可能的时候，把最后的彩色纹理样本移出内循环。不是多次纹理采样，而是把产生的样本坐标存在一个变量中，在程序结尾时做最后的一个样本。因为决定使用一种特别的颜色不是基于样本值（Voronoi 区域见第 20.3.5 节），这对程序化形状（例如圆圈）和类 Voronoi 的区域很适用。但是，当需要用 alpha 成分决定那个值是否透明时，这个技巧没有用。

在 GeForce FX 硬件上，尽可能使用固定的和半精度的变量。这允许在每个 shader 通道运行更多的指令，并减少运行一个程序所需要的寄存器数目。除非渲染浮点帧缓冲区，颜色可以使用较低的精度。但是纹理坐标通常还是应该以全浮点精度存储。

20.3　高级特性

为了有趣的效果，纹理爆炸能以多种方式扩展。

20.3.1　缩放和转动

任意缩放或转动每个图像，可以得到更多的变化。一定要小心照顾到相邻单元采样的面积，因为图像可能覆盖这个区域。至于缩放，显然只能缩小，否则每个像素测试的单元就会超过 4 个。

转动图像有相似的限制。图像的不透明部分应该放在单元中直径为 1 的圆圈里。如果它超过这个尺寸到达角上，那么所转动的图像在垂直或水平方向就可能超出 0.5 个单元宽度。于是 4 单元样本的算法可能错过这些部分。

20.3.2　可控的变量密度

有时候想要在艺术上控制图案，例如，地上的树叶。使用伪随机密度并不能把这个情

况处理得很好。这时可以使用一个指定密度的纹理和给出希望叶子出现的概率。读纹理时应该把它作为正常的基于 UV 的样本，使用那个给定值设定叶子出现的概率，如前面例子所示。

20.3.3 程序化的 3D 爆炸

一旦已经决定使用程序化纹理，就没有必要限制单元在 2D 的 UV 空间。为什么不把思路扩展到 3D 空间呢？那样可以使用像素的物体空间坐标得到颜色。在列表 20-5 中，我们把物体空间分为单位立方体。我们仍然可以使用 2D 伪随机纹理，但是在计算样本的坐标时需要包括第三维作为一个成分。缩放 z 坐标并把它加入 x 和 y 给出了一个好的结果。

列表 20-5 程序化的 3D 爆炸

```
for (i = -1; i <= 0; i++) {
  for (j = -1; j <= 0; j++) {
    for (k = -1; k <= 0; k++) {
      cell_t = cell + float3(i, j, k);
      offset_t = offset - float3(i, j, k);
      randomUV = cell_t.xy * float2(0.037, 0.119) + cell_t.z * 0.003;
      . . .
    }
  }
}
```

对于一个总数为 8 的样本，必须把采样的单元数扩展到三维。因此三维的纹理爆炸必然比二维更费时。图 20-8 的例子显示了程序化产生的球体。它使用简单的点积漫射光照明模型表示深度。

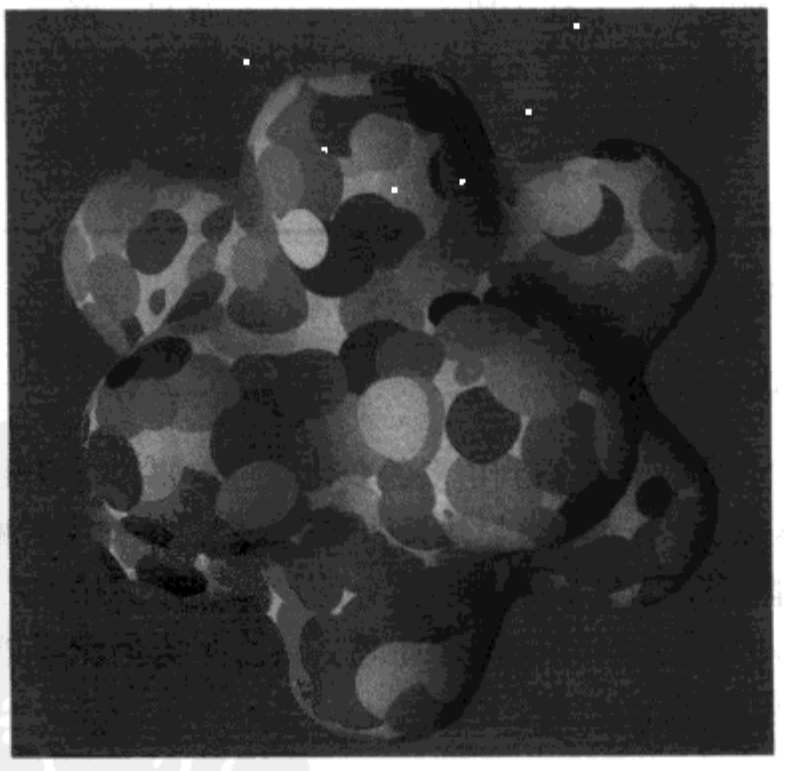


图 20-8 每单元 1 个球的程序化 3D 纹理

为了清楚，完整的 shader 在列表 20-6 中显示。

列表 20-6　程序化 3D 纹理程序

```
priority = -1;
for (i = -1; i <= 0; i++) {
  for (j = -1; j <= 0; j++) {
    for (k = -1; k <= 0; k++) {
      cell_t = cell + float3(i, j, k);
      offset_t = offset - float3(i, j, k);
      randomUV = cell_t.xy * float2(0.037, 0.119) + cell_t.z * 0.003;
      random = tex2D(randomTex, randomUV);
      offset_t3 = offset_t -(float3(0.5, 0.5, 0.5) + (float3) random);
      radius2 = dot(offset_t3, offset_t3);
      if (random.w > priority && radius2 < 0.5) {
        color = tex2D(randomTex, randomUV + float2(0.13, 0.4));
        priority = random.w;
      }
    }
  }
}

float factor = dot(normal.xyz, float3(0.5, 0.75, -0.3)) * 0.7 + 0.3;
color.xyz = color.xyz * factor;
return color;
```

20.3.4　随时间变化的纹理

动画爆炸的纹理 shader 很容易。需要做的是基于时间改变一个或更多的伪随机参数。如果采样的伪随机数纹理有双线性滤波器，那么许多性质将会平滑地运动。但是有时这个过滤存在像楼梯一样的走样，尤其对于运动的物体，因为人类眼睛对运动方向的间断很敏感。

```
random = tex2D(randomTex, randomUV * time * 0.0001);
```

在这个代码中，时间以秒表示。如果不以适当的因数缩小，动画会走得太快。缩放因子取决于动画的速率和伪随机纹理的大小。如果用于例子的纹理是 1024 × 1024，那么大约每 10 秒采样一个新的纹素：

$$\text{变化率} = (\text{缩放因子} \times \text{时间}) / \text{纹理尺寸}$$

20.3.5　Voronoi 相关的细胞法

纹理爆炸一种有趣的变化是在平面上画 Voronoi 区。关于这种纹理技术的更多信息可见 Steven Worley's 1996 SIGGRAPH 的论文。简言之，给定一个平面和那个平面上的一系列的点，接近那个点的面积就是点的 Voronoi 区。Voronoi 图案类似于树叶和皮肤上的单元形状、龟裂的泥土或爬虫类的皮，甚至看起来更规则。一个典型的 Voronoi 区视图显示在图 20-9 中。

我们可以对纹理爆炸的基本码进行一点改变来制造这些图案。从列表 20-3 中程序的圆圈代码开始，我们以到点的距离作为优先次序，代替随机的 w 数值，而且不查看是否在圆圈半径之外，见列表 20-7。

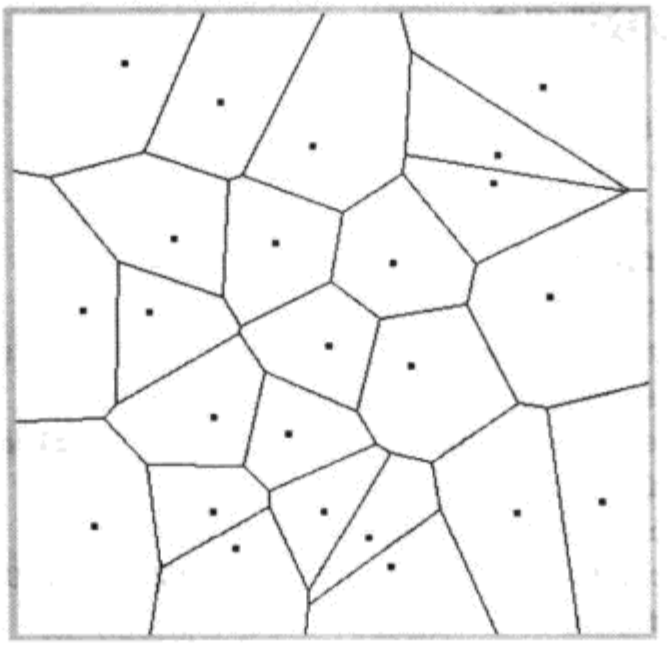
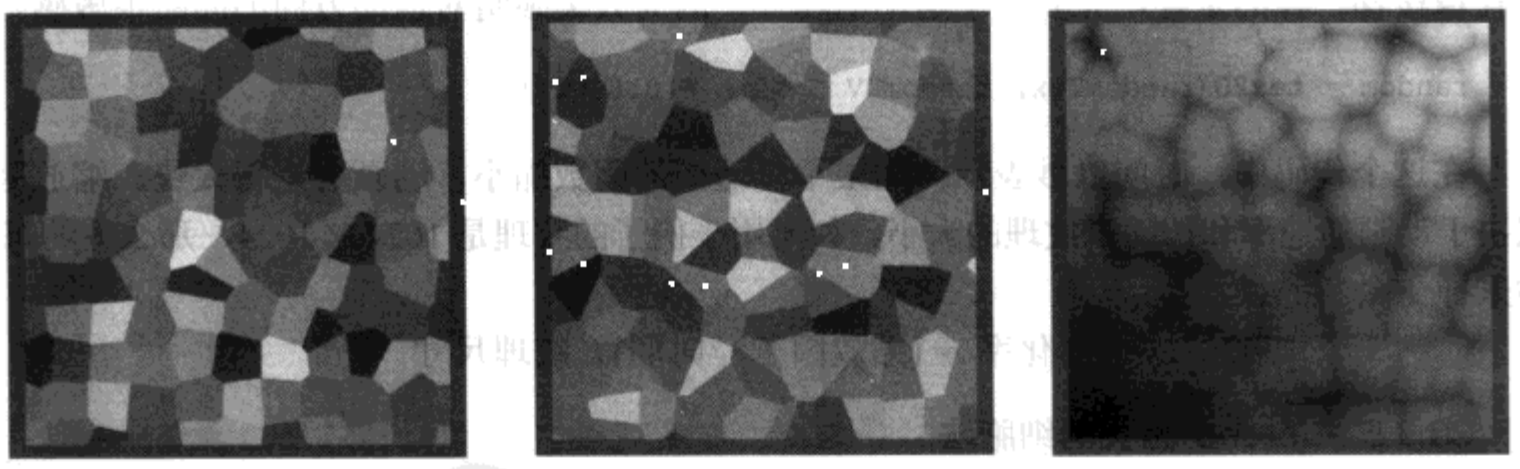


图 20-9 Voronoi 区的样本

列表 20-7 Voronoi 区域的计算

```
float priority = 999;
. . .
radius2 = dot(offset_t2, offset_t2);
if (radius2 < priority) {
  color = tex2D(randomTex, randomUV + float2(0.13, 0.4));
  priority = radius2;
}
```

我们使用距离的平方代替真实的距离，从而避免计算平方根。每个区域指定一个任意的颜色。结果显示在图 20-10 中。



（a） （b） （c）

图 20-10 Voronoi 区域

（a）每个单元 1 点的区域 （b）每个单元 3 个点的区域 （c）源于（b）的像网一样的效果

在图 20-10（a）中，可以清楚地看到算法的矩形栅格结构。通过增加样本的数目，这种规律性能明显地减少。图 20-10（b）使用每单元 3 个样本，用一个 sqrt(3)的因子缩放单元，保持 Voronoi 区域的平均尺寸相同。

要注意的是，由这个算法创建的单元不能保证是 Voronoi 区域。在相邻单元的点 P_n 可能在与我们样本相对的角上，但是相邻单元的下一个单元中的点 P_{n+1} 可能在靠近我们的边上，

结果实际上 P_{n+1} 更靠近我们的样本。但是计算不包含 P_{n+1} 所在的单元。这实际上并不经常发生，因此区域还是有用的。增加每单元的样本数可以减少发生这个问题的概率。

可以用这些单元创建许多有趣的效果。图 20-10（c）的单元图案与图 20-10（b）的相同，但是它使用最近点距离的平方调节本底颜色。

20.4　小结

纹理爆炸和相关的细胞技术可以 shader 增加视觉的多样性。使用存储在纹理中的伪随机数表和一个小程序，可以增大一个图像或一组图像的变化，并减少大块纹理区域的重复。

20.5　参考文献

Apodaca, A. A., and L. Gritz. 2002. *Advanced RenderMan*, pp. 255～261. Morgan Kaufmann.

Ebert, D. S., F. K. Musgrave, D. Peachy, K. Perlin, and S. Worley. 2003. *Texturing and Modeling: A Procedural Approach*, 3rd ed., pp. 91～94. Morgan Kaufnann.

Lefebvre, S., and F. Neyret. 2003. "Pattern Based Procedural Textures." *ACM Symposium on Interactive 3D Graphics 2003*.

Schacter, B. J., and N. Ahuja. 1979. "Random Pattern Generation Processes." *Computer Graphics and Image Processing* 10, pp. 95～114.

Worley, Steven P. 1996. "A Cellular Texture Basis Function." In *Proceedings of SIGGRAPH 96*, pp. 291～294.

第 4 部分

图像处理

简 介

提到 GPU 的功能，大多数人只想到纯粹的 3D 应用程序，而实际上在我们的监视器上见到的图片却是 2D 的。可编程像素 shader 给个人计算机提供了空前的图像处理功能，扩展了我们对 2D 和 3D 的开发能力。

图形设计者和插图画家早就在静态图像的图片编辑中使用合成控制了，但是这种程序每次只能处理一张图片。最新的图形处理器能以交互式速率执行以前很慢的运算，现在我们不但在静态图像和规划中，而且在全面动态的视频和游戏中，都能应用复杂合成和色彩控制的全部功能。以前需要一整套特殊装置才能上色的视频，现在只要一台个人手提电脑就可以完成。

无论是把 2D 图像映射成新的 2D 图像，还是把 3D 场景渲染到屏幕上，理解色彩空间是图像处理的一个方面。理解正在改变大小和形状的图像，如何能在处理期间过滤，提供平滑而又快速的结果，也同样重要。GPU 能够像基于 CPU 的图像处理工具一样，执行同样的过滤算法，但是他们还可以使用新的硬件加速方法。

在**第 21 章“实时辉光”**中，**Greg James** 和 **John O'Rorke** 给了我们一个例子，说明如何用 2D 图像处理提高 3D 游戏的可玩性。通过增加 2D 照明效果，例如辉光，表现一个小图像处理能如何完全地改变感觉和 3D 人物的动作。一旦看到如何完成这些结果，就再也不能离开它们。

在**第 22 章“颜色控制”**中，我把游戏中图像处理的讨论，扩展为技术上和艺术上控制颜色的方法和应用，包括把角色在不寻常的彩色空间移进移出，以及很快给任何 2D 或 3D 的场景加上精美的色调。今天我们在印刷品和电视上见到的颜色，几乎都是全面控制的颜色，游戏开发者应该理解这些方法并在其引擎中使用。

在**第 23 章“景深：技术综述”**中，**Joe Demers** 介绍如何使用 GPU，创建实时景深效果。遵循一些简单的过滤规则，开发者能模拟真实摄像机复杂的聚焦、底片饱和，以及在 3D 模型上的更多效果。

第 24 章“高质量的过滤”概述了图像过滤和用于任意尺寸图像的效果，把滤波器核心概念和分析计算应用于 2D 和 3D 反走样问题。最有意义的是，一些“经典的”反走样和过滤问题，使用硬件加速器而不是传统基于 CPU 的方法，在 GPU 上得到了最好的解决。

在**第 25 章“用纹理贴图进行快速过滤宽度的计算”**中，**Matt Pharr** 描述纹理在 2D 空间不寻常的应用。即使硬件 profile 对求复杂函数的局部偏导数不提供直接支持，他通过聪明的纹理操作，也能够求出结果。

在**第 26 章“OpenEXR 图像文件格式”**中，工业光影公司（ILM）的 **Florian Kainz**、**Rod Bogart** 和 **Drew Hess** 描述了 OpenEXR 标准，这是新的高动态范围图像格式，它在计算机成像的顶级电影中正在快速推广。对基于图像照明的开发者而言，OpenEXR 是关键的工具，ILM 小组显示他们如何接受 GPU 处理速率，使 OpenEXR 成为有价值的日常工具，它不仅工作于 3D，也服务于实时图像的获取、合成和回放。

当然，应用这些技术需要开发者管理图像处理。**在第 27 章“图像处理的框架”**中，**Frank Jargstorff** 提出对 2D 图像处理的安排，很灵活并可应用于多个方面。这种结构甚至能扩展到混合 GPU 和 CPU 的操作，易于游戏、电影和视频的开发者理解和使用。

Kevin Bjorke, NVIDIA

第 21 章 实时辉光

Greg James
NVIDIA
John O'Rorke
Monolith Productions

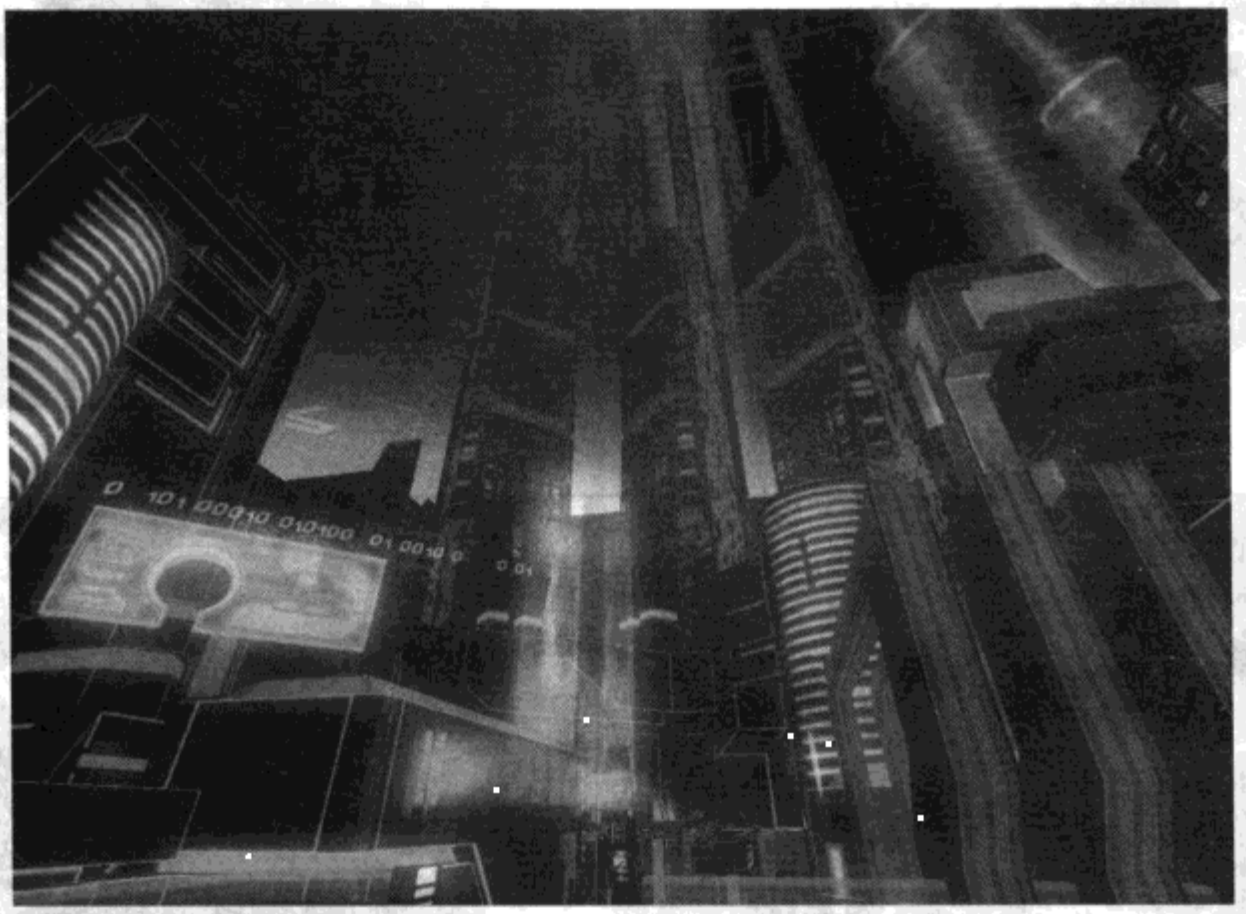
光源的辉光（glow）和晕圈（Halo）是自然界到处可见的现象，它们提供了亮度和气氛的强烈视觉信息。在观看计算机图形、胶片和印刷品时，到达眼睛的光强度是有限的，因此，辨别光源强度的惟一方法是通过它们周围的辉光和晕圈（Nakamae 等 1990）。这些辉光再生了光源强度的视觉效应，而且迷惑观察者，让他们感觉到非常明亮的光源。甚至围绕一个物体周围的微弱辉光也使人觉得它比不发光的物体更明亮。在日常生活中，这些辉光和晕圈是由于光线在大气或我们眼睛中散射而引起的（Spencer 1995）。使用现代的图形硬件，可以通过一些简单的渲染操作再生这种效果。这使得我们可以用明亮而有趣的物体填满实时渲染的场景，物体会显得更逼真或更奇妙；这是一个极好的方法，可用来克服传统低动态实时图形的单调呆板问题。

一些游戏现在正使用各种技术来产生光源的辉光和晕圈。其中有分裂细胞（Splinter Cell）、Gotham 竞赛项目（Project Gotham Racing）、Wreckless（Kawase 2003）和光晕 2（Halo 2）。另外一个著名的普遍的对辉光的使用，可以在 Pixar 的电影“海底总动员”中看到，在那里辉光表达海水的黝暗，并且帮助各种不同的场景设定氛围。这章把重点集中在最近上市的 Tron 2.0 游戏所开发的特殊技术，它是由 Buena Vista Interactive 生产的和由 Monolith Productions 开发的。该技术的设计为的是在整个屏幕上产生大面积的辉光，它很容易制造和控制一大组游戏资源，并且速度很快，足以使第一人设计的游戏以超过每秒 60 帧的速度运行。结果在图 21-1 和 21-2 中显示。在这里，这种效果表达了 Tron 2.0 计算机世界的振动和电子功率，该技术也能用来创建其他效果，包括景深、光散射、边缘检测和图像处理。

21.1 技术概述

有几种方法可以创建场景中的辉光。对于小的类似点的物体，可以把一个平滑的“辉光”纹理应用到公告牌几何体上，而让公告牌几何体在屏

幕范围内跟随着物体。在 Tron 2.0 中，这用于 Bit 角色。对于大的辉光源或复杂的辉光形状，要创建辉光，最好对 2D 场景的渲染进行后处理。这章把重点集中在后处理方式，在图 21-3 中概略说明了这种方式的步骤。



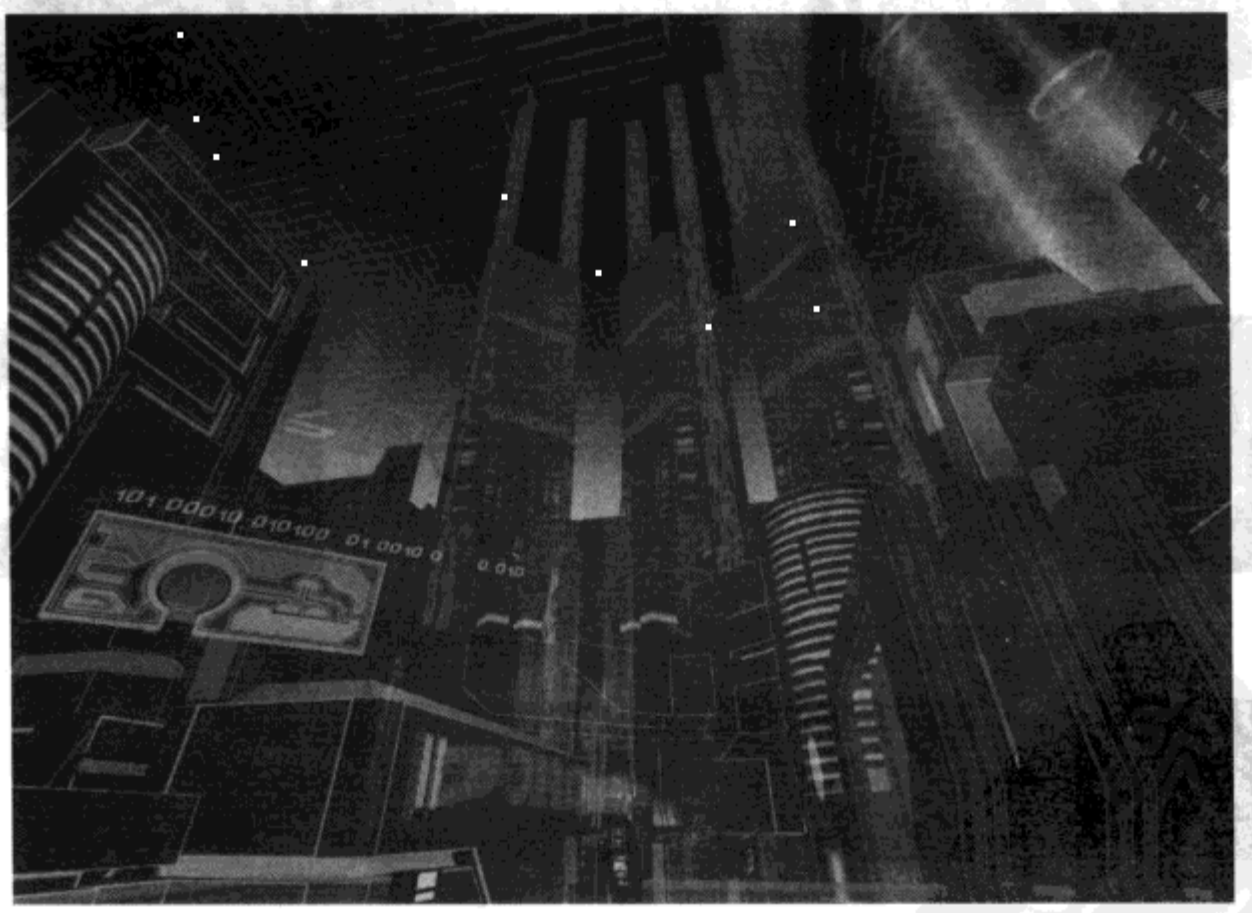


图 21-1　有辉光和没有辉光的一个 Tron 2.0 都市风景
顶部：有辉光效果的一个场景；底部：没有辉光的相同场景
图由 Buena Vista Interactive/Disney Productions 提供

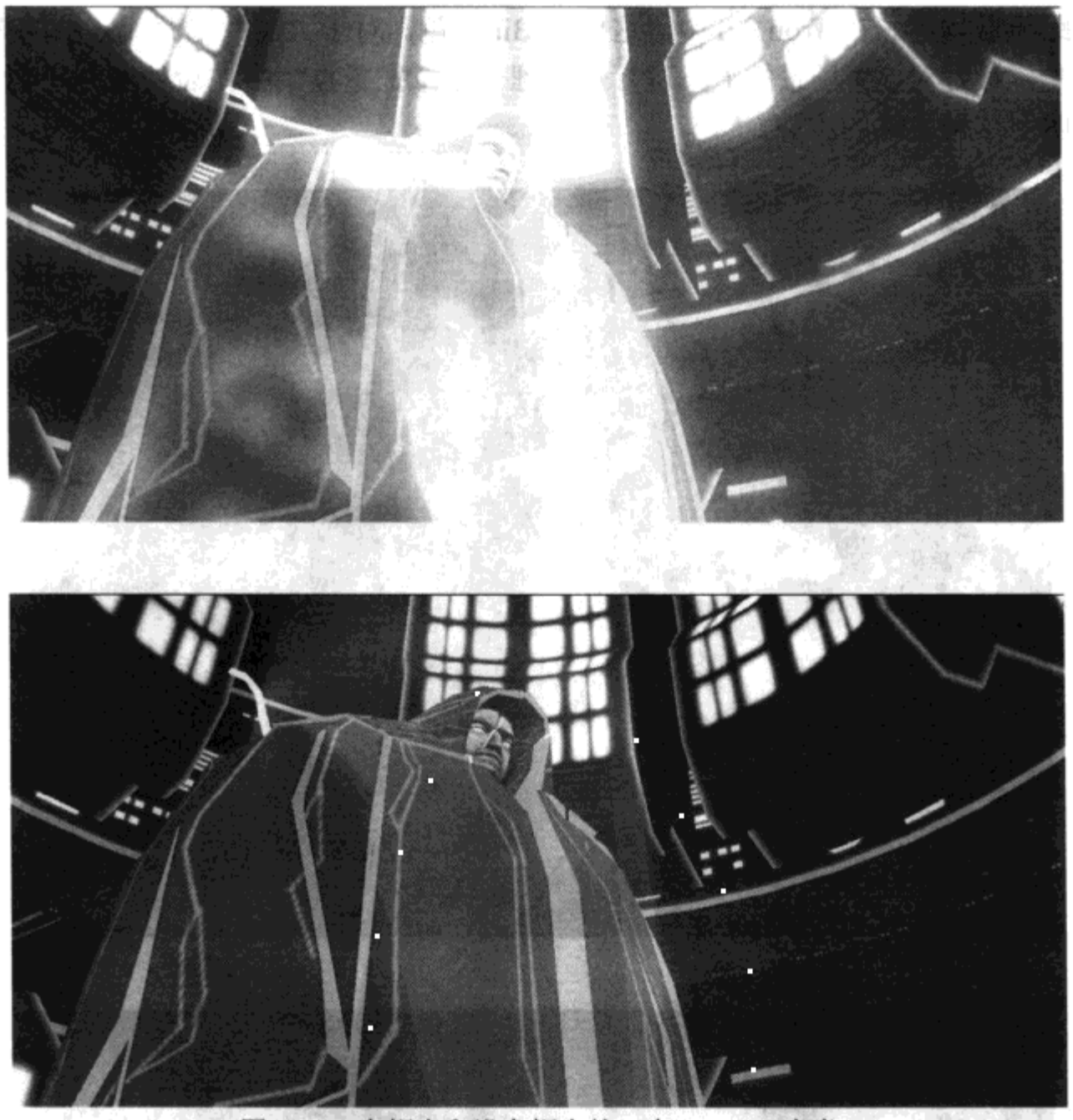


图 21-2 有辉光和没有辉光的一个 Tron 2.0 角色

顶部：辉光能提高角色的重要性；底部：没有辉光的角色魅力较小

图由 Buena Vista Interactive/Disney Productions 提供

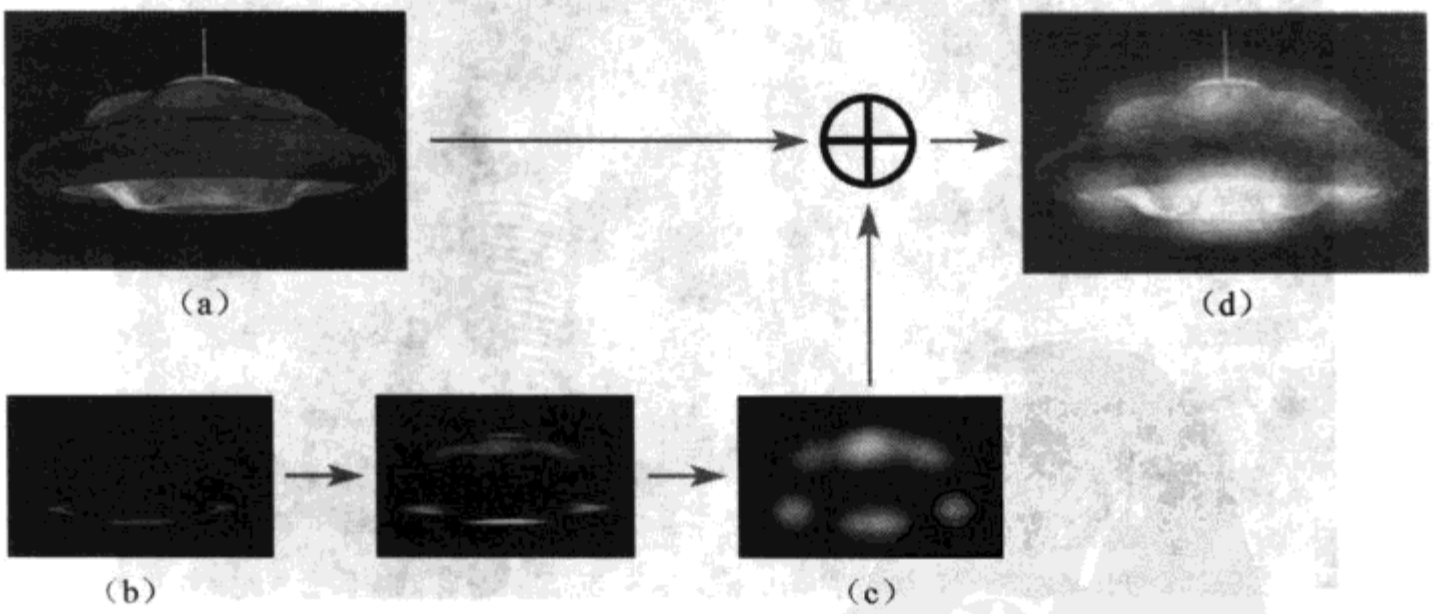


图 21-3 创建实时辉光的渲染步骤

（a）正常地渲染场景，（b）涂抹所渲染的辉光源，以产生（c）中的一个辉光的纹理，它被加到正常的场景上去，以产生（d）中的最终的辉光效果

首先必须通过一些方法，把场景或模型中发光的部分与不发光的部分隔开和分别处理。场景像没有发光一样正常地进行渲染，另外还要使用辉光源信息创建一个纹理映射，上面除了能见到辉光源，其他各处都是黑的。图 21-3b 显示的就是所渲染的纹理映射的例子。这个所渲染的纹理映射，在后面的渲染中可以当作一个平常的纹理使用。对于简单的几何体，在每个像素多次取样，进行一个两步图像卷积运算，这样就把辉光源的点向外涂抹为模糊的、大面积的辉光图案。最后，使用附加的 alpha 混合，把模糊的辉光应用到正常渲染的顶部。这样，使用硬件渲染和纹理映射，就把辉光源展开为令人信服的辉光氛围了。

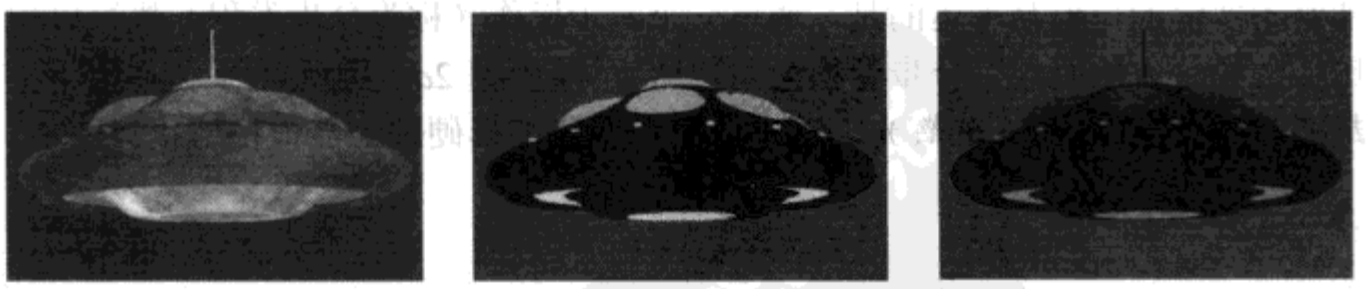
在应用范围很广的图形硬件上，以上每个步骤都能有效而快速地进行。最适用于这个技术的，是支持顶点和像素 shader 1.1 的微软公司的 Direct3D 8 硬件，但是近来对只支持固定函数的 Direct3D 7 的硬件也有一个方便的低效运行办法。旧的 Direct3D 7 硬件的填充率和纹理运行性能比较低，可以减小其目标纹理的分辨率以改善运行性能，这样图像质量不会受很大影响。因为被模糊的辉光纹理通常只包含低频特性，减小它的分辨率几乎不会降低质量。事实上，正如后面要解释的，减小纹理渲染目标的分辨率是一个好方法，它可以创建较大片的辉光，又不增加另外的运行消耗。

21.2　渲染辉光的步骤

21.2.1　辉光源的指定和渲染

渲染辉光的第一个步骤，是指定哪个物体或物体的哪一部分是辉光源。 这些辉光源的颜色和亮度将会直接地转变为最终辉光的颜色和亮度，因此，这意味着我们可以通过改变辉光源的亮度，容易地控制辉光的形貌。这些源可以是由物体特性或标记来指定的整个几何体，也可以通过纹理数据把源限制为一个物体上的某些小的部分。在后一种情况下，纹理数据遮盖了物体上不发光的部分，在辉光源的渲染中把它们转变为黑色。其余的辉光源面积可以是所需要的任何颜色和亮度。使用纹理数据是方便的和对美工友好的，因此它是我们优选的方式。

辉光源掩码可以包含在它自己分开的纹理中，但是使用平常的漫反射彩色纹理的 alpha 通道保持掩码数值也是方便的。在这种情况下，使用纹理颜色 RGB 来正常地渲染物体，不需要 alpha 混合。当渲染辉光源的时候，RGB 颜色要乘以纹理 alpha 通道。在 alpha 值是 0 的地方，没有辉光源，而当 alpha 不断变化直到最大值时，辉光源的强度不断增加。图 21-4 举例说明用纹理 RGB 和 alpha 通道来为一个 UFO 指定辉光源。另外，可以把每个顶点或每个物体的值相乘得到辉光源的值（RGB × alpha），使辉光随着时间而变化。



（a）　（b）　（c）

图 21-4　指定辉光源一个物体纹理的 alpha 通道能用来指定辉光源和辉光源亮度

（a）纹理 RGB（b）纹理 alpha（c）辉光源=RGB*A

一旦指定了辉光源，就需要把它们写入一个离屏的渲染目标纹理，我们可以处理它以创建模糊的辉光。这是用 Direct3D 9 D3DUSAGE_RENDERTARGET 标志所创建的一个纹理。有两个方法可以把辉光源放进纹理渲染目标：（1）把整个的场景渲染进纹理，用一个方法把辉光源渲染为彩色的，所有非辉光的物体渲染为黑色的；（2）使用 IDirect3DDevice9::StretchRect(. .)函数，把场景从平常的背面缓冲区复制到纹理。对于复杂的场景，把整个的 3D 立体场景再渲染为纹理可能耗费很大，而且它需要一个另外的深度缓冲区来专门用于纹理渲染目标，所以 StretchRect(..)的 2D 图像复制方法比较好。StretchRect(..)也允许我们在把场景复制进纹理的同时，缩放和过滤背面缓冲区的图像。在为创建辉光而处理纹理时，可以利用这一点减小纹理分辨率从而提高运行性能。

对于 StretchRect(..)法，可以把 alpha 值作为辉光源的掩码，渲染为平常的背面缓冲区的目的 alpha 值。这不会影响平常的场景，但是在 StretchRect(..)操作中，它将被复制到纹理的 alpha 通道。接着可以用颜色 RGB 乘以这个 alpha 通道，以滤掉场景物体，而只留下辉光源。

在这个步骤之后，就模糊辉光源纹理，以创建辉光的柔和氛围。模糊操作平滑了源纹理的高频性能，或类似点的特性，于是结果只有宽阔的、低频的特性。因此，辉光和辉光源可以渲染为低分辨率纹理，而且这么做不会降低质量。每轴上的分辨率仅为全屏幕分辨率的 1/3 或 1/4，也可以创建辉光，这极大地改善了渲染的速率。

以低分辨率渲染辉光源，确实在最终的辉光上造成走样。随着辉光源纹理的分辨率减小，辉光源纹理的走样会增加，而且当物体在场景中来回运动时，源的纹素更倾向于闪烁。辉光源的单个纹素也许代表高分辨率图像中的几个像素，而且这个单个的纹素会展开成一个大的辉光图案。这就增加了走样的效果，在物体移动时，导致辉光闪烁或晃动。在最终的图像中可以忍受多么严重的闪烁，决定了分辨率能减少到什么程度。当减小分辨率的时候，使用过滤改善质量，能够减少这种闪动。举例来说，当对高分辨率辉光源图像降低取样率时，可以使用硬件加速的双线性纹理过滤，这会大大减少那种闪烁。

21.2.2 模糊辉光源

模糊辉光源，可以将其展开成为一个平滑而又自然的辉光图案。模糊是在硬件中使用一个二维的图像处理滤波器完成的。创建辉光效果的速率主要取决于模糊执行的效率。执行模糊所需的时间则取决于应用滤波的范围（以纹素计）。当增加模糊滤波的范围、覆盖更多的纹素时，我们必须读和写更多的纹素，这与二维的模糊面积成正比。而面积与模糊直径的平方成正比，或 d^2。如果把辉光的直径加倍，则需要处理 4 倍的纹素。对于包含 50 × 50 个纹素的模糊形状，为了所创建辉光的每个像素，我们不得不读 2 500 个纹素！这对制作大面积的辉光是非常不实际的，但是幸运的是，通过一个两步操作（称为分步卷积）执行模糊，可以避免讨厌的直径平方的麻烦。分步卷积把消耗从 d^2 减少到 $2d$，因此对每个像素只读 100 个纹素，却能产生一个 50 × 50 的辉光。这个计算在现代图形硬件上能很快地执行。

21.2.3 分步卷积

设计分步卷积技术为的是减少某种特殊情况的计算，即卷积核心是几个项的乘积，而每项在各个轴上彼此独立。在这种情况下，一个 $n \times m$ 个元素的二维卷积，就能被转化为两个分别的 n 和 m 的一维卷积。这极大地减少了卷积的计算消耗。不再需要在每个点计算并加和 $n \times$

m 个样本，卷积被简化为两步，总共只需要处理 $n + m$ 个样本。第一步，对结果中的每个点沿着一根轴取样并加和 n 个样本，创建一个中间结果图像。第二步，对中间结果的邻近地区沿着另一根轴取 m 个样本，创建最终结果的每个点。n 或 m 样本的每个加权因子是沿着每个轴卷积的轮廓。我们的主要想法是选择一维的卷积轮廓，使得两步法可以实现。即使一个特殊的二维模糊形状不可能在数学上分解，我们也可以使用两个一维的轮廓去近似那个形状。仅通过做两个一维的模糊，我们可以创建许多各种不同的二维模糊形状。

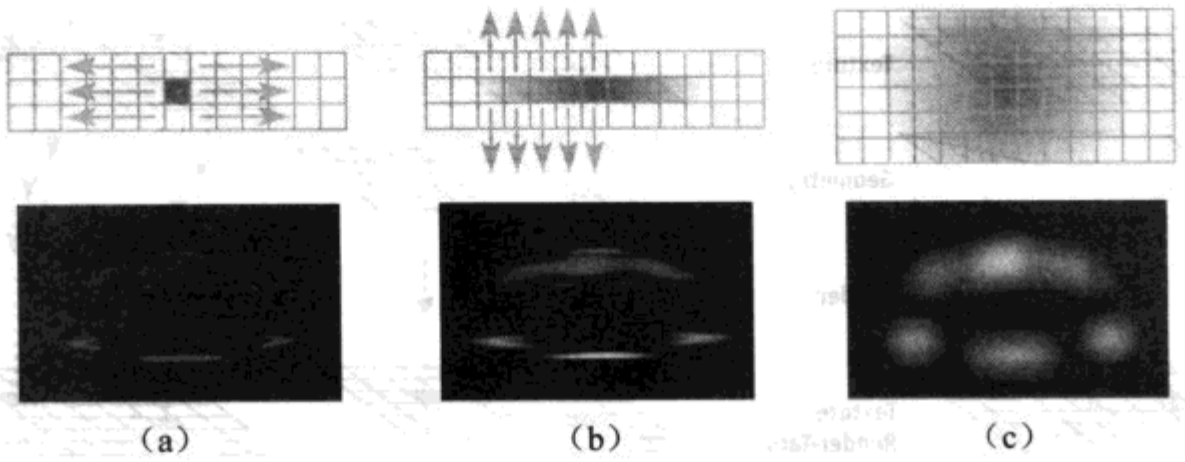


图 21-5　有效地创建模糊的两步分解法

首先，在一根轴上模糊在（a）中的辉光源的点，产生（b）中所示的中间结果，然后在另一根轴上模糊这个结果，产生显示在（c）中的最终模糊

关于分步卷积更深入的信息可以在网站 OpenGL.org 中发现。就我们的目的来看，数学微分式好像可能会限制模糊的形状。这是因为微分式典型地只以 1 个或 2 个可分解的函数为基础，例如二维的 Gaussian 函数。并不是非得进行数学微分，来把一个二维的剖面分解成两个一维的轮廓，我们可以指定任何一对喜欢的轮廓。每个轮廓是什么形状没有关系，只要它们产生一些有趣的 2D 模糊结果。对于显示在这里的图像，我们用两个高斯曲线叠加在一起。一个曲线提供平滑、宽阔的基础，而另一个在中央产生一个明亮的尖峰。我们的 Direct3D“辉光”演示（NVIDIA 2002）使用各种其他轮廓。其中有一个周期的锯齿状轮廓，产生了有趣的像衍射一样的多重图像效果。样本以及完整的源代码包括在书的 CD 和网站内。

21.2.4　GPU 上的卷积

我们在 GPU 上进行渲染到纹理的操作，先在一根轴上模糊，然后在另一根轴上模糊第一个模糊。渲染取出围绕每个像素的纹素的局部相邻地区，而且把卷积核心权数加到这个相邻地区的样本上。如果 GPU 能读出一遍中的所有相邻样本，一个卷积就能在一个单个的渲染遍中执行；不然就使用相加混合，一次累积几个相邻样本，在几个渲染遍上得到结果。

渲染由一个与屏幕平行的简单几何体驱动。几何体是一个简单的矩形，通常包含了全部渲染目标，由两个三角形组成。每个三角形顶点的纹理坐标，就是从源纹理取纹素样本的位置。也可以在一个顶点或像素 shader 中计算坐标。如果把坐标设置在渲染目标的 0.0 到 1.0 范围内，那么就可以把源纹理复制进渲染目标。如果每个被渲染的像素可以从源纹理中它本身位置上读出一个纹素，就可以做一个精确的拷贝。但是有时每个纹理样本的坐标可能彼此偏移 1 个或更多纹素。这时，每个像素将会在源纹理的相邻区域取样。相邻地区的相同图案就可能取作每个像素周围的样本。这个方法在图 21-6 中举例说明，它对在 GPU 上执行卷积

提供了一个方便的方法。关于在 GPU 上相邻取样和图像处理技术的更多信息，能在 James 2001 和 developer.nvidia.com 及 gpgpu.org 上发现。

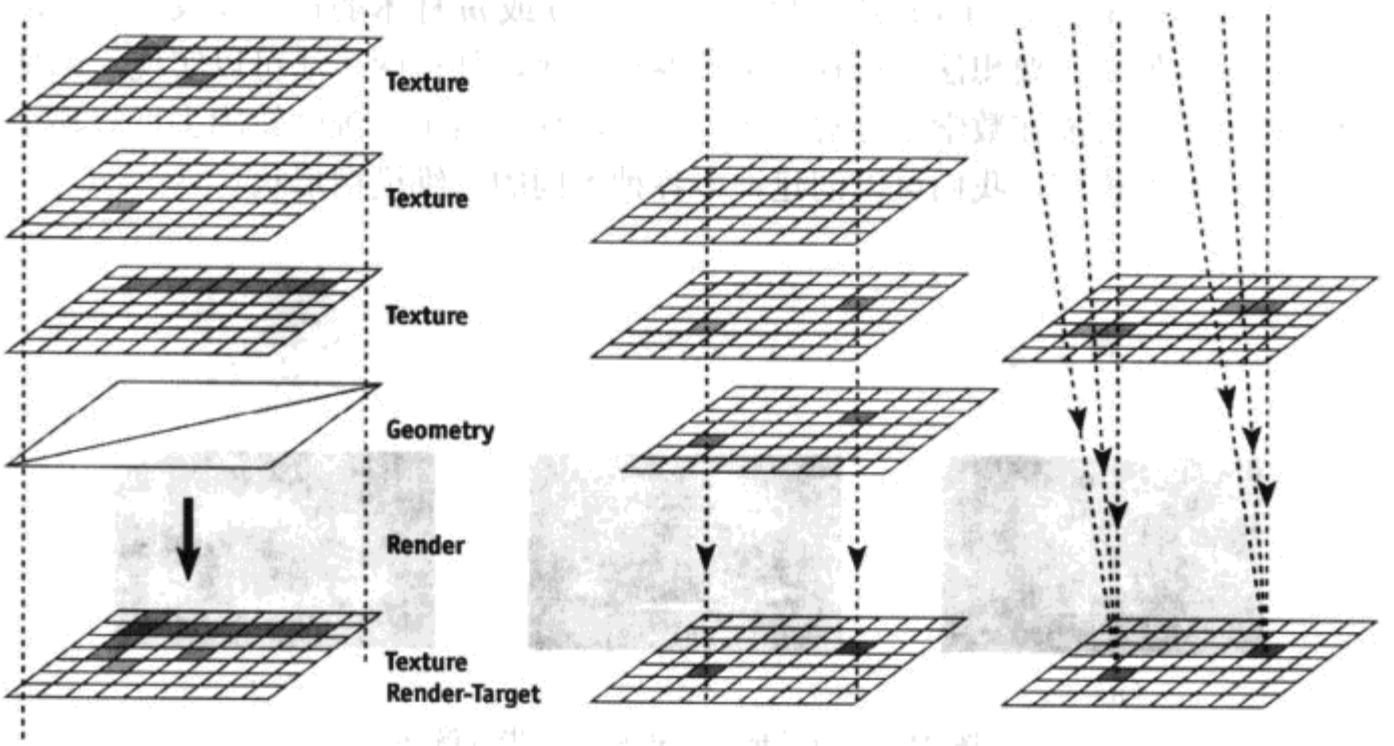


图 21-6 卷积和图像处理

（a）穿过屏幕渲染一个简单的矩形驱动着在每个像素上的纹理的处理；
（b）通过偏移纹理坐标，每个所渲染的像素可以对一个相邻区域取样；
（c）用来自单个纹理源的所有样本显示在（b）中相同的相邻取样图案

为了在图形处理器上执行模糊卷积运算，把辉光源纹理约束于 1 个或更多的纹理取样单位，而且计算纹理坐标以提供需要的相邻取样区域图案。把渲染目标设定成能够沿着第一个轴模糊的纹理。这个纹理称为水平模糊纹理。当渲染每个像素时，将几个纹理样本（相邻样本）传送到像素片元处理的硬件上，在那里它们与第一个一维卷积核心的加权因子相乘。一旦使用一个或多个渲染到纹理的遍，渲染了水平模糊之后，渲染目标就转变成另一个纹理渲染目标，它容纳了最终的模糊。水平模糊纹理与输入纹理取样约束在一起，纹理坐标及像素 shader 权重加在第二个一维卷积核心上（垂直模糊）。

在最后的模糊操作之后，辉光就准备好要混合进场景了。渲染目标又转回到平常的背面缓冲区，用附加 alpha 混合渲染一个简单的矩形，把辉光纹理加到场景上去。

21.3 特定硬件的实现

21.3.1 Direct3D 9

使用具有 ps.2.0 的 Direct3D 9 硬件，能在一个复杂的单个像素 shader 遍中读出和卷积所有的相邻样本。在一个顶点 shader 程序中可以计算相邻取样的纹理坐标偏移，但是 vs.2.0 和 ps.2.0 类型的硬件只能同时处理 8 个纹理坐标。另外的纹理坐标可以在像素 shader 中被计算，但是这并不一定比多遍方式更快（每遍只能同时处理 8 个纹理坐标）。在列表 21-1 和 21-2 中显示 shaders vs.2.0 和 ps.2.0 的例子。顶点 shader 设计成可接受简单的、覆盖整个屏幕的几何

体，它的顶点坐标在相似裁剪空间内（屏幕空间），坐标范围从$(x, y) = ([-1, 1], [-1, 1])$到覆盖整个屏幕。这些 shader 用于水平和垂直两个方向的模糊步骤，两者的差别只在于输入的常量不同。这些常量指定了相邻样本的安排和核心加权值。

列表 21-1　为 8 个相邻样本取样设定纹理坐标的 Direct3D 顶点 shader

```
vs.2.0
dcl_position     v0
dcl_normal       v1
dcl_color        v2
dcl_texcoord     v3

mov oPos, v0 // 输出屏幕空间中的顶点位置

// 通过偏移产生相邻取样的纹理坐标
// 考虑到几个常量的单个输入纹理
add oT0, v3, c0
add oT1, v3, c1
add oT2, v3, c2
add oT3, v3, c3
add oT4, v3, c4
add oT5, v3, c5
add oT6, v3, c6
add oT7, v3, c7
```

列表 21-2　用于加和 8 个加权纹理样本的 Direct3D 像素 shader

```
ps.2.0
// 取 8 个相邻样本，对它们应用 8 个卷积核心加权数
dcl t0.xyzw        // 声明纹理坐标
dcl t1.xyzw
dcl t2.xyzw
dcl t3.xyzw
dcl t4.xyzw
dcl t5.xyzw
dcl t6.xyzw
dcl t7.xyzw
dcl_2d s0          // 声明纹理取样
// 常量 c0..c7 是每个相邻样本对应的卷积核心加权
// to each neighbor sample.
texld r0, t0, s0
texld r1, t1, s0
mul   r0, r0, c0
mad   r0, r1, c1, r0
texld r1, t2, s0
texld r2, t3, s0
mad   r0, r1, c2, r0
mad   r0, r2, c3, r0
texld r1, t4, s0
texld r2, t5, s0
```

```
mad   r0, r1, c4, r0
mad   r0, r2, c5, r0
texld r1, t6, s0
texld r2, t7, s0
mad   r0, r1, c6, r0
mad   r0, r2, c7, r0
mov  oC0, r0
```

注意，为了能精确地在纹素中心取样，一定要给纹理坐标增加半个纹素的偏移量。使用 Direct3D 时一定要这样做，而对 OpenGL 就不必，因为 Direct3D 约定坐标从纹素的角上开始，而 OpenGL 约定从纹素的中心开始。这个简单的调节是为了适应实际的需要。对于列表 21-1 中的顶点 shader，从 *c*0 到 *c*7 的每个常量都加上半个纹素的偏移量。这一点应该在 CPU 上做。

21.3.2 Direct3D 8

对于支持 Direct3D 8 顶点和像素 shader 的硬件，限制每遍只取 4 个相邻样本。虽然由于这个限制，建立一个给定尺寸的卷积将要求较多的渲染遍，但是每遍能非常快地执行，渲染含有几十万个纹素（纹理尺寸从 256 × 256 到 512 × 512）的目标纹理，典型的速率为每秒几百遍。样例 shader vs.1.1 and ps.1.3 在列表 21-3 和 21-4 中显示。

列表 21-3 建立相邻取样的 Direct3D 顶点 shader 程序

```
vs.1.1
dcl_position v0
dcl_texcoord v3
mov oPos, v0 //输出屏幕空间中的顶点位置
// 通过偏移产生相邻样本纹理坐标
// 考虑到几个常量的单个输入纹理坐标
add oT0,  v3, c0
add oT1,  v3, c1
add oT2,  v3, c2
add oT3,  v3, c3
```

列表 21-4 建立相邻取样的 Direct3D 顶点 shader 程序

```
ps.1.3
tex t0     // 取 4 个相邻样本
tex t1
tex t2
tex t3
// 每个样本乘以卷积核心加权数并且输出加和
mul r0, t0, c0
mad r0, t1, c1, r0
mad r0, t2, c2, r0
mad r0, t3, c3, r0
```

21.3.3 Direct3D 7

Direct3D 7 是经典硬件，缺乏现代图形硬件方便的顶点和像素着色能力。它典型地被限

制为每遍只有两个纹理样本，而且填充率非常低。但是用了几个覆盖全屏幕的几何体的重叠三角形，卷积模糊还是可以执行的。把每对三角形安排成一个全屏幕的方块，它们有相同的顶点位置，但是不同的顶点纹理坐标。对于两个纹理的多取样硬件，每个方块携带两个纹理坐标，把每个坐标设定为针对样本的不同邻居。每个方块的顶点颜色属性设定为特殊相邻取样位置的核心加权值，而且调用固定函数 SetTextureStageState(..)API，将这个顶点颜色乘以纹理样本的值。调用 DrawPrimitive(..)，相加混合可以渲染一堆这样的方块，以建立卷积结果。

21.4　模糊的其他用途

我们已经讨论了模拟高动态强度值的感觉，此外，这个卷积和模糊的方法还可以用于多种其他效果。它能用来计算景深效果的不同聚焦度，景深的信息可用来控制模糊度。它也能用来模糊投映的纹理阴影的边缘，并且累积深度阴影映射的接近百分比过滤（percentage-closer filtering）结果。大面积的卷积能被应用于一个环境映射，以创建一个近似的辐照度映射，从而得到更逼真的场景照明（Ramamoorthi 和 Hanrahan 2001）。用大面积的卷积也可以实现许多非真实渲染技术和其他的特别效果。其中包括镀着霜的玻璃、模拟衍射的透镜摇曳，以及渲染皮肤时用的近似次表面散射。

21.5　把效果加入一个游戏引擎

下边，我们重点看看在 Buena Vista Interactive 的游戏 Tron 2.0 中如何使用视觉效果，他们在游戏的独特视觉风格中起着重要的作用。这些效果的实现简单得令人惊奇。然而，在这些效果用于游戏的各种效果之前，一些重要的问题必须谈一下。另外在开发中还发现了辉光的一些有趣效果和其他用途。

21.5.1　渲染场景

我们要谈的第一个问题是如何渲染源图像，需要把它模糊而产生最终的屏幕辉光。在 Tron 2.0 中，有两个渲染传递途径：一个是渲染世界几何体，一个是渲染模型。两者都需要扩展以支持渲染场景的概念，就是允许为不同的目的以不同的方式渲染场景。对 Tron 2.0 需要两个渲染场景。一个是正常的渲染，用来处理无屏幕辉光场景的原始渲染。第二个渲染场景是辉光渲染，当为辉光渲染源图像时要用到它。

渲染场景的实现在应用中是多种多样的，但是为了说明起见，我们描述在 Tron 2.0 中如何创建模型渲染的管道。模型渲染以一个网格和一个渲染状态的集合为基础，由渲染状态指出应该如何渲染模型。把这些渲染状态存储为一个外部资源，称为渲染风格。为了支持场景的思路，把这个系统扩展，允许美工建立一个表，用这个表把渲染风格映射为描绘辉光时所对应的渲染风格。按照这个机理，当正常地渲染场景的时候，使用原始的渲染风格，即按照禁用辉光时所见到的那样去渲染模型。然后在渲染发光的源图像时，又改为使用美工已经建立的映射（表）中的风格，去渲染模型的辉光部分。美工会发现这个系统是非常灵活和容易使用的，他们能够控制哪个物体应该发光，以及每个物体应该如何发光。图 21-7 显示简单的角色模型，由美工创建的辉光纹理信息，以及应用到模型上的辉光效果。

（a） （b） （c）

图 21-7 有辉光的和无辉光的 Tron 2.0 的英雄 Jet

（a）用标准方法渲染的 3D 模型；（b）一个由美工创建的辉光源纹理的渲染，目的是指定辉光面积的图案和强度；（c）把辉光应用到平常的渲染表达了英雄的力量和强度

图由 Buena Vista Interactive/Disney Productions 提供

21.5.2 走样问题

当把辉光效果从简单的试验扩展到整个游戏场景时，走样问题立刻出现了。Tron 世界的特点是在场景中明亮的彩色几何条纹，几乎像有一层霓虹灯的边包在角色和它们的周围。再加上大部分环境是巨大的，因此这些几何辉光条纹分布很广，从离摄像机非常近的地方到很远处都有。我们对 Tron 2.0 使用一个 256 × 256 的辉光纹理，以最小数目的通道进行相当大面积的模糊。这个方法的缺点是渲染源图像时引起了严重的走样，而在玩家运动和四处察看时，走样变得非常明显。

为了解决这个问题，我们添加了受美工控制的雾，只有在渲染屏幕辉光的源图像时才使用它。雾可以在很大的范围上启用，可以一直设置到黑暗。因为屏幕辉光的叠加混合，源图像中的黑色雾使得远处物体只对最终的屏幕辉光有微弱贡献（或者完全没有）。因此，在远处的走样很少引起人的注意。

这并不是一个最好的解决办法，例如，在远处的场景部分就不能发光了，但是比起走样的失真现象来，在视觉上它总是更可接受一点。除了避免走样之外，还可以在整个游戏的几个地方，以不曾预料到的方式使用雾来创建额外的气氛，像在图 21-8 中所显示的那样。

21.5.3 DirectX 7 的精度问题

为了把混合加权值加到一个颜色分量上，最初的 DirectX 7 实现把它从最初的[0, 1]范围放大到整数 0 和 255 之间。但是，大部分加权值都非常小，因此它不能很好地分布在整个范围上。结果，转换成整数时所发生的误差，就在 DirectX 7 和 8 的实现之间引起一个实质上的视觉差别。甚至在四舍五入时，辉光还是表现出明显的差别。

如何解决这个问题呢？执行四舍五入，但是首先要确定产生的值与真实的值之间有多大差别，然后把差值应用到下一个加权值的计算中去。例如，一个 0.13 的加权值，当放大 255 倍时是 33.15，四舍五入为整数 33。因此，用于颜色的加权值是 33，而剩余的小数部分（0.15）加到下一个加权值去渲染。结果，DirectX 7 和 8 的实现在视觉上就几乎相同了。

图 21-8　与屏幕辉光一起使用雾
在这个电影式的倒叙中，美工用了微弱的、白色的、屏幕辉光的雾，使场景显得更加朦胧
图由 Buena Vista Interactive/Disney Productions 提供

21.5.4　残留图像效应

有一个效果因为它被发现得太晚，没有应用到 Tron 2.0 中去。就是可以把前面帧的源纹理加到当前帧的源纹理上，在玩家运动和四处察看时，造成一个非常好的辉光拖尾。技术实现起来非常简单。它由两个方面构成，一个是额外的纹理表面，用于为后面的帧保护源图像，另一个是叠加混合，用于把前面的源图像加入当前的源图像。这个技术的做法是在模糊一个图像之前先马上存储这个图像，然后就模糊它。把所存储的图像加到下一帧的源图像上去，但是使它稍微地暗淡一些。当这个过程在若干帧上重复后，就使物体淡出辉光，而不是马上地消失，这不但使效应变得更令人信服，而且能够帮助隐藏源图像的走样。

在把前面帧加入当前帧之前，可以动态地改变前面帧的暗淡度，制造出很多的变化现象。举例来说，使前面帧严重地暗淡，残留图像将会非常小，而且它给使用者的视网膜留下光的短暂的不可磨灭的印象，好像看到一个非常明亮的光源。而减小对前面帧的暗淡，那么模糊和条纹就变得更加明显，给玩家的效果是喝醉了或近乎死亡。

另一个方法并不需要附加的缓冲区。因为不需要保留源图像，我们可以使用前面帧的模糊后的最终屏幕辉光纹理，这样就节省了存储器。然而，这个办法也有某种问题。如果前面帧的辉光纹理太明亮，它可能会失去控制，由于在每一帧中回归地加大、不断地模糊，最后就可能造成一个完全白屏的辉光。为了避免这个情况，在设定混合加权值和决定把前面帧变暗到什么程度时，我们必须很小心。

21.5.5　渐变效果

为了创建一个单个帧的最终模糊，屏幕辉光一定要使用一组特定参数，例如混合加权值

和雾。然而，这并不意味着参数不能从一帧到另一帧发生变化。事实上，改变这些参数之中的任何一个，都能得到多种有趣的效果。举例来说，在 Tron 2.0 的一关中，玩家处于在一个破坏了的服务器里（corrupted server），它发出病态的绿光。为了传递不稳定的感觉，模糊的混合加权值从低值渐变到高值，创建一种缓钝脉动的屏幕辉光，见图 21-9。

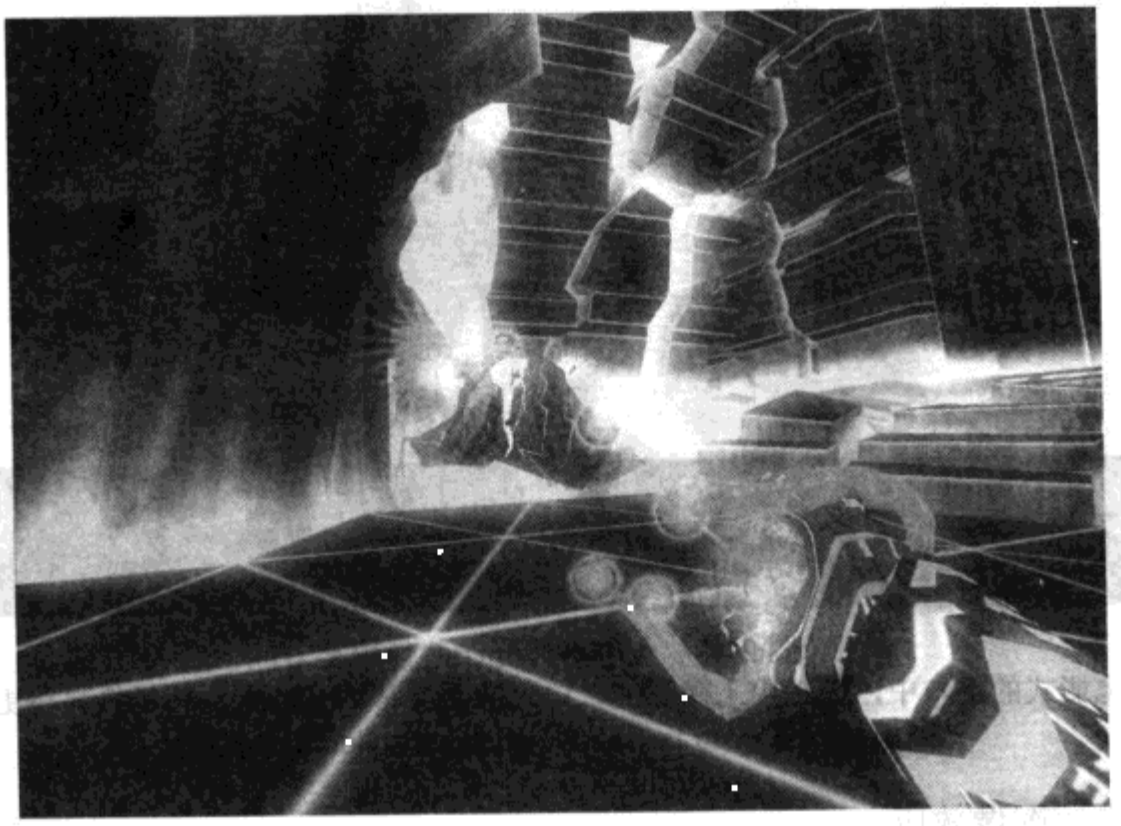


图 21-9 一个脉动的屏幕辉光产生一个病态的感觉
图由 Buena Vista Interactive/Disney Productions 提供

还有一些其他的效果，虽然尚未用在游戏中，也很容易得到。举例来说，通过浓浓淡淡地渐变屏幕辉光的雾，或改变雾的颜色，可以容易而有效地制造一个让人感觉到正在不断变化的氛围。

21.6 小结

大片的模糊和卷积能有效地在多种图形硬件上实时地计算。处理和创建这些效果的代码可以容易地封装成几个 C++类或一个小库。这些类和样本代码可以在 NVIDIA 的开发者网站 developer.nvidia.com 上得到。效果需要另外的数据来指定辉光源的亮度，而且这个数据能与现有的纹理资源合并，即纹理 alpha 通道。效果对辉光的亮度和形状提供直觉的控制。对各式各样的效果而言，能够把大面积卷积应用到全屏幕渲染是有用的。它是描述场景中明亮物体的关键，而且能大大提高实时渲染的外观和质量。

屏幕辉光是一种宝贵的效果，它能够容易地扩展到几乎每一种情形，并且它变化多端，通过它还能够创建很多另外的效果。最终的效果虽然细微但是有力，值得在任何游戏中采用。

21.7 参考文献

Chiu, K., M. Herf, P. Shirley, S. Swamy, C. Wang, and K. Zimmerman. 1993. “Spatially

Nonuniform Scaling Functions for High Contrast Images." *Graphics Interface '93*, pp. 245～253.

GPGPU web site. 2003. **http://www.gpgpu.org**

James, Greg. 2001. "Operations for Hardware-Accelerated Procedural Texture Animation." In *Game Programming Gems 2*, edited by Mark DeLoura, pp. 497～509. Charles River Media.

Kawase, Masaki. 2003. Personal web site

http://www.daionet.gr. jp/~masa/column/2003-03-21 .html

Nakamae, Eihachiro, Kazufumi Kaneda, Takashi Okamoto, and Tomoyuki Nishita. 1990. "A Lighting Model Aiming at Drive Simulators." In *Proceedings of SIGGRAPH 90*, pp. 395～404.

NVIDIA Deve loper web site. 2003. **http://developer.nvidia.com**

OpenGL.org Web site. 2003. "Advanced Graphics Programming Techniques Using OpenGL." Course notes, SIGGRAPH 99. Mathematics of separable convolution.

http://www.opengl.org/resources/tutorials/sig99/advanced99/notes/node235.html

Ramamoorthi, Ravi, and Pat Hanrahan. 2001. "An Efficient Representation for Irradiance Environment Maps." In *Proceedings of SIGGRAPH 2001*, pp. 497～500. Available online at **http://graphics.stanford.edu/papers/envmap**

Spencer, Greg, Peter S. Shirley, Kurt Zimmerman, and Donald P. Greenberg. 1995. "Physically Based Glare Effects for Digital Images." In *Proceedings of SIGGRAPH 95*, pp.325～334.

我们对 NVIDIA 的技术开发组和 Monolith Productions 表示极大的谢意，他们鼓励和协助发展了这项技术，并且把一个简单的技术演示，转化为最终商品游戏中华丽的视觉效果。

第 22 章 颜色控制

Kevin Bjorke
NVIDIA

几乎所有的印刷品和电影图像都要用到彩色校正。彩色校正可以把彩色图像从一个彩色空间转移到另一个彩色空间（比如说，从 Adobe RGB 到 sRGB）；把图像风格化（创建艺术上的幻觉，例如使胶片褪色、交叉处理或其他风格上的变化）；把不同来源的艺术元素结合起来（例如把不同来源的视频，或不同美工所做模型之间的彩色色调予以匹配）；或不改变基本的模型和纹理，就给游戏或场景整体增加一致性和氛围。

我们在电视、杂志和电影里见到的大部分图像，都经过非常小心的彩色校正和控制。对这个过程的理解，可以帮助开发者在实时应用程序中得到同样华美的视觉效果。

22.1 引言

在所有的彩色校正形式中，我们最想要的是改变个体像素的颜色。彩色校正通常有两种做法：一是各个通道的校正，分别地改变红色、绿色和蓝色各成分；二是混色操作，基于红、绿、蓝各成分的同时操作，得到每个通道的输出值。

彩色校正的机理可以简洁而容易地在一个 shader 中描述。重要的是，美工和程序员使用的普通工具就能有效地控制他们。在这章中，我们依靠 Adobe Photoshop 图像处理软件创建控制资源，然后通过像素 shader 应用到实时程序中。

22.2 基于通道的颜色校正

Photoshop 提供一些基于通道的校正工具。例如级别（Levels）和曲线（Curves）工具的特点都是基于通道的。他们既可以在一个图像中分别改变各个通道的强度，又可以将所有 3 个通道当作一个整体而改变。

22.2.1 级别

图 22-1 和 22-2 显示在 Photoshop 中典型级别指令的应用（在这种情况下，是提高对比度和改变整体亮度）。美工能从总体上控制整个图像的对比

度、亮度和变化范围，或独立地操纵每个颜色通道的特性。

对于每个通道，我们能应用下面的公式：

```
outPixel = (pow(((inPixel * 255.0) - inBlack) / (inWhite - inBlack),
                inGamma) * (outWhite - outBlack) + outBlack) / 255.0;
```

这里，inBlack、inGamma 和 inWhite 是“输入级别”值，而 outBlack 和 outWhite 是在图 22-1 中“输出级别”方框里的值（有关处理亮度值更多的信息，见第 26 章“OpenEXR 图像文件格式”）。

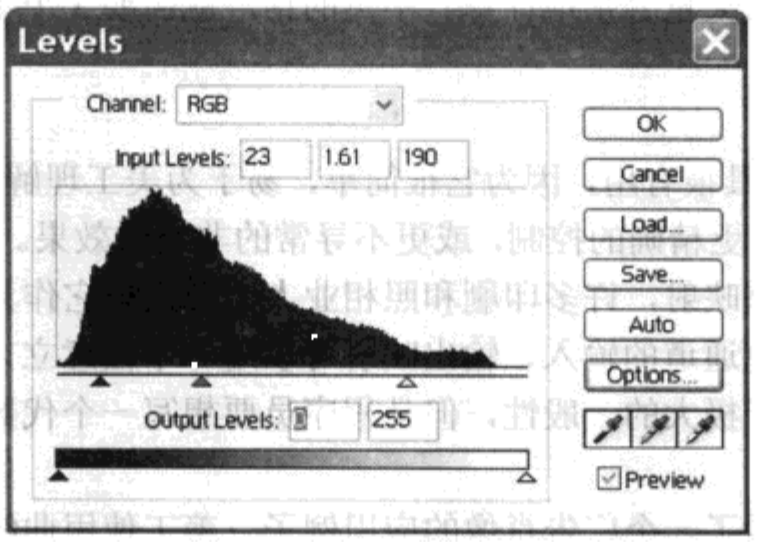


图 22-1　使用 Photoshop 的级别，调节图像的亮度和全部动态范围

图 22-2 显示对 3 个彩色通道应用同样的调节，但是我们也可以分别地调节每个通道。

（a）　　（b）

图 22-2　级别调节之前和之后

（a）原始图；（b）调节级别之后

对于使用高级着色语言的产品流水线，美工或艺术总监可以定义彩色校正，做法是：首先从 Photoshop 中打开游戏的一个截图或其他图像源，然后把校正作为 Photoshop 的新层。shader 的作者接着可以打开 Photoshop 中的那个图像，从级别对话框复制校正值，再把它们作为输入值粘贴入彩色校正代码。

在实际中，时常可以简化公式。在 Photoshop 中先做除法（除以 255），再传输到 shader，这样就可以省去做强度规范化了。默认级别值（例如输出值 0～255，或者 1.0 的亮度）可以简单地跳过去。

如果把级别应用到个别通道，输入值可以定义为矢量数据，但是除非亮度值对所有的通道都匹配，结果只能作为矢量分别地计算。不然的话，就需要 3 个不同的 pow()函数。

22.2.2 曲线

Photoshop 的级别工具很有用，因为它很简单，易于为美工理解和让 shader 程序员实现。然而，有时美工可能需要更精确的控制，或更不寻常的非线性效果。Photoshop 的曲线工具对彩色通道提供更任意的重映射，许多印刷和照相业人士都选择它作为彩色校正工具。

使用曲线工具，彩色通道的输入、输出映射可以用一个任意立方曲线来定义，或者徒手画出。这个灵活性提供了极大的一般性，但是程序员要想写一个代数表达式来定义许多可能的关系很难。

图 22-3 和 22-4 显示了一个广告肖像的应用例子。美工使用曲线对话框中一系列复杂的彩色曲线处理 RGB 图像。在这种情况下，通过两种途径应用曲线：除了同样地把整体曲线应用于 RGB，还通过 Photoshop 分别地应用红、绿和蓝色曲线。

结果仿制了化学的交叉处理（cross-processing）外观，确切地说，就是在 C41 化合物中处理 E6 胶卷所产生的假颜色外观。这样的处理多年来在印刷、电影和电视中是司空见惯的。

虽然有可能由 Photoshop 执行算法得到结果，但是更容易的方法，是使用一个处理复杂通道的 shader。由于每个彩色通道的输入值与最终输出之间存在着一对一的对应，我们可以把这些映射表达为一个“标示”映射，把它当作一个“相关的”纹理来应用。

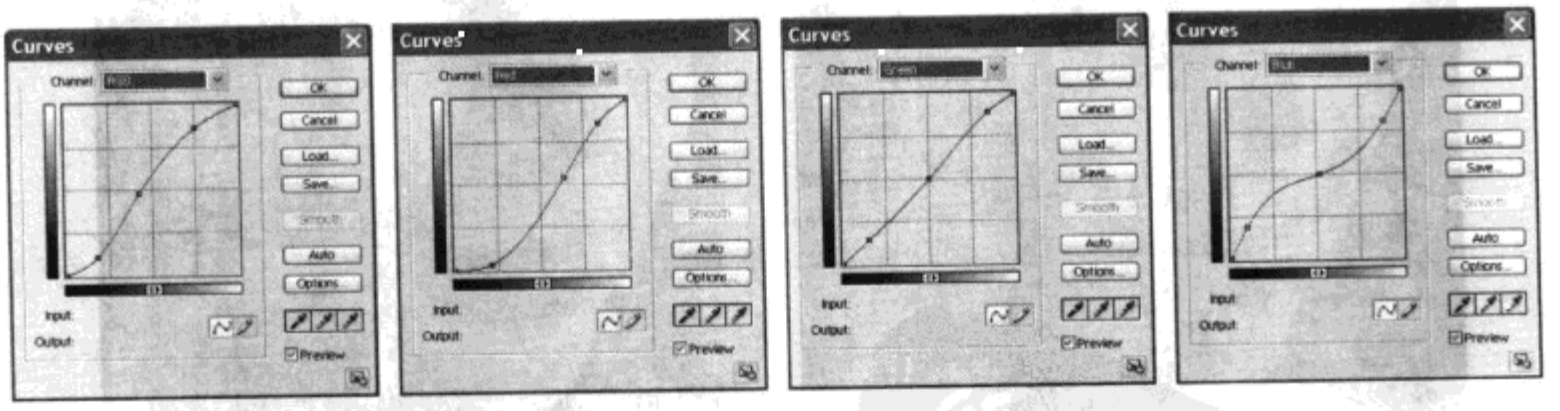
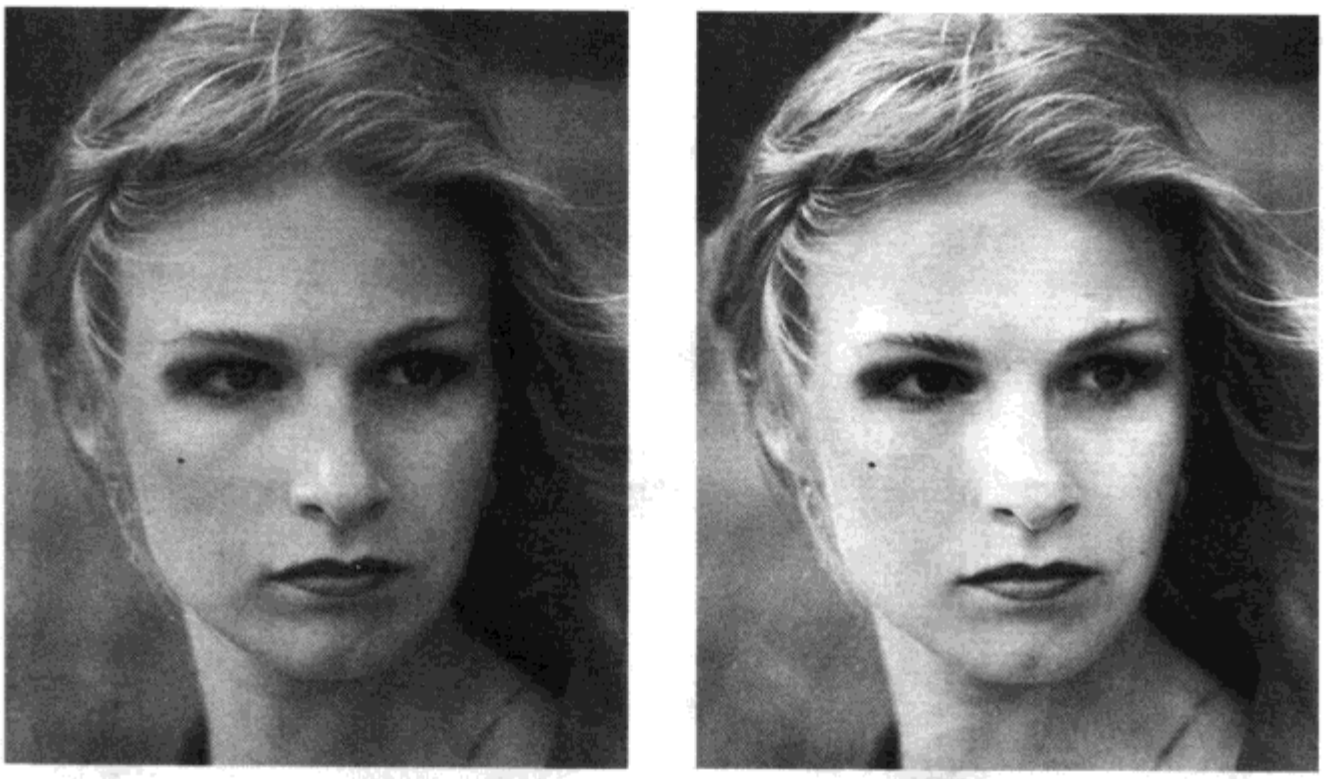


图 22-3 重新创建交叉处理效果的 Photoshop 曲线

为了最好的结果，应该定义一个 1 × 256 的纹理映射（虽然更小的映射用起来时常更有效）。在创建映射之前，美工必须首先定义彩色变换，典型地是使用 Photoshop 的曲线工具，把调节层应用到已存在的静止图像中。调节一经定义后，他们就能以 Adobe .acv 曲线文件存到磁盘中。



（a）　　（b）

图 22-4　伪交叉处理

（a）原始图；（b）曲线调节之后

如下为制作纹理的过程。

1. 在 Photoshop 中创建新的 1 × 256 个像素的 RGB 图像。

2. 从白到黑，分别地设定前景和背景颜色。

3. 使用 Gradient（梯度）工具，从图像中最左边的像素到最右面的像素应用梯度渐变，按左边黑到右边白排列（存储这个梯度作为参考：以后可能会需要它）。

4. 现在把存档的.acv 曲线文件应用到梯度文件。如果对每个彩色通道应用了不同的调节，那先前灰色的梯度现在就显示彩色带了。图 22-5 所示为该梯度的例子。

5. 以适合于程序中纹理映射的形式把新的彩色梯度图像存档（例如 DDS 或 Targa 格式）。

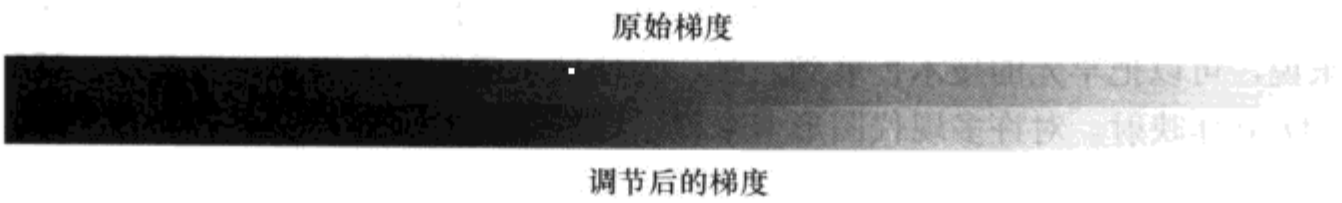


图 22-5　RGB 灰度范围和调节后的 RGB 梯度

我们现在可以把下面几行 shader 代码应用于输入颜色，使用这个校正纹理映射，能随意地用曲线工具重新创建任何彩色变化（如果需要，同样的方法也能应用于级别调节）。

```
float3 InColor = tex2D(inSampler, IN.UV).xyz;
float3 OutColor;
OutColor.r = tex1D(ColorCorrMap, InColor.r).r;
OutColor.g = tex1D(ColorCorrMap, InColor.g).g;
OutColor.b = tex1D(ColorCorrMap, InColor.b).b;
```

换句话说，使用每个原始的红、绿和蓝像素的灰度值，确定在梯度纹理中我们寻找的相关位置；然后梯度纹理本身定义对新颜色的重映射，即由复杂曲线调节所定义的新颜色，见

图 22-6。

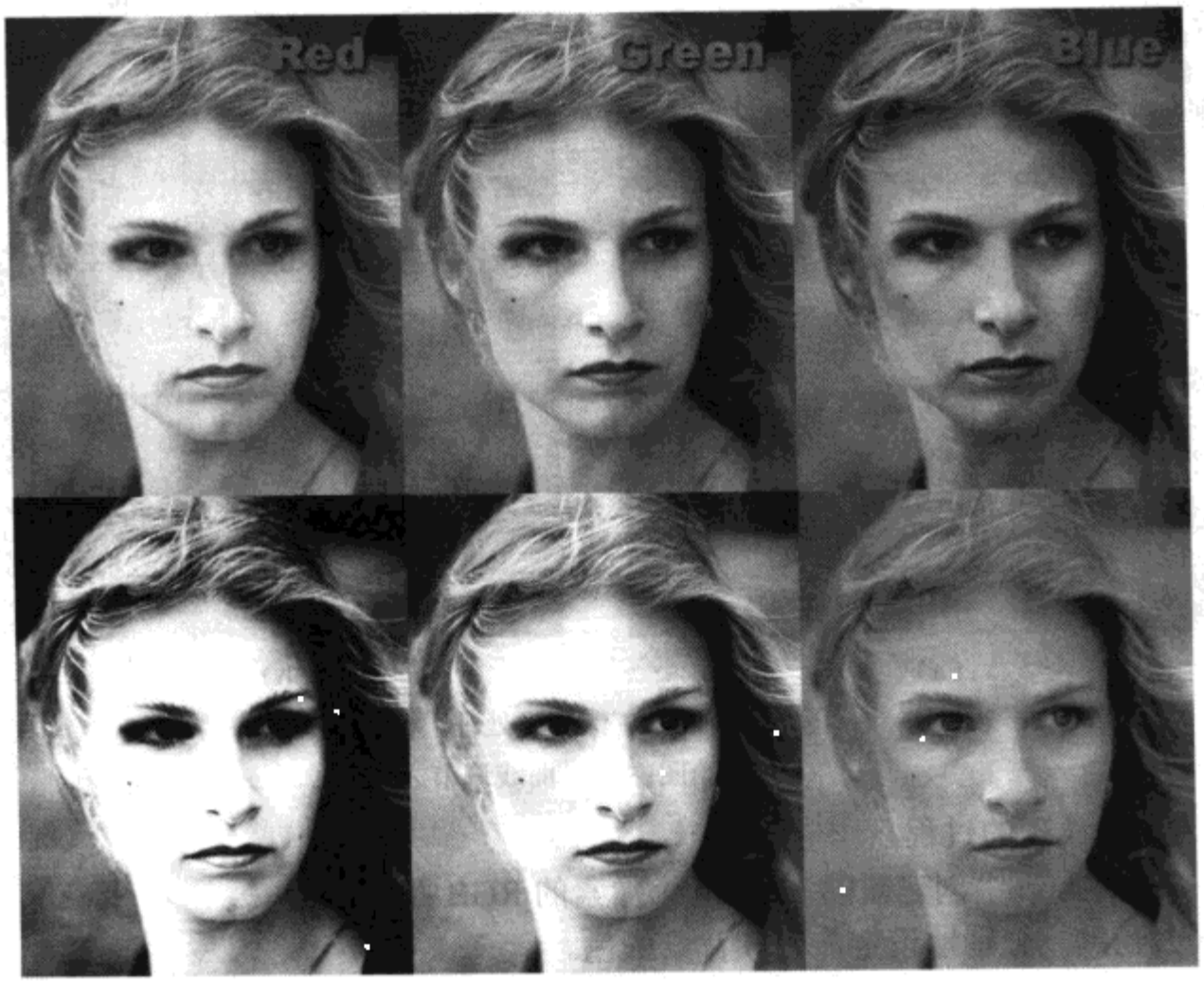


图 22-6 通道对通道的红、绿和蓝重映射结果

22.3 多通道的彩色校正和变换

有的时候，我们需要把各彩色通道混合在一起，例如调节色调（即，经过 3D 立体颜色空间的转动）从一个彩色空间转换到另一个时，或从彩色转换到灰度的时候。

一般来说，可以把早先的技术扩展到三维，为任何可能的像素提供一个 256 × 256 × 256 的完整的 3D 立体映射。对许多现代图形卡来说，这样大小的映射要求很多的存储器，而代码却极其少：

```
float3 InColor = tex2D(inSampler, IN.UV).xyz;
float3 OutColor = tex3D(colorSpaceSampler, inColor);
```

可以看出，代码很简单。实际上惟一的限制是缺乏通用工具，用来创建 RGB 到 RGB 的 3D 立体纹理映射。

幸运的是，大多数颜色空间的变换相当一致，而且可以有效地表达为点积和 3 × 3 矩阵的乘法。

22.3.1 灰度变换

从 RGB 颜色到灰度的变换通常可以用以下几个方法：可以选择一个彩色通道，或均匀地混合所有的 3 个通道，或通过改变加权值混合 3 个 RGB 通道，从而得到最后的灰度结果。

通常认为通过加权混合的选择是最好的，它可以与人类视力典型的灵敏度相匹配。

我们可以用一个点积表示这个混合：

```
float grayscale = dot(float3(0.222, 0.707, 0.071), inColor);
```

数值（0.222, 0.707, 0.071）分别表示红、绿和蓝的相对数量。这些数值遵循一个被称为 ITU Rec 709 的国际工业颜色标准（实际上有若干种标准）。注意这个变换所用的 float3 矢量成分的总和是 1.0，这使得这个变换名义上“保持能量”，尽管事实上允许以任何的数赋值而得到不同的有趣结果，并且和可以用 Photoshop Color Mixer 彩色混合工具得到的一样多。

实际上，标准化的变换意味着纯色的亮度可能是有限的，比方说，纯蓝色永远不会比 7% 的灰度更暗。因为艺术的缘故，有时可能要改变灰度变换加权值，使得重要的颜色不受限制。

使用类似上节所用的彩色查询纹理，我们也可以使用灰度变换的结果作为纹理索引，去创建其他可能的彩色贴图（映射）。考虑如下映射：

```
float grayscale = dot(float3(0.222, 0.707, 0.071), inColor);
// 把纹理的边界寻址设为“夹持”
float3 OutColor = tex1D(ColorCorrMap, grayscale);
```

使用灰度到颜色梯度的映射，允许我们创建各式各样的假颜色和印刷色调的效果，既可以是自然的（像是双色网版或三色网版的），又可以是高度程式化的（例如机器人视觉或红外线“热信号”àla 电影 Predator“铁血战士”）。

22.3.2　彩色空间的变换

在不同彩色空间之间的转换，可以计算每个合成颜色通道的点积，换句话说，把输入 RGB 值乘上一个 3 × 3 的矩阵。

```
float3x3 conversionMatrix;

//加一些代码，把数值插进这个矩阵
. . .

float3 newColor = mul(conversionMatrix, inColor);
```

代码显示依照 conversionMatrix 的内容将 inColor 转换成 newColor。许多变换矩阵是标准化的：例如，从 RGB 颜色到 CIE 颜色，从电视的 YIQ 信号到 RGB，或从彩色标准 Adobe RGB 到其他的标准 sRGB。“彩色空间 FAQ”（Bourgin 1994）含有视频的许多标准变换的信息。Autiokari 2003 是一个很好的在线资源，包含很多通用的工业彩色空间（例如用于 Photoshop 的彩色配置文件）的矩阵值。（也可以借助纹理硬件得到二次采样信号的变换，见本书第 24 章“高质量的过滤”）。

用诸如 gonioreflectometer 的工具做基于物理测量的 3D 着色时，自定义的彩色变换有时也是需要的。调节原始传感器的颜色灵敏度和典型计算机显示的输出颜色也需要彩色变换。想要最逼真地重现给定表面原始的、真实的 BRDF，最后的调节可能是至关紧要的。

使用 DCC 工具转动和缩放 3D 彩色立方体很容易。例如，如果用的是 Cg，创建一个空的 3D 节点，使用适当的 Cg 插件，然后把节点的世界空间矩阵附在如下的 Cg shader 中：

```
float4 colorCubePS(vertexOutput IN,
                   sampler2D ColorTex,
                   float3x3 RGBxform) : COLOR
{
  float3 texColor = tex2D(ColorTex, IN.UV);
  float3 result = mul(RGBxform, texColor);
  return float4(result, 1.0);
}
```

结果显示在如图 22-7 中所示。

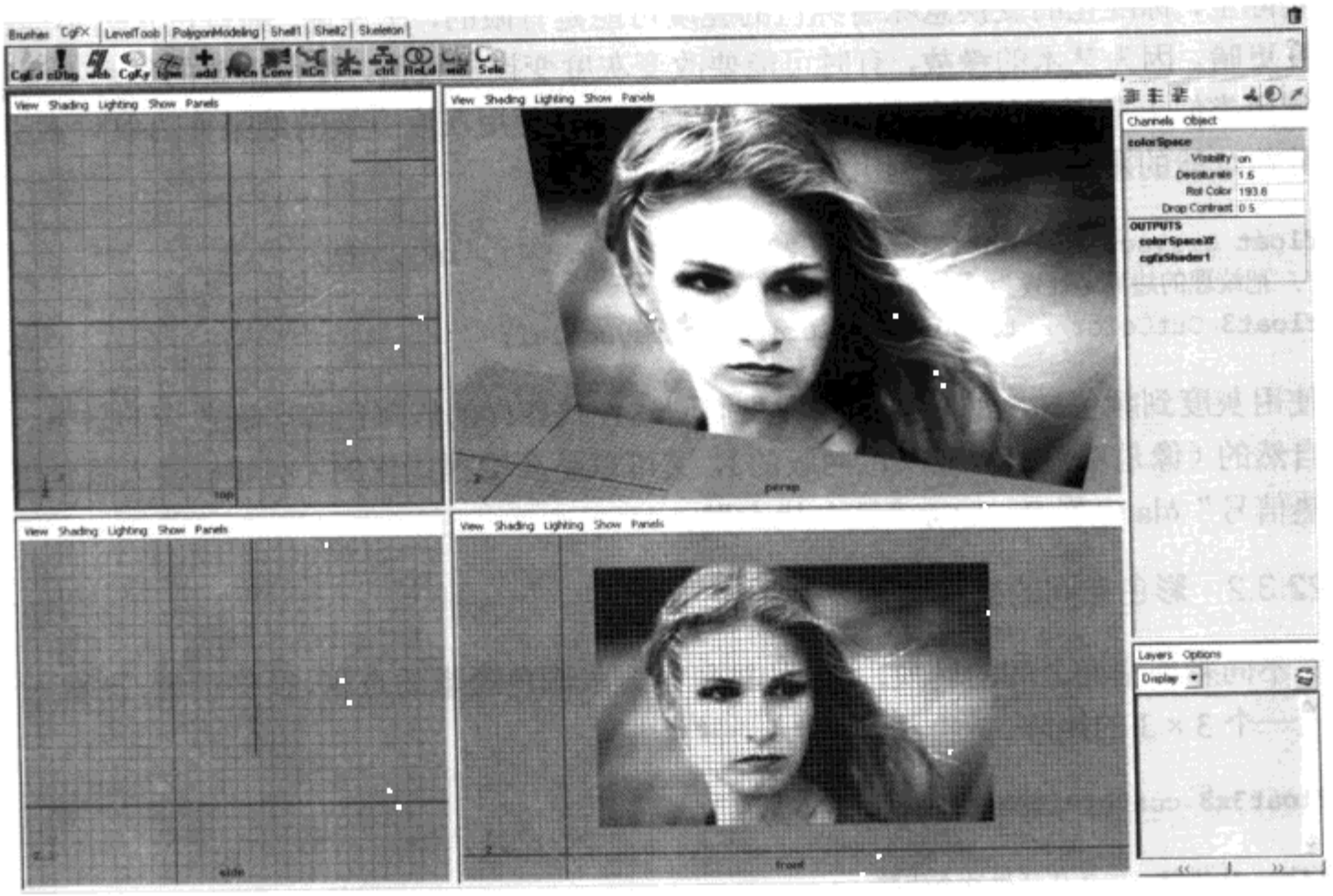


图 22-7 用 DCC 工具所做的彩色立方体的变换预览
用一个片元程序着色的大海报，程序通过彩色立方体的转换矩阵，变换纹理颜色
抓住和缩放/旋转彩色立方体物体，改变了相片的颜色，可以容易地进行交互式实验

这个技术允许你选中空的物体，随意地缩放或旋转它，以获取不同的效果。这里不仅可以任意地缩放和拖曳，享受“迷幻的”效果，还可以尝试以下设置，来控制饱和度、亮度和彩色轮旋转等。

- 为了整体亮度，在（0, 0, 0）附近缩放。
- 为了饱和度，对着对角线矢量（1, 1, 1）缩放，如果彩色立方体完全不饱和，它将简单地退化为原点到（1, 1, 1）的一条线。
- 为了旋转彩色立方体，围绕同一个对角线矢量方向（1, 1, 1）旋转。
- 为了更改整体对比度，围绕彩色立方体上任何点进行缩放。在（0.5, 0.5, 0.5）附近缩放，将会相对于中间灰度改变整体对比度，在（1, 1, 1）附近缩放，将会改变针对白色的饱和度。退饱和到其他任何的颜色也同样很直接。

任何数量这样的操作都可以连接起来，最好先在 CPU 中完成，然后再传给片元 shader。

22.4　参考文献

Albers, Josef. 1987. *The Interaction of Color*, revised ed. Yale University Press.

Autiokari, Timo. 2003. "CIE_XYZ and CIE_xyY." Web site article.
http://www.aim-dtp.net/aim/technology/cie_xyz/cie_xyz.htm

Bourgin, David. 1994. "Color Space FAQ." Web site page.
http://www.neuro.sfc.keio. ac.jp/'～aly/polygon/info/color-space-faq.html

Fraser, Bruce. 2003. *Real World Color Management*. Peachpit Press.

Hummel, Rob. 2002. *American Cinematographer Manual*, 8th ed. American Society of Cinematographers.

Margulis, Dan. 2002. *Professional Photoshop*, 4th ed. Wiley.

Poynton, Charles. 2003. Web site. **http://www.poynton.com**. Poynton 的网站包含很多有用的彩色相关的数字信息。Poynton 对现在使用的许多彩色标准贡献很大。

第 23 章 景深：技术综述

Joe Demers
NVIDIA

23.1 什么是景深

物体在距离镜头的一个范围之内能够清晰成像（聚焦了），在那个范围之外（或近或远）则成像模糊，这种效果就是景深。在照相业和电影业中，景深经常用来指示对场景的注意范围，并且提供场景深度的感觉。在本章中，把比这个聚焦范围远的区域称为背景，在这个范围前的区域称为前景，而在聚焦范围内的面积称为中景。

景深效果由透镜的物理性质产生。若要穿过摄像机透镜（或人眼睛的晶体）的光会聚到胶片（或人的视网膜）上的一个点，光源必须与透镜有特定的距离。在这个距离上的平面称为焦平面。不在这个精确距离上的任何东西，投影到胶片上的区域（不是一个点）称为模糊圈（CoC）。CoC 的直径与透镜尺寸和偏离焦平面的距离成正比。偏离距离小到一定程度，CoC 会变得比胶片的分辨率更小，摄影师和摄像师称这个距离范围为聚焦了（成像清晰）；而在这个范围之外的任何东西都是模糊的（没有对准焦点），见图 23-1 和 23-2。

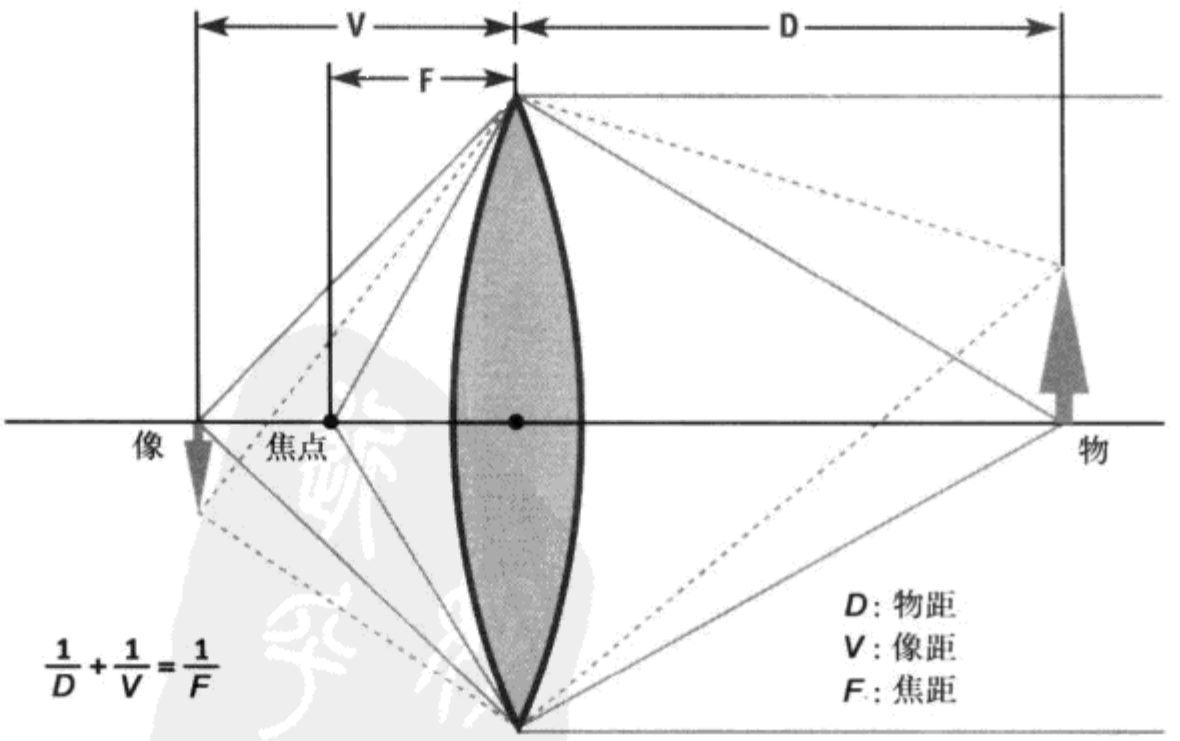


图 23-1 一个薄的透镜

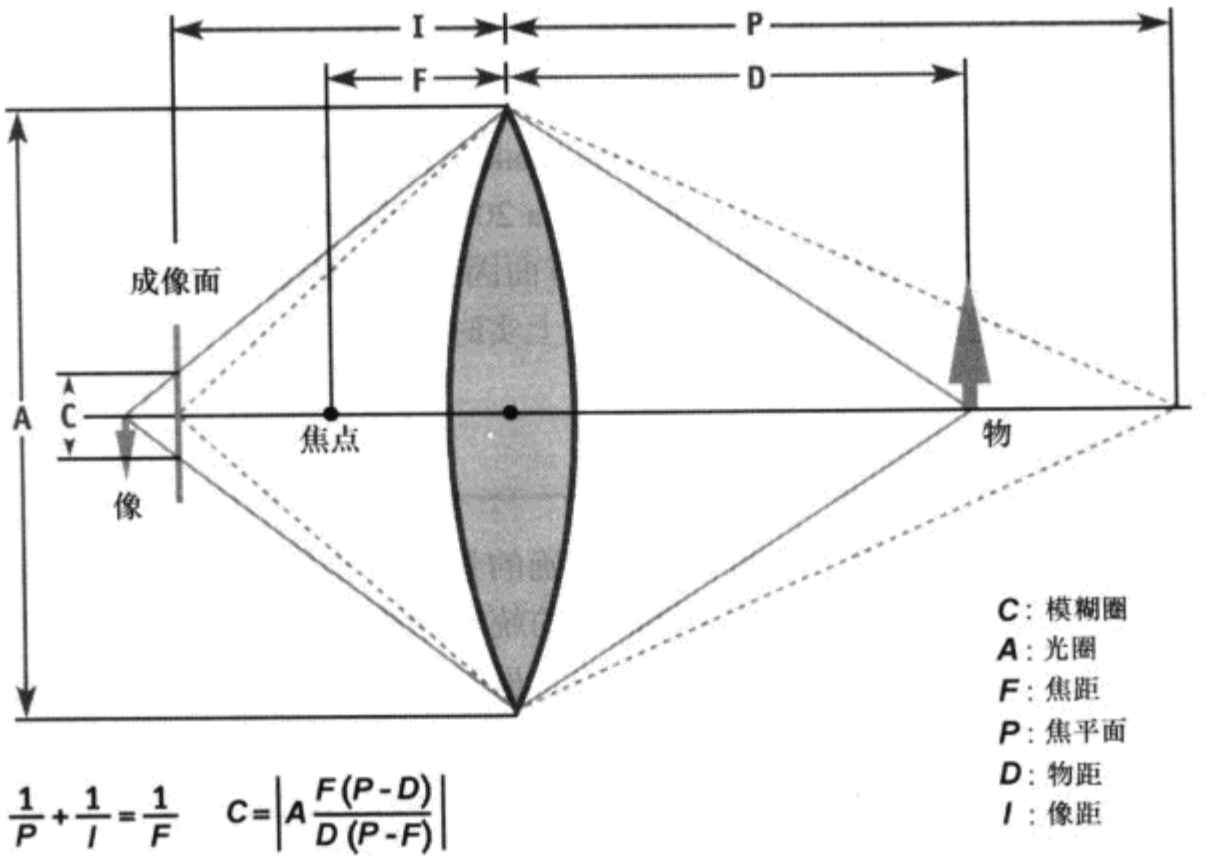


图 23-2　模糊圈

在计算机图形中，通常使用一个理想的针孔照相机（透镜尺寸为 0）向虚拟的胶片上投影，因此从场景到胶片的光线只有一条传输路径。若想得到景深效果，必须近似模拟真实透镜产生的模糊。

23.1.1　模糊圈的计算

在世界空间中，摄像机到物体的距离产生的模糊圈可以用摄像机参数来计算：

```
CoC = abs(aperture * (focallength * (objectdistance - planeinfocus)) /
          (objectdistance * (planeinfocus - focaliength)))
```

物距能通过 z 缓冲区的 z 值计算：

```
objectdistance = -zfar * znear / (z * (zfar - znear) - zfar)
```

模糊圈也能通过 z 缓冲值，以及综合摄像机参数的缩放项和偏置项来计算：

```
CoC = abs(z * CoCScale + CoCBias)
```

用摄像机参数计算缩放和偏置项：

```
CoCScale = (aperture * focallength * planeinfocus * (zfar - znear)) /
               ((planeinfocus - focallength) * znear * zfar)
CoCBias = (aperture * focallength * (znear - planeinfocus)) /
              ((planeinfocus * focallength) * znear)
```

23.1.2　主要技术

这章综述了 5 个主要的近似景深效果的技术。

- 穿过（非针孔）透镜表面的分布式光线跟踪（Cook 等．1984）

- 从多个摄像机进行渲染，也称为累积缓冲区技术（Haeberli 和 Akeley 1990）
- 多层次的渲染（Scofield 1994）
- 向前映射的 z 缓冲区技术（Potmesil and Chakravarty 1981）
- 反向映射的 z 缓冲区技术（Arce and Wloka 2002，Demers 2003）

我们把大部分时间花在 z 缓冲区技术上，一方面因为其他技术在文献中已有充分的研究，更重要的是这种技术更适用于在目前的图形硬件上实时渲染。

23.2 光线跟踪的景深

使用分布式光线跟踪的原因，是为了渲染正确的景深，因为这时光线不是穿过一个点，而是穿过一个透镜。对穿过透镜的光线进行恰当的统计分布，能够得到最正确的景深，因为这样真实地模拟了光线在摄像机和场景中的传输。虽然处理时间还达不到实时的要求，但是光线跟踪的景深产生了质量最好的图像。即使所用的光线较少，这个技术也只是精确地退化成噪音，而不产生更讨厌的走样，例如条带或重影。较之其他技术，它能更好地产生模糊的运动和阴影等效果，这是因为它能够独立地抖动采样，在时间和空间上与附近像素都不相关。图 23-3 是显示这个技术的例子，并说明了当使用太少样本的时候，噪音是如何发生的。

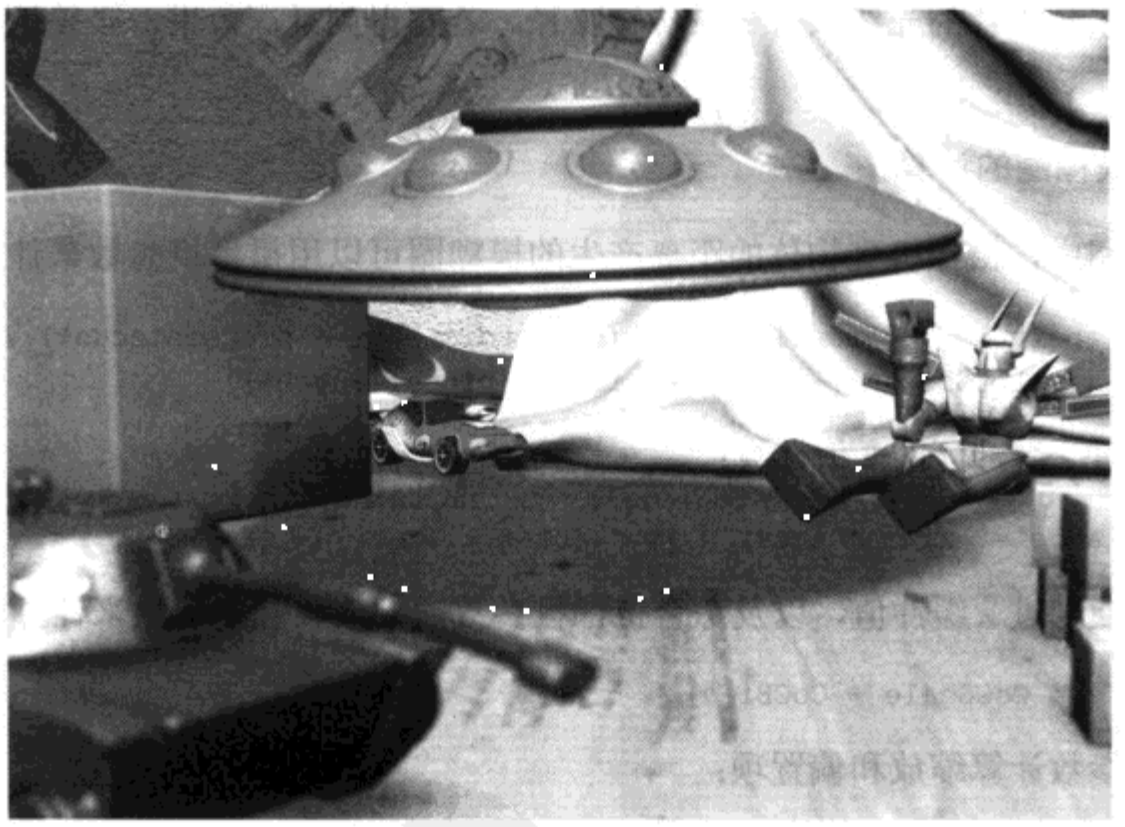


图 23-3 光线跟踪的景深

23.3 累积缓冲区的景深

累积缓冲区技术近似于使用传统 z 缓冲区的分布式光线跟踪法，差别在于前者伴随着一个附加的缓冲区，叫做累积缓冲区。累积缓冲区是高精度彩色缓冲区，在实时渲染中通常用它来累加多个图像。

一般的分布式光线跟踪法，是把穿过透镜进入场景的光线投射到多个位置；但是在这个技术中，改为从不同的位置到穿过透镜来多次渲染场景，然后使用累积缓冲区混合结果。所混合的渲染遍越多，看起来就越好，与“真实”景深越接近。如果被混合的遍数太少，在最模糊的区域，即最前面的前景和最后面的背景区域，就会看到重像，或物体的重影。由于所需要的遍数与模糊圈的面积成正比，所以对非轻微景深的场景，应该限制使用大量的遍。如果每遍对应的最大 CoC 面积为 4 个像素，所产生的景深效果通常是可接受的，几乎看不到走样，因为这样把走样条带限制为最大 2 × 2 个像素。对半径为 8 个像素、面积约为 200 个像素的 CoC，需要 50 遍。若降低渲染质量，每个遍使用 9 个样本（条带最大限于 3 × 3 个像素），对半径为 6 个像素的 CoC，只需要 12 遍。对于整数值的累积缓冲区，精度是一个问题，可能会在最终的图像上产生条带走样。

图 23-4 显示，即使图像只有很轻微的景深时，在玩具坦克炮塔周围也产生了条带。

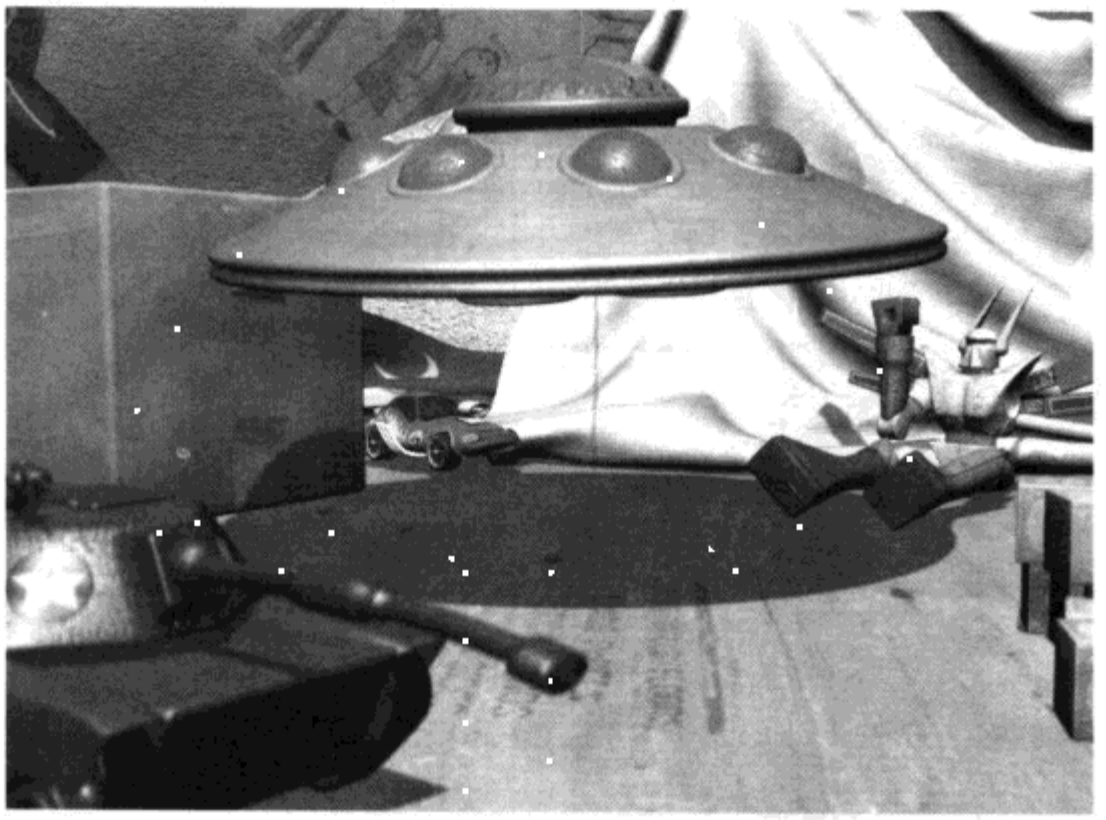


图 23-4　累积缓冲区的景深

这里介绍的只是一种有实时可能性的技术，它不是用针孔摄像机，而是用透镜（光圈）摄像机渲染正确的可见性和阴影，产生“真实”的景深。但是若想得到好的质量，就会需要太多的遍数，丧失实时操作的可能。

23.4　分层的景深

分层景深的技术基于 2D 方法（有时叫做使用层次的 2½D），该方法能以 2D 绘画和照片编辑软件包完成简单的场景。如果可以把各个物体分层排序，而层的深度互不重叠，那么就能够基于各层所代表的深度对每层进行模糊。然后把这些层组成最终的图像，给出景深的印象。

这个技术很简单，由于每层分别有一个可见物体的图像，所以可以很好地处理物体边缘的可见度。虽然这种着色法在技术上不正确，但是在大多数的场景中注意不到这一点。

这个方法的缺点是：当物体的深度跨度太大时，它的景深就显示不出来，尤其在前景中的物体更是如此。造成这种现象的原因是同层中的所有像素都均匀地进行模糊。同时，在相邻层次间的物体的模糊程度也变化得太快。这是由于各物体的典型深度是基于各层的平均深度，而不是基于相互接近的面的深度。

用这种方法不可能很好地分割所有的场景，例如有时物体的部分深度相互重叠，有时一个物体跨越所有的深度范围，像地板和墙壁。从理论上说，这种场景可以分割为许多 z 相等的平面，但是在实际上，有几种失真的效果难以克服。一种是把一个物体分割为多层，在一层上的两个边缘进行不同的模糊，结果在物体上看到了不一致的接边，这种失真还不算太严重。更严重的失真，是由于可见性沿着 z 分割可能存在的误差所造成的。在分割一些物体时，在它后面的较小物体本来应该是看不见的，却有可能被混合到层内。图 23-5 显示简单地把每个物体渲染为一层时，在各处进行同样的模糊造成的失真。尤其明显的是地板，在前景中它过于清晰，在背景中又太模糊。

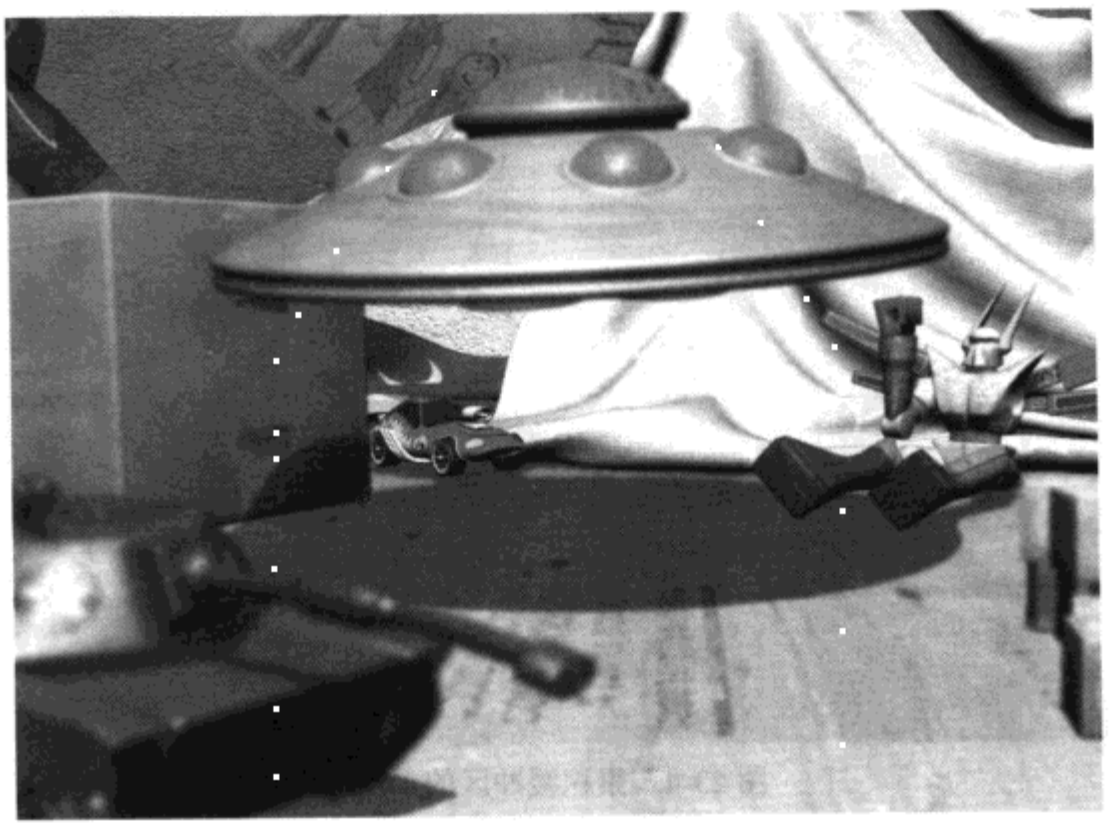


图 23-5 分层的景深

因为存在以上失真，在一般的情况下又难以解决，所以只有在能够解决这些问题的特殊情况下才可以使用这种方法。

23.5 向前映射的 z 缓冲区景深

使用向前映射（即渲染精灵）近似景深，适用于任意场景。使用后期处理软件包给渲染好的图像和影片加上景深，主要用的就是这种方法。向前映射是把源像素映射到目标图像的过程，反向映射则相反，它识别哪些源像素需要映射到目标图像上去。做法是在渲染时，把像素的彩色值和深度值存储到缓冲区，然后使用深度缓冲值为那个像素确定 CoC。把像素以一个圆圈混合到帧缓冲区内，圆圈的颜色就是像素的颜色，圆圈的直径等于 CoC，而圆圈的

面积与 alpha 值成反比。为了避免模糊像素影响它前面的清晰像素，只允许那个像素与离摄像机更远的相邻像素混合。考虑到像素的最终 alpha 值之和不等于 1，通常需要一遍进行最后的重规范化。因为当 alpha 值小于 1 时，像素的颜色出现误差，通常会比它们贡献的颜色加权平均值要小。

因为这个技术所依赖的惟一信息，来源于用针孔摄像机模型渲染的图像，所以在计算中使用了不正确的可见性和着色。于是，如果一个点光源动态地从一个物体后面经过，它将立刻从看得见变为看不见，而不是从完全看得见，过渡到从透镜的某些部分能看见，再过渡到完全看不见。但是对许多应用这个技术还是适用的，所产生的失真还可以用透镜耀斑以及其他的艺术手法掩盖。

不幸的是，由于这个技术是在典型的屏幕分辨率下，渲染数以百万计的精灵（固定的彩色圆），因此它不能很好地对应于硬件。在软件方面，虽然特殊目的的算法效率合理，但是还远远谈不上实时。有人曾经试过变革这些算法，具体做法是用基于像素 CoC 的逐像素加权值和过渡距离，多次地混合过渡的源图像。但是这也像分层的景深一样存在着一种平衡。虽然通道在几何学上是极其简单的，但是却需要更多遍：典型情况下最大 CoC 的每个像素都需要一遍。图 23-6 是显示这个技术的例子，在这个场景中它工作得相当好，看不到明显的失真。

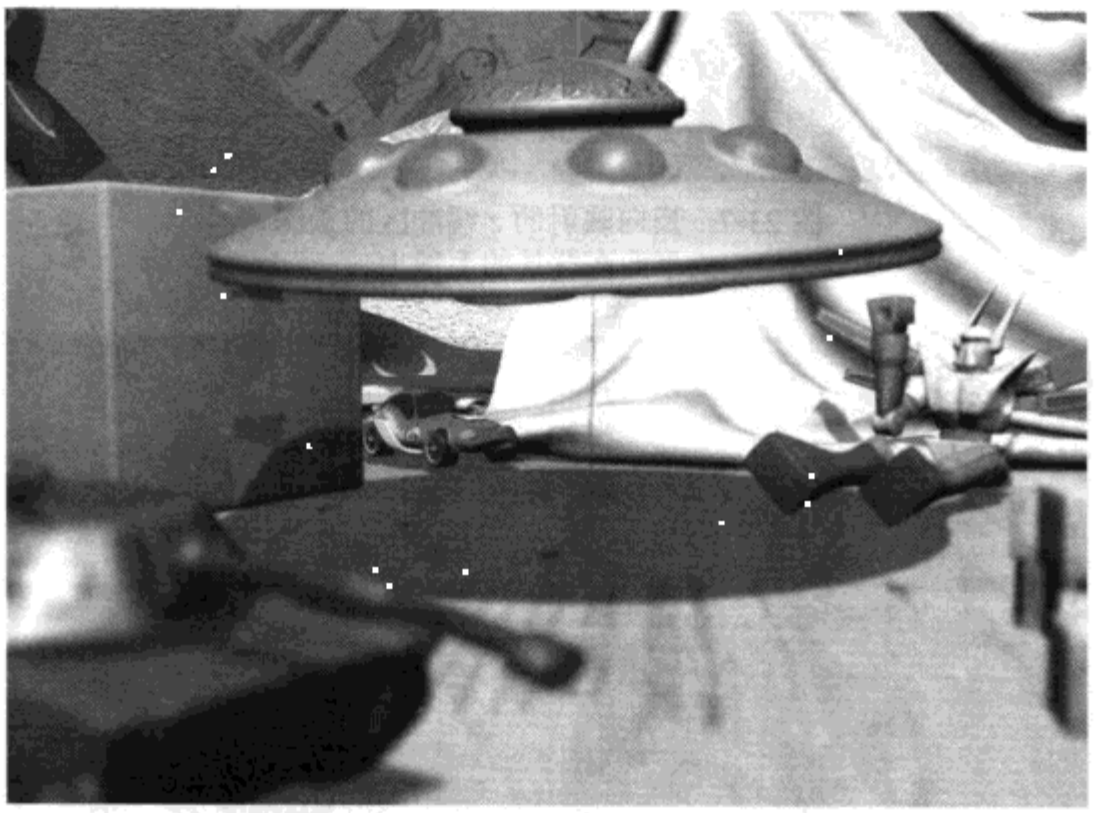


图 23-6　向前映射的 z 缓冲区的景深

23.6　反向映射的 z 缓冲区景深

最后一个技术是反向映射的（即纹理映射的）z 缓冲区技术，它在很多方面类似于向前映射技术。它也是存储一个颜色和深度的图像，但不是把混合的精灵渲染到屏幕上去，而是根据在 z 缓冲区中发现的深度值，逐个改变像素的量，去模糊被渲染的场景。对于 z 值和焦平面 z 值之间有较大差别的像素，选择较大的值作为模糊级别。这种模糊方法已经出现并且

得到应用，这里介绍主要变种。

最简单的变种允许硬件创建场景纹理的 mipmaps，然后执行纹理查询，把模糊圈作为导数来决定从哪个 mip 级别读取，见图 23-7、23-8 和 23-9。然而，直到 GeForce FX 和 Radeon 9700 发布之后，图形硬件才能够选择合适的纹理 mip 级别，以读出每个像素。

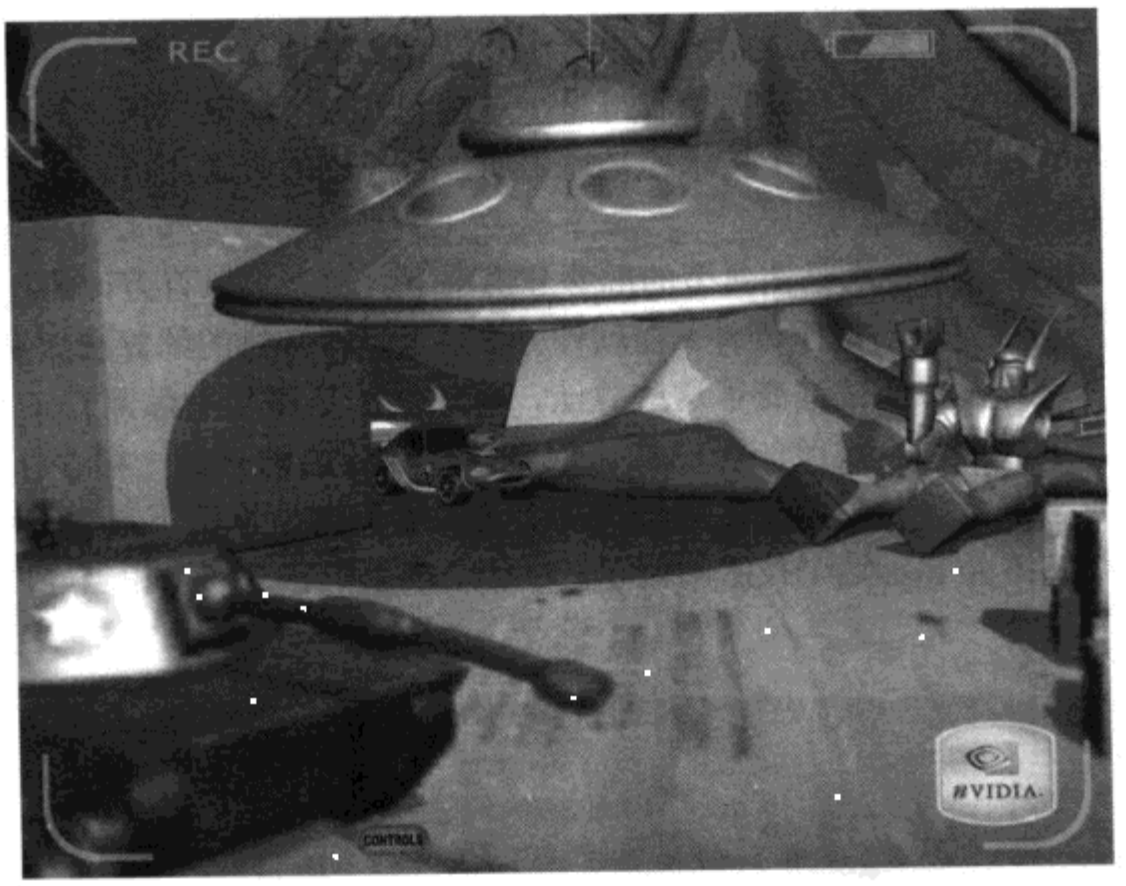


图 23-7 反向映射的 z 缓冲区的景深
焦平面在背景的毛毯上

图 23-8 反向映射的 z 缓冲区的景深
焦平面在前景的坦克炮塔上

图 23-9　无景深模拟

一些早期的方法曾使用了多个模糊的纹理来近似景深要求的模糊。有的方法用不同的 mip 级别多次地使用同样的纹理，有的则显示地创建全新的模糊纹理，有时使用更好的降采样滤波器（3 × 3 或 5 × 5 的，而不是 2 × 2 的）。这些方法通常执行 3 个查询：原始场景、1/4 的纹理（1/4 × 1/4）和 1/16 的纹理（1/16 × 1/16），并根据像素的 z 值将它们混合。代替让硬件产生 mipmap，也可以在渲染到纹理时，对每个 mipmap 使用更好的滤波器内核，以便得到质量更好的 mipmap。硬件 mipmap 生成器通常速度较快，然而，这是以降低质量为代价的。另一个有趣的方法是用硬件为彩色纹理产生一个浮点的区域加和表来代替 mipmap。尽管使用区域加和表可以改善双线内插的走样（稍后还要讨论），但是这个技术相当慢，而且它实际上不能保证产生良好定义的结果。

不幸的是，对建立区域加和表所使用的技术而言，所渲染的纹理与正在读的纹理是同一纹理，它把纹理的垂直线渲染为加和表的水平线，而把纹理的水平线渲染为加和表的垂直线。若使用目前的图形 API，读取正在写的纹理是不能保证产生良好定义的结果的（即，API 允许图形驱动程序涂黑、乱画或产生别的没用的东西）。但是在当前的 OpenGL 硬件以及驱动程序上，这种方法是可以工作的，新的硬件或新版本的驱动程序有可能会改变这一点。

虽然一般来说这个技术比其他技术快，但是也存在多种限制。某些限制较小并且在其他技术中也存在；但是有些限制很严重，并为这种技术所独有。

由于深度的不连续性产生失真

图像中深度的不连续所引起的失真是最严重的。因为图像的模糊程度（即，选择什么 mip 水平）由读取单个 z 值来决定，所以前景物体（要模糊的）不可能在中景物体（清晰的）上模糊，见图 23-10。解决的方法是多次读取深度值，但是除非深度值是个变量，否则将会出现晕圈效果，即固定半径的模糊的渐减，这几乎像深度间断本身一样严重。另一个解决方法是读取被模糊的深度缓冲区，但是这也会引起严重的失真：使清晰物体的边缘模糊，

而模糊物体的边缘清晰。通过均匀地偏置所计算的 CoC，可以给整个场景添加额外的模糊。在 GeForce FX 开始的演示 Toys（玩具）中，当把焦平面移得过近 （通常不把焦平面移得太近）时增加了小小的额外模糊，这很好地帮助隐藏这种失真，这章的图像就是从中选取的。

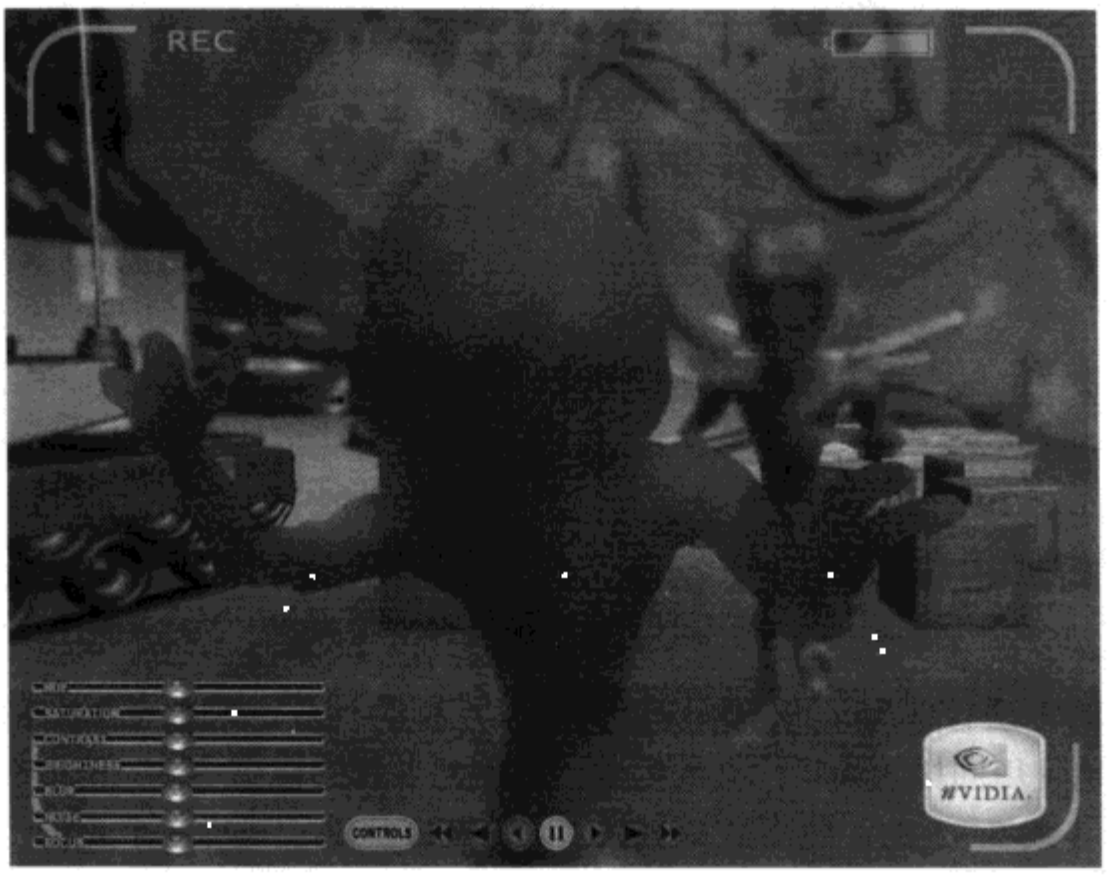


图 23-10 深度不连续产生的失真
在前景的外星人，虽然他的内部模糊，但是边缘清晰，也不能与附近的中景像素混合

双线内插的失真

把较小的 mipmap 或者模糊的图像放大会引起双线内插失真，见图 23-11。这个问题有一些较容易的解决方法：区域加和表、用实际大小的图像多次地模糊图像（而不是降采样），或在 mipmap 方法中与 CoC 成比例地变换取样位置等（潜在地多次取样）。见图 23-12。

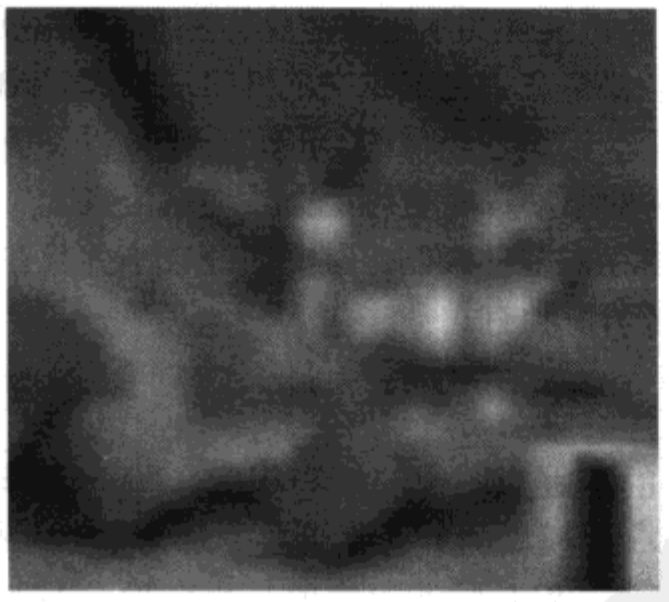


图 23-11 用三线过滤的放大失真

图 23-12 用抖动三线过滤的噪音失真

在“玩具”演示中，我们使用了抖动的 mipmap 采样方式，因为它是最快速的。虽然抖

动本身也引起失真，但是它所产生的噪音很好地克服了不匀的（crawling）、方块的，或阶梯状的失真图案，在动画期间效果尤其明显。

由于像素扩散产生的失真

像素扩散产生的失真是最轻微的。因为彩色图像的模糊是不加选择的，清晰的区域可能不正确地扩散到附近的模糊区域中去。在一个典型的场景中，这不会引起注意，但是它也容易变成非典型的情形，那就会引起注意。见图 23-13。

图 23-13　像素扩散的结果
像素从清晰的炮筒扩散到模糊的台子上，围绕炮筒产生晕圈效果

一个解决的方法是把物体分层渲染，对它们分别执行反向映射的 z 缓冲区的景深技术，然后再把它们合成。虽然这不能完全地解决问题。但是在通常情况下是有用的，因为各个物体内部有相似的色彩，而且深度差别很大时较少发生重叠。如果把每层的清楚深度设定成该层内最远处的深度，也能减少深度间断的问题。通常，如果层内的物体不在焦距上，那么该层的清楚深度也在焦距之外，于是这个物体将会在它后面的物体上模糊。一般来说间断性还会存在，但是这个技术已经把它减小了。然而，正像在分层景深技术中一样，这个层排序方法只能用于某些场景。

23.7　小结

在这章中，我们已经考察了主要的景深技术的优点和问题。用一个景深运算法则满足所有的质量和速率要求是不可能的。当必须正确着色而无须实时处理时，可以选择光线跟踪或累积缓冲区的景深算法。当速率非常重要时，最好选择分层或 z 缓冲区的景深算法。对渲染过的场景进行后期处理时，通过剖分带（splatting）景深技术可以得到最好的质量。

我们详细地考察了 z 缓冲区的景深算法，原因是在文献中缺少这方面的文章，也因为它是最有可能实现真正实时的技术。像素扩散的失真最难克服，但是它不严重。双线内插的失真也容易解决，只要花费一些运行时间，在 mipmap 或纹理之间较好地过滤就行了。最后，对于最严重的深度间断的失真，只有避开它们发生的情况（即，当焦平面在场景的远处时），这虽然会限制这种算法的一般用途，但是对许多实时应用已经足够了。

23.8 参考文献

Arce, Tomas, and Matthias Wloka. 2002. "In *Game Special Effects and Lighting*." Available online at **http://www.nvidia.com/object/gdc_in_game_special_effects.html**

Cant, Richard, Nathan Chia, and David Al-Dabass. 2001 "New Anti-Aliasing and Depth of Field Techniques for Games." Available online at

http://ducati.doc.ntu.ac.uk/uksim/dad/webpagepapers/Game-18.pdf

Chen, Y. C. 1987. "Lens Effect on Synthetic Image Generation Based on Light Particle Theory." *The Visual Computer* 3(3).

Cook, R.,T. Porter, and L. Carpenter. 1984. "Distributed Ray Tracing." In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*.

Demers, Joe. 2003. "Depth of Field in the 'Toys' Demo." From "Ogres and Fairies: Secrets of the NVIDIA Demo Team" presented at GDC 2003. Available online at

http://developer.nvidia.com/docs/IO/8230/GDC2003.Demos.pdf

Dudkiewicz, K. 1994. "Real-Time Depth-of-Field Algorithm." In *Image Processing for Broadcast and Video Production: Proceedings of the European Workshop on Combined Real and Synthetic Image Processing for Broadcast and Video Production, edited by Yakup Paker and Sylvia Wilbur,* pp. 257～268. Springer-Verlag.

Green, Simon. 2003. "Summed Area Tables Using Graphics Hardware." Available online at **http://www.opengl.org/resources/tutorials/gdc2003/ GDC03_SummedAreaTables.ppt**

Haeberli, Paul, and Kurt Akeley. 1990. "The Accumulation Buffer: Hardware Support for High-Quality Rendering." *Computer Graphics* 24(4). Available online at

http://graphics.stanford.edu/courses/cs248-02/haeberli-akeley-accumulation-buffer-sig90.pdf

Krivanek, Jaroslav, Jirf Zara, and Kadi Bouatouch. 2003. "Fast Depth of Field Rendering with Surface Splatting." Presentation at Computer Graphics International 2003. Available online at **http://www.cgg.cvut.cz/~xkrivanj/papers/cgi2003/ 9-3_krivanek_j .pdf**

Mulder, Jurriaan, and Robert van Liere. 2000. "Fast Perception-Based Depth of Field Rendering." Available online at **http://www.cwi.nl/~robertl/papers/2000/vrst/paper.pdf**

Potmesil, Michael, and Indranil Chakravarty. 1981. "A Lens and Aperture Camera Model For Synthetic Image Generation." *Computer Graphics*.

Potmesil, Michael, and Indranil Chakravarty. 1982. "Synthetic Image Generation with a Lens and Aperture Camera Model." *A CM Transactions on Graphics*, April 1982.

Rokita, P. 1993. "Fast Generation of Depth of Field Effects in Computer Graphics." *Computers dr Graphics* 17(5), pp. 593～595.

Scofield, Cary. 1994. "2½-D Depth of Field Simulation for Computer Animation." In *Graphics Gems III*, edited by David Kirk. Morgan Kaufmann.

第 24 章　高质量的过滤

Kevin Bjorke
NVIDIA

GPU 硬件可以提供一些快速过滤的访问纹理的方法，但是仅限于几种类型的纹理过滤，并不是对每种纹素（texel）格式都适用。如果用户建立自己的图像过滤方法，可以得到更好的质量和灵活性。这时应该了解在硬件和程序过滤之间存在着质量和速率的矛盾。

对过滤图像所考虑的内容也同样适用于 3D 渲染，尤其是把模型纹理化的时候，即把 3D 数据（和潜在的纹理信息）转换为一个新的 2D 图像的时候。了解这种转换对渲染反走样的图像是很重要的。

混合的过滤方法可以提供一个最佳的中间路径，即借助硬件纹理单元解析的着色。

24.1　质量与速度

在一些应用中，高质量的过滤是至关重要的。现代图形硬件可以以很高的速度进行 mipmap 化的纹理访问，但仅限于线性过滤法。对于某些图像类型，有时必须使用其他过滤方法。

如果硬件过滤不可行，或者需要极高的图像质量，可用像素着色代码随意执行任何过滤。

GPU 着色程序不同于 CPU 的主要之处在于：一般来说，CPU 数学操作比纹理访问更快。在像 RenderMan 这样的着色语言中，texture()是最费时的操作之一。而在 GPU 中的情形恰恰相反。

与 CPU 的 shader 不同，GPU 的 shader 不处理外围图形状态或磁盘系统，所以在 GPU 的着色语言中，纹理操作（例如 tex2D()）是最快的操作之一。由于速率快，我们就可以增加多种对过滤有用的元素，例如过滤器核心（kernels）和亮度校正查询表。

图像过滤的目的很简单：对于给定的输入图像 A，我们想要创建新的图像 B。把源图像 A 变换到目标图像 B 的操作就是图像滤波。最一般的变换是调整图像大小、锐化、变化颜色，以及模糊图像等。

在某些情况下（特别是在变化颜色时），新图像 B 的像素完全地与图像 A 的像素一一对应。但是其他大多数操作，是对图像 A 的多个像素采样，然后基于用户指定的函数应用这些值。源像素的图案，和它们对图像 B 像

素的相对贡献，就称为过滤核心（kernel）。把核心应用到源图像的过程叫做卷积（convolution）：即使用特殊的核心卷积源图像 A 的像素，创建新图像 B 的像素。

在通常的图像应用软件中，如 Adobe Photoshop，我们可以把核心指定为矩阵列或像素贡献等。在进行锐化（Sharpen）和模糊（Blur）的操作中，或者调整图像大小时，我们也可以让 Photoshop 计算应该使用的核心。

如果核心只是把像素简单地进行平均，我们称这个模型为盒式滤波器（box filter），因为模型是一个简单的长方形（所有的像素都在盒中）。每个采样的纹素（即，从纹理来的一个像素）的权重相等。盒式滤波器很容易构造，运行速度快，是 GPU 中硬件驱动过滤的核心。

用户自定义核心

如果我们正好使用小的核心，可以把它作为参数直接代入 shader。列表 24-1 显示的样本代码执行 3 × 3 过滤运算：对编入索引的纹素和它的邻居，赋予 W00 到 W22 的加权值，先求加权和，然后除以预计算的总和值（Sum）把它们重规范化，见图 24-1。

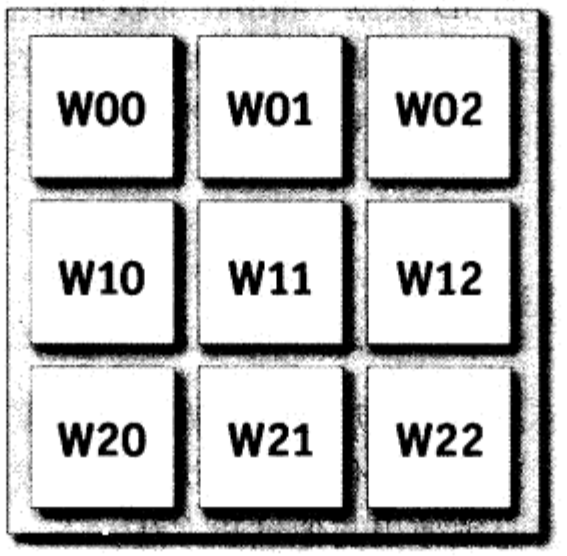


图 24-1 中心在 W11 的 3 × 3 过滤核心的像素设计
先对有关的纹素及其周围的纹素求加权和，
再除以用户给出的总和值（Sum）得到合成的像素

列表 24-1 读出 9 个纹素，计算加权和

```
float4 convolve3x3PS(vertexOutput IN,
  uniform sampler2D ColorMap,
  uniform float W00, uniform float W01, uniform float W02,
  uniform float W10, uniform float W11, uniform float W12,
  uniform float W20, uniform float W21, uniform float W22,
  uniform float Sum,
  uniform float StepHoriz,
  uniform float StepVert) : COLOR
{
  float2 ox = float2(StepHoriz, 0.0);
  float2 oy = float2(0.0, StepVert);
  float2 PP = IN.UV.xy - oy;
  float4 C00 = tex2D(ColorMap, PP - ox);
  float4 C01 = tex2D(ColorMap, PP);
```

```
    float4 C02 = tex2D(ColorMap, PP + ox);
    PP = IN.UV.xy;
    float4 C10 = tex2D(ColorMap, PP - ox);
    float4 C11 = tex2D(ColorMap, PP);
    float4 C12 = tex2D(ColorMap, PP + ox);
    PP = IN.UV.xy + oy;
    float4 C20 = tex2D(ColorMap, PP - ox);
    float4 C21 = tex2D(ColorMap, PP);
    float4 C22 = tex2D(ColorMap, PP + ox);
    float4 Ci = C00 * W00;
    Ci += C01 * W01;
    Ci += C02 * W02;
    Ci += C10 * W10;
    Ci += C11 * W11;
    Ci += C12 * W12;
    Ci += C20 * W20;
    Ci += C21 * W21;
    Ci += C22 * W22;
    return (Ci/Sum);
}
```

StepHoriz 值和 StepVert 值，定义了在纹素之间的间距（对于一个 256 × 256 的纹理，代入 1/256，以此类推）。因为想要从原始数据读出准确的纹素，所以纹理不应该被 mipmap 化。

根据硬件的 profile，如果不是像这样显式地用片元 shader 计算结果，而是预先在顶点 shader 中计算出纹理查找位置，然后将其作为内插值代入片元 shader，则 shader 可能更高效。

灰度数据的加速处理

列表 24-1 中的 shader 进行了 9 个纹理的访问，而且不得不相应地调节它的查找表。这意味着在像素之间有多余的访问（有希望被 GPU 高速缓冲存储的），还有很多小的加法和乘法。对于灰度的纹理，可以让 CPU 预先做一点工作，然后就能大大加速纹理的访问。

图 24-2 显示一个灰度图像和两个“辅助”图像，还显示了一个放大 4 倍的细节。

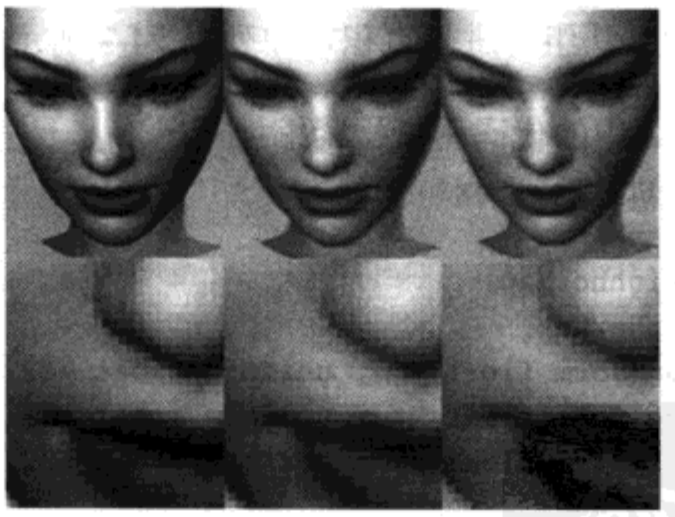


图 24-2　Grayscale 图像以及两个进行纹素数据偏移的 RGB“辅助”图像

第一个辅助图像是第一个图像的拷贝，但是它的红色通道向左偏移一个纹素，绿色通道

向右偏移一个纹素，蓝色通道向上偏移一个纹素，而 alpha（在印刷的页上看不见）通道向下偏移一个纹素。

第二个辅助图像也类似地偏移，但是向角上偏移纹素，而非向邻居偏移纹素：于是红色通道向左上偏移一个纹素，绿色通道向右（原文误为 left，译者注）上偏移一个纹素，蓝色通道向左下偏移一个纹素，等等。

通过这样安排通道，可以调用一个 tex2D()，访问所有的 4 个相邻像素（或所有的 4 个角），而不必调整纹理索引。现在的卷积 shader 变得很短，正如列表 24-2 所示。

列表 24-2 从访问 9 个纹素减为 3 个

```
float4 convolve3x3GrayHPS(vertexOutput IN,
  uniform sampler2D GrayMap,
  uniform sampler2D NeighborMap,
  uniform sampler2D CornerMap,
  uniform float W00, uniform float W01, uniform float W02,
  uniform float W10, uniform float W11, uniform float W12,
  uniform float W20, uniform float W21, uniform float W22,
  uniform float Sum) : COLOR
{
  float gray = tex2D(GrayMap, IN.UV).x;
  float4 ntex = tex2D(NeighborMap, IN.UV);
  float4 ctex = tex2D(CornerMap, IN.UV);
  float Ci = ctex.x * W00;
  Ci += ntex.z * W01;
  Ci += ctex.y * W02;
  Ci += ntex.x * W10;
  Ci += gray * W11;
  Ci += ntex.y * W12;
  Ci += ctex.z * W20;
  Ci += ntex.w * W21;
  Ci += ctex.w * W22;

  return (Ci/Sum).xxxx;
}
```

我们可以把这些加法和乘法组合进入点积，如列表 24-3 所示。

列表 24-3 把列表 24-2 中的代码用点积紧缩地写出

```
float4 convolve3x3GrayHDPS(vertexOutput IN,
  uniform sampler2D GrayMap,
  uniform sampler2D NeighborMap,
  uniform sampler2D CornerMap,
  uniform float W00, uniform float W01, uniform float W02,
  uniform float W10, uniform float W11, uniform float W12,
  uniform float W20, uniform float W21, uniform float W22,
  uniform float Sum) : COLOR
{
  float gray = tex2D(GrayMap, IN.UV).x;
  float4 ntex = tex2D(NeighborMap, IN.UV);
```

```
  float4 ctex = tex2D(CornertMap, IN.UV);

  float Ci = gray + dot(ntex, float4(W10, W12, W01, W21)) +
             dot(ctex, float4(W00, W02, W20, W22));
  return (Ci/Sum).xxxx;
}
```

得到结果的快慢取决于 profile。第一个 shader 在 DirectX ps_2_0 有 16 个指令；第二个因为构造两个 float4 项，需要许多 MOV 运算，使用了 19 个指令。

如果我们在 CPU 上预先处理那些矢量（最终它们将对整个图像平均），那么可以把函数减少为 11 条 ps_2_0 指令，仅为原始纹理版本的指令的 1/3，见列表 24-4。

列表 24-4　使用预先合并 float4 加权值的最紧缩的形式

```
float4 convolve3x3GrayHDXPS(vertexOutput IN,
  uniform sampler2D GrayMap,
  uniform sampler2D NeighborMap,
  uniform sampler2D CornerMap,
  uniform float4 W10120121,
  uniform float4 W00022022,
  uniform float W11,
  uniform float Sum) : COLOR
{
  float gray = tex2D(GrayMap, IN.UV).x;
  float4 ntex = tex2D(NeighborMap, IN.UV);
  float4 ctex = tex2D(CornerMap, IN.UV);
  float Ci = W11 * gray + dot(ntex, W10120121) + dot(ctex, W00022022);
  return (Ci/Sum).xxxx;
}
```

当核心是常量的时候

如果在 shader 的使用过程中核心不变，最好手工把核心值输入到 shader，工作时将输入值作为常量，把 0、1 或−1 这样的值分解出来，以减少指令的数目。在列表 24-5 显示的 shader 就是这种方法，基于两个常数的 3 × 3 核心做边缘检测。结果见图 24-3。

列表 24-5　边缘检测的像素 Shader

```
float4 edgeDetectGPS(vertexOutput IN,
  uniform sampler2D TexMap,
  uniform float DeltaX,
  uniform float DeltaY,
  uniform float Threshold) : COLOR
{
  float2 ox = float2(DeltaX, 0.0);
  float2 oy = float2(0.0, DeltaY);
  float2 PP = IN.UV - oy;
  float g00 = tex2D(TexMap, PP - ox).x;
  float g01 = tex2D(TexMap, PP).x;
```

```
    float g02 = tex2D(TexMap, PP + ox).x;
    PP = IN.UV;
    float g10 = tex2D(TexMap, PP - ox).x;
    // float g11 = tex2D(TexMap, PP).x;
    float g12 = tex2D(TexMap, PP + ox).x;
    PP = IN.UV + oy;
    float g20 = tex2D(TexMap, PP - ox).x;
    float g21 = tex2D(TexMap, PP).x;
    float g22 = tex2D(TexMap, PP + ox).x;
    float sx = g20 + g22 - g00 - g02 + 2 * (g21 - g01);
    float sy = g22 + g02 - g00 - g20 + 2 * (g12 - g10);
    float dist = (sx * sx + sy * sy);
    float tSq = Threshold * Threshold; // 在 CPU 上能够完成
    float result = 1;
    if (dist > tSq) { result = 0; }
    return result.xxxx;
}
```

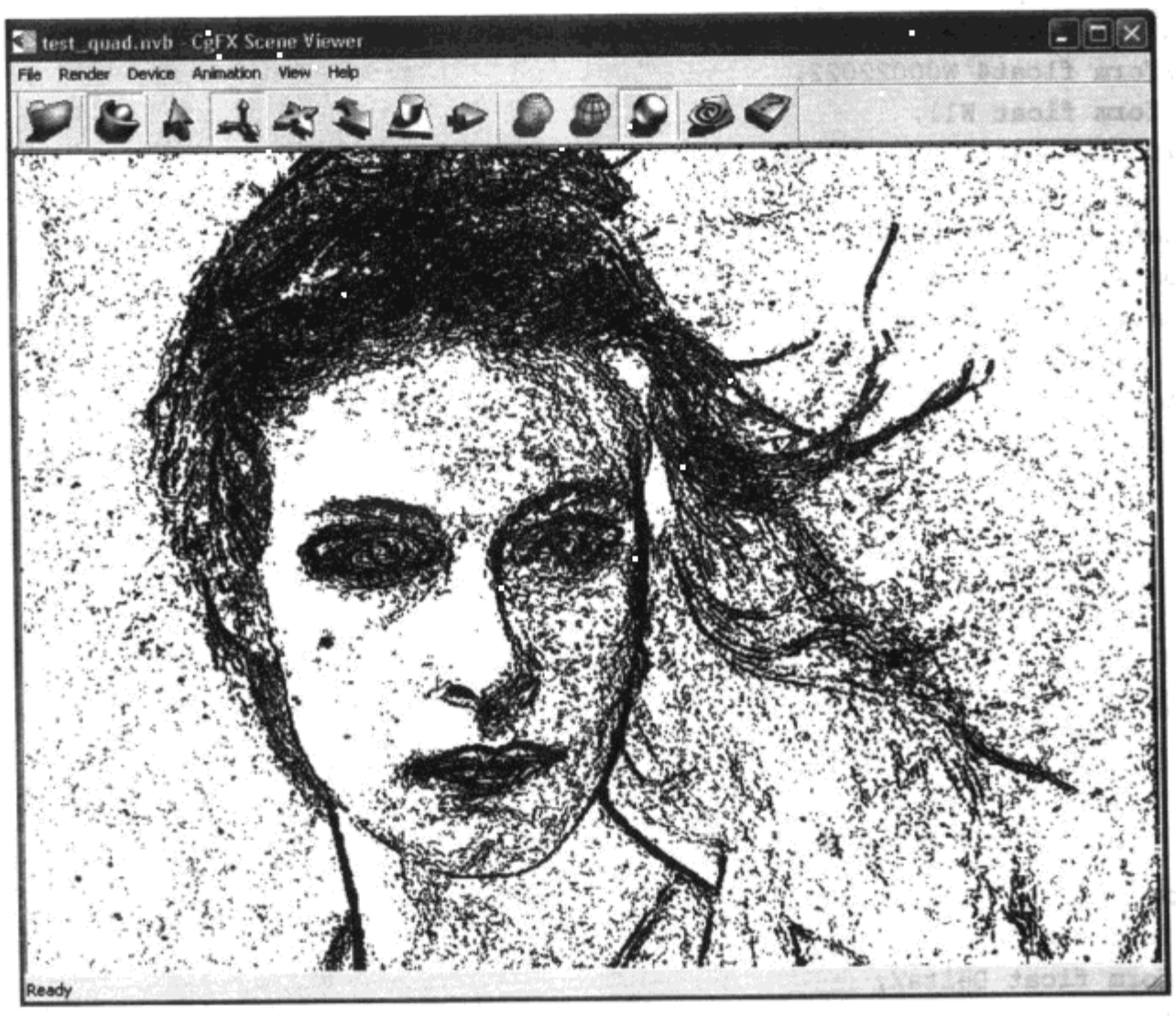


图 24-3 边缘检测 Shader 的结果

通过移去含 0 的项，我们可以减少一些乘法、加法，以及一个纹理存取指令。这个 shader 的指令少于 40 条。

如果像早先的灰度例子那样有辅助图像，指令数目大约可以减少一半，而且可以完全地去掉原始纹理，如列表 24-6 所示。

列表 24-6　为使用灰度等级辅助纹理所写的边缘检测 shader

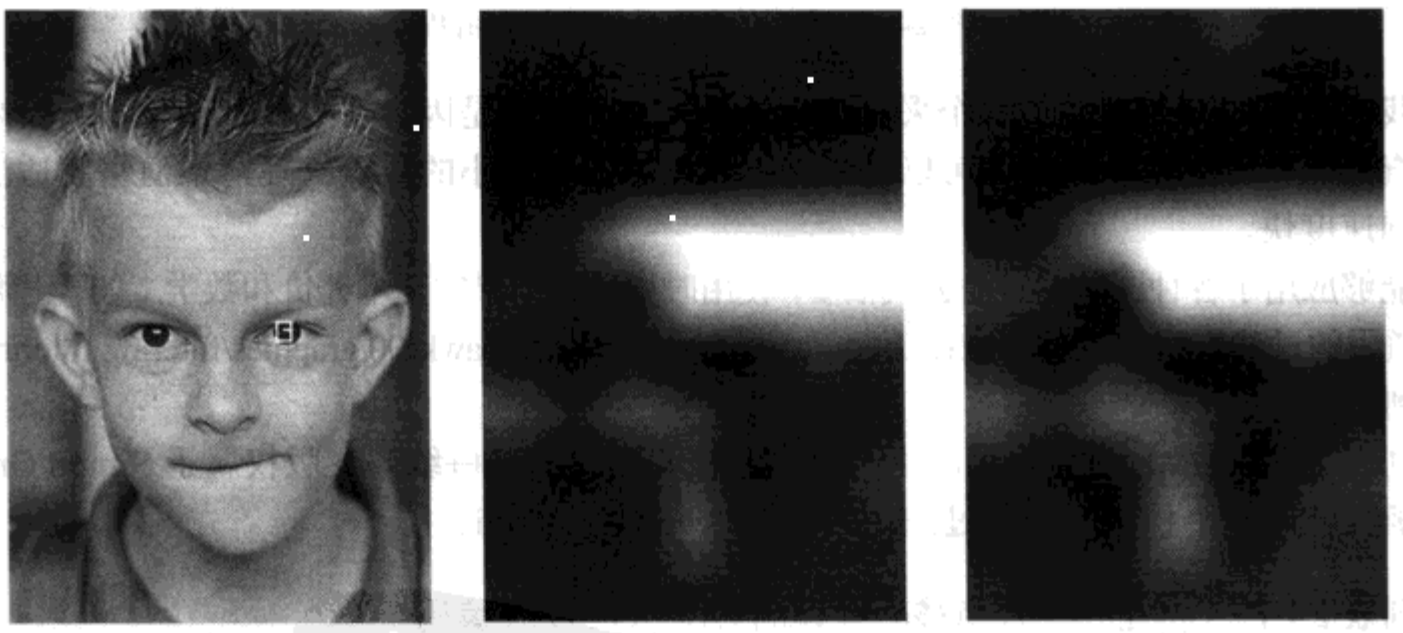
```
float4 edgeDetectGXPS(vertexOutput IN,
  uniform sampler2D NeighborMap,
  uniform sampler2D CornerMap,
  uniform float Threshold) : COLOR
{
  float4 nm = tex2D(NeighborMap, IN.UV);
  float4 cm = tex2D(CornerMap, IN.UV);
  float sx = cm.z + cm.w - cm.x - cm.y + 2 * (nm.w - nm.x);
  float sy = cm.w + cm.y - cm.x - cm.z + 2 * (nm.z - nm.y);
  float tSq = Threshold * Threshold;
  float dist = (sx * sx + sy * sy);
  float result = 1;
  if (dist > tSq) { result = 0; }
  return result.xxxx;
}
```

双立方过滤核心

盒式过滤器的问题是它产生的图像往往很差。当图像缩小时，边缘好像失去了反差。当图像放大时，盒式过滤器的矩形会显示图像 A 像素的矩形形状，所以它们看起来呈明显的“锯齿状”。

通过改变源像素的相对重要性，我们可以创建更好看的、调整过尺寸的图像。已经有一些不同的过滤方案可以做到这点，通常的做法是，越接近对应图像 B 中心的 A 像素，重要性越大。

一个大多数用户熟悉的过滤（可以“高质量”调节图像大小），一般称之为立方过滤；当在 x 和 y 两个方向中应用的时候，我们称之为双立方（bicubic）图像过滤，见图 24-4。



（a）　（b）　（c）

图 24-4　双线性和双立方过滤的效果

（a）原始的图像，注意眼睛上的长方形。（b）用双线性过滤把矩形的子图像区域放大 32 倍；（c）用双立方过滤把相同的区域放大 32 倍

我们可以通过解析的方法计算立方核心。举例来说，我们可以使用以下的式子，来创建

立方正弦近似过滤：

$$\begin{aligned}(A+2)x^3-(A+3)x^2+1.0 \quad & 0<x<1\\ Ax^3-5Ax^2+8Ax-4A \quad & 1<x<2\end{aligned}$$
$$A=-0.75$$

图 24-5 所显示的形状是用这些式子画出的曲线图。这个函数的 y 值表示赋予纹素的相对加权值，赋值基于纹素到给定的纹理坐标中心的距离。在中心的纹素权重为 100%，忽略与中心的距离超过 2 个纹素的纹素。注意，对于这个特别的过滤，一些加权值可能是负数，结果使重采样的图像更加清晰。

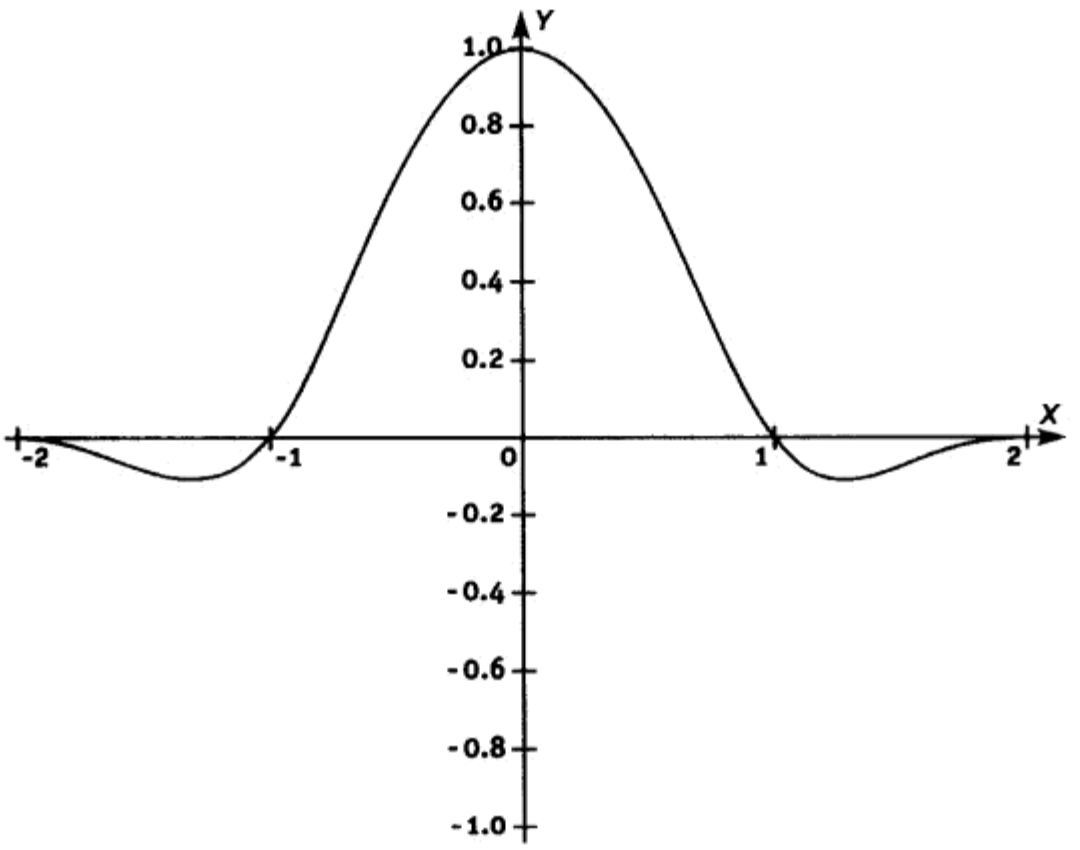


图 24-5 典型的立方体过滤核心函数

我们可以写一个函数为每个采样像素计算过滤值。但是因为在过滤范围中不需要高频精度，查询表就能工作得很好。可以简单地把这些数值写进小的浮点纹理，不但精度适当，而且查询速度快。

能够应用于各种图像的过滤公式很多，使用时要考虑结果的艺术评价和数学上的“正确”性。一篇通用的过滤方程的综述可以在 http://www.control.auc.dk/~awkr00/graphics/filtering/filtering.html 中查到。

列表 24-7 显示为 OpenGL 产生一个过滤核心纹理的 C++编码（版权属于 Simon Green）。这个函数使用一个稍微不同的过滤，叫做 Mitchell-Netravali。

列表 24-7 产生一个浮点核心纹理的 OpenGL C++编码

```
// Mitchell-Netravali 重建滤波器
// B = 1, C = 0 -立方 B 样条
// B = 1/3, C = 1/3 -推荐的值
// B = 0, C = 1/2 - Catmull-Rom 样条

float MitchellNetravali(float x, float B, float C)
```

```
{
  float ax = fabs(x);
  if (ax < 1) {
    return ((12 - 9 * B - 6 * C) * ax * ax * ax + (-18 + 12 * B + 6 * C) * ax * ax +
            (6 - 2 * B)) / 6;
  } else if ((ax >= 1) && (ax < 2)) {
     return ((-B - 6 * C) * ax * ax * ax + (6 * B + 30 * C) * ax * ax +
              (-12 * B - 48 * C) *
              ax + (8 * B + 24 * C)) / 6;
  } else {
     return 0;
  }
}

// 产生一个 1D 浮点纹理，为立方体过滤编制权重值
GLuint createWeightTexture(int size, float B, float C)
{
  float *img = new float[size * 4];
  float *ptr = img;
  for(int i = 0; i < size; i++) {
    float x = i / (float) (size - 1);
    *ptr++ = MitchellNetravali(x + 1, B, C);
    *ptr++ = MitchellNetravali(x, B, C);
    *ptr++ = MitchellNetravali(1 - x, B, C);
    *ptr++ = MitchellNetravali(2 - x, B, C);
  }
  GLuint texid;
  glGenTextures(1, &texid);
  GLenum target = GL_TEXTURE_RECTANGLE_NV;
  glBindTexture(target, texid);
  glTexParameteri(target, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
  glTexParameteri(target, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
  glTexParameteri(target, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
  glTexParameteri(target, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
  glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
  glTexImage2D(target, 0, GL_FLOAT_RGBA_NV, size, 1, 0, GL_RGBA, GL_FLOAT, img);
  delete [] img;
  return texid;
}
```

推荐的尺寸是 256，可调用 createWeightTexture（256, 0.5, 0.5）实现。

列表 24-8 是一个短的 shader 函数，它将返回 4 个颜色值过滤后的组合。浮点纹理总是使用 samplerRECT 数据类型，因此，我们对这个有 256 个元素的一维小纹理使用 texRECT()函数，而且将第 2 个索引项设定为 0。

列表 24-8　以核心纹理作为一个 Cg 的函数

```
float4 cubicFilter(uniform samplerRECT kernelTex,
                   float xValue,
                   float4 c0,
                   float4 c1,
```

```
                    float4 c2,
                    float4 c3)
{
  float4 h = texRECT(kernelTex, float2(xValue * 256.0, 0.0));
  float4 r = c0 * h.x;
  r += c1 * h.y;
  r += c2 * h.z;
  r += c3 * h.w;
  return r;
}
```

列表24-9显示一个双立方的过滤，它扫瞄任何纹素的4个相邻纹素，接着对样本周围像素的列调用cubicFilter()，基于过滤的小数位置，然后再一次混合4个返回的样本。

列表24-9 先对4个纹素行进行过滤，然后再对结果的列进行过滤

```
float4 texRECT_bicubic(uniform samplerRECT tex,
                       uniform samplerRECT kernelTex,
                       float2 t)
{
  float2 f = frac(t); // 我们想要的子纹素部分
  float4 t0 = cubicFilter(kernelTex, f.x,
                          texRECT(tex, t + float2(-1, -1)),
                          texRECT(tex, t + float2(0, -1)),
                          texRECT(tex, t + float2(1, -1)),
                          texRECT(tex, t + float2(2, -1)));
  float4 t1 = cubicFilter(kernelTex, f.x,
                          texRECT(tex, t + float2(-1, 0)),
                          texRECT(tex, t + float2(0, 0)),
                          texRECT(tex, t + float2(1, 0)),
                          texRECT(tex, t + float2(2, 0)));
  float4 t2 = cubicFilter(kernelTex, f.x,
                          texRECT(tex, t + float2(-1, 1)),
                          texRECT(tex, t + float2(0, 1)),
                          texRECT(tex, t + float2(1, 1)),
                          texRECT(tex, t + float2(2, 1)));
  float4 t3 = cubicFilter(kernelTex, f.x,
                          texRECT(tex, t + float2(-1, 2)),
                          texRECT(tex, t + float2(0, 2)),
                          texRECT(tex, t + float2(1, 2)),
                          texRECT(tex, t + float2(2, 2)));
  return cubicFilter(kernelTex, f.y, t0, t1, t2, t3);
}
```

因为我们需要对5个ftex样本和16个tex样本的系统进行纹理化，所以多次调用了texRECT()。

这里的方法允许在一遍中进行过滤，做法是沿着x过滤相邻的纹理元4次，然后将结果沿着y过滤一次。虽然这个方法可用，也是教科书中的好例子，但是远非最佳。这类的过滤中，更高效的计算方法是2遍过滤，在x和y通道上使用独立的函数。这种技术在本书的第21章“实时辉光”中有详细的描述。

对屏幕取齐的核心

在前面的例子中，我们应用过滤器“遍历”了每个纹素，我们称核心纹理是对所要生成的样本取齐的。

有些类型的图像，特别是那些录像机和数字相机产生的图像，可以受益于类似“静态核心”的方法。这些核心纹理能够把像素解码或编码，使其进入数字格式和广播格式（例如 Dl、YUV、GBGR 以及与 Bayer 模式传感器相关的，甚至如索尼公司 RGBE[①]那样复杂的 2D 模式）。在这种情况下，纹素相应的加权值是与屏幕取齐的，而不是与样本取齐的。

让我们考虑 YCbCr“Dl”的 4:2:2 编码的情况。在一个 4:2:2 图像中，把像素安排得像一个字符串：

Y Cb Y Cr Y Cb Y Cr Y Cb…

于是将这个字符串解码为 RGB 像素，我们对每个相邻像素应用不同的权重。存在许多略微不同的标准，我们考察最普通的一个：ITU-R BT.601。

对于给定的 *Y* 值，我们需要从 *Y* 和邻近的 *Cb* 和 *Cr* 值产生 RGB。我们在两边上的邻居之间内插这些值（线性内插为大多数用户的应用所接受），但是沿着帧的边缘只能使用毗连的邻居。我们可以把这些值当作一个纹理中的加权值。因为边缘的纹素需要特别的处理，所以加权纹理应该像整个屏幕一样大。对于大多数的 *Y* 样本，在任一边的邻居之间插值，将会得到有效的 *Cb* 或 *Cr* 值；但是对于沿着边的 *Y* 值，我们需要调整加权值模式，给最接近的 *Cb*/*Cr* 样本以最大的权重，而给任何“脱离屏幕”的纹理查询安排 0 权重。

给定一个复合的（Y, Cb, Cr）像素，变换如下。

```
#define MID (128.0/255.0)
#define VIDEO_BLACK (16.0/255.0)
float3 YUV = float3(1.164 * (Y - VIDEO_BLACK),(Cb - MID),(Cr - MID));
float3 RGB = float3((YUV.x + 1.596 * YUV.y),
                    (YUV.x - 0.813 * YUV.z - 0.392 * YUV.y),
                    (YUV.x + 2.017 * YUV.z));
```

这个变换也把 RGB 的范围规格化为 0～1。典型的 YUV 数据范围默认为 16 到 235，而不是计算机绘图中通常所用的 RGB 范围 0～255。超出白和黑的值通常用于特殊的视频效果：比如说，在缺少一个 alpha 通道时，superblack 时常作为视频的关键。

视频数据通常也需要亮度修正。因为亮度是常量（对于 NTSC 是 2.2，对于 PAL 是 2.8），这也可以很快地用一个 texlD()查询表实现。

24.2　对 GPU 求导的理解

复杂过滤有赖于知道究竟需要过滤多少纹理（或着色）。像 GeForce FX 那样的现代 GPU 提供的偏导函数可以帮助我们。对于在着色中用到的任何值，我们可以问 GPU：“这个值在屏幕的 x 或 y 方向上改变多少像素？”

① 索尼公司的 RGBE 缩写代表红、绿、蓝、翡翠色，这是一些索尼公司传感器采用的 4 个彩色滤波器。这个格式与高动态范围图像的 RGBE 无关，后者代表红色、绿色、蓝色和指数。

这些函数是 ddx()和 ddy()。虽然很少用到，但是它们对过滤和反走样操作很有帮助。这些求导函数提供的必需信息，便于我们执行程序式过滤，或者很快地修改大多数纹理采样中固有的过滤。

对于不直接支持 ddx()和 ddy()的 GPU，可以使用在本书第 25 章“用纹理贴图进行快速过滤宽度的计算”中概述的方法。

由 GPU 返回的 ddx()和 ddy()值是迭代值。也就是说，如果计算 ddx(myVar)，GPU 给出目前像素 myVar 值与下一个像素 myVar 值之差。这是一个线性差，由于 GPU 的 SIMD 结构使它很高效。当然，它应该适用于穿过多边形所求的插值（数值从顶点 shader 代入，那里任何相同数值的导数总是 0）。

因为这些导数总是线性的，二阶导数 ddx(ddx(myVar))是 0。如果所用的 shader 包含一些精巧的函数，可以用正确的解析计算来代替它的高阶导数，在求重要的值时（例如，用于科学计算或电影级别的渲染），应该使用这个公式。

一旦知道了在给定像素中的变化量，我们就能够确定适用的过滤。对于一个纹理，正确的过滤将不仅是在给定的 *u-v* 坐标上积分的那个纹理，而且要经过四边形的窗口积分那个纹理，水平纹理坐标是 ddx(UV)，高度方向是 ddy(UV)。当我们调用 tex2D()函数时，事实上就是自动地进行计算。高级 profile 的 Cg 允许我们选择想要过滤的范围。举例来说，通过指定：

```
float2 nilUV = float2(0, 0);
float4 pt = tex2D(myTextureSampler, IN.UV, nilUV, nilUV);
```

我们可以强使过滤的范围为 0，就是强迫这个纹理的采样总是使用“最近邻居”的过滤法，而不管 API 或 mipmaps 所设定的模式。

24.3 解析的反走样和纹理化

因为我们使用程序式纹理的 shader，所以知道在每个像素上的纹理样本尺寸，对有效的反走样是十分重要的。

为了反走样一个函数，我们需要在那个像素上对函数积分。在最简单的情况下，我们可以仅仅在像素边界上积分，然后取平均值（这就是盒式过滤器）。也可以给那个像素及附近像素的面积指定大小不同的加权值（如我们在本章开始的模糊例子中）。

考虑条纹（stripes）的简单情况。条纹函数可以表示成在有的像素上全充满，而在有的像素上全空白，但是这会产生锯齿，正如显示在列表 24-10 中的函数，它画出对物体的 *x* 坐标轴垂直的条纹（Balance 是一个小数，它定义暗纹和明纹的关系：默认值 0.5 意味着暗纹和明纹宽度相等）。可以使用条纹函数在两种颜色之间线性地插值。

列表 24-10 简单的条纹函数

```
float4 stripeJaggedPS(vertexOutput IN) : COLOR
{
  float strokeS = frac(IN.ObjPos.x / Scale);
  half stripes = 0.0;
  if (strokeS > Balance) stripes = 1.0;
  float4 dColor = lerp(SurfColor1, SurfColor2, stripes);
```

```
    return dColor;
}
```

图 24-6 显示了结果。可以看到，条纹具有明显的“锯齿状的”失真。

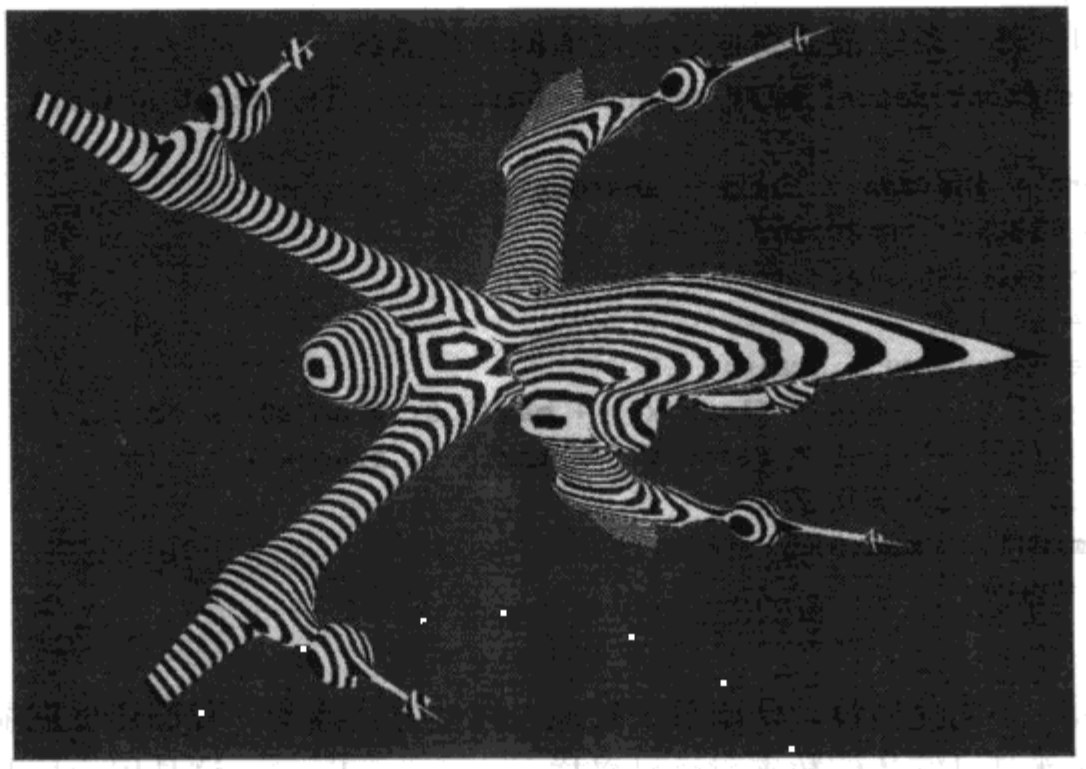


图 24-6　应用于一个模型的简单条纹

为克服这一点，可以在一个像素上对条纹函数积分，然后除以那个像素的面积得到反走样的结果。

图 24-7 显示了条纹函数及其沿着 x 的积分。条纹沿着 x 轴时有时无；而积分（即条纹覆盖面积的总和）则沿着 x 稳定增加，碰到各个条纹，面积稳定上升，然后持平，然后上升，然后持平，然后再上升……。

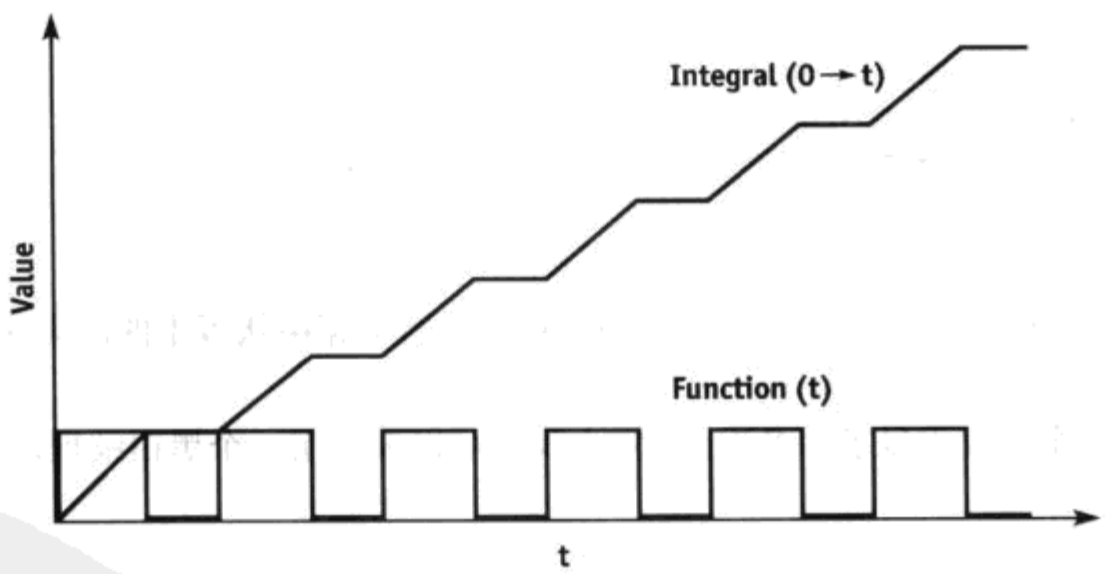


图 24-7　条纹函数的积分

定积分给我们提供了一个思路：可以通过一组像素累积条纹。

对于沿着 x 轴的任何 $x0$ 到 $x1$ 的间隔，取在 $x0$ 和 $x1$ 区间的积分，用 $x1$ 的积分减去 $x0$ 的积分，我们就可以知道在 $x0$ 和 $x1$ 之间跨过多少条纹。用面积差除以间隔宽度（$x1-x0$），将会告诉我们在整个跨距上条纹/非条纹的平均分布。这就是所需要的对单个像素的统计

结果。

把这个解法制成一个着色函数，通过计算 ddx()和 ddy()的变化，可以进行反走样，如列表 24-11 所示。

列表 24-11 基于条纹覆盖量返回灰度值的条纹函数

```
half stripe(float value, half balance, half invScale, float scale)
{
  half width = abs(ddx(value)) + abs(ddy(value));
  half w = width * invScale;
  half x0 = value/scale - (w / 2.0);
  half x1 = x0 + w;
  half edge = balance / scale;
  half i0 = (1.0 - edge) * floor(x0) + max(0.0, frac(x0) - edge);
  half i1 = (1.0 - edge) * floor(x1) + max(0.0, frac(x1) - edge);
  half strip = (i1 - i0) / w;
  strip = min(1.0, max(0.0, strip));
  return (strip);
}
```

这里我们做一个简化假设，使函数大致地以线性方式改变。我们还要把缩放比例和它的倒数分开，这样就可以缩放倒数来引进过采样（oversampling），这是用户控制的参量，根据要求模糊结果。

使用 stripe ()函数，我们可以创建另一个短的像素 shader，并在 EffectEdit 中应用它。见列表 24-12。

列表 24-12 应用过滤条纹函数的简单的像素 shader

```
float4 stripePS(vertexOutput IN) : COLOR
{
  half edge = Scale * Balance;
  half op = OverSample/Scale;
  half stripes = stripe(IN.ObjPos.x, edge, op, Scale);
  float4 dColor = lerp(SurfColor1, SurfColor2, stripes);
  return dColor;
}
```

图 24-8 看起来比图 24-6 好了许多，但是也更费时：锯齿状版本的渲染速度是这个平滑版本的 2 倍。

这也好像一个条纹重复了许多次。事实上我们可以仅使用一个黑白纹理，正如列表 24-13 所示。

列表 24-13 对于简单的条纹使用一个条纹纹理

```
float4 stripeTexPS(vertexOutput IN) : COLOR
{
  half stripes = tex1D(stripe1DSampler,(IN.ObjPos.x / Scale)).x;
  float4 dColor = lerp(SurfColor1, SurfColor2, stripes);
  return dColor;
}
```

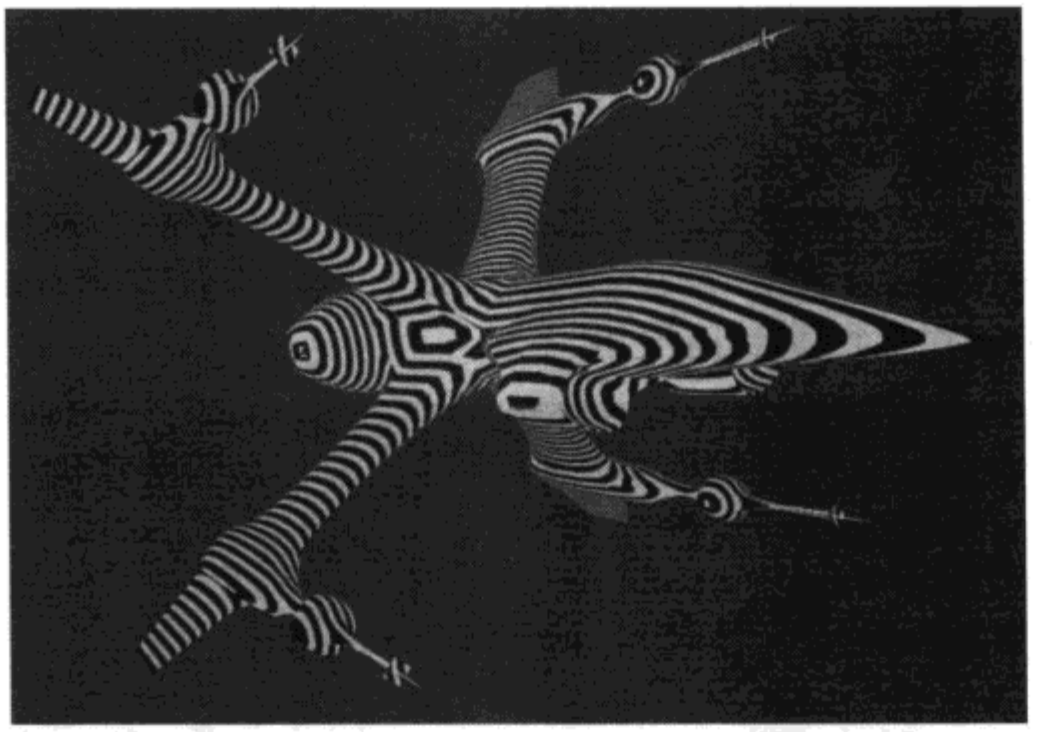


图 24-8　应用于模型的过滤后的条纹

这个版本运行得最快，其速度大约是程序式反走样版本的 3 倍。正如在图 24-9 中所见到的，产生的图像似乎丢失了它的“平衡（balance）”控制，但是这换来了速度。

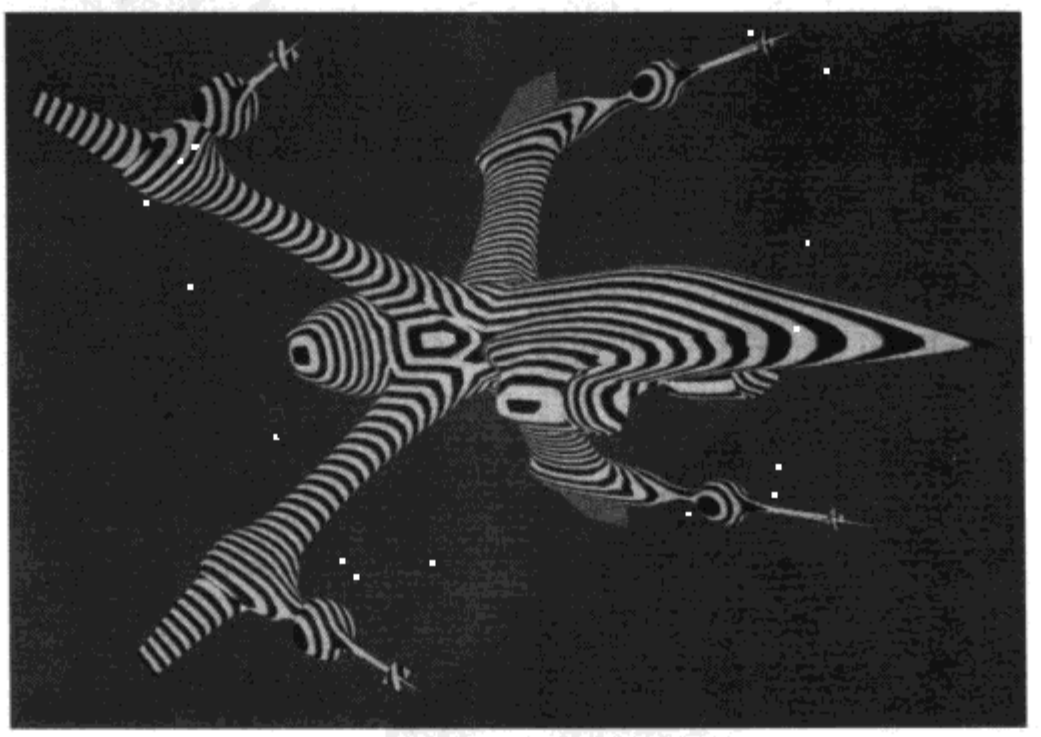


图 24-9　应用于模型的简单纹理

如果想要控制暗/明纹的平衡，作为图像的最后一部分，可以不进行用户输入，而是根据每个像素的 $(\boldsymbol{L}\cdot\boldsymbol{N})$ 改变平衡值，制作一个像雕刻一样的花纹，见列表 24-14。

列表 24-14　使用亮度控制条纹函数的平衡

```
float4 strokePS(vertexOutput IN) : COLOR
{
  float3 Nn = normalize(IN.WorldNormal);
  float strokeWeight = max(0.01, -dot(LightDir, Nn));
  half edge = Scale * strokeWeight;
  half op = OverSample / Scale;
```

```
    half stripes = stripe(IN.ObjPos.x, edge, op, Scale);
    float4 dColor = lerp(SurfColor1, SurfColor2, stripes);
    return dColor;
}
```

图 24-10 显示了产生的图像。这个 shader 再次减慢，甚至比第一个解析的例子还慢。可以应用一个纹理来加速这个函数，依据是可以在 2D 上“展开”1D 的黑白条纹纹理，然后把计算的“平衡”值当作一个 V 索引来使用。

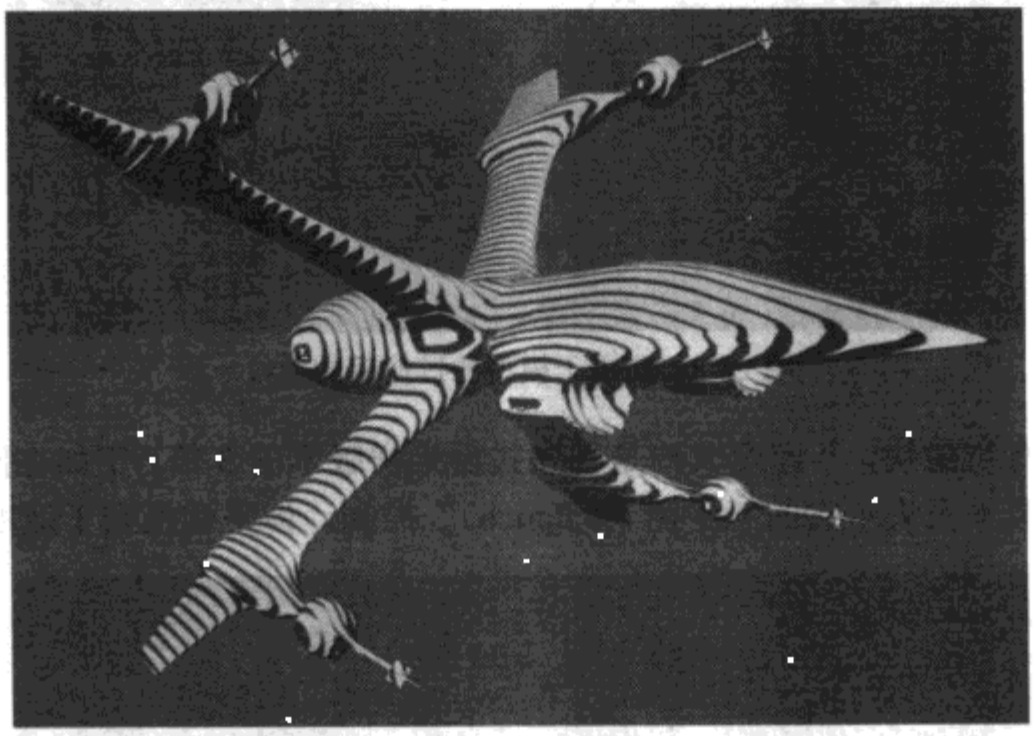


图 24-10 把明纹/暗纹平衡值应用于一个模型

条纹纹理如图 24-11 所示。这样一个纹理能由程序以 HLSL 的.fx 格式产生，如列表 24-15 所示。结果几乎像 texlD 条纹一样快，见列表 24-16。

图 24-11 条纹纹理

列表 24-15 产生可变条纹纹理的 HLSL.fx 指令

```
#define TEX_SIZE 64

texture stripeTex <
  string function = "MakeStripe";
  int width = TEX_SIZE;
  int height = TEX_SIZE;
>;
```

```
sampler2D stripeSampler = sampler_state
{
  Texture = <stripeTex>;
  MinFilter = LINEAR;
  MagFilter = LINEAR;
  MipFilter = LINEAR;
  AddressU = CLAMP;
  AddressV = CLAMP;
};

float4 MakeStripe(float2 Pos : POSITION, float ps : PSIZE) : COLOR
{
  float v = 0;
  float nx = Pos.x + ps; // 总是保持最后一列是满的
  v = nx > Pos.y;
  return float4(v.xxxx);
}
```

列表 24-16　把可变条纹纹理应用于像素 shader

```
float4 strokeTexPS(vertexOutput IN) : COLOR
{
  float3 Nn = normalize(IN.WorldNormal);
  float strokeWeight = max(0.0, -dot(LightDir, Nn));
  float strokeS = (IN.ObjPos.x / Scale);
  float2 newST = float2(strokeS, strokeWeight);
  half stripes = tex2D(stripeSampler, newST).x;
  float4 dColor = lerp(SurfColor1, SurfColor2, stripes);
  return dColor;
}
```

在图 24-12 中显示纹理化的版本，包含一些小的走样，可以用较高的分辨率或小心制定

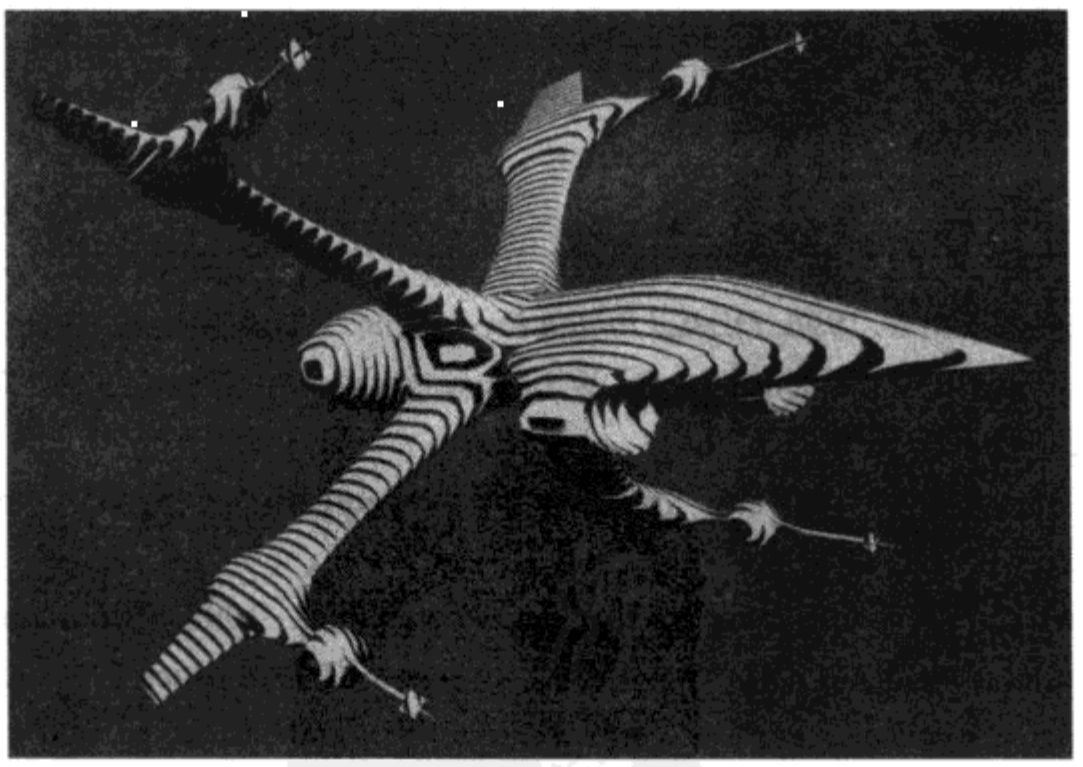


图 24-12　把可变条纹纹理应用于一个模型

的 mip 级别来改善。HLSL 的创建纹理函数 MakeStripe()，使用可选择的 PSIZE 输入。PSIZE 包含目前纹素的宽度和高度，目前纹素与整体纹理有关。它的值随 MakeStripe()目前计算的 mip 级别而改变。

结果的 shader 大小显然使运行速度加快：因为只有 14 个指令用于 PS2.0，而解析的版本却要用 45 个指令。

纹理化版本的精度受限制，特别是如果我们非常接近太空船，纹素将会开始清楚地出现。而解析的版本则总是显示清晰而平滑的边。

这里描述的过滤方法忽略了图形 API 对反走样（AA）的应用。通过 API，我们可以应用硬件图形 API 反走样到整个的帧：2X 的、4X 的等等。API 将会依照它自己的硬件核心，在每个像素中的多个子像素位置对几何采样。这种多次采样的反走样方法能校正许多问题，包括一些以别的方式难以处理的数学函数。然而，虽然多采样可以改善多边形边缘的质量，但是着色的质量却不可能得到改善，这是由于 shader 并没有对每个单独的样本重新计算。

多次采样的 AA 的缺点，是必须在表面采样和计算多次，这导致运行速度的降低。第二个缺点是对于一些应用程序，用户可能并不知道 AA 是否已经被打开。这就产生如何使质量和运行保持平衡的问题。如果图形 API 的反走样已被打开，应用程序的反走样就是多余的；而如果用户已经把图形 API 的反走样关掉，那么程序的反走样对维持质量就必不可少。shader 本身不可能知道当前 API 的反走样设置。可以选择 CPU 方面的应用程序管理 API。最后，应该在各种不同的采样环境下测试程序，选择最好的折中，如果应用程序出现问题，试着用图形 API 方面的反走样进行控制。

24.4 小结

图像处理的像素着色是 GPU 中尚未开发的领域之一。不仅游戏和相关的娱乐业可以从 GPU 图像处理中获益，许多其他的图像操作，例如图像合成、尺寸调整、数字照相，甚至图形用户接口，也能从它们的显示流水线中得到快速和高质量的图像。

图像处理中对像素的管理，也能用于 3D 图像中，进行有效的反走样，管理非常复杂的非相片着色算法。

24.5 参考文献

Castleman, Kenneth R. 1996. *Digital Image Processing*. Prentice-Hall.

Madisetti, Vijay K., and Douglas B. Williams, eds. 1998. *The Digital Signal Processing Handbook*. CRC Press.

Poynton, Charles. 2003. Web site. **http://www.poynton.com.** Charles Poynton 的优秀网站提供了许多有用的参考资料。

Young, IanT., JanJ. Gerbrands, and Lucas J. vanVliet. 1998. "Fundamentals of Image Processing." Available online at **http://www.ph.tn.tudelft.nl/DIPlib/docs/FIP2.2.pdf**

第 25 章　用纹理贴图进行快速过滤宽度的计算

Matt Pharr
NVIDIA

这章介绍在片元程序中计算导数近似值的技术。这个技术把特别的值载入纹理 mipmap 的每个级别，以特殊的方式使用纹理映射硬件，计算出导数的结果，给出在屏幕上的像素间某个量的变化率。在计算程序化图案的 shader 中计算导数是关键的操作，因为对图案进行预过滤，移除不影响像素采样率表现的细节，可以在产生图案的同时反走样。

Cg 标准程序库提供 *ddx*()和 *ddy*()函数，计算任意量关于 x 和 y 像素的导数。换言之，调用 *ddx*(*v*)，可以求出变量 v 在 x 方向的当前像素与下一像素之间的变化量；调用 *ddy*(*v*)同样也可以求出 y 方向的情况。可惜的是一些硬件 profile（比如 arbfpl）不支持这些导数的计算，当使用那些 profile 的时候，写出的 shader 总是产生失真。这章提供了一个解决方案，可以基于纹理映射功能解决这个问题。

25.1　在 shader 中求导的需求

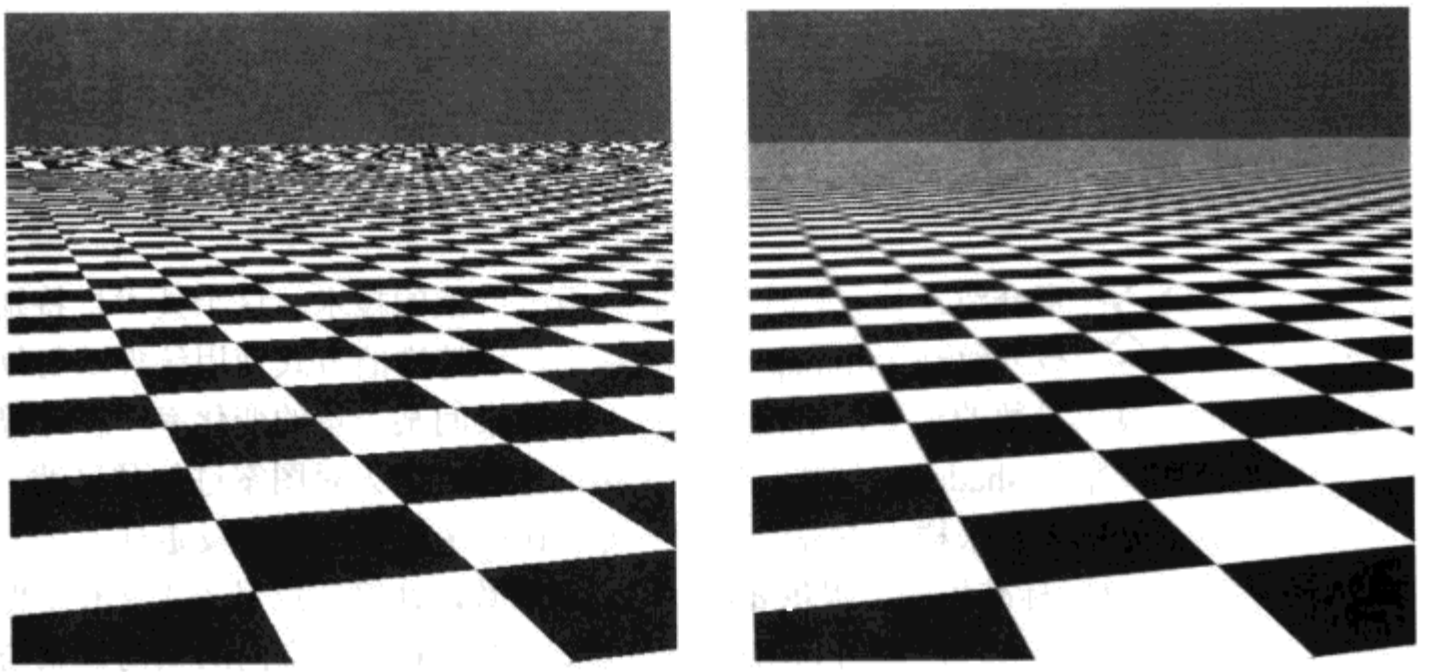
让我们看一个简单的程序化棋盘格函数，根据各个点的纹理坐标（u, v），函数返回 0 或 1。

```
float checker(float2 uv)
{
  return (fmod(floor(uv.x) + floor(uv.y), 2) < 1) ? 0 : 1;
}
```

这个函数计算点（u, v)所在单元坐标的整数部分，它们在相邻单元间交替为 0 和 1。产生在图 25-1（b）显示的棋盘格图案。用可编程的图形硬件，不再需要给纹理贴图中的图案编码；可以改在片元程序中计算纹理函数（对于简单的棋盘格纹理，不一定要程序化制作，但是一般而言，程序化计算纹理给实时渲染增加新的灵活性。这里适用于棋盘格的反走样原理，将同样用于更复杂的程序纹理）。

程序纹理的缺点是 shader 的作者需要特别注意 shader 的反走样。大致来说，不应该在单个点上计算程序函数的值，而是在面积上计算函数的平均值（第 24 章“高质量的过滤”更深入地讨论了这个问题）。更明确地说，就是要计算函数在当前像素和邻近像素之间的平均值。如果片元 shader 没

有考虑到这一点，图像就会失真，如图 25-1（a）所示。在图中棋盘格不仅在地平线上分裂为丑陋图案，在近处棋盘方格的边上也现出锯齿走样。当观察者或物体移动的时候，这些误差看起来更糟（这种走样类似于把纹理贴图缩小时不使用三线过滤而使用点采样纹理过滤模式产生的误差）。



（a） （b）

图 25-1 有反走样和无反走样的程序化棋盘格纹理

（a）程序的棋盘格纹理没有反走样，因此像素采样率不能表现水平方向上的高频细节，结果出现失真
（b）把过于高频的细节从图案中移走可以消除失真。计算出正确的过滤宽度是做到这点的关键

为了消除这些误差，需要计算棋盘格每个像素周围小面积的平均颜色。根据下面显示的基本 filterwidth()函数，可以容易地计算出任何值在像素之间变化的速率，以及程序纹理所需要过滤的面积。当调用 checker()函数求 *uv* 坐标时，得到的结果是坐标在当前像素与邻居之间变化速率的近似值（因此，它求的是各向同性的过滤宽度，而不是更正确的各向异性的宽度）。

```
float filterwidth(float2 v)
{
  float2 fw = max(abs(ddx(v)), abs(ddy(v)));
  return max(fw.x, fw.y);
}
```

实际上，我们可以假设 *uv* 是一个像素上的纹理坐标，而 *uv*1 是邻居像素上的值，它们之间的变化受到过滤宽度的限制：

```
abs(uv.x - uv1.x) < filterwidth(uv)
abs(uv.y - uv1.y) < filterwidth(uv)
```

不管所渲染的物体有怎样的几何体或如何计算 *uv*，都要遵循上述约束；也就是说，不能仅从顶点程序代入数值而不考虑这一点。filterwidth()函数通常会过高估计代入数值的变化（引起过滤的面积太大和过于模糊的结果），不会过低估计（引起过滤的面积太小和走样的结果）。当指定纹理过滤的行为时，像 OpenGL 和 Direct3D 那样的图形 API 也同样服从这个约定。

为了计算反走样的棋盘格，我们计算坐标 *uv* 在 2D 上的平均颜色，平均的范围是 *uv*–.5 * filterwidth(*uv*)到 *uv* + .5 * filterwidth (*uv*)。在列表 25-1 中的代码执行这个计算，它先对过滤区

域积分，再除以过滤区域的尺寸（在这章结尾的“进一步阅读”中，有关于这个方法如何工作的信息）。图 25-1（b）显示了使用 checker()函数所渲染的图像。注意这个图像中不再出现原先的棋盘格走样。虽然图像中间的方格边缘有点过于模糊，但是这个误差看起来尚可容忍。

列表 25-1　反走样的棋盘格函数

```
float checker(float2 uv)
{
  float width = filterwidth(uv);
  float2 p0 = uv - .5 * width, p1 = uv + .5 * width;
  #define BUMPINT(x) \(floor((x)/2) + 2.f * max(((x)/2) - floor((x)/2) - .5f, 0.f))
  float2 i = (BUMPINT(p1) - BUMPINT(p0)) / width;
  return i.x * i.y + (1 - i.x) * (1 - i.y);
}
```

25.2　用纹理计算过滤宽度

上述 filterwidth()函数，只能在支持 ddx()和 ddy()函数的 profile 下工作。技巧地使用纹理映射硬件，可以进行与 filterwidth()函数本质上相同的计算。这个技巧的关键在于，用函数 tex2D()所做的纹理贴图查询，可以自动地对纹理查询进行反走样，而不考虑所代入的纹理坐标。例如，如果调用 tex2D(map, float2(sin (*u*), tan (*v*)))，即使真实纹理坐标的像素异常地变化，适当过滤的纹理值还是能在每个像素的地方返回。基于在相邻像素上所计算的纹理坐标，硬件可以为每个查询决定要过滤的纹理面积。

纹理映射的这个性质，能够用来计算过滤宽度函数，而无须调用 ddx()和 ddy()。我们以这样一种方式执行纹理查询：通过它可以推测所过滤的纹理贴图面积，这里是推测需要过滤的程序化棋盘格纹理面积。这种解法分为两步：（1）为特定的纹理坐标组选择 mipmap 级别；（2）利用这一点决定过滤宽度。

对于第一步，考虑一个有 8 个 mipmap 级别的正方形纹理。在 mipmap 基础级别 0，有 128 × 128 个纹素；在较高的级别 1，有 64 × 64 个纹素，等等，一直到最高的级别 7，只有一个纹素。如果我们为每个 mipmap 级别赋值，让级别 0 的全部纹素为数 0，级别 1 的纹素为数 1，等等；然后，当对这个纹理做查询时，返还的数值将会告诉我们，映射纹理硬件实际上为当前查询选择了哪个 mipmap 级别。一般而言，纹理映射硬件在两级 mipmap 之间混合，即把两个相邻级别上的数按比例混合。

我们使用这个基本思路，但是改为，把每个级别的纹素编码为以 2 为底的对数。因此，贴图最高级别的单个纹素有一个 0 的值，下面一个级别的纹素的值是 1，再下面一个级别的纹素的值是 2，等等。为了在每通道 8 位的纹理贴图中得到更好的精度，我们把这个值放大到 16。列表 25-2 显示在 OpenGL 上以这种方式载入纹理贴图的 C++的源代码。

因此，当我们在 Cg 程序中用这个纹理贴图做一个纹理查询的时候，可以找到用于纹理滤波的 mipmap 级别的宽度，它是以 2 为底的对数。

这项技术的关键是，纹理滤波器选择 mipmap 级别时，要根据给定纹理坐标像素的变化速度，经过计算，使所选 mipmap 级别上纹素之间的距离，与纹理坐标相邻像素之间的距离相同。换句话说，如果确定所选的 mipmap 级别上的纹素间距，也就知道纹理坐标所改变的

像素。因为我们已经存储每个 mipmap 级别的纹素的数（以 2 为底的对数），那么在纹素之间的间距是 l/exp2 (logNumTexels) = = exp2 (−l ogNumTexels)。这就是我们所需要的滤波宽度。

列表 25-2 用计算滤波宽度的值载入 Mipmap 级别

```
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_NEAREST_MIPMAP_LINEAR);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);

#define LEVELS 10
#define RES (1 << (LEVELS - 1))
GLubyte *data = new GLubyte[RES * RES];
for (int level = 0; level < LEVELS; ++level) {
  int res = (1 << (LEVELS-1-level));
  int log2Width = LEVELS - 1 - level;
  for (int i = 0; i < res * res; ++i)
    data[i] = 16 * log2Width;
    glTexImage2D(GL_TEXTURE_2D, level, 1, res, res, 0,GL_RED, GL_UNSIGNED_BYTE, data);
}
delete[] data;
```

这样，给定初始化的纹理贴图（通过把它限于调用 filterMap 的 sampler2D 初始化），列表 25-3 的 Cg 代码首先寻找 mipmap 级别的宽度 log-2，该宽度用给定的纹理坐标来过滤纹理查询。由 tex2D()返还的数值将在 0 和 1 之间；把传给 glTexImage2D()的数值除以 255，就落入这个范围。为了在 Cg 程序中得到真实的 log-2 宽度，在填充 mipmap 级别的时候，这个 255 的缩放比例需要用因子 16 进行校正。

列表 25-3 使用从列表 25-2 来的 Mipmaps 计算滤波宽度的函数

```
uniform sampler2D filterMap;

float filterwidth(float2 uv)
{
  float log2width = (255./16.) * tex2D(filterMap, uv).x;
  return exp2(-log2width);
}
```

使用这个滤波宽度函数代替最初的函数，给出的结果在视觉上没有多大差别。它可以编译为少到 3 条片元 shader 指令，然而在整个 shader 的上下文中，两者的运行性能没有显著不同。

25.3 讨论

这种方法在很多情况下能很好地计算滤波宽度。它的运行性能几乎与基于求导的计算滤波宽度 filterwidth()方法相同，但是需要使用纹理单元和兆字节的纹理存储器。因为纹理级别保持常数，使用过的纹理可以保存为压缩的纹理格式，这可能减少存储器的使用和 GPU 在运

行时所需要的带宽。

这种方法的主要缺点是它不标准，当滤波宽度值非常大或非常小的时候，提供的结果可能不正确。当数值快速变化时，滤波宽度应该为 2。这个技术求出的滤波宽度会过小，因为只能选择最高的 mipmap 级别，它与宽度为 1 的滤波器没有差别。同样，当数值变化很慢时，求出的滤波宽度会过大：最细的 mipmap 级别限制了它能计算的最小的滤波宽度。然而，如果知道滤波宽度的一般预期范围，这在实际中就不是麻烦问题。另外，如果所有程序化纹理的滤波宽度超过某点后都取一个平均值，就无须担心上述问题。

确定何时有来自最高或最低 mipmap 级别的结果（分别对应于纹理的 mipmap 级别值 0 和小于 1 的 mipmap 级别值），可以解决这个问题。那时，可以用缩放了的纹理坐标进行另外的纹理查询，为那些坐标计算滤波宽度，然后重新缩放滤波宽度对缩放的纹理坐标予以补偿。列表 25-4 的代码显示低估滤波宽度的情况下应该怎么办，低估是由从 mipmap 棱锥的顶端工作引起。类似的方式也能用来解决从棱锥底部向上工作的问题。

列表 25-4　滤波宽度函数，它减轻低估造成的失真

```
float filterwidth(float2 uv)
{
  float log2width = (255./16.) * tex2D(filterMap, uv).x;
  if (log2width < .01) {
    log2width = (255./16.) * tex2D(filterMap, uv/512).x;
    return 512 * exp2(-log2width);
} else
     return exp2(-log2width);
}
```

这里之所以使用缩放因子 512，是因为我们在列表 25-2 中载入了 10 个 mipmap 级别。如果需要更多或更少的级别，应该相应地调整缩放因子。

25.4　参考文献

Ebert, David S., F. Kenton Musgrave, Darwyn Peachey, Ken Perlin, and Steven Worley. 2003. *Texturing and Modeling: A Procedural Approach.* Morgan Kaufmann.

本书的第 2 章对程序 shader 的反走样有很好的介绍。

Apodaca, Anthony A., and Larry Gritz, eds. 1999. *Advanced RenderMan: Creating CGI for Motion Pictures.* Morgan Kaufmann.

这是另外关于这个题目的很好资源。

Williams, Lance. 1983. "Pyramidal Parametrics." In *Computer Graphics* (*Proceedings of SIGGRAPH 83*) 17(3), pp. 1～11.

Williams 在这篇文章中把 mipmaps 介绍给图形业。

Fernando, Randima, Sebastian Fernandez, Kavita Bala, and Donald P. Greenberg. 2001. "Adaptive Shadow Maps." In *Proceedings of ACM SIGGRAPH 2001*, pp. 387～390.

这篇文章包含第一个 mipmapping 的类似应用，作者使用可比较的技术测定了阴影贴图的放大量。

Hadwiger, Markus, Helwig Hauser, Thomas Theufil, and Meister Eduard Groller. 2003. "MIP-Mapping with Procedural and Texture-Based Magnification." Sketch presented at SIGGRAPH 2003.

最近，这篇文章的作者用相似的技术测定片元程序中纹理的放大量，目的是用手应用双立方体过滤器，以得到放大纹理的更好图像。

第 26 章　OpenEXR 图像文件格式

Florian Kainz
Industrial Light & Magic
Rod Bogart
Industrial Light & Magic
Drew Hess
Industrial Light & Magic

在 GPU 上创建的图像大多是短暂的，一般只存在几分之一秒。但有时候创建的图像可能值得保留。如果保存它，最好存储在 NVIDIA half 类型的高动态范围图像文件格式中，并且可以附加另外的数据通道。

在这章中，我们将介绍 OpenEXR 图像文件格式，举出读和写 GPU 图像缓冲的例子，并讨论与图像显示有关的问题。

26.1　什么是 OpenEXR

OpenEXR 是由 Industrial Light & Magic（ILM）公司开发的高动态范围图像文件格式，目的是应用于计算机图像应用程序。OpenEXR 网站，www.openexr.org，有关于图像文件格式本身的全部细节。这节将概述高动态范围图像存储的主要特性。

26.1.1　高动态范围图像

数字图像显示设备，例如计算机监视器和电视，通常的动态范围大约为 1 到 500。这意味着，图像中最亮像素与最暗像素的亮度之比不会超过 500。

大多数图像文件格式与显示器的动态范围相当：存储在像素中的值从 0.0 到 1.0；0.0 代表“黑色”，1.0 代表显示器的最大强度，或“白色”。为了不使图像文件过大，像素值通常编码为 8 或 10 位的整数（分别为 0～255 或 0～1023），这对显示器的准确度足够了。

如果我们只是想把图像显现在显示器上，变化范围从 0 到 1 的文件格式就足矣。没有一点存储的像素比监视器的白色更亮，也不会有一点能比监视器所复制的更准确。

然而，如果我们想要处理图像，或想使用图像作为环境贴图进行 3D 渲染，那么就要求在图像文件中有更多信息。像素应该用大于 8 或 10 位的更准确值来存储，而像素值的范围在本质上应该是无限的。当文件格式允

许存储的像素值超出 0 到 1 时，它被称为高动态范围（HDR）格式。以高动态范围数据处理图像，而只以较少的位显示图像，可以得到最好的图像质量。

这里是一个简单例子，说明希望在高动态范围处理图像的原因（其他对于 HDR 图像的应用将在第 26.6 节中介绍）。

图 26-1 是一张相片显示了相当高动态范围的场景。在原始场景中，左边油灯的火焰比中间小盘子之下的阴影大约亮 100 000 倍。

图 26-1 高动态范围场景
超过 1.0 的像素值被钳制，这使非常明亮的区域退色并且失去细节

图像曝光的方式导致一些区域的亮度超过 1.0。在计算机监视器上，那些区域被夹持，并且显示为白色或不自然的饱和桔色色调。

我们可以通过把图像变暗来校正白色和橘色区域。但是如果把原始图像存储在低动态范围文件格式中（例如 JPEG 格式），把它变暗就会产生相当难看的图像。如图 26-2 所示，火焰周围的区域变成灰色。

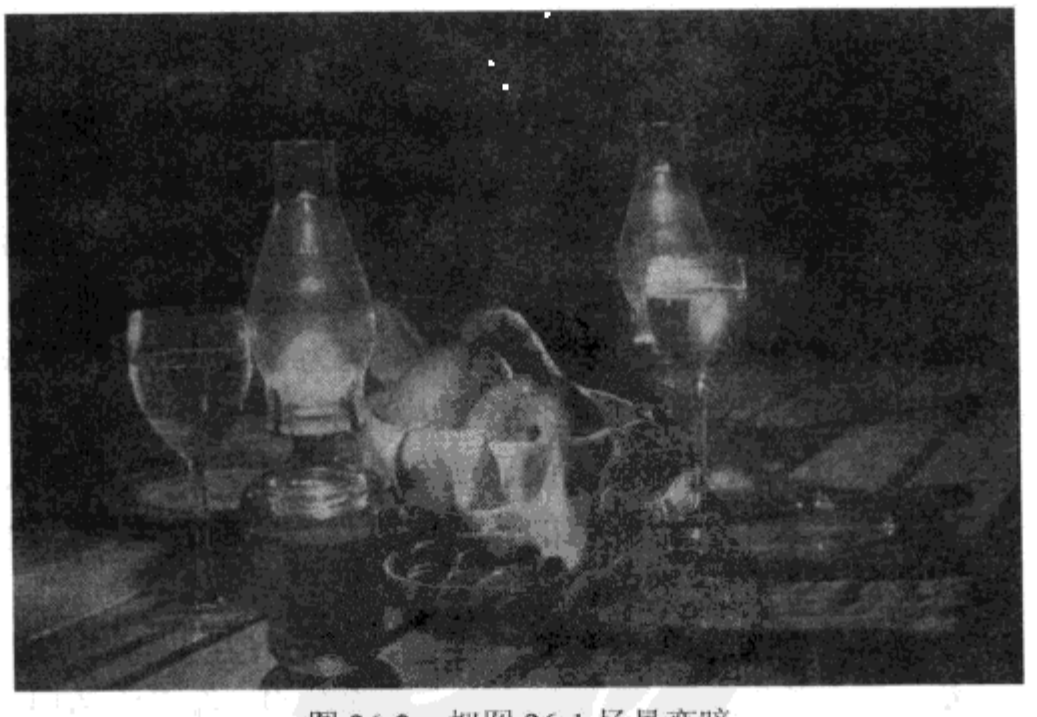


图 26-2 把图 26-1 场景变暗
由于钳制不可能恢复，明亮的区域变灰，并且细节丢失

如果原始图像存储在高动态范围文件格式中（例如 OpenEXR），保存明亮的像素值，而不是把它们夹持到 1.0，然后把图像变暗，产生的结果看起来仍然很自然，见图 26-3。

图 26-3　变暗的图 26-1 的高动态范围版本
在明亮的区域中显示出其他的细节，颜色看起来很自然

26.1.2　“半精度”（Half）格式

2003 年初，ILM 发表新的 HDR 文件格式，使用 16 位浮点颜色成分值。因为 IEEE 754 浮点规格没有定义 16 位格式，所以 ILM 创建了 half 格式，以匹配 NVIDIA 的 16 位格式。half 类型给高动态范围图像提供了很好的结构存储。OpenEXR 格式直接支持这个类型。

16 位，或称“半精度”浮点格式，建模于 IEEE 754 的单精度和双精度格式之后。半精度数字由一个 1 位符号、一个 5 位指数和一个 10 位尾数组成。保留最小的和最大的指数值是为了表现 0、不规格数，无限大数和 NaN。

在 OpenEXR 的 C++实现中，half 类型的数通常类似于 C++内建的浮点类型，float 和 double。half、float 和 double 类型可以在算术表达中自由混合。这里是一些例子。

```
half a (3.5);
float b (a + sqrt(a));
a += b;
b += a;
b = a + 7;
```

26.1.3　可表示的数值范围

半精度最明显的好处是只用 16 位就能表示数值的范围。我们可以存储的最大图像值是 65 504.0，而最小的值是 5.96^{-8}。这是从兆到 1 的动态范围。极少有图像需要这样大的范围，但是确实存在范围百万到 1 的图像，这样用半精度可以很容易地表示它们。

摄影师用 stops 测量动态范围，一个 stop 是 2 的因数。在 5 个 stops 范围的图像中，最明亮的区域比最黑暗的区域亮 32 倍。百万到 1 的范围在照相术语中是 20 个 stops。

26.1.4 彩色分辨率

半精度的第2个好处是彩色分辨率。每个stop都包含1024个清晰的值，这使得OpenEXR能很好地用于正常动态范围的图像。在范围从1.0降到0.0039（或2^{-8}）的8个stop的图像中，半精度格式会为每个通道提供8192个值，尽管一个8位、亮度2.2的图像只有235个值（21到255）。

26.1.5 C++接口

为了能容易地写和读OpenEXR文件，ILM设计了带有C++程序接口的文件格式。OpenEXR提供3种级别访问图像的接口文件：（1）通用接口，用于写和读任意图像的通道文件；（2）RGBA的专用接口（红、绿、蓝和alpha通道，或它们的子集）；（3）C可以调用的接口，支持在C语言写成的程序中读和写OpenEXR文件。

这章的例子使用的是RGBA C++接口。

26.2 OpenEXR文件结构

一个OpenEXR文件有两个主要部分：文件头和像素。

26.2.1 文件头

文件头是描述像素属性的列表。属性是任意类的被命名的数据项。为了能够用某个程序读其他程序写的OpenEXR文件，某些必要的属性必须出现在所有OpenEXR文件的头中。这些属性列在表26-1中。

表26-1 在OpenEXR头中的必要属性

属 性 名	描 述
displayWindow	图像分辨率[①]
dataWindow	修剪和开始
PixelAspectRatio	当图像以正确的屏幕高宽比显示的时候，像素的宽度和高度比
channels	描述存储在文件中的图像通道
compression	文件中所有通道的像素数据的压缩方法
lineOrder	在文件中存储扫描线的次序（y增加或y减小）
screenWindowWidth,screenWindowCenter	产生图像的透视投射

注① 细节见OpenEXR的网站：www.openexr.org。

除了必要的属性之外，程序可以在文件的头中包括选择性属性。有时必须用其他的数据注释图像，这些数据是针对特别应用的，例如计算历史、颜色描绘信息，或摄像机位置和视线方向等。作为在图像文件头中的额外属性，可以把这些数据打包。

26.2.2 像素

把一个图像的像素存储在独立的通道中。用通用的库接口，可以写RGBA通道和必需的其他许多通道。各个通道可以使用不同的数据类型，因此RGBA数据可以是half（16位），

而 z-深度通道可以写成 float（32 位）。

26.3　OpenEXR 数据压缩

OpenEXR 提供 3 种不同的数据压缩方法，在速度与压缩比的折中方面各有不同。如表 26-2 所列，3 个压缩方案分别为无损的、压缩的和不压缩的（不改变像素数据）。

表 26-2　OpenEXR 所支持的压缩选项

名　称	描　述
PIZ	将小波变换应用于像素数据，结果是 Huffman 编码的。这个方案给照片的图像提供最好的压缩比。文件的压缩和解压缩速度相同。对于电影胶片粒度的照片图像，文件减小为未压缩尺寸的 35%～55%
ZIP	使用开放源码 zlib 库压缩水平相邻像素之差。ZIP 解压缩的速度比 PIZ 快，但是 ZIP 的压缩速度慢得多。被压缩的照片图像时常是它们未压缩尺寸的 45%～55%
RLE	水平相邻像素之差的游程周期编码。这个方法快，而且对大而平的图像工作得好。然而，对于照片图像，被压缩的文件尺寸通常是未压缩文件的 60%～75%

可以选择不压缩的形式存储像素。如果存储在快速文件系统上，不压缩形式文件的读和写明显比压缩过的文件更快速。

26.4　OpenEXR 的使用

本书所附的电子材料包含简单图像重放程序的完整源代码，以及它的 Cg shader 代码。这节提供代码的摘录，示范如何读一个 OpenEXR 图像文件，用 OpenGL 显示图像缓冲，演示简单的合成操作，而且将结果写成一个 OpenEXR 文件。

26.4.1　OpenEXR 图像的读和显示

列表 26-1 的简单代码把一个 OpenEXR 文件读进内部图像缓冲。OpenEXR 库使用异常，所以像“未发现文件”的错误是在 catch 块中处理的。不需要显式地关闭文件，因为 C++析构函数会为 RgbaInputFile 对象关闭它。

列表 26-1　读一个 OpenEXR 图像文件

```
Imf::Rgba * pixelBuffer;

try
{
  Imf::RgbaInputFile in(fileName);

  Imath::Box2i win = in.dataWindow();

  Imath::V2i dim(win.max.x - win.min.x + 1, win.max.y - win.min.y + 1);

  pixelBuffer = new Imf::Rgba[dim.x * dim.y];

  int dx = win.min.x;
```

```
    int dy = win.min.y;

    in.setFrameBuffer(pixelBuffer - dx - dy * dim.x, 1, dim.x);
    in.readPixels(win.min.y, win.max.y);
  }
  catch(Iex::BaseExc & e)
  {
    std::cerr << e.what() << std::endl;

    //
    // 异常处理
    //
  }
```

缓冲一旦被填充，列表 26-2 的代码就把图像与纹理绑定在一起，目的是用 Cg 片元 shader 进行显示（更多的细节见第 26.5 节）。完整的程序调用显示代码来播放多帧。

列表 26-2 把图像与纹理绑定在一起

```
GLenum target = GL_TEXTURE_RECTANGLE_NV;

glGenTextures(2, imageTexture);
glBindTexture(target, imageTexture);

glTexParameteri(target, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(target, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

glTexImage2D(target, 0, GL_FLOAT_RGBA16_NV, dim.x, dim.y, 0,
             GL_RGBA, GL_HALF_FLOAT_NV, pixelBuffer);

glActiveTextureARB(GL_TEXTURE0_ARB);
glBindTexture(target, imageTexture);

glBegin(GL_QUADS);
glTexCoord2f(0.0, 0.0);         glVertex2f(0.0, 0.0);
glTexCoord2f(dim.x, 0.0);       glVertex2f(dim.x, 0.0);
glTexCoord2f(dim.x, dim.y);     glVertex2f(dim.x, dim.y);
glTexCoord2f(0.0, dim.y);       glVertex2f(0.0, dim.y);
glEnd();
```

26.4.2 一个 OpenEXR 图像的渲染和写入

GPU 所渲染的图像能存储在 OpenEXR 文件中。我们时常首先创建前景图像和背景图像，然后把它们结合起来。列表 26-3 中的例子使用 pbuffer 做简单的合成操作，然后把结果写入 OpenEXR 文件。在代码片段中，我们把两个输入图像（已经打开）读进已分配的图像缓冲之内，然后调用合成程序并写入结果。

列表 26-3　把两个图像合成并写为一个 OpenEXR 文件

```
//
// Read A and B.
//

Imath::V2i dim(dataWinA.max.x - dataWinA.min.x + 1, dataWinA.max.y - dataWinA.min.y + 1);
int dx = dataWinA.min.x;
int dy = dataWinA.min.y;

Imf::Array<Imf::Rgba> imgA(dim.x * dim.y);
Imf::Array<Imf::Rgba> imgB(dim.x * dim.y);

inA.setFrameBuffer(imgA - dx - dy * dim.x, 1, dim.x);
inA.readPixels(dataWinA.min.y, dataWinA.max.y);

inB.setFrameBuffer(imgB - dx - dy * dim.x, 1, dim.x);
inB.readPixels(dataWinB.min.y, dataWinB.max.y);

//
// Do the comp, overwrite image B with the result.
//

Comp::over(dim, imgA, imgB, imgB);

//
// Write comp'ed image.
//
Imf::RgbaOutputFile outC(outputFilename.c_str(),dpyWinA, dataWinA, Imf::WRITE_RGBA);

outC.setFrameBuffer(imgB - dx - dy * dim.x, 1, dim.x);
outC.writePixels(dim.y);
```

调用 Comp::over 的实现很简单，像调用列表 26-4 显示的例程（Comp::comp）一样，后者有一个 Cg shader 的名字，执行一个 over 操作。over 操作和其他各种合成操作的全部细节可以在 Porter and Duff 1984 找到。GlFloatPbuffer 类（在附带的材料中提供）隐藏着创建和删除浮点 pbuffer 的机理。注意，我们能以不损失精度、不钳制的 glReadPixels 从 pbuffer 得到半精度的数据。

列表 26-5 到 26-7 分别是 over 操作和输入、输出操作的 Cg shaders。

列表 26-4　合成为一个 Pbuffer

```
void
Comp::comp(const Imath::V2i & dim,
           const Imf::Rgba * imageA,
           const Imf::Rgba * imageB,
           Imf::Rgba * imageC,
           const char * cgProgramName)
{
  GlFloatPbuffer pbuffer(dim);
```

```
pbuffer.activate();

//
// Set up default ortho view.
//
glLoadIdentity();
glViewport(0, 0, dim.x, dim.y);
glOrtho(0, dim.x, dim.y, 0, -1, 1);

//
// Create input textures.
//
GLuint inTex[2];
glGenTextures(2, inTex);

GLenum target = GL_TEXTURE_RECTANGLE_NV;

glActiveTextureARB(GL_TEXTURE0_ARB);
glBindTexture(target, inTex[0]);

glTexParameteri(target, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(target, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

glPixelStorei(GL_UNPACK_ALIGNMENT, 1);
glTexImage2D(target, 0, GL_FLOAT_RGBA16_NV, dim.x, dim.y, 0, GL_RGBA, GL_HALF_FLOAT_
             NV, imageA);

glActiveTextureARB(GL_TEXTURE1_ARB);
glBindTexture(target, inTex[1]);

glTexParameteri(target, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
glTexParameteri(target, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(target, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);

glPixelStorei(GL_UNPACK_ALIGNMENT, 1);

glTexImage2D(target, 0, GL_FLOAT_RGBA16_NV, dim.x, dim.y, 0, GL_RGBA, GL_HALF_FLOAT_
             NV, imageB);

//
// Compile the Cg program and load it.
//

cgSetErrorCallback(cgErrorCallback);

CGcontext cgcontext = cgCreateContext();
CGprogram cgprog = cgCreateProgramFromFile(cgcontext, CG_SOURCE, cgProgramName,
                                           CG_PROFILE_FP30, 0, 0);
```

```
    cgGLLoadProgram(cgprog);
    cgGLBindProgram(cgprog);
    cgGLEnableProfile(CG_PROFILE_FP30);

    //
    // Render to pbuffer.
    //

    glEnable(GL_FRAGMENT_PROGRAM_NV);
    glBegin(GL_QUADS);
    glTexCoord2f(0.0, 0.0);       glVertex2f(0.0,   0.0);
    glTexCoord2f(dim.x, 0.0);     glVertex2f(dim.x, 0.0);
    glTexCoord2f(dim.x, dim.y);   glVertex2f(dim.x, dim.y);
    glTexCoord2f(0.0, dim.y);     glVertex2f(0.0,   dim.y);
    glEnd();
    glDisable(GL_FRAGMENT_PROGRAM_NV);

    //
    // Read pixels out of pbuffer.
    //

    glReadPixels(0, 0, dim.x, dim.y, GL_RGBA, GL_HALF_FLOAT_NV, imageC);

    checkGlErrors();

    pbuffer.deactivate();
}
```

列表 26-5　over 操作的 Cg shader

```
void
over(float2 wpos : WPOS,
     out half4 c : COLOR,
     uniform float2 dim,
     uniform samplerRECT A,
     uniform samplerRECT B)
{
  half4 a = texRECT(A, wpos.xy);
  half4 b = texRECT(B, wpos.xy);

  c = a +(1 - a.a) * b;
}
```

列表 26-6　in 操作的 Cg shader

```
void
in(float2 wpos : WPOS,
   out half4 c : COLOR,
   uniform float2 dim,
   uniform samplerRECT A,
   uniform samplerRECT B)
{
```

```
  half4 a = texRECT(A, wpos.xy);
  half4 b = texRECT(B, wpos.xy);

  c = b.a * a;
}
```

列表 26-7 out 操作的 Cg shader

```
void
out(float2 wpos   : WPOS,
    out half4 c   : COLOR,
    uniform float2 dim,
    uniform samplerRECT A,
    uniform samplerRECT B)
{
  half4 a = texRECT(A, wpos.xy);
  half4 b = texRECT(B, wpos.xy);

  c = (1 - b.a) * a;
}
```

26.5 线性像素值

最好把线性图像存储在 OpenEXR 文件中。线性这个词对不同的人有不同的意义，对我们来说，如果图像中的值与相关场景的亮度成正比，那么认为图像是在线性彩色空间中。当数值加倍时，光照也加倍。若想使场景亮度减半，就是让它变暗一个 stop，即把每个图像值除以 2。如后面所述，这说明了应该如何显示图像。

大多数图像处理算法假设图像是线性的。标准合成运算基于一个 alpha 通道 over 操作前景和背景的线性混合。当 alpha 真的是覆盖掩码（coverage mask）时，它指出应该从背景泄漏出的光照百分比。当图像表现为线性的光照时，图像处理能正确地进行。除了图像处理之外，最好在线性空间中进行反走样（Blinn 2002）。

软件渲染通常在线性空间中执行。如果在线性空间中做硬件渲染，则可以在 GPU 中得到高质量渲染。当图像可观看时，它必须经过亮度校正。虽然亮度在图形业中有许多意思，但是这里指的是对监视器亮度的校正。监视器的显示电路所输出的光强与帧缓冲区数值呈指数关系。因为假定 OpenEXR 图像是线性的，得把监视器的亮度告诉显示程序，于是程序能反向应用亮度到图像以便显示。

典型的监视器亮度为 2.2。可以用一个光度计测量它。如果没有光度计，见 Berger 2003 所描述的方法。当显示 1.0 的白色值时，仪表显示的数可能是 92nit（1 个 nit 等于每平方米 1candela）对于第 2 个数据点，显示灰色的斑点（黑色对于光读数有问题，所以不使用黑色斑点；也就是说，我们正在使用对数，0 是无意义的）。

假定 0.5 灰度值给出一个 20 个 nit（尼特）的亮度。这个 0.5 是帧缓冲区的数字值，在数字到模拟转换之前没有进行其他处理。监视器的输出亮度 $L(v)$是输入数值 v 的函数：

$$L(v) = L_m \cdot v^{\gamma}$$

这里 v 的量程从 0.0 到 1.0，L_m 是监视器的最大输出亮度。从这可以推导 gamma(γ)为：

$$L(1.0) = L_m \cdot 1.0^{\gamma} = L_m = 92\text{nit}$$

$$L(0.5) = L_m \cdot 0.5^{\gamma} = 92\text{nit} \cdot 0.5^{\gamma} = 20\text{nit}$$

$$\frac{20\text{nit}}{92\text{nit}} = 0.5^{\gamma}$$

$$\gamma = \log_{0.5}\frac{20}{92} \approx 2.2$$

为了把灰阶修正应用到线性图像数据，我们只是把图像数值上升为监视器灰阶上的 1 的幂。在列表 26-8 中的简单片元 shader 中显示了这一点。

列表 26-8　为了显示灰阶修正图像

```
struct Out
{
  half4 color : COLOR;
};

Out
main(float2 texCoord : TEXCOORD0,
     uniform samplerRECT image,
     uniform float gamma)
{
  half4 pixel = h4texRECT(image, texCoord);
  Out output;
  output.color = pow(pixel, gamma);
  return output;
}
```

重放程序让用户改变图像的曝光量。图像在亮度修正之前是线性的，因此，我们仅乘一个常数并把结果带入幂函数，见列表 26-9。

列表 26-9　调整图像的曝光量

```
Out
main(float2 texCoord : TEXCOORD0,
     uniform samplerRECT image,
     uniform float gamma,
     uniform float expMult)
{
  half4 pixel = h4texRECT(image, texCoord);
  pixel.rgb *= expMult;
  Out output;
  output.color = pow(pixel, gamma);
  return output;
}
```

最后，图像可能需要一些全面的彩色校正。这样做可能是为了美学，或让图像看起来好像在不同的装置上显示。举例来说，如果想要监视器上的图像看起来像胶片一样，可以在灰

阶修正之前执行胶片模拟。为了得到正确的结果，数字图像设计者经常使用照明技巧。相反地，我们更喜欢在线性空间做照明和渲染，模拟胶片。在列表 26-10 显示的例子中，我们提供一个 3 通道的查询表，它包含一个简单的 S 形胶片模拟曲线。

列表 26-10 使用一个查询表模拟照相胶片的样子

```
half
lookup(samplerRECT lut, half h)
{
  float f = pack_2half(h.xx);
  half4 bytes = unpack_4ubyte(f);
  return h1texRECT(lut, bytes.xy * 256.0);
}

Out
main(float2 texCoord : TEXCOORD0,
     uniform samplerRECT image,
     uniform samplerRECT lutR,
     uniform samplerRECT lutG,
     uniform samplerRECT lutB,
     uniform float gamma,
     uniform float expMult)
{
  half4 c = h4texRECT(image, texCoord) * expMult;
  c.r = lookup(lutR, c.r);
  c.g = lookup(lutG, c.g);
  c.b = lookup(lutB, c.b);
  Out output;
  output.color = pow(c, gamma);
  return output;
}
```

这个 shader 使用一个技巧执行查询函数。用作查询表的 samplerRECT 类，为每个可能的半精度输入存储一个 half。最简单的代码是 1 × 66536 的 sampler rect。因为目前的 GPU 的限制，这个 rect 太大。于是改为，使用 Cg 标准程序库函数 pack_2half 和 unpack_4ubyte 进行索引，把查询表封入一个 256 × 256 个纹理内。这样就是取给定的半精度输入值，并且把它分成 256 × 256 查询表的两个索引。

在书的 CD 和网站上，提供了这些程序的完全注释版本，以及为绑定各种纹理和图像显示的 OpenGL 代码。

26.6 创建和使用 HDR 图像

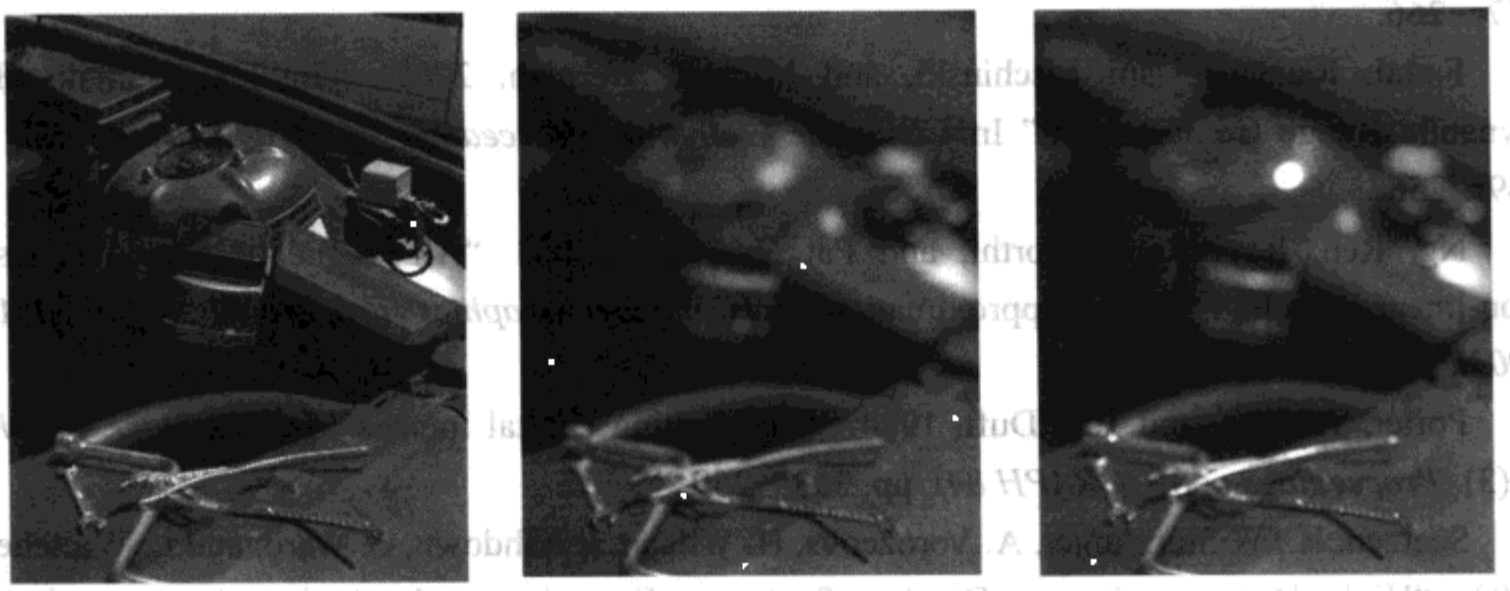
HDR 图像可以在 GPU 上或在软件中合成创建。它们也能从现实世界得到。确实存在高动态范围的摄像机。例如，Spheron 的 SpheroCam 可以取得动态范围为 26 个 stop 的全景。如果用一般的摄像机，可以取多个曝光量，使用 Greg Ward 的 PhotoSphere 或 Paul Debevec 的 HDRShop 等软件，以构造单个的 HDR 图像。

求 3D 场景中的光照明，可以用高动态图像的全局照明模型，而不是单独的光源。在

Debevec 1998 中，用 Radiance 软件渲染合成的物体。在 Ng 等 2003 中，在 NVIDIA 硬件上实时地得到漫反射表面的逼真渲染，还包括阴影。这两个方法使用环境贴图代替光源；贴图必须有足够宽的动态范围，能在环境中表现最明亮的区域（举例来说，太阳），没有夹持。

图像处理也从宽的动态范围得到好处。举例来说，景深模糊把个别像素扩展为大的面积。当个别像素非常明亮的时候，扩展它应该造成很多适度明亮的像素。但是如果一个明亮的像素，例如镜面反射的高光，在模糊之前已经被钳制为 0 到 1，那么模糊的高光就会不自然的暗淡。相似的效果也可以在运动模糊中看到。甚至进行简单的彩色校正也需要高动态范围数据，如第 26.1.1 节所述。

在图 26-4（a）中，有一个汽车引擎的高动态范围图像。高光区范围比 1.0 高得多。对于图 26-4（b），我们首先将图像转换成低动态文件格式，把像素钳制为 0 到 1。然后应用景深模糊。在图 26-4（c），我们对从 OpenEXR 文件读出数据的整个范围进行模糊。



（a）　（b）　（c）

图 26-4　模糊非常明亮高光的图像

（a）原始的 HDR 图像；镜面反射高光区的亮度超过 1.0；
（b）通过把图像（a）存储成低动态范围文件，它被钳制为 0 到 1，然后模糊它。模糊的高光区暗得不自然；
（c）对未钳制的图像（a）执行模糊。模糊的高光区仍保持明亮

在前面的章节中，我们假定显示 HDR 图像的时候，高于 1.0 的值被钳制。这个可以工作得很好，本章中的图片印刷就用了这种方法。但是，还可能做得更好。近几年已经发表一种称为色调映射（tone mapping）的有效技术，可以不使图像明显改变而减小动态范围（例如，Fattal et al. 2002 以及 Durand and Dorsey 2002）。另一种选择是，让未来的显示器直接显示 HDR 图像。例如，Seetzen 等人 2003 年介绍的一种动态范围为 120 000 到 1 的 HDR 显示器模型。

26.7　小结

OpenEXR 原来是为了 GNU/Linux 和 Irix 发表的，而经过开放源码界的努力，它被移植到 Mac OS X 和 Windows。OpenEXR 库已经在 Industrial Light & Magic 的视觉效果制造中得以验证。

除了 NVIDIA 之外，愈来愈多的厂商在他们的软件中支持 OpenEXR 格式。对于更多的信息，参见 www.openexr.org 的 OpenEXR 网站。

26.8 参考文献

Berger, Robert W. 2003. "Why Do Images Appear Darker on Some Displays?" Article on Web site. **http://www.bberger.net/rwb/gamma.html**

Blinn, Jim. *2002. Jim Blinn's Comer: Notation, Notation, Notation,* pp. 133～146. Morgan Kaufmann.

Debevec, Paul. 1998. "Rendering Synthetic Objects into Real Scenes." In *Computer Graphics (Proceedings of SIGGRAPH 98),* pp. 189～198.

Durand, Fredo, and Julie Dorsey. 2002. "Fast Bilateral Filtering for the Display of High-Dynamic-Range Images." In *Computer Graphics (Proceedings of SIGGRAPH 2002),* pp. 257～266.

Fattal, Raanan, Dani Lischinski, and Michael Werman. 2002. "Gradient Domain High Dynamic Range Compression." In *Computer Graphics (Proceedings of SIGGRAPH 2002),* pp. 249～256.

Ng, Ren, Ravi Ramamoorthi, and Pat Hanrahan. 2003. "All-Frequency Shadows Using Non-linear Wavelet Lighting Approximation." In *Computer Graphics (Proceedings of SIGGRAPH 2003),* pp. 376～381.

Porter, Thomas, and Tom Duff. 1984. "Compositing Digital Images." In *Computer Graphics* 18(3) *(Proceedings of SIGGRAPH 84),* pp. 253～259.

Seetzen, H., W. Sturzlinger, A. Vorozcovs, H. Wilson, I. Ashdown, G. Ward, and L. Whitehead. 2003. "High Dynamic Range Display System." Emerging technologies demonstration at SIGGRAPH 2003.

第 27 章　图像处理的框架

Frank Jargstorff
NVIDIA

这章将介绍在 GPU 上处理图像的一个 C++框架。使用这个框架，程序员能容易地定义图像过滤器和连接过滤器，从而形成过滤器图表。框架中大部分的类只是作为在图形板上活动结构的句柄，所以图像的实际处理几乎完全由 GPU 执行。框架基于 OpenGL API（应用编程接口）和用 Cg 语言执行处理图像的算法。本章的代表性思路同样也适用于微软的 DirectX API 和 HLSL 程序设计语言。我们希望这个框架对正在设计相似系统的读者有帮助，可以把它作为工作基础，或是沿着这个思路对问题进行分析。

27.1　引言

基于两个原因我们的框架是有价值的。首先，最新 GPU（如 GeForce FX 家族）所支持的顶点和片元 shader 灵活性正日益增加；其次，今天的 GPU 非常强大，它们在许多信号处理的应用方面超过了 CPU。

使用过滤器图表组织图像过滤器，特别适用于处理视频，因为它使用相同配置的过滤器处理很长序列图像（例如，微软的 DirectShow 是基于过滤器图表的、实时处理视频的库），见图 27-1。在批处理场合，过滤器图表也是有用的，例如合成电影质量的帧，或为网络出版处理大批图像（例如，Apple 的 Shake 排版或 Adobes ImageReady）。

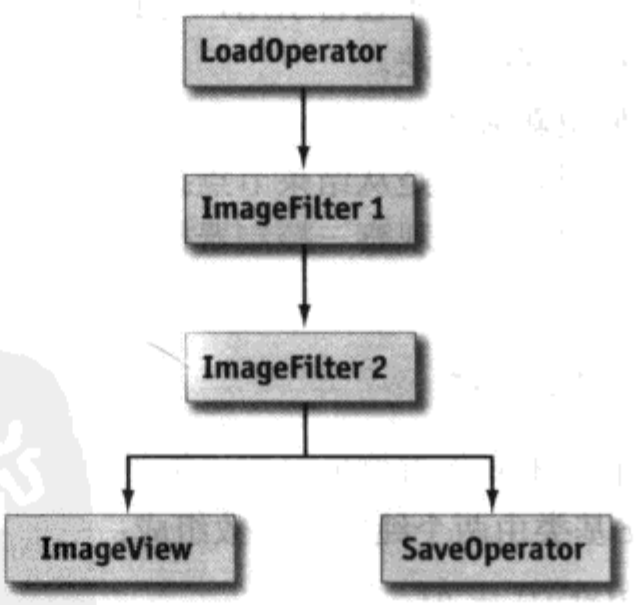


图 27-1　过滤器图表

GPU 是经过传统的 3D 应用编程接口（API）来编程的，例如 Direct3D 和 OpenGL 等。这些应用编程接口是低级的，最初的设计目的是实现 3D 应用。直接在这些编程接口中执行 GPU 的图像处理是很麻烦的。为图像处理专门设计的作为隔绝层使用的框架，极大地简化了 GPU 上的图像处理。为了简化程序，框架应该压缩所有的重复试验，而且尽可能使程序员避开基本图形应用编程接口。在下面几节中，我们介绍这种框架的基本版本。框架的作用是作为例子，用以说明最重要的设计和实现要点。然而它并没有谈到许多比较复杂的方案，这些方案对工业强度的库是必需的，例如把图像分为块（tiles），以便处理任意大小的图像。

27.2 框架设计

过滤器网络处理以下两类对象：

- 流过网络的数据。 在我们的情况下，数据流是图像。
- 运算数据的操作。存在不同的操作。有些操作没有接收输入图像就产生输出图像（例如，从磁盘下载图像的操作）。这些操作是网络的来源，因此它们叫做源操作。其他的操作消耗输入图像，却没有产生输出图像（例如，把图像显示在屏幕上的操作）。因为这些操作的作用像数据接收器，我们称它们为接收器操作，见图 27-2。

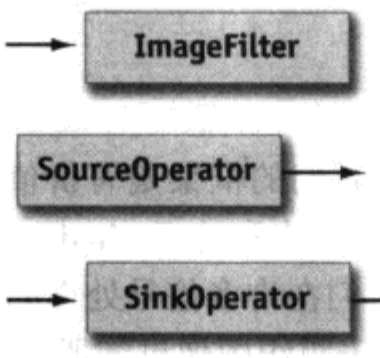


图 27-2 一个接收器操作、一个源操作和一个图像过滤器

对于 C++的实现，涉及两个问题。第一，好的设计对象应该能够产生物体，也能负责删除那些物体（Meyers 1997）。因为创建和删除图像的操作通常是不同的，所以我们需要某种对图像类的自动化内存管理。

图像是非常简单的数据，不包含引用，尤其不包含对它们本身的引用。因此简单的引用计数足可以作为内存管理的办法。

第二是应该如何更新流水线。也就是说，是什么机构使数据通过图表的过滤？有两个选择，一是把数据推出图表，一是从结果节点拉入数据，即依次从输入数据拉入必要的数据。因为对于应用通常使用模型－观察－控制器模式（Gamma et al. 1995），我们通过拉入数据实现更新。

27.2.1 操作器和过滤器

上节中所描述的基本操作直接地对应于框架设计的抽象基类。

SourceOperator 基类由两个纯虚拟函数组成：

```
virtual bool dirty() = 0;
virtual Image image() = 0;
```

image()方法获得操作器的输出图像。dirty()方法决定最新获得的图像是否仍然有效。SinkOperator 基类只包含一个虚拟函数和一个对 SourceOperator 的被保护引用。

```
virtual void setSourceOperator(SourceOperator * pSourceOperator);

protected:
  SourceOperator * _pSourceOperator;
```

接收器操作从_pSourceOperator 所指向的源操作找到它的输入图像。setSourceOperator()方法设定这个引用。

ImageFilter 类由 SourceOperator 和 SinkOperator 推导出来，如图 27-3 所示。

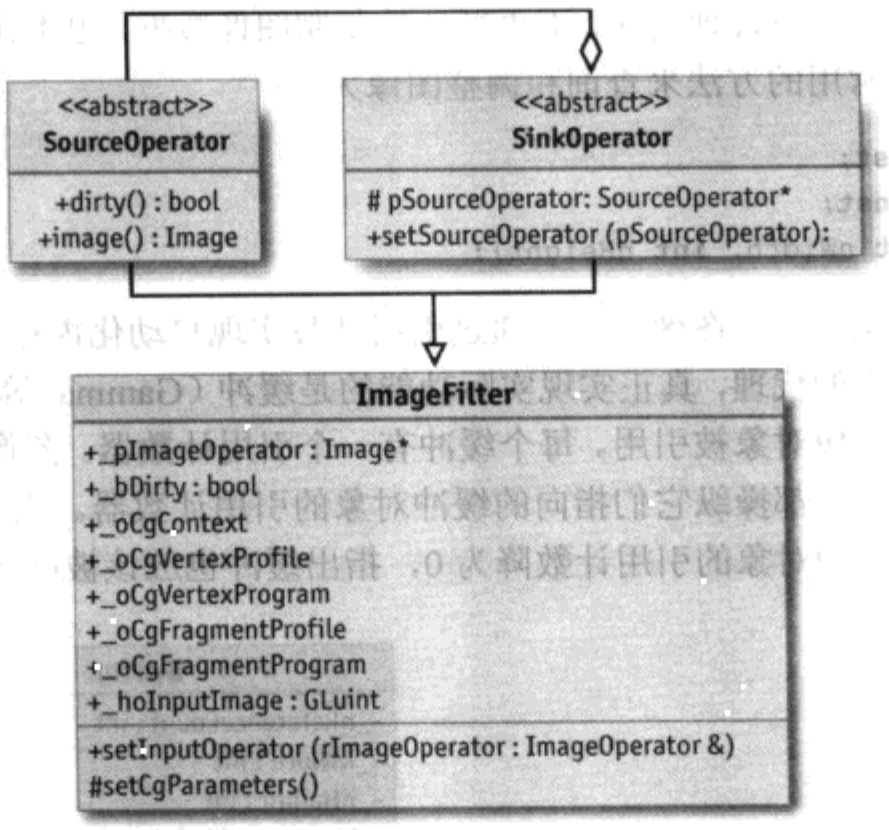


图 27-3　ImageFilter 类图

对它所推导的所有过滤器，ImageFilter 用默认执行使 image()和 dirty()方法超载。

- image()方法从输入操作找到输入图像，并画一个完全覆盖整个输出图像的长方形。画长方形的代码建立 Cg 顶点和片元程序，并使用输入操作的图像作为一个纹理。片元程序决定如何把纹理渲染到输出图像。因此，通过简单地使用不同 Cg 片元程序，可以执行不同图像过滤器。
- dirty()方法向上递归地查询所有操作的 dirty()方法。如果发现查询的图像过滤取决于 dirty 操作的输出，查询返回到真。某个上游操作的任何参数改变时，该操作就变污染了。一个操作器或过滤器实施的责任，就是保证正确地设定 dirty 标记。

为了建立及执行片元和顶点程序，image()方法执行所需要的一切。它使用 ShaderManager 类重新建立共享的 Cg 资源。大部分必需但乏味的 Cg 流水线压缩在图像方法中。一个新过滤器类所必须进行的所有执行，就是在类的结构中为过滤器建立实际的片元 shader。

27.2.2　图像数据

图像类的作用正如图像数据的容器。因此，它必须支持两个基本功能：一是找到图像数

据，二是把新图像数据画进图像。因为图像类是对图形板上实际图像的起码操作，对这些功能的支持是高级别的（与之相反的是低级访问，例如设定和找到单个像素的颜色值）。

图像数据的高级接口由以下 3 个方法组成：

```
GLuint textureID() const;
void renderBegin();
void renderEnd();
```

这些方法允许程序员寻找一个 OpenGL 纹理句柄，它指向包含图像数据的纹理。

这个渲染方法制造当前 OpenGL 渲染场景的图像，所以在两个指令之间发出的任何 OpenGL 画图指令，将直接地画图像。使用几个 OpenGL 扩展时，有可能执行这个接口，使得画入图像和把它重复作为纹理使用时，不再循环地复制图像数据，也不让图形卡内存这样做。

此外，图像类有常用的方法来查询和调整图像大小：

```
int width() const;
int height() const;
void setSize(int nWidth, int nHeight);
```

我们在本节的开头指出，图像类必须通过引用计数实现自动化内存管理。事实上，图像类是一个称作缓冲的类的代理，真正实现实际功能的是缓冲（Gamma 等，1995）。见图 27-4。缓冲对象只可能通过图像对象被引用。每个缓冲有一个引用计数器。图像对象的构造、破坏、副本构造和操作安排，全都操纵它们指向的缓冲对象的引用计数器。当引用缓冲的最后一个图像被破坏的时候，缓冲对象的引用计数降为 0，指出缓冲也应该被破坏。

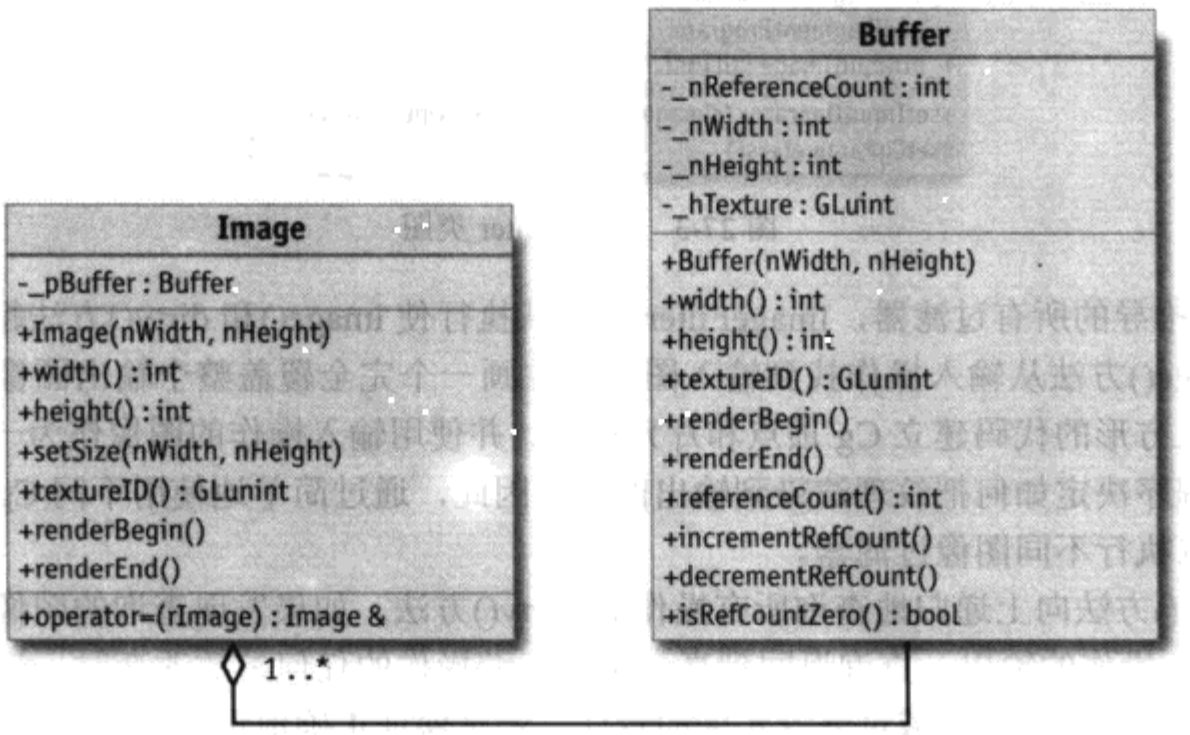


图 27-4 图像类和缓冲类的框图

27.2.3 丢失的块

我们现在有了数据类型和一堆描述过滤器图表的抽象类。但是为了真正实现一个应用，我们需要图像输入和输出的具体手段。一个简单的应用是从磁盘下载图像，可能还应用一个过滤器，并把结果显示在屏幕上。为了这样做，我们需要一个下载磁盘的操作和把图像渲染

到屏幕的显示操作。

LoadOperator 显然是一个数据源，是从源操作（Source-Operator）类得来。下载操作取两个参数：一个文件名（包括路径）和一个文件类型的标识符。操作器从文件读数据，并创建一个图像实例。操作器把这个图像完全地返回它的 image()方法的每个程式。只有当文件名或类型改变的时候，操作器才再从磁盘下载新的图像，见图 27-5。

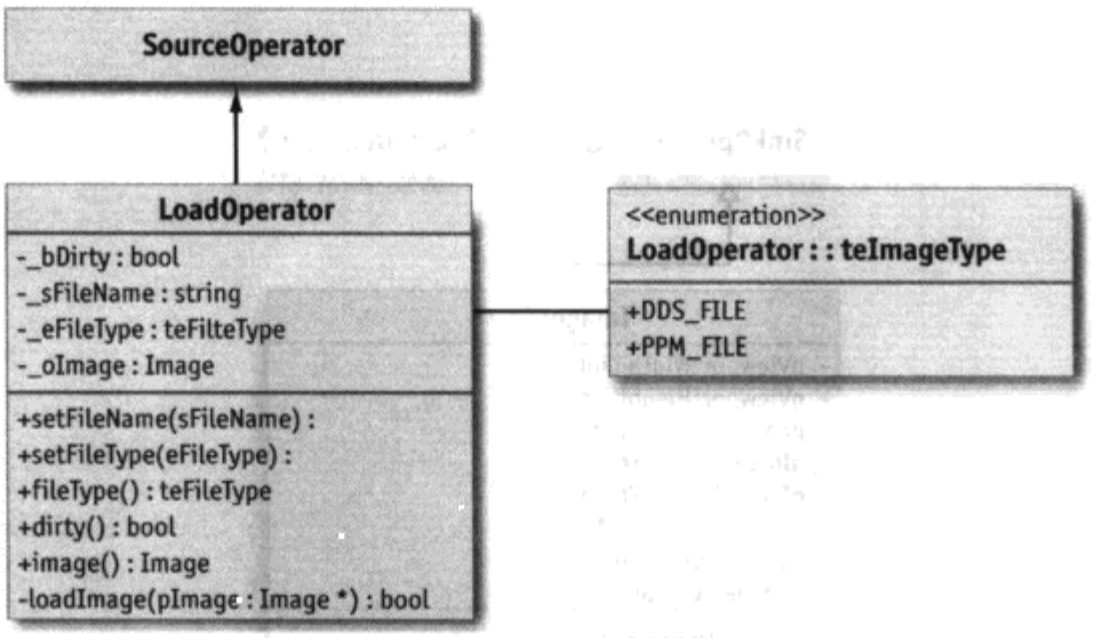


图 27-5　LoadOperator 类图

ImageView 类显示所处理的图像。这个类也负责让用户调节图像尺寸（放大和缩小），并在视口中来回移动图像。因为观察只消耗加工的图像，而不对流水线产生任何输出，所以它来自 SinkOperator 类。

支持这个类的方法如下：

```
int   imagePositionX() const;
int   imagePositionY() const;
void  setImagePosition(int nX, int nY);
float zoomFactor() const;
void  setZoomFactor(float nZoomFactor);
```

对位置及放大因子的设置和取得无须解释。

更有趣的是下列两个方法：

```
void center();
void changeZoomFactor(float nPercent);
```

center()方法（中心）把当前的图像放在视口中央。changeZoomFactor(nPercent)方法允许程序员按照某个百分比逐渐地改变放大因子。因为它允许基于当前尺寸进行调节，而无须询问信息，所以是很方便的。

为了在用户改变窗口大小时仍然能正确地显示图像，ImageView 需要更新视口的转换。通过一个简单函数把任何视口尺寸的变化通知给 ImageView 做到这点。程序员必须确保应用程序把任何视口类的变化通知给观众。

```
void reshape(int nViewportWidth, int nViewportHeight);
```

显示视口由下列两个方法实现：

```
void display();
void update();
```

display()方法只是渲染当前存储在视图中的图像。流水线处理不是由 display()的代码启动的。存储在视图中的图像使用 update()方法来更新。这个方法只是从流水线取回一个新的图像，见图 27-6。

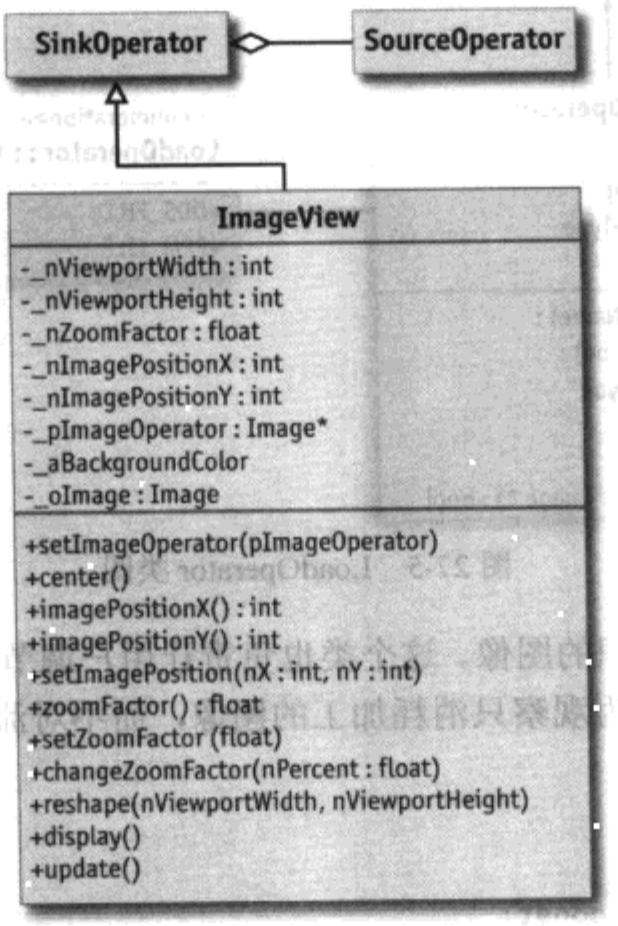


图 27-6 ImageView 类图

27.3 实现

处理一个图像是非常简单和直接的。过滤器把与屏幕取齐的方块渲染进一个不可见像素缓冲区。与屏幕取齐的方块有约束为纹理的输入图像。对每个被渲染的像素，执行一个 Cg 片元程序，由它做实际的图像处理。因为使用作为纹理的输入图像来渲染方块，Cg 程序可以通过纹理读出访问输入图像中的像素。

这就是实现新的图像过滤非常简单的原因。它由写入一个新 Cg 片元程序和一些 C++编码构成，目的是把必要的（uniform）参数提供给 Cg 程序。

要保证图像不进行没必要的复制，也不在系统内存和图形卡之间移动，所有的魔法都隐藏在图像类的实现中。

基本机理是把所处理的图像渲染进一个不可见缓冲内。渲染一经完成，图像就可以通过流水线的下一个过滤器进行处理，为了下一条渲染路径，不可见缓冲的内容需要约束到一个纹理。

27.3.1　Image 类

这个渲染进不可见缓冲和使用该缓冲作为纹理的机理，可以在 OpenGL 中使用两个 ARB 扩展 WGL_ARB_pbuffer 和 WGL_ARB_render_texture 来实现。

像素缓冲（或使用 WGL_ARB_pbuffer 扩展创建的 pbuffers）是 OpenGL 的不可见缓冲。可见帧缓冲所能做的一切，像素缓冲也都可以做。既然可以渲染进图形板上不可见缓冲，我们就需要找到一个方法把这个缓冲作为纹理使用。恰好那就是渲染到纹理的扩展所允许我们做的事情（WGL_ARB_render_texture）。

为了使事情变得更有趣，我们支持非 2 的幂（NPOT）的纹理，因为典型的视频和图像分辨率不是 2 的乘方。OpenGL 通过 NV_texture_rectangle 扩展支持非 2 的幂（NPOT）的纹理。

现代 GPU 日益支持浮点数据和彩色值算法。浮点数在表达彩色方面比定点数更优越。在线性彩色空间，浮点数沿着数值大小的不均匀分布与人类的知觉系统一致。浮点数在 0 附近非常密集，随着数字变大，在连续值之间的增量也相应变大。类似地，我们的知觉系统在低强度区可以识别小的强度变化，在高强度区就逐渐丧失敏感性。当用浮点算术用于图像处理的时候，对过程的任何中间步骤可以不把值钳制为 1 到 0 的范围。只有对非常靠后的步骤才必须把动态范围“规范化”为固定间隔。在最容易的情况下，这是简单的仿射映射。但是确实存在精细的色调映射方案，这时基于局部的强度分割图像，而且有差别地映射每个片段。这种算法通常避免把图像的暗区映射为黑色，也避免把亮区映射为白色。

NVIDIA 硬件支持两种类型的浮点数：浮点精度（IEEE 32-bit 浮点）和半精度。为了实现这个框架，我们使用半精度数据类型作彩色表达。半精度比浮动精度优越的地方如下。

- 在速度和精度之间有很好的折中。
- 半精度数比 32bit 浮点精度数占用内存少。
- OpenEXR 图像格式使用与 NVIDIA 硬件同样的 16-bit 浮点数。这允许我们无任何质量损失地把高动态图像下载和存储为标准格式（对于这种资源的完全讨论，见本书的第 26 章“OpenEXR 图像文件格式”）。

为了创建 16bit 的 pbuffers，我们仍然需要使用另外一个 OpenGL 扩展：WGL_ARB_pixel_format，它根据使用缓冲的目的，为像素缓冲查询所要用的像素格式（见下面的示范代码）。

这样，一旦 WGL 扩展被初始化（示范代码见 DVD 或 NVIDIA SDK），我们就可以为缓冲类写构造函数和析构函数了，见列表 27-1。

列表 27-1　设置 OpenGL 纹理属性和过滤器的代码

```
Buffer::Buffer(int nWidth,
               int nHeight): _nReferenceCount(0),
                             _nWidth(nWidth),
                             _nHeight(nHeight)
{
  // 设置纹理属性
  glGenTextures(1, &_hTexture);
  glBindTexture(GL_TEXTURE_RECTANGLE_NV, _hTexture);
  glTexParameteri(GL_TEXTURE_RECTANGLE_NV, GL_TEXTURE_MAG_FILTER, GL_NEAREST);
```

```
glTexParameteri(GL_TEXTURE_RECTANGLE_NV, GL_TEXTURE_MIN_FILTER, GL_NEAREST);
glTexParameteri(GL_TEXTURE_RECTANGLE_NV, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
glTexParameteri(GL_TEXTURE_RECTANGLE_NV, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
```

列表 27-1 为与缓冲捆绑在一起的纹理设定性质。在写入时，浮点纹理除了支持GL_NEAREST外，不支持任何其他过滤。

因为像素缓冲需要它们自己的设备和渲染场景，我们存储当前的场景：

```
_hOldDC = wglGetCurrentDC();
_hOldRenderContext = wglGetCurrentContext();
```

现在我们必须寻找一个合适的像素格式，如列表27-2所示。

列表27-2 选择像素格式的代码

```
int aIntegerAttributes[15] = {
  WGL_DRAW_TO_PBUFFER_ARB, GL_TRUE,
  WGL_BIND_TO_TEXTURE_RECTANGLE_FLOAT_RGBA_NV, GL_TRUE,
  WGL_FLOAT_COMPONENTS_NV, GL_TRUE,
  WGL_RED_BITS_ARB, 16,
  WGL_GREEN_BITS_ARB, 16,
  WGL_BLUE_BITS_ARB, 16,
  WGL_ALPHA_BITS_ARB, 16,
  0 };

float aFloatAttributes[2] = {0.0f, 0.0f};
int nPixelFormat;
unsigned int nPixelFormats;

if (0 == wglChoosePixelFormatARB(
  _hOldDC,
  aIntegerAttributes,
  aFloatAttributes,
  1, &nPixelFormat, &nPixelFormats))
{
  std::cerr << "Error: Couldn't find a suitable pixel format."
            << std::endl;
  exit(1);
}
```

调用wglChoosePixelFormatARB()要求相当多的准备。其中最有趣的是像素格式必须支持的属性列表。这个属性列表指定为数组形式。数组由成对的简单参数值打头，以0结尾。

wglChoosePixelFormatARB()方法可以返回完整的数组值，但是我们只要求单个值，即把第4个参数设定为1。nPixelFormat的变量接收返回的值；而nPixelFormats的变量能够接收由硬件支持的适应要求的像素格式的总数。但是我们只需要一个格式，所以舍弃后者。

列表27-3的代码用于创建实际的缓冲。为了创建一个数组的函数，给出缓冲属性。这些属性告诉创建方法如何使用缓冲。gnFormatRGBA包含像素格式数字。我们用ARB_pixel_format

扩展求出这个数。

列表 27-3　创建 Pbuffer 的代码

```
int aPBufferAttributes[7] = { WGL_TEXTURE_TARGET_ARB,
                              WGL_TEXTURE_RECTANGLE_NV,
                              WGL_TEXTURE_FORMAT_ARB,
                              WGL_TEXTURE_FLOAT_RGBA_NV,
                              WGL_PBUFFER_LARGEST_ARB,
                              0, 0 };
_hPBuffer = wglCreatePbufferARB(_hOldDC, nPixelFormat,
                                _nWidth,
                                _nHeight,
                                aPBufferAttributes);
```

现在存储 pbuffer 的设备和渲染场景。wglShareLists()指令告诉 OpenGL，应用和缓冲要共享资源（例如纹理等）。

```
_hDC = wglGetPbufferDCARB(_hPBuffer);
_hRenderContext = wglCreateContext(_hDC);
wglShareLists(_hOldRenderContext, _hRenderContext);
```

最后一步，我们把这个缓冲与一个纹理捆绑在一起。缓冲的默认模式是约束为一个纹理。只有当渲染它的时候，我们才解开它：

```
wglMakeCurrent(_hDC, _hRenderContext);
glBindTexture(GL_TEXTURE_RECTANGLE_NV, _hTexture);
wglBindTexImageARB(_hPBuffer, WGL_FRONT_LEFT_ARB);
wglMakeCurrent(_hOldDC, _hOldRenderContext);
}
```

析构函数释放所有的资源：

```
Buffer::~Buffer()
{
  wglReleasePbufferDCARB(_hPBuffer, _hDC);
  wglDestroyPbufferARB(_hPBuffer);
  wglDeleteContext(_hRenderContext);
}
```

到了这一步，把缓冲转为渲染模式及转回纹理模式就变得很简单，如列表 27-4 所示。

列表 27-4　把活动的 Pbuffers 和捆绑的纹理转变为容易使用的开始/结束渲染机制

```
void Buffer::renderBegin()
{
  glBindTexture(GL_TEXTURE_RECTANGLE_NV, _hTexture);
  wglReleaseTexImageARB(_hPBuffer, WGL_FRONT_LEFT_ARB);
  wglMakeCurrent(_hDC, _hRenderContext);
}

void Buffer::renderEnd()
{
```

```
  glBindTexture(GL_TEXTURE_RECTANGLE_NV, _hTexture);
  wglBindTexImageARB(_hPBuffer, WGL_FRONT_LEFT_ARB);
  wglMakeCurrent(_hOldDC, _hOldRenderContext);
}
```

27.3.2 ImageFilter 类

如前所述，我们把与屏幕取齐的嵌块渲染进入 pbuffer 内，进行图像处理。因为实际图像处理是由 Cg 程序所做，我们使用下面的技巧，可以立刻实现 ImageFilter 类的 image()方法。我们定义一个纯虚拟函数 cgFragmentProgram()，当它被一个所推导的类超载时，这个函数返回一个到片元程序的句柄。image()方法把这个程序约束到渲染与屏幕取齐的嵌块。由 ImageFilter 基类推导具体图像过滤器，只是要超载这个方法，并使它返回到处理图像要使用的片元程序。

下面就是 image()方法的一个简化版本：

```
Image oOutputImage;
Image oInputImage = _pSourceOperator->image();

oOutputImage.setSize(oInputImage.width(), oInputImage.height());
```

首先，我们为输出图像创建对象，并且从过滤器的输入操作得到输入图像。然后我们指定输出图像的尺寸与输入图像相同（这意味着，不能再使用一个调整大小的过滤器）。见列表 27-5。调用 renderBegin()准备渲染我们的图像（即我们的 pbuffer）。下面的代码为简单渲染与屏幕取齐的嵌块设置了渲染流水线。

列表 27-5 为处理一个图像而设置视口和投射

```
oOutputImage.renderBegin();
{
  glViewport(0, 0,
             (GLsizei) oOutputImage.width(),
             (GLsizei) oOutputImage.height());

  glMatrixMode(GL_PROJECTION);
  glLoadIdentity();
  gluOrtho2D(0, oOutputImage.width(), 0, oOutputImage.height());
  glMatrixMode(GL_MODELVIEW);
  glLoadIdentity();
```

然后我们告诉 Cg 运行时间，使用一个非常简单的顶点 shader。如下列的代码片断所示，shader 和 profile 从 ShaderManager 类得到。所有顶点 shader 根据观察模型矩阵来变换顶点坐标，并把纹理坐标代入片元 shader。

```
cgGLEnableProfile(ShaderManager::gVertexIdentityProfile);
cgGLBindProgram(ShaderManager::gVertexIdentityShader);
```

现在我们告诉片元程序所使用的 Cg 运行时间：

```
cgGLEnableProfile(cgFragmentProfile());
cgGLBindProgram(cgFragmentProgram());
```

然后运行时间从输入图像找到 OpenGL 纹理 ID，并把它制成片元程序所渲染的纹理：

```
cgGLSetTextureParameter(_hoInputImage, oInputImage.textureID());
cgGLEnableTextureParameter(_hoInputImage);
```

以下是另一个 ImageFilter 类的纯虚拟函数。它应该用于所推导的类中，以设定片元程序的其他参数：

```
setCgParameters();
```

然后我们使用下列的代码，简单地画一个与屏幕对齐的嵌块。关于纹理坐标要注意的是，虽然纹理坐标通常是在[0, 1]范围中，但是 NV_texture_rectangle 纹理需要以像素表示坐标。

```
glBegin(GL_QUADS);
  glTexCoord2f(0.0f, 0.0f);          glVertex2f(0.0f, 0.0f);
  glTexCoord2f(nWidth, 0.0f);        glVertex2f(nWidth, 0.0f);
  glTexCoord2f(nWidth, nHeight);     glVertex2f(nWidth, nHeight);
  glTexCoord2f(0.0f, nHeight);       glVertex2f(0.0f, nHeight);
glEnd();
```

最后，我们结束渲染并返回输出图像：

```
  }
  oOutputImage.renderEnd();

  return oOutputImage;
}
```

27.3.3　过滤的实现

通过一般的 ImageFilter 类实现过滤需要完成以下任务：

1．添加过滤参数。这包括定义必需的成分变量、设置方法和获得方法。因为每个参数都需要代入 Cg 片元程序，最好有代表 Cg 参数的成分。

2．实现 Cg 片元程序。载入这个程序的编码应该被添加到滤波器的构造函数中。

3．超载 setCgParameters()方法。这个方法把存储在滤波器数据成分中的参数值，代入到随时调用的 Cg 程序中去。

下面的示范编码表示一个简单滤波器实现高斯模糊。高斯模糊滤波器使用一个有二维高斯曲线的滤波器核心卷积图像。为了避免处理速度太慢，我们将滤波器核心的最大尺寸限制为 7 × 7。

过滤器参数

模糊度由术语 sigma 确定，sigma 是高斯标准偏差，即从高斯曲线的中点到某特定点的距离，特定点上的高斯值降为中点高斯值的 50%。只要 sigma 改变，实际滤波器必须随着改

变，于是我们决定在 CPU 上计算实际过滤器核心。因此也必须用浮动数组存储核心。GaussFilter 的完整类定义显示在列表 27-6 中。

列表 27-6 对一个高斯过滤器的类的声明

```
class GaussFilter: public ImageFilter
{
  public:
    // Construction and destruction
    GaussFilter();
    virtual ~GaussFilter() { ; }

    // Set/Get sigma
    float sigma() const;
    void setSigma(float nSigma);

  protected:
    virtual void setCgParameters();
    void recalculateKernel(float nSigma);

  private:
    float _nSigma;          // Standard deviation
    float _aKernel[49];     // The filter kernel

    CGparameter _hmModelView;
    CGparameter _hKernel;
};
```

推导一个新过滤器类的花费显然非常少。除了_hmModelView 之外，每件事都直接与高斯过滤器有关。为了正确地变换顶点，顶点程序要求观察模型矩阵的 Cg 参数。

片元程序

高斯过滤器的 Cg 程序，用原始图像的像素简单计算滤波器核心的卷积。对于每个像素，用过滤器核心中对应的值乘以过滤器核心所覆盖的所有像素，然后加和所有这些数值形成新的像素颜色[1]。程序在列表 27-7 中显示。

列表 27-7 用于高斯过滤器的片元程序

```
void gauss(in float2 vUV  : TEXCOORD0,
           out half4 vOut : COLOR,
           const uniform samplerRECT oImage,
           const uniform half aKernel[N])
{
  half4 vSum;
  int i, j, k;
```

1. 这是实现高斯过滤器的最简单的方法，但是也是最无效的。高斯过滤器像许多其他简单过滤器核心一样，属于可分解的过滤器。也就是说，它们是 2D 过滤器核心，但是可以重写为两个 1D 过滤器之积。这个技巧将卷积从 $O(n^2)$问题变成一个 $O(2n)$问题，几乎在任何情况下总能使运行变得更快。关于卷积和分解过滤的更多信息，见本书第 21 章“实时辉光”。

```
  vOut = half4(0, 0, 0, 0);
  k = 0;
  for (i = -RAD; i <= RAD; i++)
    for (j = -RAD; j <= RAD; j++)
    {
      vSum += texRECT(oImage, vUV + half2(i, j)) * aKernel[k];
      k++;
    }
  vOut = vSum;
}
```

为了可读性更好，使用常数 N 和 RAD。在我们的实例中将它们定义如下：

```
#define RAD 3 // Filter kernel radius
#define N 49  // Number of elements in the filter kernel
```

设置 Cg 程序并给出它们在过滤器中默认构造函数参数的代码，如列表 27-8 所示。

设置 Cg 参数

如前所述，把它 invocation 上的数据提供给 Cg 程序代码，应用 setCgParameters()方法，这种方法需要被 ImageFilter 所导出的每个类超载。

```
void GaussFilter::setCgParameters()
{
  cgGLSetParameterArray1f(_hKernel, 0, 0, &(_aKernel[0]));
  cgGLSetStateMatrixParameter(_hmModelView,
                              CG_GL_MODELVIEW_PROJECTION_MATRIX,
                              CG_GL_MATRIX_IDENTITY);
}
```

列表 27-8　GaussFilter 构造函数的实施

```
GaussFilter::GaussFilter(): _nSigma(1.0f)
{
  _oCgVertexProgram = cgCreateProgramFromFile(_oCgContext, CG_SOURCE,
    "VertexIdentity.cg", _oCgVertexProfile, "vertexIdentity", 0);
  cgGLLoadProgram(_oCgVertexProgram);

  _oCgFragmentProgram = cgCreateProgramFromFile(_oCgContext, CG_SOURCE, "Gauss.cg",
    _oCgFragmentProfile, "gauss", 0);
  cgGLLoadProgram(_oCgFragmentProgram);

  _hmModelView = cgGetNamedParameter(_oCgVertexProgram, "mModelView");

  setSigma(_nSigma);
}
```

这就是我们用第一个图像过滤器所做的结果。在图 27-7 中可以看到把这个过滤器应用到图像的效果。

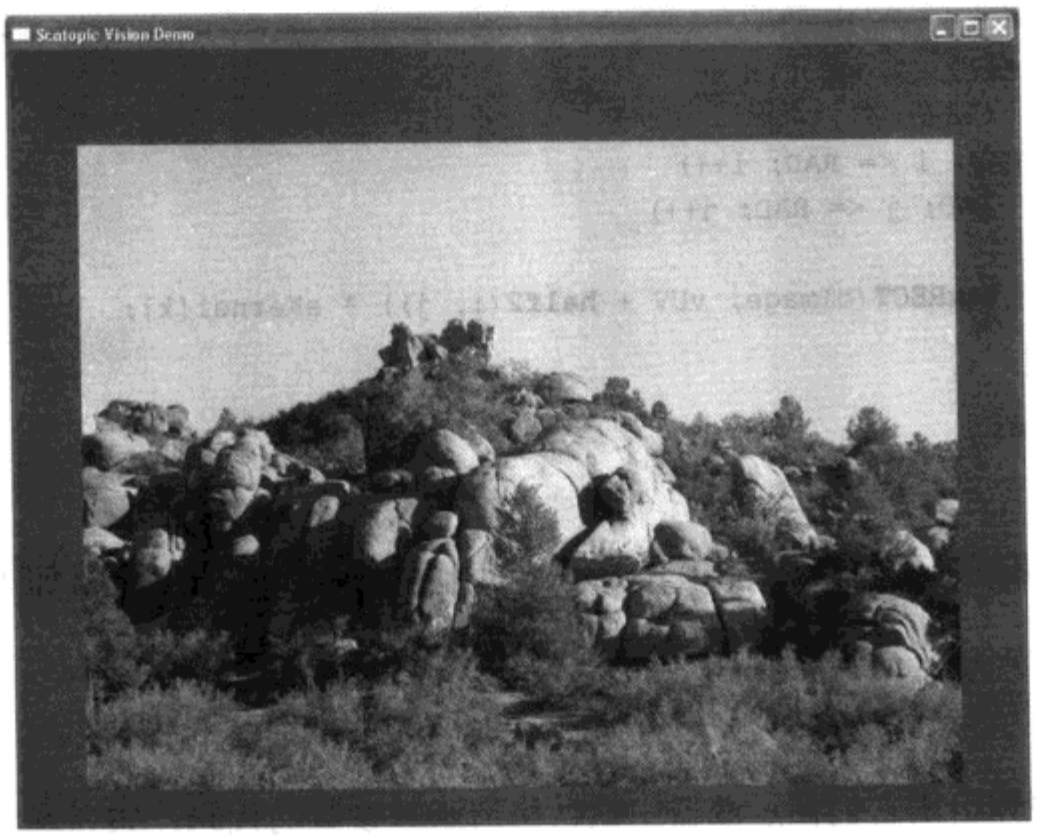


图 27-7 应用一个高斯过滤器
上图：原始的输入图像；下图：对上图图像应用高斯过滤器

27.4 一个示例应用程序

为了演示框架的用途，我们创建一个简单演示程序。应用程序实现一个过滤器，它模拟夜晚的景象。当人类的眼睛适应黑暗环境时，必须依赖暗视觉（scotopic），或在微弱照明下的视觉，这时人类丧失大部分识别颜色的能力。模糊的场景处于蓝色朦胧中。电影制作利用人类视觉的这个特点，在白天用蓝色滤光片拍摄夜晚场景。基于 William B. Thompson 等人（2002）的文章，我们实现了这个 *Hollywood Night*（好莱坞夜晚）的先进版本。

Thompson 等人（2002）首次提出的后处理算法，将彩色图像转换成蓝色笼罩的图像。接着，模拟在黑暗中眼睛分辨率的减小，模糊这个图像。然后把一个边缘清晰的过滤器应用于图像。根据这篇文章，这个过程类似于人类大脑对图像的处理，让视觉系统在黑暗中传送模糊图像。

我们在称为夜过滤的操作器中实施颜色的转变过程。完整的暗视觉过滤器（ScotopicFilter）是一个合成过滤器：在它本身中实施一个小的次级流水线，见图 27-8。

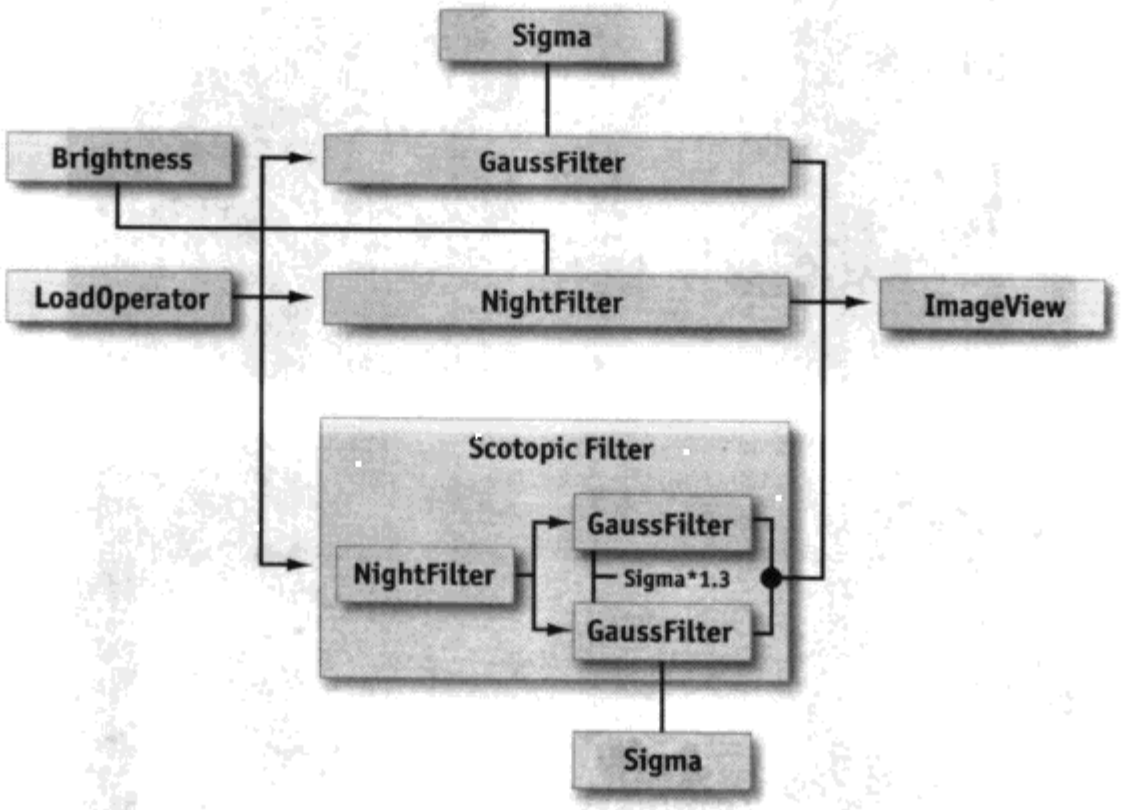


图 27-8　暗视觉过滤流水线

完整的样本应用程序允许用高斯过滤器、夜视（night-vision）过滤器和暗视觉过滤器处理输入图像。建立 3 条“流水线”，根据用户的选择把流水线插进观察操作，从而实现图像处理。

在图 27-9 中，可以见到把暗视觉过滤器用于图像的效果。

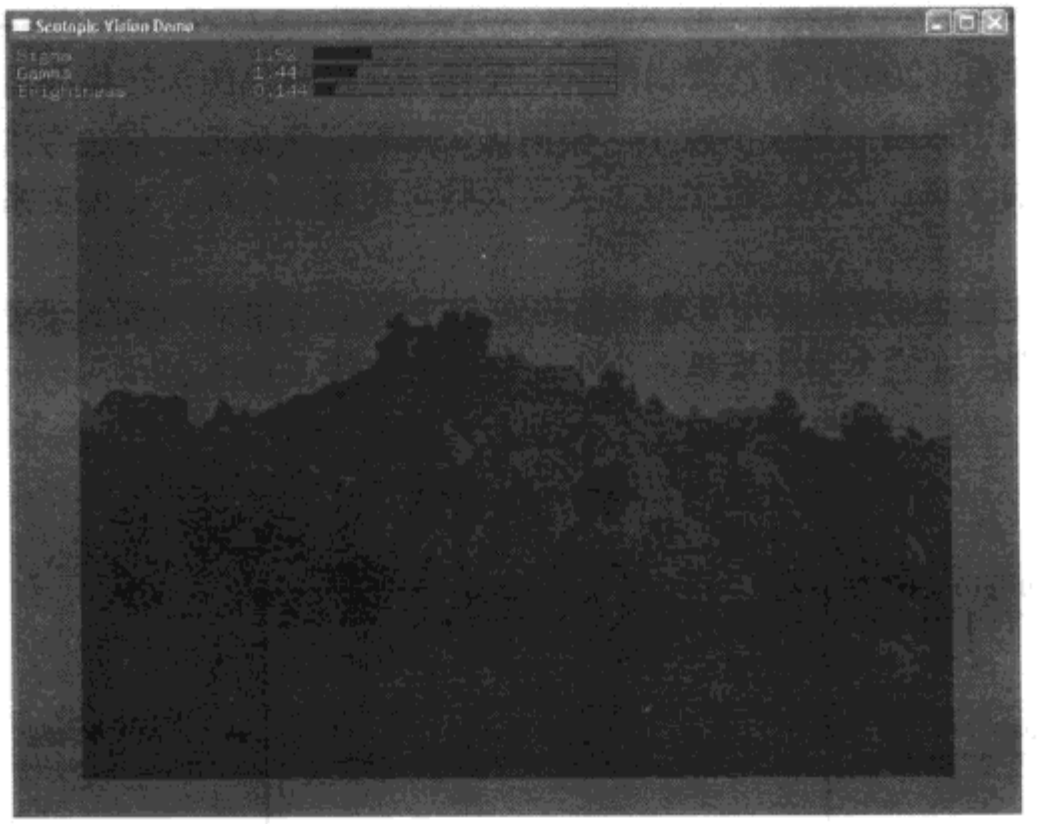
样本应用程序是 NVIDIA 开发的 SDK 的一部分，在本书的 CD 和网站上可以得到。

27.5　性能和局限性

本章进行到此，我们还没有谈到把技术应用到真实世界中的关键问题，这些问题包括运行性能和硬件限制（例如最大纹理尺寸、有限的图像存储器等等）。图形硬件持续地高速发展，使得绝对地讨论这些议题几乎是不可能的。今天的硬件限制或许明天就会消失。

当设计利用 GPU 的算法时，有几件事应该考虑到：图形处理器是极其专门化的第二位处理器；多处理器系统的代码当然比平常的代码写起来更复杂；图形处理器主要是为渲染 3D 立体场景而设计的。只有能够满足硬件主要目的的解决方法才会带来最大好处。

许多图像处理方法在今天的 GPU 上得到很好的应用，使它的速度翻了 10 倍（Colanroni 等 2003）。为了缓解其他不能很好匹配的问题，还可以把某些工作量转移到 GPU。为了得到最好的运行性能，算法应该使 CPU 和 GPU 两者的工作量都饱和。



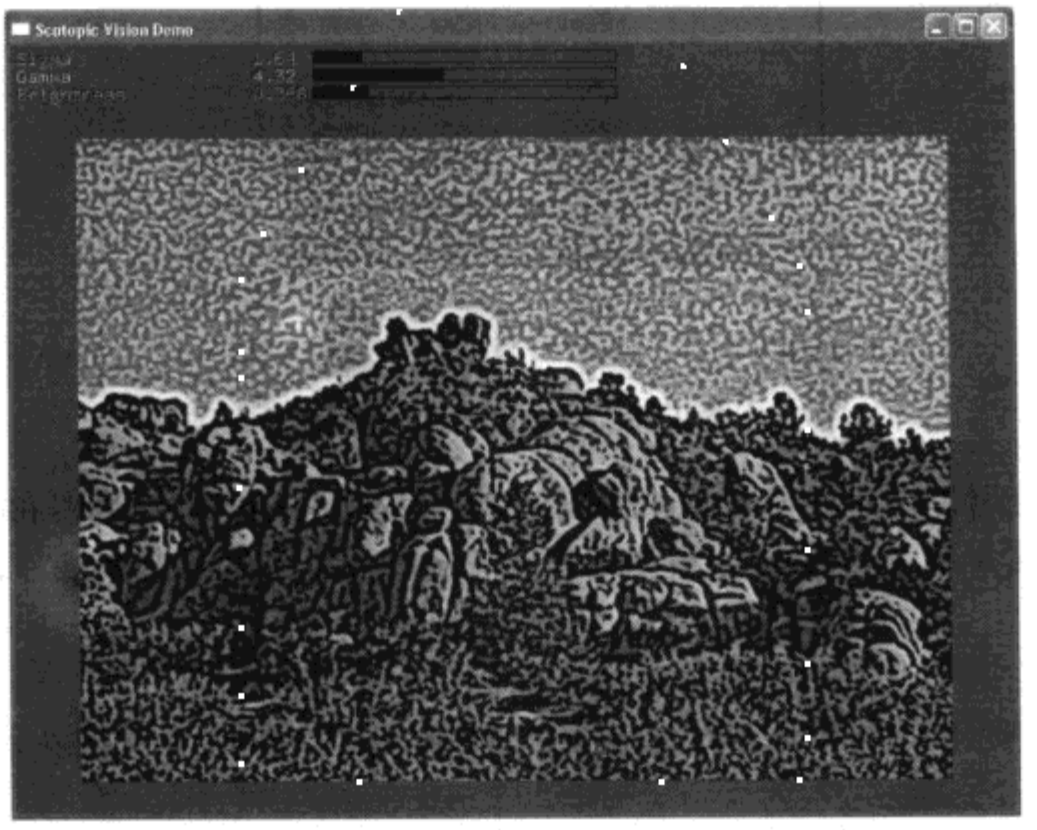


图 27-9 把暗视觉过滤器应用于图 27-7 的相片

上图：调节参数以产生逼真的夜晚场景；下图：把一组不同的参数应用于过滤器，产生更“艺术的”效果

图像处理算法的实施，过去面临着硬件限制，最明显的是内存有限和高速缓冲存储器尺寸的限制。通常是设计专门的方法来解决这些问题。在最初的限制解除之后（例如虚拟内存的管理），由于较好的运行性能，大部分方法仍然继续使用。相信 GPU 的开发也会沿用类似的方式。

27.6 小结

这章所述的处理图像框架，说明现代 3D 图形硬件可以用来进行 2D 图像处理。框架的设

计和实施方式表明，目前的图形应用编程接口（主要用于 3D）支持清晰和直接的实现。

与目前硬件有关的其余限制可以通过巧妙的设计来克服。未来的硬件和应用编程接口将进一步改善运行性能和简化编程工作。

27.7　参考文献

Colantoni, Philippe, Nabil Boukala, and Jerome Da Rugna. 2003. "Fast and Accurate Color Image Processing Using 3D Graphics Cards." In *Proceedings of Vision, Modeling, and Visualization 2003.*

Gamma, E., R. Helm, R. Johnson, and J. Vlissides. 1995. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley.

Meyers, Scott. 1997. *Effective C++: 50 Specific Ways to Improve Your Programs and Designs.* Addison-Wesley.

Thompson, William B., Peter Shirley, and James A. Ferwerda. 2002. "A Spatial PostProcessing Algorithm for Images of Night Scenes."*Journal of Graphics Tools* 7(1), pp.1～12.

第 5 部分

性能及实践

简 介

由于GPU的复杂性日益增加，在应用中有效地使用GPU也变得更富挑战性。本书的这篇提供在shader管理和整合上的一些远景，以及对构成整合决定的图形性能特征的宏观看法。

在**第28章“图形流水线性能”**中，**Cem Cebenoyan**提出对现代图形流水线的宏观看法，包括可编程的流水线，由它引发了本书所讨论的许多技术。在这章中，他描述了测试图形处理器流水线瓶颈的过程，而且为一些瓶颈提供可能的克服办法。

Croteam的**Dean Sekulic**在**第29章“有效的遮挡剔除”**中讨论了遮挡查询，它虽然是很有效的，但是时常被误用。遮挡查询使GPU得到在屏幕上描绘一个物体的像素量。如果经过z测试或模板测试，无须描绘那个物体的像素，就可以跳过那个物体。但是由于CPU和GPU的分离性质，不可能像调用单线程函数一样进行遮挡查询，否则大部分或所有的运行优点都会丧失。Dean讨论了几种方法，以确保能快速和有效地应用GPU遮挡查询的结果。

在**第30章“FX Composer的设计”**中，**Christopher Maughan**讨论了创作shader的有力工具。FX Composer 1.0为shader作者提供完整的IDE，以及美工可调节的图形用户接口，用来调节shader属性。Chris对工具的设计方面进行了介绍，对最先进的shader整合进行了很深的认识。

第31章“FX Composer的使用”也由**Christopher Maughan**撰述，他深入研究FX Composer用法的细节，包括写shader、设定简单场景，以及把shader应用到物体。这章对shader创作和工具的用法提供了很好的介绍。

在**第32章“Shader接口入门”**中，**Matt Pharr**介绍了shader对象，通过shader接口的概念把shader的整合简化到应用程序中。通过把shader片段指定为对象，加上定义明确的接口，可以在运行期自动有效地组合这些片断，提高了灵活性和性能。

第33章“将产品的RenderMan shader转化为实时的shader”，Sony Pictures Imageworks（索尼图片公司）的**Stephen Marshall**介绍在一个GPU型制作流水线中如何改进和权衡RenderMan风格的离线shader。以前的离线shader利用了CPU的优点，在思想上有些局限；只有借助现代GPU重新构思shader，才能实现运行的最高速度。

Cinema 4D是另一个现代化的能够写shader的工具。**Jörn Loviscach**在**第34章“将硬件着色整合进Cinema 4D”**中，讨论了如何整合GPU shader，使它尽可能接近地模拟现有的CPU着色流水线，Jörn提供了一个

引人注目的例子，说明如何无缝地把 GPU 的能力加入到更传统的现存工作流程中去。

虽然 GPU 逐年变得更灵活、更强大，但是要想所有基于内容创建的渲染任务都能在图形卡上处理，恐怕还需要一段时间。在**第 35 章“在实时应用程序中使用高质量软件渲染效果”**中，**Alexandre Jean Claude** 和 **Marc Steven** 讨论了如何在仍然保持现有成熟软件渲染和建模流水线的灵活性的基础上权衡地融入 GPU shader 的马力。

最后是 **John O'Rorke 的第 36 章“将 Shader 整合到应用程序中去”**，重点是 DirectX.fx 文件格式及如何使用。John 示范如何使用.fx 文件的特性，如语义和注释，使用它们能够简化 shader 整合。作者为适应用户要求和扩展.fx 文件（包括 shader 继承）提出了一些建议。

Sim Dietrich, NVIDIA

第28章 图形流水线性能

Cem Cebenoyan
NVIDIA

28.1 概述

在过去的几年里，硬件加速的渲染流水线日益复杂，它的性能特征也越来越难以理解。以前改善性能只意味着减少用于渲染的CPU内循环周期，现在变成确定瓶颈的周期和系统地处理它们。这个鉴别和优化的循环基本上是调整一个异构的多处理器系统，主导思路是，流水线的速度由最慢的阶段决定。因此，过早和盲目的优化只能给单个处理器系统带来最小的性能增益；而对多处理器系统而言，这种优化得到的增益时常只是0。

辛苦地进行图形优化，但得到的结果却为0，这不是一件有趣的事。本章的目的就是完全避免这种尴尬。

28.1.1 流水线

从根本上说，流水线可以分成两部分：CPU和GPU。虽然优化CPU是应用优化的关键，但是这种优化的许多内容与图形流水线完全无关，所以它不是本章的重点。

图28-1显示在GPU里有许多平行操作的功能单元，本质上它们就像独立的专用处理器，其中存在许多可能产生瓶颈的地方。包括顶点和索引的取得、顶点着色（变换和照明，或T&L）、片元着色和光栅操作（ROP）。

28.1.2 方法

未很好确认瓶颈就进行优化，将造成对开发资源的浪费，因此我们把这个过程归纳为以下基本的确认和优化循环：

1. 识别瓶颈。对于流水线的每个阶段，改变它的负载或计算能力（即，时钟速度）。如果性能发生了改变，就发现了一个瓶颈。

2. 优化。指定发生瓶颈的阶段，减小这个阶段的负载，直到性能不再改善，或者达到所需要的性能水平。

3. 重复。重复第1步和第2步，直到达到所需要的性能水平。

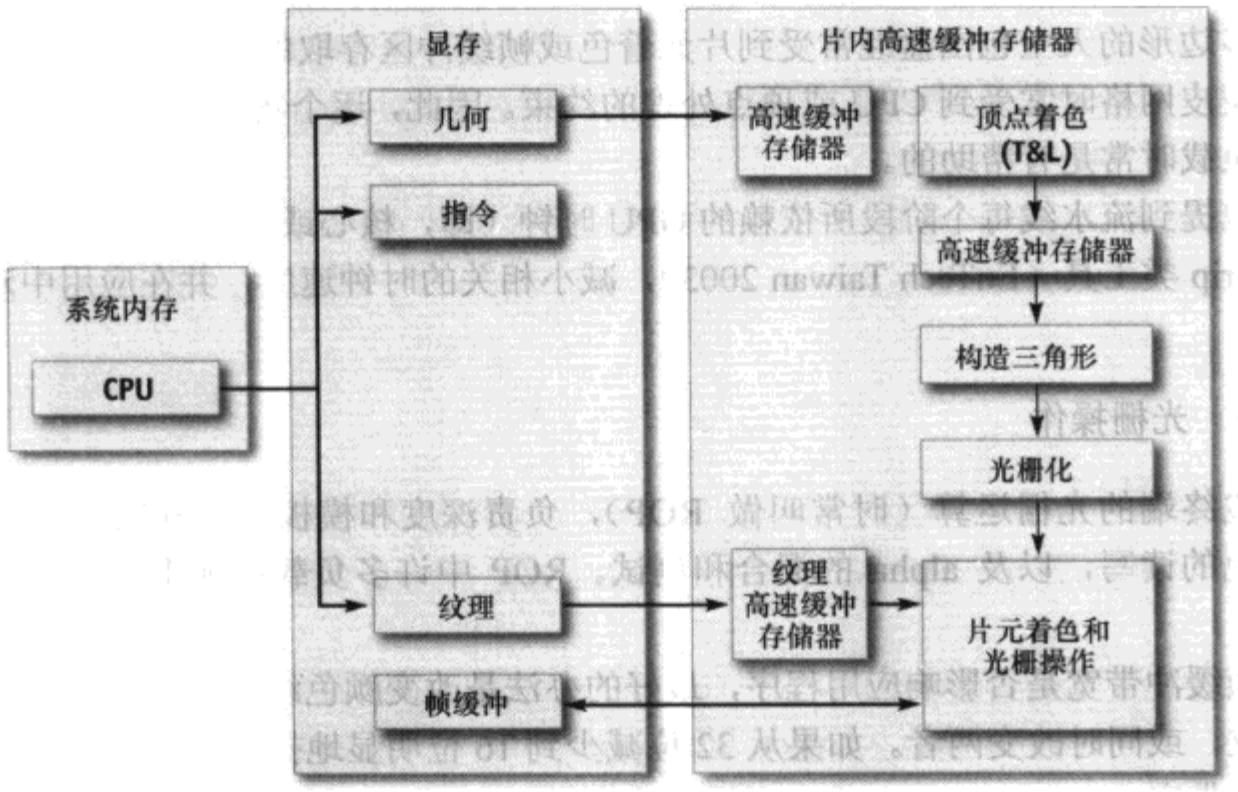


图 28-1　图形流水线

28.2　定位瓶颈

定位瓶颈是优化的一半，因为可以通过它对专注于真正的优化做出明智的决定。寻找瓶颈精确位置的流程图显示在图 28-2 中。它始于流水线的尾端，经过帧缓冲区的操作（也称为光栅操作），终于 CPU。虽然根据定义，某个图元（通常是一个三角形）只有一个瓶颈，但在帧的整个流程中瓶颈有可能改变。因此，修改流水线中多个阶段的负载常常会影响性能。例

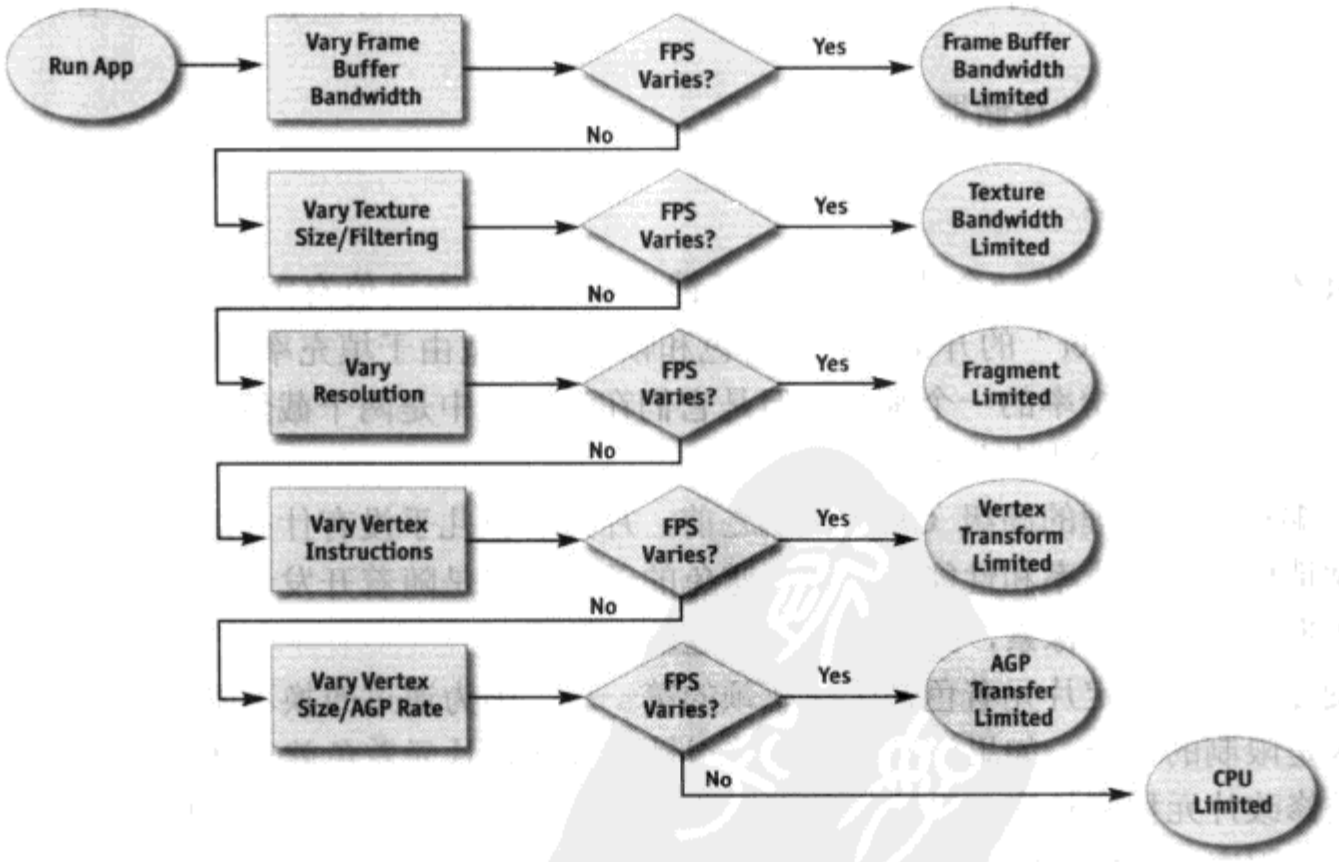


图 28-2　确认瓶颈流程

如，少数多边形的天空包围盒经常受到片元着色或帧缓冲区存取的限制；只映射为屏幕上几个像素的蒙皮网格时常受到 CPU 或顶点处理的约束。因此，逐个物体地改变负载，或逐个材质地改变负载时常是有帮助的。

我们也提到流水线每个阶段所依赖的 GPU 时钟（即，核心或存储器）。这个信息可以辅助 PowerStrip 类工具（EnTech Taiwan 2003），减小相关的时钟速度，并在应用中观察性能的变化。

28.2.1 光栅操作

流水线终端的光栅运算（时常叫做 ROP），负责深度和模板的读和写、深度和模板的比较、颜色的读写，以及 alpha 的混合和测试。ROP 中许多负载都加重了帧缓冲区带宽的负担。

测试帧缓冲带宽是否影响应用程序，最好的办法是改变颜色缓冲的深度位，或深度缓冲区的深度位，或同时改变两者。如果从 32 位减少到 16 位明显地提高了性能，那么必然是受限于帧缓冲带宽。

帧缓冲带宽是 GPU 存储器时钟的函数，因此，修改存储器时钟也可以帮助识别瓶颈。

28.2.2 纹理带宽

纹理带宽是取得纹理请求占用内存所消耗的时间。虽然现代 GPU 配有纹理高速缓冲存储器，设计它的目的是将无关内存要求减到最小，但是它们显然仍有发生，而且消耗相当数量的内存带宽。

在检查 ROP 时，修改纹理格式比修改帧缓冲区格式更麻烦。另外我们推荐使用大量 mipmap 细节级别（LOD）的正偏差，改变有效纹理尺寸。这使得纹理获取访问 mipmap 锥体的粗糙级别，有效地减小纹理尺寸。如果这个修改显著地改善性能，则意味着纹理带宽是限制。

纹理带宽也是 GPU 存储器时钟的函数。

28.2.3 片元着色

片元着色关系到产生一个片元的实际开销（与颜色和深度值有关）。这就是运行“像素 shader”或“片元 shader”的开销。片元着色和帧缓冲带宽由于填充率的关系经常混在一起，因为这两者是屏幕分辨率的一个函数。但是它们在流水线中是两个截然不同的阶段，分辨两者之间的差别对有效优化十分关键。

在可编程片元处理的高级 GPU 出现之前，片元着色几乎没有什么局限性，时常是帧缓冲区带宽造成屏幕分辨率和性能之间不可避免的矛盾。但是随着开发者利用新的灵活性，花费许多周期制造奇异的像素，片元着色的问题就开始了。

改变分辨率是确定片元着色是否为瓶颈的第一步。因为通过改换不同深度位，已经排除帧缓冲区是限制的可能；如果调整分辨率使得性能改变，片元着色就可能是限制。辅助的鉴别方式是修改片元程序长度，看这样是否影响性能。但是小心不要再增加指令，那很容易被有效的驱动程序导致偏离优化。

片元着色速度是 GPU 核心时钟的函数。

28.2.4　顶点处理

渲染流水线的顶点变形阶段通过输入的一组顶点属性（例如模型空间的位置、顶点法线、纹理坐标等等），产生一组适合剪切和光栅化的属性（例如各向同性剪裁空间的位置、顶点照明结果、纹理坐标等等）。这个阶段的性能自然是所处理顶点数和在各顶点所做工作的函数。

用可编程的转换，只要简单地改变顶点程序的长度，就能确定顶点处理是否是瓶颈。如果这时性能变化，顶点处理就是限制。如果增加指令，必须小心地增加富有意义的指令；否则，编译器或驱动可能将指令优化掉。例如，因为驱动在编译程序时通常不知道常数值，所以提交给常数寄存器的空操作（例如添加 0 值常数寄存器）不会被优化掉。

使用固定函数的转换则有些麻烦。调整负载可以通过改变顶点的工作进行，例如镜面高光照明或纹理坐标的生成状态。

顶点处理速度是 GPU 核心时钟的函数。

28.2.5　顶点和索引传输

让 GPU 得到顶点和索引，是 GPU 流水线的第一步。得到顶点和索引的操作随其实际位置变化很大。它们的位置通常是在系统内存中（通过 AGP 或 PCI Express 总线传送到 GPU），或在局部帧缓冲内存中。在 PC 平台上做出决定的是驱动程序而不是应用程序，当然现代图形 API 允许应用程序提供用途暗示，以帮助驱动选择正确的内存类型。

可以通过调整顶点格式大小，来确定得到顶点或索引是否是应用程序的瓶颈。

如果数据放在系统内存内，得到顶点或索引的性能是 AGP/PCI Express 速率的函数；如果数据位于局部帧缓冲区内存，它是内存时钟的函数。

如果这些测试对性能没有明显的影响，限制可能在 CPU 上。这个事实可以通过下调 CPU 的时钟来确认，如果性能按比例改变，CPU 就是限制。

28.3　优化

既然我们已经识别了瓶颈，就必须对那个特殊的阶段实施优化，以改善应用程序的性能。下面根据解决问题的阶段把技巧分类。

28.3.1　在 CPU 上优化

许多应用受到 CPU 的限制，有时是正当理由（例如复杂物理学或人工智能 AI）；有时是因为不好的批处理或资源管理。如果已经发现应用程序受到 CPU 限制，可以试行下列建议，以减少渲染流水线上 CPU 的工作。

减少资源锁定

每当执行一个需要访问 GPU 的同步操作，就有可能严重堵塞 GPU 流水线，这将消耗 CPU 和 GPU 两者的周期。CPU 必须保持在一个循环中，等待 GPU 流水线工作，直到它闲下来并返回所请求的资源，这种等待会造成 CPU 周期的浪费。然后 GPU 等待对流水线的再填充，这种等待又造成 GPU 周期的浪费。

这种锁定发生在以下情况下：

- 对前面正在渲染的表面进行锁定或读出时
- 对 GPU 正在读的表面进行写入，例如纹理或顶点缓冲区

一般应该避免访问渲染期间 GPU 正在使用的资源。

批的尺寸的最大化

我们也称这个技巧是“将批数减到最小”。一个批是调用单个 API 渲染所做的一组基本渲染（例如，在 DirectX 9 中的 DrawIndexedPrimitive）。批处理的尺寸是它包含的基本渲染数目。正如一个智者所提出的“整批”的概念（Wloka 2003）。为画几何体调用每个 API 功能，都同时有 CPU 的消耗，因此如果每次画画调用所提交的三角形数目达到最大，CPU 渲染给定数目三角形的消耗就减到最小。

使批的尺寸最大化的技巧如下。

- **如果使用三角形条带，使用退化的三角形把分开的条带缝合在一起**。假如它们在单个的绘画调用中共享材质，这样能够发送多个条带。
- **使用纹理页**。不同物体使用不同纹理时，批时常被打破。通过把许多纹理安排进单个的 2D 纹理内并适当设定纹理坐标，就能在单个绘画调用中发送使用多个纹理的几何体。这个技术可能有 mipmapping 和反走样的问题。回避大部分这类问题的技术是，把单个的 2D 纹理包进一个立方体贴图的各个面内。
- **使用 shader 分支增加批大小**。现代 GPU 具有灵活的顶点和片元处理流水线，允许 shader 里有分支。例如，两个分开的批处理，一个要求 4 块骨头蒙皮顶点 shader，另一个要求 2 块骨头蒙皮顶点 shader，可以改为写 1 个顶点 shader，让它在所需要的骨头数目上循环，同时累加混合权重因子，当加权总和为 1 时，则跳出循环。这样，2 个批处理可以结合为 1 个。在不支持 shader 分支的架构上，也可以实现相似的功能，以上两种情况都使用 4 块骨头的顶点 shader，对骨头数不足 4 块的顶点，将其骨头加权因子设置为 0。
- **将顶点 shader 常数内存作为矩阵查询表使用**。当许多小物体共享全部材质属性、只是矩阵状态不同时，批时常遭到破坏（例如具有相似树的森林，或粒子系统）。在这种情况下，可以把不同矩阵的 n 载入顶点 shader 的常数内存中，为每个物体把索引存储进顶点格式的固定内存内。然后使用这个索引查询顶点 shader 的常数内存，并使用正确的变换矩阵，这样可以一次渲染 n 个物体。
- **尽可能把决定延迟到流水线的下端再做**。要想使速度更快，应该使用纹理的 alpha 通道作为发光系数，而不是打破批，为光泽设定一个像素 shader 常数。同样，把着色数据放入纹理和顶点可以使批提交量更大。

28.3.2 减少顶点传输的开销

顶点传递是瓶颈的可能性很小，但是也偶有发生。假如顶点或索引（可能性更小）的传递是应用瓶颈，试用下列各项技巧：

- **在顶点格式中使用尽可能少的位**。位数足够即可，不需要对一切都使用浮点格式（例如对颜色）。

- **在 Vertex-progrom 里产生可推导的顶点属性，而不是把它们存储在输入顶点格式内。**例如，正切线、副法线和法线时常不需要存储。给出任意两个，能用 Vertex-progrom 简单叉积推导出第 3 个。这项技术以顶点处理速度换取顶点传输速率。
- **使用 16 位的索引代替 32 位的索引。**16 位索引更容易查找，移动起来更轻量，而且占用的内存更少。
- **以相对连续的方式访问顶点数据。**当访问顶点数据时现代 GPU 可以进行高速缓存。因为在任意内存层次中，引用的空间局部性有助于最大化高速缓冲存储器的命中率，这可以减少对带宽的要求。

28.3.3　顶点处理的优化

顶点处理是现代 GPU 瓶颈的可能性很小，但是也偶有发生，这取决于所使用的模式和目标硬件。

如果发现顶点处理是应用瓶颈，试用下列各项技巧。

- **对 T&L 后的顶点存储进行优化。**现代 GPU 有一个小的先入先出（FIFO）高速缓冲存储器，存储最近所转换的顶点结果；命中这个高速缓冲存储器可以节省所有的变换和照明，以及所有流水线早先完成的工作。为了利用这个高速缓冲存储器，必须使用索引的图元，而且必须排序顶点以最大化网格上的引用局部性。可以帮助完成这个任务的工具有 D3DX 和 NVTriStrip（NVIDIA 2003）。
- **减少所处理的顶点数。**这基本不成问题，但是使用简单的细节级别安排，例如一组静态的 LOD，确实有助于减少顶点处理的负担。
- **使用顶点处理的细节级别。**在使用细节级别减少所处理的顶点数时，试把细节级别用于顶点计算本身。例如，对远处人物没必要完全做 4 块骨头的蒙皮，或许可以使用更轻量的光照近似。如果要多次代入材质，减少在远处较低细节级别的代入次数，也能减少顶点处理的开销。
- **把每个物体的计算留给 CPU 去做。**每个物体或每帧都改变的计算时常在顶点 shader 中进行。例如，方向光矢量到视点空间的转换有时在顶点 shader 中进行，虽然计算结果只是每帧改变一次。
- **使用正确的坐标空间。**坐标空间的选择时常影响计算顶点程序值所需的指令数。例如，计算顶点照明时，如果顶点法线存储在物体空间中，而光矢量存储在视点空间中，就必须在顶点 shader 中转换其中一个矢量。如果改为在 CPU 上对每个物体一次性地把光矢量转换到物体空间，逐个顶点的转换就没有必要了，这就节约了 GPU 顶点指令。
- **使用顶点分支来“早结束”计算。**如果在顶点 shader 中循环数个光源，在正常的低动态范围[0.1]中求光照明，当检查出饱和度到达 1 时，或正在背离光源时，则摆脱以后的计算。对于蒙皮过程可以进行相似的优化，当加权和达到 1 时，停止计算（因此所有后来的加权值是 0）。这取决于 GPU 如何实现顶点分支，而且它不保证改善一切架构上的性能。

28.3.4　加速片元着色

如果正在使用的是长而复杂的片元 shader，片元着色时常有可能是限制。如果确实是，

则应尝试如下做法。

- **首先渲染深度**。在渲染主要着色遍之前，渲染深度（而不是颜色）遍能显著地提高性能，尤其在深度复杂性高的场景中，可以减少片元着色量和帧缓冲存储器的存取量。为了得到优先深度遍的全部优越性，仅仅禁用帧缓冲区的彩色写入是不够的；还应该禁用片元上的所有着色，包括影响深度以及颜色的所有着色（例如 alpha 测试）。
- **帮助 early-z 优化避免片元处理**。现代 GPU 配有设计好的芯片，以避免对被遮挡片元的着色，但是这些优化依赖场景知识。以从前面到背面的次序渲染，可以明显地改善它们。也就是说，把后来通道的遮挡深度复杂度减小为 1，首先放弃个别遍中的深度（见前面的技巧），可以充分地帮助给后来的遍加速（在那里做所有昂贵的着色）。
- **在纹理中存储复杂功能**。纹理作为查询表非常有用，而且可以无消耗地过滤它们的结果。典型的例子是规范化立方体贴图，它允许仅花费单个纹理查询，用高精度规范化任意矢量。
- **把每个片元的工作转移到顶点 shader 中去**。正如顶点 shader 中每个物体的工作应该移到 CPU，每个顶点的计算也应该被移到顶点 shader（连同在屏幕空间中正确的线性插值计算）。通常的例子包括计算矢量和在坐标系统之间转换矢量。
- **使用必需的最低精度**。API（例如 DirectX 9）允许在片元 shader 代码中指定精度，使用减少的精度进行定量和计算。许多 GPU 可以利用这种办法，减少内部精度和改善性能。
- **避免过度归一化**。以“归一化为乐”通常是一种错误。它执行计算时，对每个步骤的每个矢量都要规范化。应该识别不改变长度的变换（例如在标准正交基上的变换）和不依赖矢量长度的计算（例如立方体贴图的查询）。
- **考虑使用片元 shader 细节级别**。虽然片元 shader 的细节级别不像顶点细节级别影响那么大（由于透视，在远处物体本身的细节级别自然地与像素处理有关），但是减少远处 shader 的复杂性和表面上的通道数，可以减少片元处理的负载。
- **在不必要的地方禁用三线过滤**。在最现代 GPU 结构的片元 shader 中计算三线过滤，即使不消耗额外的纹理带宽，也要消耗额外的循环。在 mip 级别转换不容易辨别的纹理上，关掉三线过滤以节省填充率。
- **使用尽可能简单的 shader 类型**。在 Direct3D 和 OpenGL 中，制作阴影片元有几种不同方法。举例来说，在 Direct3D 9 中，可以指定片元着色的使用，随着复杂性和功率的增加，有纹理阶段、像素 shader 版本 1.x（ps.1.1～ps.1.4）、像素 shader 2.x 版，以及像素 shader 3.0 版等。一般而言，应该选择最简单的 shader 类型创建所希望的效果。shader 越简单，由 GPU 驱动编译的本地像素处理代码则越快速。一个附带的好处是，这些 shader 可以在范围更宽的硬件上工作。

28.3.5 减小纹理带宽

如果发现内存带宽是瓶颈，但是大部分结果又要从纹理中得到，那么考虑以下方面的优化。

- **减少纹理尺寸**。考虑目标分辨率和纹理坐标。如果使用者不是真的需要见到最高的 mip 级别，应该考虑缩减纹理大小。如果帧缓冲区的内存已经超载，纹理将被迫放在非局部内存（例如系统内存，通过 AGP 或 PCI Express 总线），缩减纹理大小尤其有

帮助。NVPerfHUD 工具（NVIDIA 2003）可以帮助诊断这个问题，因为它显示各个堆（heaps）中由驱动所分配的内存量。

- **压缩所有的彩色纹理**。应该压缩用作贴图或细节的一切纹理，根据特定纹理 alpha 的需要，选用 DXT1、DXT3 或 DXT5 进行压缩。这个步骤将减少对内存的使用，减少对纹理带宽的需求，而且将改善纹理缓冲存储器的效率。
- **避免不必要的昂贵纹理格式**。64 位或 128 位的浮点纹理格式，显然要花费更多带宽，仅在不得已时才可以使用它们。
- **在可能缩小的表面上总是使用 mipmapping**。mipmapping 除了可以减少纹理走样改善质量外，还可以改善纹理缓冲存储器的利用，通过把纹理内存访问模式局限在缩小的纹理上达到这一点。如果发现某个 mipmapping 使表面看起来模糊，不要禁用 mipmapping，或增加大的细节级别负偏差。最好改为各向异性过滤，并适当调整每批各向异性的级别。

28.3.6　优化帧缓冲带宽

流水线的最后阶段 ROP，直接与帧缓冲存储器衔接，而且最大地消耗帧缓冲带宽。因此如果带宽出了问题，经常会追踪到 ROP。这里讲如何优化帧缓冲带宽。

- **首先渲染深度**。这个步骤不但减少片元着色的开销（见上一节），也会减少帧缓冲带宽消耗。
- **减少 alpha 混合**。当 alpha 混合的混合目标因子非 0 时，要求对帧缓冲区的读出和写入，这样可能消耗加倍的带宽。所以只有必要时才可以进行 alpha 混合，并且要防止高深度级别的复杂性。
- **可能时关闭深度写入**。深度写入消耗额外的带宽，在渲染 alpha 混合效果时（例如粒子）、把物体渲染进阴影贴图时（实际上，渲染进基于颜色的阴影映射时，最好也关掉深度读取），应该关闭多遍渲染中的深度写入（那里最终深度已经在深度缓冲区中了）。
- **避免无关的颜色缓冲区清除**。如果每个像素在帧缓冲区都要被重写，那么就不必清除颜色，因为它消耗宝贵的带宽。但是，只要可能就应该清除深度和模板缓冲区，这是因为许多早期 z 值优化都要依赖被清空的深度缓冲区的内容。
- **大致从前到后进行渲染**。除了前面提到的片元着色会从中受益外，帧缓冲区带宽也得到类似好处。早期 z 值硬件优化能去掉无关的帧缓冲区读出和写入。实际上，没有优化功能的老硬件也会从这个步骤受益，因为通不过深度测试的片元越多，需要写入帧缓冲区的颜色和深度就越少。
- **优化天空包围盒的渲染**。天空包围盒常常受限于缓冲区带宽，因此必须决定如何优化它们：（1）最后渲染天空包围盒，读深度（但是不写），而且允许与一般的深度缓冲一起进行早期 z 优化，以节约带宽；或（2）首先渲染天空包围盒，而且禁用所有深度读和写。究竟哪个选择能节省更多的带宽，取决于目标硬件的功能和在最终帧中有多大部分天空包围盒可见。如果大部分天空包围盒被遮挡，第一个技术比较好；否则，第二个可以节省更多带宽。
- **只有在必须时才使用浮点帧缓存器**。这种格式比起较小的整数格式来，明显地消费更多的带宽。在多渲染目标时也是如此。

- **尽可能使用16位的深度缓冲区。**深度处理消耗大量带宽，因此使用16位代替32位是极有好处的，16位对小规模的室内场景（不需要模板）时常是足够的。16位的深度缓冲区对于需要深度的渲染到纹理的效果（例如动态的立方体贴图）时常也足够了。
- **尽可能使用16位的颜色。**这个忠告尤其适用于对纹理的渲染效果，因为这些工作的大多数，用16位的颜色能工作得很好，例如动态立方体贴图和彩色投射阴影贴图。

28.4 小结

现代 GPU 能力和可编程性的增强，使得改善机器性能变得更复杂了。无论是打算加速应用的性能，还是希望无成本地改善图像质量，都需要对图形流水线的内部工作原理有深刻理解。GPU 流水线优化的基本思路首先是，通过改变每个单位的负荷或计算能力来识别瓶颈，然后，运用对每个传递单元工作原理的理解，系统地解决那些瓶颈。

28.5 参考文献

EnTech Taiwan. 2003. Web site. **http://www.entechtaiwan.com.** 这里可得到 PowerStrip 软件包的信息。

NVIDIA. 2003. Developer Web site. **http://developer.nvidia.com.** 在这里可以发现有助于性能优化的工具。

Wloka, Matthias. 2003. "Batch, Batch. Batch: What Does It Really Mean?" Presentation at Game Developers Conference 2003. Available online at

http://developer.nvidia.com/docs/IO/8230/BatchBatchBatch.pdf

在以下的网址可以找到详细的信息。D3DX: Available in the Microsoft DirectX 9 SDK. See **http://msdn.microsoft.com/directx**

NVTriStrip: **http://developer.nvidia.com/object/nvtristrip_library.html**

NVPerfHUD: **http://developer.nvidia.com**

第 29 章　有效的遮挡剔除

Dean Sekulic
Croteam

29.1　什么是遮挡剔除

大多数图形算法告诉我们如何很快地渲染物体，或怎样改善渲染质量。有些技术倾向于改善运行性能，另一些追求质量，但是维持两者之间的平衡才是真正的挑战。

遮挡剔除可以增加渲染性能，它不渲染视锥之外的几何体，或被更靠近摄像机的物体挡住的几何体。有两种通常的遮挡剔除方法，遮挡查询和早期 z 值拒绝法。

这章研究遮挡查询及适当的使用途径，还提供两个例子证明遮挡剔除的效果。由于一个例子主要用到包围盒，因此为加深理解还要介绍包围盒。

29.1.1　遮挡查询

从 GeForce3 GPU 开始，所有 NVIDIA 的 GPU（和其他厂商的许多 GPU）都提供遮挡查询功能，从而避免渲染被遮挡的物体。跳过被遮挡物体的整个渲染过程，可以减少 GPU 的负担。

29.1.2　早期 z 值拒绝

GeForce3 和大多数图形处理器也支持早期 z 值拒绝。在渲染过程的光栅化阶段，早期 z 法对所渲染的片元深度值和存储在 z 缓冲区的当前值进行比较。如果片元不可见（即深度测试失败），则拒绝，于是不为片元取纹理或运行片元程序，从而节省在每个片元级别的存储器带宽。

遮挡查询和早期 z 值拒绝之间有很大差别。遮挡查询在几何体级别上拒绝多边形，而早期 z 值拒绝发生在光栅化阶段。另外，通过遮挡查询剔除几何体要求应用程序更多的控制。

两种方法都要求程序员把物体从前到后排序。但是由于深度缓冲区和模板缓冲区时常存储在一起，使用模板缓冲可能会减少早期 z 值拒绝的运行性能（另外，早期 z 值拒绝要求更多的中止，但是这与本章的主题无直接关系）。对于早期 z 值拒绝法，程序员做完工作后，由 GPU 自动进行其余的事。

但是对于遮挡查询，还有很长的路要走……

29.2 遮挡查询如何工作

在OpenGL中，ARB_occlusion_query扩展（基于NVIDIA的NV_occlusion_query扩展，比Hewlett-Packard最初的HP_occlusion_test扩展大为改善）执行GPU遮挡查询的想法，得到查询结果（更多内容请见NVIDIA 2003）。微软公司已经将相同的功能纳入DirectX 9（更多内容请见微软公司Microsoft 2003）。

遮挡查询实质上很简单。就是由GPU告诉我们最终在荧屏上可见像素的数量。像素在流水线中经过各种检测，例如视域锥体可见检测、剪裁检测、alpha检测、模板检测和深度检测等，最终通过各种检测的像素是荧屏可见像素。

遮挡查询的工作过程如下。

1. 创建查询。
2. 禁用屏幕渲染（把所有通道的颜色屏蔽设定为False）。
3. 禁用深度缓冲的写入（只能对深度缓冲区检测，但是不能更新数据）。
4. 查询开始（重新设定可见像素的计数器）。
5. “渲染”物体的包围盒（只做深度测试；通过检测的像素不能渲染在荧屏上，因为禁用渲染和深度写入）。
6. 结束查询（停止统计可见像素）。
7. 启用屏幕渲染。
8. 启用深度缓冲写入（如果需要）。
9. 得到查询结果（“可见”像素数）。
10. 如果可见像素数大于0（或某个阈值），则渲染整个物体。

在这些步骤中，我们可以用简单的包围盒代替几何形状复杂的物体。检测包围盒能决定是否需要渲染物体本身。如果最终物体包围盒被其他荧屏上的多边形遮挡，就完全不用渲染物体。

检测物体包围盒时，必须非常小心不变更颜色或深度缓冲区。因此我们禁用颜色和深度写入。

下面，我们开始使用遮挡查询，实际应用该方法。

29.3 初步使用遮挡查询

GPU流水线的流畅是真正的问题。正常情况下CPU和GPU是平行工作的：CPU把指令发给GPU后（例如“渲染某些三角形”），它不等待GPU完成。相反，CPU的驱动器把指令和三角形发给GPU做指令排队，同时CPU继续其他的任务。当GPU处理好所排队的指令时，三角形渲染在屏幕上。

然而，遮挡查询的渲染过程是不同的。GPU需要告诉CPU画在荧屏上的像素数目，这些像素是GPU必须首先渲染的内容。问题是CPU必须等待，在GPU完成渲染用于遮挡查询的三角形之前（不只是那些三角形，还包括在遮挡测试之前提供给GPU的一切三角形），CPU

不能做其他工作。

即使避免 GPU 渲染被遮挡的物体，也不能加速渲染过程。实际上，当大部分测试物体是被遮挡的时候，因为 CPU 和 GPU 不再平行工作，渲染变得非常慢。

很显然，这种简单的方法不能很好地工作。下面寻找真正的解决办法。

29.3.1　恰当地使用遮挡查询

OpenGL 和 DirecrX 9 支持多个独立的查询。这意味着我们可以先启动几个计数器来计算可见像素，以后再去查询那些计数器。

现在，算法变化如下。

1. 创建 *n* 个查询。
2. 禁用屏幕渲染。
3. 禁用深度缓冲写入。
4. 对 *n* 个查询中的每个，
   a. 查询开始。
   b. “渲染”物体的包围盒。
   c. 结束查询（查询自循环结束）。
5. 启用屏幕渲染。
6. 启用深度缓冲写入（如果必须）。
7. 对 *n* 个查询中的每个，
   a. 得到查询结果。
   b. 如果可见像素的数目大于 0，渲染整个物体（结束查询自循环）。

虽然这个方法比最初那个好得多，但是它依然没有给 GPU 充足时间去实际上渲染 CPU 提交给它的所有几何体，这意味着延迟还没有完全消除。

还存在其他的问题……

29.3.2　遮挡物和被遮挡物的比较

基本方法的问题是包围盒不能写入深度缓冲区，因为存在一些真正怪异的情况，这时某个包围盒完全地遮挡另一个包围盒，但是在遮挡包围盒里的物体事实上并没有挡住被遮挡包围盒里的物体，如图 29-1 所示。

这样，需要把某些几何体在前面渲染，不进行遮挡消除。这些几何体起的作用是遮挡物，因此，它在最终屏幕上应该是大东西（好的例子是高墙或大厦）。在渲染这些物体之后，可以开始进行较小物体的遮挡查询。

但是这容易使事情复杂化，而且它使遮挡消除变得无效（甚至一个大物体本身有时可以完全被遮挡）。因此，我们需要另外的方法。

29.4　更进一步的应用

为了确保 GPU 渲染我们所要求的每样东西，我们要检查下一帧的遮挡查询！

目前的游戏循环如下所示（为了简单起见，我们只显示一个查询）。

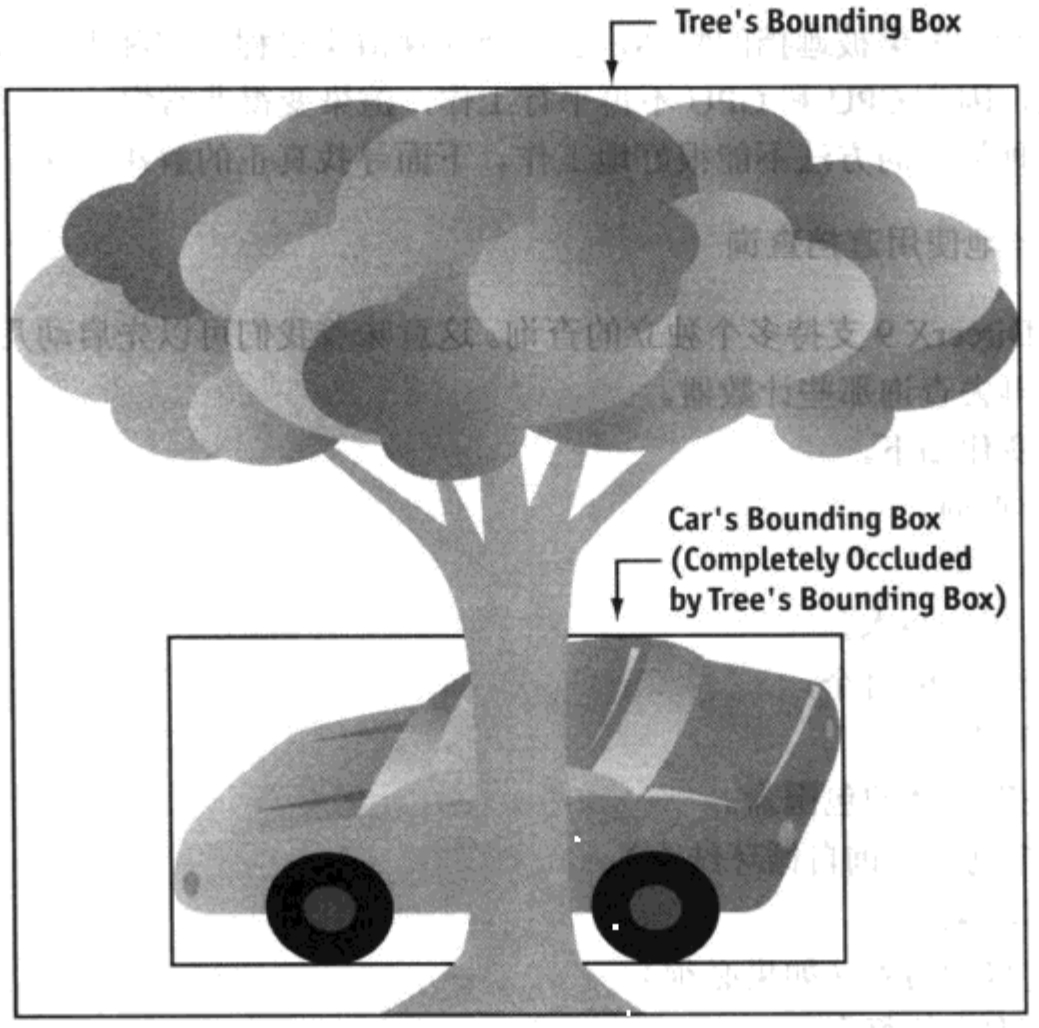


图 29-1 一个不正确的遮挡查询结果

1. 创建查询。
2. 游戏内循环。
   a. 做 AI。
   b. 做物理。
   c. 渲染。
      i. 检查来自前一帧的查询结果。
      ii. 查询开始。
         (1) 如果物体在最后一帧中可见：
            (a) 启用屏幕渲染。
            (b) 启用或禁用深度缓冲的写入（取决于物体是半透明的或不透明的）。
            (c) 渲染物体本身。
         (2) 如果物体在最后一帧不可见：
            (a) 禁用屏幕渲染。
            (b) 禁用深度缓冲的写入。
            (c) “渲染”物体的包围盒。
      （结束查询）
      （对场景中的每个物体进行重复）
   d. 交换缓冲。
（游戏循环结束）

频繁地检查物体是否仍然可见是不需要的，我们可以假定，在前一帧中可见的物体在当前帧中仍然可见。然而这种方法存在一个缺点，有可能把不可见的几何体传送到 GPU 中去。因此，我们可以增加另一个优化，每隔几帧就检查一下物体是否可见。这样，我们会在特定时间进行一下遮挡查询。但是每帧都需要进行被遮挡物的检测（不可见物）！不可以假定在当前帧中物体被遮挡，因为这可能引起过分“渲染”的错误（即，物体无中生有地显现）。

总而言之，落后一帧和消除可见物体（“所见非所得”）的问题不严重。在一帧中看不见一个物体，在下一帧中又见到它也不那么重要，尤其当帧速率超过每秒 30 帧的时候。它只不过是一种“若隐若现”的常见情况，因此这种突然进入不会引起注意。

因为有些驱动器通过队列化的 SwapBuffers（交换缓冲）请求，让 GPU 渲染后面的 2 或 3 帧，所以有时检测下一帧也会引起延迟。这个问题还是可以接受的，因为运行性能只降低不到 10%，何况遮挡查询还提高了速率。

对上一帧可见物体的遮挡检测在物体本身上做，而对以前不可见物体的遮挡检测在物体包围盒上做。虽然这不符合优化原则（不需要对物体和包围盒两者都渲染！），但是这个方法对确定哪个物体是遮挡物，哪个物体是被遮挡物很必要。

29.4.1　将物体排序

这个好方法免去从被遮挡物中分出遮挡物的需要（如第 29.3.2 节所述），因为作为遮挡物的每个可见物体，在渲染时已经被写入深度缓冲区。因此，在被遮挡物之前不需要渲染遮挡物（未经过遮挡测试的物体）。但是根据算法的要求，需要按距离把物体排序。

一旦有了在目前场景中需要渲染的全部物体（即通过视锥剪裁检测的物体），就应该把它们分成不透明物和透明物两组。因为我们能看穿透明物体（而且它们不能写到深度缓冲区），它们只能是被遮挡物而不可能是遮挡物。另一方面，不透明物既可能是遮挡物又可能是被遮挡物，这意味着应该将它们从前到后排序，并且在透明物之前渲染。然后，再将透明物从后到前排序，也在荧屏上渲染。

这个方法很好地利用了遮挡查询的功能，因为遮挡物首先被渲染，这使得许多其他物体变成潜在被遮挡物，其中一些渲染就可能由遮挡测试消除。所有不透明物一经被渲染，它们就自动成为所有透明物的潜在遮挡物。

29.4.2　一个防止误解的说明

有时从某观察点看，一个物体被遮挡，但是它的包围盒可见。当包围盒比物体大很多时，这种现象常常发生。这时，物体每隔一帧就会被渲染一次。由于在这帧包围盒可以“看得见”，因此需要在下一帧渲染它的物体。然而，在下一帧渲染的物体检测结果是“不可见”，又意味着需要在再下一帧渲染包围盒等等。尽管我们的方法可能改变一点帧速率，因为不一定要在每帧中都画物体和包围盒两者，它仍不失为很好的优化方法。

29.5　关于包围盒

静态的或动画的包围盒，时常比它们包含的物体大得多，因此，值得为它们做一些优化。

29.5.1 静态的物体

静态物体的包围盒通常不比物体大很多。但是在经典的棱锥体例子中，可以使用一个以上的包围盒来表现它的内容，如图 29-2 所示。

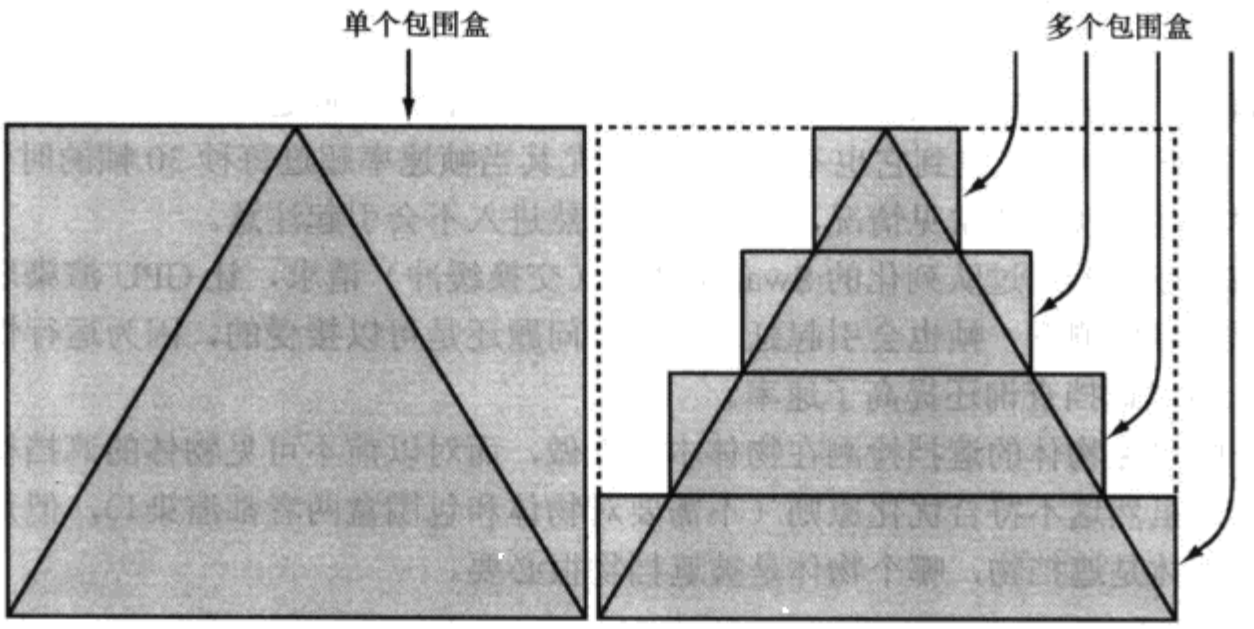


图 29-2 一个棱锥体的包围盒（截面）

必须很小心地使用这些子包围盒，因为检测它们比仅检测一个大包围盒要消耗更多的资源。当然，这完全取决于物体几何形状的复杂程度。棱锥体不过是用来清楚说明问题的例子，用几个包围盒表达棱锥体的 5 个多边形并非最佳。

29.5.2 动画的物体

动画的物体是更复杂的。计算动画物体包围盒的好方法是什么？存在许多方法，这里介绍几个。

1．为每个动画的每帧计算包围盒。

这可能使问题复杂化并加重 CPU 的负担，当物体的骨骼动画系统被合并入 GPU 顶点 shader 中时尤为严重。用 CPU 计算它的包围盒动画物体，造成的消耗比渲染物体要高得多。

2．为每个动画的所有帧存储一个包围盒。

这是最简单和最快速的方法。但有时结果可能是不正确的，因为包围盒可能比物体大得多，当指出物体可见时，实际上它是被遮挡的。

我们可以让物体在原地动画减小包围盒，换句话说，可以通过动画程序转变物体，代替在动画中转变物体。例如，如果我们射击一个敌人，它将向空中飞出，并落在两 m 开外。如果在动画中使敌人在空中飞过，最终会有一个极大的包围盒。另一种做法是，我们在一个地方实现敌人在空中飞行的动画（不变动它），这样将有一个小的包围盒，实际的转变可以在动画启动的时候，通过动画代码实现，见图 29-3。

3．为每个动画存储一个包围盒。

这是最好的方法，但是它也引起一个小问题，当混合几个动画的时候，需要为每个动画计算包围盒的连接。但是一个动画的包围盒比全部动画的包围盒小得多，计算连接的消耗比起这样的节约效果可谓失小得大。

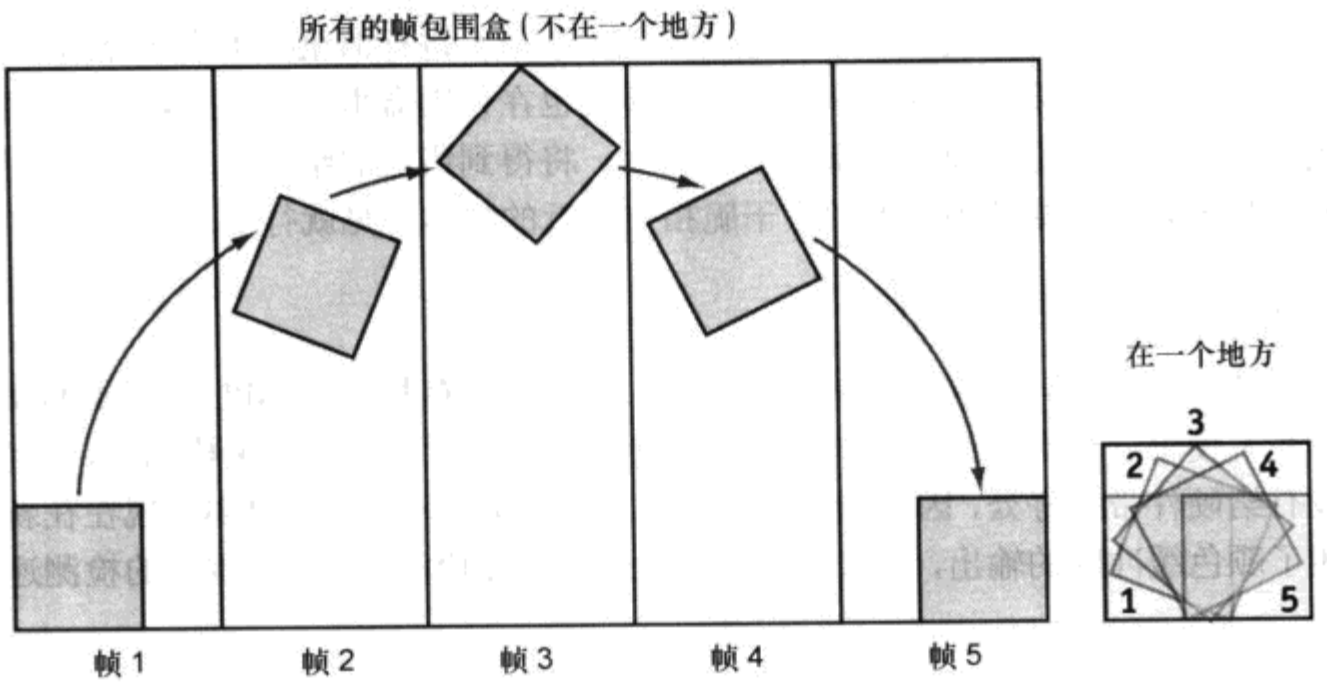


图 29-3　各帧包围盒与在原地包围盒的比较

最好的方式是结合第二和第三个方法，对某个特殊物体选择最佳方法。

在某些情况下，如果渲染一个物体需要多个通道，而第一个通道又非常简单，我们可以使用第一个通道进行可见检测（代替一个包围盒）。但是因为我们正试图节省几何体，所以仍然需要小心对待单个通道中的多边形数量。

29.6　其他问题

29.6.1　CPU 消耗太高

当把渲染请求发送到 GPU 时，需要考虑引起的 CPU 消耗。如果某个物体容易渲染，那么渲染它本身比渲染包围盒和执行遮挡检测要便宜。例如，渲染 2 个三角形（一个包围盒）和渲染约 200 个三角形（或更多），在目前 GPU 上花费的时间几乎相同。准备渲染物体的大部分时间都花费在 CPU 上。

但是有时不管实际物体有多少多边形，都必须用包围盒。为什么呢？看这样一种情况：如果物体有非常复杂的像素 shader，并且占据屏幕的显著的部分，这时虽然包围盒比物体本身有更多像素，但是包围盒像素比物体本身渲染得更加快速（因为它们只被渲染到深度缓冲区），而物体本身的每个像素有许多运算，会加重 GPU 的负担。

29.6.2　高分辨率的渲染

在高分辨率渲染但使用简单像素 shader 的时候，应用程序可能受到填充速率的限制。可能渲染整个包围盒比渲染其中的物体需要更多的像素，需要 GPU 在每个像素级别上做更多工作。

单凭经验的方法（rule of thumb）是，未受到填充速率限制的物体（即，不使用复杂片元程序和许多纹理层的物体，等等）不应该在高分辨率进行遮挡检测。因为在高分辨率填充的像素越多，比起物体本身来，GPU 花费在包围盒的渲染时间也越多。如果物体被遮挡，早期

z 值拒绝将在每个像素级别做它的工作，这样就不必处理复杂的片元程序。

因为有时包围盒需要从两面（朝前和朝后的多边形）进行渲染，那么填充速率就更成问题了！当检测非常接近视点的物体时，观察者最终也在包围盒里。这就是需要渲染背面的原因；否则，因为背面消除，没有像素画在荧屏上，将得到错误的结果。为了安全起见，过于接近观察模型的遮挡检测就不必做了，干脆指定接近的物体可见就行了。

29.6.3 快速深度写入的性能

也有好的方面。当只有深度测试或模板写入发生时，有些新的 GPU（例如 GeForce FX 系列的产品）使用一种更高运行性能的渲染路径。过去进行包围盒可见检测时，颜色缓冲区的关闭没有给硬件带来好处，因为颜色和深度值的输出处于相同的流水线。现在在新硬件上，如果关闭了颜色缓冲区的输出，流水线仍可用来进行深度检测，而且包围盒的检测速度更快。

29.6.4 锥体剔除

尽管遮挡检测可以剔除视域锥体之外的像素，用它做锥体剔除也是不合适的。较之 CPU 传统方法以锥体剪裁平面测试包围盒，渲染包围盒的消耗更大。另外，有的包围盒可以用包围球代替，这使得 CPU 方法的锥体检测更快。

一个方法是首先对视域锥体检测包围球。如果最终包围球完全在锥体内，我们继续渲染整个物体（当然用遮挡剔除）。如果球完全在视域锥体外，这就是拒绝状态，我们进行下一个物体。但是，如果球与锥体相交（更精确地说，与一个或更多的剪切平面相交），我们必须检查物体的包围盒，看它的一维是否比另外两维大许多。如果是那种情形，则意味着包围球比物体本身大很多！因为最后可能剔除包围盒，最好再用视域锥体检测包围盒。一个好的例子是高大的柱子，如图 29-4 所示。

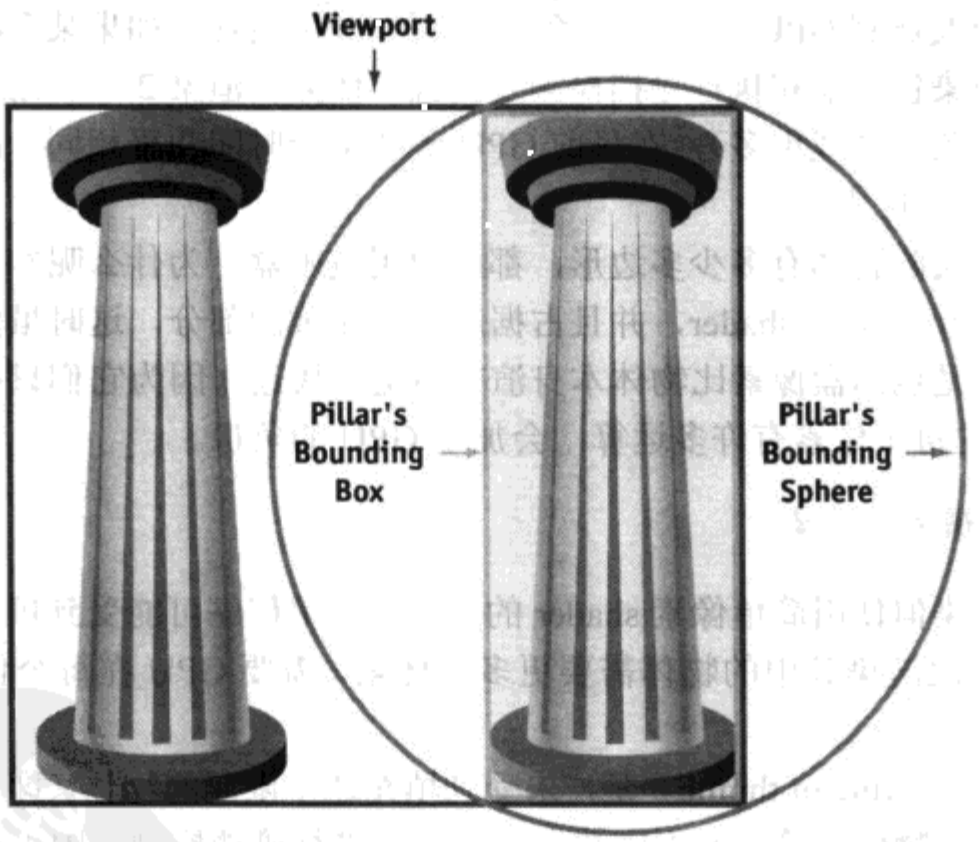


图 29-4 包围盒比包围球能更接近地表达物体

29.7　一点小忠告

当物体有很多多边形时，遮挡消除工作得最好。对于只有 300 个三角形的物体，即使荧屏上有数以百计的三角形，也几乎看不出遮挡查询的作用。速度可以适当地改善，但是它取决于 CPU 的负荷以及 CPU 和 GPU 之间的速度比。快速的 GPU 加上慢速的 CPU，应用受到 CPU 的限制，即使遮挡测试也无济于事。

另一方面，如果每个模型有很多多边形（1000 个是一个好起点），遮挡消除就显示出威力。速率的提高不只是加倍（当然，取决于场景），而且它几乎不花费什么。即使最终所有检测物体都可见，也感觉不到渲染速度变慢！

29.8　一个应用：透镜耀斑

遮挡查询技术的另一个应用是渲染透镜耀斑，见图 29-5 和图 29-6。透镜耀斑对于增加游戏逼真度很有效，但是要想正确地渲染它们也很容易引起严重的问题。

图 29-5　来自 Serious Sam 的一个透镜耀斑：第二次相会

问题在于透镜耀斑不只是围绕光源的美好光晕：它们是摄像机的透镜效果，这意味着它们不能够部分地被物体遮挡。因此，透镜耀斑或是完全看得见或是完全看不见，于是透镜耀斑必须被覆盖在先前渲染的场景上。

透镜耀斑的渲染不是什么大事，只需要渲染一个长方形，用好的透镜-耀斑辉光或环状纹理适当地填充它就行了。然而，当确定透镜耀斑的可见度时，就出现了问题。理论上没有问题，所需要做的“一切”就是渲染整个场景，然后用每个光源的深度值与存储在深度缓冲区中的深度值进行比较，以控制透镜耀斑。如果光源的深度值更靠近视点，则耀斑可见。

图 29-6 来自 Serious Sam 另一个透镜耀斑：第二次相会

可惜的是，检查深度缓冲区的值需要从深度缓冲区读数，那对 GPU 可是不友好的。另一个选择是不检查深度缓冲区的值，而是改为从每个光源投一道光线到观察点。如果光线到达观察点，则透镜耀斑可见。这听起来简单，但是投射的光线经过某种多边形，它的纹理用 alpha 测试，问题又出现了（例如树的叶子）。因为在纹理的 alpha 值低于某个临界值（通常为 0.5）的地方，它的像素遭到 alpha 测试拒绝，纹理只在某些地方写深度缓冲区。为了用光线的投射正确地模拟这一点，必须在 CPU 中做完全的纹理映射，这是一个痛苦的缓慢运算。

那么怎样才能有效地检测透镜耀斑是否可见呢？

29.8.1 渲染透镜耀斑的旧方法

不使用遮挡查询，当然需要访问深度缓冲区来决定是否可见。但是从深度缓冲区读数将不可避免地引起图形流水线的堵塞，因为 GPU 需要排队渲染所有多边形，以产生正确的深度缓冲区图像。Direct3D API 实际上不允许首先读深度缓冲区，这更增加了困难（虽然只在很少场合中发生，但是在实际中完全无法使用）。解除这个限制的方法十分麻烦，而且它也超出了这章的范围。

OpenGL 可以容易地读深度缓冲区，因此我们有机会避开这个 GPU 延迟。所有深度缓冲区读数可以放到帧的结尾处批处理，甚至放到实际渲染透镜耀斑以后。这使透镜耀斑的可见度结果滞后在一帧之后，但是在上下文中完全可以接受。可惜的是这个优化没有明显改善速率，我们不得不再寻找其他根本的解决方法。

检查深度缓冲值的时机，可以选在调用 SwapBuffers（交换缓冲）之后及开始下一帧渲染之前。因为那时 GPU 实际上已经渲染完上一帧，而 CPU 仍然从事目前帧的 AI 计算和类似工作，这将为我们赢得一些时间。渲染一帧的运算法则如下：

1．做 AI、物理和碰撞检测。
2．做渲染之外的其他事。
3．读透镜耀斑的深度值（它们来自上一帧）。
4．清除深度缓冲区。
5．做渲染。
6．渲染透镜耀斑。
7．交换缓冲区。

虽然这个方法可以消除相当的 GPU 延迟，但是还存在一些问题。GPU 延迟的开销取决于 GPU 和 CPU 速率之比。意外的是，实际上 GPU 越快，它的延迟费用反而越大。另一个问题更严重，在调用 SwapBuffers 后，不能保证深度缓冲区的内容保持完好！在今天所有的 GPU 上，深度缓冲区的内容在 SwapBuffers 调用后依然存在，但是不能保证不发生变化。更糟的是，当多样本反走样方法在交换之后进入深度缓冲区时，可能产生某些不好的副作用。这基本上是一种危险的方法，只能在迫不得已时使用。

由此可见，我们确实需要遮挡查询的帮助!

29.8.2　渲染透镜耀斑的新方法

通过“渲染”一个像素到屏幕位置和透镜耀斑光源的深度（启用深度测试，但是禁用深度写入和颜色写入），可以用遮挡查询检测透镜耀斑是否可见。然后，检查通过遮挡检验的像素数目。如果一个像素通过，则透镜耀斑可见。为了减小 GPU 延迟效应，在这帧发出遮挡查询，在下一帧读它的结果。

由于运行性能遭到重创，会使应用程序停止。深度的直接读出并没有给出很多实验空间，因此，为每个透镜耀斑读多个像素是不可能的。但是使用一个遮挡查询，我们能发现一个透镜耀斑实际可见的像素数目。不是只检查一个像素得到“是或非”的回答，我们可以渲染一批像素（16 × 16），并看实际所渲染的比例。然后就使用这个比例来调整透镜耀斑的亮度。现在我们得到更平滑的耀斑，而且不必使用特殊的淡入、淡出程序，以防止耀斑的突然可见和突然消失。检测的像素数目可以容易地由这些因素决定：透镜耀斑在荧屏上的尺寸，或者它的光源到观察者的距离，或由两者共同决定。

这是很好的实验基础，结果会看起来很好，而且完全不会损坏 GPU。

29.9　小结

第一眼就觉得遮挡查询好像是非常有力的工具，它确实是。但是，它也像任何其他工具一样，要求一定的技巧和知识才能得以有效地应用。这需要识别在怎样的情况下它可能有用。

在这章中，我们举出两个例子说明如何有效使用这项技术。当然，不应止步于此，还可以用它做更多的事。例如，遮挡查询可以用来基于物体像素数目决定几何细节级别，或者用于可见面判别算法。

像任何其他技术一样，遮挡查询将会进一步发展。最初，它用于相当简单的任务。但是随着时间的推移，各种障碍将会克服，我们也期待程序员能够发现令人激动的新应用。这章正是投石问路，现在可以开始真正的探险了。

29.10 参考文献

Maughan, Chris. 2001. "Texture Masking for Faster Lens Flare." In *Game Programming Gems 2,* edited by Mark DeLoura, pp. 474～480. Charles River Media.

Microsoft. 2003. Web site. **http://msdn.microsoft.com/directx/default.asp**

NVIDIA. 2003. Web site. **http://developer.nvidia.com/view.asp?IO=nvidia_opengl_specs**

第 30 章　FX Composer 的设计

Christopher Maughan
NVIDIA

FX Composer 是一个为帮助开发者和美工产生 Direct3D 效果而设计的工具。这些效果存储在.fx 文件中，包含了如何对特定的 3D 模型应用 shader 的全部信息。FX Composer 本质上是一个 IDE，其外观和感觉都与 Microsoft 的 Visual Studio .NET 应用程序类似。在开发 FX Composer 的过程中，我们想要使人们在友好的环境中得到互动式的编辑效果，而且想要给未来将会扩展的方面打下一个坚实的基础，例如 shader 调试和剖析（profiling）等。

这章将描述 FX Composer 工具的设计方法，并且解释这样做的理由。通过讨论所面临的问题和所采取的解决办法，希望能让人们对开发一个大型 Direct3D 应用程序的复杂性有所了解。注意，Direct3D 9 SDK 的帮助文件和其他的支持材质已经包含了.fx 文件的细节，而且解释了如何在 3D 应用程序中使用它们。因此，本章不考察使用.fx 文件的细节。

30.1　工具的开发

一个工具开发者的主要工作，是给使用者提供一种应用程序，使他们能够用来生产其他的东西。因此，工具的设计和开发与典型的游戏相当不同，游戏通常主要集中于开发具有固定特性集的快速引擎。对他们有个限制性的咒语是“必须以每秒 x 英尺的速度跑”。而工具的开发则有趣得多，因为它允许工具开发者集中于工程软件而无须理会那个咒语。在工具程序中，功能性和稳定性优先于运行的速度。同样地，FX Composer 的开发主要集中于产生一个有用的工具，它可以扩展以跟上 Direct3D API 将来的变化，它对将来的扩展提供了一个清楚和公用的接口。

30.2　设计初衷和使用对象

当我们开始 FX Composer 项目的时候，主要的设计目标如下。

1. 一个与.NET 的外观和感觉相似的熟悉的 IDE。
2. 一个可扩展的引擎。
3. 一个稳定的环境，用于构建.fx 文件并使结果可视化。

FX Composer的用户是软件开发者和具有一定编程知识的技术美工。基于这些考虑，工具需要把编辑.fx文件的能力和用户接口结合在一起，以编写shader的特性。为了使这个功能更加便利，那个类似.NET的IDE允许用户添加一些特性，这些特性既可以方便地引入窗口面板，又可以在必要的时候隐藏起来。从一开始，就对FX Composer计划了下列的应用特点。

- 有一个材质面板，显示被渲染在一个3D物体上的多个.fx文件。
- 有一个纹理面板，表现渲染目标和材质的纹理。
- 有一个编辑窗口，具有通常的特色：撤销/重复、书签、语法加亮，等等。
- 有一个渲染窗口，以所应用的效果表现简单的场景。

下面就是对FX Composer的应用和引擎的描述。

30.3 对象设计

FX Composer几乎完全由接口和对象构造。模型非常忠实地遵循Microsoft的COM规范，带有一个核心运行期执行对象创建和注册的特性。这样的以接口为基础的方式有几个好处，而且正如我们后面显示的那样，它对非常清楚的设计有益。我们设计的关键结果是一个清楚的插件模型，整个FX Composer应用程序处处都用到它。几乎应用中的每个组件都是一个插件，它很容易把新特性和外部所写的扩展整合在一起。

来自SDK的一个简化的接口显示在列表30-1中。

列表30-1 一个接口样例

```
class INVImportScene : public INVObject
{
public:
  virtual bool ImportScene(INVScenePtr& pScene,
                           const char* pszFileName) = 0;
  virtual bool GetFileExtension(unsigned int i,
                                const char** pszName,
                                const char** pszExtension) const = 0;
  virtual unsigned int GetNumFileExtensions() const = 0;
};
```

系统中任何希望执行INVImportScene的对象，都可以通过继承纯抽象的基类以及实现方法做到这一点。所有的引擎接口都是纯抽象，而且全部从基础的接口INVObject继承而来。INVObject有3个任何COM程序员都熟悉的方法：AddRef、Release和QueryInterface。这些允许生命期控制引用计数的始终，而且它们允许查询系统中任何实例化的对象是否实现了一个特别的接口。因此，系统中的任何对象必须最少地执行INVObject接口，所以查询它总是为了其他接口的指针和引用计数。

列表30-2所示的是一个典型的引擎对象执行INVImportScene接口。

另一个函数CreateNVObject通常用一个宏来声明，而且运行库调用它来构建这个对象的新实例，如果它被支持，则给被请求的接口返回一个指针。

在例子中的所有的插件，即动态链接库（DLLs），被要求输出两个函数：即RegisterNVObjects和UnRegisterNVObjects。在启动时运行库将会装载插件，而且要求它们对

所构造的每种对象类型给以注册。根据 COM，每个对象和每个接口均有全局惟一标识符（GUIDs）。

列表 30-2　XFileImporter 对象

```
class XFileImporter : public INVImportScene
{
public:
  // 实现 INVTmportScene 方法
  . . .

  // 声明 INVObject (addref/release/queryinterface)
  . . .

  // 一个静态的建立函数
  static bool CreateNVObject(INVCreatorPtr,
                             NVGUID& Interface,
                             void** ppObject);
};
```

本质上，在 FX Composer 框架的每个对象都是一个插件，从复杂的对象如文件的输入，到简单的对象如字符串容器。可装载的 DLL 包含许多这样的对象，每个对象执行许多接口。另外，所有的对象都具有易读的字符串和一个可选择的种类（category），它把特定类型的对象分在一起，使它们容易被发现。一个执行 INVCreator 接口的对象工厂（object factory）对每个对象类型声明一次；可以根据需要用它来制造被注册的对象。FX Composer 系统是一个可插（pluggable）类工厂的例子。

这里是一个例子。可以在 FX Composer 中构建的所有形状都在 geopipeobject_shape 类中。下列的代码显示 FX Composer 怎样找到适用的形状插件的目录：

```
INVCreatorArrayPtr pCreators = GetSYSInterface()->
                               GetCreatorsInCategory("geopipeobject_shape");
```

对 GetSYSInterface 的调用是全局性的，并且可以在任何地方调用它，以便给 nv_sys 的运行库返回指针。对象创建所返回的数组可以构建显示在材质和渲染视屏上的形状。因此通过非常简单的过程就可以制作这样的插件对象，它能够产生特定的形状，并使这些形状可以立即在 FX Composer 里使用。进一步的是，能够对项目文件存储这个形状，并且再装载它时还保留它的所有相关参数。

讨论这些组件的技术实现超出了本章的范围，但是它归结为几个简单的用来声明对象和它们接口的宏。我们取 COM 规格的有用部分，以避免更复杂的特性，例如聚合和通过注册表注册组件。为了进一步简化用法，我们用了智能指针来免去对 AddRef/Release 的调用，并且使得接口的查询自动进行。例如，以下的代码片断就用来检查一个场景输入对象是否支持 INVProperties 接口。

```
INVSceneImporterPtr pImporter = GetImporter();
INVPropertiesPtr pProps = pImporter;
if (pProps)
```

```
{
  . . .
}
```

对输入指针到特性指针的分配，将调用输入对象上的一个隐式 QueryInterface，如果成功，它将返回一个有效的指针。智能指针（由接口名中的 Ptr 后缀来表示）的引用计数特性将清除一切其他的引用。每个 FX Composer 接口由一个等价的智能指针来声明，而且它们的引用几乎都为 1。

不是每个人都喜欢基于接口的方式/COM 方式。但是对于 FX Compose，这种方式已经变成一个了不起的特性，它已经使得基本的实现既清楚又容易。使用基于接口的程序，我们可以以非常容易理解的方式看到 FX Composer 对象，也使得添加扩展和新特性变得容易。

下列各项是这种方式的一些好处。

对象查询

在 FX Composer 中，任何对象可以支持任意接口，具有非常好的扩展性。IDE 的特性面板就是很好的证明。当选择一种材质的时候，就有一个包含 INVObjectPtr 的“对象选择”信息到达特性面板。特性面板可以给它加上 INVPropertiesPtr，如果成功，它就可以使用这个标准接口遍历一串特性值，而且在图形用户接口中编辑它们。对于任何一个对象，只要它选择让系统看见其特性，就会用到特性面板。例如，当我们添加形状插件时，显示它们的尺寸、布局等是一件平常的事。当装载和选择了这些形状的时候，特性面板就允许用户改变它们的值。因此，FX Composer 系统中的许多对象都支持特性接口，从而在用户接口上得到有用的信息。

这个特性已经再三地带来好处。当我们给一个 XML 文件添加存储场景的能力时，简单的办法是使序列化对象支持一个把对象流入 XML 格式文件的 INVXMLStreamable 接口。当我们想要复制一个对象的时候，就添加一个 INVClone 接口，对如何复制予以控制。

未来接口的思路是包含一个 INVScript 接口，它使得系统对象显露它们脚本的功能。

对象生命期

因为所有的组件都是引用计数的，而且智能指针通常总是指向它们，我们几乎没有内存泄漏，即使有，时常也中止在外部代码中。当然，引用计数也存在缺点，就是在传送接口指针时总有一项固定的开销；有时想清除一个对象，也很难做到；而且当两个对象互为引用的时候，循环引用时常可能是个问题。当这些问题出现的时候，我们通过“弱”指针来解决，即对非“智能”的、没有一个引用的对象装载指针。这种方式感觉起来确实有点像一个变通办法，但是实际上时常是不需要的。正如早先提到的那样，对于这个工具不需要考虑额外开销。

插件管理

用我们正在使用的对象模型时，FX Composer 的插件本质上是自由的。每个对象用系统注册其本身，而且以后可以被任何感兴趣的部分使用。创建一个新插件很简单，不过就是声明一个新接口、指派一个相关的类、写简单的代码把那个对象装载到类中和使用它们而已。

核心运行库一直在追踪对象，而事实上，总是有一个内部目录，它记录系统中每个实例化的对象和所有被装载的插件模块。因为可以随时分析运行库系统，所以这可能是一个有用的调试工具。FX Composer 中另外一个调试对话框，提供对当前系统对象的树状结构，已经在许多场合下使用它来分析行为和追踪程序缺陷了。

我们的方法有一个缺点，就是确实需要对它有点理解，才能用好它，如果一个开发者不熟悉这类程序就会感到困难。为了使调整过程变得容易，我们添加了一些宏，尽可能地简化对象和接口的创建；也对样品插件做了尽可能多的说明。

连接参数

从一开始，我们就决定 FX Composer 应该处理场景的动画、蒙皮的人物和材质中的参数。于是，我们设计了一个连接参数类型，与 D3DX 效果参数类型紧密地匹配，充分地支持动画。结果使用时间键和各种类型的内插，可以使 FX Composer 中的任何参数动画在两个分开键之间产生中间值。比方说，可以应用一个效果到一个蒙皮的网格，然后变动一个值，例如在效果中的“凹凸高度”。人物将会活动起来，而效果将会适时地变化。连接参数有几个其他的属性，例如语意、注释和名字，正像在.fx 文件中一样，把它们构成在一个对象中任何数据项的有用的通用容器。

FX Composer 支持的连接参数在几个方面是有用的。

- 特性窗口可以为了一串参数而查询任何对象，并且以一个巧妙的方式显示它们。只要一个对象支持 INVProperties 接口，就能通过一个普通的特性窗口编辑它的参数。当编辑不同类型的对象时，例如材质、形状、光源、摄像机和其他对象等，这给用户一致的外观和感觉。
- 项目文件流代码可以零散地存储或恢复连接参数，使大多数对象的序列化成为一个简单的过程。
- 可以就类型、语义、名字、注释和更多项目来比较连接参数。由于时常要把一列连接参数映射为另外一列连接参数，这一点特别有用。当一个重装载的项目文件包含对于已经修改的.fx 效果的引用时，一个典型的情景就发生了。在这样的情况下，我们既想要维持尽可能多的存储数据，又要用新的.fx 文件工作。为了达到这个目的，我们把旧的参数目录映射为新目录。当一个冲突发生的时候，用户就及时地解决它。另一个映射情景就发生在当试图以一种巧妙的方式把材质参数与场景中的对象匹配时，例如把一个光源钩到一个材质的位置参数上时。

消息的发送和接收

对于 FX Composer 引擎这样复杂的系统，不同组件之间的良好通信是很必要的。我们采取一个两端分叉的方式解决这个问题。首先，我们开发了一个全局的信息传播系统，它允许任何对象以“发射并忘却”的方式传达消息给所有感兴趣的组件。NewScene 消息是关于这点的好例子，许多组建都希望知道在哪里已经产生了一个新的场景。

这个方案一开始工作得很好，但是很快由于传播的泛性而变得过于膨胀。为了提高系统性能，开发了另一个消息传播方案，它使得两个感兴趣的组件之间有一个密切连接。在系统中，任何对象都能执行 INVMessageReceiver 接口，也能执行 INVMessageSender 接口。然后

感兴趣的组件可以用对象注册它们本身，并找到它们希望知道的事件。这个方案的基本要求是，必须小心地保持通信两端之间的连接。一定要注意不使参与双方同时脱离，也要防止其本身与不再存在的或不再有关的对象发生关系。消息发送/接收系统是《设计模式》中所描述的“出版/订购”模式的例子（Gamma et al.1995）。

30.4 文件格式

为了存储和恢复当前的场景、材质调色板、文件路径和可选择的当前媒体集，我们确定FX Composer需要它自己的文件格式。这种文件会支撑整个的工作空间，而且允许用户把纹理、.fx文件和其他的项目打包到一个文件内，这是我们开发者关系小组格外需要的，因为我们时常想要给开发者们发送独立成套的例子。这个工作空间格式的文件扩展名是.fxcomposer。

我们所考虑的第一个文件格式完全由XML数据组成，但是由于在给定场景中有大量的二进制数据，它被否决了，原因是这样的数据不能很好地用XML编码。我们采用了混合方式，它可以提供两个语言里最好的东西。FX Composer项目文件是一个混合格式，所有的数据实际上存储在格式里一个单独的.zip文件中（虽然带有.fxcomposer扩展）。.zip文件包含一个XML场景描述，引用了一些外部的二进制数据，这些数据以偏移的形式进入一个二进制块内。这个二进制块也编码在相同的.zip文件中。为了更进一步简化设计，系统中的对象能支持一个XML流接口，而且能够以XML的格式写出它们的数据。这样做所使用的接口也允许把数据加入.zip二进制的文件，为了以后恢复而返回一个偏移。存储整个的工作空间是一件简单的事，即遍历对象的继承结构并为每个对象构建XML。恢复也很简单：即每个对象基于XML数据装载其本身。随着进一步抽象化，XML的每种元素有一个可以从文件的结尾取回的ObjectID。这样，在装载必需的插件以及它可能支持的对象从工作空间文件被流出之前，FX Composer可以预先进行检查。这个步骤防止出现这种问题：存储不含有“茶壶形状”插件的场景，并试图在没有安装那个插件的FX Composer的版本上再装载它。在这样的情况下，程序能通知用户，一个插件不见了，然后试着装载其余的场景。

新文件格式一个有趣的附带作用是它的效率。因为通常的数据被压缩进二进制块内，它没有交叉存取的场景数据，文件已经变得相当紧凑，对存储器的要求远小于被压缩时的原始网格文件。这种紧凑是十分惊人的，.fxcomposer文件包含的不只有网格，还有调色板的材质。较之于我们内部开发的.nvb文件格式，在输入.x文件的情况下增益是最大的，而且实际上被存储的文件以切线和副法线矢量形式包含了更多的网格数据。

表30-1显示了把一个.nvb和一个.x输入文件存储为.fxcomposer文件的例子，文件大小以字节计量。所显示的节约是以占被压缩形式的原始文件的百分比（.zip文件）来表示的。注意，通常被压缩的原始文件不是存储在用户硬盘上的，在这种情况下的节约更大。

表30-1 文件大小的比较

文件名	文件大小	原始文件大小	被压缩的原始文件大小	节约
Gooch_alien.fxcomposer	906 486	1 561 000 (.nvb)	968 696	62 210(6%)
Tiny.fxcomposer	211 819	1 533 978 (.x)	338 252	126 433(62%)

我们也用XSLT变换做了一些实验。通过把数据转换到一个易读的网页之内，这种声明

的语言能够显示 XML 文件。将来，用.fxcomposer 项目把.xslt 文件打包，我们可以制作自描述的项目文件。添加一个 XML 的 schema 是另一个有趣的选择，可以对再次附在混合.zip 文件里的 XML 格式予以校验。

30.5　用户接口

一个工具项目通常涉及许多用户接口工作。这些工作可能是麻烦的，而且如果做得不好，时常会由于接口很难使用而导致整个工具不完善。结果经常取决于开发者的 Windows 编程经验。从一开始我们就意识到 FX Composer 需要复杂的 UI，而且我们打算支持浮动和停靠的窗口，以此节省空间并给用户提供熟悉的开发环境。

我们曾考虑写自己的 MFC 代码，但是这样一件复杂的工作需要专门立项。另一种可能是写一个 VC.NET 的扩展，但是我们不能完全确信.NET IDE 能否满足即时 3D 应用的各种要求，而且仅仅为了解答简单的问题，我们还得进行艰苦的学习。

最后，BCGSoft（www.bcgsoft.com）以其 BCGControlBar 专业的 MFC 扩展库解了我们的围。这个威力强大的组件实现了许多在高级的 IDE 中发现的特性，例如 Visual Studio.NET。它能支持浮动和停靠的窗口，而且还有一个意外的收获，它具有一个编辑器能支持书签、撤销/重复，甚至 IntelliSense。到目前为止，我们真的对结果感到很满意。BCGControlBar 是非常大的，实际上，我们并不用打开工具箱的所有功能，只是根据需要使用必要的功能即可。比方说，SDK 支持一个完全可定制的菜单和热键版面安排，就像在.NET 上一样，用户可以对它进行修改，使其外观和感觉适合于自己。

使用这样复杂的工具箱，不但附加了应用复杂性，测试也是一个负担。有这么多以各种方式安排的窗口，每一个都在应用时启动，又可能要随时打开、关闭或隐藏，的确需要一个完整的测试方法，以保证所有版面安排的组合能够工作。我们通过几个方面解决这个问题：自始至终的质量保证、应用手工修改暴露潜在的问题，以及自动化的测试等。我们希望实现的计划是一个在对象层次的单位测试框架。使用一个像 INVUnitTest 这样简单的接口，有可能让系统的组件以一个清楚的方式进行自动化的测试。FX Composer 对系统的所有对象的测试，既可以借助整体的调试构造定期地进行，也可以响应一个按键进行。

30.6　Direct3D 图形的实现

FX Composer 本质上是一个图形应用程序，具有多个设备窗口，是相当复杂的。它也负责管理一列.fx 文件和所有相关媒介。在当前系统上的任何时间最多有 4 个 Direct3D 设备可以使用：媒体面板、纹理面板、渲染面板和 Shader Perf 条——这是一个特性面板，能够分解.fx 文件和为 NVIDIA GPU 显示剖析（profiling）信息。

30.6.1　设备窗口

我们清楚需要简化各个组件的实现，所以决定在应用中对每个面板使用单独的 Direct3D 设备。虽然这种方式可能需要很多的资源，但是它确实允许用户“分解”应用中的任何单独的面板，而且在第二个监视器的完整屏幕上运行它。为了确认效果或当前硬件不支持的

shader，有时在加速硬件的设备窗口上使用参考的光栅器也变得比较容易。

各个设备的消耗显示在材质和渲染窗口面板上。这两个窗口可能包含着由于共享材质造成的纹理的重复。举例来说，把 DiffTex.fx 文件装载进材质面板之内，也就把纹理装载到材质面板上了。如果材质以后应用于场景中的一个网格，那么纹理也将例示在渲染窗口上。紧接着，如果那个材质被选择，在纹理面板中也将产生纹理。当装载材质去分析效果的时候，它又可能出现在 Shader Perf 面板中；尽管并不一定要通过装载纹理做这一点，或在任意长的时间中把纹理保持在材质上，但由于 Shader Perf 面板和纹理面板只显示一次材质，因此这个问题还不大。

因此，材质面板和渲染面板是资源可能被重复使用的主要地方，特别是重复纹理。在实际中，这还不是一个问题，只有在取相当大的场景时它才会变成一个问题。另外，除了渲染目标之外，大多数的纹理是管制的，在使用它们之前，不必在设备上实例化它们，而且如果需要，还可以把它们翻页。将来，我们可能研究各个设备共享的纹理，那时 Direct3D 就允许这样做，或者激活一个能被各个窗口共享的设备。这可能在某种程度上减小设计的灵活性并增加复杂性，所以我们认为现在这还是不必要的，尤其因为 FX Composer 是力图为开发者和美工提供开发机器，它具有相当现代的硬件，包含许多图像存储器。

30.6.2 Direct3D 效果

FX Composer 的主要工作是管理.fx 文件。为了与.fx 文件合作，D3DX 提供了丰富的应用编程接口，FX Composer 使用它得到了很好的效果。D3DX 的关键是能够以两个步骤装载效果。虽然还有其他方式可以这样，但是 FX Composer 只采用了这种方式。

首先，FX Composer 创建一个材质。我们把该材质看作一个.fx 文件的实例，而相关的参数是那种材质独有的。我们随时可能从同样的.fx 文件创建几个材质，其中每个都有一组不同的连接参数。举例来说，假定我们有两个不同的卡通材质，两个都引用一个叫做 cartoon.fx 的文件。每个材质可能有一种在.fx 文件中漫反射参数不同的颜色，目的是给一个卡通人物不同的外观。这一点模仿美工用典型的 CAD 包工作的方式，首先选择材质的类型，然后调节参数得到想要的外观。此时材质不与任何特别的 Direct3D 设备相连，无论有多少窗口引用它，只有一个对象代表一种材质。

30.6.3 ID3DXEffectCompiler

我们一旦有了材质，就可以使用 ID3DXEffectCompiler 接口装载.fx 文件了。这使我们能够编译效果文件，而且无须在设备上例示效果就能检查其错误。效果编辑器创建一个紧凑的效果表达式，以后能用它来产生特定设备的效果。但是这里必须强调一点：因为我们想要那个效果的一组材质参数，所以必须为那个.fx 文件存储一个连接参数集。这就是我们的主要参数组，引擎的所有部分都根据它来修改。这样做是必须的，因为一旦把效果加到一个特殊的设备上去，就要为那个设备创建一组新的参数，那么我们就需要把当前参数组映射为设备参数组。

为了简化在设备上的材质渲染，我们把一个叫作 ApplyToDevice 的 API 加到我们的材质接口上。刚好在材质用于一个设备窗口之前，调用这个应用编程接口。如果那时材质还没有用在窗口上，材质将使用由 ID3DXEffectCompiler 所编译的效果，并为设备创建效果。此时，

D3DX 将把压缩的效果表达式转换给一个 ID3DXEffect 接口，我们可以用它来渲染效果。因为我们事先编译过那个.fx 文件，这个步骤显然比从.fx 文件直接到 ID3DXEffect 更快速，这也意味着不管多少窗口正在渲染那个效果，我们只编译一次。

30.6.4　ID3DXEffect

我们现在有一个设备专用接口，还附带几个应用编程接口。我们能在效果应用于当前设备时观察它，并检查那个设备的能力能否渲染它。如果设备满足不了效果要求的能力，FX Composer 将以蓝色的线框渲染对象；如果先前的编译步骤失败了，结果就是红色的线框。D3DX 效果被分解成几个技术，每个可能有不同的资源要求。因此，我们也保存一系列当前设备可用的技术，这样我们可以通过用户接口提供它们，而且如果当前的选择不能用，我们还有可能选择另一个不同的技术。

30.7　场景管理

FX Composer 包含它自己的场景图，它实现的方式非常简单。附加的输入端将格式为.nvb 和.x 的文件输入到场景图中。将来，我们还会根据开发者的要求，提供更多的格式。场景本质上是“一堆节点”，包含几何体、摄像机、光源等等。连接节点的是变换链接，它使场景图能够来回移动。系统是十分灵活的，所以我们将来可以根据需要，以不同的方式来回移动节点。比方说，我们可能希望从前到后地从光源的观察点穿过场景。由于节点和链是独立的实体，所以当前的场景图可以满足这种要求。

节点和节点链也有标准的动画连接参数，因此，它们可以在不同的关键帧，产生场景动画效果、蒙皮、摄像机摇摄等动画。

通过语义，D3DX 效果完全地指定了从应用到顶点 shader 的输入接口。使用一个叫做 D3DXGetShaderInputSemantics 的应用编程接口，找到效果所需要的顶点缓冲分布是可能的。得到这个以后，可能 FX Composer 就可以处理复杂的几何体流，并且把它们映射到当前的.fx 文件了。为了使这样做更容易，FX Composer 使用了一个几何体管道节点（geometry pipe node）的概念。一个几何体管道节点由几个几何体管道对象组成，其中每个都可以修改为一个几何体管束。束是一个几何体数据的集合，对于每个数据类型都有一个流对象。典型的束可能包含一条位置流，一条法线流和一条纹理坐标流。当几何体在流水线上流动时，几何体管对象能够通过增加数据、修正数据，或除去数据来修正束。然后把流水线顶部的最终的束转换到一个 Direct3D 顶点缓冲中去，使它与在几何体管上使用的效果文件相匹配。为了安排几何体束与.fx 文件的输入格式，几何体束流具有语义和用法索引。

这种方式允许创建插件，以满足当前效果的用法情景。举例来说，应该可能创建一个把“鳍”加入毛皮 shader 的插件。插件作者也可以把一条几何体流加入束，指示出现了一个“鳍”，或出现了一个渲染它的三角形流。

一个特殊类型的几何体管道插件总是放置在堆栈的底部。这个插件或是一个“形状”，或是一个“网格”。在它流上堆栈之前，它是几何体束的出发点。形状插件非常紧凑，包含如何建立形状的简单信息。通常通过输入预载的几何体数据来产生网格插件，当然插件作者也可以通过程序来创建一个网格物体。

蒙皮

.fx 文件的一个通常的问题是如何处理蒙皮。FX Composer 可以采取以下 3 个方法。

1. 如果当前的几何体束包含骨头和它的加权，但是指定的.fx 文件没有把它们当作输入，在把数据代入效果之前，FX Composer 会先把蒙皮数据代入主机。

2. 如果当前.fx 文件有骨头加权，而当前的几何体流没有，FX Composer 将会把骨头加权设定为 1，而且支持骨头矩阵调色板中的当前变换，由此保证一个未蒙皮的人物在蒙皮效果中的工作。

3. 如果当前.fx 文件包含骨头加权，而模型也有相同的骨头加权数，在假定当前的硬件能够控制骨头调色板大小的条件下，FX Composer 将把数据直接地传给效果。

30.8 小结

我们开发 FX Composer，就是希望为开发者创造一个工具，可以用之创建美妙的应用效果。我们也力图设计这样的应用程序，它将随着 API 和 GPU 一起发展，使我们能够持续地提供有用的工具。插件特性赋予开发者整合工作流程和 FX Composer 的能力，以及用简单的方式获得资产的能力。

我们希望这章已经使读者对编写这个复杂的 Direct3D 工具的方法有所了解，并且激发起读者使用.fx 文件进行实施的兴趣。Direct3D 效果能提供强有力的效果写作特性，而 FX Composer 一类的工具应该有助于促使它们在 3D 应用中的推广使用。由于具有构建效果的丰富的 IDE 和用于未来开发工具的坚实框架，我们相信 FX Composer 已经担当起了这一任务。

30.9 参考文献

Gamma, E., R. Helm, R. Johnson, and J. Vlissides. 1995. *Design Patterns.* Addison-Wesley.

Rogerson, Dale. 1997. *Inside COM.* Microsoft Press.

Tennison, Jeni. 2002. *Beginning XSLT.* Wrox Press.

Wyke, R. Allen, Sultan Rehman, and Brad Leupen. 2002. *XML Programming.* Microsoft Press.

第 31 章　FX Composer 的使用

Christopher Maughan
NVIDIA

这章对本书所附 CD 中的 FX Composer（也能在书的网站上发现）做一个基本介绍。可以借助 FX Compose 用.fx 文件产生高级的着色效果，.fx 文件是 Direct3D 的图形 API 状态的本文描述，通过 D3DX 效果框架任何 Direct3D 应用程序都可以载入.fx 文件。效果文件包含渲染状态、纹理状态、顶点和像素 shader，以及参数等。FX Composer 能用来产生这些文件，并把它们显示在输入的模型上。用.fx 文件也可以尽可能多地发展出其他的工具。

31.1　开始

我们推荐安装 DirectX 9 的“2003 夏季升级”版，它能够使所支持的全部.fx 文件运行。FX Composer 安装软件包含了所需要的所有东西，其中有许多媒体文件的示例，它位于 FX Composer 主安装位置的 MEDIA 目录下。为了达到最好的效果，应该使用流行的支持 DirectX 9 的图形卡。图 31-1 演示的是 FX Composer 运行一个应用了材质的载入模型。

从一开始 FX Composer 就被设计为一个开发 shader 的有用工具。尽可能灵活的设计提供了几个提高生产能力的高级特性。可以使用 FX Composer 观察 shader 的示例和项目，开发自己的.fx 文件，或通过高级的芯片剖析（profiling）特性，分析和改善 shader 库的运行性能。我们希望 FX Composer 能成为一个有价值的工具，为了符合要求，我们试图保证应用程序的扩展性尽可能强。也许想要做的不仅仅是产生 shader，也许对几何数据还有特殊的需要，也许想要使用 FX Composer 导入自己的应用程序数据格式，这些要求都不成问题。我们还力图提供一个完整的 SDK，用它可以创建插件程序，从而有权使用 FX Composer 引擎的所有特性。因为事实上，许多 FX Composer 功能本身就是插件程序，场景输入就是一个例子。

随之而来的是第一版的 FX Composer IDE 的简短概述。我们分别地描述每个部分，组合式地介绍所有的程序特点。在启蒙阶段和开始使用 FX Composer 构建效果之前，这种概述将会有助于对每个部分的特性获得初步感性认识。在本章结束的时候，我们装载一个简单的项目，以显示各部分

如何组合在一起。另外，对于使用的更深入信息，FX Composer 具有完整的用户指南。

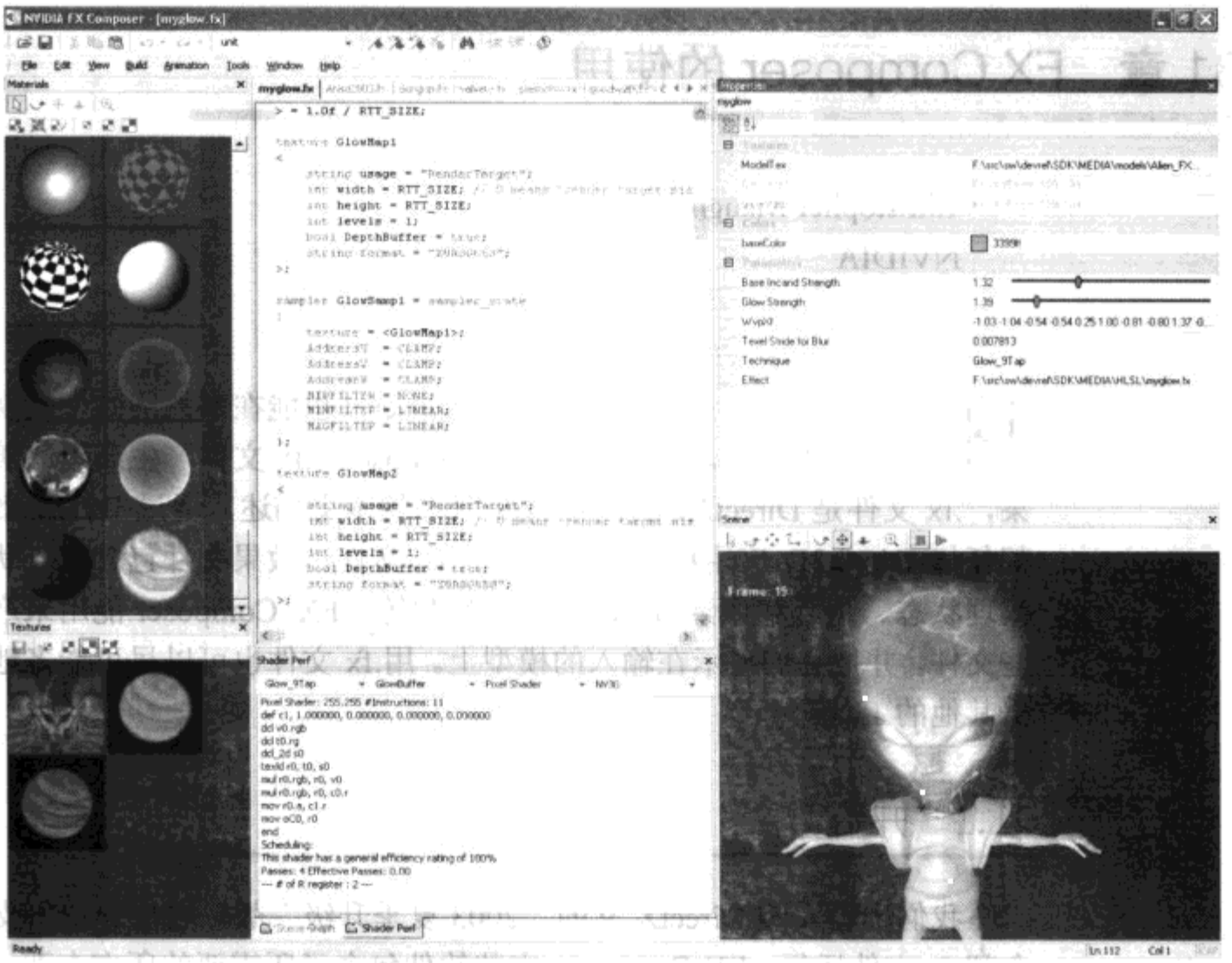


图 31-1 FX Composer 的主 IDE

31.1.1 材质面板

在把.fx 文件应用到场景的局部之前，可以装载.fx 文件并且事先查看它们。事实上，可以使用在图 31-2 中显示的材质面板，它是 shader 开发窗口，能够改变材质的大小，在把它们直接应用到场景之前就能快速地看到效果。因此，材质面板是一个有效的材质调色板。每个材质通过一组独特的参数表达效果文件的一个实例。多个材质可以归结于同一个.fx 文件，但是具有不同的参数。用这种办法，可以装载同一类型的多个材质，但是要修改它们的颜色和纹理，以适应场景中的模型。

一开始，材质面板只包含一个简单的材质，即一个基本的照明范例。注意，可以选择材质、旋转它、改变它的大小等等。每个材质实际上是一个缩小的 3D 视野，给出真实情景的感觉效果。一开始这看起来好像有点奇怪，因为许多 CAD 应用程序只显示静态的、二维的材质版本，但是效果却时常取决于光照明变化和凹凸映射，因此我们的 3D 版本使材质的观察变得非常容易。

用鼠标右键单击材质和物体的选择菜单，就可以给附着材质的物体选择不同的预览形状。现在，可以产生像茶壶、球体和立方体等形状的物体。任何人都可以写一个形状插件程序，而 FX Composer 引擎可以支持这里的任何一种几何体。删去材质和创造新的材质都很容易，一旦新的材质产生，它归属的效果文件就可以从属性面板上改变。然而，一个更快的产

生材质的方法，是从一个.fx 文件直接地装载它，只要选择“文件”|“打开材质”（File | Open Material）即可。

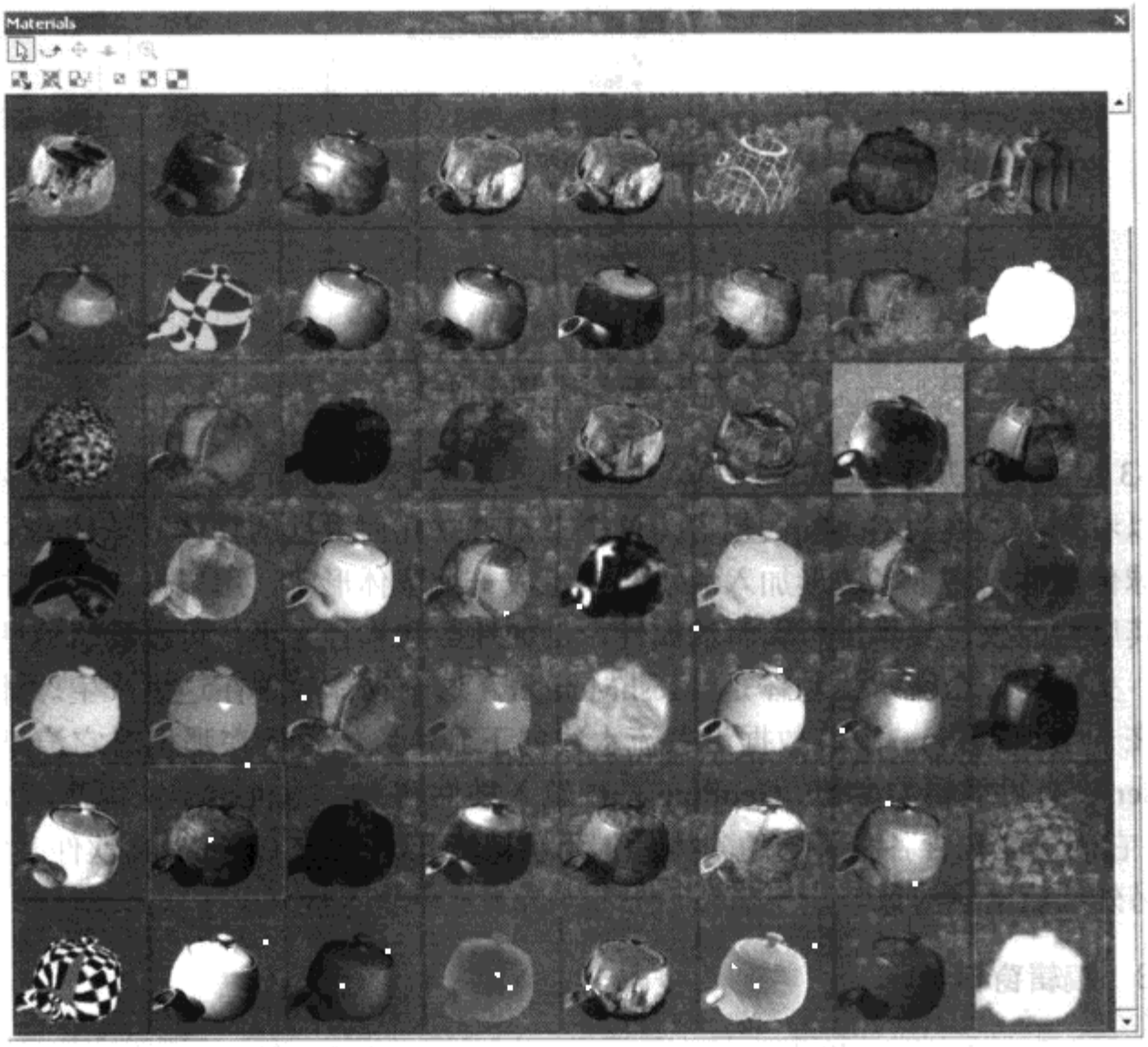


图 31-2　材质面板

有些材质可能显示为红色的线框或蓝色的线框。这些是特殊显示模式：红色指出无效的效果文件，而蓝色指出不能在图形设备上运行的效果文件。这些显示模式使单击坏的材质很容易，并把它们调到编辑器中进行修理。

当选择一个材质的时候，它在材质面板上的预览周围就出现一个红框。而选择当前场景的局部的时候，在与那个局部相关的任何材质的周围出现一个蓝框。这样，从面板上场景里的物体找寻材质就变得很容易。

31.1.2　场景图形面板

图 31-3 显示的场景图形面板，可以对树状表达的当前场景进行访问。在这里可以看到在场景中有什么物体，观察它们的属性，还能选择它们。从场景图形面板可以选择 FX Composer 的大多数物体，这时常是在复杂的场景中寻找物体的一种有用的方法。使用这个面板和属性面板，可以巧妙地处理场景中的任何物体。举例来说，图 31-3 的场景图形面板表达的场景中，有一个单个的默认值聚光灯。沿着场景图形树，就能找到光源体，要改变它的属性，就用鼠标右键单击它来进行属性选择。于是现在的属性面板显示光源信息，这时就可以改变光源的颜色或任何其他参数了。

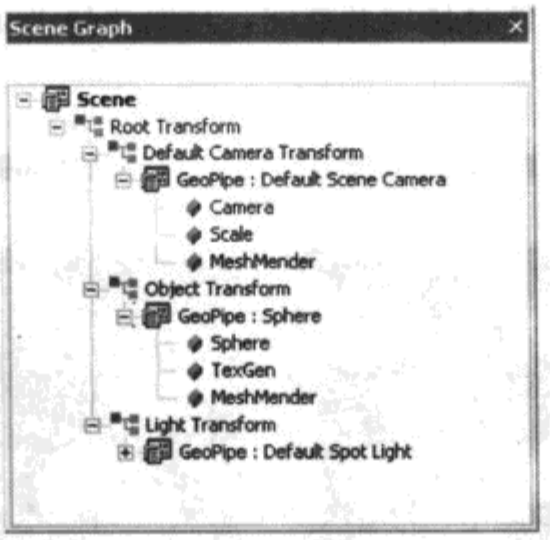


图 31-3 场景图形面板

图 31-3 也显示了几个 GeoPipe 物体。GeoPipe 是在 FX Composer 中保存几何体的节点。从图中可以看到，扩展的 GeoPipe 包含为这个节点建立几何数据的物体。球体的 GeoPipe 的第一项是球体插件，它把多边形加入流水线来组成一个球体形状。用鼠标右键单击选择插件可以得到其属性面板，显示出可编辑的属性，例如环数和半径大小。可以编辑这些属性从而改变球体。另外的一个流水线物体是 MeshMender。这个物体放置在这里产生球体的切线空间矢量。在一个 GeoPipe 中，数据沿着流水线向下到达终端，就在那里对它渲染。目前对 MeshMender 没有可编辑的参数。GeoPipe 是由输入插件和 FX Composer 来构建，以适应所产生的形状。它们可能包含各种不同的安排。使用为未来计划的 FX Composer 软件开发工具包，可以产生另外的流水线对象。

31.1.3 编辑窗口

在图 31-4 中所显示的编辑窗口，用来修改和创建.fx 效果文件。语法加亮用不同颜色显示关键字，使 shader 源代码更容易读出。IntelliSense 是为 Direct3D 设备状态内建的，可以在编辑.fx 文件的时候出现建议的关键字。举例来说，如果要把 Direct3D 渲染状态 fillmode 键入一个.fx 文件之内，编辑器会建议下列 3 个选项：

- 实体
- 线框
- 点

它们等同于 D3DFILL_SOLID，D3DPILL_WIREFRAME 和 D3DFILL_POINT，设置在渲染状态 D3DRS_FILLMODE 内。

书签、查找和撤销/重复也在编辑中起着重要作用。撤销/重复记忆所有的编辑，而且允许返回到刚开始编辑文件的地方。书签提供一个很快地跳到 shader 关键点的方式，而使用在工具栏上的搜索栏可以很方便地在当前文件中找到本文字符串。

排在窗口顶部的标签允许在多个.fx 文件之间快速转换。选择材质面板上的新材质，将会自动地把在编辑器中相关的.fx 文件调出来。当按工具栏上的编译键的时候，所有的相关材质都会根据对文件所做的修改而刷新。关于构建过程的所有信息输出在日志和任务面板上；可以单击任务面板上的错误，自动地加亮它们在编辑器中的文件和位置。这使得找到和改正语法错误变得容易。

图 31-4　编辑窗口

31.1.4　Shader Perf 面板

时常需要从给定的效果文件观察所产生的像素和顶点 shader 汇编代码。图 31-5 的 Shader Perf 面板显示当前所选择的材质 shader。有 4 个可以顺着向下移动的盒子。第 1 个选择所检查的技术，第 2 个选择遍，第 3 个是那遍的顶点或像素 shader。一旦着手处理一个像素 shader，就能使用第 4 个下拉框看时序安排信息。这里能看到 shader 如何在所支持的 GPU 配置上运行。一连串的芯片专用信息被附加在 shader 上，而且提供了效率总结。这样，就可以调整怀疑的.fx 文件以达到更优的性能。图 31-5 显示一个效果文件在 GeForce FX 5900 芯片上的时序安排。

将来，我们会在 Shader Perf 面板上增加对新设备的支持，扩展这个工具的用途，以便对所有 NVIDIA 芯片上的 shader 代码的性能予以评估。

31.1.5　属性面板

图 31-6 显示的是 FX Composer 的属性面板，通过一个美工友好的界面，使得用户能够调节各种不同的参数。当选择了一种材质的时候，属性面板将会显示相关.fx 文件的参数目录。可以对参数进行调节，或许编辑一种颜色或浮点值，从而改变某个效果的外观。图 31-6 显示一种称为 check3d 的材质的属性。因为.fx 文件的作者指定参数 UIType 是颜色，所以属性面

板知道应该被显示为颜色的值。由于作者为了指出每种彩色的 UIName 而附加了注释，因此颜色在面板上具有易读的名字。这些信息和其他对.fx 文件增加的细节，都包含在 FX Composer 用户指南中。

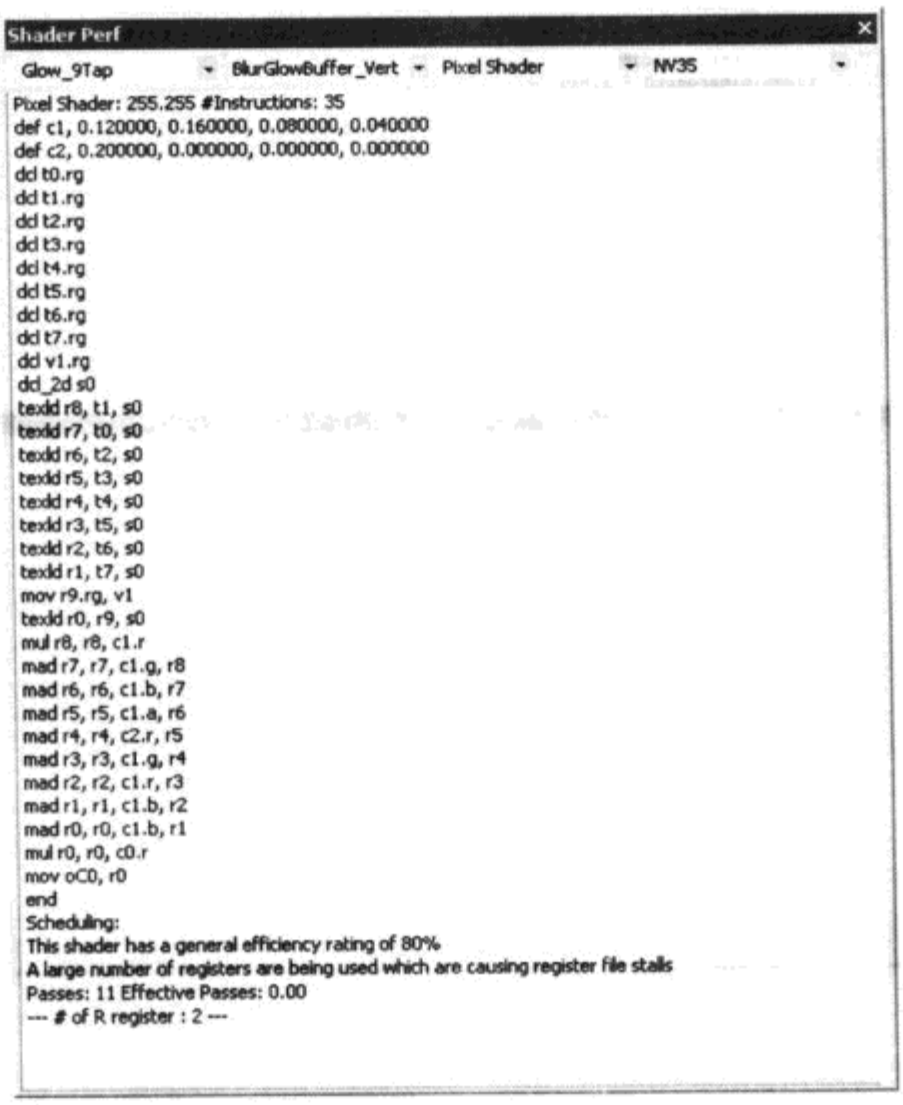


图 31-5 Sharder Perf 面板

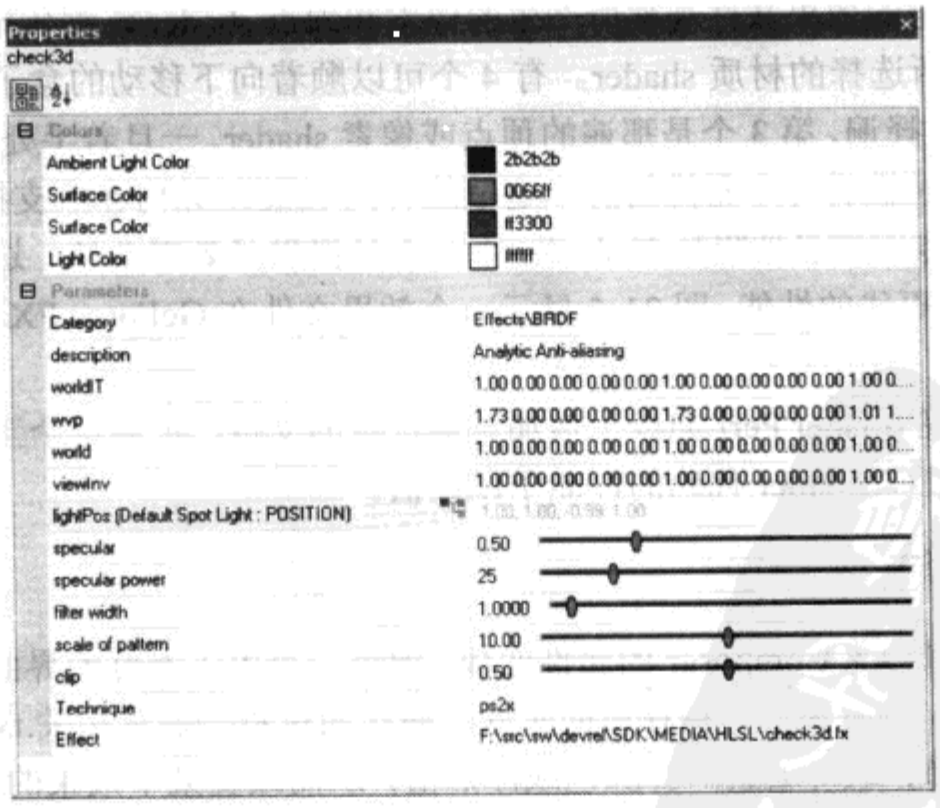


图 31-6 属性面板

注意图 31-6 中在 lightPos 属性上附加了一个小图标。单击图标就会打开图 31-7 所显示的参数对话框，从而能将场景中的物体与属性相连接。因为在.fx 文件中对这个参数提供注释 UIOBJECT = POINTLIGHT，所以可以进行对话。图 31-7 显示的参数对话框使人们可以打开光源选择项，还提供了根据所选择的项如何计算参数值的详细信息，也显示了当前值。从对话框我们可以看到，现在调用了场景里的“默认点光源”，正在计算一个光源的位置。也可以用这个对话框把参数与场景“断开”，自己用手编辑它们。

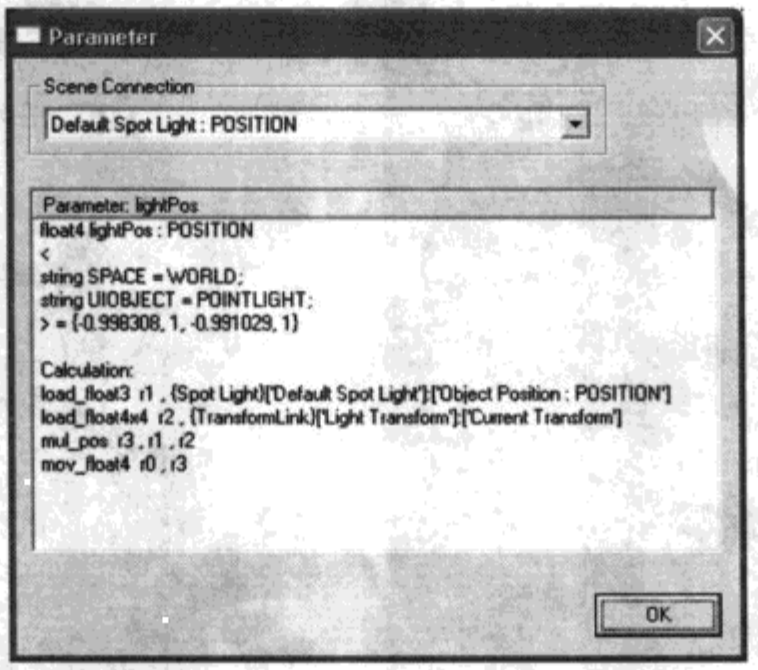


图 31-7　参数连接对话框

除了材质的属性之外，属性面板还会显示物体的属性：如果选择的物体是一个球体，面板将能够改变球体的尺寸和它的镶嵌布置。如果编辑球体的属性，可以在主要的显示窗口看到其尺寸的变化。同样地，选择一个纹理将会提供关于那个纹理的信息；但是一般而言，不能在当时从属性面板修改纹理，它们要通过效果文件注释来设置。除了修改 shader 中的值之外，也可以为当前的场景局部指派面板中的属性。例如存在一个光源，则需要 shader 总是在有的地方保持当前光源的位置，这时可能就得这样做了。

属性面板是 FX Composer 参数的场景相关的编辑器，它能够编辑颜色值、数字、矢量和矩阵。对于矢量和矩阵，属性面板提供滚动条和编辑器；其他的控制器根据被编辑属性的要求而出现。注意不是所有的属性都可以被编辑；不能够被编辑的那些显示为灰色。FX Composer 的典型工作流程是先选择要修改的某样事物，然后使用属性面板进行修改。总是在当前的关键帧中修改数值，如果合适，则浏览场景的其他关键帧对它们插值。属性面板只是修改 FX Composer 中的信息的关键部分，而且它总是活动的：当移动场景中的物体时，面板上的对应值也同步地变化。

31.1.6　场景面板

场景面板包含当前场景的几何体和应用的材质。在这个窗口中，可以选择输入模型的区域，并对它们应用材质。最初的场景是一个简单的球体，伴有一个单光源，光源位于左上部。可以进行控制来调节整个的场景和个别的物体。通过右击弹出菜单，可以得到所有的这些选项。

图 31-8 显示一个示范场景面板。注意，几何体的范围显示为蓝色，场景的范围显示为红色，而所选择的形状由小的白色记号标出。另外，我们已经打开“显示变换”选项，使物体的移动能够看得见。在这个场景中，它引起从世界中心到左上角光源的一条线。在有一个蒙皮人物的场景中，这些线代表骨头的移动。

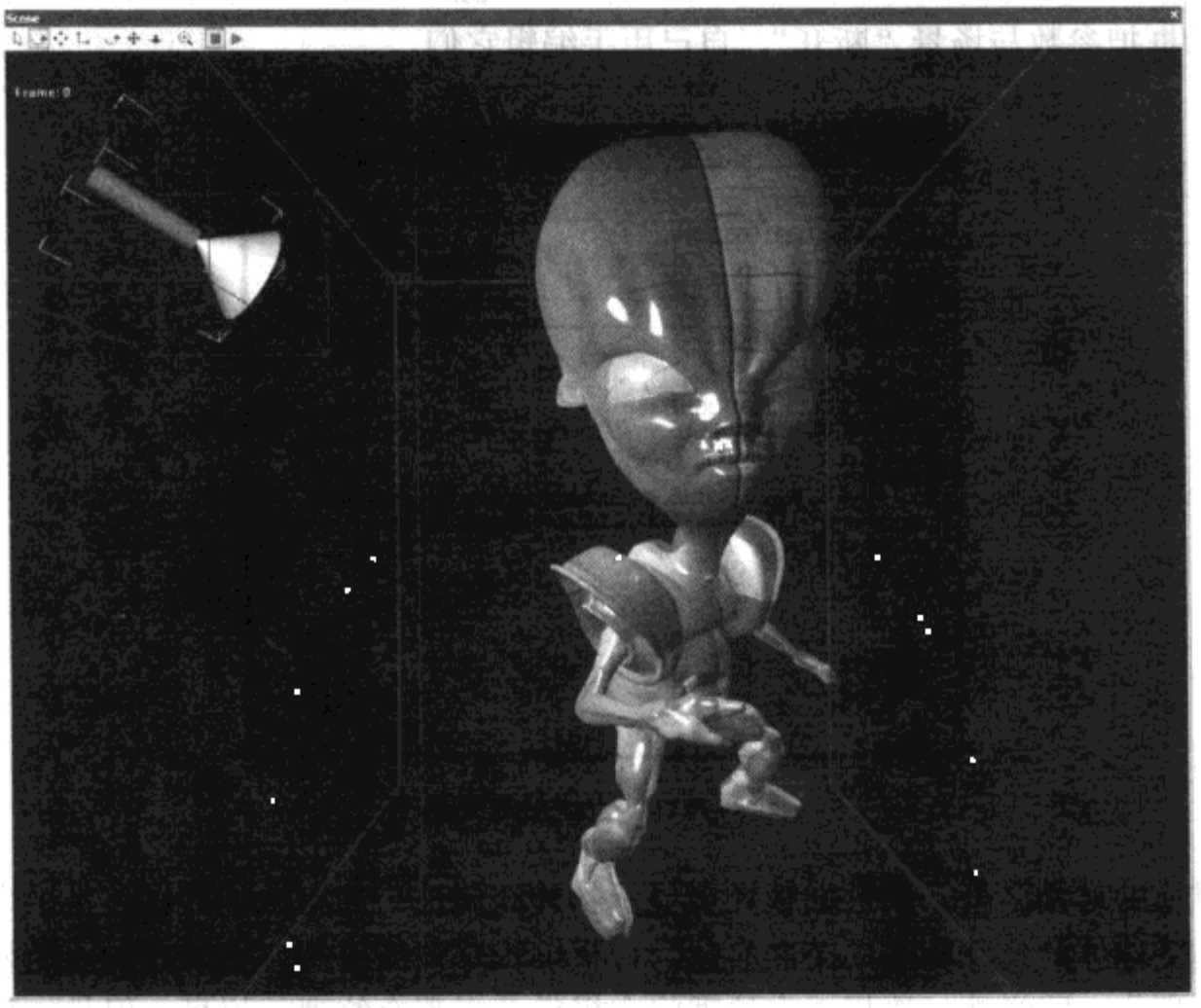


图 31-8 场景面板

FX Composer 支持简单的形状插件程序，使得用户在简单的测试场景中显示它们自己的物体，这些物体也可以用在材质面板上。鼠标右键总是提供用这些形状构建场景的快速方法，允许用户在不同类型的几何体上很快地测试一个 shader。使用所提供的.fx 文件和.nvb 文件的输入插件也能输入几何数据。开发者还可以为定制的场景格式添加它们自己的输入插件。图 31-8 的场景就是从一个.nvb 文件输入的。

用户可以用场景窗口顶部的按钮操纵场景。可以用摄像机运动（摇摄/移动/转动）和物体运动（转换/缩放/转动）的按钮来挑选场景中的物体。如果场景包含动画数据，在工具栏上的播放钮能够重放，而窗口中的计数器显示动画运行的当前关键帧。若想要看个别场景物体的属性，用鼠标右键单击它们并选择属性，时常会把在属性面板上的有用控制调出。甚至场景的背景也有它自己的属性，可以使人们看到场景范围，并且对背景颜色一类的选项进行修改。用鼠标右键单击菜单能够选择和删除场景项目，也可以产生物体，例如点光源、聚光灯和定向光源，使用属性面板可以把它们与场景中的材质连接起来。

31.1.7 纹理面板

当选择了一个材质的时候，纹理面板显示所有与该材质有关的纹理。这些纹理可以是从

文件装载的，也可以是通过.fx 文件里的函数程序式地产生的，或者是渲染目标。图 31-9 显示的是一个渲染目标的例子，带有从 glow.fx 文件来的材质。把“模糊”的纹理装载入材质面板并选择它，就可以在纹理面板中看到它。在材质面板中旋转发光材质，接着会显示即时更新的模糊纹理。可以容易地看到正在产生的纹理效果，如果必要的话还可以调试运算法则。单击纹理面板工具栏上的保存图标，就能以纹理本身格式的.dds 文件存储纹理。这个属性使人们能够用效果文件 shader 制作一个纹理。可通过工具栏的其他钮改变纹理的观看尺寸，也包括对检查像素准确性很有用的“真实的尺寸”，尽管它可能很大。

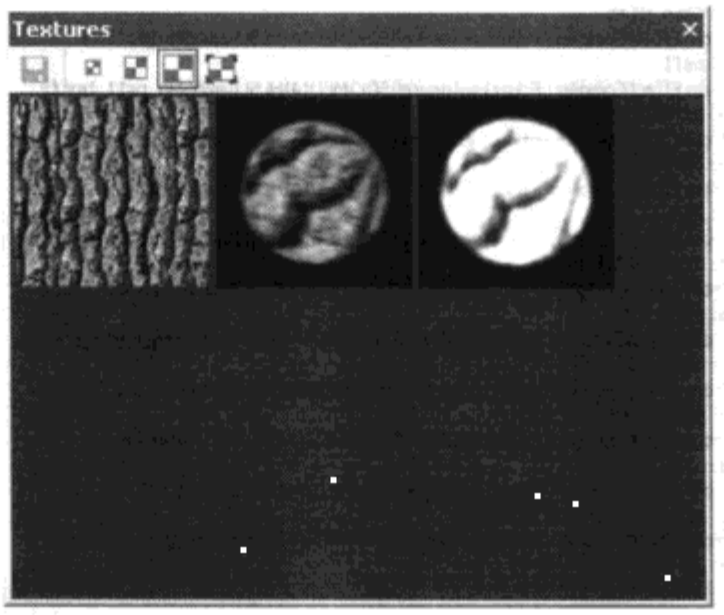


图 31-9　纹理面板

31.1.8　任务面板

任务面板（启动时它藏在日志面板之下）包含当前错误和警告的目录。见图 31-10。举例来说，对.fx 文件的不正确的编辑和装载纹理资源的失败，都可能引起错误。警告提供更一般的指导，例如可以指出因为当前的图形硬件不支持某效果，所以不能够使用它；或在一个.fx 文件中发现了可疑的语法，例如由于打字错误而不能识别的语义或注释等。

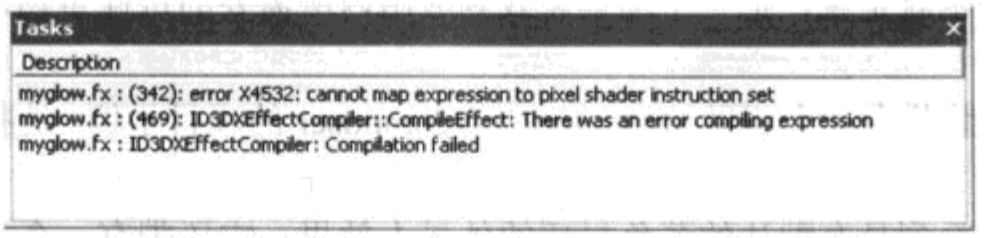


图 31-10　测试面板

如果显示在任务面板的错误起因于一个 shader 编辑问题，可以用鼠标单击错误，而 FX Composer 编辑器会自动地装载相关的.fx 文件，并且用红色标注错误。对于刚编辑完的.fx 文件新出现的错误，任务面板将会自动地用加亮来标注。

31.1.9　日志面板

图 31-11 的日志面板显示报告事件的细节，例如载入项目和材质这类的事件。在这里可以追踪构建的过程、模型的装载（例如所输入的.x 文件），以及获得 FX Composer 现在正在

做什么的信息。

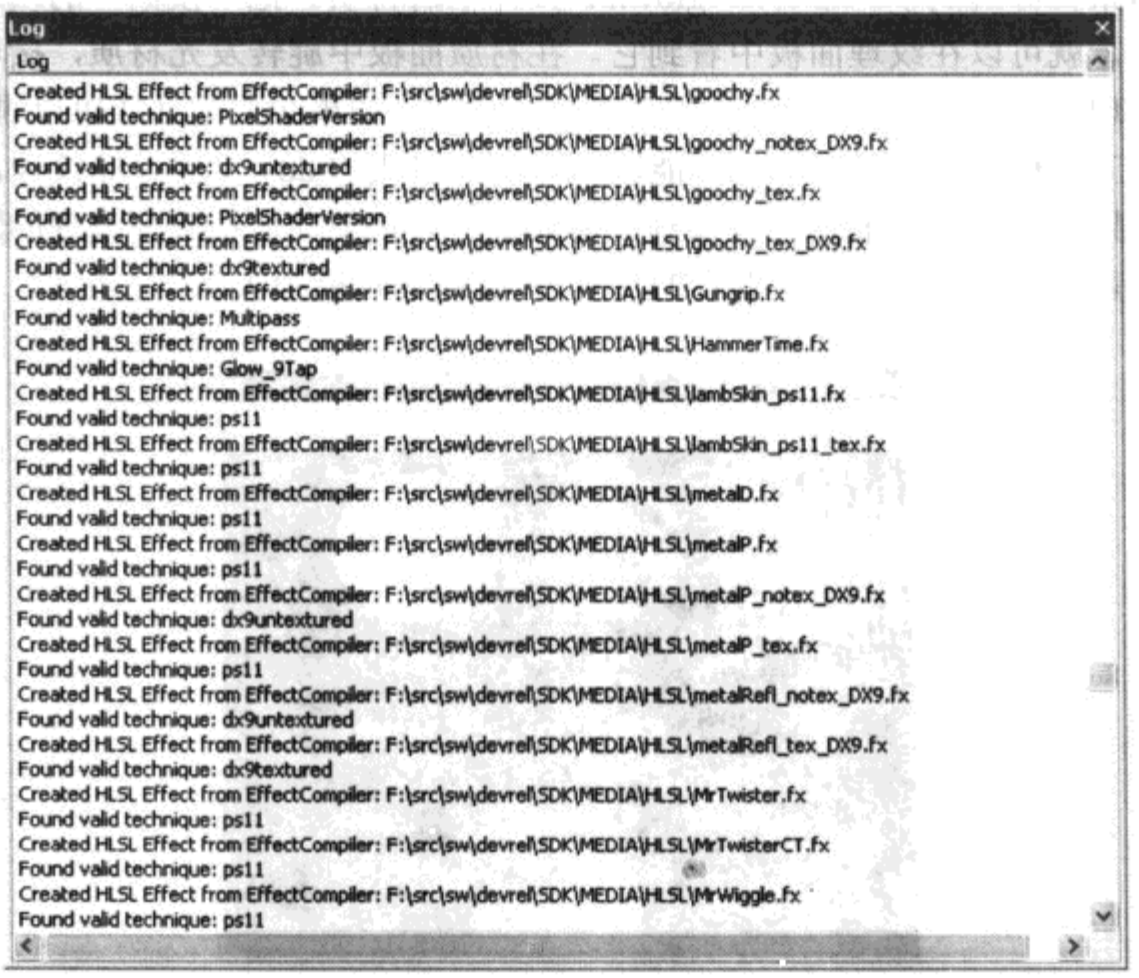


图 31-11 日志面板

31.2 项目示例

熟悉了 FX Composer 的基本面板和窗口后，现在来试着装载一个项目示例。示例媒体装载在 FX Composer 目录下，其文件夹叫做 MEDIA。在 MEDIA 下，可以找到一个包含项目示例的.fxcomposer 目录。FX Composer 项目文件有.fxcomposer 扩展名，包含几何体、材质、纹理和其他的元素，也就是说，把一个场景装载到应用内所需要的每样事物，而且它还带着应用的材质。

FX Composer 装载有一个叫做 alien_gooch.fxcomposer 的项目示例。选择“文件”|“载入”项目来装载它。图 31-12 中显示了被装载的部分 FX Composer 应用窗口。

这里我们可以看到具有默认的黄色材质的外星人效果，它还带有一个逐像素的镜面反射高光。我们在材质面板中选择了材质，并使用属性面板将颜色换成绿色。我们还放映了 10 帧的动画，可以看到外星人正忙着在地板上蹑手蹑脚地移动呢。

为了对外星人应用不同的材质，我们可以用鼠标右键单击材质面板（或使用文件菜单），打开新的材质。一旦打开材质，我们选择外星人去应用新的效果，并双击新材质去应用它。也可以不用.fxcomposer 文件，而使用文件菜单上的输入场景指令输入另外的模型。现在 FX Composer 能输入.x 和.nvb 文件。如果准备好了要对另外一个 FX Composer 用户发送自己的场景，可以选择“文件”|“保存”项目，完全地存储所设置的工作空间。重载这个工作空间时，将正确地恢复最后一次存储的场景。

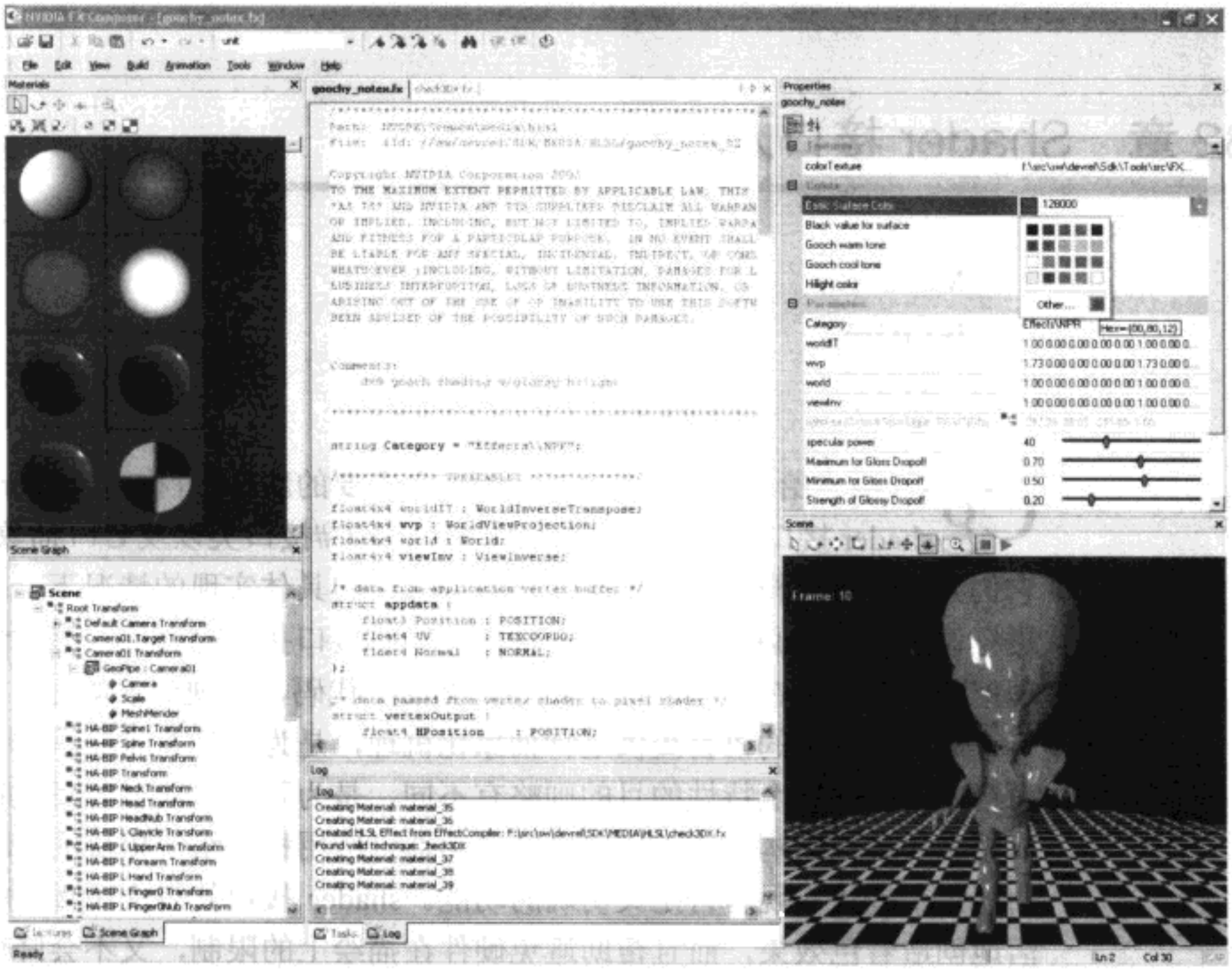


图 31-12　一个在 FX Composer IDE 上的示范项目

31.3　小结

这章是使用 FX Composer 的第一版而编写的，它包含在书的 CD 上（在书的网站也可得到）。FX Composer 的应用正在日益增长，而且它能够满足一个功能强大的 shader IDE 的要求。可以通过 http://www.fxcomposer.com 得到它并进行升级。若需要 FX Composer 的新特性，或想报告一个程序缺陷，请发电子邮件给 fxcomposer@nvidia.com。

希望大家喜欢使用 FX Composer！

第32章 Shader接口入门

Matt Pharr
NVIDIA

Cg 1.2版本给Cg编程语言引入了一个重要的新特性，称为shader接口。shader接口提供的功能类似于Java或C#接口类以及C++抽象基类，它们允许代码这样写：可以在未知抽象接口的具体实现的情况下，就对其进行调用。用户必须提供在Cg程序运行库接口执行的对象，然后经过处理把各部分合并在一起，形成最后的shader代码，在硬件上运行。

给面向对象的语言提供这种抽象化机理，是为了开发大的软件系统；而给Cg加入这个特性的目的则略有不同，是为了使它能够容易地在运行库用多块源代码构造shader。shader是通过应用程序组合模块的功能构建的，每个模块实现各自定义明确的功能。shader接口使得应用程序能更灵活地创造着色效果，而且帮助避免硬件在描绘上的限制，又不会降低最终的编译代码在GPU上运行的质量或效率。

举例而言，我们考虑写一个片元程序，目的是计算一个表面的着色颜色，同时还要能应用于各种不同类型的光源：点光源、聚光灯以及复杂的投射光源，它既带有阴影映射，又程序化地定义了随距离衰减的函数等。一种极无效率的方法是把“光源类型”的参数代入程序，然后用一系列的if检测，来决定使用哪种光源模型。可以采用的另一种方法是使用预处理程序，而且应用程序可以使用#ifdef的防卫功能，以便手动编译程序的一个专门版本，来针对实际上正在使用的那种光源。遗憾的是这种方法稍微有点笨拙，而且它不能很好地扩展到多个光源，尤其是如果事前不知道光源的数目就更难办了。

shader接口对这种问题提供了一个类型安全的解决办法。这种方法既减小程序的负担又不牺牲性能。开发者可以借助用shader对象接口所表示的大功能块（例如基本的“光源”接口）来写一般的shader。当对这些接口开发出新的特殊实现时，旧的shader可以使用它们而不需要重写。使用这种特性的应用程序，不再需要实现字符串连接的程序，就可以立刻合成Cg源代码，这样就避免了那种既会出错又很笨拙的方法。

在这章中，我们描述shader接口的基本语法，并指出如何用Cg的编译器和运行库来使用它们。为了获得一个应用程序shader接口可能的感觉，我们演示了3个如何把它们应用于Cg的例子：（1）在两个矢量规范化的方法之间进行选择；（2）支持在shader中可变数量的光源；（3）以

一般的、分层的模式描述材质和纹理。

在 Cg 1.2 中两个另外的新特性补充了 shader 接口，改善了 shader 的灵活性和效率。第一个特性是，不规定数组大小，允许程序员决定和使用大小不固定的数组。在程序运行之前，必须向 Cg 运行库提供数组大小。在编写的 shader 使用变化数量的光源参数组、蒙皮矩阵时，这个特性显得格外方便。第二个特性是能够把 shader 参数标记为常数值；如果知道一个特殊的参数是常数，编译器可以用它来充分地改善 shader，将代码优化。举例来说，如果一个 if 测试只依赖于常数参数，而且这个参数可以在编译期充分地计算，那么编译器可以根据检测值抛弃模块里面的所有代码，或者无条件地执行它。

32.1　shader 接口的基础

我们从一个片元程序的简单例子开始，它使用 shader 接口对两个矢量规范化的方法进行选择：一个方法基于立方体映射的纹理查询，另一个则基于数字计算得到规范化的矢量。现在应用要做的是，根据图形硬件的能力，在运行库从这两个方法之中选出性能最佳的方法。这虽然是一个简单的例子，但是它有助于理解在 shader 接口背后的一些主要思路。

首先，我们描述一个用于矢量规范化的一般接口。关键字 interface 引出一个新的接口，它必须由一个有效的标识符来命名。然后在大括号{}之间声明由接口提供的方法，接口的声明以一个分号结束。

```
interface Normalizer {
  float3 nrm(float3 v);
};
```

在接口里可以声明多个方法，同时应用通常的函数重载规则。比方说，假如我们已经声明了一个 half 3 nrm(half3)函数，那个适合的函数可能要基于代入它的矢量类型来进行选择。

给出了这个声明，我们就可以开始写一个 Cg 程序，它取 Normalizer 接口的一个实例作为一个参数。列表 32-1 显示一个非常简单的片元程序，它计算从一个点光源来的漫反射项。Normalizer 接口内的 nrm()函数被调用两次：一次规范化从光源位置到着色点的光矢量，另一次规范化从顶点 shader 来的被插值的法线。

列表 32-1　简单的片元程序，它使用 shader 接口来隐藏矢量规范化技术细节

```
float4 main(float3 Pworld : TEXCOORD0,
            float3 Nworld : TEXCOORD1,
            float3 Kd     : COLOR0,
            uniform float3 Plight,
            uniform Normalizer normalizer) : COLOR
{
  float3 L = normalizer.nrm(Plight - Pworld);
  float3 C = Kd * max(0, dot(L, normalizer.nrm(Nworld)));
  return float4(C, 1);
}
```

通过使用一个 shader 对象接口，尽管并不知道在运行库实际上如何进行规范化，也可以基于某些 Normalizer 实现写出这个 shader。

接着需要定义那个接口的一个或两个实现。第一个是 StdNormalizer，它使用 Cg 标准库程序 normalize()来规范化矢量，用矢量的分量除以它的长度。我们定义一个新的结构，并通过用冒号和接口名跟随结构名，来指定它实现的 Normalizer 接口。在已经承诺实现接口以后，结构就必须对接口中所声明的所有函数的实现给以定义。正像在 Java 和 C#中一样，必须在结构定义里定义方法的实现。

```
struct StdNormalizer : Normalize {
  float3 nrm(float3 v) { return normalize(v); }
};
```

我们也可以定义一个 shader 对象，它使用一个立方体映射纹理来规范化矢量，已经把纹理的面初始化了，这样它们就能为规范后的矢量保持其对应的方向。在下一节中，CubeNormalizer 声明它结构里的一个 samplerCUBE 参数，来容纳这个立方体映射，并且它的 nrm()实现以平常的方式使用这个参数。

```
struct CubeNormalizer : Normalize {
  samplerCUBE normCube;
  float3 nrm(float3 v) { return texCUBE(normCube, v).xyz; }
};
```

如果给定一个程序，它的接口参数像前述代码中的 Normalizer 一样，那么在程序能运行之前，需要使用 Cg 运行库去指定接口的一个特别实现。当我们调用 cgCreateProgram()或 cgCreateProgramFromFile()的时候，首先需要告诉运行库，不要按照默认那样，立刻着手编译我们给它的程序。否则，编译将会失败，因为还没有把接口约束到一个特定的实现。cgSetAutoCompile()程序是控制这个行动的；这里把它设定为：直到我们明白地告诉它要编程时重编译程序（cgSetAutoCompile()也有另一种可行的设定，即根据需要自动地进行重编译，但是额外的重编译可能要耗费资源）。

```
cgSetAutoCompile(context, CG_COMPILE_MANUAL);
CGprogram prog = cgCreateProgramFromFile(context, CG_SOURCE, "frag.cg", profile, NULL,
                 NULL);
```

被 cgCreateProgramFromFile()返回的 CGprogram 句柄还不能被载入 GPU。首先需要把 StdNormalizer 结构的一个实例与程序的 normalizer 参数连接起来。所幸的是，运行库允许单纯地通过 Cg 运行库的应用编程接口的调用，来创建 StdNormalizer 结构的一个实例。我们调用 cgGetNamedUserType()，把字符串型的名字“StdNormalizer”转变成 CGtype 值，然后与那个类型一起使用 cgCreateParameter()，从无到有地创建一个 StdNormalizer 实例。

```
CGtype nrmType = cgGetNamedUserType(prog, "StdNormalizer");
CGparameter stdNorm = cgCreateParameter(context, nrmType);
```

如果 StdNormalizer 有结构里的参数，我们现在可以把它们初始化，初始化时使用 cgSetParameter* ()程序，或 profile 特定的参数设定程序，其中包括把纹理单元绑定到取样器的那些程序，虽然并不需要对 StdNormalizer 做这些（但是，我们也许需要把一个合适的立方体映射纹理与 CubeNormalizer 结构的 normCube 参数约束到一起）。

一旦创建了 StdNormalizer 的实例，剩下的一切就是从 main()程序的参数列表中为接口获

得一个参数句柄，以及连接我们刚刚产生的 StdNormalizer 了。

```
CGparameter normIface = cgGetNamedParameter(prog, "normalizer");
cgConnectParameter(stdNorm, normIface);
```

我们现在已经有了一个充分详细说明的程序，它的接口参数全都受实现约束了。我们通过手工程序编译完成了这个工作。在这之后，我们就能以正常的方式使用这个程序，把它载入 GPU，并能使用它进行渲染。

```
cgCompileProgram(prog);
```

如果以后想要对程序的这个实例改换一个不同的 Normalizer 实现，我们就再一次调用 cgConnectParameter()，连接新的参数和重编译新的程序。此外，如果我们在同一个 CGcontext 有多个程序，它们全都可以分享 StdNormalizer 的同一个实例，通过对 cgConnectParameter() 的多次调用，把它连接到多个参数，而不需要产生 StdNormalizer 的多个实例。

32.2　一个灵活的光源描述

shader 接口的一个更有趣的应用，是用它们写一个支持任意类型光源的 Cg 程序。一个常见的问题是要写一个程序能支持任意数目的光源，而其中每个光源的类型又可能是完全不同的。shader 接口结合 Cg 1.2 的另一个新特性，即不规定尺寸的数组，给这个问题提供了一个清楚的解决办法。

以前这个问题的解决办法是对每个光源渲染一次场景，再把每个通道的结果加在一起计算最终的图像。这种方式效率很低，因为它需要多次运行顶点程序，每次运行又导致片元程序不必要的重复计算。它也不容易使用，因为每增加一种光源的新类型，就必须针对新光源模型，给每一个现有表面 shader 重写一个新实例。而对于 shader 接口，如果我们已经用同样的光源接口实现了多种不同类型的表面反射模型，那么在增加一个新类型的光源时，就不需要修改现有 shader 的任何源代码了。

一个基本的、一般的接口能够很好地抽象不同类型的光源行为，建立正在着色的 P 点与光源的联系。光源依次影响 P 点的入射光方向的变量 $\boldsymbol{L}$ 的初始化，并且由它得到 P 点的光照量。图 32-1 显示以这种模式计算光照明的基本设置。

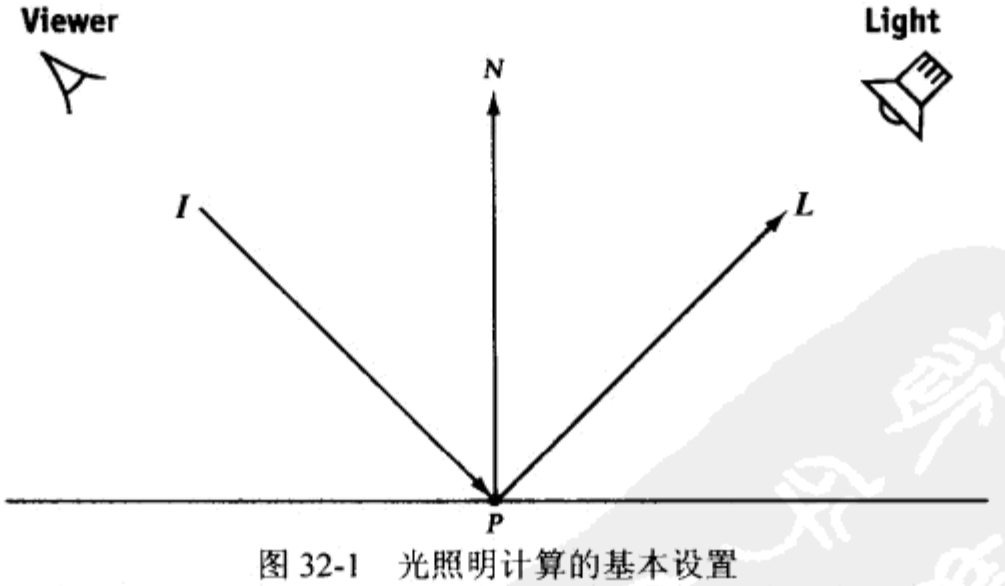


图 32-1　光照明计算的基本设置
光线沿着方向 $\boldsymbol{L}$ 到达点 P；表面法线是，从视点来的入射方向是 $\boldsymbol{I}$

```
interface Light {
  float3 illuminate(float3 P, out float3 L);
};
```

给出这个接口，我们就可以定义一个简单的阴影映射的聚光灯以实现接口。这个聚光灯为它的阴影映射保存一个 sampler2D，并且使用一个立方体映射纹理来描述光线的分布角。光源的三维位置和它的颜色构成光源的定义。对于这个例子，我们假定光源位置和着色点的位置已经表达在相同的坐标系中，所以不需要再把它们转换到一个共同的坐标系。

这个光源的实现相当简单，使用光源的位置很容易计算光线方向 $\boldsymbol{L}$，而到达 P 的光线数量是三者的乘积，这三者是：光的颜色、从阴影映射查询的可见度因子，以及在 P 点受光方向上聚光灯强度的纹理查询。见列表 32-2。

列表 32-2 根据光源接口的实现而定义的聚光灯光源

```
struct SpotLight : Light {
  sampler2D shadow;
  samplerCUBE distribution;
  float3 Plight, Clight;
  float3 illuminate(float3 P, out float3 L) {
    L = normalize(Plight - P);
    return Clight * tex2D(shadow, P).xxx * texCUBE(distribution, L).xyz;
  }
};
```

用这个接口能容易地实现各式各样的光源类型：点光源、方向光、投射一个纹理到场景的光源、线性地或随距离平方而衰减的光源、由一个一维纹理映射定义而衰减的光源，以及许多其他光源。

在列表 32-3 的片元程序中，Cg 代码在所有照亮表面的光源上循环，调用每个光源的光照明函数 illuminate()，然后累积每个光源基本漫反射着色项得到总和。Cg 新的大小不定数组的特性使得我们这样声明一个长度不定的数组：在数组名字 lights 后使用[]句法表示数组长度。我们立刻就会看到，在 shader 最后执行之前，如何使用 Cg 运行库来设定大小不定数组中元素的数目。也会注意到数组里有一个给出其中元素数目的长度成分，这让我们要为一个大小不定数组的所有项写一个循环，甚至在本身不支持可变循环迭代的 profile 上写这个循环。

列表 32-3 为任意数目的光源取接口参数的片元 shader

```
float4 main(float3 Pworld : TEXCOORD0,
            float3 Nworld : TEXCOORD1,
            float2 uv     : TEXCOORD2,
            uniform sampler2D diffuse,
            uniform Light lights[]) : COLOR
{
  float3 C = float3(0, 0, 0);
  Nworld = normalize(Nworld);
  float3 Kd = tex2D(diffuse, uv).xyz;
  for (int i = 0; i < lights.length; ++i) {
    float3 L, Cl;
    Cl = lights[i].illuminate(Pworld, L);
```

```
    C += Kd * Cl * max(0, dot(Nworld, L));
  }
  return float4(C, 1);
}
```

在这个 shader 能被用于一个程序之前，一定要使用 Cg 运行库做两件事。首先，必须设定大小不定的 lights[]数组的最后长度，第二，数组中每个接口实例必须与一个实现 Light 接口结构的实例绑定在一起。对于第一个问题，Cg 运行库有一个程序设定大小不定数组的长度：

```
void cgSetArraySize(CGparameter param, int size);
```

第二个问题按照早先的例子解决，通过调用 cgCreateParameter()创建光源的具体实例，通过调用 cgConnectParameter()把那个实例与光源接口的数组中的输入连接起来，对于数组中每个光源都这样做。正如在那个例子中一样，同样的光源实例可以被相同的 Cg 上下文中的多个程序共享。

由于已经使用了 shader 接口和大小不定数组，我们可以编写这样的 shader，它能进行光源接口的任何实现；也很容易编写这样的 shader，它的编码不必受光源数目固定的约束。我们已经能够以清楚的方式写非常灵活的 Cg 程序，并且不会影响运行的效率。只有当光源的特定数目和它们的具体类型已经设定之后，才会产生最终的 GPU 指令，因此使用 shader 对象和大小不定数组特性时，性能不会降低。

注意，假如改为对每个光源重新渲染一次场景来解决这个问题，那么我们就不得不为每遍重规范化 Nworld，重做 Kd 纹理查询。这还仅仅是简单的操作，如果片元程序做更为复杂的纹理操作，那么运行消耗会相应地变得更糟。

32.3　材质树

最后的例子显示了接口的构成如何引出描述材质的有趣方法。描述图形中复杂材质和纹理的最成功方法之一，是把它们分解成树或网络，一堆节点联系在一起，通过执行简单操作的各个节点来计算一个复杂的着色模型（Maya 的 HyperShade 和 3d max 的材质编辑器，是这种方法的两个众所周知的例子）。这不但是一个在内容创建应用程序中描述复杂材质的方便方法，而且也是一个在应用程序中构成材质和纹理库的优秀方法，使得它们具有极大的灵活性在运行库建立表面的描述。在 shader 接口增添到 Cg 之前，没有一个清楚的办法，能够在 Cg 程序中表示以这种方式描述的材质。

在这一节中，我们将显示 shader 接口是怎样把这种材质描述的方法清楚地映射到 Cg 中去的。在这里，我们先对材质和纹理之间的区别作一个说明，材质是全过程地描述一个表面如何对光反应，而纹理则表示用来计算一个点上的某些值的函数，材质用点来说明表面性质的变化。

首先，我们需要定义一个材质接口。一个材质要负责计算给定点的反射光的颜色，计算时要考虑到材质的性质和从光源来的照明。它需要关于表面的局部几何信息（至少需要着色的点 P、那点的表面法线 N 和纹理坐标 uv），以及入射的视线方向 I 和那点的光照明。注意，

我们把一个 Light（光源）接口的大小不定数组 lights[]，代入材质接口的 color()方法之内。

```
interface Material {
  float3 color(float3 P, float3 N, float3 I, float2 uv, Light lights[]);
};
```

使用 Material（材质）计算着色的片元程序很容易。我们只需要一点粘合代码，把正确的插值参数代进材质之内，而返回的结果作为片元的颜色，如列表 32-4 所示。

列表 32-4 一个片元 shader，它表示了对材质接口实现的几乎所有的计算

```
float4 main(float3 Pworld : TEXCOORD0,
            float3 Nworld : TEXCOORD1,
            float3 Iworld : TEXCOORD2,
            float2 uv     : TEXCOORD3,
            uniform Light lights[],
            uniform Material material) : COLOR
{
  Nworld = normalize(Nworld);
  float3 C = material.color(Pworld, Nworld, Iworld, uv, lights);
  return float4(C, 1);
}
```

我们可以首先执行一个基本的DiffuseMaterial，它抽象出漫反射颜色Kd从哪里来的信息。这样我们就不需要区分 DiffuseMaterial 对象的表面是带有常数的 Kd，还是一个由纹理映射定义的 Kd，还是由一个程序的棋盘格函数定义的 Kd，等等；我们定义一个纹理接口，去隐藏关于这些可能改变的颜色值是如何计算的细节。

```
interface Texture {
  float3 eval(float3 P, float3 N, float3 uv);
};
```

现在 DiffuseMaterial 调用接口的 eval()函数，然后做平常的漫反射计算，如列表 32-5 所示。

以这种方式设计材质，能够很容易地把 DiffuseMaterial 和任何类型的纹理一起使用，即使我们未来可能发展纹理，也不需要修改它的源代码。随着我们大量收集不同材质的实现（例如实现复杂的 BRDF 模型的材质），这种正交性（orthogonality）日益变得更加重要。

列表 32-5 漫反射着色模型，它被表示为一般材质接口的实现，这是纹理接口的一个实例，用来计算在着色点的漫反射系数

```
struct DiffuseMaterial : Material {
  Texture diffuse;
  float3 color(float3 P, float3 N, float3 I, float2 uv, Light lights[]) {
    float3 Kd = diffuse.eval(P, N, uv);
    float3 C = float3(0, 0, 0);
    for (float i = 0; i < lights.length; ++i) {
      float3 L;
      float3 Cl = lights[i].illuminate(P, L);
      C += Kd * max(0, dot(N, L));
    }
```

```
    return C;
  }
};
```

为了把这个 DiffuseMaterial 与列表 32-4 的 main()中的 material 参数连接到一起，首先需要构建 DiffuseMaterial 结构的一个实例，然后把纹理接口的一个实现实例连接到它的漫反射参数。接着我们把 DiffuseMaterial 连接到 main()中的 material 参数。

一个纹理的自然实现是一个 ImageTexture，它返回来自 sampler2D 的一种颜色。

```
struct ImageTexture : Texture {
  sampler2D map;
  float3 eval(float3 P, float3 N, float3 uv) {
    return tex2D(map, uv).xyz;
  }
};
```

为了方便，也许需要一个只返回颜色常量的纹理。当颜色值为一个常量时，这允许我们只使用 D:ffuse Material，而不需要对 Material 进行独立实现，幸运的是，按下面代码实现不会影响性能。

```
struct ConstantTexture : Texture {
  float3 C;
  float3 eval(float3 P, float3 N, float3 uv) { return C; }
};
```

为了更有趣，我们可以定义一个纹理实现，它本身就使用纹理接口完成它的工作。在下面的代码片断中，按照第 3 个纹理指定的混合量，一个 BlendTexture（混合纹理）进行两个纹理之间的混合。当然，在这里 3 个纹理中的每一个都可能是完全不同的类型：一个可能从 2D 图像映射来查寻一个值，一个可能总是返回恒定的值，而第三个可能程序式地计算一个值。

```
struct BlendTexture : Texture {
  Texture map1, map2, amt;
  float3 eval(float3 P, float3 N, float3 uv) {
    return lerp(map1.eval(P, N, uv), map2.eval(P, N, uv), amt.eval(P, N, uv));
  }
};
```

shader 接口允许我们以一般的方式实现 BlendTexture，无须事先知道 BlendTexture 的任何具体实例到底使用哪种特别类型的纹理。在把 DiffuseMaterial 中的纹理接口参数初始化时，需要使用 Cg 运行库，把实现纹理接口的结构实例连接到 map1、map2 和 amt 上去。扩展系统中现有的纹理类型很容易，只要实现新的纹理类型并扩展其应用，创建它们并把它们连接到接口参数就行了。不需要对先前存在的类型加以修改。

正在开始写一种接口的实现，而它们本身又支持其他接口，这种思路可以应用于许多其他方面。举例来说，我们可以实现一种材质，它又把雾的气氛模型应用到任何其他的材质上去，如在列表 32-6 中所做的一样。

因此，不需要把对雾的支持加入到所有的材质上去。可以只用 FogMaterial 这样做一次，而且通过把 FogMaterials 的基本实例变量设定到原始的没被雾笼罩的材质中去，把一个

FogMaterial 实例与我们程序的 main()函数的参数连在一起。

列表 32-6 对其他材质所返回的值进行修改的材质，同时又混合在一个雾项中

```
struct FogMaterial : Material {
  Material base;
  float3 fogAtten, fogColor;
  float3 color(float3 P, float3 N, float3 I, float2 uv, Light lights[]) {
    float3 C = base.color(P, N, I, uv, lights);
    float fogFactor = exp(-P.z * fogAtten);
    return lerp(C, fogColor, fogFactor);
  }
};
```

32.4 小结

尽管 shader 接口和大小不定数组加入 Cg 语言之后，并不可能写出什么在这之前写不出的程序。然而，由于它们使得应用程序更容易在运行库从代码块构建 shader，于是实现经典方法的程序着色实际上变得更容易了。它们是以有效的方式提供这个功能的，基本上没有降低 GPU 的性能，惟一花费的是一些运行库和编译器的编译时间。

这章已经描述了几个这种功能的例子。一个应用可能要求更复杂的光源接口，例如像 Barzel 1997 所要求的那样说明光源：只贡献漫反射而不投射镜面的高光。一个应用也许要求一个更复杂的纹理接口，它具有在着色的点表面的偏导数的信息。还可以增加许多其他的生成，来改善在这些例子中已经使用的接口。

举例来说，增加一个描述纹理坐标生成的接口（例如球面映射、圆柱形映射，等等），就产生了能够抽象纹理坐标来源的纹理接口。而通过一个程序或一个纹理映射的查询，这个接口的纹理坐标变换实现又可以转换所产生的纹理坐标的矩阵。

也可能实现许多有用的材质接口，例如根据材质表面的哪一边能被看到，可以从其他两个材质之间选择出一个双边的材质；也可以根据纹理或 Fresnel 反射项，在两个材质之间混合，得到一个混合材质。而以一般的方式用这些接口表示凹凸贴图也许很有用。

Cg 1.2 的用户手册有许多信息，涉及到 shader 接口和大小不定数组的语言语法，以及由于增加这些特性，而对新的 Cg 运行库调用的描述。

32.5 参考文献

Barzel, Ronen. 1997. "Lighting Controls for Computer Cinematography." *Journal of Graphics Tools* 2(1), pp. 1～20. 这篇文章用许多有用的参数描述一个复杂的光源模型。

Cook, Robert. 1984. "Shade Trees." *Computer Graphics (Proceedings of SIGGRAPH 84)* 18(3), pp. 223～231. 这是第一篇用着色节点树来介绍建模中的层次结构思路的文章。

第 33 章　将产品的 RenderMan shader 转化为实时的 shader

Stephen Marshall
Sony Pictures Imageworks

33.1　引言

自 20 世纪 80 年代后期以来，视觉效果工作室已经用着色语言为电影业创建出令人信服的图像。尽管这些着色语言极其有力，但是要产生创新的图像需要的语言太复杂，这时常要花费很长的渲染时间。如果要求更大和更好的效果，这些 shader 的复杂性还会进一步增加。

如果渲染每帧要花费几个小时，那么所需要的全部时间有多么长就可以想见了。因此，视觉效果工作室正在考虑使用实时的游戏技术作为最佳的可视技术。特别是最近图形硬件技术的进步和硬件着色语言的发展，已经提供了实时产生效果的可能性。

这章说明把一个 Pixar RenderMan 表面 shader 转化成一个硬件 shader 是可能的。所用的硬件着色语言是 Cg。Cg 的使用并不限于本章的范围，因为 Cg 编译器能产生许多不同形式的 shader 指令，包括 OpenGL ARB 片元程序适用的代码。详细内容请见 NVIDIA 2002。我们提出的概念也适用于其他的着色语言。

在书的 CD 或网站上的适当子目录中，可以找到这章中所提到的每个列表。举例来说，列表 33-1 包含的 *Stuart Little* 毛皮 shader，就在第 33 章的列表 1 的子目录中。

这章始终使用电影 *Stuart Little* 中的 RenderMan 毛皮，作为一个把 RenderMan shader 转换成 Cg 程序的例子（对于毛皮 shader 的描述请看 Berney 和 Redd 2000）。所涉及到的题目如下：

- 频率独立性的问题（Frequency-independent），尤其在处理多个光源时
- 顶点程序和片元程序的比较，对硬件特有的优化问题予以密切的关注（这对好的性能是必须的）
- 对结果的分析，包括最初的 Cg shader 及其优化结果之间的差异，以及 Cg 和 RenderMan 结果之间的不同

33.2 光照

正如本文所述，用 Cg 代码写光照和处理它们的方法，与 RenderMan 有极大的不同。这是因为 RenderMan 的结构没有等效的对应。下面谈谈光源和光源 shader。

33.2.1 光源

在 RenderMan 中，光源由 illuminance 语句迭代。illuminance 语句让表面 shader 聚集在着色点上对场景中所有光源取样的总和。光源颜色和光源方向矢量之外的其他信息，可以通过消息传递在表面和光源之间交换。关于这个问题的更多信息可以在 Upstill 1990 中找到。

在 RenderMan 毛皮 shader 的代码中单个光照明语句是：

```
// RenderMan illuminance 语句
illuminance (P, norm_hair, radians(illum_width))
```

这个语句指示：下面的代码将对满足照明条件的场景中的所有光源运行。不幸的是，在 Cg 中没有对应的语句。在写一个 shader 之前，场景中光源的数目必须是已知的。同时，每个光源可能有一组相似的参量，但是具有不同的数值。举例来说，如果在场景中的所有光源都是简单的方向光，每个光源的参数列表中至少需要有一个光的方向矢量。如果在场景中有 10 个光源，参数列表可能看起来如下：

```
// Cg shader 参数列表例子
Output cgProgram(Input In,
                 uniform float3 lightDir[10],
                 . . . , // 其他的程序参数
```

有可能在 10 个光源上循环，但是过程不变。如果所需要的光源超过 10 个，就必须修改 Cg 程序了。

一个代替的方法是以多遍方式运行程序。如果所有的光源都有相似的参量，那么只把当前光源的参数约束于当前渲染状态。然后把当前遍的结果加入帧缓冲区内。这样就不需要为 Cg 程序指定场景中的光源数目了。现在可以让应用程序去处理这个信息了。这样产生的附加好处是 Cg 的参量列表程序可以简化为：

```
// 多遍 Cg shader 参量列表的例子
Output cgProgram(Input In,
                 uniform float3 lightDir,
                 . . . , // 其他的程序参数
```

注意，尽管这个方法解决了需要重写 Cg 程序的问题，但是程序的多遍运行必然要重创性能。如果 Cg 程序复杂，当场景中的光源数目增加时，可以预见应用的性能会极大地降低。

33.2.2 光源 shader

同样地，也没有与 RenderMan 光源 shader 相对应的 Cg 语句。像对光源一样，也可以对光源 shader 做一个相似的假设：场景中所有的光源共享相同的 shader。

这看起来好像是对产品使用的严重限制。但是，通常存在着一个主要的光源 shader，它把所有可用的光源参数都压缩在其中。而计算普通的复杂光照方程，可以重写一个单个的 Cg 函数。关于实施这个题目的更多信息可以在 Barzel 1997 和 Gritz 1998 发现。

33.2.3　其他的光源参数

一个复杂的光源 shader 可能把许多参数加入 Cg 程序。通过定义多个光源 shader 参数表，shader 预处理程序可以管理这种复杂性，并且对可读性有帮助。下面的两个列表分别用于定义和简化程序的参数表。

```
// uber_light parameter list
#define DEFINE_UBER_LIGHT_PARAMS        \
  uniform float3 lightColor,            \
  uniform float lightIntensity,         \
  uniform float lightConeInner,         \
  uniform float lightConeOuter,         \
  uniform float lightConeRoundness,     \
  . . .

// uber_light parameters
#define UBER_LIGHT_PARAMS \
  lightColor,             \
  lightIntensity,         \
  lightConeInner,         \
  lightConeOuter,         \
  lightConeRoundness,     \
  . . .
```

Cg shader 的入口点可以简化为：

```
// 多遍 Cg shader 参数列表的例子
Output cgProgram(Input In,
                 DEFINE_UBER_LIGHT_PARAMS,
                 . . . , // Other program parameters
```

可以如下地调用在 cgProgram 块中的光源实施：

```
// 多遍 Cg shader 参量列表的例子
float3 lightColor = uber_light(UBER_LIGHT_PARAMS,
                               . . . , // Other parameters
```

最后，RenderMan 中的消息传递的灵活性必然会被取消。这不是大问题，因为 shader 知道当前光源的参数值。这些参数值压缩在 Cg 程序中早先的参数列表定义中。

33.3　顶点程序与片元程序的比较

写 Cg 程序（而不是 RenderMan shader）涉及到计算频率的不同概念。RenderMan 中 shader 的运行速率由 ShadingRate 参数控制（Apodaca and Gritz 2000）。这个参数是一个浮点值，它

指定一个着色样本可能代表的区域（以像素表示）。

与之形成对比的是，在 GPU 中着色的速率由使用的程序类型确定。例如顶点程序的所有运算在顶点数据上执行，这些数据是通过 API（OpenGL、Microsoft DirectX 等等）代入到 GPU 的。而所有片元程序的运算在图元上执行，这些基元直接涉及到最终图像的像素。

在硬件中，顶点和片元程序不一定非要有相同的能力。在许多情况下，这些程序必须一起工作，才能支持单个的 RenderMan 表面 shader 所提供的同样功能。一个顶点程序优先执行图形流水线的变换和照明部分。因此，必须至少提供一个齐次剪裁空间位置给输入顶点。这样，必须手动产生流水线的变换部分，包括把 modelview projection 矩阵代入程序，转换引入的顶点，把新的顶点位置赋给输出顶点寄存器。这与 RenderMan 不同，因为除了最终的颜色以外，在 RenderMan 中无须担心 shader 的输出。

与之对应的是，一个片元程序覆盖了图形流水线的多纹理部分，于是任何纹理操作必须手工执行。一个片元程序运行的结果，是片元的颜色和一个可能的新的深度。这个新颜色一定被限制在输出颜色寄存器中。

Cg 是一个与硬件独立的语言。只有通过硬件 profile，编译器才能知道它为什么平台而构建。关于硬件限制的更多细节可以在 NVIDIA 2002 的附录 B 中找到。建议读者在写硬件 shader 之前阅读和了解这些细节。

33.4 使用顶点和片元程序

如前面所述，要达到一个与 RenderMan shader 相同的功能，通常必须写一个顶点程序和一个片元程序。对于毛皮的例子，一个顶点程序准备光栅化过程，而片元程序实现着色计算。列表 33-2 显示设置一个顶点程序的初始的实施，为的是让片元程序开始光栅化。

顶点程序处理两种类型的数据。第一类是在顶点程序中产生的属性值。例如光线方向（$\boldsymbol{L}$）和视线方向（$\boldsymbol{V}$），它们是片元 shader 所需要的，取决于输入顶点的位置。由于这种相关性，因此在顶点程序的运行库中计算这些数值。

其他的逐顶点数据可以作为数据流，通过顶点 shader 代入片元 shader。不用经过进一步处理，直接把这些属性代入并分配到顶点程序的输出。对于毛皮的例子，这些属性是归一化的毛发正切（$\boldsymbol{T}$）、一个毛发参数值（v）和一个表面法线（$\boldsymbol{nSN}$）。这类似于在 RenderMan 中把额外的原始变量分配给几何体。RenderMan 和顶点程序之间的不同是，可以通过 RenderMan 的插值方法产生数值（例如，通过插值法得到指定在 NURBS 表面的控制点的数据）。而顶点程序则只接收从应用程序代入的非插值数据。

为 GPU 进行的数据插值，发生在片元程序之前，发生在光栅化阶段中。用户不控制如何计算中间的片元值。通过标准的图形流水线的数值，是穿过一条扫描线线性地内插的。这预示着输入几何体的取样率将会影响图像的最终质量。

最后的部分是片元程序本身，那里是执行大部分着色计算的地方。列表 33-3 显示了 Cg 片元程序，它几乎和原始的 RenderMan shader 相同。因为 Cg 提供许多与 RenderMan 相同的标准库函数，所以初始的变换非常容易。惟一重要的不同是与顶点程序的交互作用、不同名字的程序类型，以及 saturate、dot、cross 函数的引入。饱和函数是消耗最小的函数，应该使用在把一个值钳制在 0 和 1 之间的时候。dot 和 cross 函数用来代替 RenderMan 着色语言中

· 和∧的操作符。

33.5　片元程序的优化技术

目前片元程序的实现似乎是合适的，但是它将会执行得很慢。这在开始可能令人失望，但是请记住，GPU 不是一个典型的处理器。虽然转换毛皮 shader 简单，但硬件结构还没有被考虑在内。GPU 由两个处理器组成：顶点处理器和片元处理器。顶点处理器仅在每个顶点上运行，因为模型中的顶点一般比最终图像的片元少得多，所以最好是在顶点处理器上做尽可能多的计算。因为片元程序在产生的全部片元上运行（包括看得见的和看不见的），所以片元程序中某些高质量的处理可能补充顶点处理。同时，应该把在互动进程中不变的计算挪到应用层去；因为每次顶点或片元程序运行时，都要重计算它们是一种浪费。

考虑一下 GPU 的矢量化本性也很重要。一般而言，GPU 可以执行一个包含 4 个算法指令的组，同时也可以执行包含 1 个、2 个或 3 个算法指令的组。应该组织 shader 源代码以帮助编译器识别和优化这些情况。最后，因为在 GPU 上纹理映射的查询是极其有效的，可以对 1、2、3 个变量的复杂函数分别使用一维、二维、三维纹理作为查询表。

我们的目标是减少片元程序的大小。一般地，程序越短，则运行越快。如果使用 fp30 profile，最初编译的片元程序是 104 个指令。如果用户对硬件有更深入的理解，一定能够减少指令并提高效率。

33.5.1　把代码转移到应用层

在片元程序的第一部分，如果视点或光源改变，代码显然不会改变：

```
// Fragment Program lines
float3 S = cross(IN.nSN, IN.T);
float3 N_hair = cross(IN.T, S);

float l = saturate(dot(IN.nSN, IN.T));
float3 hairNorm = normalize((l * IN.nSN) + ((1.0f - l) * N_hair));
```

因此可以把这部分移进应用层，新的属性 hairNorm 将会在那里被带入顶点程序。

33.5.2　把代码转移到顶点程序

因为 L 和 V 矢量分别依赖光源和视点位置，所以下面的点积计算不能够移进应用层：

```
float T_Dot_nL = saturate(dot(IN.T, IN.L));
float T_Dot_nV = saturate(dot(IN.T, IN.V));
```

然而，可以将这组计算移进顶点程序，因为矢量一般沿着由曲线段产生的小段慢慢地变化。大体上，沿着这条曲线的取样率越高，由这个插值所引进的误差就越小。

我们可以用严格的方式估计由于这个内插产生的潜在误差量。因为点积等于两个归一化矢量夹角的余弦，使用 Crenshaw 2000 的分析，单值余弦函数的误差函数是：

$$e_{\max} = \cos\left(x_1 + \frac{\Delta x}{2}\right) - \frac{1}{2}\left[\cos(x_1) + \cos(x_1 + \Delta x)\right] \qquad (1)$$

假定最大的误差产生在曲率最高的点上，把 x_1 和 x_2 放在 cos(0)上。那么：

$$x_1 = -\frac{\Delta x}{2}$$
$$x_2 = \frac{\Delta x}{2} \tag{2}$$

因此：

$$e_{\max} = 1 - \frac{1}{2}\left[\cos\left(-\frac{\Delta x}{2}\right) + \cos\left(\frac{\Delta x}{2}\right)\right] \tag{3}$$

使用三角公式：

$$\cos(a+b) + \cos(a-b) = 2\cos(a)\cos(b) \tag{4}$$

这个例子里 $a = 0$ 和 $b = \Delta x / 2$，方程式（1）变成：

$$e_{\max} = 1 - \cos\left(-\frac{\Delta x}{2}\right) \tag{5}$$

假定Δx 足够小，方程式（5）的余弦项能由余弦函数的无穷级数的前两项近似，得到：

$$E = \frac{\Delta x^2}{8} \tag{6}$$

正如预料的，方程式（6）表明最大的误差发生在取样率最低的地方。特别是在那里，当取样点变化时，需要它们的正切几乎是相同的值。这意味着应用必须保证，在曲率越高的区域，毛发的取样率也越高。如果这个假设成立，那么点积就能如下所示地移进顶点程序。

```
float2 dots = float2(saturate(dot(IN.T, IN.L)), saturate(dot(IN.T, IN.V)));
```

33.5.3 通过纹理查询优化

Kajiya-Kay 的毛发着色模型基于一个复杂函数，该复杂函数包含两个变量，T_Dot_nL 和 T_Dot_e 。

```
float Alpha = acos(T_Dot_nL);
float Beta = acos(T_Dot_e);
float Kajiya = T_Dot_nL * T_Dot_e + sin(Alpha) * sin(Beta);
```

一个更详细的推导可以在 Kajiya and Kay 1989 上发现。

这个函数项似乎很适合做一个纹理查询。为了这个应用可以产生一个浮点纹理，而这个纹理函数可以在两个独立变量的 0 到 1 之间取样。

可以用一个类似于前节所讲的分析，来为我们的函数查询确定取样率和适当的纹理尺寸，但是它太复杂了。代替的办法是再写一个分开的简单的片元程序，用来求纹理查询与完全的 Kajiya-Kay 项的计算之间的差。这个差可以显示为最终的片元的一种颜色。通过简单的视觉比较，可以选出误差最小的合适的纹理尺寸，如图 33-1 所示。

通过这个查询，现在可以发现纹理的 Kajiya-Kay 项：

```
float2 Kajiya = texRECT(kajiyaTexture, IN.dots);
```

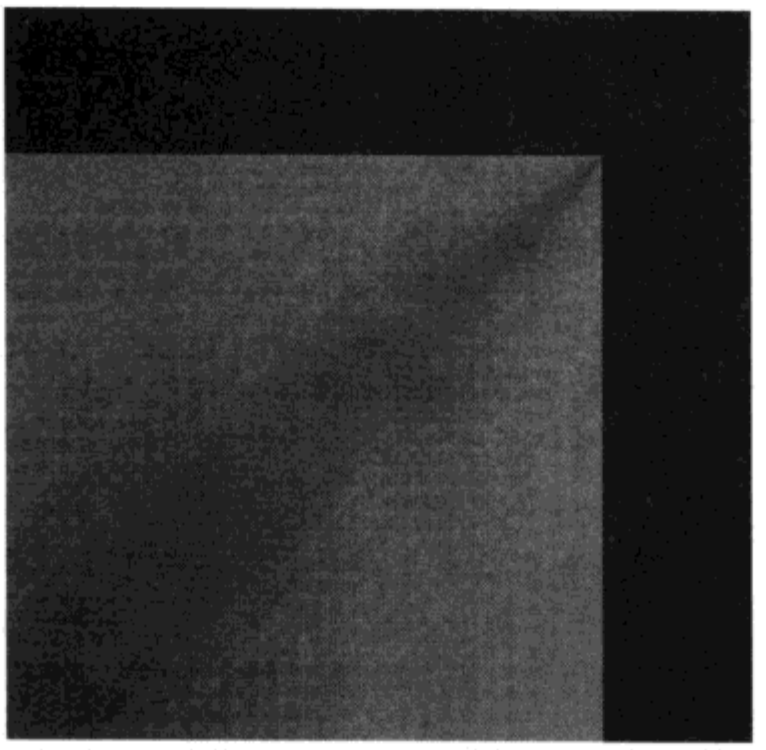


图 33-1　纹理查询和完全计算之间不同的图像
轴表示在 0 和 1 之间的点积。蓝色的区域（两边）表示误差小于 0.0001f；
绿色区域（中间）表示误差小于 0.01f

注意，第 33.5.2 节顶点程序中所计算的点积，是当作纹理查询参数使用的。也要注意，texRECT()函数是用作纹理查询的。结果，必须在一个非归一化的纹理空间中做这个纹理查询。用纹理查询的参数（顶点程序中的点积）乘以 Kajiya-Kay 纹理的尺寸，就得到纹理查询的结果。

33.5.4　向量化的优化

下面的 power 函数用于计算着色项的镜面反射成分：

```
float3 Cspec = (spec.x * lightColor * pow(Kajiya, iroughness.x)) +
               (spec.y * lightColor * pow(Kajiya, iroughness.y));
```

为了利用乘法的矢量形式可以将这个函数重排列。特别要注意的是，power 函数的结果可以平行地与 spec 成分连乘：

```
float2 powVal = spec * pow(Kajiya, iroughness);
float3 Cspec = lightColor * powVal.x + lightColor * powVal.y;
```

33.5.5　最终的优化

在列表 33-3 顶部所定义的函数 fnc_diffuselgt，用来计算着色项的漫反射成分。这个函数也使用光线方向和毛发法线之间的点积。从第 33.5.2 节点积的分析，我们知道可以把这个函数移进顶点程序。引起变化的属性被传给片元程序叫做 dComp。

可以把 Kd 乘法和 final_c 计算（减去变暗的项）移入顶点程序，以提高性能。Kd 必须与早就移进顶点程序的漫反射成分相乘。中间的毛发颜色值存储在变化属性 hairCol 中。顶点程序代码现在看起来如下：

```
OUT.hairCol = ((1.0 - IN.v) * rootColor) + (IN.v * tipColor);
OUT.dComp = saturate(dot(lDir, IN.hairNorm)) * Kd;
```

最后再看看完整的片元颜色：

```
OUT.col.xyz = saturate((Kd * Cdiff + staticAmbient) * final_c +
                       (In.v * Kspec * Cspec * specularColor));
```

IN. v * Kspec 的计算可以移进顶点程序，而镜面反射值能被存储在变化属性 specColor 中以便产生：

```
OUT.col.xyz = saturate((Cdiff + staticAmbient) * final_c +
                       (In.specColor * Cspec * specularColor));
```

注意，没有条件语句可用来除去不必要的计算。这是不幸的，现在的 GPU 本身在片元程序中不支持条件代码。于是不管条件的结果是什么，条件中的两个子句 if 和 else 都会被计算。这样对运行不利 （甚至可能由于条件而减速），因此应该使用更复杂的版本进行计算。

33.6 小结

这个应用在带有 2.8 GHz Xeon 处理器的 Intel Pentium 4 上运行，在有 NVIDIA Quadro FX 2000 和 NVIDIA Quadro 4980 的 Linux 上测试。用 10 个均匀取样品点，在 Cg 1.1.0003 版的编辑器上产生 27 488 个 b 样条曲线。

虽然从 RenderMan shader 转化为一个片元程序是简单的，但是最初的性能是令人失望的。但是，通过本章中讨论的各种优化技术，片元程序从 104 个指令减少到 42 个。这明显地提高了性能。

为了优化在图像质量方面做了一些让步。幸好实际上这些让步没有引起明显的差异。

图 33-2 和图 33-3 分别显示最初的片元程序和优化的片元程序，图 33-4 显示了两个版本产生的图像之间的差异。可以注意到，差别是可以忽略的。

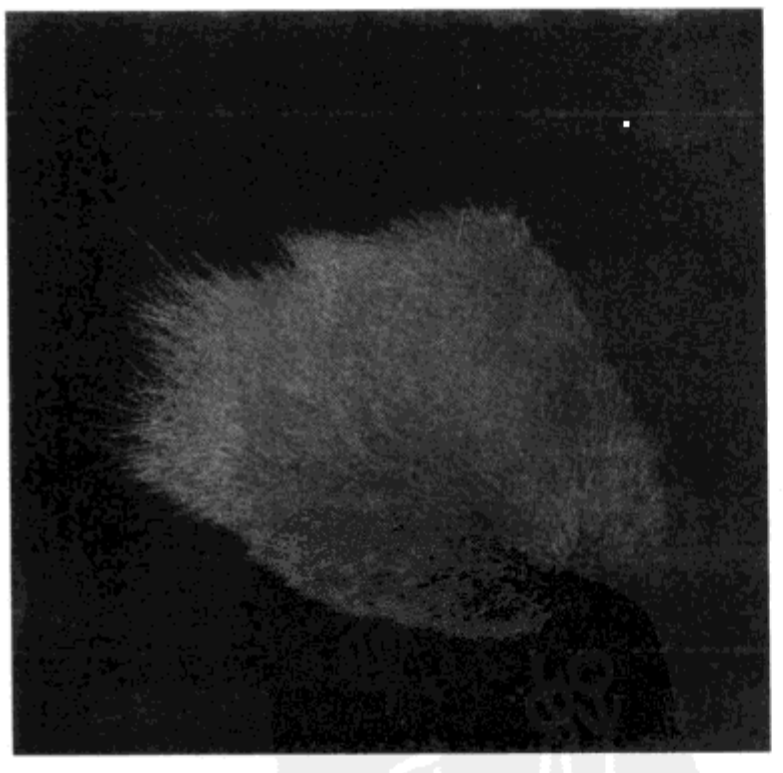


图 33-2 最初的片元程序的结果

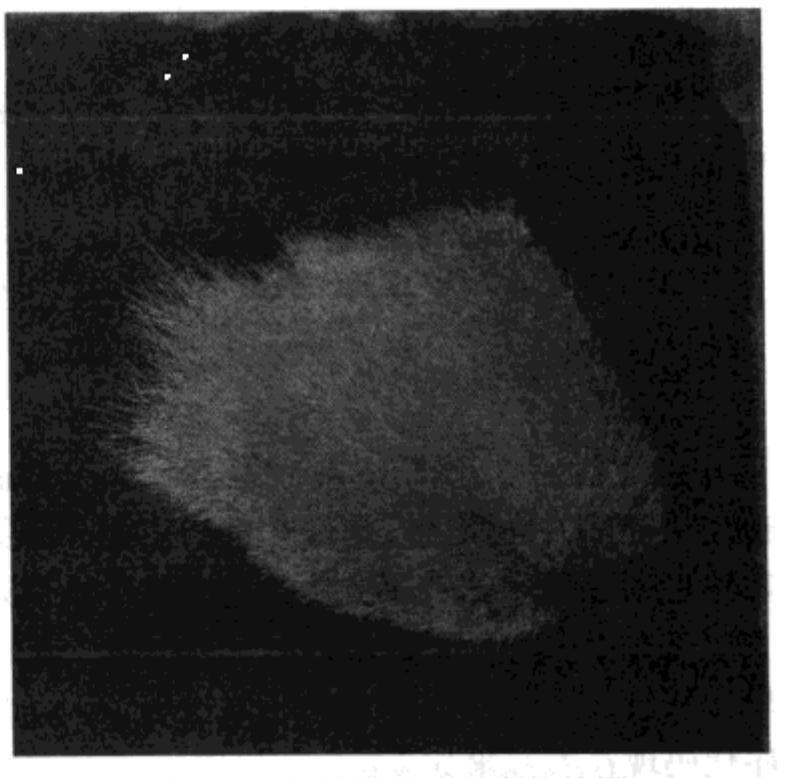


图 33-3 优化的片元程序的结果

片元程序产生的渲染和 RenderMan 产生的渲染之间的差别，也很有趣。使用相同的曲线

几何体，用 Pixar 的 PRMan 在 20 秒内所渲染的图像显示在图 33-5 中。虽然片元程序的图像与 RenderMan 的图像稍微有点不同，但是在 RenderMan 图像中，所有重要的加亮区和彩色变化的位置，都与片元程序图像中的位置相同。容易看出，只用了很少的时间，硬件版的渲染就恰如其分地近似了 PRMan 的渲染。

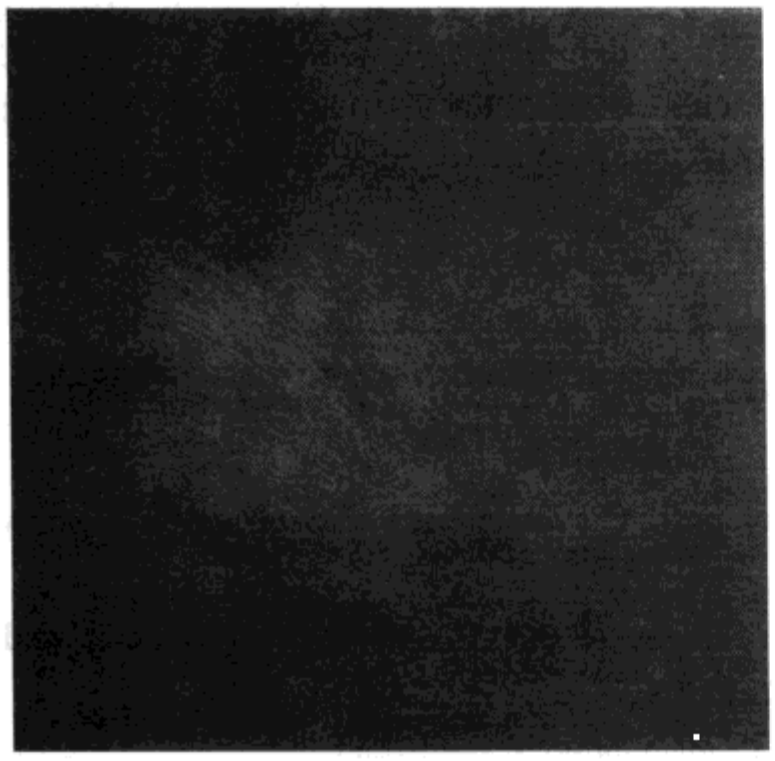


图 33-4　最初的和优化的图像之间的不同为了显示细节增强了差异性

图 33-5　PRMan 渲染的结果

达到产品质量的角色时常有数以百万计的毛发。甚至在剔除之后，仍然有数十万计的毛发要着色。对这类角色而言，优化后的片元程序变得太慢，因此，一个完全的毛皮着色是在一个顶点程序中实现的，如列表 33-4 所示。结果见图 33-6。虽然这个方法的正确性比不上片元程序，但能明显地改善性能。

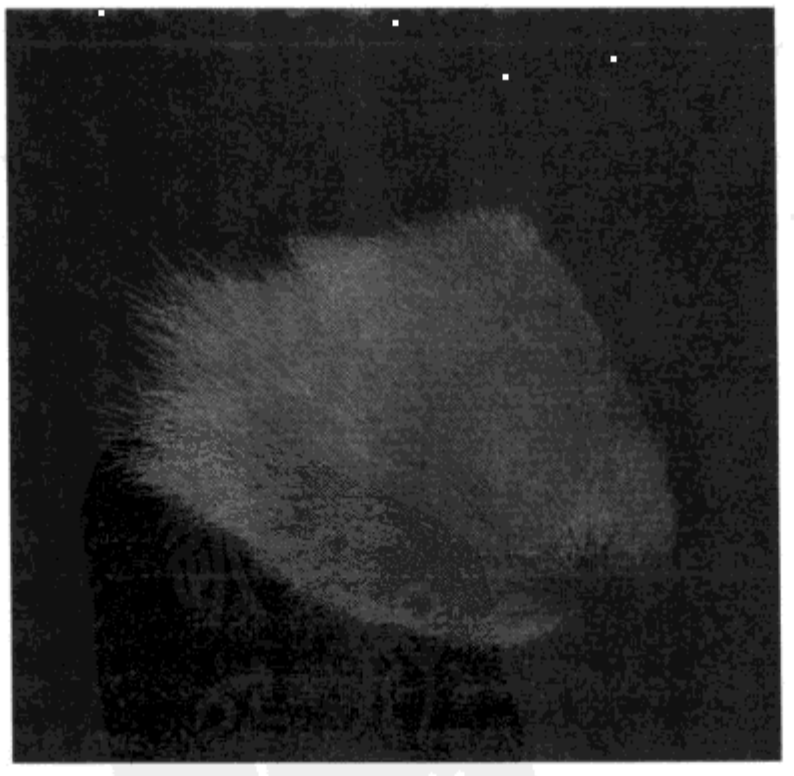


图 33-6　毛皮 shader 顶点程序的结果

注意，如果 shader 作者进行手工优化，片元程序与原始的 RenderMan 表面 shader 会变得非常不同。如果必须修改最初的 RenderMan shader 代码，那么要想辨认出在硬件 shader 中应该修改哪一部分代码才可以得到相同的变化是很难的。这意味着作者必须十分了解 RenderMan 和硬件 shader 是如何工作的。

在这章中没有提到 RenderMan shadeop。shadeop 是一个以 C 或 C++写的、由用户定义的函数，它把功能加入 RenderMan 的标准库中。它取一个用户创建的共享库的形式，在运行期由 RenderMan 渲染器动态地载入。问题是目前的 GPU 不允许这种类型的编程。如果能够简化 shadeop 参数的列表，就有可能用第 33.5.3 节中所描述的纹理查询的方法代替它。

33.7 参考文献

Apodaca, A., and L. Gritz. 2000. *Advanced RenderMan: Creating CGI for Motion Pictures.* Morgan Kaufmann.

Barzel, Ronen. 1997. "Lighting Controls for Computer Cinematography." *Journal of Graphics Took* 2(1), pp. 1～20.

Berney, J., and J. Redd. 2000. "Stuart Little: A Tale of Fur, Costumes, Performance, and Integration: Breathing Real Life Into a Digital Character." SIGGRAPH 2000 Course 14.

Crenshaw, J. 2000. *Math Toolkit for Real-Time Programming.* CMP Books.

Gritz, L. 1998. "uberlight.sl". Available online at the RenderMan Repository:
http://www.renderman.org/RMR/Shaders/BMRTShaders/uberlight.sl

Kajiya, J., and T. Kay. 1989. "Rendering Fur with Three Dimensional Textures." *ACM Computer Graphics (SIGGRAPH 89)* 23(3), pp. 271～280.

NVIDIA Corporation. 2002. *Cg Toolkit User's Manual: A Developer's Guide to Programmable Graphics.* Available online at **http://developer.nvidia.com**

Upstill, S. 1990. *The RenderMan Companion: A Programmer's Guide to Realistic Computer Graphics.* Addison-Wesley.

这个工作得到索尼图片 Imageworks 公司的支持。我还要感谢 Murilo Coutinho、Maria Giannakouros 和 Hiro Miyoshi，他们提供了许多有益的建议和贡献。

第 34 章　将硬件着色整合进 Cinema 4D

Jörn Loviscach
Hochschule Bremen

大多数的 3D 图形设计软件使用图形硬件来加速交互式预览，以便显示正在构造中的场景。尽管目前的 3D 芯片具有极其强大的计算能力，但是如何能使硬件影响高质量的渲染，仍然存在许多有趣的问题需要解决。我们的目的是既加速离线渲染，又提供较高质量的交互式显示，从而给设计者提供较好的反馈。这章概略说明如何使用 CgFX 工具箱把这些能力加到 Maxon Cinema 4D 中去，并且讨论了已经用这个办法解决的问题。所用的方法可以容易地推广到相似的 3D 图形设计软件中去。

34.1　引言

要研究基于硬件的渲染，没有必要从头发展新的 3D 图形设计软件，只要模拟图形硬件的嵌入式渲染器，增加一个具有这种功能的现成产品就行了。这章就描述这种方法的实现，作为 Maxon Cinema 4D 的插件，称为 C4Dfx。这个开发主要不取决于宿主软件的选择；相反地，我们把重心集中在 shader 的设计、几何体的计算和面向对象。因此，我们的技术可以容易地应用于许多不支持可编程硬件着色的 3D 图形解决方案。

Cinema 4D 提供一个建模、动画和渲染的环境，其功能可与 Alias Maya 和 discreet 3ds max 媲美。然而，Cinema 4D 目前对图形硬件特性的支持很有限。我们借助于插件来扩展它的功能，同时这个软件也是如何应用 CgFX 的很好的测试用例。

图 34-1 的左上图显示了 Cinema 4D 的预览窗口，它基于 Gouraud 插值，使用标准的 OpenGL 进行渲染。硬件加速的渲染器 C4Dfx（在图 34-1 的右上图中）显示了平滑的、Phong 插值的高光区、凹凸映射、阴影，以及环境映射。除了柔和的阴影之外，这个渲染的质量与 Cinema 4D 离线的渲染效果几乎相当（在图 34-1 左下边显示）。

用于硬件加速渲染的插件提供了可以接近交互速率的显示更新，而离线渲染可以产生.avi 文件。尽管插件并不能准确地再生 Cinema 4D 的离线渲染结果，但是它的快速和相当精确的预览是有帮助的。插件的离线渲染结果可以用作快速预览效果，甚至可以当作最终的结果。

插件建造在 OpenGL 和 CgFX 工具箱上。3D 软件的表面材质显然是用

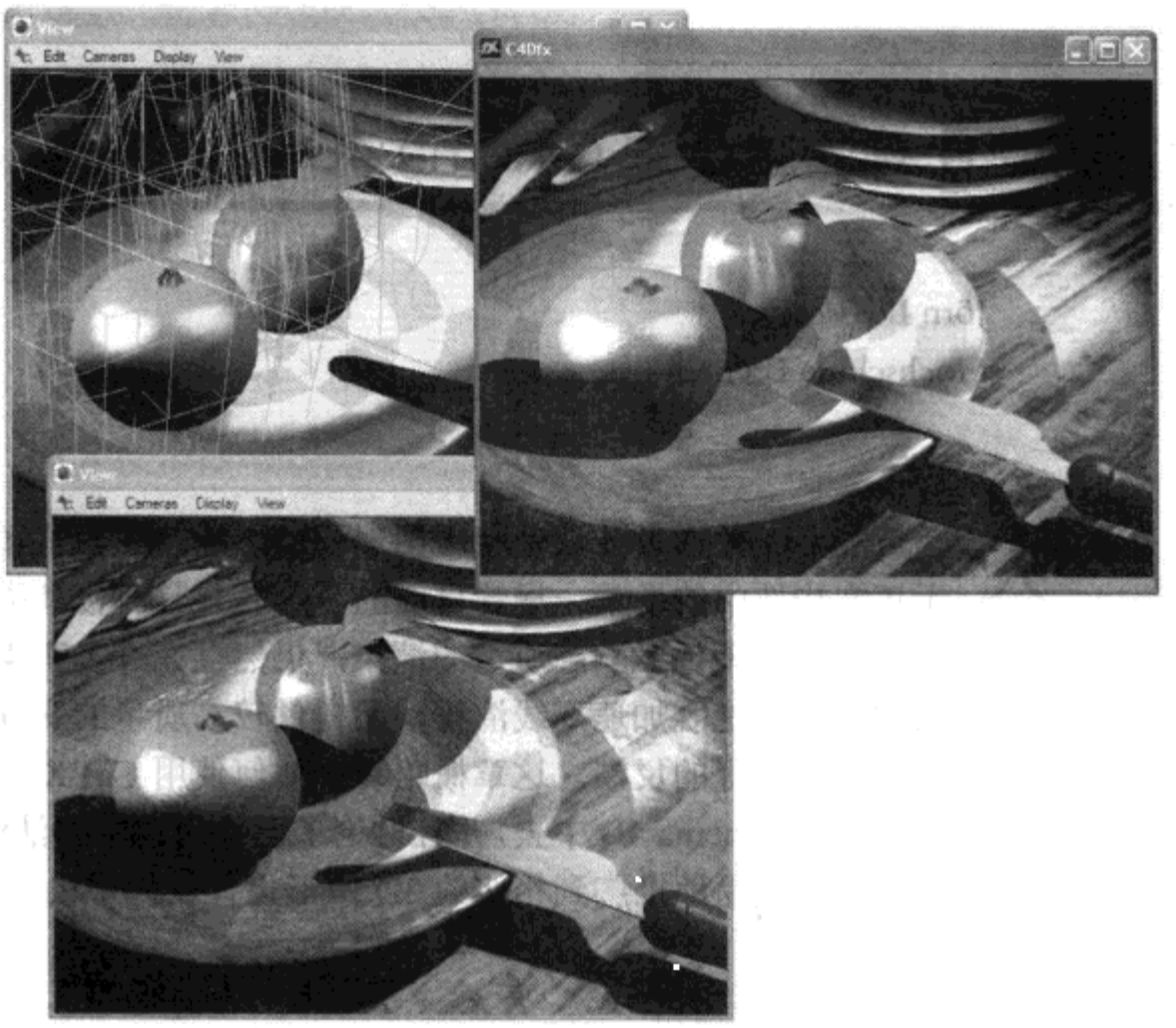


图 34-1 渲染的比较

CgFX shader 模拟的，包括凹凸映射、环境映射和阴影效果。也可以从.fx 文件装载 CgFX shader。在这种情况下，快速产生一个图形用户接口，让设计者能够控制一个 shader 的可调节参数。这个方法不但可以将越来越多的.fx shader 在游戏开发中使用，也可以作为常规材质在高质量渲染中使用。它们也可以把阴影投射到带有模拟 Cinema 4D 材质的物体上。图 34-2 就显示了这个结果，以表现 7 个.fx shader，它们包含在 Maya 的 NVIDIA CgFX 插件之中。该图也表明能以相同的方式处理各种类型的 3D 物体。

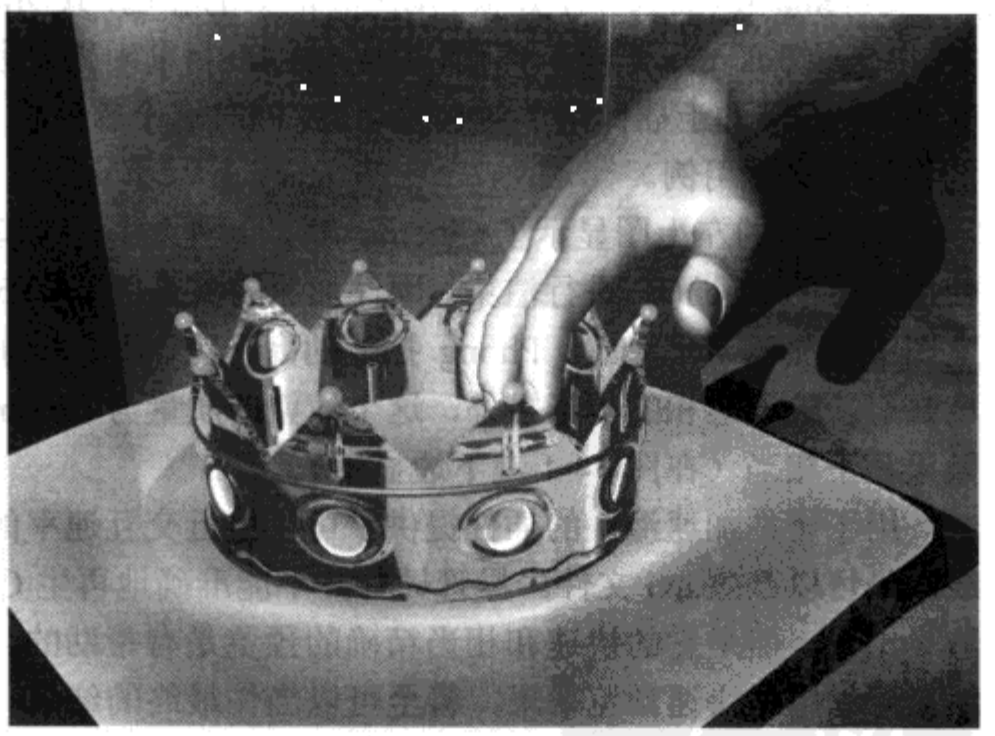


图 34-2 CgFX shader 和一个带有阴影的模拟 Cinema 4D 材质

34.2　把 Cinema 4D 连接到 CgFX 上去

当试着把 Win32、OpenGL 和 CgFX 调用，与 Cinema 4D 的 C++ API 一起使用的时候，出现了一个小的障碍：对应的头文件互相冲突，并且由于类型重新定义而不能够把它们包括在一起。这个问题通过引入包裹函数（wrapper function）和类而彻底解决了。它们封装了 API 所用到的函数和类。

Cinema 4D 只允许第三方的程序员有限制地访问它的窗口。为了得到对设备场景的完全控制，插件要建立它自己的 Win32 线程。它的高质量预览和它的离线渲染与 Cinema 4D 协同操作；它们使用离屏缓冲，而且通过位图把结果传送给 Cinema 4D 的对话框进行显示。这种设计使得不同的反走样（多重取样）模式之间能够飞速变换。此外，如果用户缩放某个窗口，只有所显示的位图被重新放缩，才可以立即得到视图反馈，而在调整大小的事件发生时重新渲染场景是不可能这么快的。

多亏了多线程，用户无须等待交互的或离线的渲染完成，就可以在构建的窗口上工作。但是这种同时操作意味着插件必须接到场景的复制拷贝。不然的话，插件可能会试图访问用户刚刚删除的对象。

插件使用 CgFX 框架构建 shader，并用它们渲染场景。每次为了一帧要调用交互式渲染，或为了一个帧序列要调用离线渲染时，对所有的材质构建的 ICgFXEffect 的实例（下面有更多例子）或者来自.fx 文件，或者来自模拟标准的 Cinema 4D 材质。在渲染的时候，针对每个物体调用对应的 ICgFXEffect。CgFX 框架的每个渲染遍调用一个 OpenGL 的索引顶点数组，其中包括法线和（多）纹理坐标数组。

这些数组是这样准备的：当 Cinema 4D 以样条表面、metaball 等工作时，它的 API 的 Hierarchy 类使插件开发者容易地遍历场景数据结构，并收集所有物体的镶嵌的和变形的版本，以及它们全局的变换材质。顶点和多边形结构可以立刻用于 OpenGL，作为四边形的索引顶点数组使用。注意 Cinema 4D 不能支持任意多边形，而只能支持四边形和三角形；后者可以看成是退化的四边形。

除了空间坐标之外，典型的.fx shader 需要许多其他的各个顶点的数据：法线矢量、切线矢量、副法线矢量和 *u-v* 纹理坐标。如果我们要求对象是一个内置的参数类型（例如一个球体或一个人体模特儿），或要求用户给物体指派 *u-v* 坐标，这些数据是可以立刻访问的。

通过 Cinema 4D 提供的邻接信息，可以容易地确定面法线，而通过求一个顶点周围面法线矢量的平均值可以得到顶点法线矢量。为了计算在一个顶点上的切线和副法线矢量，需要知道纹理坐标的指向 *u* 和 *v* 方向的矢量。这对以样条为基础的物体是容易的：只要求偏导数就行了。然而，在这里我们必须处理已经镶嵌的几何体，或者从一开始就是多面体的几何体。在这些情来况下，*u* 和 *v* 方向是没有良好定义的。可以对每个顶点把它们推断出来，推断时要考虑顶点本身和周围邻居的 *u* 和 *v* 坐标，试着求得一个最好的线性近似值。

原理如下：从位置 x_i 和与顶点 0 相邻的顶点 i 的纹理坐标 u_i、v_i，计算辅助的量 a、b、c、$\boldsymbol{d}$ 和 $\boldsymbol{e}$（黑体字是矢量）：

$$a=\sum_{i=1}^{n}(u_i-u_0)^2,\ b=\sum_{i=1}^{n}(u_i-u_0)(v_i-v_0),\ c=\sum_{i=1}^{n}(v_i-v_0)^2$$

$$\mathbf{d} = \sum_{i=1}^{n}(u_i - u_0)(\mathrm{x}_i - x_0), e = \sum_{i=1}^{n}(v_i - v_0)(\mathrm{x}_i - x_0)$$

从这些信息，确定在 u 和 v 方向上的非归一化矢量 u 和 v，它们可能还不垂直于中心顶点 0 的法线 n：

$$u = cd - be, v = ae - bd$$

为了构造一个正交坐标系（法线、切线、副法线），计算 u 的切线分量：

$$u_t = u - (u \cdot n)n$$

而且使用一个 3 矢量组：n、u_t^0、$n \times u_t^0$，这里 u_t^0 表示规范化的 u_t，而 x 表示矢量积。对于从凹凸映射所产生的法线映射，切线和副法线矢量可以用 u^0 和 v^0 代替，或用矢量$(v \times n)^0$和$(n \times u)^0$代替，它们与凹凸贴图的最初形式更近似，而且对于局部不成直角的 u-v 坐标系工作得更好。至于其他的选择，见 Kilgard 2000。

34.3 shader 和参数管理

如上所述，正是有了 CgFX 工具箱，装载一个.fx 文件和把它用作 shader 变得很容易。然而，还有一些关于它的参数的工作要做：用户需要使用一些可调节参数，包括它们的动画；以及一些不可调节参数，它们必须从场景来计算。

为了处理.fx 文件，插件把独有的材质类型添加到 Cinema 4D 上，它们可以像往常一样被附到物体上去。一个这种材质的实例存储.fx 文件名，以及它的可调节参数的值和动画。当用户重装载一个或选择一个新的.fx 文件的时候，插件使用 ICgFXEffect 的实例装载和分解文件。

为每个参数建造一个特别的对象，用来管理参数的 GUI 和它的值等。这个对象属于一个新定义的 C++基类的子类，叫做 ParamWrapper，这个基类的子类用于不同类型的参数，例如 3 颜色组或纹理文件名。每个 ParamWrapper 子类包含的虚拟方法，可以调用来做以下事情：

- 如果这个参数类型是可调节的，建立和初始化一个 GUI
- 从一个 GUI 重新得到动画的值
- 当开始渲染一个帧的时候（例如载入一个纹理），分配资源和初始化设置
- 当开始渲染一个物体的时候（例如，把它的世界矩阵传递给 CgFX），分配资源和初始化设置
- 在渲染一个物体之后，释放资源
- 在渲染一帧之后，释放资源

此外，可以要求每个 ParamWrapper 子类，用一个 ICgFXEffect 给定的参数例示它本身。接着构造函数读出参数的类型、给出注释并且报告是否成功。在分解一个.fx 文件时，插件为每个参数调用这个方法，直到发现一个合适的子类。这个方法有可能把关于一个参数类型的所有知识都压缩进对应的子类之中。

一些可调节的参数，可以通过子类来接受由 Cinema 4D 提供的特殊 GUI 元素。对于纹理，存储.dds 文件的名字。位置和方向与其他物体的位置或 z 轴相连接，这一点特别适用于光源，

可以把它们拖曳到 GUI 的输入域并放在那里。不可调节的和因此没有 GUI 的矩阵参数会得到特别的处理。世界矩阵由 API 传递，是遍历场景的副产品。观察和投影矩阵是根据摄像机的设置来计算的。根据 shader 的需要，来计算这些矩阵的积、逆和转置。

为了方便起见，插件提供在用户指定的编辑器上打开的.fx 文件。重装载一个编辑过的.fx 文件时，只要它现有参数值的类型正确并在允许的范围内，就可以保存。如果 CgFX 工具箱报告在装载的.fx 文件中有一个错误，那么这个文件在编辑器中被打开，并且光标放置在错误的行上。关于错误的所有信息以及类似未知参数类型等问题，全被写到 Cinema 4D 的文本控制台上。

34.4　模拟离线渲染

用户也可以不装载.fx 文件，而是简单地使用 Cinema 4D 的标准的表面材质。插件在内部明显地把它们转换到基于内存的.fx shader 中去，很适合外部 shader 文件所描述的过程。插件应用字符串流建立内部的.fx shader。

一个 Cinema 4D 材质可能包含位图文件、电影文件，或作为贴图的程序 shader。插件在一个用户选择的分辨率（典型地为 256 × 256 个纹素）下逐个读这些像素，而且建立对应的纹理。可以转换为 3 种类型的映射：漫反射映射、凹凸映射和环境映射。后者被用于光线跟踪中的球体映射；插件计算来自它们的立方体映射，以求得优化的分辨率。凹凸映射是灰度高度场。对于这些，从凹凸映射的水平和垂直的逐像素差形成一个法线映射，为了在较小分辨率时较陡的梯度予以补偿，可以缩放它的大小。

为了产生阴影，我们依靠与 Everitt 等 2002 类似的一个阴影映射方法。这个方法与一个硬件加速的阴影体的算法（Everitt and Kilgard 2002）不同，以后也许可以相当容易地扩展它，去处理被偏移映射或顶点 shader 变形的物体。

对于标注为阴影投射体的场景中的每个圆锥体光源，都产生一个由用户控制尺寸的深度映照（典型地是 256 × 256 个纹素）。结果，不管场景中物体携带着何种表面材质，把它们全都渲染进没有纹理和 shader 的离屏缓冲内。这要求遍历场景，正如早先描述的最终渲染一样，尽管在这一过程中没有计算纹理坐标、法线、切线或副法线矢量。事实上，阴影映射的渲染完全不使用 CgFX。为了使用一个屏幕以外的缓冲作为一个深度映射的纹理，它是用 OpenGL 扩展 WGL_NV_render_depth_texture 创建的。

另外的一个纹理用来存储 Cinema 4D 的镜面高光区的特殊形状。它包括一些超越函数，例如幂函数和弧余弦函数。这个复杂的关系是作为 1024 个纹素的一维的纹理，每帧都进行预计算的。

为了使得光源数目不受限制，一个可能的办法是对每个光源使用一个单独的渲染遍，也许为自照明还要增加另外的遍。然而，这要进行大量的重新计算，例如要重新计算插值的归一化矢量。而连接一些遍又会增大消耗。因为目前的图形卡只支持固定数量的纹理，所以投射阴影的光源的数目受到严重的限制。

所产生的 CgFX shader 的顶点 shader 部分是相当简单的：

1. 使用 WorldViewProjection 矩阵，把位置转换到归一化的屏幕坐标系。
2. 经过世界矩阵把切线和副法线矢量转换到世界坐标系。

3．为了保持正交性，经过逆转置 World 矩阵把法线矢量转换到世界坐标系。

4．使用 World 矩阵计算世界坐标系的位置。

5．最后，在世界坐标系中决定一个从视点（它可以从逆转置的 View 矩阵读出）到顶点的单位矢量。

当存在一个投射阴影的圆锥体光源时产生的像素 shader 见列表 34-1。当存在更多光源时，加上像 depthSampler1 那样的参数，并且因此重复在括号{…}之间的代码块。列表的开始部分是标准；对每个光源所做的计算应该进行一些解释。Ld 是从顶点到光源的世界空间矢量。baseCol 累加了光源的贡献：首先是漫反射，然后镜面反射部分。后者使用 specShadeSampler 读出预先计算的高光区形状。如果反射光线的指向远离视点，specShadeSampler 的纹理坐标就会变成负的，于是把它钳制为 0，因此那里的纹理也是 0。

列表 34-1 像素 shader，用于场景中有一个投射阴影的圆锥体光源的材质

```
pixelOutput mainPS(vertexOutput IN,
                   uniform sampler2D diffuseSampler,
                   uniform sampler1D specShapeSampler,
                   uniform sampler2D normalSampler,
                   uniform samplerCUBE enviSampler,
                   uniform sampler2D depthSampler0,
                   uniform float4 lumiCol,
                   uniform float4 diffCol,
                   uniform float bumpHeight,
                   uniform float4 enviCol,
                   uniform float4 specCol,
                   uniform float4 lightPos0,
                   uniform float4 lightCol0,
                   uniform float4 lightParams0,
                   uniform float4 lightUp0,
                   uniform float4 lightDir0,
                   uniform float4 lightSide0)
{
  pixelOutput OUT;
  float3 Vn = normalize(IN.view);
  float3 Nn = normalize(IN.norm);
  float3 tangn = normalize(IN.tang);
  float3 binormn = normalize(IN.binorm);
  float3 bumps = bumpHeight * (tex2D(normalSampler, IN.uv.xy).xyz * 2.0 - float3(1.0,
                 1.0, 2.0));
  float3 Nb = normalize(bumps.x * tangn + bumps.y * binormn + (1.0 + bumps.z) * Nn);
  float3 env = texCUBE(enviSampler, reflect(Vn, Nb)).rgb;
  float3 colorSum = lumiCol.rgb + env * enviCol.rgb;
  float3 baseDiffCol = diffCol.rgb + tex2D(diffuseSampler, IN.uv.xy).rgb;
  {
    float3 Ld = lightPos0.xyz - IN.wPos;
    float3 Ln = normalize(Ld);
    float3 baseCol = max(0.0, dot(Ln, Nb)) * baseDiffCol;
    float spec = tex1D(specShapeSampler, dot(Vn, reflect(Ln, Nb))).r;
    baseCol += specCol.rgb * spec;
```

```
    float3 L1 = (Ln / dot(Ln, lightDir0.xyz) - lightDir0.xyz) * lightParams0.z;
    float shadowFactor = max(lightParams0.x, smoothstep(1.0, lightParams0.w, length(L1)));
    float d = dot(Ld, lightDir0.xyz);
    float z = 10.1010101 / d + 1.01010101;
    float2 depthUV = float2(0.5, 0.5) + 0.5 * float2(dot(L1, lightSide0.xyz), dot(L1,
                     lightUp0.xyz));
    shadowFactor *= max(lightParams0.y, tex2Dproj(depthSampler0, float4(depthUV.x,
                    depthUV.y, z - 0.00005, 1.0)).x);
    colorSum +=shadowFactor * baseCol * lightCol0.rgb;
  }
  OUT.col = colorSum;
  return OUT;
}
```

其余的计算处理柔和的圆锥体和阴影，把它们结合到一个变量 *shadowFactor* 之中。***L1*** 量度的是顶点到光源的光学轴的距离，如果顶点完全在圆锥体的边上，这个矢量有长度 1，如图 34-3 所示。结果，*LightParams0.z* 必须被设定成 1/2 圆锥体孔径角的反正切。为了产生一个柔和的圆锥体，*lightParams0.w* 被设定成 1/2 圆锥体内孔径角的正切除以前面的正切所得到的商，内孔径角指的是锥体的内部完全光照的部分。由此 *smoothstep* 函数就创建了一个柔和的转变。*lightParams0.x* 对泛光灯设定成 1.0，而对圆锥体光设定为 0.0，所以只有后者被限制为一个圆锥体。

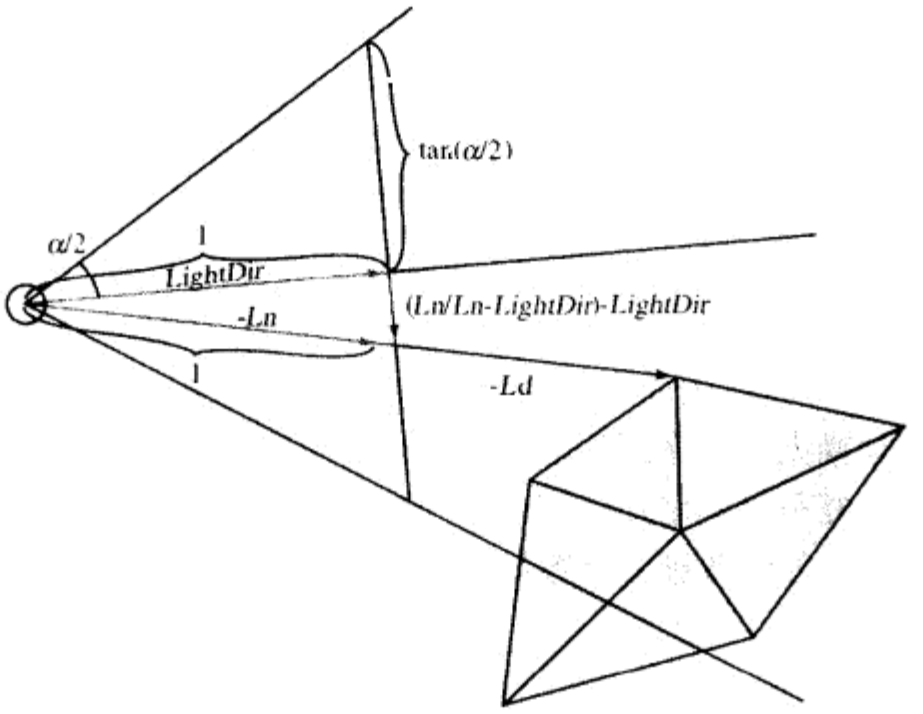


图 34-3　圆锥体光源的几何关系

为了产生阴影，*d* 测量在光源坐标系中的顶点的 *z* 坐标，接着将其转换为深度映射中所用的深度坐标，然后与在映射中所发现的真实值相比较。注意为了 tex2Dproj 能工作，纹理必须用 glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_COMPARE_SGIX, GL_TRUE)定义。

34.5　结果和性能

图 34-1 显示所渲染的图像可以与原始的离线渲染相媲美。像透明和反射效果一类复杂的

特性仍然没有表现出来，但是可能会在一个后来的版本中增加它们，例如，可以经过即时渲染的环境映射和分层渲染（Everitt 2001）。可能进一步做的改善包括柔和的阴影（Hasenfratz等2003）、基于物理的照明模型，以及间接照明（Kaurz 2003）。

当然，主要的问题是硬件加速方法的速度如何。按照每秒兆像素的速率，图形卡几乎可以超过离线渲染的9倍。这一点可以从图34-4中渲染时间随帧尺寸的增长而看出。然而，图34-4也显示，即时编译CgFX shader，特别是从Cinema 4D纹理映射和程序的shader构建OpenGL纹理，也会在渲染任何一帧之前产生启动成本。因此，硬件加速的方法只对尺寸大的帧和序列帧有优越性，但是对单个的帧就不好了。为了改进这一点，需要关于纹理的智能方法。应该尽可能地保存纹理，而不是重新建造每一帧，这需要对动画进行智能的、深入的考查。

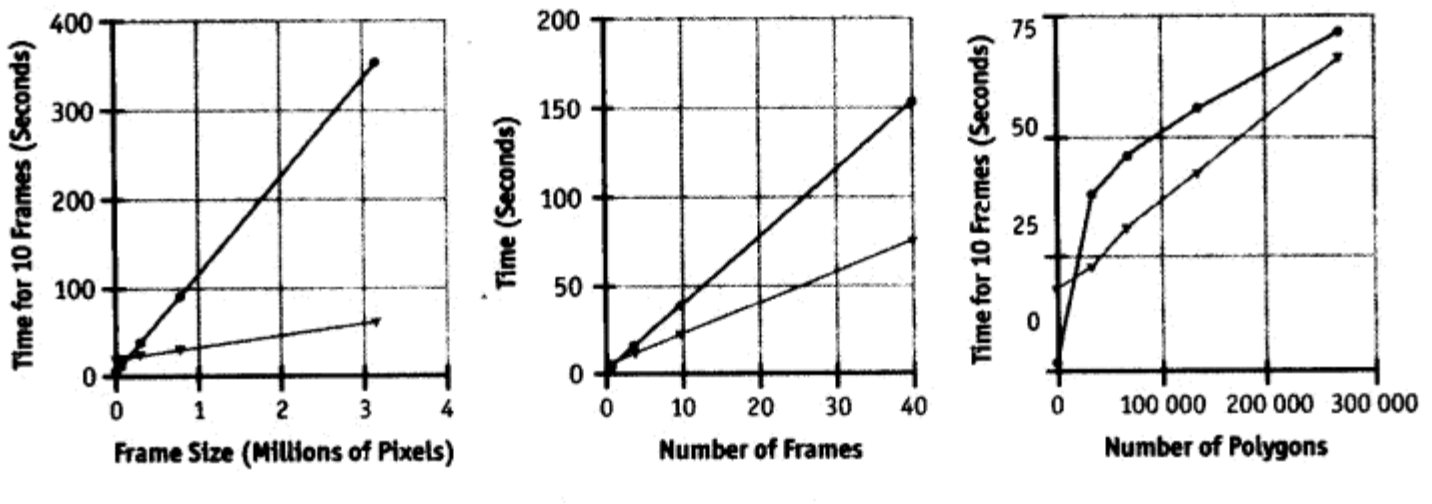


图34-4 基准结果的比较

运行的基准结果使用了大约33 000个多边形（几乎全都是四边形）的一个基本场景，和没有反走样的640 × 480像素（0.31百万个像素）的10个帧的动画。所用的系统是带有GeForce FX 5900的2.5GHz的Pentium 4 PC，运行的是Cinema 4D 8.207版本。图34-4的3个部分，分别对应于帧幅大小、序列长度和复杂性的变化。

随着场景中几何体复杂性的增加，Cinema 4D的离线渲染明显地得益于一些优化技术。硬件加速的方法也需要类似的改进。最容易执行的是视域四棱锥剪裁和基于硬件的逐物体的隐藏剔除。

34.6 收获和教训

CgFX工具箱灵活地把加速硬件的高质量显示和硬件加速的离线渲染都加入现存的软件中来了。这里提出的解决方法大概使用了150KB的C++源代码。

通过定义在.fx文件中的一组技术，CgFX允许程序员使用不同厂家出品的各种图形卡。但是为了优化操作性能，最好预先小心地统计一下有多少参数和纹理会同时需要。甚至最新的图形卡在这方面也只能传递有限的功率。

shader通常只是程序极小的一部分。软件的其余部分对硬件有非常专门的要求。这里介绍的项目以CgFX提供的广泛兼容的硬件为起点，但是增加了一些较高级的特性，例如渲染

屏幕之外的缓冲、多次取样，或产生阴影、减小目标范围等。可能缺少一些 OpenGL 的扩展，或是在一些图形卡上表现不同，或是它们的实现包含严重的缺陷。

尽管实际上 Cinema 4D 可以在 Windows 和 Mac 两个操作系统上使用，但是我们的插件只能在 Windows 上工作。CgFX 在 Windows 之外的一些平台上不可用的问题（在开发阶段）只是问题的一部分。Cinema 4D 的跨平台 API 可能隐藏许多在窗口、事件和线程的处理上的不同。然而，我们的目标是对下面的结构提供足够的功能并允许充足的访问，从而完全支持复杂的基于 OpenGL 的软件，但是并没有达到目的。

尽管图形设计软件的用户可能创建任意的复杂场景，但是它们仍然希望软件有恰当的响应时间。在开发 C4Dfx 插件的时候，我们体会到理解和计划对于精细安排在一起的线路是至关紧要的，也懂得了数据和事件之间是如何共享和传输的。

从一开始就要求深入考虑的另一个领域是对纹理和 shader 的管理。如基准测试所示，创建纹理、把它们载入图形卡，以及编译 shader 需要没有任何损耗，这个限制要求高度智能化的设计。当然，这样一个设计仅用扩展功能的附件加在现有软件上是难以实现的。

即时的 shader 能近似地模仿许多通常在 CPU 上计算的函数。举例来说，这里描述的用插件渲染的光照圆锥体的柔和边缘和镜面高光区，看起来就很像用软件渲染的。但是这样一个完美的再现需要使用原始的算法，或大量反复的工作。甚至像凹凸贴图那样的标准特性都可以用许多的方式实现，但会产生略有差异的结果。如何形成切线和副法线？如何依靠凹凸强度参数，将凹凸的法线矢量与原始的混合？如何从凹凸映射计算法线映射？要使用原始的、可能是非 2 的幂的凹凸映射图像，或一些 mipmap 水平吗？在这个方面以及其他的许多方面，Cg 已经成为用计算机图形算法进行实验的一个理想工具。

34.7　参考文献

Everitt, C. 2001. "Order-Independent Transparency." Available online at **http://developer.nvidia.com/view.asp?IO=order_independent_transparency**

Everitt, C., and M. J. Kilgard. 2002. "Practical and Robust Stenciled Shadow Volumes for Hardware-Accelerated Rendering." Available online at **http://developer.nvidia.com/object/robust_shadow_volumes.html**

Everitt, C., A. Rege, and C. Cebenoyan. 2002. "Hardware Shadow Mapping." Available online at **http://developer.nvidia.com**

Hasenfratz, J.-M., M. Lapierre, N. Holzschuch, and F. X. Sillion. 2003. "A Survey of Real-time Soft Shadow Algorithms." *Eurographics 2003 State of the Art Reports,* pp.1～20.

Kautz, J. 2003. "Hardware Lighting and Shading." *Eurographics 2003 State of the Art Reports,* pp. 33～57.

Kilgard, M. J. 2000. "A Practical and Robust Bump-Mapping Technique for Today's GPUs." Available online at **http://developer.nvidia.com**

Maxon. Cinema4D. **http://www.maxon.net**

第35章 在实时应用程序中使用高质软件渲染效果

Alexandre Jean Claude
Softimage
Marc Stevens
Softimage

35.1 引言

今天的图形硬件能够执行复杂的顶点和像素着色程序，从而显示高级的实时效果。这些效果不仅要求有一个顶点或像素着色程序，也要有许多供这些程序使用的输入数据，这样才能得到高质量的输出。这些输入数据很丰富，可以是顶点属性（比如法线和正切）、纹理坐标、纹理映射等（比如漫反射映射、法线映射和反射映射）。为硬件渲染生成组件的方法和工具也林林总总，从手工建立的、整合的，到自动的解决方法不一而足。

高质量的输入很重要。由于增加了对高质量的硬件渲染输出的要求，对产生它的美工的要求也相应提高了。因此今天的美工需要工具，以有效地产生高质量的输出，而这种工具要提供一种环境，能够灵活地、不断地反复修改结果。

最好能从高端建模和动画系统进行输入，而不是专为实时效果特制另外的工具。已经存在的流行工具是独立程序的软件包，它能够产生属性映射，从而得到复杂的实时着色效果。但是它强制进行一个高耗时的变换过程，有时还会丢失关键数据，即主要的 DCC 工具的 3D 场景数据和独立程序之间的数据。而一个整合的工作流程，有助于美工想象最终着色程序中的不同操作的效果，并且能够容易地反复修改结果。

现有的高端渲染方法，例如 mental images 的 mental ray 和 Pixar 的 RenderMan，是多年发展起来的，具有丰富的功能组合。这些渲染工具擅长于产生高精度和高细节的照片级图像。从草图产生这些视觉信息的方法，涉及到重写许多已经由大部分高端渲染方案实现的代码。这些方法具有一些优点，例如它支持把着色结点的任意网络渲染进单个映射，这个单个映射可用作漫反射映射；也支持把渲染器的镶嵌算法所产生的用于偏移映射的微多边形固化（bake）成法线映射；或把全局光照明模型的效果固化为映射。

这章将讨论如上的过程、其在实际中的应用、潜在的问题，以及应该如何围绕它们工作。

35.2　用于硬件渲染的内容流水线

用来创建适合于硬件渲染的内容流水线，时常会迫使美工在目标引擎的限制之内工作。美工通常受到纹理和三角形的预算的限制。普通的工作流程是在脑子里开始建立引擎所限制的内容。虽然对于产生硬件渲染的最佳内容，这个工作流程是了不起的，但是它限制了美工的自由，要想使用所有可行的工具，产生出完美的对象是很难的。另一方面，要求工作的内容是高质量的，但是又试图将其缩水于硬件渲染的限制中，这是一件很困难的任务，而且并不总能产生满意的结果。

我们的方法是让美工使用所有可行的工具，以产生高质量的对象，并把它的细节带入现成的硬件渲染对象版本之中。图 35-1 中显示的是流程图，它的工作过程如下。

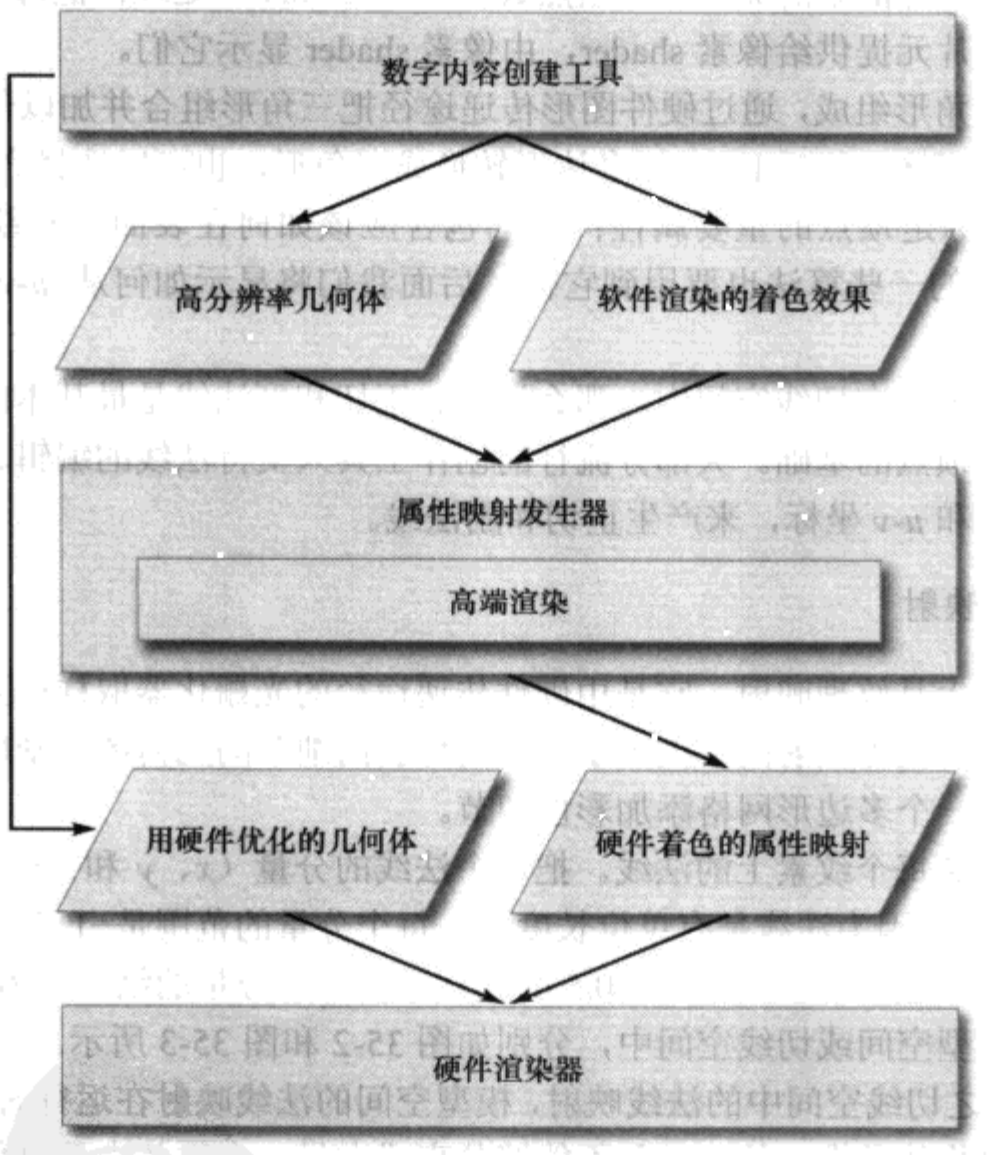


图 35-1　硬件渲染的内容流水线

- 美工使用 DCC 工具和高端软件渲染的着色能力，创建高质量的物体和着色效果。这个对象不直接用在硬件渲染中，而是作为原料用来产生硬件渲染的组件。
- 美工创建原始的高质量内容的现有硬件渲染版本。此时，美工只创建基本的数据，它们稍后被自动化的工具更新为包含有高质量的细节。举例来说，美工产生一个低分辨率物体，它与原始的内容有大致相同的形状，然后让工具为硬件渲染把属性映射固化。

该工具使用高质量几何体和着色效果，并且编辑必要的属性，把它们应用在物体的低分辨率版本上。硬件渲染的最终结果与高质量内容非常相近。该工具依靠软件渲染的办法，能够使用软件渲染的所有特性，再把结果固化在最后的属性映射上。本章的其他部分会详细地解释这一过程。

35.3 硬件渲染的组件

我们现在要好好看一下硬件渲染的内容传递途径。在这节中，我们要更详细地讨论用于硬件着色效果的不同组件的规范。组件分为两类：几何数据和属性映射。

35.3.1 几何数据

几何数据是由多边形网格结构和它的顶点属性组成的。这个数据被硬件光栅化用来产生要显示的像素。在硬件渲染的流程中，顶点数据由顶点 shader 处理；把每个三角形光栅化成片元；然后把这些片元提供给像素 shader，由像素 shader 显示它们。

网格结构由三角形组成，通过硬件图形传递途径把三角形组合并加以传送。DCC 工具的使用者产生多边形网格，然后 DCC 把多边形转换成三角形，也可以选择产生三角形条带。

u-v 的纹理坐标是顶点的重要属性；它们包含应该如何在表面上对纹理图像取样的映射信息。产生正切的一些算法也要用到它们。后面我们将显示如何从 *u-v* 的纹理坐标产生正切。

如果法线映射表示在切线空间中，那么法线、正切和副法线是最基本的。法线、正切和副法线是确定每个顶点的基础。大部分流行的创作工具只支持法线的编辑。稍后我们将显示如何使用表面法线和 *u-v* 坐标，来产生正切和副法线。

35.3.2 属性映射

属性映射本质上是纹理映射，它是由硬件传递途径的光栅化器取样得到的。当不增加物体的几何学密度时，纹理映射的密度大于创建物体表面细节的几何图形密度。例如我们可以使用纹理映射，为一个多边形网格添加彩色细节。

法线映射将存储每个纹素上的法线。把每个法线的分量（x、y 和 z）编码成纹理的像素组分（红、绿和蓝）。因为法线是有单位长度的，每个分量的范围是−1 到 1。红、绿和蓝的值域是 0 到 1。法线编码时每个分量有 0.5 的偏移，而每个分量的缩放率为 0.5。法线映射可以随意地表达在模型空间或切线空间中，分别如图 35-2 和图 35-3 所示。

一般来说，较之切线空间中的法线映射，模型空间的法线映射在运行时需要的计算较少。但是，每个映射对物体是惟一的，并且不能够共享或在另一个物体上重复使用。虽然切线空间中的法线映射需要较多的顶点运算，但是另一方面，它们能被重复排列，而且能在其他的物体上重复使用。

如图 35-4 所示的漫反射映射会被当作标准的彩色纹理映射使用。按照一般的红、绿、蓝和 alpha 组分给它们编码，并用这些组分代表物体表面的颜色。

反射映射也被当作标准的彩色纹理映射使用。它们也被编码成一般的红、绿、蓝和 alpha 组分，并且表现被物体所反射的环境颜色。

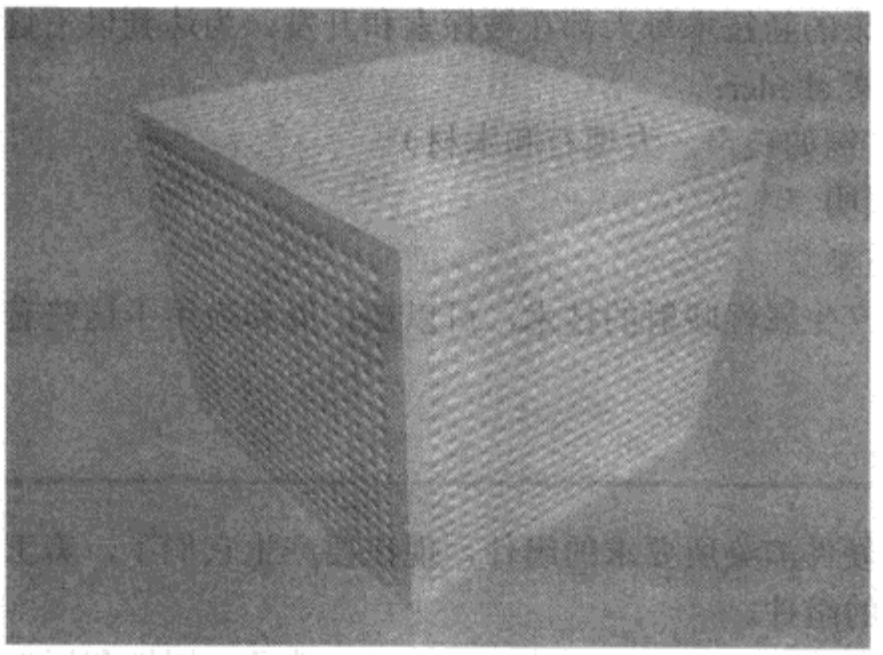


图 35-2　模型空间中的法线映射
因为法线表示在模型空间中，而且指向不同的方向，所以立方体各个面的法线映射非常不同

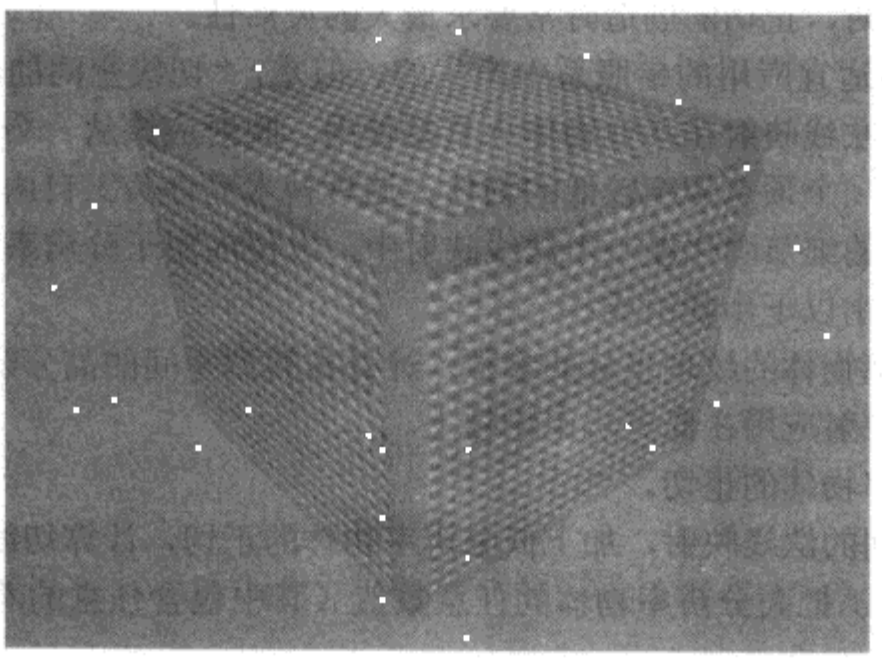


图 35-3　切线空间的法线映射
因为法线表示在切线空间中，不管面的朝向如何，法线总是指向相同的方向，所以立方体的每个面的法线映射相似

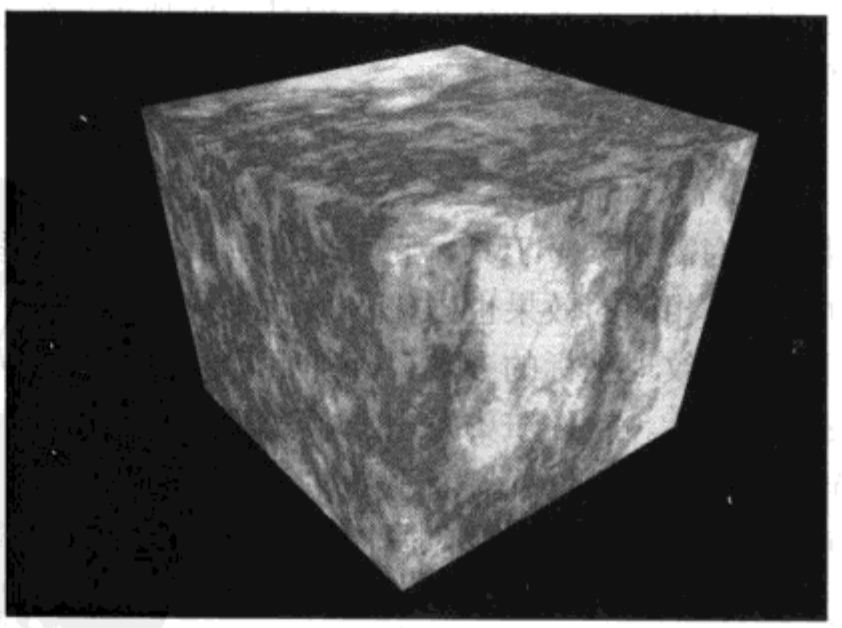


图 35-4　一个表面的漫反射映射

高级实时着色效果的新技术每天都在被探索和开发。为达到以下这些效果，其他类型的映射也被用于硬件像素 shader：

- 程序式纹理（例如噪声、大理石和木材）
- 基于环境的照明
- 非相片真实渲染

这章中所描述的产生属性映射的技术，可以被扩展和应用于这些着色效果。

35.4 组件的产生

我们已经说明了硬件渲染所要求的组件，现在要产生它们了。美工产生一些组件；而自动化的工具产生其他的组件。

理想的组件生成步骤是开放的和灵活的，凡是创造顶点属性和属性映射时人们所用到的运算法则，都应该包括在内。在不损失灵活性的前提下，尽可能自动的过程是我们的目标。在规定如何形成正切时，正切的创造时常要求最大的灵活性。

法线映射是在最适宜应用的参照系中产生的。但是，“切线空间的法线映射”是最常用的技术，因为它允许法线映射在其他物体上重复使用。映射应该从一个顶点密度相当高的物体上产生，而应用于一个顶点密度较低的物体。这种技术允许美工自由地模拟细节很高的物体，并且将这些物体的细节固化在一个法线映射中，以便应用于分辨率低的模型。

产生映射的技术由以下步骤组成：

1．使用低分辨率物体的纹理坐标的参数，计算在模型空间的高分辨率物体的表面法线。

2．把那个法线映射应用在低分辨率物体上。

3．计算低分辨率物体的正切。

4．使用模型空间的法线映射，加上低分辨率物体的正切，计算切线空间的法线映射。

工作流程允许美工把高分辨率物体的任意数值（其中包含任意的着色效果），固化入简单的纹理映射之中。

35.4.1 创建几何图形

在几何学方面，美工用 DCC 工具产生物体。但是一些顶点属性是很难用手建立的，因此，美工常借用一组辅助工具来创建它们。

35.4.2 对纹理和顶点的渲染

为了产生属性映射，我们需要一个工具，它可以使用目标物体的参数，把软件渲染的着色效果渲染进纹理映射之内。相同的效果也固化入顶点属性之内。

对纹理的渲染使用现有的纹理坐标的参数，外加目标纹理映射。为了得到纹理的每个纹素，算法计算在物体表面的坐标。软件渲染发出一条光线并在坐标上对属性取样。渲染顶点前，对这些属性的取样是在每个顶点位置，而不是在每个纹素位置。

对一个纹理和顶点的渲染只产生表面颜色属性，因此需要寻找一个方法来渲染自定义的属性，例如正切和法线。解决办法是产生一个自定义的软件着色，它把表面的正切和法线渲染成表面颜色。因为可编程硬件的着色最后用 4 组分的寄存器来工作，任何的自定义属性可

以在顶点颜色或纹理映射中编码为一个 RGBA 值。

图 35-5 是一个用设置成表面颜色的法线所渲染的物体的例子。

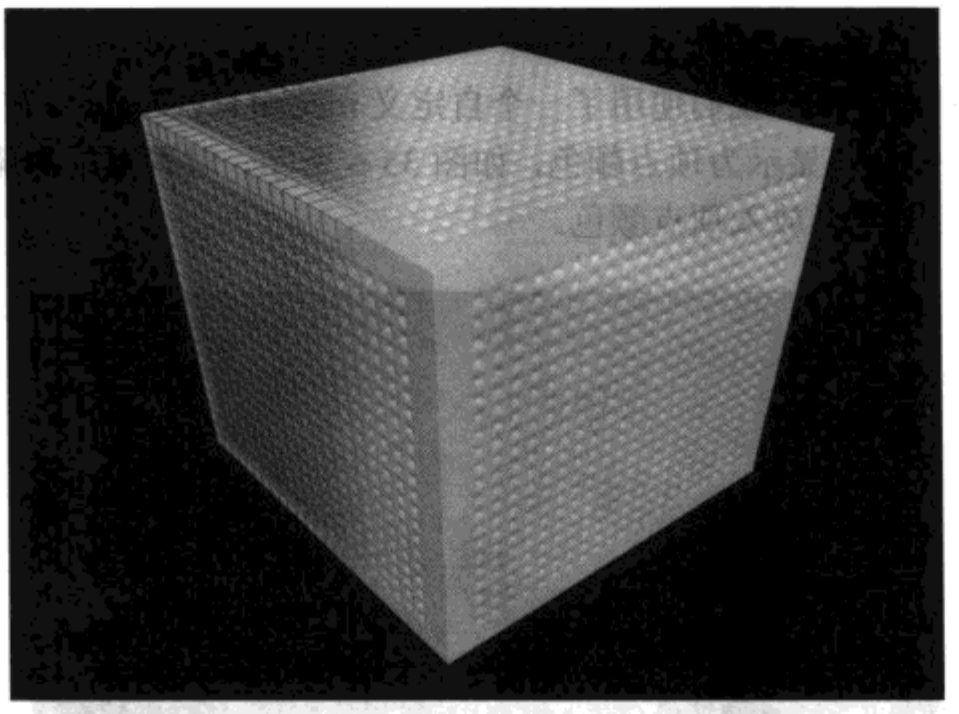


图 35-5　用 mental ray 渲染的物体法线
将产生的颜色与线框显示混合在一起以表现几何体的复杂性

我们可以使用一个纹理坐标编辑器来创建 *u-v* 坐标。还存在一些解决办法，它能够自动地显示几何图形而且产生很好的纹理坐标组，并可以毫无问题地用在硬件流程中。

以下是 *u-v* 坐标的主要标准：

- *u-v* 坐标应该有大体好的 *u* 流和 *v* 流（好流是指在连续的 *u* 值或 *v* 值之间，间断性最小）。这对基于 *u-v* 坐标产生正切的工具是至关紧要的。大多数产生正切的算法使用给定的纹理坐标系的 *u* 流和 *v* 流，以确定物体表面的正切方向。
- *u-v* 坐标集合也许有一些极小的岛（island），即 *u-v* 空间中连在一起的三角形组，岛会引起纹理映射的间断性。当算法对靠近间断的棱上的纹素取样时，这些间断可能引起问题。虽然这个问题可以通过溢出（spilling）或平均纹素的值绕过，但是一般而言，我们建议避免间断。

创建法线通常是物体建模过程的一部分。正切和副法线能容易地由 *u-v* 的坐标和法线矢量推导。在这个试验中，我们根据应用所提供的法线和正切，计算顶点程序里面的副法线矢量。因为 DCC 工具的使用者可能需要覆盖这个计算，所以对它们来说，正切必须是可保存的。我们通过把正切存储在顶点颜色属性中做到这一点，因为在凹凸贴图中用不着顶点颜色。

提交给顶点程序的最终顶点格式，显示在列表 35-1 中。副法线是用法线和正切的叉积计算的。

列表 35-1

```
struct app2vertex
{
  float4 position : POSITION;
  float4 normal   : NORMAL;
  float4 tangent  : COLOR0;
  float4 uv0      : TEXCOORD0;
```

```
    float4 uv1      : TEXCOORD1;
    float4 uv2      : TEXCOORD2;
    float4 uv3      : TEXCOORD3;
};
```

为了计算试验中的正切，我们使用了一个自定义 mental ray shader（它暂时地覆盖了当前的物体着色），而且把正切显示为顶点颜色，如图 35-6 所示。对顶点的渲染显示了 mental ray shader 的魔力，最后把结果放入顶点颜色。

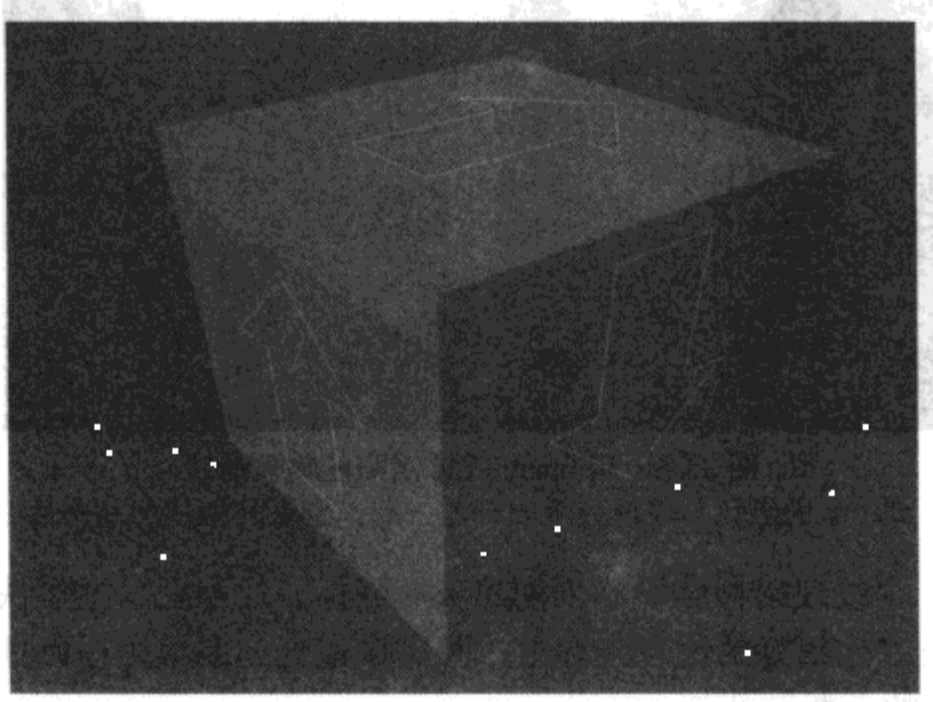


图 35-6 由 mental ray 渲染的正切，并且设置为顶点颜色
箭头说明正切的流动。在这种情况下，在左前面和顶面正切正在无缝地流动，而在右前面它朝一个不同的方向流动

使用一个插件，或者把正切的结果写入顶点颜色属性的脚本，也可以重计算正切。

凹凸映射效果的硬件所要求的创建纹理映射的过程，应该尽可能地自动化。目标是把复杂的几何细节和着色效果固化入最终的低分辨率物体。高和低分辨率的模型不需要在它们的 *u-v* 映射或拓扑上进行任何校正。

法线映射是在两个通道产生的。第一个通道使用低分辨率物体的纹理坐标的参数，计算在模型空间中的高分辨率物体表面的法线。

我们在试验中对第一遍使用的技术如下：

1．使用一个把法线渲染为颜色的 mental ray shader，暂时遮蔽高分辨率物体。

2．由于低分辨率物体被渲染为完全透明，所以渲染高分辨率物体的细节时，不用考虑它的几何体。

3．使用低分辨率物体的纹理坐标的参数，调用对一个纹理的渲染，把高分辨率物体的细节渲染进模型空间的法线映射之内。

高分辨率物体的几何细节被渲染进法线映射内。最好的是也考虑到由置换映射和几何体 shader 等软件执行的几何操作。用这个方法有可能“固化”像头发或粒子这样的程序式几何体。

第二遍使用先前在低分辨率物体上计算的正切，加上模型空间的法线映射，产生切线空间的法线映射。

我们使用的第二遍的技术如下：

1．除了低分辨率物体之外，暂时使所有的物体都看不见。

2．在低分辨率物体的表面上，对模型空间的法线映射取样。

3．在低分辨率物体的表面上，使用顶点颜色属性对正切取样。

4．为了创建所取样法线的新参考值，使用法线和正切计算副法线。

5．把取样法线从模型空间转换到切线空间。

产生漫反射映射的过程类似。在我们的试验中，计算这个映射时产生了一个问题。因为软件渲染可能运行一个任意材质的 shader 的网络，所以当没有任何光照在上面时，不可能抽取光照明方程的不同项，并返回漫反射颜色。但是我们可以提供遍历物体表面的平均照明，技巧地进行渲染。这个技巧允许漫反射颜色均匀地显示，而不必考虑场景光源的设置或表面的法线。

执行下列步骤来产生漫反射映射：

1．暂时禁用高分辨物体上的环境和镜面反射的贡献。这使我们能得到一个纯的漫反射映射。

2．暂时禁用场景中的光照明，并把一个光源放到适当的地方，使高分辨物体得到均匀的照明。

3．把低分辨率物体渲染成完全透明，所以当描绘高分辨物体的细节时，不必考虑低分辨率物体的几何体。

4．调用对一个纹理的渲染。使用低分辨率物体的纹理坐标的参数，把高分辨物体的细节渲染进模型空间的法线映射内。

即使光源设置允许我们在没有太多光照明贡献时对漫反射的颜色取样，还是有一些照明的失真。可以使用颜色校正工具纠正这些失真。

在进行漫反射映射通道的渲染时，要考虑到影响物体漫反射颜色的着色效果。美工可以使用程序式的效果（例如大理石、木材或噪声纹理），这些效果自动地被固化进表面的映射中。

当环境是静态的时候，可以使用环境映射来模拟反射材质。在这种情况下，我们可以有效地利用高端渲染软件产生高质量环境映射，这些映射甚至可以把高级的渲染效应考虑在内，例如全局的光照明和最终聚合（final gathering）等。两个最常用的环境映射是立方体映射和球体映射。

对于立方体映射，必须要把它的渲染设置得适于物体的包围立方体，并且把环境渲染在那个立方体上。对于立方体的每个面，我们从反射物体的观点渲染所反射的图像。所产生的映射以后将被用于图形传递途径上的硬件反射映射的 shader 中。

为了得到球体映射，必须设置摄像机视域来渲染一个半球，半球能够覆盖物体朝着视点的部分，然后将这个映射用在硬件图形的传递途径上，用作一个环境 shader 的球体映射。

这个方法把全部高级的渲染功能，例如全局的光照明模型、使用软件渲染的刻蚀，固化进生成的纹理映射之内。

35.5　试验情况和结果

为了举例说明在本章中所述的技术，我们提出一个实在的试验例子，然后评论结果。

在这个试验中，我们使用的高分辨率模型有 440 个显性的多边形网格，还有略多于 260 000 个的三角形，请见图 35-7 和图 35-8。模型将不同类型的材质附在人物的每一边。盔甲板是用反射的 Phong 类型的材质做成的；在双臂上的织物是用非反射的、Lambert 类型的材质做成的。

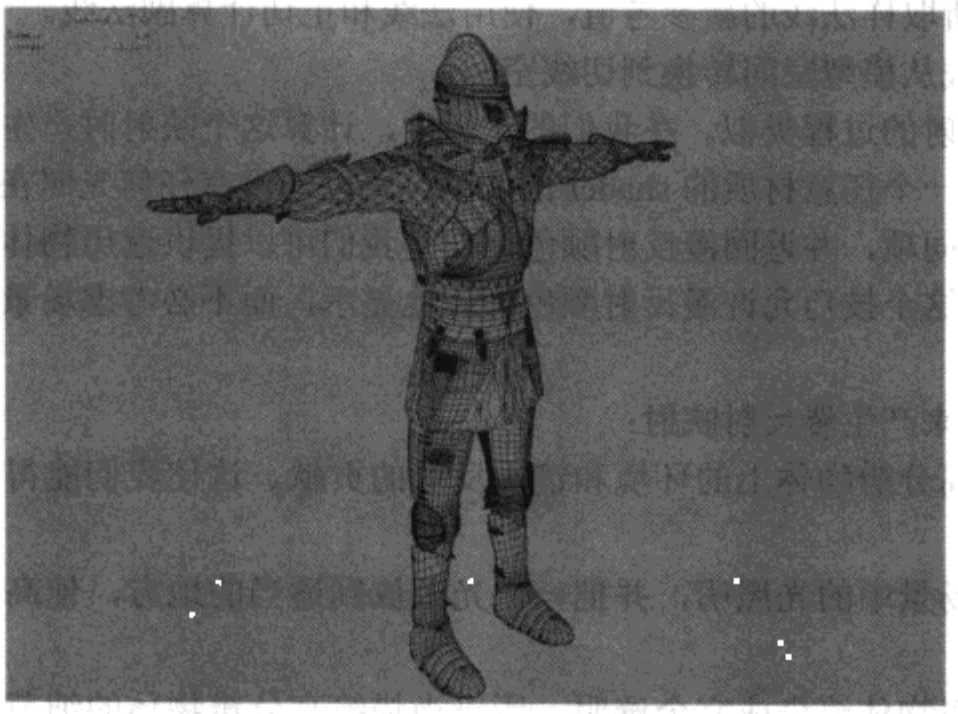


图 35-7 高分辨率模型的线框视图
这个模型将作为实时输出而产生属性映射的来源

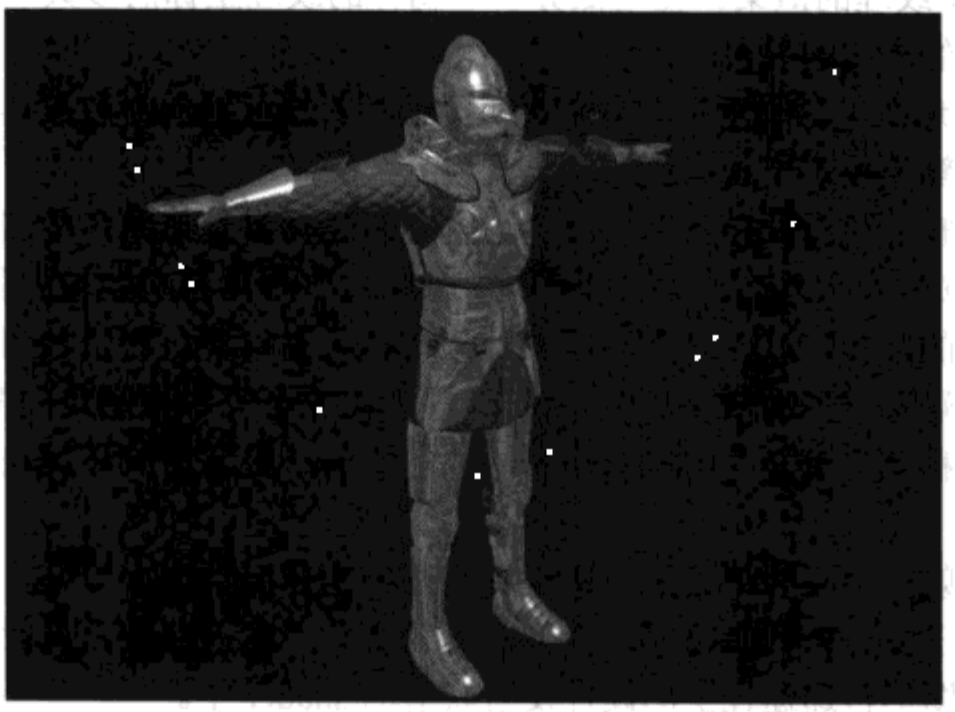


图 35-8 高分辨率模型的最终渲染
最终的渲染是用 mental ray 做的

低分辨率模型由单个的网格和少于 4 000 个的三角形组成，如图 35-9 所示。两个类型的实时 shader 将附在这个模型上：（1）对于盔甲，使用一个反射凹凸映射的实时版本；（2）为双臂上的织物，将使用一个简单的非镜面反射凹凸映射。

高分辨率和低分辨率模型的拓扑非常不同。高分辨率模型本身没有纹理坐标，而且它不是由纹理映射引起。已经把高分辨率模型上的所有细节建模为多边形的网格。另一方面，低分辨率模型完全依靠纹理映射得到结果。在本例中，我们使用两个不同的映射：一个是在切

线空间中的法线映射，另一个是为了光照明颜色的漫反射映射。最终的实时着色把这些映射当作输入，并产生最后的图像，显示在图 35-10 中。

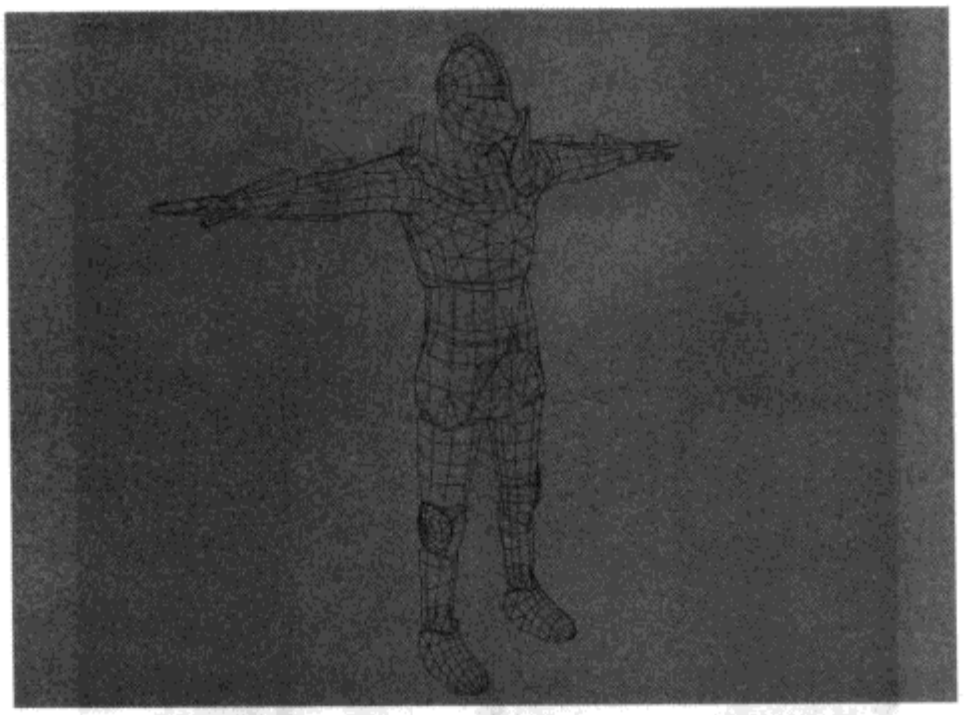


图 35-9　低分辨率模型的线框视图
实时的着色技术将会用于这个模型，以近似原始的高分辨率模型

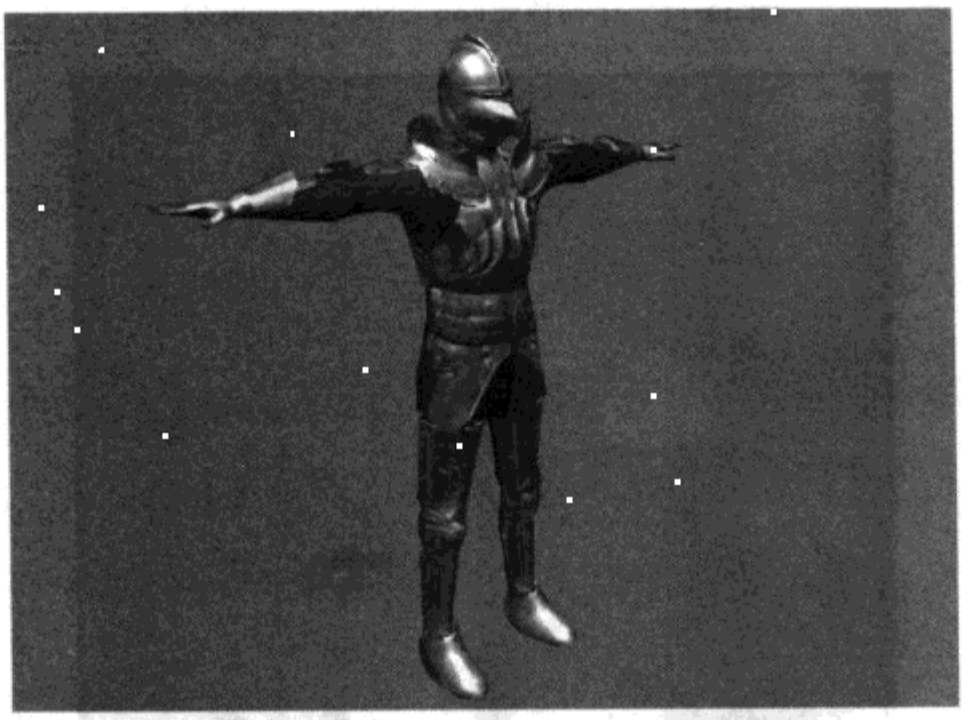


图 35-10　低分辨率模型的实时着色视图
实时的着色技术使得这个模型很像原始的高分辨率模型。最后的渲染是使用 Cg shader 做的

请注意，盔甲上的小细节，以及布料上缝制的花纹，实际上不是作为几何体而存在的。GPU 表面 FX 插件把这些细节固化进法线映射之内，而法线映射又用于从高分辨率模型而来的凹凸映射的实时效果。同样的法线映射也用于制作反射凹凸贴图，因此，表面细节呈现为变形的反射映射。

当接近去看所产生的映射时，我们可以看到在物体空间的中间法线映射（显示在图 35-11 中），包含了高分辨率模型的所有表面细节。高分辨率模型的表面被投射到低分辨率模型的 *u-v* 空间之中。

图 35-12 所显示的切线空间中的法线映射，是由物体空间中的法线映射和低分辨率模型的切线推导出来的。

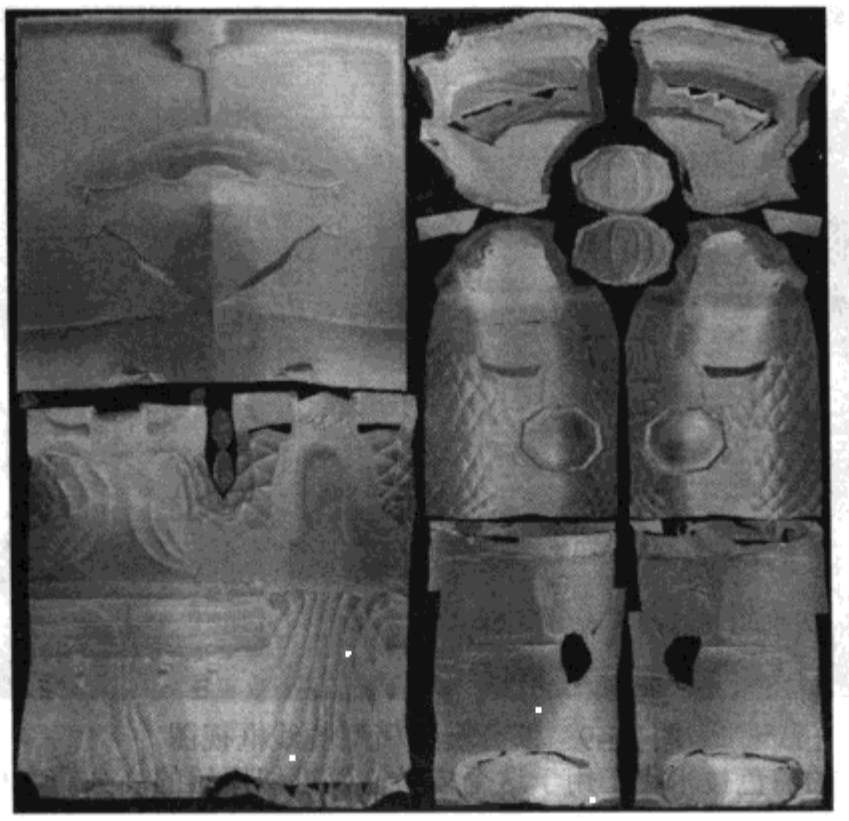


图 35-11 在物体空间所生成的法线映射

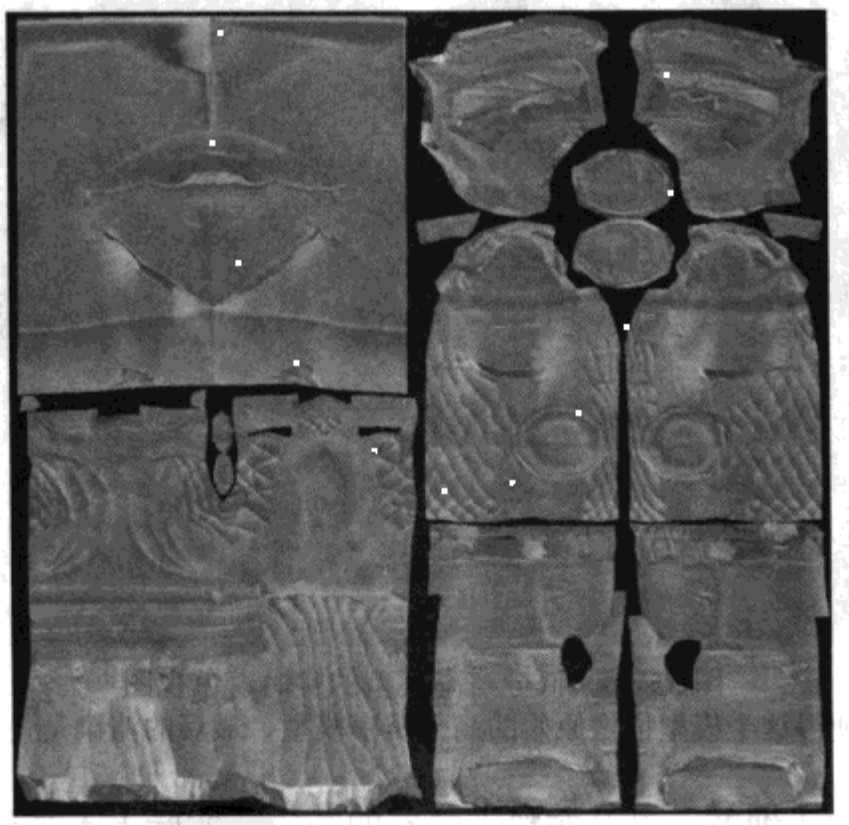


图 35-12 在切线空间所生成的法线映射

图 35-13 显示的是用作实时效果的颜色组分的表面映射。可以看到，从图 35-8 的原始模型抽取颜色，我们已经获得了很好的结果。

图 35-14 比较了软件渲染和硬件渲染的模型。结果与原始的非常接近。要产生好的属性映射，系统要求几个条件。第一个要求是低分辨率模型所使用的 *u-v* 空间必须适当：它应该是非重叠的，并且间断最小。第二个要求是高分辨率模型的形状一定要与低分辨率模型相似。

虽然不需要形状有一个拓扑的相互关系，但是两个模型的形状应该是相似的。在把高分辨率模型的属性投射到低分辨率模型上时，这个要求可以把属性映射生成的误差减到最小。在这个试验中，美工用手编辑 *u-v* 的坐标，目的是最大限度地在模型的表面上使用纹理映射。根据形状变量的 delta（增量），我们发现了从高分辨率模型投射到低分辨率模型的最大距离应该是相对小的。

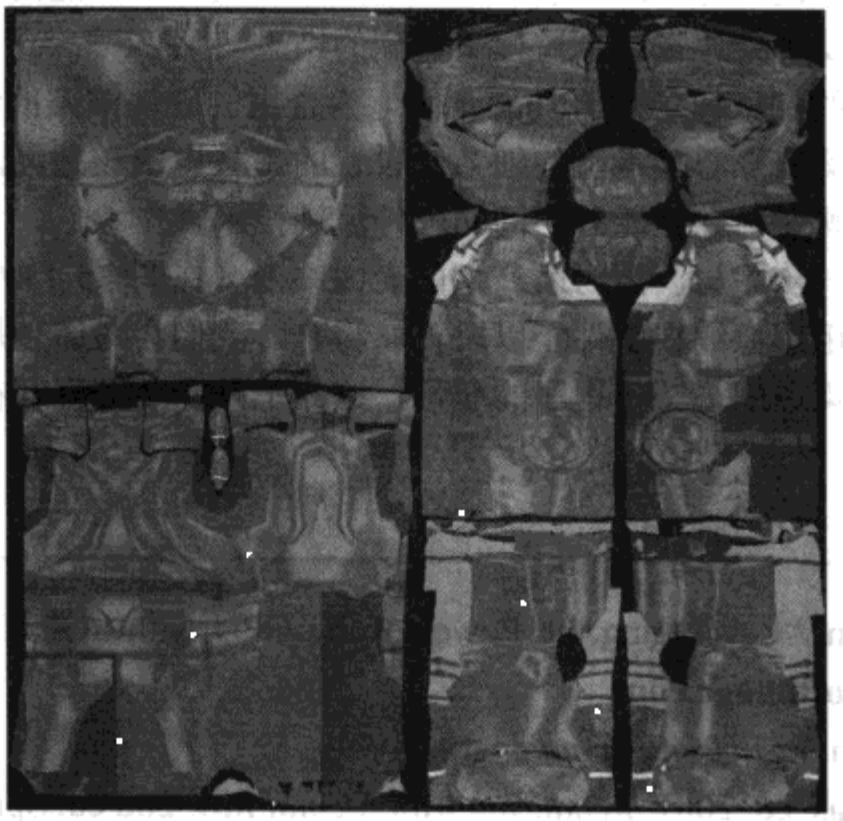


图 35-13　所生成的近似的漫反射映射

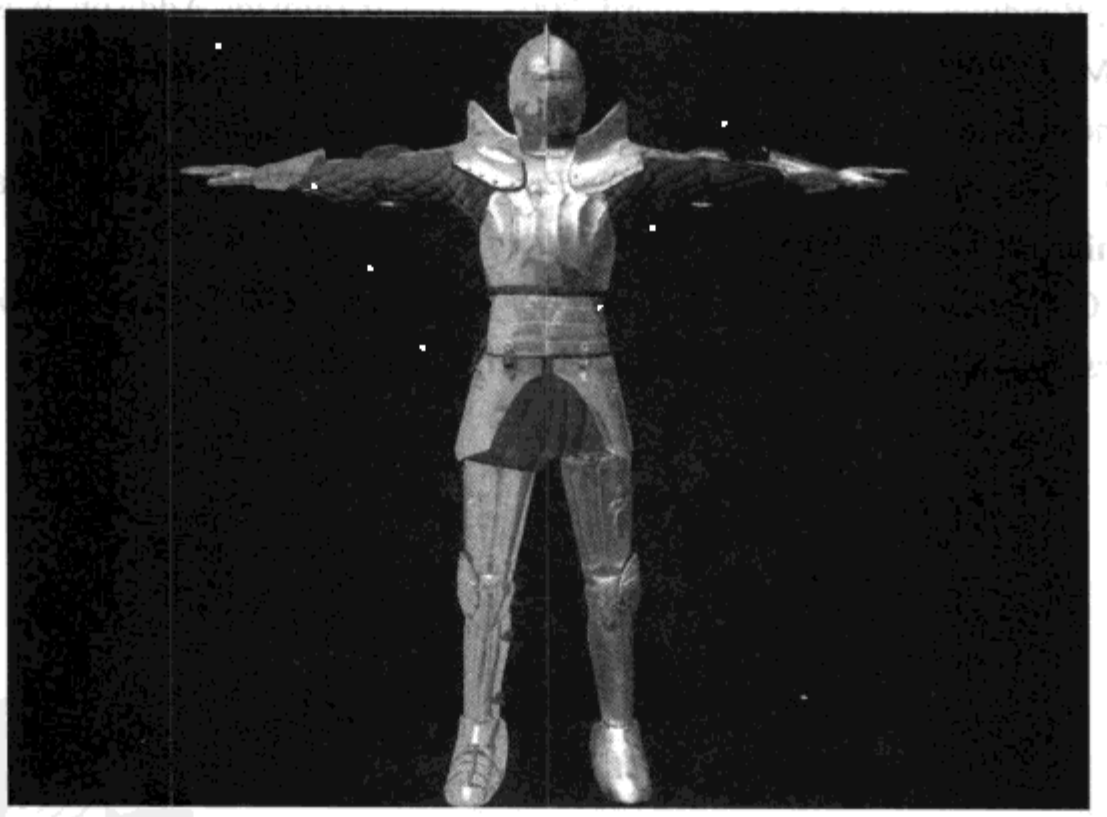


图 35-14　渲染的比较

软件渲染的高分辨率模型（在左边）与硬件渲染的低分辨率模型（在右边）的比较。注意两者手和腿的颜色不同。右手臂和左边的腿是红色的，而左手臂和右腿在原始的来源材质上是黑色的

35.6 小结

高端软件渲染工具提供了一个有力的和灵活的传递途径，用来产生简单的属性映射，例如漫反射映射和法线映射。上述工作流程说明，在产生用于硬件渲染传递途径的属性信息时，将一组任意的高分辨率模型与一组任意的 shader 网络一起使用，能提高美工的创造力；而且通过它们还能容易地反复处理在这个过程中得到的结果。

把高分辨率模型的信息和复杂的 shader 固化为合用的硬件渲染的组件，所使用的工具需要是自动化的，无须美工操心技术上的细节。设置的工具也应该是灵活的，它应该能够适于产生其他类型的属性映射，并且能与不同的硬件渲染技术合作。

该技术还需要进一步完善。因为所产生的表面映射（在图 35-13 中显示）仍然显示出一些着色的失真，需要用图像编辑软件予以矫正。将来，我们会考虑使用不同的技术消除这些失真。另外，因为渲染速度是迭代程序的瓶颈，这几乎是一个一直存在的问题。

35.7 参考文献

AVID Corporation. "Leveraging the Power of the GPU with SOFTIMAGE XSI." Available online at **http://www.softimage.com**

AVID Corporation. *XSI User Manual.*

Driemeyer, Thomas, ed. 2001. *Rendering with Mental Ray,* 2nd ed. Springer.

Driemeyer, Thomas, and Rolf Herken, eds. 2003. *Programming Mental Ray,* 2nd ed. Springer.

Fernando, Randima, and Mark J. Kilgard. 2003. *The Cg Tutorial.* Addison-Wesley.

Kilgard, M. J. 2000. "A Practical and Robust Bump-Mapping Technique for Today's GPUs." Available online at **http://developer.nvidia.com**

Microsoft Corporation. *DirectX* 8.0 *Programmer's Reference.* Available online at **http://msdn.microsoft.com/directx**

NVIDIA Corporation. 2002. *Cg Toolkit User's Manual: A Developer's Guide to Programmable Graphics.* Available online at **http://developer.nvidia.com**

第 36 章　将 Shader 整合到应用程序中去

John O'Rorke
Monolith Productions

36.1　引言

这章讨论把 shader 的支持添加到应用程序中去，目的是改善 shader 的灵活性，方便对它的维护。这里不包括处理 shader 的个别 API 或 shader 的语言。而是高屋建瓴地考察应用程序和 shader 之间所需的通信层，创建一个灵活而又有力的数据驱动渲染器。编写这章的原因很简单：当设计一个被 shader 驱动的渲染器时，我发现许多文档是关于编写 shader 和使用相关 API 的。但是没有讨论应用需要提供什么，才能很好地管理不同的 shader。许多文章讨论了如何从 effect 得到变量名和评注（annotation），但是没有提到应用程序应该如何使用这些信息，才能把必需的数据提供给 shader。要想使应用程序以灵活的方式支持 shader，必须给它加入合适的工作。这章试图了解这个领域的各个方面，并提出一个有效又有力的解决方案。

这里描述的管理 shader 的技术，来自反复试验的个人经验，与其他工程师的讨论，以及对类似应用程序的分析。它们所起的作用类似于一种黏合剂，把应用的技术设计和所选择的 shader API 黏合起来。

36.2　关于 shader

在将 shader 整合进应用之前，让我们考查一下 shader 的作用。shader 的目的是指定一种方法去渲染几何性质的集合。如果取的范围适当，shader 允许把渲染状态作为一种资源来处理。这种威力简直令人吃惊，因为它竟然允许一个应用的渲染器几乎完全成为受驱动的资源。

过去，只有外部资源文件能够控制渲染的外观，例如纹理和网格。但是现在使用 shader，甚至渲染实际几何图形的设备的设置，都能被当作外部资源来处理。这个技术早就给电影业带来了好处（它们使用像 RenderMan 那样的着色语言），近来它又引人瞩目地增加了实时渲染的灵活性。因为这种被 shader 驱动的方式不需要把自定义代码加入应用程序，所以它允许更加多样性地渲染场景。另外，它让我们容易地增加和保持更多的渲染方法，而且允许对新效果和优化进行极快的实验。随着这个工具的日臻完善，将来它甚至可能使美工可以完全地控制渲染。

现在已经存在各式各样的实时 shader 语言，而未来必定会有更多的语言问世。这些 shader 语言可以分成两类：独立程序和 effect 文件。

独立程序

独立程序由一个文件集合组成，其中每个都在渲染的一遍中执行一个顶点 shader，或一个像素 shader。Cg 编程语言和标准的顶点和像素 shader 文件就是相关的例子。另外，为了简化实现，许多 shader 语言的自定义实现也放在这一类。然而，这种方式有一个缺点，就是以这种方式构造和管理大量的复杂效果很困难，因为每遍通常需要两个源文件，一个是顶点单元的，另一个是像素单元的。

因为其他的设备状态（例如填充模式或剔除模式）不依靠一个单独顶点或像素 shader 程序，所以很难使用独立的文件控制这种状态，这是这个方式的主要问题。例如画一个物体要渲染两遍：第一遍是实体的 Gouraud 填充，第二遍是平面的线框涂色。这个简单的过程需要 4 个独立的 shader：2 个为顶点单元，2 个为像素单元。当使用这些 shader 的时候，需要在应用程序中把设备的填充模式状态改变为第二个渲染遍上的线框状态。

如你所见，如果用于一种特殊渲染风格的所有程序都可以巧妙地放进一个单独的 shader 资源之内，而它又能够控制整个的设备状态，那就太好了。这就是启用 effect 文件的原因。

effect 文件：为什么使用它们？

effect 文件的最普遍实现是 Microsoft 的 effect 框架（Microsoft 2003）和 NVIDIA 的 CgFX（NVIDIA 2003），后者是 Microsoft effect 框架的一个超集。effect 文件的方法允许将独立的像素和顶点程序放入一个单独的文件之内，然后将这些顶点和像素 shader 按 pass 和 technique 分组。通过引入这些 technique 和 pass 的概念，effect 文件允许逐个遍地指定设备状态，这样在各种不同顶点和像素 shader 文件的集合上，就产生了更易于管理的、更资源驱动的、最终是更加有力的方法。列表 36-1 是一个 effect 文件的例子。

列表 36-1 一个简单的 effect 文件

```
// Our vertex format
struct SVertex {
  float3 Position : POSITION;
  float4 Color : COLOR0;
} ShaderVertex;

// Scene parameters
float4x4 CameraTrans : WorldViewProjection;

// Material parameters
float4 FillColor : MaterialParam <string desc = "object color">;

//Selected rendering technique
struct SVSOut {
  float4 Position : POSITION;
  float4 Color : COLOR0;
};
```

```
SVSOut Selected_VS(MaterialVertex IN) {
  SVSOut OUT;
  OUT.Position = mul(CameraTrans, float4(IN.Pos, 1.0f));
  OUT.Color = FillColor * IN.Color;
  return OUT;
}

struct SPSOut {
  float4 Color : COLOR;
};

SPSOut Selected_PS(SVSOut IN) {
  SPSOut OUT;
  OUT.Color = IN.Color;
  return OUT;
}

technique Selected
{
  pass p0
  {
    CullMode = CCW;
    BlendEnable = false;

    VertexShader = compile vs_1_1 Selected_VS();
    PixelShader = compile ps_1_1 Selected_PS();
  }
}
```

在本章的剩余部分，我们以 Microsoft 和 NVIDIA 所建立的标准为基础，考察把 effect 文件纳入应用程序的技术。本章的 effect 文件所提供的每个特性，从调整必须用于各遍的 shader，到建立设备的状态等，都需要得到某个有活力的 shader 系统的支持，而系统则要整合到应用之中。

根据经验，最好使用这些现成的效果系统，而不要试图创造一个自定义的实现。如果现成效果系统使用得当，它们可能非常有效，应用起来非常简单，写作起 shader 来也会非常快速，并且比较不易出错。而这些 shader 语言的 API 也非常容易应用，同时它们明显地缩短了开发时间。此外，使用外部实现有助于应用将来的检验，因为它可以把应用程序和基础平台专用的 API 相隔离。如果在目前的某个平台上，还不能使用现成的 API，就不得不执行自定义的方式。即使在这种情况下，也应该把 effect 文件的标准小心地加以研究，将其看作有活力和灵活的 shader 系统的模型基础。

36.3　一个 effect 文件的剖析

这节对用到的 effect 文件的各个部分进行分析，并且考查应用与 shader 之间通信的各个成分。这里仅了解有关的成分，而不讨论 effect 文件格式的全部语法。

同样地，即使未使用 effect 格式的系统，也可以并且应该包含类似的、允许整合进应用

中的结构。

36.3.1 变量

effect 文件中的变量起的作用与任何其他编程语言中的一样。每个都有一个独特的名字和相关的值，每个都是一种类型（类型的种类很多，有字符串、结构和多维的数组等）。应用可以列举全局范围的一切变量，并且读或写出它们的值。这些全局的变量是把数据传达给 shader 的主要方法，例如所应用的纹理是哪一种，或当前摄像机的变换是什么等。

一个应用不仅能决定全局参数的值，而且也能决定每个变量的类型，所以当把数据写到 shader，或从中读出时，应用能够确保使用有把握的类型。effect 文件中的全局变量有联合的标记，称为语意，这是其他语言所没有的。语意是附加的本文符号，这个符号可以与应用所查询的变量联合。应用可以因为任何目的使用语意，但是它时常指定如何使用一个变量，或变量需要什么数据。以下是一个变量的例子，它带有叫做 LightRadius 的语意，可以由一个应用在运行时间读入：

```
float Radius = 1.0 : LightRadius;
```

36.3.2 结构体

effect 文件的结构体与 C 风格的结构体类似。它们包含一个变量集合，各个变量要由结构体的不同实例来实例化。结构中的每个条目（entry）也可以包含一个语意，这一点与全局参数十分相似。然而，应用不能够直接访问结构。结构的信息必须通过结构的实例来收集。如果一种结构的定义对某个应用是重要的，那么必须在 shader 里定义这个结构的变量实例，然后由应用程序检验那个变量。

36.3.3 pass

一个 pass 代表一个 shader 的单个渲染遍的设备状态。这包含相关的顶点和像素传递途径的设置。每个 pass 有一个独特的名字，并且可以被应用访问。pass 是原子的渲染单位。它是设备状态的最根本的来源，如果要渲染一个物体，必定有一个激活的 pass 指出如何渲染它。

36.3.4 technique

某种风格的渲染时常需要多个遍，因此需要一些方法将遍分组。这就是 technique 起的作用。technique 是有名字的遍集合，并且用作验证设备的原子单位。这意味着，当 effect API 检查在设备上的支持时，也要检查在 technique 里的每个 pass，看它是否能够在设备上操作。如果设备不支持任何 pass，则认为整个 technique 也不被支持。因为存在这个验证，所以 pass 不可能在 technique 外存在。在运行期，technique 也对在所包含的每 pass 上的迭代提供支持，并访问关于每个 pass 的信息。

36.3.5 评注

评注是附在 effect 文件上的 technique、pass 和全局变量的用户数据。评注使得 shader 作者可以提供从 shader 所输出的另外数据；它们采取的形式是类型名=值；以后应用能够通过评注查询任何的 technique、pass 或参数。在运行期，可以根据名字、类型和值来查询每个

评注。这些评注是不变的，而且应用程序只能以只读形式访问它们。下面的代码是一个注释的例子：

```
float Alpha = 1.0 <string Description = "Opacity of the object";>;
```

36.4　shader 数据的类型

我们已经知道了能在 shader 和应用程序之间通信的基本元素（primitives），但是还没有描述通信的信息。这节来了解信息本身，而把通信的技术留待以后讨论。图 36-1 是一个框图，高级别地观察数据如何在应用和 shader 之间来回运行。

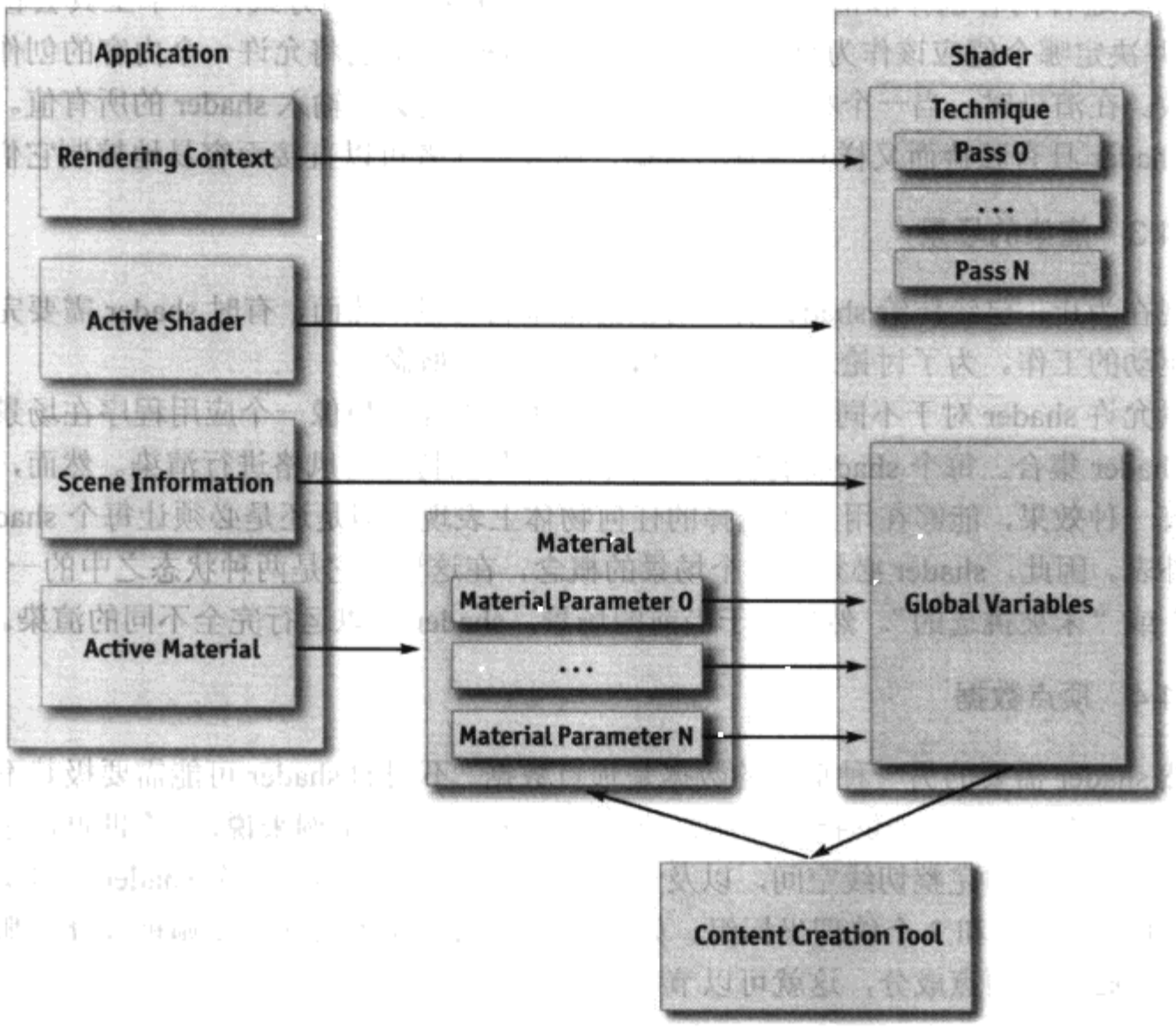


图 36-1　shader 和应用之间通信的流程

36.4.1　场景信息

必须从应用程序传递到 shader 的第一组数据是场景数据。场景数据包含渲染一个 shader 所需要的各种渲染状态，就好像在应用程序和 shader 之间的一份契约。shader 知道，它必须依靠应用通过特定变量来提供确定的数据，才能完成它的任务。没有这种场景数据，shader 不能凭空操作，甚至连最简单的操作也不能够执行。

物体空间到剪裁空间的转换、骨骼的变换、摄像机位置、当前物体的颜色，以及应用所

保持的任何与 shader 有关的数据，都是场景信息的例子。这种数据是单向的，只能从应用到 shader 去，而且每当场景数据变化一次，在 shader 中的数据也必须随之更新。

36.4.2 材质

场景信息包含应用所产生的信息，但是它不包含一个至关紧要的数据集，那就是按客户具体要求而制造 shader 的数据。这种数据包括了允许美工控制的 shader 的任何一方面的参数。例如 shader 将会使用的纹理、表面的镜面反射程度或移动纹理的速度等。比起场景数据来，材质数据的通信具有更多的双向性。这是因为 shader 定义的变量可以为材质所支持，它们必须由一些应用或工具来读出。接着由材质来定义变量的值，它也必须被代入到 shader 中去。

材质要适合内容创作和渲染的传递途径，时常采取下列的方式：一个工具会检验一个 shader，并决定哪个值应该作为材质参数提供给 shader。然后它将允许一个内容的创作者指定这些参数。在渲染时，当一个材质被激活的时候，它填进美工输入 shader 的所有值。这个流程允许 shader 具有独特而又样式多变的数据，内容创作者可以直接而容易地控制它们。

36.4.3 渲染的场景

到现在为止，已经传给 shader 的所有信息都来自变量。然而，有时 shader 需要完成不能被数据驱动的工作。为了讨论这个问题，要介绍场景的概念。

场景允许 shader 对于不同的情况应用不同类型的渲染。想像一个应用程序在场景中有一个大的 shader 集合。每个 shader 有特殊的外表，并且以特殊的风格进行渲染。然而，尽管应用需要有一种效果，能够在用户所选择的任何物体上表现，但是还是必须让每个 shader 有它的独特外表。因此，shader 必须有一个场景的概念，在这里，它是两种状态之中的一个："被挑选的"或"未被挑选的"。然后基于当前的场景，shader 可以运行完全不同的渲染。

36.4.4 顶点数据

当然 shader 需要的另一种形式的数据是顶点数据。不同的 shader 可能需要极其不同的数据，因此一个 shader 必须能够指出它需要的是哪种顶点成分。举例来说，一个凹凸映射 shader 可能需要每个顶点的完整切线空间，以及一个纹理坐标组；而另外一个 shader 可能只需要 1 个位置、1 个法线，和 2 个纹理坐标组。如果每个 shader 能指出它需要哪种成分，那么应用可以只产生必需的顶点成分，这就可以节省带宽和内存。

36.5 与 shader 的通信

我们已经了解了需要来回传送于 shader 的数据，以及进行这种通信的工具，现在考查一下允许 shader 支持的应用的设置过程。

36.5.1 场景信息

首先是如何把场景信息传送给 shader。如果没有正确的变换和其他必需的数据，一个 shader 不可能做任何事情。数据将会经过全局变量传给 shader，这是能够把应用的数值提供

给 shader 的惟一可行的通信方法。窍门是在 shader 中找到那个含有所需场景信息的变量。

使用变量名

确定哪个变量含有场景信息的最直接方法，就是简单使用全局变量的名字和类型。举例来说，应用程序可以列举在 shader 里的全局变量，可以找到一个叫做 CameraTransform 的变量，它是一个 4 × 4 矩阵，摄像机变换矩阵就安装在这个变量内。

这个方法要求考虑到所有的 shader，因为变量的取名在应用所用到的所有 shader 中应该是一致的，否则就不可能很好地工作。这个方法对由多人共同维护和工作的大 shader 库是很有用的。

使用语意或评注

更灵活的方法是使用与全局变量有关的语意或注释。例如，在评注里可能是键/数值（key/value），它类似于 string Usage = "CameraTransform"。或者变量有指出用法的专门语意。这种用法的值可以被读出，从而确定在运行期应该被载入变量的值。

这种方法比较灵活，因为它让 shader 作者自己命名变量，他们可以根据喜好使用任何名字，只要能正确地指出用法就行。语意方法是通过 NVIDIA 开发的 FX Composer 使用的（见第 31 章，“FX Composer 的使用”）。下列的示例变量声明使用的语意，表示在运行期接收摄像机变换，它是通过 FX Composer 工作的：

```
float4x4 wvp : WorldViewProjection;
```

优化场景通信

因为求解一个变量的数值，也许需要在变量上多次迭代，还要进行很多字符串操作，所以可能花费很多资源。因此这种求解应该只在创建 shader 的时候运行一次。所有流行的 effect API 都支持句柄，它们可以直接使用变量、technique、pass，甚至评注。但是这些句柄在 shader 之间是不可转移的，因此，不可能创建一个单独句柄，让它在所有的 shader 中引用一个特定的变量。相反地，必须为每个 shader 的每个变量都产生一个句柄。这种存储机制的简单实现，是对每个 shader 产生一张句柄表，表的每个条目（entry）都引用一个特定的变量。那么当场景信息的值改变时，激活的 shader 表可以根据所传送的变量而直接变址，根据所传送的变量找到适当句柄，接着用它向 shader 传达新数据。另外，当激活的 shader 变化时，所有的场景信息必须在新的 shader 中设置。可以通过运行句柄表和给每个场景信息提供数据做到这一点。

36.5.2　材质的参数

类似于场景信息参数，也把材质参数交流到 shader 中去；但是与场景信息不同的是，这些数据通常是从材质资源而来，材质资源是由内容创作者所建立的。因此，要把这些数据从资源映射到 shader 的全局变量上去，必须有一个机制。对于材质参数，使用名字不是一个好方法，因为材质参数在各 shader 中可能变化很大，而时常只与一个单独的 shader 有关。

不使用名字，可以使用注释或语意指出所用的材质参数。那么可以把具有特定的注释或语意组的任何变量，作为一个材质参数来处理，即允许一个值在材质中指定。

还可以为了材质参数的许多其他目的而改写注释。一个字符串注释可以描述帮助参数的工具。或者在没有值存在时，指定默认值。注释中所需要的信息取决于应用打算支持的功能，但是通常最好每个变量至少有一个默认值和一个描述。下面的例子显示了如何用一个语意设置一个材质参数，语意标志它是一个材质参数；而注释提供的信息、创建材质的工具或在运行时间的应用都可以使用。

```
float Specular = 4.0 : MaterialParam <
  float Default = 4.0;
  string Desc = "Specular power of the surface";
  float Min = 2.0;
  float Max = 8.0;>;
```

36.5.3 顶点格式

确定一个特殊的 shader 需要什么顶点数据，存在许多不同的方法。第一个方法是为了明确地指出不同顶点成分而给变量命名。例如，一个预定义的变量名集合，像 Vertex_Normal、Vertex_Position 等。应该检查每个 shader，确定它包含哪个变量；而对每个 shader，适当的顶点信息会包含在 shader 渲染的任何顶点中。

更巧妙的方法是给所需要的顶点元素创建一个包含一个条目的结构。这些变量的名字或者它们相关的语意，以后可以用来确定必需的顶点成分。因为该顶点数据的概念很好地对应于一个结构，而大多数的 shader 已经使用某种结构来表达输入的顶点数据，所以这或许是最直接的方法。需要注意的是不能够直接考查结构，因此，必须定义结构的实例。这里是一个例子：

```
struct SVertex
{
  float3 vPosition : Position;
  float3 vNormal : Normal;
};
SVertex VertexDecl;
```

还有其他一些方法可以指定必需的顶点数据：用一个字符串变量，它包含有限的能被解析的成分目录，或用一个单独变量并使用注释来指定成分。

使用这些方法，我们可以确定需要提供给各个顶点的信息，即 shader 将渲染的任何几何体顶点的信息。如果当时就知道 shader 将会渲染哪个几何体，那么相关数据就可以在预处理时使用，这样我们就能只产生和挑选那些 shader 需要的顶点成分了。这种方法能显然地减少大多数 shader 存储几何体所需要的内存，同时又允许 shader 要求复杂的顶点信息。但是如果在预处理期还不知道 shader，我们就需要使用一种顶点格式，其中包含所有可能的顶点数据。否则就需要运行期支持以产生顶点数据，但是这两者都不是最佳的解决办法。

36.5.4 场景

场景的概念直接地对应于一种 technique。对于应用程序所要求的每个场景，我们要把对应的 technique 放入相关的 shader。让我们使用一个应用程序的原始场景的例子，根据所选择

的物体，应用需要各个 shader 以不同的风格渲染。在 shader 里可能是两种不同的技术：被选的和不被选的。当要渲染某物体时，基于物体的状态激活相应的 technique，然后使用该 technique 渲染物体。这允许每个 shader 针对应用所需要的场景，实现一种适当的渲染风格。另外，应用程序也使 shader 可以改变其执行功能的方式，以适应具体情况的需要。为了使这一点工作，在 shader 和渲染之间必须存在一份契约。它的作用更像场景变量，渲染器在变量里调用特殊情况下的场景，以进行某种方式的渲染；而作为回报的是，shader 在每种情况下都能很好地工作。但是在 shader 和渲染之间的这份契约取决于每个应用程序，这使得场景约束法对每个应用都是特殊的。

36.5.5　就场景对 technique 和 pass 的比较

可能产生这样一个观点：应该用 pass 代替 technique 在一个 shader 里表现场景。理由是什么呢？就整体而言技术是有效的，它们避免了这样一种情况：当一种场景可能正确时，另一种场景却无效。这可能导致物体在一种场景中可见，而在另一种中却不可见。

使用 pass 作为场景，略微增加了工作量而且降低了清晰度，但是它通过使用一个对 pass 的命名约定，能够很容易地实现。命名约定可以指出场景的名字和遍数。下列的代码是一个 technique 的例子，它持有两个场景，以及被选的场景所需要的两个通道：

```
technique SomeShader
{
  pass Selected_0 { . . . }
  pass Selected_1 { . . . }
  pass Unselected_0 { . . . }
}
```

然后，当应用程序装载一个 shader 的时候，对当前设备有效的 technique 可以被挑出来，并分析 pass。分析决定需要渲染哪个 pass、以什么次序渲染、什么时候使用某个特定的场景等。这个方式更容易在相同的 shader 文件里添加备份。当支持的硬件很多时，我们推荐这种方式。

36.6　effect 文件格式的扩展

使用本章中的方法，我们可以把完整的 shader 系统整合到应用中去，并创建一个非常有力和灵活的渲染。我们还可以给应用和 shader 再添加几个扩展，使 shader 的开发和管理更加容易。

36.6.1　对预处理程序的支持

应用程序支持对 shader 提供预处理程序是非常重要的。所有流行的 effect API 都支持预处理程序，如果应用程序含有某种 include 命令的挂钩。这些挂钩让应用程序实现自身的文件系统，而增加对挂钩的支持通常也很简单。对当前的工作目录进行定义，要相对于当前正在编译的文件。一旦把对预处理程序的支持加入应用，就能很快创建一个完全的标准定义和函数的实用库。这有助于减小独立 shader 的大小和复杂性。例如我们可以创建一个 shader 文件，

让它包含蒙皮模型的功能函数。可以在任何需要蒙皮的 shader 中包含这个文件，这就减小了每个 shader 的大小，也减少了必须在 shader 间复制的代码量。

36.6.2 对 shader 变化的支持

经过小心地设置，预处理程序允许同一个 shader 用于刚性和蒙皮两种物体。蒙皮网格和刚性网格之间惟一的差别是它们获得各种几何数据的方式不同，包括顶点的位置和切线空间等。因此，对于刚性网格，可以直接使用在顶点中所指定的几何数据；但是对于蒙皮网格，在使用顶点数据之前，一定要经过一个蒙皮过程。于是，可以用这样的条件编译代码，来访问所有的几何信息：要么执行顶点数据的蒙皮，要么简单地返回所存储的顶点数据。

骨头加权数据（可能还有骨头索引数据）也需要同样处理。可以使用一个宏添加上这种数据：对于刚体 shader，宏不扩展任何东西，而在骨骼 shader 中声明顶点数据时，宏扩展为所需的适当定义。只需要开发一个单个的 shader 就可以做到这一点。用一组不同的预处理程序定义来编译这个 shader，可以使它要么支持蒙皮网格，要么支持刚性网格，如列表 36-2 所示。

列表 36-2 使用预处理程序支持骨骼 shader 的变化

```
#ifdef SKELETAL
  //Used in the vertex struct to indicate the weights of the vertex
  #define DECLARE_SKELETAL_WEIGHTS float4 Weights : BLENDWEIGHT
  //Used anywhere that point data is accessed from a vertex
  #define SKIN_POINT(Point, Vert) SkinPoint(Point, Vert.Weights)
  //Used anywhere that vector data is accessed from a vertex
  #define SKIN_VECTOR(Vector, Vert) SkinVector(Vector, Vert.Weights)
#else
  //We aren't supporting skeletal animation
  #define DECLARE_SKELETAL_WEIGHTS
  #define SKIN_POINT(Point, Vert) (Point)
  #define SKIN_VECTOR(Vector, Vert) (Vector)
#endif
```

在渲染多个光源时，可以采用类似的方法。一个宏可以定义所支持的光源数目，而 shader 可以简单地迭代到宏所指定的数目。这样就可以从一个单个 shader 源文件得到许多光源数量的变化。

36.6.3 shader 继承的添加

另外一个有用的扩展是继承的概念。思路是，如果一个 shader 不是很有效，或需要使用一个较轻量的 shader，可以用一个备用 shader 替换原先那个 shader。在全局级别上，可以用一个字符串参数把备用 shader 指定到将要撤销的 shader 文件上。也可以在各 technique 级别上，使用适当的注释指定一个备用 shader。但是如果一个 shader 使用继承，除了它的数据之外，决定材质参数和顶点声明时，还必须考虑到所有派生出那个 shader 的集合，因为事先并不知道什么时候需要使用备用的 shader。

继承的实现通常很简单，而且它出人意料地改善了硬件的可伸缩性及性能。由于将来市场上不可避免地要出现许多性能各异的硬件，这一点会越来越重要。

36.7　小结

我们在支持 shader 的通信层的开发中加入了必要的工作。如果遵循这个指导，工作就会很简单，并且会产生一个意想不到的灵活又有力的渲染器。

shader 是实时渲染的未来。现在把 shader 整合进渲染传递途径，就是让应用向有力而灵活的下一步迈进。这样不仅使得应用更加富有竞争性，而且简化了渲染器的开发，使其能适应多种多样的效果。

36.8　参考文献

Microsoft Corporation. 2003. “Effect Reference.” MSDN Online.

http://msdn.microsoft.com/library/default.asp?url=/library/en-us/directx9_c/directx/graphics/reference/EffectFileReference/EffectFileReference.asp

NVIDIA Corporation. 2003. “CgFX Overview.” Available online at **http://developer.nvidia.com/object/cg_users_manual.html**

第 6 部分

超越三角形

简 介

图形处理器不再仅仅用于处理图形。可完全编程和支持浮点数学运算的图形处理器（GPU）的出现，已经使其能够进行许多更通用类型的计算。实际上，现在的研究者对 GPU 的使用已非当初设计者所能想象的。

这篇将讲述 GPU 的一些非传统应用，包括从物理模拟到立体图像渲染等内容。虽然某些应用在今天的硬件上可能没有广泛使用，但是它们提供的看法对图形处理器未来的应用可能会有启发。

第 37 章是 Ian Buck 和 Tim Purcell 写的“**用于 GPU 计算的工具箱**”，介绍在 GPU 上进行通用计算，而且描述了一些程序基元，用来在 GPU 上实现通用的算法。这章还将介绍如何在 GPU 上编写排序和搜寻数组的程序。

第 38 章是 **MarkJ. Harris** 的“**在 GPU 上的快速流体动力学模拟**”，详述如何实现流体的精确物理模拟，并且完全在 GPU 上实现。物理建模本身是并行的问题，与 GPU 处理并行的能力相符。随着图形硬件变得更快速、更并行和更灵活，我们可以看到越来越多类型的物理模拟将移植到 GPU 上去。因为许多物理模拟的最终输出是图形，所以把计算与显示拉近很有意义。

这篇也包含关于体渲染的两章。1991 年 Jim Kajiya 发出著名的预言：“在 10 年内，所有渲染都将是体渲染”。虽然这个预言没有成真，但是要可视化那些不能以多边形来描画的数据，体渲染确实还是最有用的。随着纹理内存更加便宜和着色能力不断增加，体渲染将变得更普遍。**第 39 章“体渲染技术”是 Milan Ikits、Joe Kniss、Aaron Lefohn 和 Charles Hansen** 所提供的关于体渲染的极好概述。**第 40 章**是 **Thilaka Sumanaweera** 的“**用于三维超声波可视化的实时着色**”，讲述了使用体渲染来渲染超声波数据时出现的一些实际问题。

第 41 章是 Fabio Policarpo 的“实时立体图”，解释了如何使用 GPU 产生单个图像的随机点的立体图（SIRDs）。这些图像很有趣，因为它们证明人类视觉系统能纯粹从立体图像的匹配特性推测出深度，甚至在缺乏一切其他深度线索（如纹理和透视）时也能做到这一点。Bela Julesz 在 1960 年发明随机点立体图的时候，也许从未想到过，由此可能产生出实时动态 3D 场景的立体图。

最后，在**第 42 章“变形”**中，**Eugene d'Eon** 通过把几个技术包含到“变形”中去，产生了动画的概念。使用这个定义，他开发了一个标准方法，使用 Jacobian 矩阵来得到顶点法线，因此不再需要使用有限微分法求梯度矢量。

通过这篇可以感到，GPU 可以应用于视频游戏的正常范围之外，我们也希望它将激发读者的灵感，找到自己使用硬件的方法并成为行家里手。

Simon Green, NVIDIA

第 37 章 用于 GPU 计算的工具箱

Ian Buck
Stanford University
Tim Purcell
Stanford University

从传统上来说，GPU 是一个渲染的协处理器，或如名字所示，是 PC 主机的图形处理器。本书的许多章节都谈到如何用 GPU 真正快速地渲染极棒的效果。但是谁说 GPU 只能进行渲染呢？有关的研究清楚地表明，与图形无关的计算也能在 GPU 上运行，已经有一打以上的这方面论文在 SIGGRAPH 和 Graphics Hardware 会议中发表。这章讨论了在 GPU 上实现一般计算的基本方法，并且举例说明如何实现这些基本的算法，以满足许多基于 GPU 的应用的需要。

37.1 用 GPU 进行计算

为什么想要使用 GPU 做一般的计算？第一个原因是性能。现代的 GPU 运行浮点计算比今天的 CPU 快得多。举例来说，一个 3GHz 的 Pentium 4 理论上大约能进行每秒 60 亿（6 gigaflops）次浮点乘法运算。然而，一个在 NVIDIA GeForce FX 5900 Ultra 上运行的片元程序可达的速率为每秒 2 百亿次乘法运算[1]，大约相当于一个 10GHz 的 Pentium 4。这个计算的性能结合增加的内存带宽，GPU 峰值可达 25.3GB/s，而 Pentium 4 的峰值仅为 5.96GB/s，GPU 和 CPU 孰优孰劣就很清楚了[2]。

第二个理由是负载的平衡。如果 CPU 限制了应用程序的性能，使得 GPU 有空闲周期，可以把工作卸给 GPU，让我们的应用程序得以全速运转。

最后，展望未来，GPU 性能的增加将比 CPU 快得多。到 2003 年 6 月，两年来 Pentium 4 的性能从 4 gigaflops 增加到 6.3 gigaflops，或者说每年增加 0.65 gigaflops。然而，随着 GeForce FX 5900 Ultra 的上市，仅在 6 个月内 NVIDIA GPU 就从 8 gigaflops 增加到 20 gigaflops，相当于每年增加

1. 每秒 2 百亿的乘法是运行一个片元 shader 所观察到的结果，片元 shader 由单独在 262 144 个像素上进行 100 次 1000 条 MULR R0, R0, R0 指令所组成，并累计结果时间。
2. GeForce FX 5900 Ultra 内存系统：
   256-bit DDR @ 425MHz = 425 × 2 × 32B = 25.3GB/s
   Pentium 4 前端总线：
   64-bit QDR @ 200MHz = 200 × 4 × 8B = 5.96GB/s
   $1GB = 2^{30}B$；$1MHz = 10^6Hz$

24 gigaflops。换言之，在性能方面，GPU 已经超过 CPU，而且在未来会继续保持超过的势头。

37.1.1　编程模型

众所周知，在编程方面，GPU 和 CPU 之间有很大的不同。我们不能仅仅改变几个编译器标记，然后就重新编译。原因是编程模型略有不同。

主要的区别在于 GPU 不是一个串行处理器，而是一个流式处理器。von Neumann 体系结构的串行处理器（von Neumann 1945）连续地运行指令，更新它访问过的内存。而一个流式处理器的工作方式略有不同，它在一组输入记录（片元）上运行一个函数（例如一个片元程序），并行地产生一组输出记录（着色的像素）。流式处理器通常要提到这样一个称为核心（kernel）的函数，以及作为流的一组记录。数据流入处理器，经过一个核心函数的运算，输出到内存，如图 37-1 所示。进入处理器的每个元素都是独立地处理，在元素之间没有任何相关性。这允许体系结构并行地运行程序，而不需要程序员了解并行单元或任何并行结构。

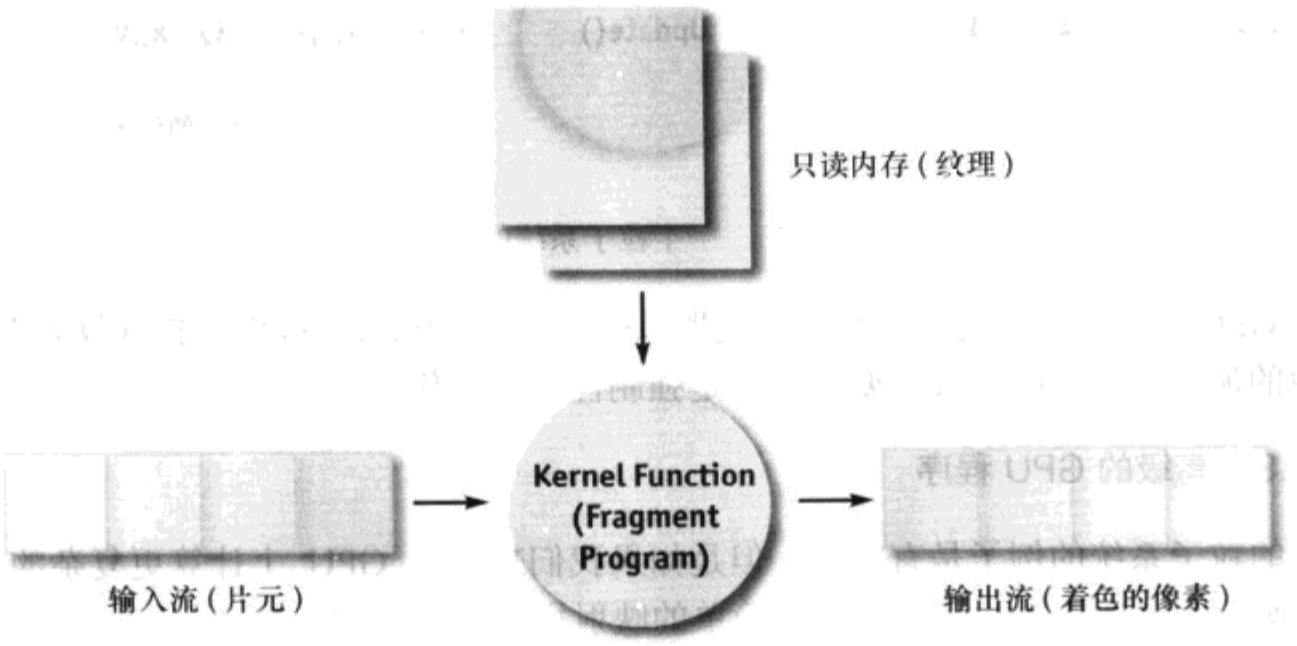


图 37-1　在 GPU 上的一个流式程序的模型

然而，正如我们在这章中所描述的，这些函数和记录不一定必须是像素和 shader，它们可以是任何数据，例如网格点和物理方程式。举例来说，我们想要实现一个简单的粒子模拟，并让 GPU 执行大部分的物理操作。我们使用浮点纹理和像素缓冲（pbuffers）存储粒子位置、速度和方位。一个片元程序执行必要的计算得到单个粒子的新位置。为了计算所有粒子的新位置，我们简单地渲染一个大的方块，在方块里的片元数目等于粒子数目，如图 37-2 所示。方块的纹理坐标指示片元程序，它需要处理哪个被存储的粒子。被存储在 pbuffer 里的结果包含新位置的值。

我们给出一个粒子系统的例子，说明在通用计算方面 GPU 的一个明显而有用的应用。这个更新粒子位置的操作，可以推广为把一个函数应用到一批数据的过程。这种操作，在功能程序设计语言中称为一个映射（map），可以有多种应用。Simon Green（2003）举例说明了如何使用类似粒子模拟的技术在 GPU 上模拟布料。

37.1.2 并行编程

有一些限制可能反映到 GPU 上。最重要的是函数必须是并行的。因为片元程序不能够在不同片元之间共享写入的内存，在每个粒子、网格点，或无论任何东西上执行的计算必须是彼此独立的。正如在着色时我们不能够访问相邻片元所计算的结果，应用于流数据的函数也必须遵循相同的规则。幸运的是，正像粒子系统的例子一样，大多数用于图形的物理模拟本来就是并行的。每个粒子不需要知道它邻居的新计算的位置，就可以更新它本身。

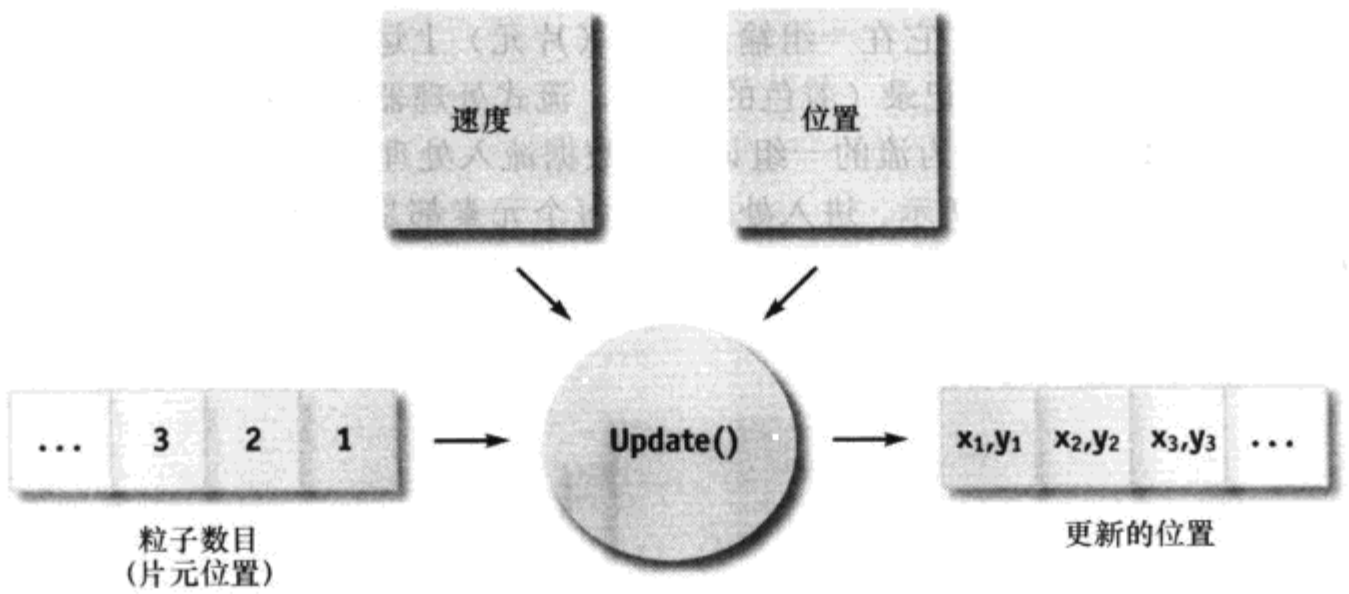


图 37-2 一个粒子系统的例子

使用 GPU 作通用计算还会引起其他问题，这一点会在稍后谈到，但是把计算映射到 GPU，或大多数的流式处理器时，最重要的考虑是强制性的并行化。

37.1.3 高级的 GPU 程序

简单的粒子系统的例子是有趣的，但是如果我们想要在 GPU 上计算更复杂的东西，应该怎么办呢？不是所有的应用都可以用简单的映射操作来实现的。

这章的其余部分描述了几个基本工具，在它的并行程序环境中，可以使用它在 GPU 上执行更高级的算法。“约减”提供一个方法，从一组数据中计算单个值。“排序和搜索”提供机制来建立和使用 GPU 上的数据结构。使用这些操作，我们可以在 GPU 上执行许多一般的计算操作，这些曾经是由 CPU 计算的。

37.2 约减

一般而言，GPU shader 是把一个函数应用到一组数据产生一组结果，例如应用一个片元程序来着色像素，或把一个顶点 shader 应用到顶点。在这两种情况下，我们代入一组值而得到一组结果。但是如果我们想从一组值只得到一个结果，该怎么办呢？举例来说，考虑 tone mapping 的问题。我们使用浮点缓冲 pbuffer 渲染一个高动态范围场景，并需要把场景显示在帧缓冲内，这就限制了精度。一个直接的方法是把最大的浮点值映射为白色，而把其余的值随之缩放。

怎样才能得到最大值呢？可以回读整个场景，并让 CPU 进行计算。但是如果我们不得不对每一帧都这样做，回读会严重地减少帧速率。一个更好的解决办法，是使用 GPU 从场景的所有像素中计算最大值。换言之，使用 GPU 把那组浮点像素约减为一个单个的结果，即最大值。

大体上，约减是从一组数据计算单个结果的任何运算。其中包括最小/最大、平均、加和、乘积或其他任何用户定义的函数。这一节解释如何使用 GPU 在几遍内计算约减。

37.2.1　并行的约减

考虑我们的明暗映射的例子。流行的图形硬件没有内置的特性，可以在浮点缓冲 pbuffer 提供最大浮点值。但是我们可以使用可编程的片元流水线做一个部分的约减，这能够减少需要做约减的像素数目。

列表 37-1 的片元程序输出 4 个浮点值中的最大值。

列表 37-1　部分约减的执行

```
float main(float2 texcoord : TEXCOORD0,
  uniform samplerRECT img)  : COLOR
{
  float a, b, c, d;
  a = f1texRECT(img, texcoord);
  b = f1texRECT(img, texcoord + float2(0, 1));
  c = f1texRECT(img, texcoord + float2(1, 0));
  d = f1texRECT(img, texcoord + float2(1, 1));
  return max(max(a, b), max(c, d));
}
```

使用这个程序，我们可以对 4 个像素做约减，而保留惟一的大值像素。这里是具体做法：（1）把图像缓冲与对应于变量 img 的纹理单元捆绑在一起，同时渲染一个方块，其大小为图像的 1/4（图像的每一边除以 2）；（2）设置纹理的坐标，使方块的每个片元对应于一个纹理中不同的 2 × 2 长方形。所渲染的方块是原始图像尺寸的 1/4，只保留其中最大的值。对新的图像重复这个算法，直到我们得到一个单独的浮点值，可以把它回读到 CPU 去，如图 37-3 所示。

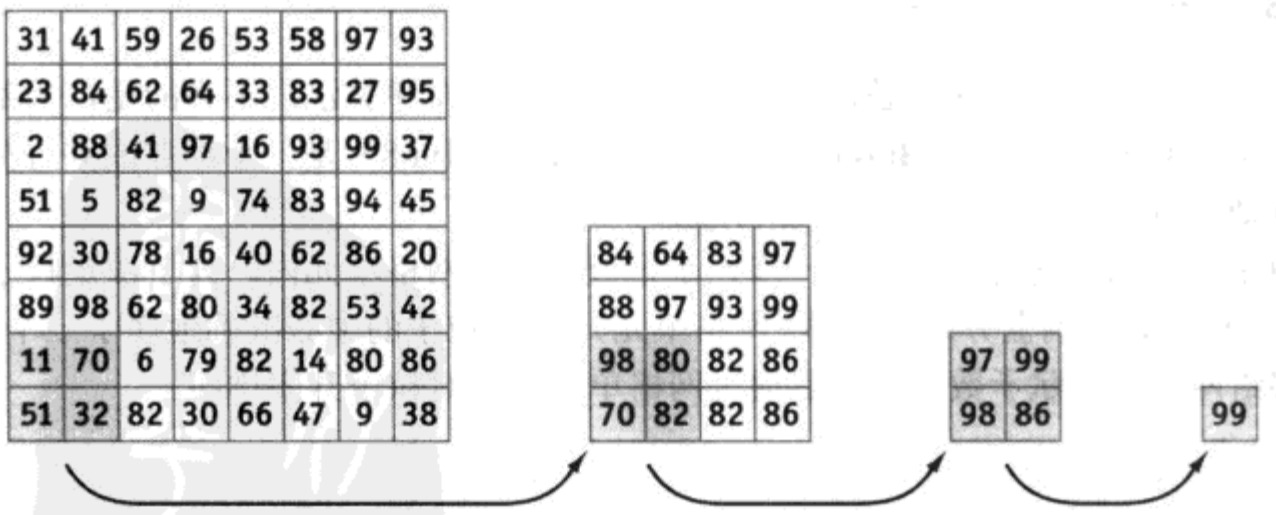


图 37-3　以多遍执行最大值约减

性能

执行 1024 × 1024 图像的最大值约减只需要 10 遍，每遍是前一个遍大小的 1/4。一般来说，为了运行 n^2 个元素的约减需要 $O(\log n)$遍。我们不需要为回读走完到达一个像素的所有的路。在某些情况下，回读一个小方块，而让 CPU 做后面的约减可能是比较快的。我们也可以增加在片元程序里执行的约减数目，在每个遍做 1 个以上的约减，而减少所需要的遍总数。

37.2.2 有关约减的注意事项

虽然我们的例子是关于 tone mapping 的，但是基本的算法能用于任何需要做约减的地方，只要没有超出硬件的限制。因为需要把约减写成一个片元程序，因此显然要受到指令数目和输出大小的限制。如果 GPU 只支持一个 4 个组分的浮点输出（即片元颜色），我们就不可能为一个 4 × 4 的矩阵乘法实现约减行动（至少，是不容易的），它需要来自 shader 的 4 个 4 组分的浮点输出。同时，如果要做约减的数据值不是 2 的幂，情形就会更复杂。

37.3 排序和搜索

假设我们想要为 GPU 的粒子系统加上碰撞检测。编写片元程序实现粒子与粒子的交互作用是比较容易的。但是随着越来越多的粒子增加到系统中来，粒子之间的交互作用很快就会形成巨大的消耗。遍的总数与系统中的粒子的数目的平方成正比。更糟的是远处的粒子对给定的粒子影响很小或简直就没有影响。因为这些粒子并不发生碰撞，计算它们之间的交互作用所花费的时间，是一种浪费。

一个较好的主意是只计算那些相邻粒子之间的互动，因为它们更可能发生碰撞。一个这样做的方法是把粒子组织进一种空间数据结构中去，结构类似于均匀网格。相邻的粒子放入相同的网格单元或网格单元组。这个方法可以把粒子碰撞测试的数目从 n^2 减少到一个常数，它的大小取决于网格单元内的粒子数目和所要搜寻的网格单元数目。

这节描述两个算法：排序和搜索，它们是构造均匀网格数据结构所需要的。我们在一个小的多遍 shader 上实现排序，而在一个单独渲染通道上实现搜索。

37.3.1 Bitonic 归并排序

排序是一个经典的算法问题，已经具有许多的不同排序算法。不幸的是，大部分排序算法都不适于在 GPU 上实现。Bitonic 归并排序（Batcher 1968）是一个经典的并行排序算法，它能很好地适合于 GPU 所限制的程序环境。

为粒子系统构建均匀网格的第一步，是把数据分到网格单元内。为了决定粒子在网格单元的位置，把场景包围盒分成立方体（网格单元）。每个粒子的中心准确地属于一个网格单元。我们能够给网格单元赋予数字，然后通过网格单元数字对粒子升序排序。

算法

Bitonic 归并排序可以使得 n 个数组的处理器在 $O(\log^2 n)$个步骤中排列出 n 个元素的顺序：

log *n* 步骤中每一步最多有 log *n* 个阶段。每个阶段并行地运行 *n* 个比较和交换。

图 37-4 中显示的是对 8 个元素的数据组运行 Bitonic 归并排序。箭头指出要进行比较的两个元素（位于每支箭的头部和尾部）并交换它们的方向，目的是结束时要把最小的元素放在尾部。举例来说，第 1 步骤的第 1 阶段，我们比较 4 和 8，并交换它们的位置，因为需要把较小的元素（4）放在箭尾。

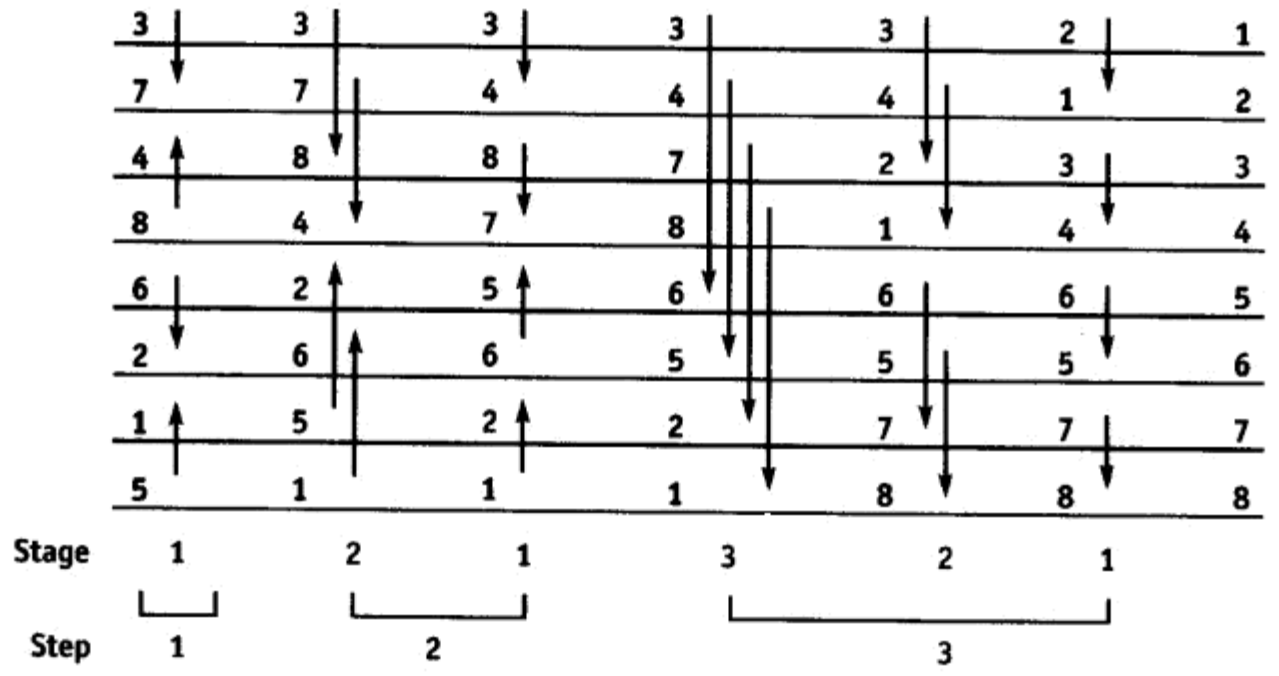


图 37-4　8 个元素的 Bitonic 归并排序

Bitonic 归并排序是这样工作的：先是对两个已经排序的序列进行重复的分组，由此制成一个 Bitonic 序列，然后对该序列排序，形成一个单个的排序好的列表。要记住 Bitonic 序列或是单调地增加然后减少，或是先减少然后增加。以图 37-4 为例，我们能看到，在第 1 个步骤结束的时候，开始的序列已经被排序为 4 个小组：（3, 7）、（8, 4）、（2, 6）和（5, 1）。这 4 个小组实际上组成两个 Bitonic 序列：（3, 7, 8, 4）和（2, 6, 5, 1）。在第 2 步骤中排列这两个序列。而两个排序过的序列形成一个 Bitonic 序列（3, 4, 7, 8, 6, 5, 2, 1）。这个序列在第 3 步骤中被排序，得到最后的排列好的列表。

GPU 的实现

Bitonic 归并排序（以及稍后描述的二分搜索）需要一个辅助函数 convertIdto2d，可以把 1D 的数组地址改变为 2D 纹理坐标。它的实现是简单的，如列表 37-2 所示。

列表 37-2　Bitonic 归并排序的辅助函数

```
float2 convert1dto2d(float coord1d, float width)
{
  float2 coord2d;
  coord2d.y = coord1d/width;
  coord2d.x = floor(frac(coord2d.y) * width);
  coord2d.y = floor(coord2d.y);
  return coord2d;
}
```

这个排序需要 3 个步骤，每个步骤中最多有 3 个阶段。

Bitonic 归并排序由一个多遍 shader 实现。列表 37-3 显示了 Bitonic 归并排序的 Cg 代码。一个优化的实现能在书的 CD 和网站上找到。

我们正在排序的数据存储在纹理内存中。每个渲染遍对应一个并行的比较阶段。片元程序根据排序的当前步骤和阶段，计算哪两个元素需要比较。又根据排序的当前的阶段和步骤，写出比较的最小或最大值。把新产生的纹理用作下一个比较阶段的输入。n 个元素的排序需要$(\log n \times (\log n + 1))/2$ 个渲染遍。

性能

排序一百万个以上的元素（存储在 1024 × 1024 的纹理中）需要 210 个渲染遍。甚至今天最快速的图形卡也很难实时地在一个兆像素（megapixel）区域上渲染 210 遍。但是排序少量的元素仅需要非常小的区域上的几遍。排序 4096 个元素的数据组只需要 20 毫秒。基于 CPU 的好的排序程序有运行时间渐近线 $O(n \log n)$，这意味着它们时常能够排序得比 GPU 更快。然而，如果被排序的数据需要放在 GPU 内存中，与主机来回传递的时间就抵消了这个优越性。

列表 37-3 Bitonic 归并排序

```
float BitonicSort(float2 elem2d : WPOS,
                  uniform float offset,   // offset = 2^(stage - 1)
                  uniform float pbufwidth,
                  uniform float stageno, // stageno = 2^stage
                  uniform float stepno,  // stepno = 2^step
                  uniform samplerRECT sortedlist) : COLOR
{
  elem2d = floor(elem2d);
  float elem1d = elem2d.y * pbufwidth + elem2d.x;

  half csign = (fmod(elem1d, stageno) < offset) ? 1 : -1;
  half cdir = (fmod(floor(elem1d/stepno), 2) == 0) ? 1 : -1;

  float adr1d = csign * offset + elem1d;
  float2 adr2d = convert1dto2d(adr1d, pbufwidth);

  float val0 = f1texRECT(sortedlist, elem2d);
  float val1 = f1texRECT(sortedlist, adr2d);

  float cmin = (val0 < val1) ? val0 : val1;
  float cmax = (val0 > val1) ? val0 : val1;

  return (csign == cdir) ? cmin : cmax;
}
```

37.3.2 二分搜索

搜索是另一个经典的算法。像排序一样，在 GPU 所限制的程序环境中，大部分搜索的算法也不起作用。然而，二分搜索能很好地映射到 GPU 上，它也是一种最常用的搜索算法。

有了一个有序的粒子列表，我们就可以完成均匀网格的数据结构。虽然有序的列表把粒子根据网格单元进行了排序，但是没有告诉一个粒子的子表在哪里结束，下一个列表在哪里

开始。我们在排序好的粒子列表上并行地进行几个连续的二分搜索，建立均匀的网格。每个网格单元在排序好的列表上开始对本身的搜索。对每个网格单元的搜索返回到粒子列表的开始位置。

算法

二分搜索在元素的有序列表上进行。算法寻找列表的中间元素，然后决定哪一半列表要进一步搜索。对后面的每个子表都这样检测，直到子表的长度为 1 才停止。这时，如果在最后位置上的值不等于我们的搜索值，就需要一个最后的“校正”比较，来决定是否增加位置。这是对一个给定元素的第一种情况的人工搜索，如图 37-5 所说明。算法要求 $O(\log n)$次比较。

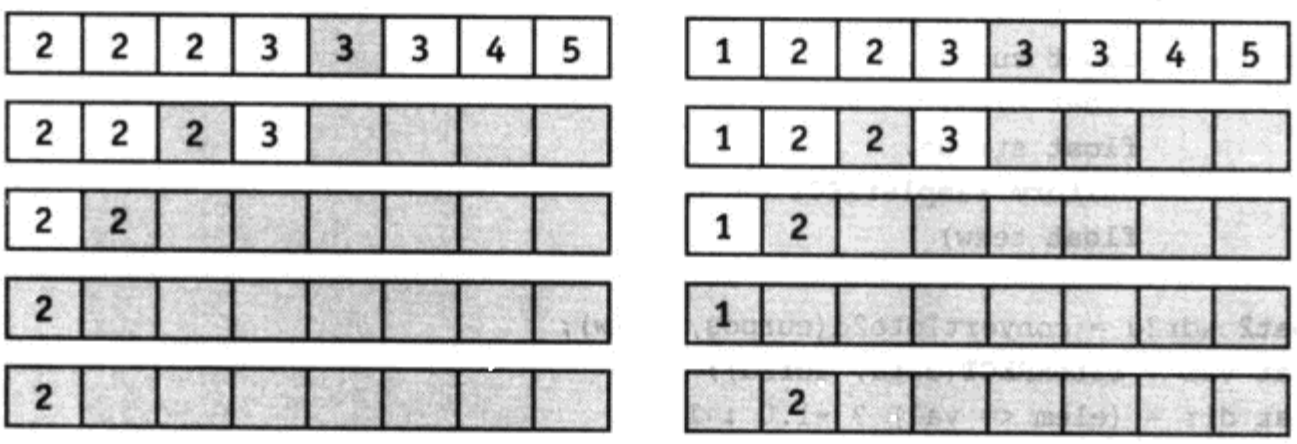


图 37-5　在 8 个元素上对 2 值的二分搜索

可以看出，在图的右半边需要一个最后的校正步骤，因为那里数组中的第一个元素是 1 而不是 2

GPU 的实现

二分搜索很容易写成一个单遍片元程序。把搜索的元素列表存储在纹理内存中。在片元程序的一次运行中，一个单片元执行完整的二分搜索。片元程序是一个简单的开循环，它在列表中准确地迭代 $\log n + 1$ 次。程序归结为一个序列的相关纹理查询。最终被返回的值或是被搜索的单元，或是下一个单元的起始位置，见列表 37-4。

列表 37-4　二分搜索

```
float BinarySearch(float2 elem2d : WPOS,
                   uniform float stride,
                   uniform float pbufwidth,
                   uniform float sortbufwidth,
                   uniform samplerRECT sortlist) : COLOR
{
  elem2d = floor(elem2d);
  float elem1d = elem2d.y * pbufwidth + elem2d.x;
  float curpos = stride;

  //循环 logn - 1 次搜索遍
  for(int i = 0; i < logn - 1; i++){
    stride = floor(stride * 0.5);
    curpos = Search(curpos, elem1d, stride, sortlist, sortbufwidth);
  }
```

```
  //第 logn 个遍
  curpos = Search(curpos, elem1d, 1.0, sortlist, sortbufwidth);

  //清除遍
  curpos = SearchFinal(curpos, elem1d, 1.0, sortlist, sortbufwidth);

  return curpos;
}
```

在列表 37-5 所显示的搜索程序，只是进行相关纹理的抽取，然后计算和返回下面要搜索的位置。

列表 37-5 Search 例程

```
float Search(float curpos,
             float elem,
             float stride,
             uniform samplerRECT data,
             float texw)
{
  float2 adr2d = convert1dto2d(curpos, texw);
  float val = f1texRECT(data, adr2d);
  float dir = (elem <= val) ? -1.0 : 1.0;
  return dir * stride + curpos;
}
```

记住在完成搜索之后，为了决定是否增加最后的位置，需要一个最后的“校正”比较。SearchFinal 的代码和 Search 的代码基本相同，只是在以下一行存在差别：

```
float dir = (elem <= val) ? 0.0 : 1.0;
```

性能

二分搜索非常有效。与约减和排序不同，搜索时常可以在一个单渲染遍中执行。在超过一百万个元素（存储在一个 1024 × 1024 纹理中）中搜索一个元素只需要不到 1 毫秒。构建一个网格数据结构和并行地执行几个搜索只消耗几毫秒。

37.4 挑战

使用当前的 GPU 进行计算存在一些挑战，即，有限的输出和缓慢的回读。

37.4.1 有限的输出

片元程序所允许的输出数目是一个难题。今天的 GPU 能够支持的输出数目很不一致，有的只支持一个 4 组分的浮点输出，而最新的硬件可以支持 4 个或更多的这种输出。有限的输出数目会限制我们在一遍中所能计算的东西。举例来说，如果我们正在执行光线跟踪，每条入射光线都可能产生多条出射光线（反射、传输和阴影），我们很容易受到被 GPU 支持的输出数目的限制。随着 GPU 技术的进展，这个缺点也应该随之减弱。

37.4.2　缓慢的回读

回读可能是在 GPU 上计算的最大限制之一。在计算新的粒子位置之后，我们的应用程序有可能使用 CPU 做一些挑战性的操作，例如进行场景物体的碰撞检测。为了这样做，我们也许可以使用适当的 API 指令从图形卡内存把数值读回来。然而，这需要在 AGP 连接上输送所有的数据，虽然现代的 GPU 支持快速的 8 × AGP 传输，但是回读的路径只有 1 × AGP[3]。这种缓慢的传输抵消了由于把物理计算卸到 GPU 应该获得的任何运行优越性。为了避免这种回读造成的惩罚，我们希望把尽可能多的计算都放在 GPU 上。即将上市的 PCI-Express 兼容主板应该能减轻这个限制。（译者注：实际情况表明，即使在 PCI-Express 主板上，回读慢的问题仍然存在，因为 GPU 本质上还没有完全支持高速回读。）

37.4.3　GPU 和 CPU 的比较

正如我们在本章中见到的，许多传统上在 CPU 上的优秀算法，不能很好地映射为 GPU 的流式程序模型。除了在这节中提到的一些限制之外，流式程序模型强制的并行性使我们要重新考虑解决问题的办法。在一些情况下，不选择 GPU 作为执行算法的平台。像调试器和剖析器一类的程序支持工具更适用于 CPU。然而，我们期待继续为 GPU 写出许多新的好的算法，学会把算法映射到流式程序模型上。本章“参考文献”一节包括一些很好的内容，可供那些从事 GPU 的人们参考。这些程序包括光线跟踪、光子映射、流体流动的解决方法，布料模拟和全局照明计算等。同时，我们期待 GPU 继续进展，将来能够更充分地支持通用计算——包括通常在 CPU 编程上可以找到的程序员支持和工具。

37.5　小结

这章举例说明了 GPU 可以做相当多的工作，不仅仅是渲染着色的像素。我们已经显示了 4 种有用的操作：映射、约减、排序和搜索，而且用例子说明如何使用它们。随着 GPU 性能的继续高速发展，使用它做通用的计算有可能变得轻而易举。

37.6　参考文献

注意：www.gpgpu.org 已经链接了最新的结果，它是 GPU 上通用计算的一个非常有用的信息库。

Batcher, Kenneth E. 1968. “Sorting Networks and Their Applications.” *Proceedings of AFIPS Spring Joint Computer Conference* 32, pp. 307～314.

Bolz, Jeff, lan Farmer, Eitan Grinspun, and Peter Schroder. 2003. “Sparse Matrix Solvers on the GPU: Conjugate Gradients and Multigrid.” *ACM Transactions on Graphics* 22(3), pp. 917～924.

Carr, Nathan A., Jesse D. Hall, and John C. Hart. 2002.“ The Ray Engine.” *Proceedings of the*

3. 回读限制起因于原来如何定义 AGP 的规格，而且受所有 AGP2.0 主板的限制，而不是受图形卡限制。

SIGGRAPH/Eurographics Workshop on Graphics Hardware 2002, pp.37～46.

Carr, Nathan A., Jesse D. Hall, and John C. Hart. 2003. "GPU Algorithms for Radiosity and Subsurface Scattering." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003,* pp. 51～59.

Goodnight, Nolan, Rui Wang, Cliff Woolley, and Greg Humphreys. 2003. "Interactive Time-Dependent Tone Mapping Using Programmable Graphics Hardware." *Eurographics Symposium on Rendering: 14th Eurographics Workshop on Rendering,* pp.26～37.

Goodnight, Nolan, Cliff Woolley, Gregory Lewin, David Luebke, and Greg Humphreys. 2003. "A Multigrid Solver for Boundary Value Problems Using Programmable Graphics Hardware." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003,* pp. 102～111.

Green, Simon. 2003. "Stupid OpenGL Shader Tricks." Presentation at Game Developers Conference 2003. Available online at **http://developer.nvidia.com/docs/IO/8230/GDC2003-OpenGLShaderTricks.pdf**

Harris, Mark J., Greg Coombe, Thorsten Scheuermann, and Anselmo Lastra. 2002. "Physically-based Visual Simulation on Graphics Hardware." *Proceedings of the SIGGRAPH/ Eurographics Workshop on Graphics Hardware 2002,* pp. 109～118.

Harris, Mark J., William Baxter III, Thorsten Scheuermann, and Anselrno Lastra. 2003. "Simulation of Cloud Dynamics on Graphics Hardware." *Proceedings of the SIGGRAPH/ Eurographics Workshop on Graphics Hardware 2003,* pp. 92～101.

Hillesland, Karl E., Sergey Molinov, and Radek Grzeszczuk. "Nonlinear Optimization Framework for Image-Based Modeling on Programmable Graphics Hardware." *ACM Transactions on Graphics* 22(3), pp. 925～934.

Kruger, Jens, and Rudiger Westermann. 2003. "Linear Algebra Operators for GPU Implementation of Numerical Algorithms." *ACM Transactions on Graphics* 22(3), pp. 908～916.

Ma, Vincent C. H., and Michael D. McCool. 2002. "Low Latency Photon Mapping Using Block Hashing." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2002,* pp. 89～99.

Moreland, Kenneth, and Edward Angel. 2003. "The FFT on a GPU." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2003,* pp. 112～120.

Purcell, Timothy J., lan Buck, William R. Mark, and Pat Hanrahan. 2002. "Ray Tracing on Programmable Graphics Hardware." *ACM Transactions on Graphics* 21 (3), pp. 703～712.

Purcell, Timothy J., Craig Donner, Mike Cammarano, Henrik Wann Jensen, and Pat Hanrahan. 2003. "Photon Mapping on Programmable Graphics Hardware." *Proceedings of the SIGGRAPH/ Eurographics Workshop on Graphics Hardware 2003,* pp.41～50.

Sen, Pradeep, Michael Cammarano, and Pat Hanrahan. 2003. "Shadow Silhouette *Maps*." *ACM Transactions on Graphics* 22(3), pp. 521～526.

von Neumann, John. 1945. "First Draft of a Report on the EDVAC." Moore School of Electrical Engineering, University of Pennsylvania, Philadelphia. W-670-ORD-4926.

第 38 章　在 GPU 上的快速流体动力学模拟

Mark J. Harris
University of North Carolina at Chapel Hill

这章介绍一个快速的稳态流体模拟方法，它完全在 GPU 上运行。对流体动力学和相关的数学做了介绍，而且描述了在 GPU 上模拟的技术细节。通过学习这章，应该对流体动力学有基本的了解，并且知道如何使用 GPU 模拟流体。本书所附的源代码演示了这章中所描述的技术。

38.1　引言

流动无处不在：沿岸流淌的河水、香烟燃烧的青烟、壶嘴直泻的茶水、升腾成云的水汽，以及罐中混合的油漆。所有这些全都是运动的流体，全都是我们想要在交互式图形应用中逼真地描绘的现象。图 38-1 演示的例子就是使用本书提供的源代码模拟的流体。

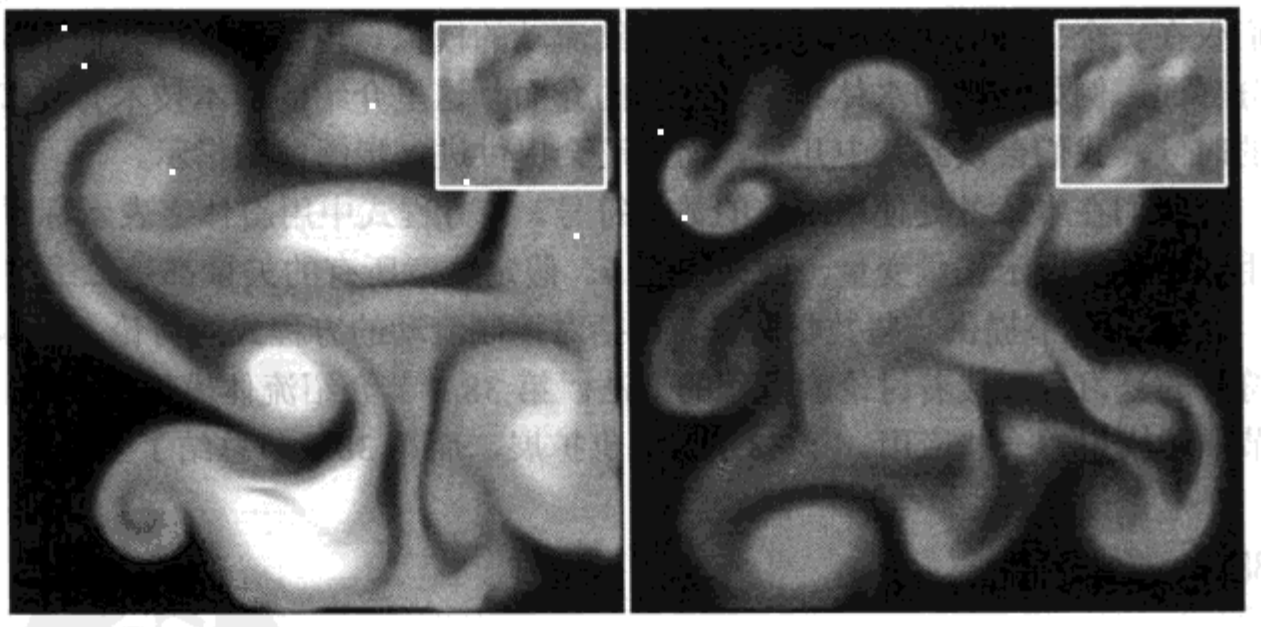


图 38-1　涡流携带的彩色“染料”
小插图：每个流体的速度场。x 和 y 方向的速度分别存储在红色和绿色通道中

流体模拟是一种有用的建筑积木，它是模拟多种自然现象的基础。因为图形硬件的并行性极大，所以我们所说的模拟在 GPU 上运行起来要比在 CPU 上快得多。运行同样的模拟，在 NVIDIA GeForce FX 上速度要比 CPU 快达 6 倍之多。

38.1.1 目的

我们的目的是帮助大家学习一个有力的工具，而不是只教一个新的技巧。流体动力学对更复杂的模拟十分有用，把它当作黑箱处理就大错特错了。不了解流体的基本物理和数学，很难使用和扩展我们所介绍的算法。因此，在这里我们没有对数学予以删减。结果，这章可能包含许多令人厌烦的方程式。我们会尽可能地提供清楚的解释，并且把数学和它的实现联系起来。

38.1.2 假设

我们希望读者至少具备大学的微积分基础知识，能够基本掌握微分方程。如果了解矢量微积分学的原理则更好，但不是必须的（我们将会复习所需要的内容）。同时，理解求导的有限差分近似是有帮助的。如果已经实现过任何实际的模拟，例如发射运动或刚体动力学，那么我们所使用的许多概念都是很熟悉的。

38.1.3 方法

我们描述的技术以 Stam 1999 的“稳态流动”方法为基础。但是 Stam 的模拟使用 CPU 实现，而我们则选择在图形硬件上实现模拟，这是因为 GPU 很适合流体模拟所必需的计算类型。模拟在单元的网格上执行。我们把可编程 GPU 优化为进行像素计算，这里可以把单元的网格看作像素。GPU 的并行性使它能够同时处理多个顶点和像素，达到很高的运行性能。为了每个周期能够执行多个纹理查询，对它们也进行了优化。因为模拟网格是存储在纹理中的，所以这种速度和并行性正是我们所需要的。

这章不可能教授有关流体动力学的全部内容。这里对模拟概念的范围做必要的限制。我们仅在一个二维的矩形区域做一个连续流动的模拟。也就是说，我们不模拟两种流体的自由分界面，例如晃动的水体和空气之间的介面。这里介绍的基本技术有许多扩展。在本章结束的时候，我们会谈到其中几种，而且还会提供进一步阅读的指示。

我们在本章各处使用统一的数学符号。在方程式中用斜体字表示标量变量，例如压力 p；用黑斜体字表示矢量变量，例如速度 $\boldsymbol{u}$。假定这章所有的矢量都是二维空间的。

第 38.2 节提供一些数学背景，包括对控制流动的方程式的讨论，基本矢量微积分概念和符号的复习等。接着讨论解方程的方法。第 38.3 节介绍流体模拟在 GPU 上的实现。第 38.4 节描述模拟的一些应用。第 38.5 节提供扩展，而第 38.6 节总结了全章。

38.2 数学背景

为了模拟流体，必须有一个数学表达式，以表示任何给定瞬间流体的状态。流体的速度是最重要的量，因为速度决定了流体如何移动其本身，以及在其中的东西。流体的速度既随时间又随空间而改变，因此我们把它表示为一个矢量场。

矢量场是矢量函数在一个参数化空间上的映射（有若干种参数化空间，但是为了本章的目的，我们假定为二维的笛卡尔坐标空间）。流体的速度矢量场是对于每个位置 $\boldsymbol{x} = (x, y)$而定义的，也存在与时间 t 相关的速度，$\boldsymbol{u}(\boldsymbol{x}, t) = (u(\boldsymbol{x}, t), v(\boldsymbol{x}, t), w(\boldsymbol{x}, t))$，如图 38-2 所示。

流体模拟的关键是正确地、及时地决定当前的速度场。通过解一组方程式可以做到这一点，方程式描述由于多种力的作用，速度随时间的变化。一旦有了速度场，我们就能用它做有趣的事了，例如用它移动物体、改变烟的密度，还能变动其他的量，只要这些量能显示在我们的应用中。

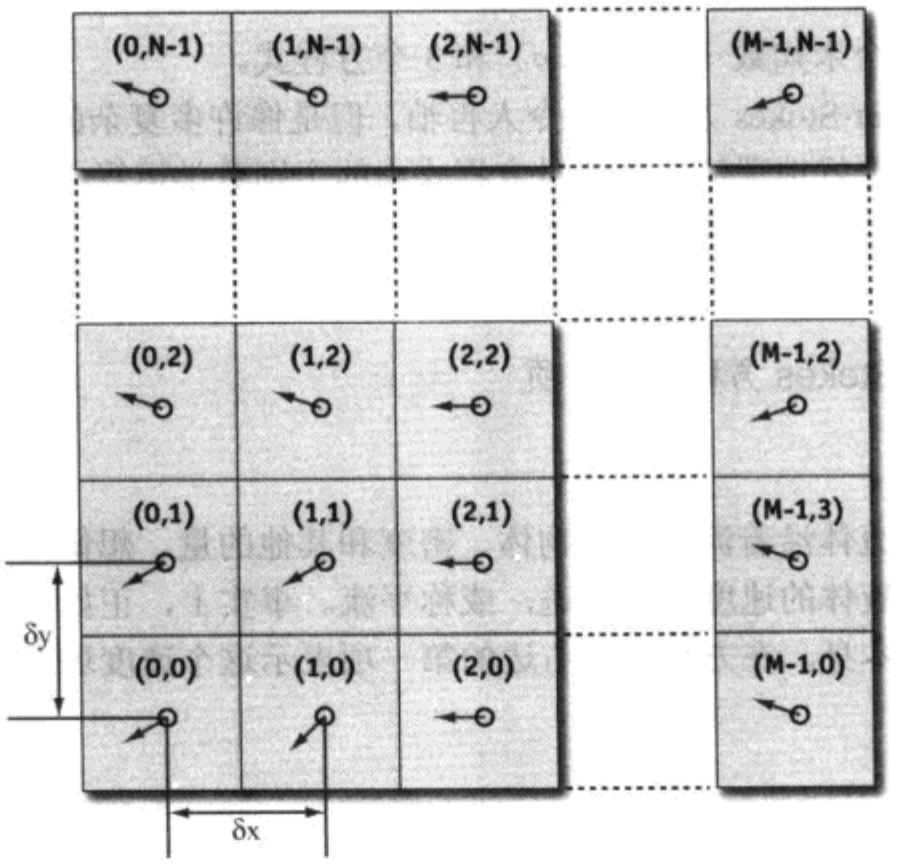


图 38-2　流体速度的网格

流体模拟的状态表示在一个 M × N 的网格上。箭头代表速度

38.2.1　不可压缩流体的 Navier-Stokes 方程式

在物理学中，在为复杂现象建模时，通常要进行简化假设。流体模拟也不例外。我们假定流体是不可压缩的和均匀的。

如果某流体的任何局部区域（subregion）的体积不随时间而改变，那么这种流体是不可压缩的。如果它的密度 ρ 在空间中是常数，则流体是均匀的。既有不可压缩性又有均匀性，则意味着密度对时间和空间都是常数。这是流体动力学中普遍的假设，也适用于水和空气等真实流体的模拟。

我们用空间坐标 $\boldsymbol{x} = (x, y)$和时间变量 t，在一般的笛卡尔坐标系上模拟流体动力学。如前所述，流体由它的矢量速度场 $\boldsymbol{u}(\mathbf{x}, t)$和标量压力场 $p(\boldsymbol{x}, t)$表示。这些场既随时间又随空间而改变。不可压缩的流体，其速度和压力在起始时间 $t = 0$ 已知，那么流体状态随时间的变化，可以用 Navier-Stokes 方程式描述：

$$\frac{\partial \boldsymbol{u}}{\partial t} = -(\boldsymbol{u} \cdot \nabla)\boldsymbol{u} - \frac{1}{\rho}\nabla p + v\nabla^2 \boldsymbol{u} + \boldsymbol{F} \tag{1}$$

$$\nabla \cdot \boldsymbol{u} = 0 \tag{2}$$

这里ρ是流体密度（常数），v 是动力黏度，而 $\boldsymbol{F} = (f_x, f_y)$代表作用于流体的任何外力。注意由于 $\boldsymbol{u}$ 是矢量，方程式 1 实际上是两个方程式：

$$\frac{\partial u}{\partial t} = -(\boldsymbol{u}\cdot\nabla)u - \frac{1}{\rho}\nabla p + v\nabla^2 u + f_x$$

$$\frac{\partial v}{\partial t} = -(\boldsymbol{u}\cdot\nabla)v - \frac{1}{\rho}\nabla p + v\nabla^2 v + f_y$$

因此，我们有 3 个未知数（u、v 和 p）和 3 个方程式。

可能一开始 Navier-Stokes 方程式会令人害怕，但是像许多复杂的概念一样，把它们分解为简单的部分有助于更好地理解。即使对方程式中的个别数学运算不理解，也不必担心。下面，我们首先了解影响流体流动的不同因素。在方程式 1 右边的 4 个项是加速度。我们将依次看其中的每一项。

38.2.2 Navier-Stokes 方程式的各项

平流项

流体的速度使得流体沿着流动传送物体、密度和其他的量。想像把染料喷射进入流动的液体之内。染料沿着液体的速度场被传送，或称平流。事实上，正如液体的速度携带染料一样，它也携带着液体本身。在方程式 1 右边的第一项表示这个速度场的自身平流，于是被称为平流项。

压力项

因为流体的分子能够彼此运动，它们倾向于“挤压”和“碰撞”。当外力施加于流体时，外力不是立即地传导到整个体积，而是离外力近的分子把那些远一点的分子推开，从而产生压力。因为压力是单位面积上的力，所以在流体上的任何压力自然地产生加速度（根据牛顿第二定律，$\boldsymbol{F}=\boldsymbol{ma}$）。第 2 项代表这个加速度，称为压力项。

扩散项

根据对真实液体的经验可以知道，有些液体比其他的更“粘稠”。举例来说，糖蜜和糖浆流动很慢，而医用酒精却流动很快。即稠的液体黏度高。黏度是液体流动受到阻碍的衡量。这个阻碍造成动量的扩散（由此而引起速度的扩散），因此，我们认为第 3 个项是扩散项。

外力项

第 4 项是由于外力施加到流体上而增加的加速度。这些力或许是局部力，或许是整体力。局部力施加到液体的特定区域，例如，吹动空气的电扇风力。而像重力这样的整体力则均匀地施加于整个液体。

在很快地复习矢量微积分之后，我们将会回到 Navier-Stokes 方程式。对于详细的推导和更多的细节，我们建议阅读 Chorin and Marsden 1993 和 Griebel et al. 1998。

38.2.3 矢量微积分的简要复习

方程式 1 和 2 包含符号∇（时常读 del 的音）的 3 次不同的使用，∇即差分运算符（劈形算符 nabla）。∇分别应用于梯度、散度和拉普拉斯（Laplacian）算子，如表 38-1 所示。表中

公式里的下标数字 i 和 j 表示笛卡尔坐标系中离散的位置，δx 和 δy 分别是 x 和 y 方向上的网格间距（见图 38-2）。

表 38-1　流动模拟使用的矢量微积分算符

算符	定　义	有限微分形式
梯度	$\nabla p = \left(\frac{\partial p}{\partial x}, \frac{\partial p}{\partial y}\right)$	$\frac{p_{i+1,j} - p_{i-1,j}}{2\delta x}, \frac{p_{i,j+1} - p_{i,j-1}}{2\delta y}$　（3）
散度	$\nabla \cdot \boldsymbol{u} = \frac{\partial u}{\partial x} + \frac{\partial v}{\partial y}$	$\frac{u_{i+1,j} - u_{i-1,j}}{2\delta x} + \frac{v_{i,j+1} - v_{i,j-1}}{2\delta y}$　（4）
拉普拉斯算子	$\nabla^2 p = \frac{\partial^2 p}{\partial x^2} + \frac{\partial^2 p}{\partial y^2}$	$\frac{p_{i+1,j} - 2p_{i,j} + p_{i-1,j}}{(\delta x)^2} + \frac{p_{i,j+1} - 2p_{i,j} + p_{i,j-1}}{(\delta y)^2}$　（5）

标量场的梯度是一个标量场的偏导数矢量。在方程式 2 中出现的散度有重要的物理意义。它是“密度”离开给定空间区域的速率。在 Navier-Stokes 方程式中，它被应用于流速，它测量的是经过一小块流体表面的速度的变化。让流体的散度永远是 0，连续性方程式 2 就可以表达不可压缩的假设。在散度算子中的点积得到的是偏导数的总和（不像梯度算子是一个矢量）。这意味着散度算子只能适用于一个矢量场，例如速度 $\boldsymbol{u} = (u, v)$。

注意，标量场的梯度是一个矢量场，而矢量场的散度是一个标量场。如果把散度算子应用于梯度算子的结果，得到的是拉普拉斯算子$\nabla \cdot \nabla = \nabla^2$。如果网格单元是正方形（即，如果 $\delta x = \delta y$，本章的其余部分基于这个假设），拉普拉斯算子简化为：

$$\nabla^2 p = \frac{p_{i+1,j} + p_{i-1,j} + p_{i,j+1} + p_{i,j-1} - 4p_{i,j}}{(\delta x)^2}. \qquad (6)$$

拉普拉斯算子在物理学中普遍出现，最著名的是扩散方程的形式，例如热传导方程。$\nabla^2 x = b$ 的形式是 Poisson（泊松）方程。$b = 0$ 是拉普拉斯方程的情形，即拉普拉斯算子的起源。在方程式 2 中，拉普拉斯算子被应用于一个矢量场。这是一个简化符号：把算子分别地应用于矢量场的每个数量成分。

38.2.4　解 Navier-Stokes 方程式

只有对于一些简单的物理结构，才能分析地解 Navier-Stokes 方程式。在其他的情况下，可以用数值积分法递增地来解它们。因为我们感兴趣的是流动随时间的变化，递增数值法可以满足需要。

像任何算法一样，我们必须把 Navier-Stokes 方程的解法分成简单的几步。我们使用的方法基于 Stam 1999 描述的稳定流体技术。在这节中我们介绍每个步骤的算法，在第 38.3 节中我们介绍 Cg 语言在 GPU 上对它们的实现。

首先要把方程式转变成更适宜数值积分法的形式。已经知道 Navier-Stokes 方程式是可以求 u、v 和 p 的 3 个方程式。但是该如何求解呢？下面就介绍一种变换，通过它可以得到直接的算法。

Helmholtz- Hodge 的分解

基本矢量微积分学告诉我们，任何矢量 $\boldsymbol{v}$ 能被分解为总和为 $\boldsymbol{v}$ 的一系列基本矢量分量。

例如，在笛卡尔坐标系上，我们通常用 $\boldsymbol{v}=(x, y)$表达矢量，它可以写成 $\boldsymbol{v}=x\hat{\boldsymbol{i}}+y\hat{\boldsymbol{j}}$，这里 $\hat{\boldsymbol{i}}$ 和 $\hat{\boldsymbol{j}}$ 是沿着坐标轴排列的基本单位矢量。

我们不但能把矢量分解成矢量的和，同样地，我们也能把矢量场分解成矢量场的总和。设 D 是空间的区域，在此处即是流体被定义的平面。让这个区域有平滑的（即，可求导的）边界∂D，法线的方向为 $\boldsymbol{n}$。我们可以使用下面的由 Chorin 和 Marsden1993 提出的定理。

Helmholtz-Hodge 分解定理

在 D 上的矢量场 $\boldsymbol{w}$ 能惟一地分解为这样的形式：

$$\boldsymbol{w}=\boldsymbol{u}+\nabla p \tag{7}$$

这里在 $\boldsymbol{u}$ 的散度为 0，并且与∂D 平行；也就是说，在∂D 上 $\boldsymbol{u}\cdot\boldsymbol{n}=0$。

这里就不证明这个定理了。关于它的细节和证明，参见 Chorin 和 Marsden 1993 的第 1.3 节。

这个定理指出任何的矢量场可以分解为两个其他矢量场之和：一个是无散度矢量场，另一个是标量场的梯度。它也指出无散度场在边界为 0。这是一个有力的工具，由它我们得到两个有用的结果。

第一个结果

解 Navier-Stokes 方程，包括在每个时间段计算 3 个量来更新速度：移流、扩散和外力的施加。得到的结果是一个散度非 0 的新的速度场 $\boldsymbol{w}$。但是连续方程要求在每个时间段结束时得到散度为 0 的速度。好在 Helmholtz-Hodge 的分解定理告诉我们，减去压力场的梯度可以改变速度的散度：

$$\boldsymbol{u}=\boldsymbol{w}-\nabla p \tag{8}$$

第二个结果

定理也引出一个计算压力场的方法。如果我们把散度算子应用到方程式 7 的两边，我们得到：

$$\nabla\cdot\boldsymbol{w}=\nabla\cdot(\boldsymbol{u}+\nabla p)=\nabla\cdot\boldsymbol{u}+\nabla^2 p \tag{9}$$

但是因为方程式 2 规定 $\nabla\cdot\boldsymbol{u}=0$，上式简化为：

$$\nabla^2 p=\nabla\cdot\boldsymbol{w} \tag{10}$$

这是流体压力的 Poisson 方程式（见第 38.2.3 节），有时称为 Poisson 压力方程。这就是说在得到散度速度 $\boldsymbol{w}$ 之后，我们可以解方程式 10 得到 p，然后把 $\boldsymbol{w}$ 和 p 代入方程式 8，计算新的无散度速度场 $\boldsymbol{u}$，我们稍后还会回到这一点。

现在我们需要一个方法计算 $\boldsymbol{w}$。为了这样做，让我们回到对矢量和矢量场的比较。从点积的定义，我们知道能通过计算 $\boldsymbol{r}$ 和 $\hat{\boldsymbol{s}}$ 的点积，求得矢量 $\boldsymbol{r}$ 在一个单位矢量 $\hat{\boldsymbol{s}}$ 上的投射。求点积是把矢量 $\boldsymbol{r}$ 投射到 $\hat{\boldsymbol{s}}$ 方向得到分量。我们可以使用 Helmholtz-Hodge 的分解定理定义一个投射算符 $\mathbb{P}$，意义是把一个矢量场 $\boldsymbol{w}$ 投射到它的无散度分量 $\boldsymbol{u}$ 上。如果我们把 $\mathbb{P}$ 应用到方程式 7，我们得到：

$$\mathbb{P}\boldsymbol{w}=\mathbb{P}\boldsymbol{u}+\mathbb{P}(\nabla p)$$

但是根据 $\mathbb{P}$ 的定义，$\mathbb{P}\boldsymbol{w}=\mathbb{P}\boldsymbol{u}=\boldsymbol{u}$。因此，$\mathbb{P}(\nabla p)=0$。现在让我们使用这些来简化 Navier-Stokes 方程式。

首先，我们把投射算符应用到方程式 1 两边：

$$\mathbb{P}\frac{\partial \boldsymbol{u}}{\partial t}=\mathbb{P}\left(-(\boldsymbol{u}\cdot\nabla)\boldsymbol{u}-\frac{1}{\rho}\nabla p+\nu\nabla^2\boldsymbol{u}+\boldsymbol{F}\right)$$

因为 $\boldsymbol{u}$ 散度为 0，左边的导数也是无散度的，因此 $\mathbb{P}(\partial \boldsymbol{u}/\partial t)=\partial \boldsymbol{u}/\partial t$。同时，$\mathbb{P}(\nabla p)=0$，所以去掉压力项。留下下列的方程式：

$$\frac{\partial \boldsymbol{u}}{\partial t}=\mathbb{P}\left(-(\boldsymbol{u}\cdot\nabla)\boldsymbol{u}+\nu\nabla^2\boldsymbol{u}+\boldsymbol{F}\right) \tag{11}$$

这个方程式最棒的一点，是它用符号封装了模拟流体的整个算法。我们首先计算在右边括弧里的东西。从左到右，我们计算移流、扩散和外力项。计算完这 3 项得到一个散度速度场 $\boldsymbol{w}$，对它应用投射算符得到一个新的无散度的速度场 $\boldsymbol{u}$。具体做法是，解方程式 10 求 p，然后按照方程式 8 从 $\boldsymbol{w}$ 减去梯度 p。

一个典型的实施，不是像方程式 11 那样计算各个分量再把它们加在一起。相反地，是经过在状态上的变换的合成来求解的。换句话说，每个分量都是一个步骤，它取一个场作为输入，而产生新的场作为输出。我们可以定义一个算子 $\mathbb{S}$，它就是方程式（11）的一个步骤的解。各分量算子定义如下：平流 $\mathbb{A}$、扩散 $\mathbb{D}$、外力 $\mathbb{F}$ 和投射 $\mathbb{P}$。

$$\mathbb{S}=\mathbb{P}\cdot\mathbb{F}\cdot\mathbb{D}\cdot\mathbb{A} \tag{12}$$

因此，模拟运算法则的一个步骤能表示为 $\mathbb{S}(\boldsymbol{u})=\mathbb{P}\cdot\mathbb{F}\cdot\mathbb{D}\cdot\mathbb{A}(\boldsymbol{u})$。从右到左进行运算；首先是平流，接着是扩散、外力和投射。注意，为了清楚起见，在这里省略了时间，但是在实际中，时间必须要用于每个算子的计算。现在让我们更进一步地看平流和扩散步骤，然后着手解 Poisson 方程。

平流

由速度传送流体本身和在流体中的其他量，这个过程就是平流。为了计算平流量，我们必须更新每个坐标点上的量。在计算一个量如何沿着速度场移动时，可以想像每个网格单元由一个粒子代表。最初打算这样来计算平流：像更新一个粒子系统那样来更新网格。每个粒子在时间段 δt 沿着速度场向前移动的距离 $\boldsymbol{r}$ 为：

$$\boldsymbol{r}(t+\delta t)=\boldsymbol{r}(t)+\boldsymbol{u}(t)\delta t$$

这就是 Euler 方法；它对于通常的微分方程的显式（或正向）积分，是一个简单的方法（还有更正确的方法，例如中点法和 Runge-Kutta 法）。

但是这个方法有两个问题：第一个是与模拟有关的问题，使用显式的方法计算平流，在大的时间段是不稳定的，如果 $\boldsymbol{u}(t)\,\delta t$ 比单个网格单元的尺寸大，这个问题还可能会“放大”。第二个是与 GPU 实现有关的特殊问题。我们的模拟是在片元程序中实现的，它不能够改变正在写的片元位置。而向前积分法需要能够“移动”粒子，因此，它不能够在目前的 GPU 上实现。

解决的办法是逆向处理和使用隐式积分法（Stam 1999）。不是通过计算粒子在当前时间段移动到哪里来得到移流量，而是从每个网格单元逆时追踪粒子轨道，得到它以前的位置，并到开始的网格单元，复制那个位置的量。为了更新量 q（它可能是速度、密度、温度或被液体携带的任何量），我们使用下列的方程：

$$q(\boldsymbol{x},t+\delta t)=q(\boldsymbol{x}-\boldsymbol{u}(\boldsymbol{x},t)\delta t,t) \tag{13}$$

我们不但能够容易地在 GPU 上实现这个方法，而且如 Stam 所示，它对于任意时间段和速度都是稳定的。图 38-3 描绘的是如何计算平流，为了计算以双圈标记的单元的平流，逆时追踪速度场到绿色的×。对最靠近绿色×的 4 个网格值（在图中连接成一个绿色的正方形）进行双线性插值，而对出发网格单元写下结果。

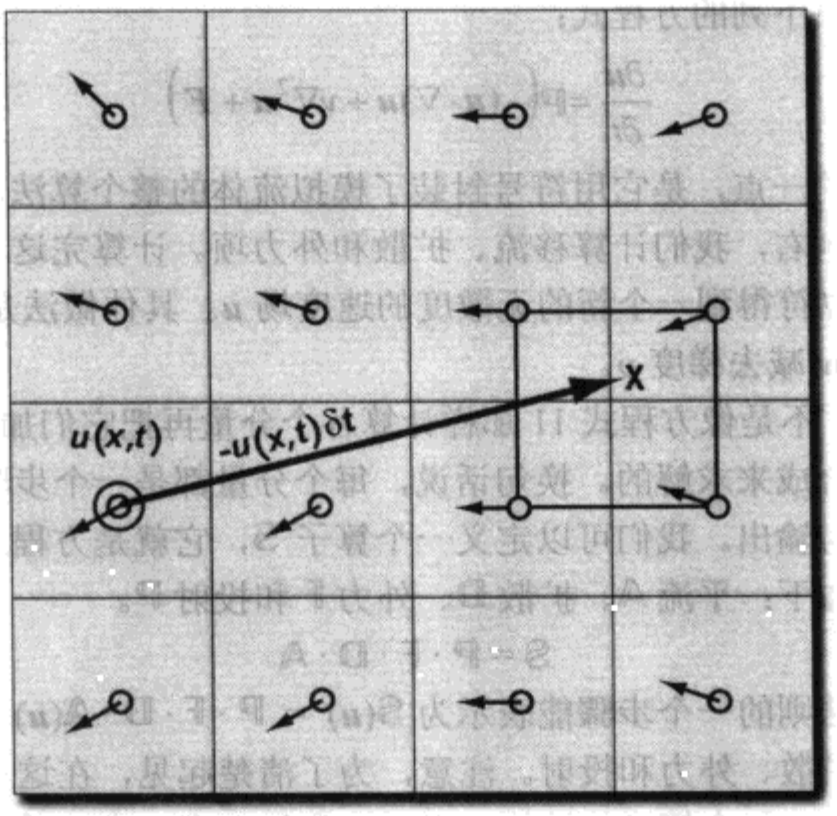


图 38-3 计算流体的平流

隐式的平流步骤是穿过速度场向后追踪，以确定是如何向前携带量的

黏性的扩散

如前所述，黏性的液体对流动有某种阻碍，这会造成速度的扩散（或分散）。粘性扩散的偏微分方程式是：

$$\frac{\partial \boldsymbol{u}}{\partial t}=\nu\nabla^2\boldsymbol{u} \tag{14}$$

与求解平流一样，我们可以选择该如何解这个方程。一个显而易见的方法是为显式的、不连续的方程式制定一个简单的算法：

$$\boldsymbol{u}(\boldsymbol{x},t+\delta t)=\boldsymbol{u}(\boldsymbol{x},t)+\nu\delta t\nabla^2\boldsymbol{u}(\boldsymbol{x},t)$$

在这个方程中，∇^2 是方程 3 拉普拉斯算子的不连续形式。像计算移流的显式 Euler 法一样，这个方法对大值的 δt 和 ν 是不稳定的。我们再次遵循 Stam 的引导，使用方程 14 的隐式形式：

$$(\boldsymbol{I}-\nu\delta t\nabla^2)\,\boldsymbol{u}(\boldsymbol{x},t+\delta t)=\boldsymbol{u}(\boldsymbol{x},t) \tag{15}$$

这里 $\boldsymbol{I}$ 是单位矩阵。这个公式对于任何时间段和黏度都是稳定的。这个方程是速度的 Poisson 方程（有些晦涩）。还记得我们用 Helmholtz-Hodge 分解得到压力的 Poisson 方程。还可以用迭代松弛法解这些方程式。

Poisson 方程的解法

我们需要解两个 Poisson 方程式：Poisson 压力方程式和黏度扩散方程式。Poisson 方程式

在物理学中是常用的和众所周知的。我们使用迭代法，它从一个近似的解开始，每迭代一次都更接近准确值。

Poisson 方程式是 $\boldsymbol{Ax} = \boldsymbol{b}$ 形式的矩阵方程式，这里 $\boldsymbol{x}$ 是我们正在解的数值矢量（在我们的例子中是 p 或 $\boldsymbol{u}$），$\boldsymbol{b}$ 是一个常数的矢量，而 $\boldsymbol{A}$ 是一个矩阵。在此处，$\boldsymbol{A}$ 在拉普拉斯算子 ∇^2 中隐式地表示，因此它不需要显式地存储为一个矩阵。迭代法从一个最初的“猜测”值 $\boldsymbol{x}^{(0)}$ 开始，每一步 k 产生一个改进的解 $x^{(k)}$。上标符号代表迭代次数。最简单的迭代法叫做 Jacobi 迭代。对于一般矩阵方程式的 Jacobi 迭代的推导可以在 Golub 和 Van Loan 1996 中发现。

更复杂的方法会聚得更快，例如共轭梯度和多栅格法，但是我们使用 Jacobi 迭代法，因为它简单且易于实现。对于更复杂解法的细节和例子，见 Bolz 等 2003、Goodnight 等 2003 和 Kruger and Westermann 2003。

方程式 10 和 15 显得不同，但是使用方程式 3 可以使两者离散，而且重写为以下形式：

$$x_{i,j}^{(k+1)} = \frac{x_{i-1,j}^{(k)} + x_{i+1,j}^{(k)} + x_{i,j-1}^{(k)} + x_{i,j+1}^{(k)} + \alpha b_{i,j}}{\beta} \tag{16}$$

这里 α 和 β 是常数。x、b、α 和 β 的值对两个方程式是不同的。在 Poisson 压力方程式中，x 代表 p，b 代表 $\nabla \cdot \boldsymbol{w}$，$\alpha = -(\delta x)^2$，而 $\beta = 4$。[1] 对于黏度扩散方程式，x 和 b 代表 $\boldsymbol{u}$，$\alpha = (\delta x)^2 / \nu \delta t$，而 $\beta = 4 + \alpha$。

我们这样来表示方程，目的是使用相同的代码解这两个方程式。为了解方程式，我们只是简单地运行若干次迭代，把前一个迭代结果作为下一个迭代的输入（$\boldsymbol{x}^{(k+1)}$变成 $\boldsymbol{x}^{(k)}$），在每个网格单元应用方程式 16。因为 Jacobi 迭代会聚得很慢，所以需要运行许多次。所幸的是，Jacobi 迭代在 GPU 上运行消耗很小，因此在很短时间内能运行许多次。

初始条件和边界条件

在有限域上定义的任何微分方程都需要边界条件。边界条件决定如何在模拟区域的边缘计算数值。同时，为了计算流量随时间的变化，我们必须知道它如何开始，换句话说，它的初始条件。对于我们的流动模拟，假定最初流体的速度为 0，而且各处压力为 0。边界条件需要更多的讨论。

在每个时间段，我们解方程求两个量：速度和压力，因此需要两者的边界条件。因为是在一个矩形网格上模拟流体，我们假定它是在一个方格里的流体，并且不能够流过方格的边。对于速度，我们使用不穿越（no-slip）条件，它指定速度在边界变成 0。Poisson 压力方程的正确解法需要纯 Neumann 边界条件：$\partial p / \partial \boldsymbol{n} = 0$。这意味着，在边界上，在法线方向上压力对边界的变化率是 0。我们在第 38.3 节结尾处再谈边界条件。

38.3　实现

既然了解了问题和它的基本解法，我们就能进一步去实现了。首先对这个算法安排一些伪代码。每个时间段的算法都是相同的，因此这个伪代码就代表单个时间段。速度和压力场

1. 注意这个方程式的解实际上是 $p\delta t$，而不是 p。因为只是在投射步骤计算梯度时用到压力，所以这不是一个问题。因为 δt 在网格上是常数，它不影响梯度。

的变量是 u 和 p。

```
// 方程式 12 的前 3 个算子
u = advect(u);
u = diffuse(u);
u = addForces(u);
// 现在应用投射算符到结果
p = computePressure(u);
u = subtractPressureGradient(u, p);
```

因为这些运算的大部分不能够适当地执行，所以在实际中，需要暂时存储。举例来说，在伪代码中平流步骤更准确地写成：

```
uTemp = advect(u);
swap(u, uTemp);
```

这个伪代码没包含特定的实现细节。事实上，同样的伪代码在 CPU 和 GPU 上都实现得很好。我们的目的是在 GPU 上执行所有步骤。一些读者可能不熟悉在 GPU 上的这类计算，因此，我们会对典型的 CPU 流体模拟运算与它们在 GPU 上的版本做一些类比。

38.3.1 CPU-GPU 的类比

任何计算机的基础是它的内存和处理模型，所以任何应用程序必须考虑数据的表达和计算。让我们简略谈谈这两者在 CPU 和 GPU 之中的不同。

纹理 = 数组

我们的模拟表现二维网格上的数据。在 CPU 上对这个网格的自然表达是一个数组。数组在 GPU 上的类比是纹理。虽然纹理不像数组那么灵活，但是随着图形硬件的发展，它们的灵活性正在提高。实现流动模拟所需的所有基本运算，都得到目前 GPU 上的纹理的支持。因为纹理通常有 3 或 4 个彩色通道，所以对有 2 到 4 个分量的矢量数据类型，它们可以提供一个天然的数据结构。也可以选择把多个标量场存储在单个纹理中。最基本的运算是数组（或内存）的读取，可以使用一个纹理查询来完成。因此，一个数组的偏移在 GPU 中的类比是一个纹理坐标。我们至少需要两个纹理来表现流体的状态：一个用于速度，一个用于压力。为了使流量能看得见，我们用另一个纹理来包含一个由流体携带的量。我们可以把它想成“墨水”。图 38-4 显示了这些纹理的例子，还有一个另外的表现旋涡纹理，在 38.5.1 节进行了讲述。

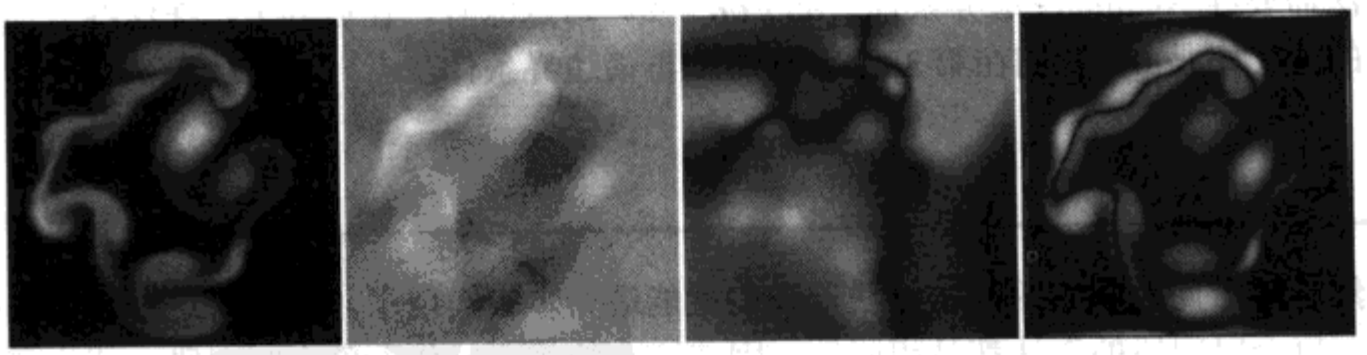


图 38-4 存储在纹理中的流动模拟的状态场

从左到右：场是“墨水”、速度（缩放和偏差在范围[0, 1]之内，于是 0 速度是灰色）、压力（蓝色代表低压，红色代表高压）和旋涡状态（黄色代表反时针方向旋转，蓝色代表顺时针转）

循环体 = 片元程序

CPU 的模拟实现是通过循环执行算法，使用一对嵌套的环路在网格中每个单元上迭代。在每个单元执行的是相同的计算。GPU 没有在纹理的每个纹素（texel）上执行这个内循环的能力。然而，片元传递途径可以在每个片元上执行相同的计算。对程序设计者而言，好像每个片元都有一个处理器，而所有的片元又同时更新。依照并行程序的说法，这个模型即是用单指令多数据（SIMD）计算。因此，类比于 CPU 在一个数组上的嵌套循环计算，在 GPU 中就是以 SIMD 形式把片元程序应用于每个片元。

反馈 = 纹理更新

在第 38.2.4 节中，我们描述了如何使用 Jacobi 迭代法解 Poisson 方程式。这类迭代法使用这次迭代的结果作为下一次迭代的输入。这种反馈在数学方法中是很普通的。但是在 CPU 的典型实施中一般不考虑反馈，因为它实施起来很麻烦，要对变量和数组两者都进行读和写的操作。在 GPU 上，片元处理器的输出总是写入帧缓冲区。把帧缓冲区看作不能够直接读取的二维数组。有两个方法能够把帧缓冲区的内容放入一个可读的纹理之内：

- 复制到纹理（CTT），把纹理的复本从帧缓冲区复制到一个纹理。
- 渲染到纹理（RTT），把一个纹理当作帧缓冲区使用（使 GPU 能直接对它写入）。

CTT 和 RTT 的功能是相同的，但是有一个运行开销问题。为了一般性起见，我们假定不使用任何一种，而是把写入一个纹理的过程当作一个纹理更新。

实际上，早先我们提到算法的 5 个步骤，其中的每一步都更新一个暂时网格，然后执行一个交换。RTT 要求使用两个纹理来实现反馈，因为渲染一个正在读的纹理，结果是不确定的。在这种情况下的交换只是纹理 ID 的交换。因此 RTT 的运行开销是固定的。而对于 CTT 则不同，它只需要一个纹理。帧缓冲区的作用就是一个暂时网格，交换是通过把数据从帧缓冲区复制到纹理执行的。这个复制的运行开销与纹理大小成正比。

38.3.2　片运算

我们把模拟步骤分解为片运算（slab，是 slabop 的缩写）[2]。每片（slabop）可以处理帧缓冲区内的 1 个或更多的（时常是所有的）片元，通常是激活一个片元程序，跟着更新一个纹理。片元处理由渲染几何图元（primitives）所驱动。对于这个应用，我们渲染的几何体很简单，只是方块图元和线图元。

在任何的片运算中要处理两种类型的片元：内部片元和边界片元。为了存储和计算边界条件，在 2D 网格上保留单个单元的周长。在单元的内部和边界上通常执行不同的计算。为了更新内部片元，我们渲染一个四边形，它几乎覆盖全部像素，除了帧缓冲区周边上一个像素宽的边界。为了更新边界单元，我们渲染 4 个线图元。我们对内部片元和边界片元分别地应用片元程序，见图 38-5。

38.3.3　片元程序的实现

既然我们知道了算法的步骤、数据表达式和该如何执行一个片运算，我们就可以写在每

2. 我们称它们为片运算，是因为帧缓冲区仅限于二维，所以如果用 GPU 模拟三维就必须把三维网格分成一堆二维的“片（slab）”。

个单元执行计算的片元程序了。

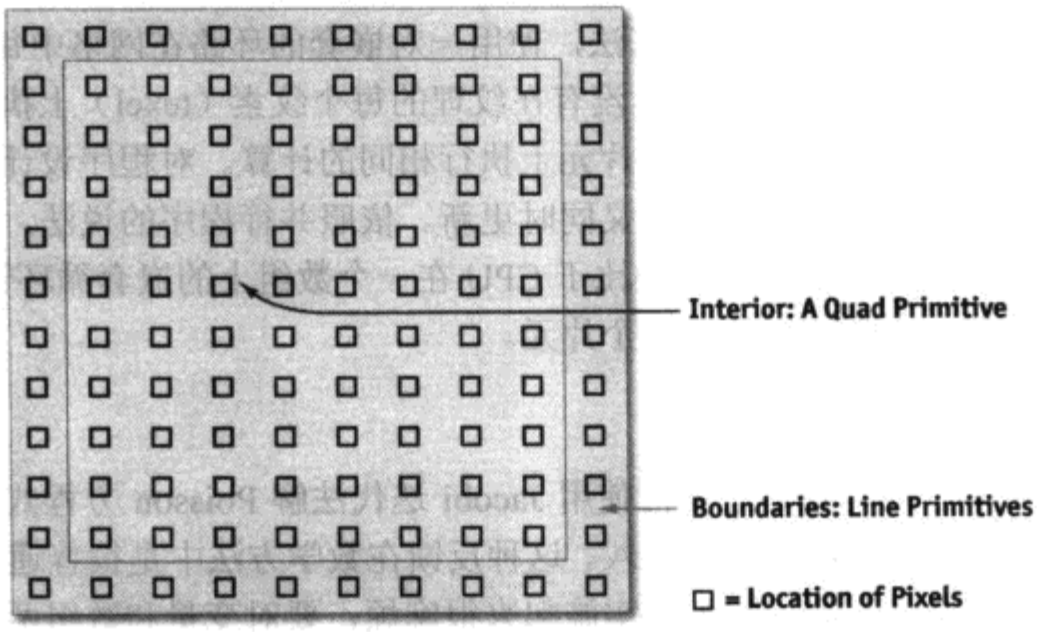


图 38-5 用于更新网格内部和边界的图元

网格的更新包括一个内部方块和几条边界线的渲染。把独立的片元程序应用于内部片元和边界片元

平流

列表 38-1 显示的平流片元程序的执行，几乎完全遵从方程式 13，这里重复一下：

$$q(\boldsymbol{x},t+\delta t)=q(\boldsymbol{x}-\boldsymbol{u}(\boldsymbol{x},t)\delta t,t) \tag{13}$$

也存在一点微小的不同。由于纹理坐标与模拟区域的单位不同（纹理坐标是在 [0, *N*]范围内，这里 *N* 是网格分辨率），必须把速度缩放进网格空间。在 Cg 代码中就是用局部速度乘以参数 rdx，rdx 代表网格比例尺 δx 的倒数。必须把纹理约束（wrap）模式设定成 CLAMP_TO_EDGE，以便把超出范围[0, *N*]之外的向后追踪夹持在边界线的纹素上。下面描述的边界条件正确地更新了这些纹素，使得这种情况能正常地进行。

列表 38-1 平流片元程序

```
void advect(float2 coords : WPOS,          // 网格坐标
            out float4 xNew : COLOR,       // 平流后的量
            uniform float timestep,
            uniform float rdx,             // 网格大小的倒数
            uniform samplerRECT u,         // 输入速度
            uniform samplerRECT x)         // 平流的量
{
  // 按照“逆时”速度场
  float2 pos = coords - timestep * rdx * f2texRECT(u, coords);

  // 修正并写入输出片元
  xNew = f4texRECTbilerp(x, pos);
}
```

在这个代码中，参数 ***u*** 是速度场纹理，而 ***x*** 是被平流的场。它可能是速度或另外的量，例如染料浓度。用函数 f4texRECTbilerp()来执行双线性插值法，对代入的纹理坐标附近的 4 个纹素进行插值。因为目前的 GPU 不支持浮点纹理中的自动双线插值法，我们不得不用这个类型的代码实现它。

黏度的扩散

用第 38.2.4 节描述的 Jacobi 迭代法编写 Jacobi 迭代片元程序很简单，如列表 38-2 所示。

列表 38-2　用于解 Poisson 方程式的 Jacobi 迭代片元程序

```
void jacobi(half2 coords : WPOS,          // 网格坐标
            out half4 xNew : COLOR,       // 结果
            uniform half alpha,
            uniform half rBeta,           // beta 的倒数
            uniform samplerRECT x,        // x 矢量(Ax = b)
            uniform samplerRECT b)        // b 矢量(Ax = b)
{
  // 左、右、下和上的 x 样本
  half4 xL = h4texRECT(x, coords - half2(1, 0));
  half4 xR = h4texRECT(x, coords + half2(1, 0));
  half4 xB = h4texRECT(x, coords - half2(0, 1));
  half4 xT = h4texRECT(x, coords + half2(0, 1));

  // b 样本，从中间取
  half4 bC = h4texRECT(b, coords);

  // 求 Jacobi 迭代
  xNew = (xL + xR + xB + xT + alpha * bC) * rBeta;
}
```

注意 rBeta 参数是从方程式 16 得到的 β 的倒数。为了解扩散方程，我们将 alpha 设定为 $(\delta x)^2 / v\delta t$，rBeta 设定为 $1 / (4 + (\delta x)^2 / v\delta t)$，把 $\boldsymbol{x}$ 和 $\boldsymbol{b}$ 参数设置到速度纹理。然后运行迭代若干次（通常是 20 到 50 次，但是为了减少误差，可以增加次数）。

外力的应用

计算由外力所引起的加速度是最简单的步骤。在本书附带材料的演示程序中，可以看到通过鼠标的点击和拖曳，把一个脉冲加到流体上。具体过程如下：单击速度纹理内的一点，画一个斑点。斑点的颜色给脉动的方向和大小编码：红色的通道包含 x 上的量，而绿色的通道包含 y 上的量。斑点实际上是一个二维的高斯“条”。

我们使用一个片元程序检查每个片元与脉冲位置的距离。然后把量 $\boldsymbol{c}$ 添加到颜色中去：

$$\boldsymbol{c} = \boldsymbol{F}\delta t \exp\left(\frac{(x - x_p)^2 + (y - y_p)^2}{r}\right)$$

这里，$\boldsymbol{F}$ 是根据方向和鼠标拖曳的长度计算出来的力，r 是所要求的脉冲半径，而 (x, y) 和 (x_p, y_p) 分别是在窗口坐标中的片元位置和脉冲（点击）位置。

投射

在本章的开始，我们知道投射步骤分为两个运算：解 Poisson 压力方程求 p，以及从中间的速度场减去梯度 p。这需要 3 个片元程序：上述的 Jacobi 迭代法程序、一个计算中间速度场散度的程序和一个从中间速度场减去梯度的程序。

在列表 38-3 中显示的散度程序，把中间速度场取作参数 w，把网格缩放率倒数的一半取作参数 halfrdx，根据表 38-1 给出的有限微分公式计算散度。

列表 38-3 散度片元程序

```
void divergence(half2 coords : WPOS,          // 网格坐标
                out half4 div : COLOR,        // 散度
                uniform half halfrdx,         // 0.5/网格大小
                uniform samplerRECT w)        // 矢量场
{
  half4 wL = h4texRECT(w, coords - half2(1, 0));
  half4 wR = h4texRECT(w, coords + half2(1, 0));
  half4 wB = h4texRECT(w, coords - half2(0, 1));
  half4 wT = h4texRECT(w, coords + half2(0, 1));

  div = halfrdx * ((wR.x - wL.x) + (wT.y - wB.y));
}
```

把散度写入一个暂时的纹理，然后用它作为 Jacobi 迭代程序的 b 参数的输入。把 Jacobi 程序的 x 参数设定为压力纹理，一开始把它清 0（换句话说，用 0 作为压力场的初始值）。alpha 和 rBeta 参数分别设定为$-(\delta x)^2$和 1/4。

为了求解时能够很好地收敛，典型的迭代次数为 40 到 80 次。迭代次数的改变会影响模拟的准确度。迭代次数低于 20 不好，因为那样会引起明显的误差。迭代次数越多，产生的旋涡细节越多，准确度也越好，但是所需要的计算时间也越多。在完成 Jacobi 迭代法之后，我们在以下的程序中把压力场纹理与参数 p 捆绑在一起，根据表 38-1 的定义计算梯度 p，并从参数 w 中的中间速度场纹理减去 p，见列表 38-4。

列表 38-4 减去梯度的片元程序

```
void gradient(half2 coords : WPOS,       // 网格坐标
              out half4 uNew : COLOR,    // 新速度
              uniform half halfrdx,      // 0.5/网格大小
              uniform samplerRECT p,     // 压力
              uniform samplerRECT w)     // 速度
{
  half pL = h1texRECT(p, coords - half2(1, 0));
  half pR = h1texRECT(p, coords + half2(1, 0));
  half pB = h1texRECT(p, coords - half2(0, 1));
  half pT = h1texRECT(p, coords + half2(0, 1));

  uNew = h4texRECT(w, coords);
  uNew.xy -= halfrdx * half2(pR - pL, pT - pB);
}
```

边界条件

在第 38.2.4 节中，我们决定，在“一个方格中的流体”需要不穿越（no-slip）（零）速度边界条件和纯 Neumann 压力边界条件。在第 38.3.2 节中我们知道可以用下面的方法执行边界条件，即为存储边界值而保留网格的 1 个像素宽的周长。我们使用一个片元程序，适当地设

定数值，通过画在边界上的线基元，更新这些数值。

首先应该看一下网格的离散化如何影响边界条件的计算。不穿越条件规定在边界上的速度等于 0，而纯 Neumann 压力条件要求在边界的压力法向导数为 0。规定边界值落在边界单元和最近的内部单元之间的边上，而网格值定义在单元中心。因为任何满足边界条件的边有两个相邻单元，因此我们必须求两个单元的平均值才能得到边界值。

举例来说，对于左边的速度边界，我们有：

$$\frac{\boldsymbol{u}_{0,j} + \boldsymbol{u}_{1,j}}{2} = 0 \quad j \in [0, N] \tag{17}$$

这里 N 是网格分辨率。为了满足这个等式，我们必须设定 $\boldsymbol{u}_{0,j}$ 等于 $-\boldsymbol{u}_{1,j}$，压力方程式也同样。使用求导的向前微分的近似值，我们得到：

$$\frac{p_{1,j} - p_{0,j}}{\delta x} = 0 \tag{18}$$

在解这个方程求 $p_{0,j}$ 时，可以看到应该把每个压力边界值设定为正好在边界里的值。对于压力和速度边界，我们可以使用同一个简单的片元程序，如列表 38-5 所示。

列表 38-5　边界条件的片元程序

```
void boundary(half2 coords : WPOS,      // 网格坐标
              half2 offset : TEX1,      // 边界偏移
              out half4 bv : COLOR,     // 输出值
              uniform half scale,       // 缩放参数
              uniform samplerRECT x)    // 状态场
{
  bv = scale * h4texRECT(x, coords + offset);
}
```

图 38-6 示范了这个程序的工作过程。参数 x 表示一个纹理（速度或压力场），我们从那个纹理中读取内部值。*offset* 参数含有对一个内部单元的正确偏移，那个单元与当前边界相邻。

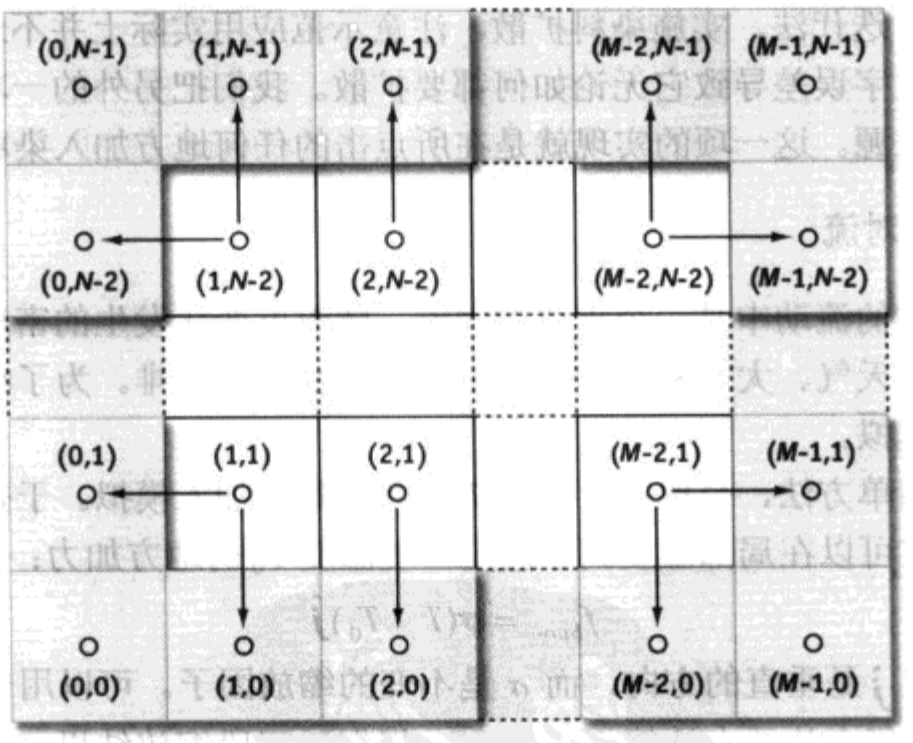


图 38-6　在一个 $M \times N$ 网格上的边界条件
箭头指出如何使用偏移去复制数值，即从恰好在边界里到边界单元来复制

coords 参数含有一个纹理坐标的位置，即片元程序正在处理的纹理坐标，因此把偏移加入该纹理坐标对附近的纹素寻址。在边界上，我们设定 offset 把纹理坐标调节到恰好在边界里的纹素。对于左边的边界，我们把它设定到（1, 0），所以它仅仅对右边的纹素寻址；对于底部边界，我们使用（0, 1），等等。scale 参数能用来缩放一个值，即我们复制到边界的值。对于速度边界，把 scale 设定成-1，而对于压力，它被设定成 1，这样我们分别正确地实现方程式 17 和 18。

38.4 应用

在这节中我们研究在本章中讨论的多种 GPU 模拟技术的应用。

38.4.1 模拟液体和气体

模拟技术最直接的使用是模拟连续的液体或气体。如果模拟只表现流体的速度，还不是很有趣的。如果把别的东西放入流体，就变得比较有趣了。示范应用是通过维持另一个标量场这样做的。这个场表现流体所携带的染料浓度（因为它是一个 RGB 纹理，它实际上是 3 个标量场：每个代表 3 种染料颜色中的一种）。像这样的量称为被动的标量，因为它们只是被流体携带向前，而不影响流体的流动。

如果 d 是染料的浓度，那么染料的变化由下列的方程式决定：

$$\frac{\partial d}{\partial t} = -(\boldsymbol{u} \cdot \nabla) d \tag{19}$$

为了模拟染料如何由流体所携带，我们把平流算子应用到标量场，正如对速度所做的一样。如果还要考虑染料在流体中的扩散，增加一个扩散项：

$$\frac{\partial d}{\partial t} = -(\boldsymbol{u} \cdot \nabla) d + \lambda \nabla^2 d + S \tag{20}$$

这里 γ 是染料在水中的扩散系数（或任何我们所假定的液体）。像计算速度的粘度扩散一样，我们使用 Jacobi 迭代法，实施染料扩散。注意示范应用实际上并不运行染料的扩散，只是因为平流项中的数字误差导致它无论如何都要扩散。我们把另外的一项 S 加入方程式 20，这一项代表染料的来源。这一项的实现就是在所点击的任何地方加入染料。

38.4.2 浮力和对流

温度在许多流体的流动中是一个重要的因素。随温度变化发生的密度变化引起了对流。这些对流影响我们的天气、大海和湖泊，甚至影响到我们的咖啡。为了模拟这些效果，需要把浮力加入我们的模拟。

加入浮力的最简单方法，是把关于温度 T 的新标量场加入模拟。于是我们可以插入一个另外的浮力算子，它可以在局部温度高于给定环境温度 T_0 的地方加力：

$$f_{buoy} = \sigma(T - T_0)\hat{j} \tag{21}$$

这个方程式中，$\hat{j}$ 是垂直的方向，而 σ 是不变的缩放因子。可以用一个简单的片元程序实现这个力，程序在每个片元上计算方程式 21，根据时间段缩放结果，再把它加入当前的速度中去。

烟和云

模拟烟所需要的一切，现在我们几乎都有了。这里我们介绍的烟模拟技术，类似于 Fedkiw 等 2001 上介绍的烟模拟技术。除了计算速度和压力场之外，烟模拟还必须有烟密度 d 和温度 T 的标量场。烟密度是由速度场平流输送的，正如我们先前描述的染料一样。把重力加在浓密的烟上，浮力修正为：

$$f_{buoy} = (-\kappa d + \sigma(T - T_0))\hat{j} \tag{22}$$

这里 κ 是质量比例常数。

在网格上给定位置添加上烟的密度和温度的来源（可能表现为一个烟囱或烟头），我们就可以模拟烟了。Fedkiw 等人的文章与我们的基本模拟有两点不同。他们使用一个交叉排列的网格改善准确性，还使用一个限制旋涡状态的力，来增加烟的旋涡运动。两者的扩展将在下一节讨论。

如 Harris 等 2003 所示的，一个更复杂的模拟能用来在 GPU 上模拟云。在图 38-7 中显示了 GPU 的 2D 云模拟的序列图片。云模拟把流体模拟和热力学模拟（包括浮力），以及一个水的蒸发和冷凝的模拟结合起来。这个 128 × 128 的云模拟是在 NVIDIA GeForce FX 5950 Ultra GPU 上，以每秒 80 多次迭代进行的。

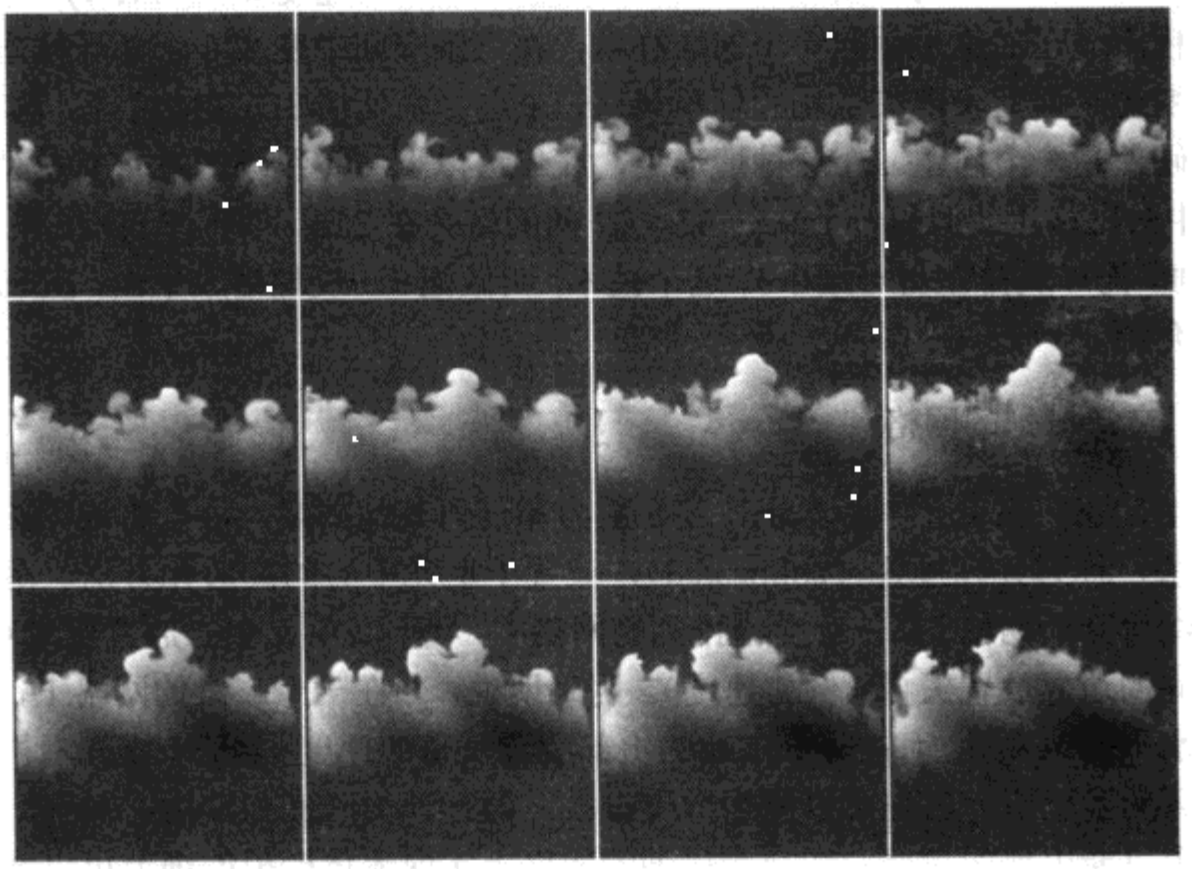


图 38-7　云模拟
从一个二维云模拟而来的一系列帧画面（各帧以 20 次迭代分开），在 GPU 上实现

38.5　扩展

在这章中所呈现的流动模拟是一块建筑积木，可以作为更复杂模拟的基础。可以用许多方法来扩展这个基础模拟。为了抛砖引玉，下面我们描述一些有用的扩展。

38.5.1 旋涡状态的限制

烟、空气和其他低黏度流体的运动，通常会含有一些大小不一的旋流。这种旋流是旋涡状态。如 Fedkiw 等人所说（Fedkiw et al. 2001），由粗网格模拟所引起的数字离散性，减弱了这些有趣的特性（Fedkiw et al. 2001）。因此，他们使用旋涡状态的限制，来恢复这些精细的运动。旋涡状态的限制，首先用于计算旋涡 $\boldsymbol{\omega} = \nabla \times \boldsymbol{u}$。从旋涡状态我们计算规范化的旋涡状态矢量场：

$$\boldsymbol{\Psi} = \frac{\boldsymbol{\eta}}{|\boldsymbol{\eta}|}$$

这里，$\boldsymbol{\eta} = \nabla |\boldsymbol{\omega}|$。在这个矢量场中的矢量，从低旋涡状态区域指向较高旋涡状态区域。从这些矢量我们计算一个力，它能用来恢复消散的旋涡的近似值：

$$\boldsymbol{f}_{vc} = \varepsilon(\boldsymbol{\Psi} \times \boldsymbol{\omega})\delta x$$

这里 ε 是一个用户控制的比例参数。“旋度”算子$\nabla\times$，可以使用梯度的定义和叉积算子推导出来。附带的源代码实现了旋涡状态限制。

38.5.2 三维

在这章中所出现的所有模拟都是二维的。我们可以把他们扩展到三维空间中去。方程式本质上保持相同，但是我们必须将其扩展，得到一个三维的速度 $\boldsymbol{u} = (u, v, w)$。必须根据这一点重写片元程序，在二维平面的 4 个邻居样本，在三维空间中变成 6 个邻居样本。

最大的变化是如何表现矢量场和标量场。一个选择是使用 3D 立体纹理。这对不支持 3D 浮点纹理的硬件是一个问题。在这种情形下，我们可以把 3D 纹理的片重复排列在一个存储在 2D 纹理中的网格上（例如一个 $32 \times 32 \times 32$ 的网格会在 256×128 的 2D 纹理上重复排列，8 块排在一维，而 4 块排在另一维中）。这个技术，叫做把 3D 纹理铺成平面，在 Harris 等 2003 中有详细的介绍。

38.5.3 交错排列的网格

在模拟中，我们在单元中心表现速度、压力和任何其他的量。这是将连续流体变成离散化的一种方法，称为并列排置（collocated），或置于单元中心的离散化方法。另外的一个方法是使用错综排列的网格。在一个交错排列的网格中，我们在单元中心放置标量（例如压力），而在单元之间的边界放置矢量（例如速度）。明确地说，即在一个二维的网格上，在每个单元的右边放置水平的速度 u，而在每个单元的上部放置垂直的速度 v。交错排列网格的离散化增加了许多计算的准确度。它也可以减少数值振荡，即把外力（例如浮力）加在一个置于单元中心的网格上可能出现的振荡。在交错排列的网格上实现流体模拟的细节可以在 Griebel 等 1998 中发现。

38.5.4 任意边界

到现在为止，我们一直假定流体存在于矩形方格中，矩形方格有平而实的边。如果需要边界的形状和位置是任意的，则需要扩展模拟。

构成任意的边界需要把边界条件应用于任意位置（边界条件已在第 38.3.3 节中讨论过）。这意味着，在每个单元我们必须决定把边界放在哪个方向，才能计算正确的边界值。这需要

在每个单元做更多的判断，导致模拟更加缓慢且更加复杂。但是这样可以创建许多有趣的效果，例如烟围绕障碍物的流动。甚至能加上运动的边界，见 Fedkiw et al. 2001。关于实现细节可以参考该论文和 Griebel et al. 1998。

38.5.5　流体的自由表面

我们所做的另一个假设是流体是连续的，所表现的惟一边界是方格的固定边界。因此我们不能够模拟像大海表面那样的事物，因为有一个水体和空气之间的界面。这类界面称为自由表面。要把模拟扩展到包含一个自由表面，需要在表面穿过单元时追踪它的位置。实现自由表面流体的方法可以在 Griebel et al. 1998 中找到。

38.6　小结

目前 GPU 的功能和编程能力，使它可以快速模拟各种各样的现象。许多现象的基础就是流体动力学现象。

在读过本章之后，大家应该基本了解了在 GPU 上实现基本流体模拟所需要的数学和技术。从这些思路出发，可以实验自己的模拟观念，并将流体模拟纳入图形应用。我们希望这些技术成为项目开发中有力的新工具。

38.7　参考文献

Bolz, J., I. Farmer, E. Grinspun, and E Schröder. 2003. "Sparse Matrix Solvers on the GPU: Conjugate Gradients and Multigrid." In *Proceedings of SIGGRAPH2003.*

Chorin, A.J., and J.E. Marsden. 1993. *A Mathematical Introduction to Fluid Mechanics.* 3rd ed. Springer.

Fedkiw, R., J. Seam, and H.W. Jensen. 2001. "Visual Simulation of Smoke." In *Proceedings of SIGGRAPH 2001.*

Golub, G.H., and C.F. Van Loan. 1996. *Matrix Computations.* 3rd ed. The Johns Hopkins University Press.

Goodnight, N., C. Woolley, G. Lewin, D. Luebke, and G. Humphreys. 2003. "AMultigrid Solver for Boundary Value Problems Using Programmable Graphics Hardware." In *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware* 2003.

Griebel, M., T. Dornseifer, and T. Neunhoeffer. 1998. *Numerical Simulation in Fluid Dynamics: A Practical Introduction.* Society for Industrial and Applied Mathematics.

Harris, M.J., W.V. Baxter, T. Scheuermann, and A. Lastra. 2003. "Simulation of Cloud Dynamics on Graphics Hardware." In *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware* 2003.

Kruger, J., and R. Westermann. 2003. "Linear Algebra Operators for GPU Implementation of Numerical Algorithms." In *Proceedings of SIGGRAPH* 2003.

Stam, J. 1999. "Stable Fluids." In *Proceedings of SIGGRAPH* 1999.

第39章 体渲染技术

Milan Ikits
University of Utah
Joe Kniss
University of Utah
Aaron Lefohn
University of California, Davis
Charles Hansen
University of Utah

这章将介绍基于纹理的体渲染技术，主要是用于三维数据集的可视化和高质量特效的创建。

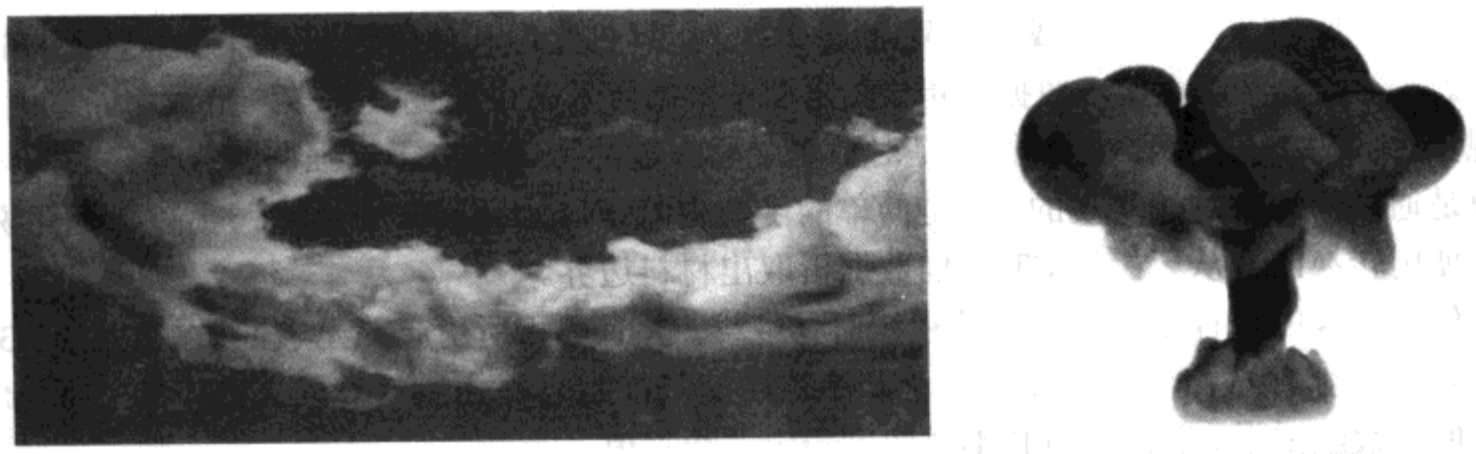
39.1 引言

许多自然界的视觉效果是体的。液体、云彩、火焰、烟、雾和灰尘很难使用基本几何元素建模，体模型更适于创建这些效果。这种模型假定体中的大量粒子可以发射、吸收和散射光线。图 39-1 显示两个这样的例子。

除了体现象的建模和渲染之外，体渲染在科学和工程中也有很重要的应用，许多情况下都要求把三维数据集视觉化。例如，将医学图像仪器获得的数据可视化，或流体力学模拟计算的数据视觉化。互动式体渲染应用程序的用户必须依靠现代图形加速器的运行，才能进行有效的数据探索并得到重要发现。

这章介绍了体渲染技术，它用到现代 GPU 灵活的程序模型和 3D 纹理化能力。虽然在 GPU 上还可能实现其他流行体渲染算法，例如光线投射（Roettger et al.2003，Kruger and Westermann 2003），但是这章只介绍基于纹理的体渲染。基于纹理的技术很容易与多边形算法相结合，只要求很少的渲染遍（render pass），又能提供很棒的交互性，还不需要牺牲渲染的质量。

39.2 节介绍了术语和解释直接体渲染的过程。39.3 节描述了典型的基于纹理的体渲染应用，并用一个简单例子加以说明。39.4 节提供了其他的实现细节，用于扩展基本体渲染能力。39.5 节介绍了更高级的技术，给渲染附加更逼真的光照效果和添加程序细节。39.6 节包含相关性能考虑的总评。



（a）积云的模拟　　（b）火焰的模拟

图 39-1　体效果

39.2　体渲染

直接体渲染法不需要显式地从数据里抽取几何表面，就可以产生 3D 体数据集的图像（Levoy 1988）。这些技术使用光学模型把数据值映射为光学性质，例如颜色和不透明度（Max 1995）。在渲染的时候，沿着每道视线累积光学性质，从而形成数据的图像（见图 39-2）。

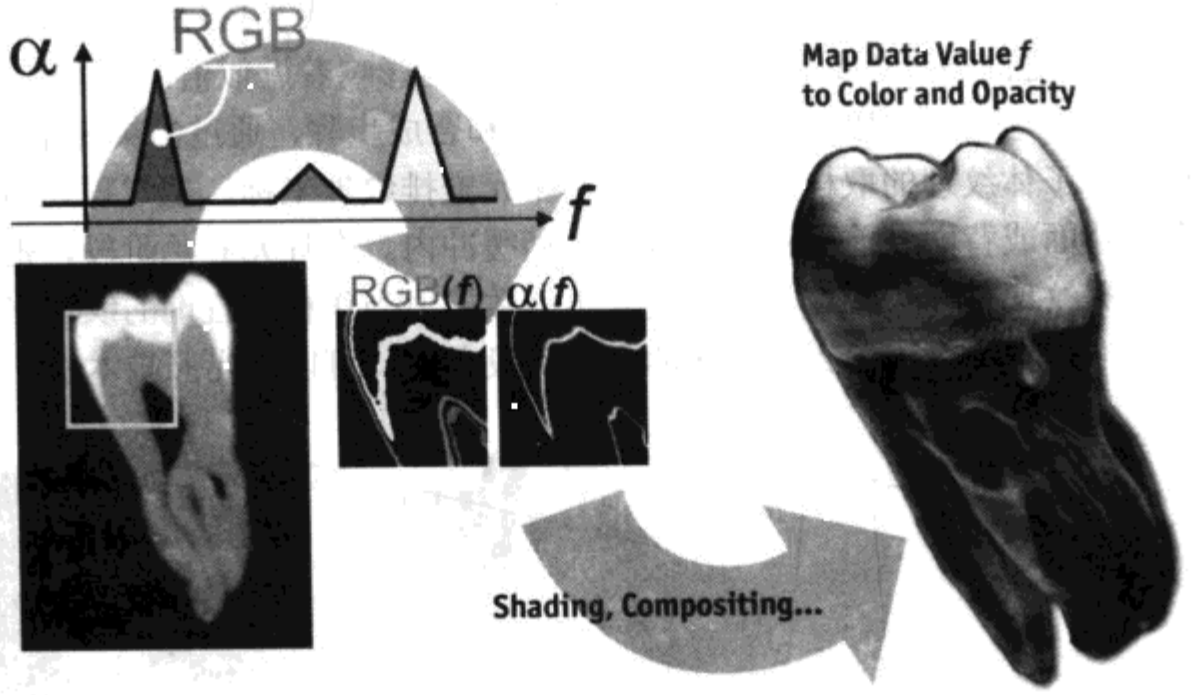


图 39-2　体渲染的过程

转换函数把数据值映射为颜色和不透明度，然后沿着视线方向着色和合成加以调节（感谢 Gordon Kindlmann 帮助渲染牙齿）

虽然一般把数据集解释为在空间的连续函数，但是在实际应用中，它统一由采样的 3D 数组代表。在图形内存中，体数据作为 2D 纹理片的堆栈而存储，或者作为单个的 3D 纹理对象存储。术语体素（voxel）指的是单独的“体元素”，类似于术语像素（pixel）为“图片元素”，和术语纹素（texel）是“纹理元素”。每个体素对应于数据空间中的一个位置，并且有 1 个或更多的数据值与它相关。中间位置的数据值通过在邻近的体元素上插值获得。这个过程即是重建，它在体渲染和处理应用中起着重要作用。

光学模型基本上是描述体中的粒子如何与光相互作用。比方说，最通用的模型假定体由

同时发射和吸收光的粒子组成。更复杂的模型加上局部光照和体阴影，而且还考虑光的散射效果。光学参数可以直接用数据值指定，或者将数据代入 1 个或更多的转换函数而计算出来。在视觉化应用中转换函数的目标，是对数据中感兴趣的特殊部分予以强调或分类。通常转换函数是通过纹理查询表实现的，但是简单的函数也能在片元 shader 中计算。图 39-2 举例说明如何使用转换函数从牙齿的 CT 扫描数据析取出材料边界。

创建图像的过程是沿着所有的视线对体取样，并且累积所引起的光学性质，如图 39-3 所示。对于发射-吸收模型，累积的颜色和不透明度是依照方程式 1 计算的，其中的 C_i 和 A_i 是由转换函数给采样 i 上的颜色和不透明度指定的数据值。

$$C=\sum_{i=1}^{n} C_i \prod_{j=1}^{i-1}(a-A_j)$$
$$A=1-\prod_{j=1}^{n}(1-A_j) \tag{1}$$

离散体的渲染方程式

不透明度 A_i 是吸收的近似值，而不透明度加权的颜色 C_i 则是沿着光线取样点 i 到 $i+1$ 之间发射和吸收的近似值。对于颜色成分，在加和式中的连乘表示取样点 i 发出的光到达眼睛之前所衰减的数量。把采样沿着视线排序，这个公式有效地迭代计算累积的颜色 C 和不透明度 A。39.4 节描述如何经过 alpha 混合执行合成步骤。因为方程式（1）是对连续光学模型的数字近似，所以与采样之间距离 l 成反比的取样率 s，极大地影响着近似值的准确性和渲染质量。

如图 39-3 所示，基于纹理的体渲染技术的取样和合成步骤，通过渲染体里 2D 几何体基本单元组进行。为了体纹理的取样，给每个基本单元安排纹理坐标。把代理几何体光栅化，并且以从前面到背面或背面到前面的次序混合到帧缓冲内。在片元着色阶段，把插值求出的纹理坐标用于数据纹理查询。接着，把插值求出的数据值当作相关查询的纹理坐标，代入转换函数纹理。在把产生的颜色送到流水线的合成阶段之前，可以用光照技术修正它。

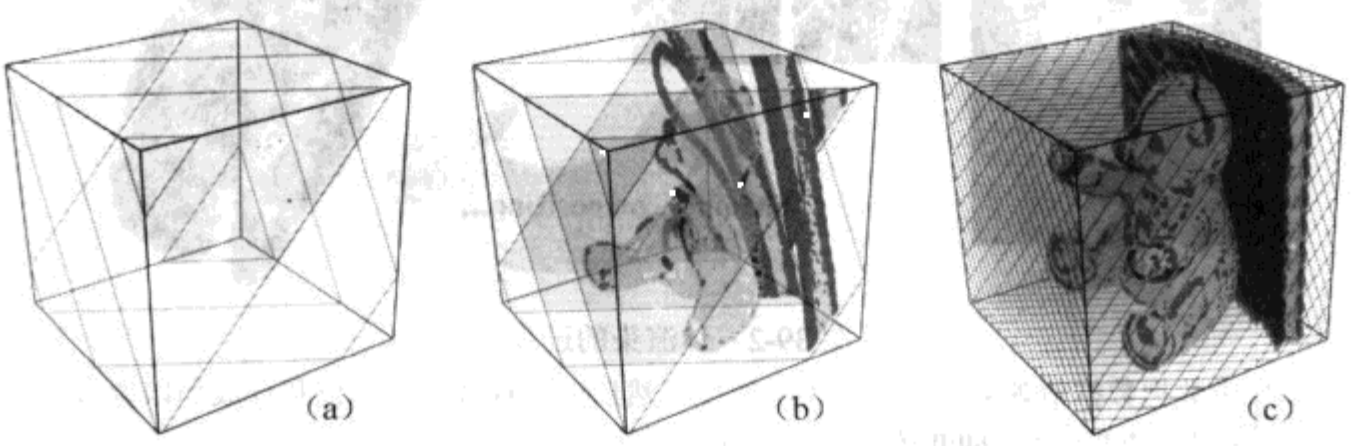


图 39-3 体的取样和合成

（a）通过渲染垂直于视线方向的代理多边形取得采样数据组；
（b）转换函数把颜色和不透明度指排为取样位置的数据值；
（c）沿着视线方向累积所指派的颜色和不透明度，产生数据的图像

39.3 基于纹理的体渲染

如图 39-4 所示，基于纹理的体渲染算法大体上分成 3 个步骤：（1）初始化；（2）更新；

（3）绘图。初始化通常只运行一次。而更新和绘图则要运行多次，每当应用程序接受用户输入，例如观察或渲染参数改变时，更新和绘图步骤就要重新运行一次。

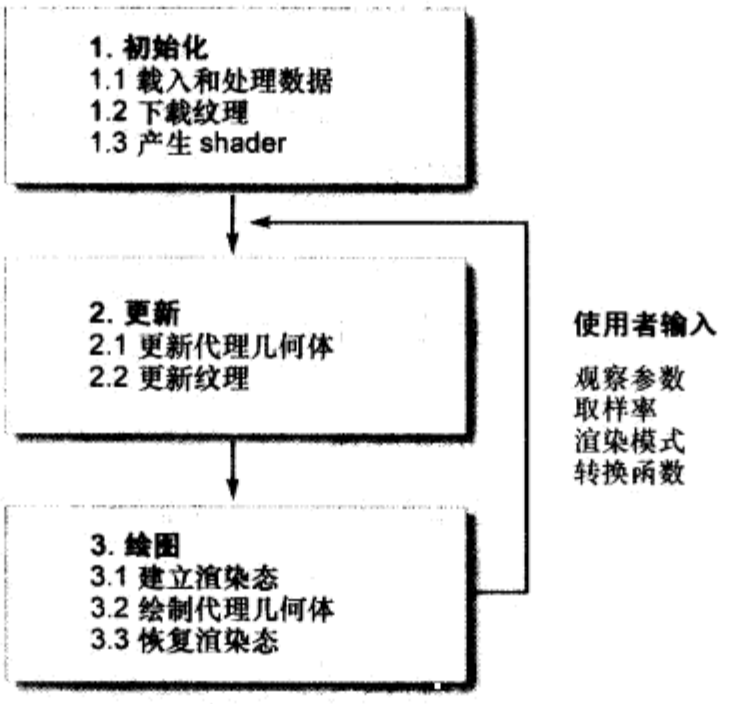


图 39-4　典型基于纹理的体渲染实现步骤

在应用开始，数据量被载入 CPU 内存。某些情况下，在把数据组压缩和下载到纹理内存之前需要对它们进行处理。例如用户可以选择计算梯度或减少该阶段的取样数据。也可以在应用之外做一些数据处理操作。转换函数查询表和片元 shader 通常在应用的初始化阶段创建。

在初始化后和每次观察参数发生变化时，都要重新计算代理几何体，然后把它们存储在顶点数组中。在把数据组当作 3D 纹理对象存储时，代理几何体由多边形组构成，而多边形则是垂直于视线方向把体切成薄片形成（见 39.4.2 节）。多边形薄片的计算分为两个步骤：首先薄片平面与体包围盒的边相交产生顶点，然后围绕多边形薄片的中心以顺时针或反时针方向把顶点排序。每个顶点对应的 3D 纹理坐标，借助于顶点程序或自动纹理坐标发生器在 CPU 上计算。

当把数据组存储为 2D 纹理薄片组时，代理多边形就是简单的长方形薄片。这种方法尽管速度很快，却存在一些缺点。第一，因为需要沿着每个主要方向复制数据薄片，所以还需要增加 3 倍的内存。消耗某些运行性能，在运行中（on the fly）重建薄片可以避免数据复制（Lefohn et al. 2004）。第二，取样率要视体分辨率而定。这个问题可以通过添加中间薄片数和用片元 shader 进行 3 次曲线插值来解决（Rezk-Salama et al. 2000）。第三，取样距离随视点而变，导致强度随摄像机的移动而变化，以及在变换薄片组时出现图像间断的失真现象（Kniss et al. 2002b）。

在更新阶段，如果渲染模式或转换函数参数变化，纹理就要被刷新。同时，如果取样率变化，就要校正转换函数纹理的不透明度（见方程式 3）。

在按照排序的顺序画多边形薄片之前，需要适当设置渲染状态。这个步骤通常包括禁用光照和剔除功能，以及建立 alpha 混合。为了在不透明的几何体中进行混合，必须激活深度检测，但必须禁用写入深度缓冲的功能。体和转换函数纹理必须约束于纹理单位，片元 shader 用纹理单位进行输入。在这时指定 shader 的输入参数，并为渲染建立顶点数组。最后，按照

排序的顺序画完薄片之后，就重新恢复渲染状态，使得算法不至于影响场景中其他物体的显示。

要理解基于纹理的体渲染的实现细节，可以用下面的例子作为出发点。在这个例子里，转换函数是固定的，数据组直接地代表不透明度，而发射颜色被设定为恒定的灰色。此外，观察方向沿着数据坐标系的 z 轴；因此，代理几何体由沿着 z 轴均匀放置的 x-y 平面的长方形组构成。算法由显示在算法 39-1 中的步骤构成。

算法 39-1　简单的体渲染步骤

1. 创建和下载作为 3D alpha 纹理的数据组。
2. 装载列表 39-1 中的片元程序。
3. 装载观察模型（modelview）和投影矩阵。
4. 对于源片元使用 1，而对于目标片元使用（1−源 alpha），激活 alpha 混合。
5. 禁用光照和深度检测功能（这个例子中没有不透明几何体）。
6. 把数据纹理约束于纹理单位 0。
7. 启用并约束片元程序，指定它的输入。
8. 沿着 z 轴画纹理的方块。

   x-y 的顶点坐标是(−1, −1), (1, −1), (1, 1), (−1, 1)

   对应的 x-y 纹理坐标是(0, 0), (1, 0), (1, 1), (0, 1)

   z 顶点和纹理坐标分别从−1 到 1、0 到 1 均匀增加。

列表 39-1　用于简单的体渲染的片元程序

```
void main(uniform float3 emissiveColor,
          uniform sampler3D dataTex,
          float3 texCoord : TEXCOORD0,
          float4 color : COLOR)
{
  float a = tex3D(texCoord, dataTex);   // 读 3D 纹理数据
  color = a * emissiveColor;            // 用不透明度乘以颜色
}
```

图 39-5 显示了由简单体渲染产生的图像。下面示范如何使该例中的每个阶段更适于各种应用。

图 39-5　使用熟悉物体的 Voxelized 模型进行简单体渲染

39.4　实现细节

这节概述基于纹理的体渲染中通常用到的各个量。目的是提供充分的细节，便于对流行图形硬件（例如 GeForce FX 系列的卡等）所用体渲染的实现有所了解。

39.4.1　数据的表达和处理

体数据组有不同的大小和类型。为了体渲染，应该把数据以适当的格式存储起来，这样就容易作为纹理下载到图形内存中去。通常体存储为单独的 3D 数组。根据代理几何体的类型，创建 3D 纹理对象，或者 1 到 3 组 2D 纹理的切片。开发者还必须选择有利于渲染的纹理格式。例如，通常使用 2 的幂作为纹理的尺寸，以达到最佳的渲染运行性能。数据组经常未处于正确的格式和尺寸，也不适于 GPU 上可行的纹理内存，所以需要进行处理。简单的实现中，在渲染以外的独立步骤进行数据处理。在更复杂的游戏的关中，例如在运行中产生数据值的时候，或从原始数据直接创建图像的时候，数据处理与应用变成一个整体。

为了改变数据组的大小，可以再取样，使它进入更粗的或更细的网格，或把它填补在边界上。填补就是把数据放进较大体，并用值填充空的区域。重取样需要访问体（即，在与体素相邻的位置插值计算数据值）。虽然普遍使用的 3 次曲线插值技术容易实现，但是它可能引起视觉失真，所以并不永远是重取样的最好选择。如果重取样的质量对于应用是决定性的，则需要更复杂的插值函数，例如分段三次多项式。所幸的是，这些操作很容易借助可获得的公有工具箱来运行。例如，Teem 工具箱包含各种可直接从命令行得到的数据处理工具，具有基础库的功能而不需要写任何代码（Kindlmann 2003）。使用 Teem 做体数据处理的例子在书所附的 CD 上和网站中有详细介绍。高级的数据处理也能在图形处理器上运行，例如创建高质量的图像（Hadwiger et al.2001）。

局部光照技术和多维空间转换函数在渲染时使用梯度信息。大多数的实现使用中心差法获得在每个体素的梯度矢量。中心差法是求梯度的近似法，用沿坐标轴两个相邻体素之间的数值差除以实际距离。例如，下列公式用于计算体素位置上梯度矢量的 x 分量 $\bar{p}_{(i,j,k)}$：

$$g_x(\bar{p}_{(i,j,k)}) = \frac{v(\bar{p}_{(i+1,j,k)}) - v(\bar{p}_{(i-1,j,k)})}{2h}, \tag{2}$$

用中心差法计算梯度

这里 h 是 x 轴上体素之间的距离。为了获得数据边界上的梯度，用重复的边界值填补体。中心差法所引起的视觉失真现象与 3 次曲线差值重取样所引起的类似。如果十分在乎视觉的质量，则需要更复杂的求导函数，例如 Teem 提供的那种。根据所用的纹理格式，可能需要量化、缩放和偏移所计算的梯度，以使它与可用值范围相适应。

通过把颜色和不透明度指派给数据值，转换函数可以强调体中的部位。直方图可以帮助分析数据中哪些范围的值更重要。大体上，直方图显示数据值的分布和其他有关的数据范围。创建一维直方图是把数值范围分割为许多小间隔。给每个小间隔指定上限和下限，每个间隔包含上下限之间的体素数。通过直方图可以看出哪些值经常出现。然而直方图不显示体中采样的空间分布。

数据处理步骤的输出是纹理组，在稍后阶段把它下载到图形处理器中。有时把几个纹理

结合成单个纹理更有效。举例来说，为了减少纹理查询和插值的消耗，经常把数据值和规范化的梯度纹理存储在一起，并且一起用在单个 RGBA 纹理中。

39.4.2 代理几何体

在渲染阶段，按照排序的次序画代理几何体创建体的图像。当数据组存储在 3D 纹理中时，用排列的观察平面把包围盒切片，产生多边形组进行体的取样。算法 39-2 可以通过观察模型（modelview）矩阵把顶点从物体坐标系转换到观察坐标系，从而计算观察空间中的代理几何体。为了更有效地渲染，代理多边形棋盘格化为三角形网格，把产生的顶点存储在顶点数组中。

算法 39-2 为了进行体渲染，用排列的观察平面切片

1．使用观察模型矩阵，把体包围盒顶点转变为观察坐标。

2．找到所转换顶点的最小和最大 z 坐标。使用与观察原点相等的间距，计算这两个值之间的取样面数目。取样距离由体素的大小和当前的取样速率计算。

3．以前面到背面或背面到前面的次序处理每个平面：

a．测试与包围盒的边相交的点。把各个相交点都加入暂时顶点列表。最多产生 6 个相交点，这样确定列表的最大尺寸。

b．对相交点求平均值，计算代理多边形的中心点。把相交点投影到 x-y 平面上，使用第一个顶点或 x 轴作参照，计算它们围绕中心点的角度，从而对多边形顶点按顺时针或反时针方向排序。为了避免三角法计算，角度正切和坐标符号可以结合成一个数量值（又被称为伪角），它可以用于顶点排序（Moret and Shapiro 1991）。

c．把代理多边形棋盘格化为三角形网格，并把产生的顶点加入输出的顶点数组。切片多边形可以棋盘格化为三角形条带，或以中心为公共点的三角形扇面。根据渲染算法的要求，这个步骤中可能需要把顶点转换回物体空间。

图 39-6 用两个切片多边形举例说明算法 39-2。第一个多边形包含 3 个顶点，第二个多边形有 6 个顶点并且棋盘格化为 6 个三角形。

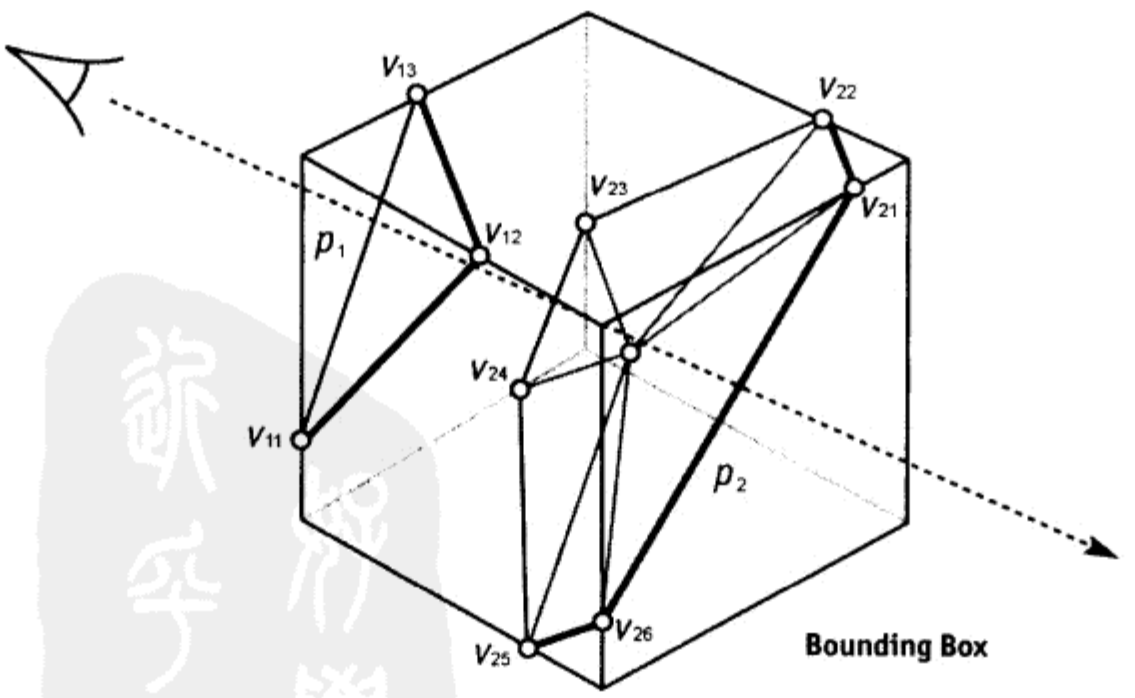


图 39-6 有两个取样平面的排列观察面切片

有几个方法可以为多边形顶点产生纹理坐标。例如可以利用算法 39-2 的第 3（c）步骤所计算的顶点位置和体包围盒，在 CPU 中计算纹理坐标。这时，可以用独立的顶点数组，或者与顶点数据交叉，把坐标传送到 GPU。也有不同的方法可以在 GPU 上计算纹理坐标，包括自动纹理坐标的产生、纹理矩阵或顶点程序等。

39.5 节中所描述的那种高级算法，可以使用与观察方向不同的切片轴。在这种情况下，算法工作方式相同，但是观察模型（modelview）矩阵需要被适当地修正。

39.4.3　渲染

转换函数

转换函数的作用是强调数据的特征，做法是把数据值和其他的数据测量映射为光学性质。最简单和最广泛使用的转换函数是一维的，而且把数据值映射为颜色和不透明度。这些转换函数通常用一维纹理查询表实现。在建立查询表时，颜色和不透明度通常分别由转换函数指派。因为颜色近似于一段光线中的发射和吸收，所以为了正确渲染，颜色组分需要乘以不透明度（称为不透明度加权的颜色）（Wittenbrink et al. 1998）。

列表 39-2　使一维转换函数与纹理相关的片元程序

```
void main(uniform sampler3D dataTex,
          uniform sampler1D tfTex,
          float3 texCoord : TEXCOORD0,
          float4 color : COLOR)
{
  float v = tex3d(texCoord, dataTex);   // 读 3D 纹理和转换函数纹理
  color = tex1d(v, tfTex);              // 转换函数纹理
}
```

仅仅使用数据值来控制颜色和不透明度的分配，可能会限制将数据特征分类的效果。将其他的数据表达方式（例如梯度）纳入转换函数，可以更精细地控制和得到更精确的视觉效果（Kindlmann and Durkin 1998，Kindlmann 1999）。图 39-7 举例说明基于数值和梯度的一维转换函数和二维转换函数之间的不同。

（a）　（b）　（c）　（d）

图 39-7　一维和二维转换函数之间的不同
用基于数值和梯度大小的二维转换函数来区分数值的不同特征（b, d），用一维的转换函数不可能做到（a, c）

转换函数设计是困难的迭代近似程序，它要求全面洞察所关系到的数据组。数据值的直方图提供一些信息，指出哪些范围的数据值应该予以强调。用户接口是互动式设计程序的重要组成部分。通常接口由一个一维的曲线编辑器构成，借助一组控制点指定转换函数。另外的方式是使用直接处理窗口小装置，直接地画转换函数纹理（Kniss et al. 2002a）。图 39-7 下部的图像说明了后一种技术。小装置提供的视图，沿水平轴表现数据值的分布，而沿垂直轴表现梯度大小。在数值和梯度大小分布图里的拱形指出了材质的边界。一个小树丛用来绘制二维转换函数相关的纹理，它通过相应数值范围和梯度大小，把产生的颜色和不透明度分配给体素。

所分配的不透明度也与取样率相关。举例来说，当使用较少的切片时，必须放大不透明度，才能保持图像的整体强度相同。每当用户改变取样率 s，使它偏离基准取样率 s_0 时，方程式 3 用来校正转换函数的不透明度：

$$A = 1 - (1 - A_0)^{s_0/s} \tag{3}$$

校正不透明度的公式

为了控制渲染的质量和速度，可以通过用户接口改变最大切片数，或相对取样率 s / s_0。

光照

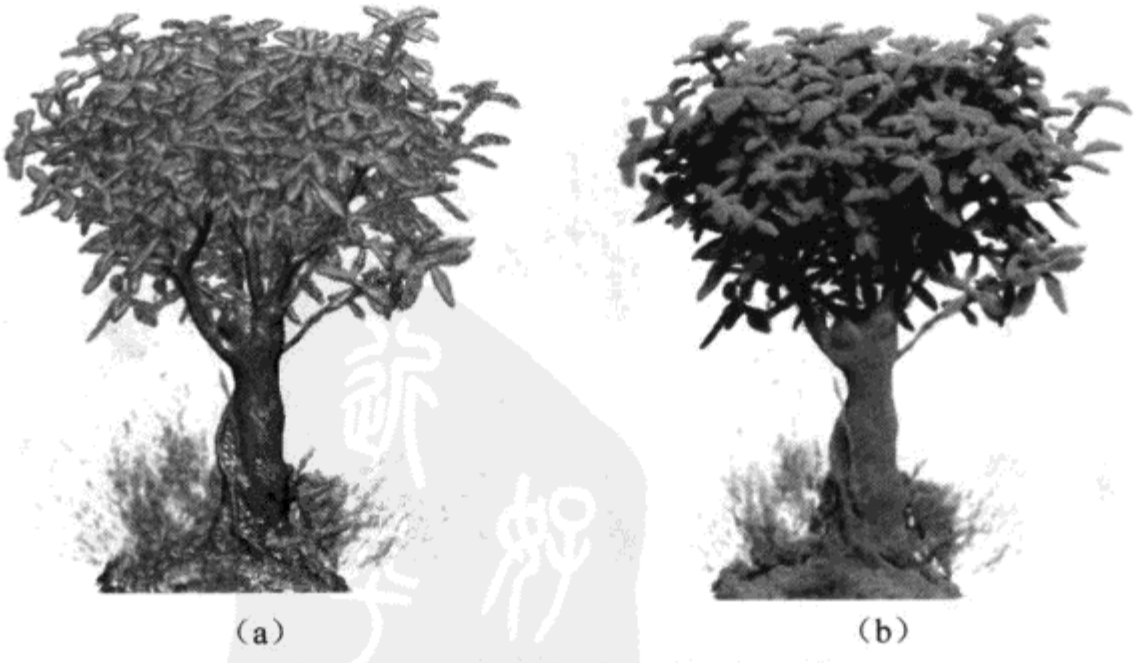
光照模型用来改善物体的视觉表现。简单模型局部地近似于从物体表面反射的光强度。最普通的近似是 Blinn-Phong 模型，它使用局部表面法线 $\vec{n}$ 、方向 $\vec{l}$ 、点光源强度 i、环境系数 k_a、漫反射系数 k_d、镜面高光反射系数 k_s 和发光系数 n 的函数，计算反射强度：

$$I = k_a + I_L k_d (\vec{l} \cdot \vec{n}) + I_L k_s (\vec{h} \cdot \vec{n})^n \tag{4}$$

局部光照 Blinn-Phong 模型

所计算的强度用来调整来自转换函数的颜色组分。在片元 shader 中计算方程式 4，需要每个像素的法线信息。在体渲染应用中，规格化的梯度矢量被当作表面法线使用。不幸的是，在均匀的体中梯度不能很好地确定。为了体渲染，时常对 Blinn-Phong 模型做些修正，只让梯度大的区域产生明暗效果（Kniss et al. 2002b）。

局部光照忽略间接光的存在、阴影和其他全局效果，如图 39-8 所示。39.5 节描述如何将简单的全局光照模型纳入渲染模型，从而创建高质量的体渲染效果。



（a） （b）

图 39-8 比较简单的和复杂的光照效果

（a）在体渲染中使用 Blinn-Phong 模型；（b）简单的光照模型不能取得重要的全局效果，例如自阴影和半透明效果

合成

为了有效地计算体渲染方程式（方程式 1），把采样从背面到前面排序，而且迭代地计算累积的颜色和不透明度。合成过程的单个步骤即称为上算子（Over operator）。

$$\begin{aligned}\hat{C}_i &= C_i + (1 - A_i)\hat{C}_{i+1} \\ \hat{A}_i &= A_i + (1 - A_i)\hat{A}_{i+1}\end{aligned} \tag{5}$$

从前面到背面的合成方程式

这里，C_i 和 A_i 分别是从片元着色阶段沿着视线获得片段 i 的颜色和不透明度，而 $\hat{C}_i$ 是从体背面开始所累积的颜色。如果采样从前面到背面排序，那么使用下算子（Under operator）：

$$\begin{aligned}\hat{C}_i &= (1 - \hat{A}_{i-1})C_i + \hat{C}_{i-1} \\ \hat{A}_i &= (1 - \hat{A}_{i-1})A_i + \hat{A}_{i-1}\end{aligned} \tag{6}$$

从前面到背面的合成方程式

这里，$\hat{C}_i$ 和 $\hat{A}_i$ 分别是从体的前面到背面累加的颜色和不透明度。

用硬件 alpha 混合实现方程式的合成是很容易的（方程式 5 和 6）。对于上算子，源混合因子设定成 1，而目的混合因子设定成（1−源 alpha）。对于下算子，源混合因子设定成（1−目的 alpha），而目的混合因子设定成 1。如果允许硬件在相同的缓冲读和写，也可以通过把代理多边形顶点投影到视口矩形上，在片元着色阶段运行合成（见 39.5 节）。

为了把不透明几何体混合进体，需要在体之前画几何体，因为深度检测将会剔除在物体里的代理片元。下算子（Under operator）需要把几何体和体画入独立的颜色缓冲之内，最后再把它们合成起来。在这种情况下，用几何体通道的深度值来剔除体渲染通道里的片元。

39.5　高级技术

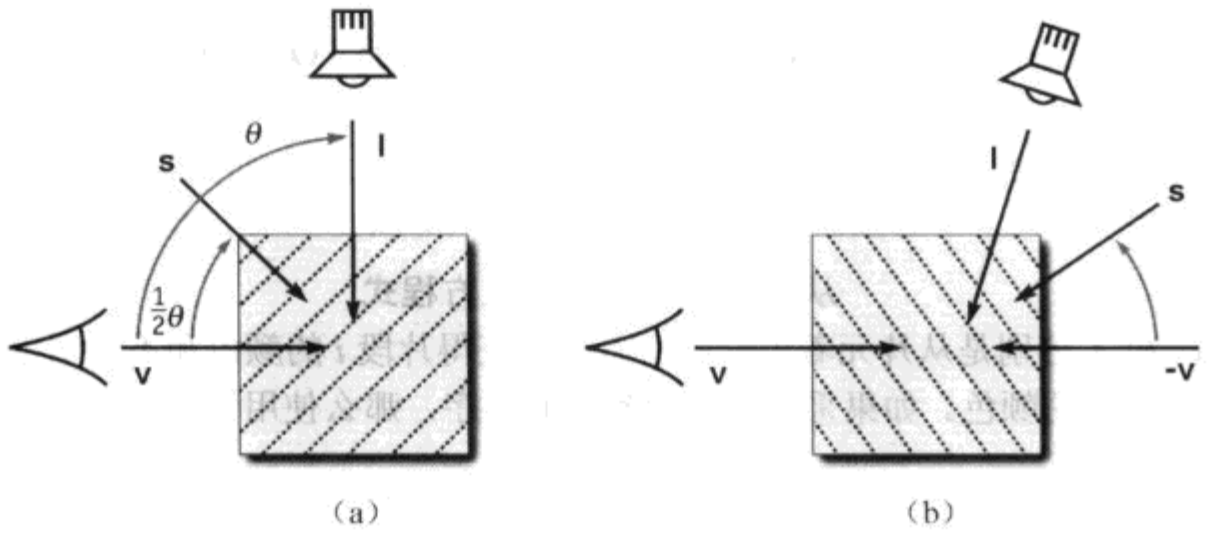
这节描述的技术用于改进渲染质量和创建体的特效。

39.5.1　体光照

前面提出的局部光照模型给渲染加入了重要的视觉提示。但是它假定光到达样点时，不与体的其他部分相互作用，所以这个模型虽然简单，却不切合实际。此外，这种光照假定的模型基于表面，未对体的材质予以适当考虑。添加复杂光照效果的方法（例如加上体阴影），是预先计算阴影体，存储经过材质衰减之后到达每个样点的光量。渲染时，用这个体阴影贴图的内插值乘以转换函数求得的颜色。但是这个方法除了使用额外的内存之外，体阴影贴图还会造成视觉的失真现象，比如阴影模糊和图像变暗。

更好的代替办法是使用像素缓存，累积从光源观点看所衰减的光量（Kniss et al. 2003）。为了有效地做到这一点，把切片轴设定在视线和光线的中间，如图 39-9（a）所示。这允许从眼睛和光源两个观点来渲染相同的切片。到达某个特定切片的光线数量，等于 1 减去前面各切片的不透明度的加和。每个切片首先从眼睛观点来渲染，用从光源观点渲染的前面通道结果，调整当前切片上样点的亮度。接着从光源观点渲染同一个切片，计算到达下一个切片

的光强度。算法 39-3 使用两个缓冲：一个用于眼睛，一个用于光源。图 39-10 表现在算法 39-3 中所描述的设置。



（a） （b）

图 39-9 为增加的光照计算所做的半角切片

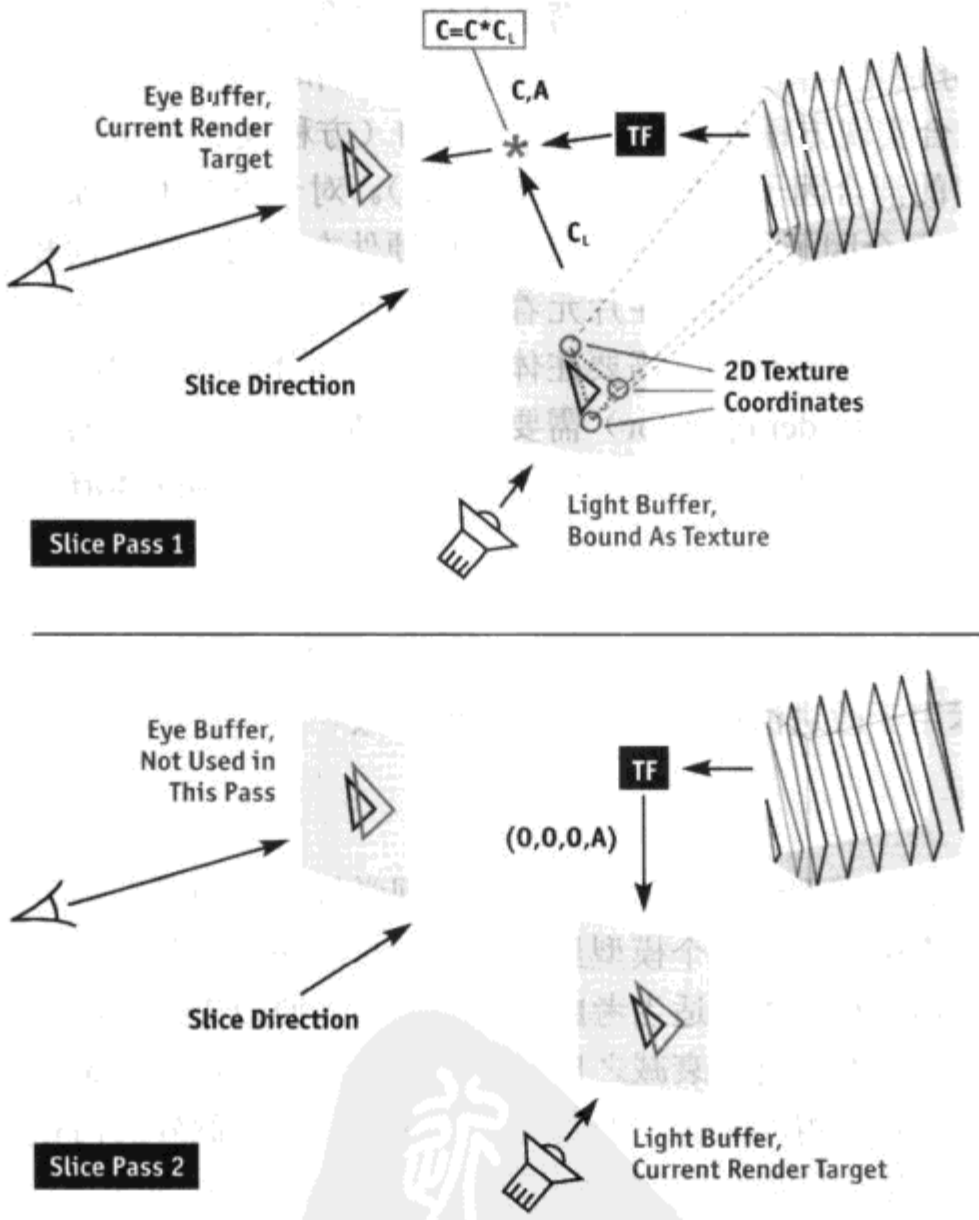


图 39-10 2 遍的体阴影设置

算法 39-3 有阴影的两个通道的体渲染

1．把眼睛缓冲清 0，把光源缓冲初始值设定为光源颜色 C_L。为创建聚光灯之类的特效，可以使用纹理贴图初始化光源缓冲。

2．使用算法 39-1 计算物体空间的代理几何体。当光源和视线方向的点积为正的时候，把切片方向设定在光源和视线方向的中间，如图 39-9（a）所示。在这种情况下，对于眼睛而言从前面到背面渲染体使用下算子（见第 39.4.3 节）。当点积为负的时候，切片在光源和视线反方向的中间，使用上算子从背面到前面渲染体（见图 39-9（b））。在这两种情况下，对于光源而言从前面到背面渲染体都使用上算子。

3．对于每个切片，

a．通道 1：把切片渲染并混合进眼睛缓冲之内。

i．使用光源的观察模型和投影矩阵把切片顶点投影到光源缓冲。根据光源的视口大小和光源缓冲将产生的顶点位置转换成 2D 纹理坐标。

ii．把光源缓冲当作纹理局限于一个可行的纹理单位，并且使用在 3（a）（i）步骤中所计算的纹理坐标。数据纹理查询也需要 3D 纹理坐标组。

iii．在片元 shader 中计算转换函数，求出反射的颜色 C 和不透明度 A。接着，用从光源缓冲来的颜色 C_L 乘以 C，并使用对当前切片方向合适的算子把 C 和 A 混合进眼睛缓冲之内。

b．通道 2：使用上算子把切片渲染和混合进光源缓冲之内。在片元 shader 中，为求 alpha 组分计算转换函数，并且把片元颜色设定为 0。

如图 39-11 所示，体阴影极大地改善了渲染场景的逼真度。注意，当观察方向和光源方向之间的夹角改变时，为了维持取样率恒定，需要调整切片间隔。如果沿观察的视线方向所要求的取样距离是 d_v，在观察方向和光源方向之间的角度是 θ，切片间隔 d_s 由方程式（7）给出：

$$d_s = \cos(\theta/2)d_v \tag{7}$$

在半角切片时，切片间隔和取样距离之间的关系

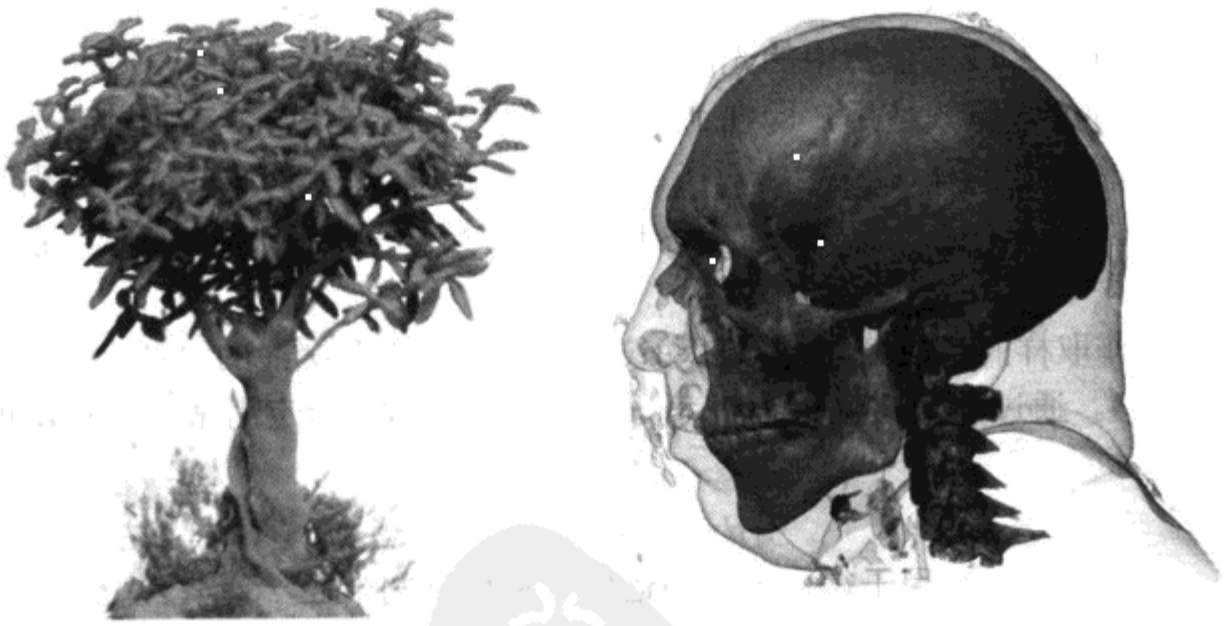


图 39-11　带阴影的体渲染的例子

不幸的是，因为忽略了体内部散射光的贡献，算法 39-3 还是产生发暗的失真图像。散射效果可以完全基于体光照模型从物理上取得，可惜这对互动式渲染太复杂。但是把算法 39-3 扩展到近似某种散射现象还是有可能的。这种现象是半透明，是光传播进材质并在其中散射的结果。一般散射计算要考虑从所有方向来的入射光，而对于半透明只需考虑光源方向圆锥体内的入射光就足够了，如图 39-12 所示。这个简化结果反映这样一个事实：在特殊样点上

间接散射光量的大小，取决于算法 39-3 在前面迭代中计算的局部邻居采样区域。因此，在体中切片之间传播和模糊间接光照光，有可能合成半透明效果。

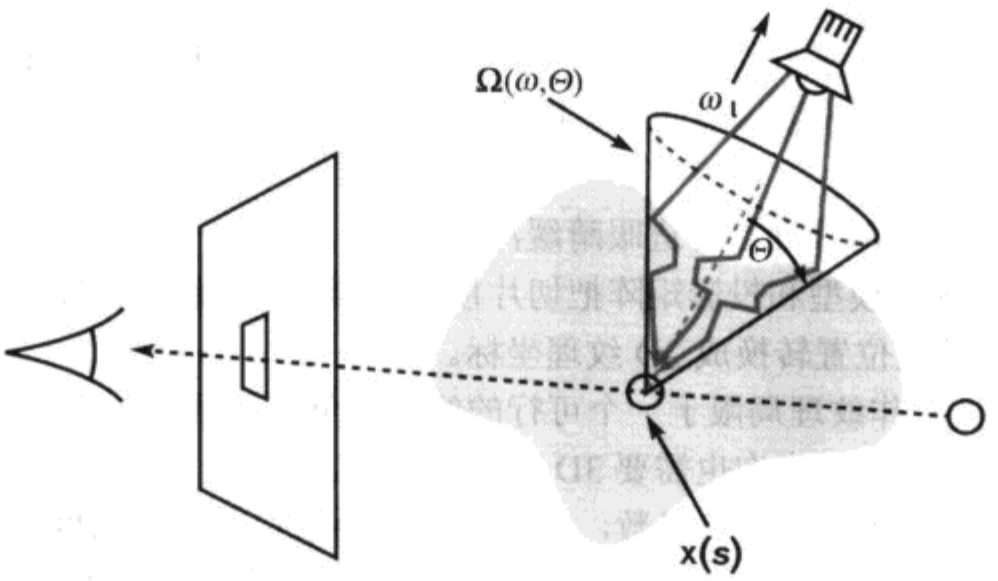


图 39-12 为近似半透明所做的设置

为了将半透明纳入算法 39-3，第一，添加间接衰减参数 A_i。这些参数是对于每个 RGB 颜色通道的 alpha 值，与算法 39-3 中所用的单个的 alpha 值 A 相对应。第二，使用 $1-C_L$ 代替光源颜色 C_L，作为光源缓冲的初始值。第三，在第 3（a）（iii）步骤中，改为用 $1-A_i$（即 1 减去从光源缓冲读出的颜色值）乘以 C。在第 3（b）步骤中，用 A_i 代替 0 设定为片元 shader 的颜色，并用下列混合操作代替上算子（Over operator）：

$$C = C_1 + (1 - C_1)C_0 \tag{8}$$

用于半透明模型的光源缓冲合成

在方程式 8 中，C_1 是入射片元的颜色，而 C_0 是在目标渲染缓冲中当前的颜色。

接着，在更新第 3（b）步骤中的光源缓冲内容时，用另一个缓冲来模糊间接衰减成分。这两个缓冲以某种交替方式使用，即在为眼睛的第 3（a）步骤中，对当前光源缓冲取样一次，而在为光源的第 3（b）步骤中，对当前光源缓冲取样多次。下一个光源缓冲是 3（b）步骤的渲染目标。在每个切片之后都改变这种关系，这样，下一个缓冲变成当前的缓冲，反之亦然。

为了模糊操作取样间接成分，使用扰动纹理修正当前光源缓冲的纹理坐标（对所有的纹理单位进行修正）。通过用户定义的模糊角 φ 和采样距离 d，指定用来缩放扰动的模糊圆半径 R：

$$R = d \tan(\varphi / 2) \tag{9}$$

用于得到半透明近似值的模糊圆半径

在 3（b）步骤的片元 shader 中，使用修正过的纹理坐标读当前的光源缓冲。对从光源缓冲读出的值求加权和得到模糊的衰减，接着把它混合入下一个光源缓冲之内。

图 39-13 是显示使用鱼的 CT 数据组做半透明体渲染的例子。这个技术对于创建云等其他大气现象的逼真图像很重要（见图 39-1 和 39-l4（a））。因为 3（b）步骤中每个颜色成分有独立的不透明度，有可能调整 A_i 的值，使它比 3（a）步骤所用的 alpha 值 A 更小或更大，从而控制光进入某个区域内的量。当 A_i 比 A 小时，光较深地穿入体之内，即使用眼睛观察，材质也表现出很强的视觉反差。

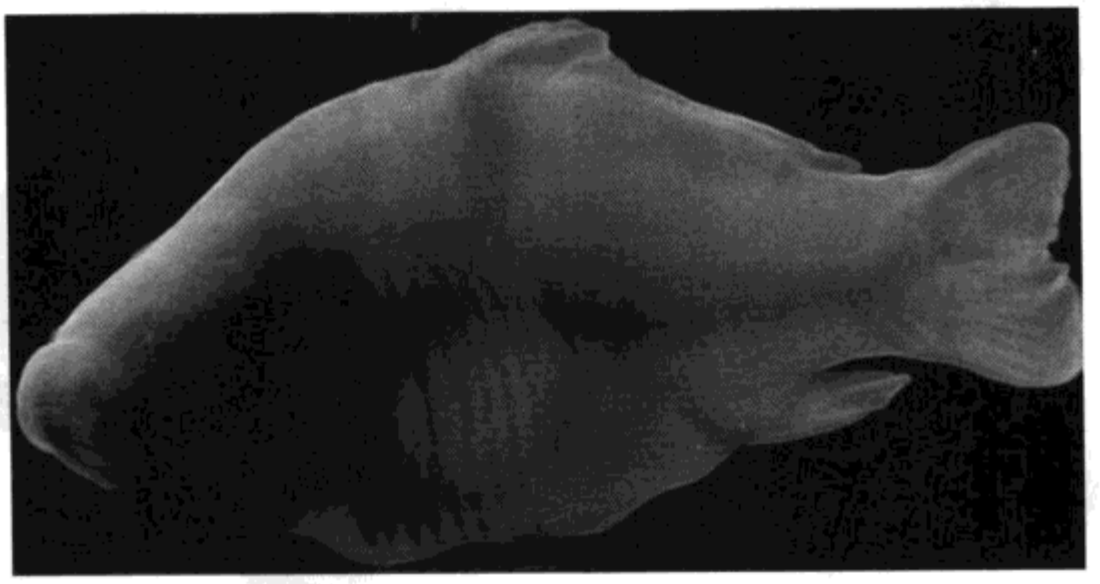


图 39-13　半透明材料的体渲染的例子

同时，因为对每个彩色通道指定一个独立的 alpha，光在较深地穿入体内时会改变颜色。这很像握在手里的电筒效果：入射的光是白色的，而射出的光是红色的。让红光的 A_i 值小于绿光和蓝光的 A_i 值，使得红光组分比绿光的和蓝光组分衰减更小，可以形成这种效果。

使用 2D 缓冲计算体光在屏幕空间的传输，有许多好处。它可以使光传播的分辨率与视口相匹配，产生失真最小的明晰阴影。这节提出的方法把光传输分辨率与 3D 数据网格分离开，而且允许程序化体纹理效果使用正确的光照模型，如下节所述。

39.5.2　程序化渲染

体渲染的缺点之一是低分辨率体不能够表现小的高频细节。对云、烟、树、头发和毛皮等许多体现象而言，要得到他们的特征，高频细节是必不可少的。程序化噪波模拟是给低分辨率体数据添加细节的有效技术（Ebert et al. 2002）。一般方法是对宏观结构使用粗糙的模型，而对于微细结构使用程序化噪波。下面介绍两个方法把程序化噪声加入基于纹理的体渲染。第一个方法扰动着色阶段的光学性质；第二个方法扰动体本身。

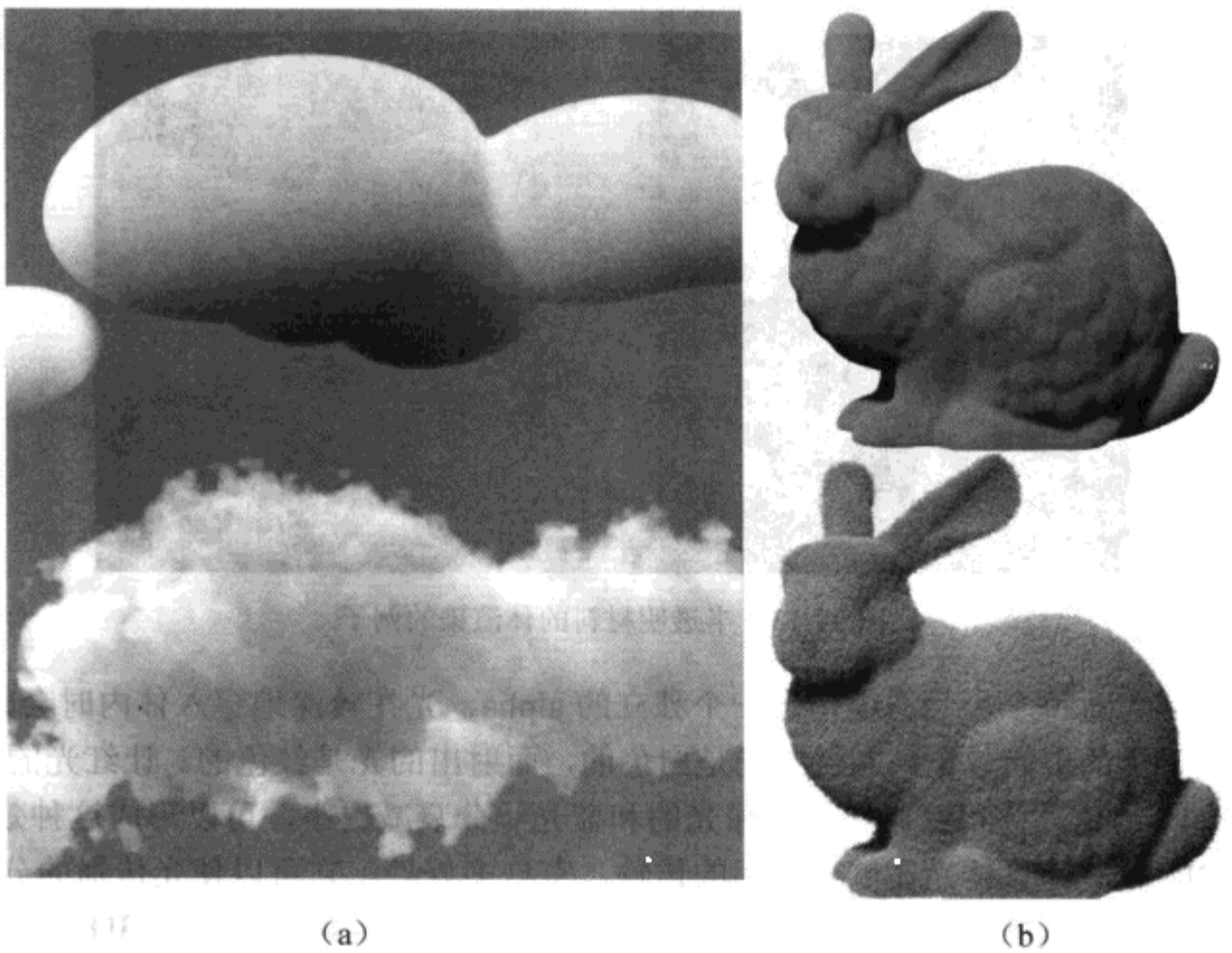
两个方法都使用一个小噪波体。在这个体中，每个体素的初始值设定为 4 个任意数，作为 RGBA 分量存储，并且稍加模糊以隐藏由 3 次曲线差值所引起的失真。由于在渲染时每个切片缩放为不同大小，因此应使用噪波纹理的不同复本。计算各个噪波组分的加权并求得每个像素的扰动。为了动画扰动，把变化的偏移矢量添加到片元 shader 中的噪波纹理坐标上去。

第一个技术在计算转换函数之后，使用各个像素的 4 个噪声分量修正体的光学性质，这样可以使材质表现出不规则性。通过选择所修正的光学性质，可以得到不同的效果。

第二种方法使用噪波修正数据在体中存取的位置（Kniss et al. 2003）。在这种情况下，噪波纹理的 3 个组分形成一个矢量，把它加到纹理坐标上去，用于数据纹理查询。图 39-14 举例说明如何用体扰动把复杂的细节加入粗糙的体模型。

39.6　对性能的考虑

基于纹理的体渲染很容易使现代 GPU 达到运行极限。这节将对 GPU 上体渲染做些特殊考虑。



（a） （b）

图 39-14 体扰动技术的例子

（a）最初的低分辨率数据组是用多个椭圆产生的，它们定义了大尺寸云的结构；（b）使用体扰动增加“史丹福 *Stanford* 兔子”的毛皮效果

与大多数图形应用不同，基于纹理的体渲染使用少量相当大的几何体基本单元。光栅化在每个基元中产生许多片元，很容易变成流水线的瓶颈。此外，用于体渲染的透明代理几何体不像不透明物体，它不能够使现代 GPU 早期深度剔除能力发挥杠杆式作用。由于渲染高质量图像需要大量切片，光栅化瓶颈还会进一步恶化。另外的麻烦是，每当用片元 shader 或合成硬件处理片元时，还必须向后读一次合成阶段帧缓冲的内容。

基于这些原因，一定要注意避免产生不必要片元的代理几何体。简单地画个大的方框覆盖体，会使得实现非常缓慢。缩小视口、降低取样率、使用预先整合的分级（Engel et al. 2001），以及不画体的空白区域（Li et al. 2003，Kruger and Westermann 2003）等措施可以减小光栅化的压力。跳过空白的区域能有效地在所用几何体和片元处理带宽之间求得平衡。

片元程序的限制

片元 shader 的复杂性极大地影响着体渲染的运行性能。预先计算的查询表可能比有许多复杂指令的片元程序要快。然而，相关的纹理读出，可能延迟流水线的运行，明显地减小渲染速度。要想达到最高的运行速度，需要发现片元操作和纹理阅读之间的正确平衡，这可是一件挑战性的剖析（profiling）工作。

纹理内存的限制

在体渲染中使用 3 次曲线插值（当运行 mipmaps 时，使用 4 点式曲线 quadrilinear），至少需要 8 次纹理查询，这比标准 2D 纹理映射所用的双线插值更耗时（2D 纹理映射运行

mipmaps 时，使用 3 次曲线）。另外，高速缓冲存储器用于 2D 纹理时，能有效地隐藏纹理内存访问的反应时间，用于大 3D 纹理时就不那么有效了。当渲染的速度至关重要时，较小的纹理、纹理的压缩，以及较低的精度类型可以减少对纹理内存子系统的压力。最近已经出现有效的压缩方案，它既能达到很高的纹理压缩比，又不影响渲染的运行性能（Schneider and Westermann 2003）。最后，在现代 GPU 上，算法和内存系统可以同时操作一个 RGBA 元组的所有值。把数据装进 RGBA 元组之内，能够减少对带宽的要求，从而提高运行性能。

39.7　小结

体渲染是一种重要的图形和可视化技术。它不但能够显示模型的表面，而且能够显示包含在里面的复杂细节。这章的前半部分用排列的观察面上的代理几何体，介绍基于纹理的体渲染的典型实现。39.5 节给出了建立在基本实现上的两个高级技术。通过把体阴影、半透明效果和随机细节加到标准渲染模型中去，改善了图像质量。

体渲染已经问世逾十载了，现在仍然是图形和可视化研究的一个活跃领域。对更多细节和最新技术感兴趣的读者可以参考相关的文献。

39.8　参考文献

Ebert, D., F. Musgrave, D. Peachey, K. Perlin, S. Worley, W. Mark, and J. Hart. 2002. *Texturing and Modeling: A Procedural Approach.* Morgan Kaufmann.

Engel, K., M. Kraus, and T. Ertl. 2001. "High-Quality Pre-Integrated Volume Rendering Using Hardware-Accelerated Pixel Shading." In *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2001*, pp. 9～16.

Hadwiger, M., T. Teußl, H. Hauser, and E. Groller. 2001 "Hardware-Accelerated High-Quality Filtering on PC Hardware." In *Proceedings of the International Workshop on Vision, Modeling, and Visualization* 2001, pp. 105～112.

Kindlmann, G., and J. Durkin. 1998. "Semi-Automatic Generation of Transfer Functions for Direct Volume Rendering." In *Proceedings of the IEEE Symposium on Volume Visualization*, pp. 79～86.

Kindlmann, G. 1999. "Semi-Automatic Generation of Transfer Functions for Direct Volume Rendering." Master's Thesis, Department of Computer Science, Cornell University.

Kindlmann, G. 2003. The Teem Toolkit. **http://teem.sourceforge.net**

Kniss,J., G. Kindlmann, and C. Hansen. 2002a. "Multidimensional Transfer Functions for Interactive Volume Rendering." IEEE Transactions on Visualization and Computer Graphics 8(4), pp. 270～285.

Kniss, J., K. Engel, M. Hadwiger, and C. Rezk-Salama. 2002b. "High-Quality Volume Graphics on Consumer PC Hardware." Course 42, ACM SIGGRAPH.

Kniss, J., S. Premo e, C. Hansen, P. Shirley, and A. McPherson. 2003. "A Model for Volume Lighting and Modeling." *IEEE Transactions on Visualization and Computer Graphics* 9(2), pp.

150～162.

Kruger, J,, and R. Westermann. 2003. "Acceleration Techniques for GPU-Based Volume Rendering." In *Proceedings of IEEE Visualization,* pp. 287～292.

Lefohn, A., J, Kniss, C. Hansen, and R. Whitaker. 2004. "A Streaming Narrow-Band Algorithm: Interactive Deformation and Visualization of Level Sets." *IEEE Transactions on Visualization and Computer Graphics* (to appear).

Levoy, M. 1988. "Display of Surfaces from Volume Data." *IEEE Computer Graphics & Applications* 8(2), pp. 29～37.

Li, W., K. Mueller, and A. Kaufman. 2003. "Empty Space Skipping and Occlusion Clipping for Texture-Based Volume Rendering." In *Proceedings of IEEE Visualization,* pp. 317～324.

Max, N. 1995. "Optical Models for Direct Volume Rendering." *IEEE Transactions on Visualization and Computer Graphics* 1(2), pp. 97～108.

Moret, B., and H. Shapiro. 1991. *Algorithms from P to NP.* Benjamin Cummings.

Rezk-Salama, C, K. Engel, M. Bauer, G. Greiner, and T. Erd. 2000. "Interactive Volume Rendering on Standard PC Graphics Hardware Using Multi-Textures and Multi-Stage Rasterization." In *Proceedings of the SiGGRAPH/Eurographics Workshop on Graphics Hardware 2000,* pp. 109～118.

Roettger, S., S. Guthe, D. Weiskopf, T. Erti, and W. Strasser. 2003. "Smart Hardware-Accelerated Volume Rendering." In *Proceedings of the Eurographics/IEEE TVCG Symposium on Visualization,* pp. 231～238.

Schneider, J., and R. Westermann. 2003. "Compression Domain Volume Rendering." In *Proceedings of IEEE Visualization,* pp. 293～300.

Weiskopf, D., K. Engel, M. Hadwiger,J. Kniss, and A. Lefohn. 2003. "Interactive Visualization of Volumetric Data on Consumer PC Hardware." Tutorial 1. *IEEE Visualization.*

Wittenbrink, C., T. Malzbender, and M. Goss. 1998. "Opacity-Weighted Color Interpolation for Volume Sampling." In *Proceedings of the IEEE Symposium on Volume Visualization,* pp. 135～142.

第 40 章　用于三维超声波可视化的实时着色

Thilaka Sumanaweera
Siemens Medical Solutions USA, Inc.

现代可编程 GPU 有极大的计算能力，用于三维医学数据可视化，有助于更快和更有效地诊断疾病。例如，使用超声波数据的体渲染可以看到心肌功能和胎儿畸形（比如颚裂）。我们在这章介绍一种体渲染技术，它使用 GPU 的顶点和片元处理，渲染随时间变化的医学超声波三维体积（或称为 4 维数据）。我们使用三维透视纹理贴图进行超声波数据的体渲染，利用真正的胎儿超声波数据给出结果。

40.1　背景

体渲染是使体积数据集可视化的技术，体积数据可以由计算机控制的 X 线断层摄影术（CT）、核磁共振成像（MRI）和三维超声波等医学仪器得到。体渲染也可以用于油井探测中的体数据可视化。

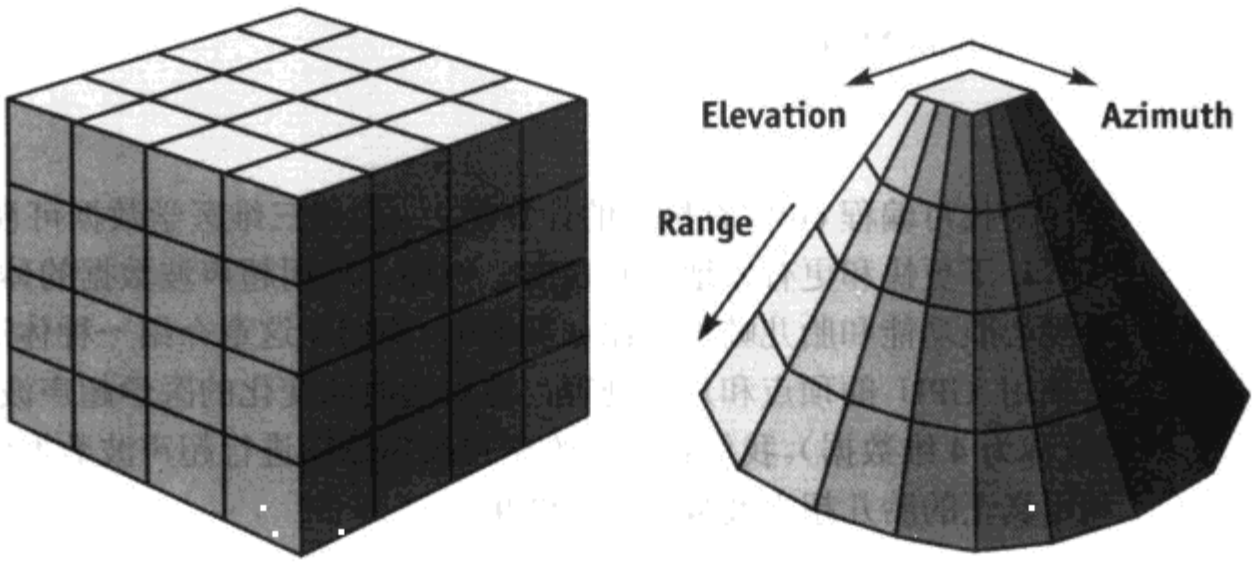
体积数据集的可视化是一种挑战。尽管二维图像的可视化和解释已被广为接受，但是大多数人仍然很难理解三维体数据集的可视化。在医学体积图像技术发明之前，简单的二维 X 光图像在诊断医学的图像领域占据统治地位。放射线学者可以很好地识别和解释二维 X 光图像。但是体积图像技术发明以后，他们必须观看的二维图像数量就爆炸式地增加，因为每个体积都包含一系列的二维图像或切片。显然迫切需要更直观地观察体积数据，体渲染由此而得以开发。

自 20 世纪 80 年代后期以来，CT 和 MRI 等医学成像形式，已经从使体积数据可视技术（体渲染）中获益。CT 和核磁共振成像数据在三维笛卡尔坐标网格中获得，如图 40-1（a）所示。然而超声波数据更为复杂，因为典型的超声波数据在非笛卡尔坐标网格获得（称为声学网格），如图 40-1（b）所示。因此，CT 和核磁共振成像的体渲染也比超声波数据的体渲染简单。

假如想对身体的某个部位进行三维超声波扫描。在临床上，首先把声谱仪的超声波传感器放在皮肤上。传感器基本上是一个天线，它对 1MHz（百万赫兹）或更高频率的声波敏感。它对身体发出超声波脉冲，并接收回波，与雷达很相似。正如雷达一样，当传感器在身体上移动的时候，获得超声波体数据的网格产生发散。图 40-2 所示胎儿的二维超声波图像，可以

给出关于超声波图像的概念。可以把三维超声波体积数据想象为截棱锥或圆锥体，如图 40-1（b）所示。

三维超声波数据是用超声波扫描仪得到的，并且作为三维数组存储在扫描仪本身的内存中。这个三维数组中移动最快的数值对应于离开传感器的距离，叫做射程方向（见图 40-1(b)）。移动第二快的数值对应于传感器的方位角方向。移动最慢的数值对应于传感器的仰角方向。



（a）CT 或核磁共振成像　　（b）超声波

图 40-1　CT、核磁共振成像和超声波数据取样的网格

图 40-2　胎儿头的二维超声波图像

超声波数据也可能是 4 维数据，3 个为空间方向，1 个为时间方向。大家可能见过临床上胎儿面部的 4 维超声波图像。胎儿面部图像所需要的每秒体积数据，比人类心脏图像所需要的数据少得多。这是因为心脏在快速运动，为了看到心脏运动，对于每个心动周期，都需要获得几组三维数据，并进行体渲染。

所幸的是，在现代 GPU 上（像 NVIDIA 的 GeForce FX 和 Quadro FX），可编程的顶点和片元处理器目前已经提供体渲染方法，能够渲染从非笛卡尔坐标网格中获得的 4 维超声波数据，并且渲染速率能够满足观看人类心脏的要求。存在几种体渲染 4 维超声波数据的技术。我们在本章中将介绍这些技术。

40.2　引言

对 CT、核磁共振和超声波等三维网格数据组，可以使用纹理映射进行体渲染。存在多种这类技术。本书第 39 章“体渲染技术”和 Hadwiger et al 2002 对此做了很好的综述。这章使用三维纹理。我们首先概述如何使用 CT 和核磁共振等笛卡尔网格数据做体渲染。以 OpenGL 程序 VolumeRenderCartesian.cpp 为例，解释如何使用笛卡尔网格做体渲染（见书所附的 CD 和网站）。我们不讨论这个程序的细节，而是介绍一些基础。

40.2.1　笛卡尔网格数据的体渲染

图 40-3 显示使用 3D 纹理对笛卡尔数据进行体渲染的要点。首先使用 glTexImage3D 把数据作为三维纹理载入图像存储器。然后用激活的 alpha 混合，把一系列与观察方向成直角的剖面进行纹理映射，从离视点最远的剖面开始，到离视点最近的剖面为止。所有的剖面都与计算机屏幕平行。与计算机屏幕相联系的坐标系称为剖面空间。数据组可以相对于计算机屏幕旋转。与数据相联系的坐标系称为数据组空间。

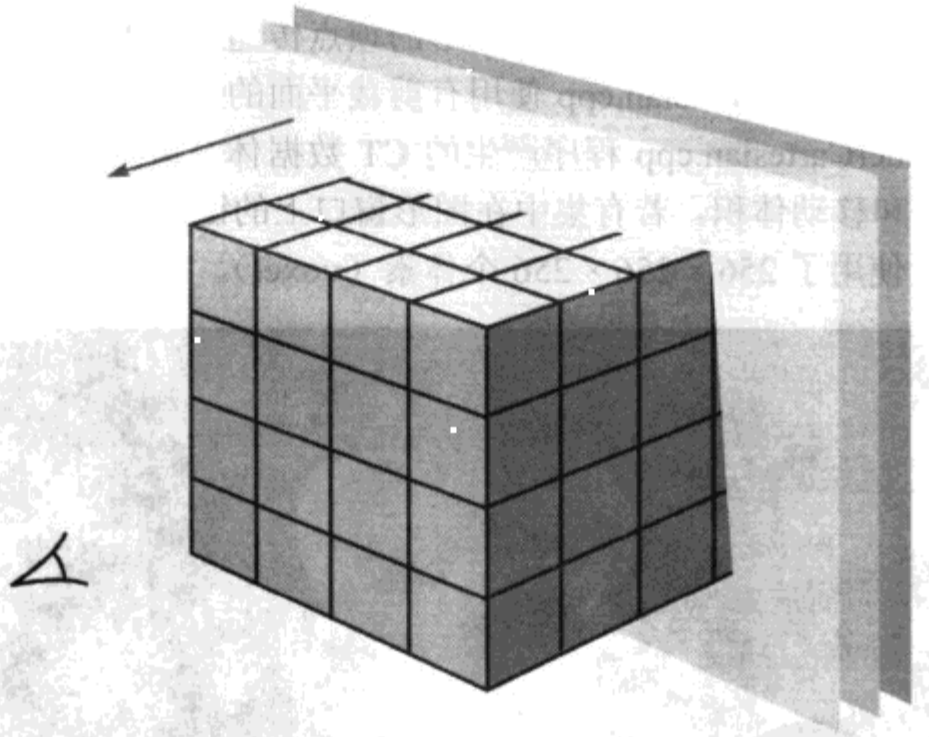


图 40-3　三维笛卡尔数据的体渲染

在第 i 个剖面渲染之后，帧缓冲中像素 P_i 可以表示成：

$$P_i = [1 - f(I_i)]P_{i-1} + g(I_i) \tag{1}$$

这里 P_{i-1} 是在第 $i-1$ 个剖面渲染之后的像素内容，I_i 是从第 i 个剖面引入的片元，$f()$是不透明性函数，而 $g()$是不透明加权的传输函数。假定 $f()$和 $g()$两者都是在[0, 1]范围中。可以直观地看到，如果引入的片元是不透明的，它后面的片元就不再做贡献。可以在 OpenGL 开

放式绘图界面中，借助混合函数 GL_ONE_MINUS_SRC_ALPHA，使用 alpha 混合实现方程式 1。不透明加权的传输函数为像素产生 RGB 值，而不透明函数产生像素的 alpha 值。

当对剖面进行纹理映射时，可以使用两个不同的方法指定他们：矩形闭包和相交的多边形。

矩形闭包

矩形闭包是最直接的纹理映射剖面方法。范例程序 VolumeRenderCartesian.cpp 使用了这种技术。在这个方法中，从背面到前面画一系列闭包着数据组范围的方块。为了在快速渲染中做快速访问，可以把方块的顶点坐标存储在图像存储器中。惟一要记住的是，原始数据体之外的数据不能做纹理映射（即任何纹理坐标（s, t, r）小于 0.0 或大于 1.0 时）。可以采用以下 3 个办法这样做：（a）使用剪裁平面；（b）使用 GL_CLAMP_TO_BORDER_ARB 包裹纹理坐标，并设定边的颜色为黑色；（c）用 GL_CLAMP_TO_EDGE 包裹纹理坐标，并如下进行设置，使边上的数据等于 0：

如果 $s = 0.0$，$t = 0.0$，$r = 0.0$，$s = 1.0$，$t = 1.0$ 或 $r = 1.0$，则数据= 0

相交的多边形

这个方法更复杂，需要在给定剖面和数据立方体之间找到相交的多边形。可以用 CPU 或 GPU 寻找（使用顶点程序）。一旦找到相交多边形的顶点，接着就要纹理映射那个多边形。从片元程序的观点来看，这个方法比其他方法快，因为只访问那些在数据立方体里的片元。然而，从光栅化的观点来看，这个方法更麻烦，因为现在有更多的三角形需要光栅化。此外，每当相对于数据组空间改变观察方向时，程序都需要再计算多边形。如果使用 CPU 重新计算多边形，那么在渲染它们之前，也需要把多边形的顶点传递到 GPU，这可是耗时的。

示范程序 VolumeRenderCartesian.cpp 使用有剪裁平面的矩形闭包来渲染剖面。图 40-4 显示了使用 VolumeRenderCartesian.cpp 程序产生的 CT 数据体渲染图像。在运行这个程序的时候，可以旋转、缩放和移动体积。若有集中在图形窗口上的键盘时，键入 h，就能获得帮助。这个可视化的数据组使用了 256 × 256 × 256 个体素（voxel）。

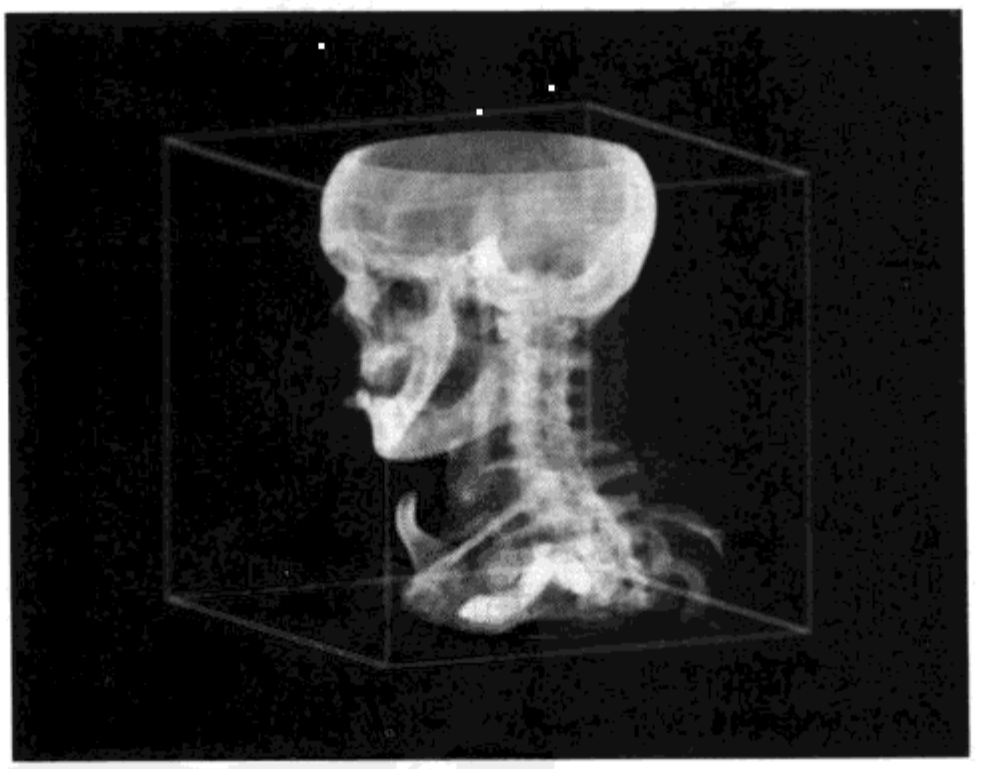
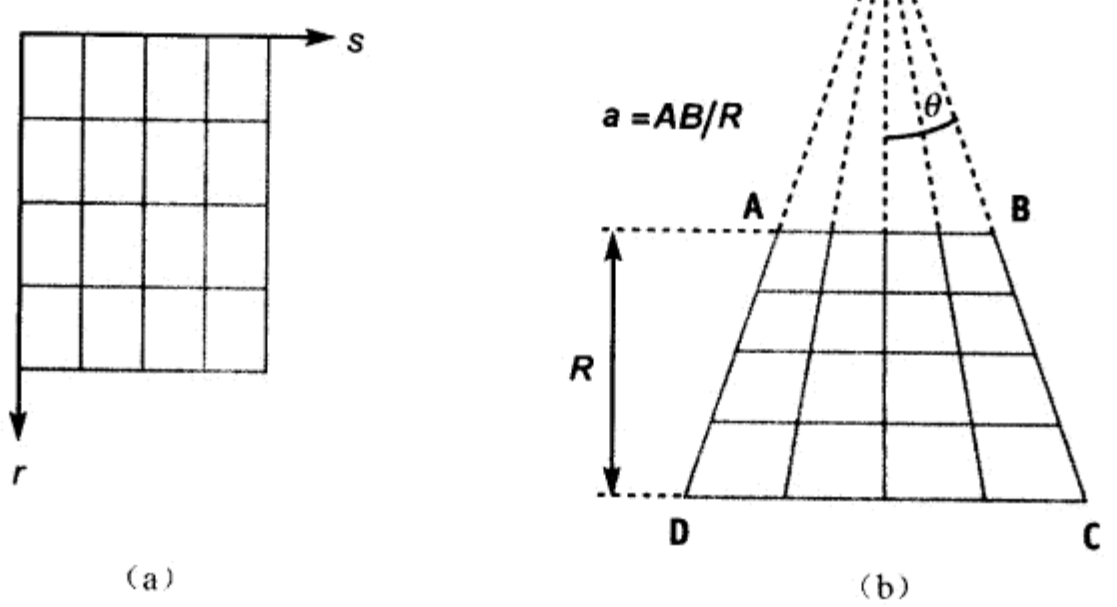


图 40-4 在笛卡尔坐标系获得的 CT 数据组的体渲染

40.2.2　体渲染锥体网格中的数据

现在研究在锥体网格中所得数据的体渲染。进入身体的时候，有几种有发散性质的网格，包括斜棱锥体的（锥形）、圆锥形的、球形的和圆柱形的网格。为了简化起见，我们在这章中使用斜棱锥体的网格，因为它可以线性处理，这对 GPU 很有利。此外，图 40-1（b）和图 40-2 中所显示的其他类型网格，也可以通过沿着射程方向对超声波数据重取样，转换成斜棱锥体网格，而不造成什么损失。

让我们先看两个二维的情况：（a）笛卡尔网格；（b）斜棱锥体网格，如图 40-5 所示。对于每个二维网格，我们在每个网格点得到一个超声波数据样品，形成二维数组。一般超声波图像具有许多网格点。但是这里我们只看一个 5 × 5 的网格。在图 40-5（a）中显示的笛卡尔网格中，对应于常数 s 的垂直线叫做超声波线。传感器位于 $r = 0.0$ 平面。超声波扫描仪顺序地从左到右发射超声波线，在每个网格点取数据，从而收集一个二维图像。纹理坐标（s, r）对应于传感器的方位角和射程方向。



（a）　　（b）

图 40-5　二维的笛卡尔和锥形网格

在图 40-5（b）显示的锥形网格中，超声波线全部会聚于一个公共点 O，称为顶点。传感器的宽度 AB，称为孔径尺寸。在本章中，我们使用规格化孔径尺寸：$a = AB/R$，R 是锥体的高度（$0 \leqslant R \leqslant 1.0$），角 AOB 叫做顶角 2θ。我们可以想象锥体形网格与笛卡尔网格等效，只是把 s 坐标作为 r 坐标的函数进行线性缩放。如后面所要讨论的那样，使用投射的纹理映射可以容易地做到这点。

图 40-6 显示的是三维锥形网格。为了给它产生正确的纹理坐标，类似于二维锥形网格，我们将纹理坐标 s 和 t 作为 r 的线性函数进行缩放。下面就是这个运算法则的具体步骤。

1．假定我们使用笛卡尔网格，通过模仿的 OpenGL 自动纹理坐标生成函数 g1TexGen，产生顶点程序里的纹理坐标。

2．目标是把 s 和 t 坐标当作 r 的线性函数而进行缩放。为了做到这点，计算以下顶点程序里的新纹理坐标：

$$q = a + 2r\tan\theta \tag{2}$$

$$s' = (s - 0.5) + 0.5q \tag{3}$$

$$t' = (t - 0.5) + 0.5q \tag{4}$$

这里 s'、t'和 q 是新的坐标。

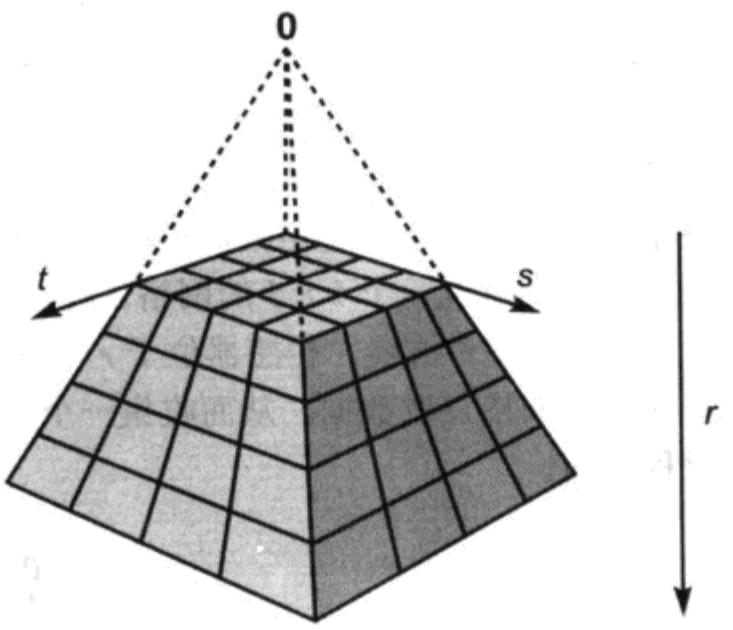


图 40-6 三维锥形网格

3．接着在片元程序中，使用投射三维纹理查询指令，在一个步骤中做纹理缩放和查询。必须避免在 r 坐标上进行缩放。假如也沿着 r 方向缩放，随着对身体的进入，产生的锥体数据体积会越来越多地沿着 r 方向伸展。只要在片元程序中预先用缩放率乘以 r 坐标，就可以避免这一点：

$$r' = rq \tag{5}$$

这里 r'也是一个新的纹理坐标（因为这个步骤是非线性操作，所以我们不能够在顶点程序中进行，不然以后 GPU 会在每个片元中内插出不正确的 r 值）。

4．完成以上步骤后，投射纹理查询指令使用下列纹理坐标进行纹理查询：

$$s_{\text{proj}} = (s - 0.5)/q + 0.5 \tag{6}$$

$$t_{\text{proj}} = (t - 0.5)/q + 0.5 \tag{7}$$

$$r_{\text{proj}} = r \tag{8}$$

如前所述，剖面被指定为一系列的矩形闭包。选择的这些长方形尺寸使得它们能够完全包裹锥体形网格。通过使用前面 VolumeRenderCartesian.cpp 同样的 CT 数据集，开放式绘图界面样本程序 VolumeRenderPyramid.cpp、顶点程序 VolumeRenderPyramidV.cg 和片元程序 VolumeRender-PyramidF.cg 阐明我们的运算法则。在 VolumeRenderPyramidF.ocg 中也有片元程序的汇编版本。现在我们研究顶点和片元程序。

顶点程序

我们现在逐条研究列表 40-1 所示的顶点程序。这个顶点程序的输入是可变的位置，包括顶点坐标。缩放因子（ZoomFactor）、锥体（Pyramid）、剪裁平面（ClipPlane）0～5、观察模型（ModelView）和投射观察模型（ModelViewProj）等 10 个统一的变量也输入到这个程序中。缩放因子 ZoomFactor 包含用于缩放的比率值；锥体 Pyramid 包含两个描述锥体形网格的

几何参数：Pyramid.x 是规格化的传感器孔径尺寸 a，Pyramid.y 是 1/2 顶角正切的 2 倍，$2\tan\theta$；剪裁平面 ClipPlane 0 到 5，包含 6 个剪裁平面的方程式；观察模型 ModelView 含有数据集空间的观察模型矩阵，而投影观察模型 ModelViewProj 含有与剖面空间投影矩阵连接的观察模型矩阵。

列表 40-1　VolumeRenderPyramidV. cg

```
void VertexProgram(in float4 Position    : POSITION,
                   out float4 hPosition : POSITION,
                   out float4 hTex0      : TEX0,
                   out float hClip0      : CLP0,
                   out float hClip1      : CLP1,
                   out float hClip2      : CLP2,
                   out float hClip3      : CLP3,
                   out float hClip4      : CLP4,
                   out float hClip5      : CLP5,
                   uniform float4 ZoomFactor,
                   uniform float4 Pyramid,
                   uniform float4 ClipPlane0,
                   uniform float4 ClipPlane1,
                   uniform float4 ClipPlane2,
                   uniform float4 ClipPlane3,
                   uniform float4 ClipPlane4,
                   uniform float4 ClipPlane5,
                   uniform float4x4 ModelView,
                   uniform float4x4 ModelViewProj)
{
  // 计算剪裁空间位置
  hPosition = mul(ModelViewProj, Position);

  // 从位置坐标移去缩放因子
  Position = Position * ZoomFactor;

  // 使用笛卡尔网格计算纹理坐标
  hTex0 = mul(ModelView, Position);

  // 存储原始纹理坐标
  float4 hTex0_Orig = hTex0 - float4(0.5, 0.5, 0.0, 0.0);

  // 计算锥体网格中的纹理坐标
  // pyramidal grid
  hTex0.w = Pyramid.x + hTex0.z * Pyramid.y;

  // 调节纹理坐标偏移
  hTex0.x = hTex0.x - 0.5 + 0.5 * hTex0.w;
  hTex0.y = hTex0.y - 0.5 + 0.5 * hTex0.w;

  // 剪裁锥体体积
  hClip0 = dot(hTex0_Orig, ClipPlane0);
  hClip1 = dot(hTex0_Orig, ClipPlane1);
  hClip2 = dot(hTex0_Orig, ClipPlane2);
```

```
  hClip3 = dot(hTex0_Orig, ClipPlane3);
  hClip4 = dot(hTex0_Orig, ClipPlane4);
  hClip5 = dot(hTex0_Orig, ClipPlane5);
}
```

这个顶点程序的输出是顶点在剪裁空间中的位置 hPosition、纹理坐标 hTex0，以及剪裁距离 CLP0 到 CLP5。

在第 1 行我们计算顶点在剪裁空间的位置；在第 2 行去掉由开放式绘图界面程序加到位置坐标的任何缩放；在第 3 行模仿开放式绘图界面的 glTexGen 函数，为笛卡尔坐标系计算纹理坐标；在第 4 行把纹理坐标的原始数据在 s 和 t 方向上偏移 0.5，存在 hTex0_Orig 中，留待以后使用。下面是有趣的部分。在第 5 行针对给定的 r 坐标，对 s 和 t 纹理坐标做必要的拉伸和压缩；在第 6 和第 7 行偏移 s 和 t 纹理坐标的原点，以便片元程序中的 s 和 t 纹理坐标的缩放围绕锥体的中轴进行；第 8 行到第 13 行计算包围锥体体积的 6 个平面的剪裁值。片元包含的剪裁值小于 0 时，不画到帧缓冲中去。

片元程序

在顶点程序运行完后，就该执行列表 40-2 的片元程序了。输入 inTex 是一个包含纹理坐标的变量。还有两个一致的输入：USTexture 和 ColorMap。USTexture 包含三维纹理，它描述三维超声波体积。ColorMap 是彩色映射表，包含不透明加权的传输函数和不透明函数。输出 sColor0 包含片元颜色。

列表 40-2 VolumeRenderPyramidF.cg

```
void FragmentProgram(in float4 inTex : TEXCOORD0,
                     out float4 sColor0 : COLOR0,
                     const uniform sampler3D USTexture,
                     const uniform sampler1D ColorMap)
{
  // 预先把 r 坐标乘以缩放因子
  inTex.z = inTex.z * inTex.w;

  // 投影纹理贴图查询
  float val = tex3Dproj(USTexture, inTex);

  // 颜色贴图查询
  sColor0 = tex1D(ColorMap, val);
}
```

在第 1 行中，我们预先把 r 坐标乘以缩放因子；在第 2 行使用投影纹理贴图查询三维超声波体积的纹理值；在第 3 行做一维彩色映射查询，按照超声波数据值产生颜色以进行渲染。

体渲染速度通常受到片元程序复杂性的限制（即它的填充率限度）。因此，有时最好用手调整片元汇编代码，以求达到最大能力，如列表 40-3 所示。

列表 40-3 VolumeRenderPyramidF.ocg

```
!!FP1.0
# NV_fragment_program generated by NVIDIA Cg compiler
# cgc version 1.1.0003, build date Mar 4 2003 12:32:10
```

```
# command line args: -q -profile fp30 -entry FragmentProgram
#vendor NVIDIA Corporation
#version 1.0.02
#profile fp30
#program FragmentProgram
#semantic FragmentProgram.USTexture
#semantic FragmentProgram.ColorMap
#var float4 inTex : $vin.TEXCOORD0 : TEXCOORD0 : 0 : 1
#var float4 sColor0 : $vout.COLOR0 : COLOR0 : 1 : 1
#var sampler3D USTexture : : texunit 0 : 2 : 1
#var sampler1D ColorMap : : texunit 1 : 3 : 1
MOVX H0, f[TEX0];
MULX H0.z, H0.z, H0.w;
TXP  R0, H0, TEX0, 3D;
TEX  o[COLR], R0.x, TEX1, 1D;
END
# 4 instructions, 1 R-regs, 1 H-regs.
# End of program
```

在第 1 行把引入的纹理坐标载入一个 12bit 定点寄存器 H0。在下一行预先把纹理坐标的 *r* 分量乘以缩放率。使用 12bit 定点计算可以明显地提高渲染速度。在第 3 行使用三维投射纹理查询表查询片元值。在最后一行，我们使用一维的彩色贴图查询表查询片元的颜色值。

图 40-7 显示了用开放式绘图界面示范程序 VolumeRenderPyramid.cpp 产生的体渲染图像，它可以调节孔径尺寸（按键 a/A）和顶角（按键 b/B），而且可以看到屏幕上的效果。按 h 键可以看到 Help（帮助）。

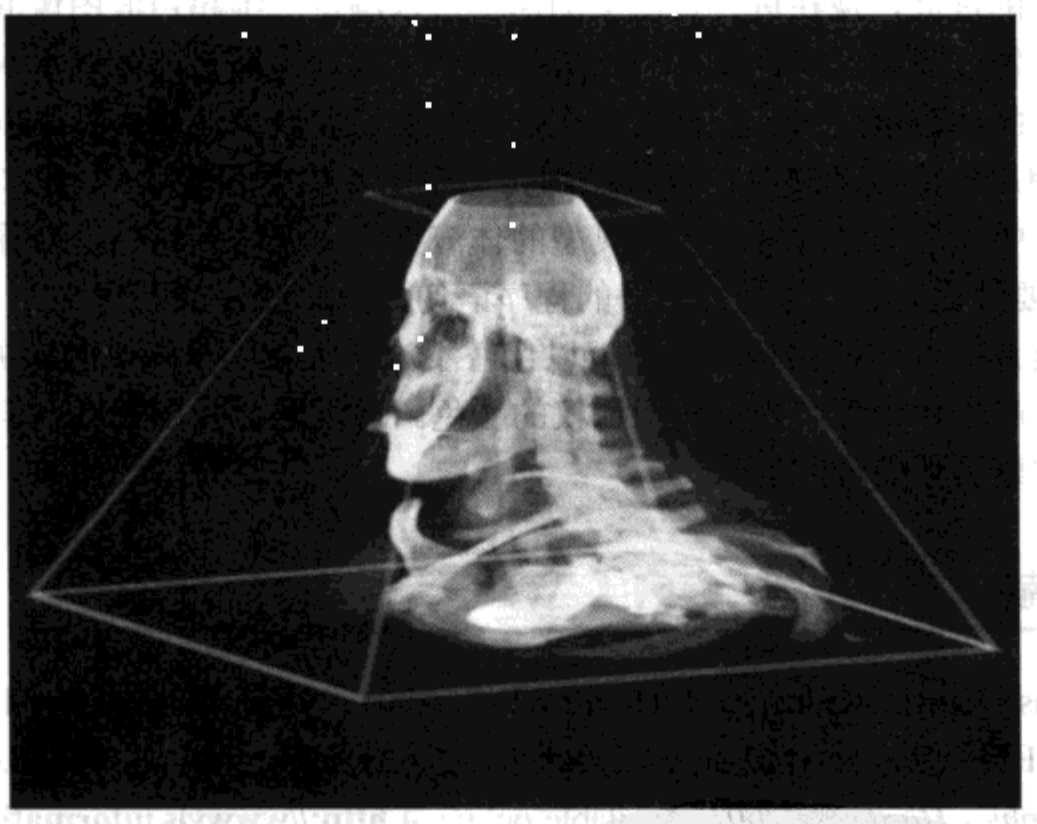


图 40-7　在锥体坐标系中的 CT 数据组的体渲染

超声波数据的体渲染

示范程序 VolumeRenderPyramidUS.cpp 说明如何使用超声波数据进行体渲染。子宫中胎儿超声波数据使用 ACUSON Sequoia 512 超声波系统获得，有 256 × 256 × 32 个体积元（voxel）。

在这种情况下，由于超声波探头的特殊几何形状，不需要方程式 7 那样改变 t 坐标，只需要按照线性函数缩放 s 坐标。这样产生的锥体几何体，包含一个楔形（V 形）体积。

40.3 结果

图 40-8 显示了体渲染的胎儿面部图像，使用的投影纹理映射技术在本章中已经概略说明。样本程序 VolumeRenderPyramidUS.cpp 使用 181 个剖面产生这个图像。在影片剪辑 VolumeRenderPyramidUS.avi 中，如果近看，可以看到胎儿伸舌头和抓挠头。

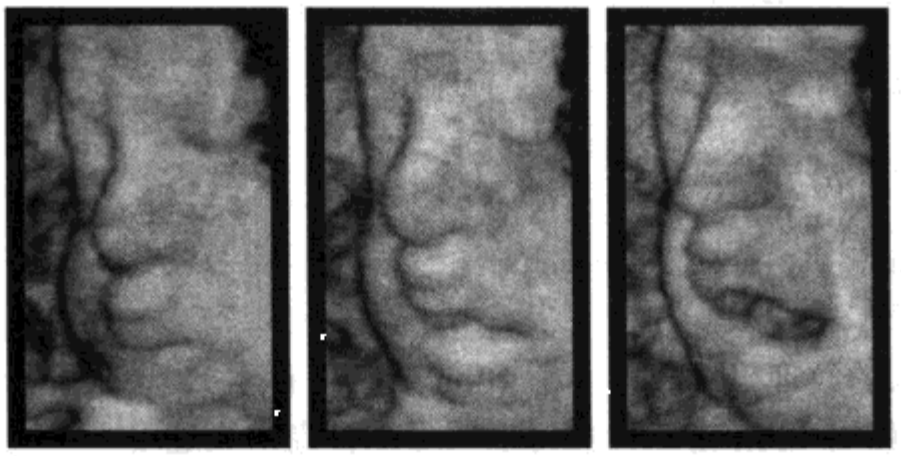


图 40-8 使用投影纹理映射体渲染的胎儿面部图像

40.4 小结

随着 GPU 的进展，现在进行体渲染的速率可以满足三维超声波图像的要求。我们已经介绍如何使用投影纹理映射对锥体形网格做体渲染。以此为框架，可以加入着色模型，也可以使用预先整合的体渲染技术，仅损失一点帧速率就可以减少波纹失真（Engel et al.2001）。

也可以不使用投影纹理映射，直接在片元程序中实现方程式（6）和（7）。但是这种方式比我们在本章中采取的方式慢。因为典型的体渲染应用已达到填充率边界，而实现方程式（6）和（7）需要转化 r 值，这需要另外的程序语句，因此运行得更慢。

我们可以给片元程序提供传输函数，以代替不透明加权的传输函数，并用颜色 sColor0 乘以不透明值，产生片元程序里不透明加权的颜色。这可能会减少失真，但是要牺牲一定的帧速率，因为需要另外的程序语句。

40.5 参考文献

Engel, Klaus, Martin Kraus, and Thomas Ertl. 2001. "High-Quality Pre-Integrated Volume Rendering Using Hardware-Accelerated Pixel Shading." *Proceedings of the SIGGRAPH/Eurographics Workshop on Graphics Hardware 2001*. Available online at **http://wwwvis.informatik.uni-stuttgart.de/~engel/pre-integrated/ paper_GraphicsHardware2001 .pdf**

Hadwiger, Markus, Joe Kniss, Klaus Engel, and Christof Rezk-Salama. 2002. "High-Quality Volume Graphics on Consumer PC Hardware." Course Notes 42, SIGGRAPH 2002. Available online at **http://www.cs.utah.edu/~jmk/sigg_crs_02/course_42/course_42_notes_small.pdf**

第 41 章　实时立体图

Fabio Policarpo
Paralelo Computação Ltda.

41.1　什么是立体图

立体图是一种特殊的二维图像，它将立体信息编码，在正确观察时，可以显露出隐藏的三维场景。一切开始于 1960 年，当时 Bela Julesz 在 AT&T 公司 Bell 实验室研究人类视觉，特别是深度知觉和模式识别，结果创建了随机点立体图（RDS）。立体图从立体摄影发展而来，即从略微不同的摄像机位置拍摄两张相片，表现我们两只眼睛之间的位置差异。

41.1.1　立体摄影

古老的立体摄影术诞生于 1838 年，现在古玩店仍然有图 41-1（a）中所示的那种古董立体摄像机和照片观看器。立体摄影的思路类似于我们的两只眼睛，从略微不同的水平位置拍摄两张相似的相片。我们的两眼大约分开 65 毫米，这点差距产生略有差别的图像被送到大脑后，就产生深度的感觉。

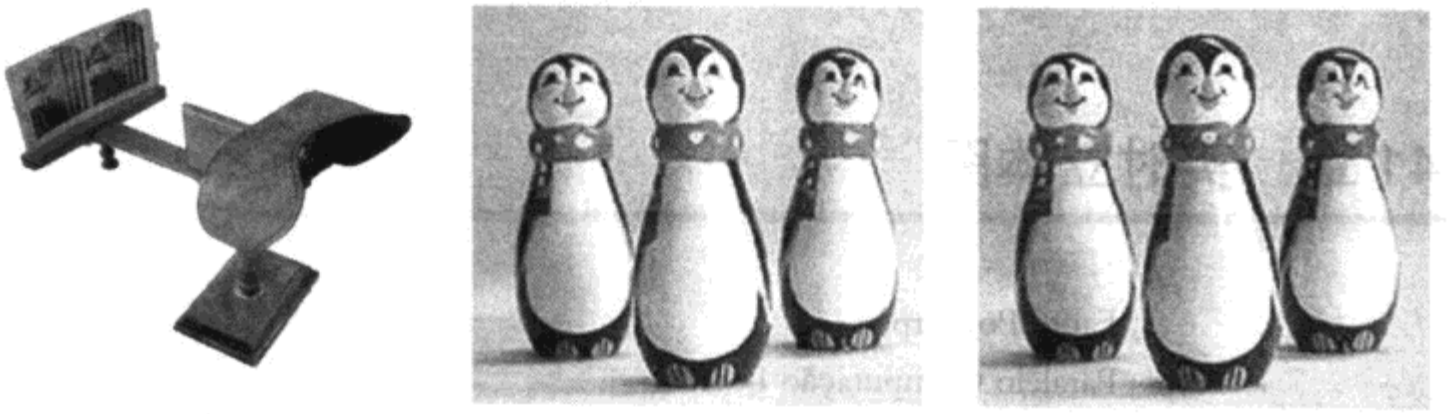
在图 41-1（a）中，两个图像（立体对）在透镜前面并排放置。透镜帮助观察，分别给每个眼睛呈现各自的图像。看立体对时，让左眼看左边的相片（图 41-1（b）），右眼看右边的相片（图 41-1（c））。如果交换图像（用右眼看左边的图像，而左眼看右边的图像），深度感觉将会颠倒。

41.1.2　随机点立体图

为了产生随机点立体图，我们从创建由任意点组成的图像开始，如图 41-2（a）所示。然后复制图像，并通过选择区域和水平移动它而进行修正，如图 4l-2（b）（移动越大，它看起来就越深）。然后用更多随机产生的点，填充由于移动图像区域所造成的缝隙，就得到它了，如图 41-2（c）所示。

我们现在有两个图像：最初的和复印/移动的版本。对每个眼睛显示各自的图像得到深度感觉，而且能看见隐藏的三维立体场景。

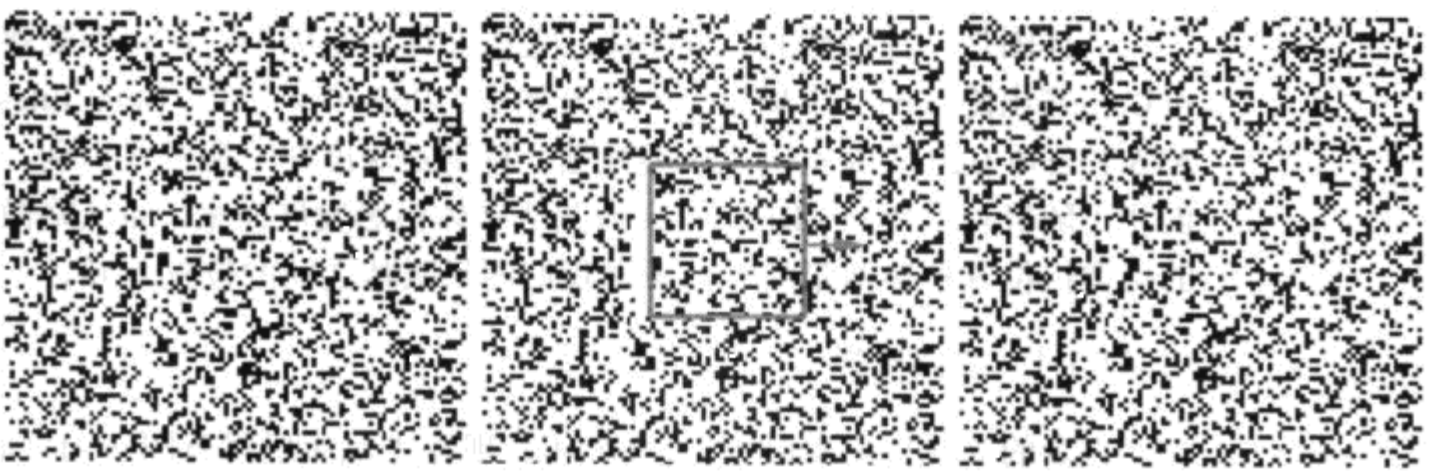
看随机点立体图可以像看标准立体相片一样。为了仅用眼睛而不用额外的仪器看随机点立体图（或标准立体相片），把眼睛靠近图像平面，看图 41-3 中的两个图像。结果看到的是 4 个图像而不是两个图像，原因是每个图像都复制了一份（就好像正在看自己的鼻子）。



（a） （b） （c）

图 41-1 立体摄影

（a）维多利亚女王时代的立体相片观看器；
（b）左边的和（c）右边的立体对图像。注意左边和右边的企鹅图像之间的微小不同



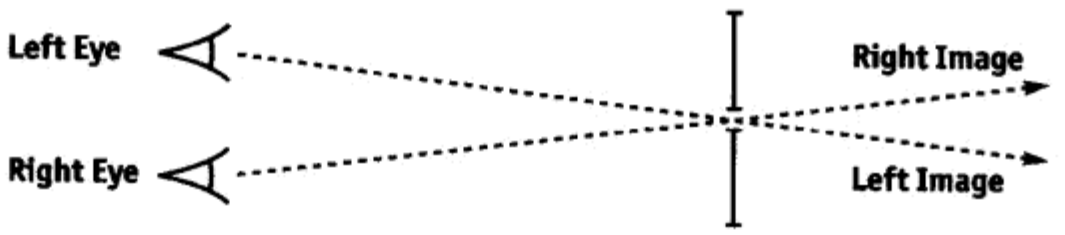
（a） （b） （c）

图 41-2 随机点立体图的产生

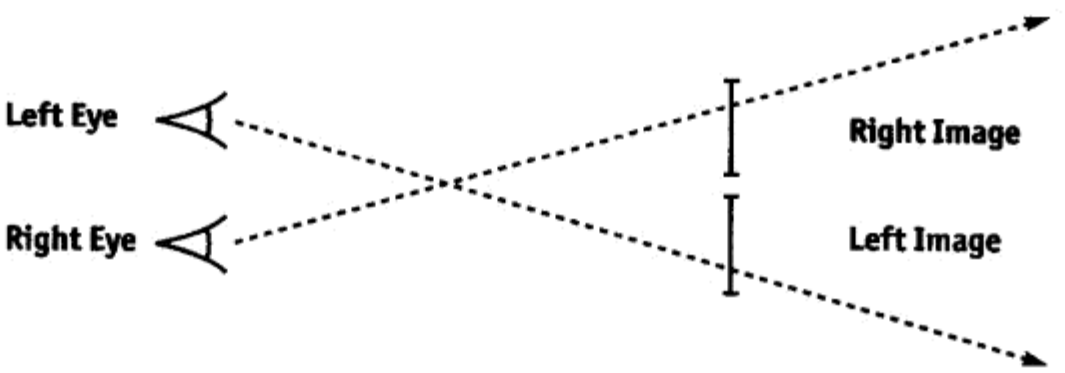
（a）最初的随机点图像（左眼的图像）；（b）选择区域和水平移动；
（c）最后的图像，缝隙由另外的随机点填充（右眼的图像）

图 41-3 用于可视化的随机点立体对
正确观察时，这两个图像将会在图像的中央显示立方体

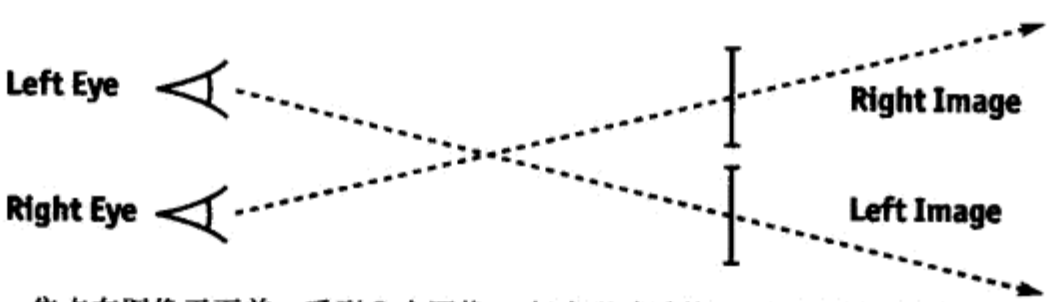
改变焦点使复制的图像彼此移动，当图像在中间合并的时候，将会看到隐藏的三维立体场景。在正确的焦点上，只会看到 3 个图像：一个在中间的三维立体相、两个在边上的二维相。图 41-4 详细解释了这是如何工作的。



焦点在图像平面上。看到两个图像。每个眼睛看着相同图像的相同位置
看不到三维图像



焦点在图像平面前。看到 4 个图像。 每个眼睛看着不同图像的不同位置
看不到三维图像



焦点在图像平面前。看到 3 个图像。 每个眼睛看着不同图像的相同位置
看见三维图像!

图 41-4　立体图像对的观察

41.1.3　单个图像的立体图

单个图像的立体图（SIS），由标准随机点立体图演变而来，只需要一个图像。思路是图像的相邻垂直切片会匹配图案，于是产生深度感觉，见图 41-5（上图）。在单个图像的立体图中，我们把图像切成垂直的条，减少观察时所需要的偏移。举例来说，如果我们把图像切成 8 条，为了观察它，我们需要 1/8 的偏移。这允许眼睛的交叉点更靠近图像平面，这样观察起来比较舒服。

一个随机点立体图比起单个图像立体图来，眼睛的交点必定在图像平面的更前面（即，更靠近观者），所看到的置换图像是图像本身的尺寸。实际上，一个随机点立体图对的工作原理像具有两条的单个图像立体图一样。

对于立体摄影和经典的随机点立体图像，观众必须总是在图像平面之前让眼睛相交。但是在单个图像立体图中，因为条之间的分离比我们眼睛之间的距离更小，有另一个比较舒服的看图像方法。观众可以让眼睛在图像平面的后面相交，由此使深度感觉颠倒，但是仍会形成 3D 图像，见图 41-5（下图）。大多数流行的单个图像立体图是通过这种观察方法产生的。

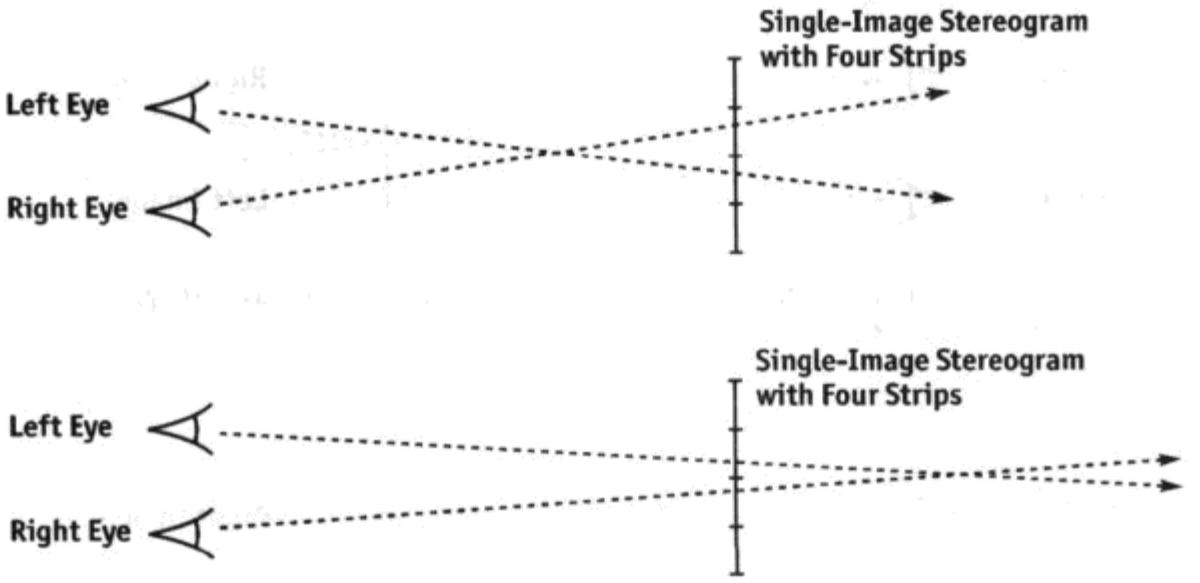
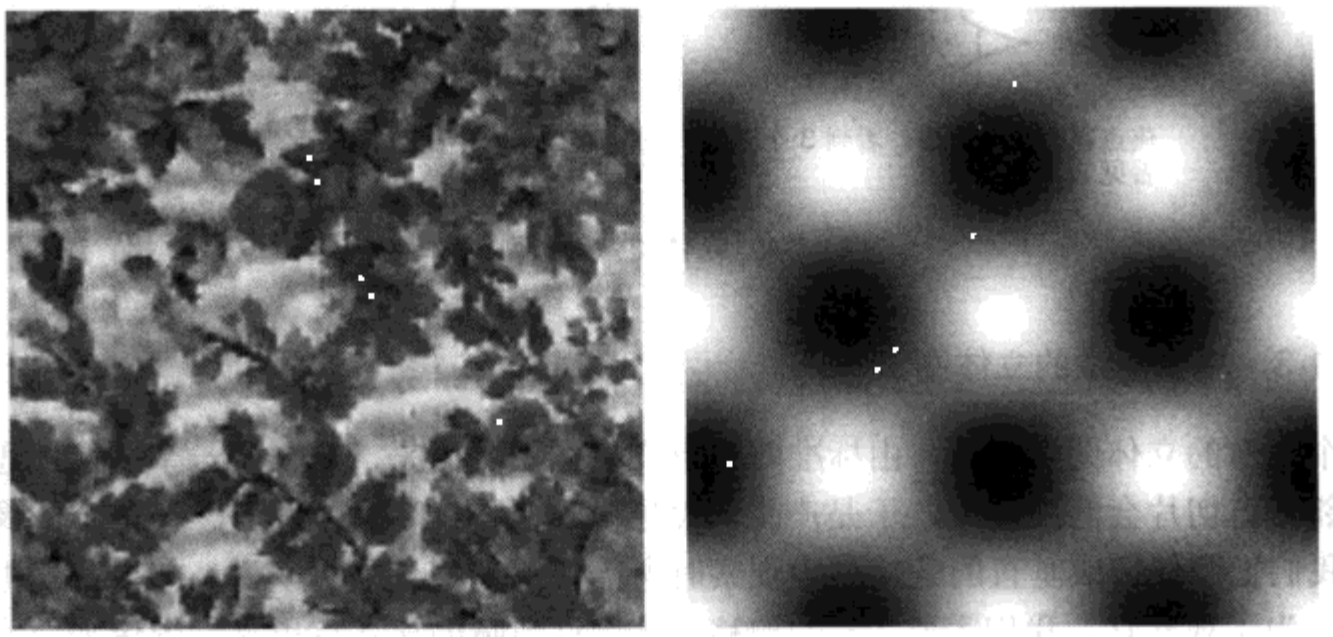


图 41-5 用 4 条观察立体图

上图：眼睛交点在图像平面之前；看不到 3D 场景。
下图：眼睛交点在图像平面之后；3D 场景可见！

41.2 单个图像立体图的创建

单个图像立体图是从给定的深度贴图（即，带有深度信息的灰阶图像）和平铺图案而产生的（通常是彩色平铺图像），如图 41-6 所示。



（a）彩色平铺图像 （b）灰阶深度贴图

图 41-6 输入图像

在图 41-7 中平铺图案是基于深度贴图重复的和变形的。如果随机点的图像作为平铺图案使用，产生的立体图是单个图像随机点的立体图（SIRDS）。

41.2.1 参数

当创建新的单个图像立体图时，需要考虑参数：包括使用的条带数目、能增加或减少深度感觉的深度因素（它又反过来控制应用于图案平铺的变形量），以及是否颠倒深度值（把白色作为深度 0 或最大深度 1）。

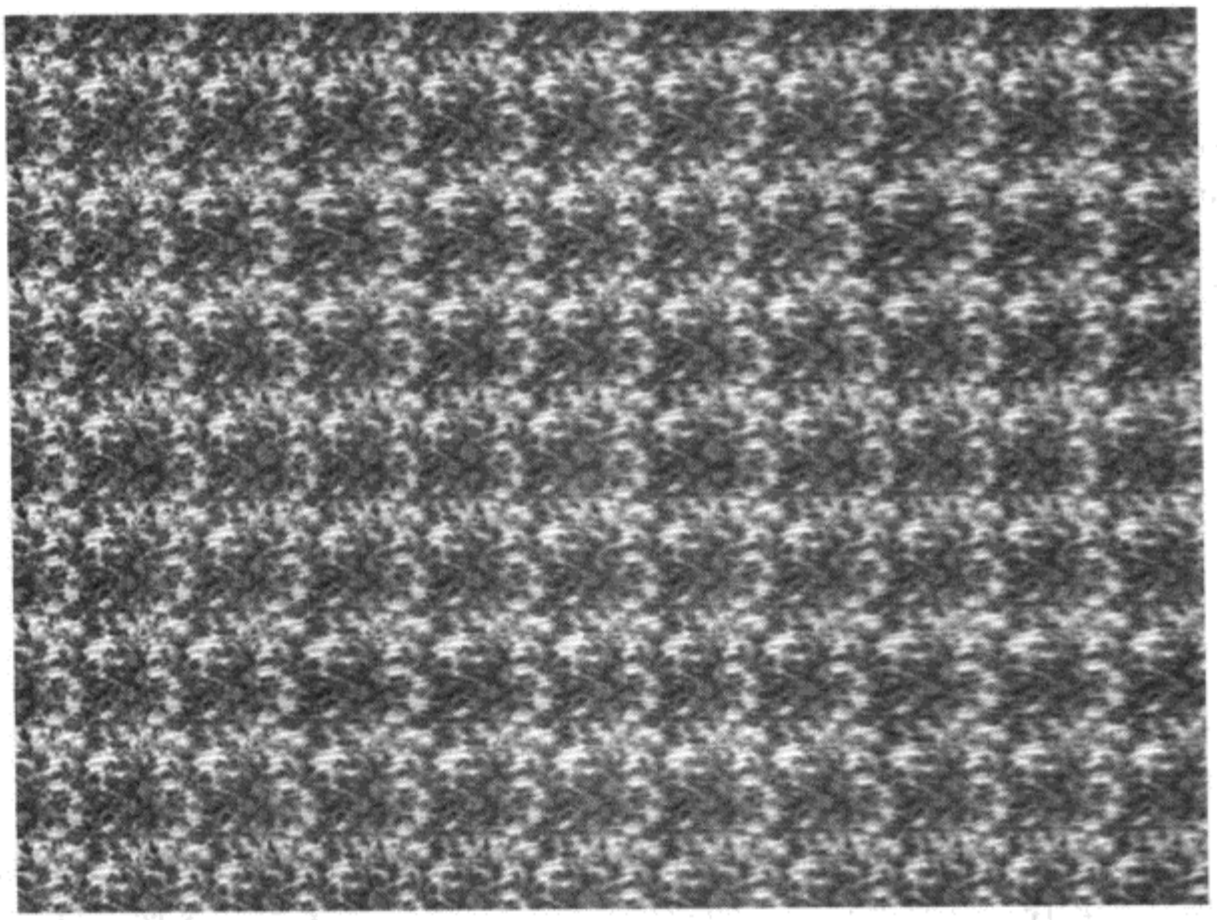


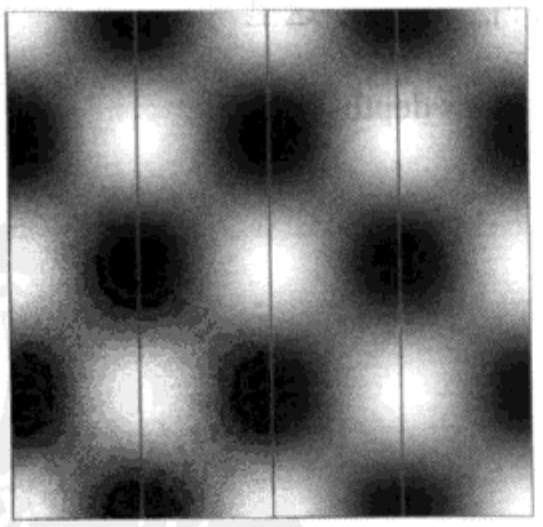
图 41-7　立体图的结果

这里是一些参数：

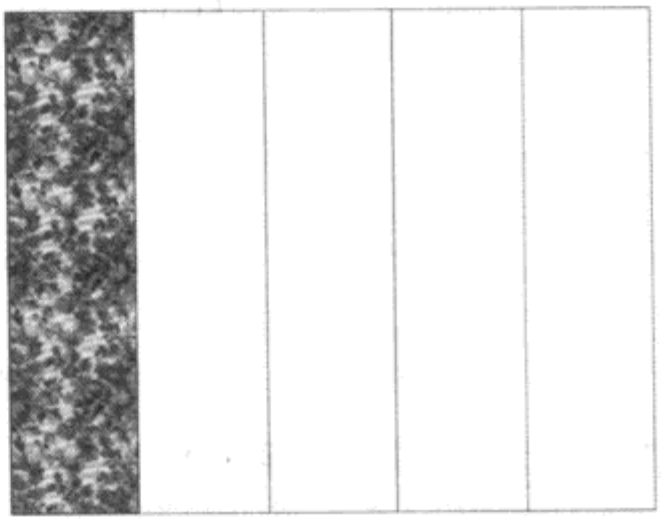
- 条带的数目（num_strips）：通常是从 8 到 24 的整数值
- 深度因子：范围为 0.0 到 1.0 的浮点值，1.0 为最大深度
- 颠倒深度：布尔（Boolean）值指出深度值是否应该被颠倒（1–深度）

41.2.2　渲染

为了渲染单个图像立体图，我们把深度贴图和结果图像细分为垂直的条带。真实的单个图像立体图可能会使用很多条带，为了简化，这个例子我们只使用 4 个（num_strips=4）。如图 41-8 显示的那样，我们把深度贴图细分为 4 个条带（num_strips），而把结果贴图分为 5 个条带（num_strips+1），因为开始时需要一个参考长条。



（a）深度贴图



（b）结果贴图

图 41-8　渲染图像

这里是渲染单个图像立体图的伪代码：

```
select_pattern_texture();
draw_strip(0);
read_strip_to_result_texture(0);

enable_fragment_program();
select_depth_texture();
select_result_texture();

for(i = 0; i < num_strips; i++)
{
  draw_strip(i + 1);
  read_strip_to_texture(i + 1);
}

disable_fragment_program();
draw_result_texture();
```

在伪代码的开始，我们简单地画第一个条，用平铺图案把它纹理映射出来，如图 41-8（a）所示（不需要片元程序，只要进行正常的纹理映射就行了）。接着把长条像素复制到结果纹理贴图上去（使用 glCopyTexSubImage2D 复制我们刚刚修正的区域），得到如图 41-8（b）所示的结果图像。

然后，我们启用片元程序并设置深度贴图、结果贴图，及作为参数的深度因子。我们循环地画来自结果图像的其余 4 个条带。因为每个新长条使用前面长条的图像作为参考，所以必须把我们所画每个长条的像素复制到结果纹理贴图中去。

当画图 41-8（b）中缺失的条带时，我们将复制前面长条的内容，并根据当前像素的深度，水平地置换像素。举例来说，如果给定的像素深度是 0，我们将复制前面长条中相同像素位置的准确颜色。但是对于其他深度值，我们将从前面长条的同一扫描线位置得到颜色，但是与深度成比例地置换它。

现在考虑坐标范围[0, 0]到[1, 1]的深度贴图和结果贴图图像。给定的具有坐标$[x, y]$的片元使用以下的计算，求结果贴图中前面长条（即从那里取得颜色）的坐标：

$$[x,y]_{\text{result_map}} = [x + a - c, y]_{\text{result_map}}，\text{这里}$$

$$a = \left[b \times \left(\frac{x}{c-1} \right), y \right]_{\text{depth_map}} \times c \times \text{depth_factor}$$

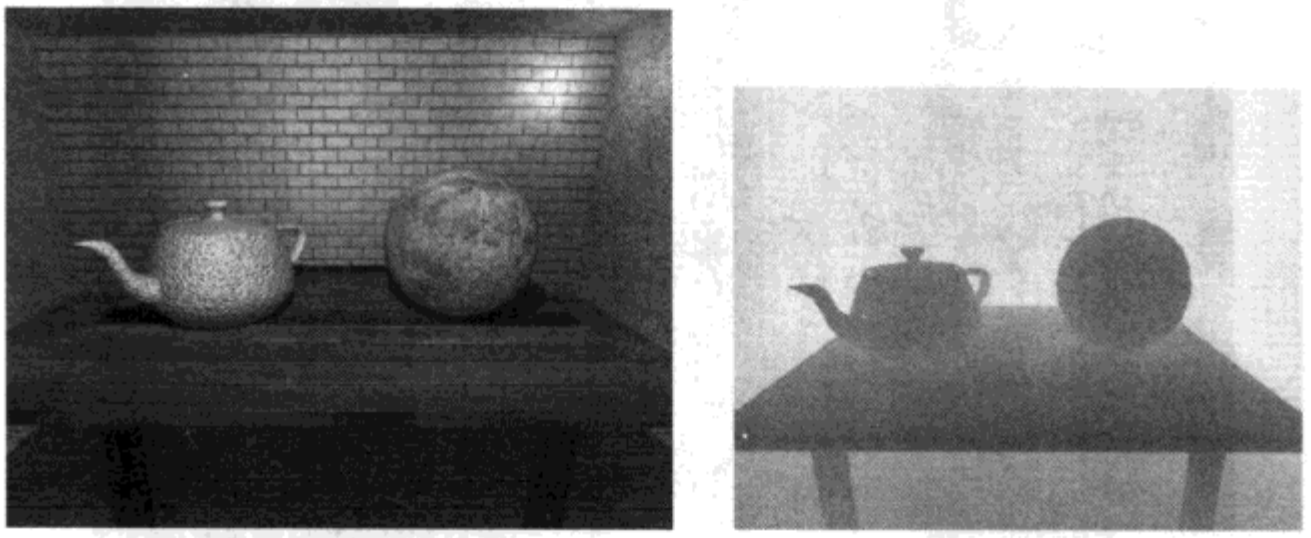
$$b = \frac{1}{\text{num_strips}}$$

$$c = \frac{1}{\text{num_strips} + 1}$$

41.2.3 动画的单个图像立体图的创建

因为使用 GPU 的片元程序可以实时地产生单个图像立体图，我们现在可以制作一个动画的单个图像立体图（ASIS）。我们将使用由三角形组成的正常三维场景，并且使用它的 z

缓冲作为单个图像立体图的深度贴图，也就是说从给定的视点渲染 3D 场景，并且把 z 缓冲的内容读入深度贴图纹理，见图 41-9。



（a）　　（b）

图 41-9　制造动画的单个图像立体图
（a）三维场景的正常渲染；（b）从渲染三维场景后的 z 缓冲内容获得的深度贴图

当使用实时渲染场景的 z 缓冲作为深度贴图图像的来源时，我们可以交互地移动 3D 场景（或预先确定摄像机动画路径），并逐帧更新立体图以反映变化。这就产生了实时动画的立体图（动画中的单个帧显示在图 41-10 中）。

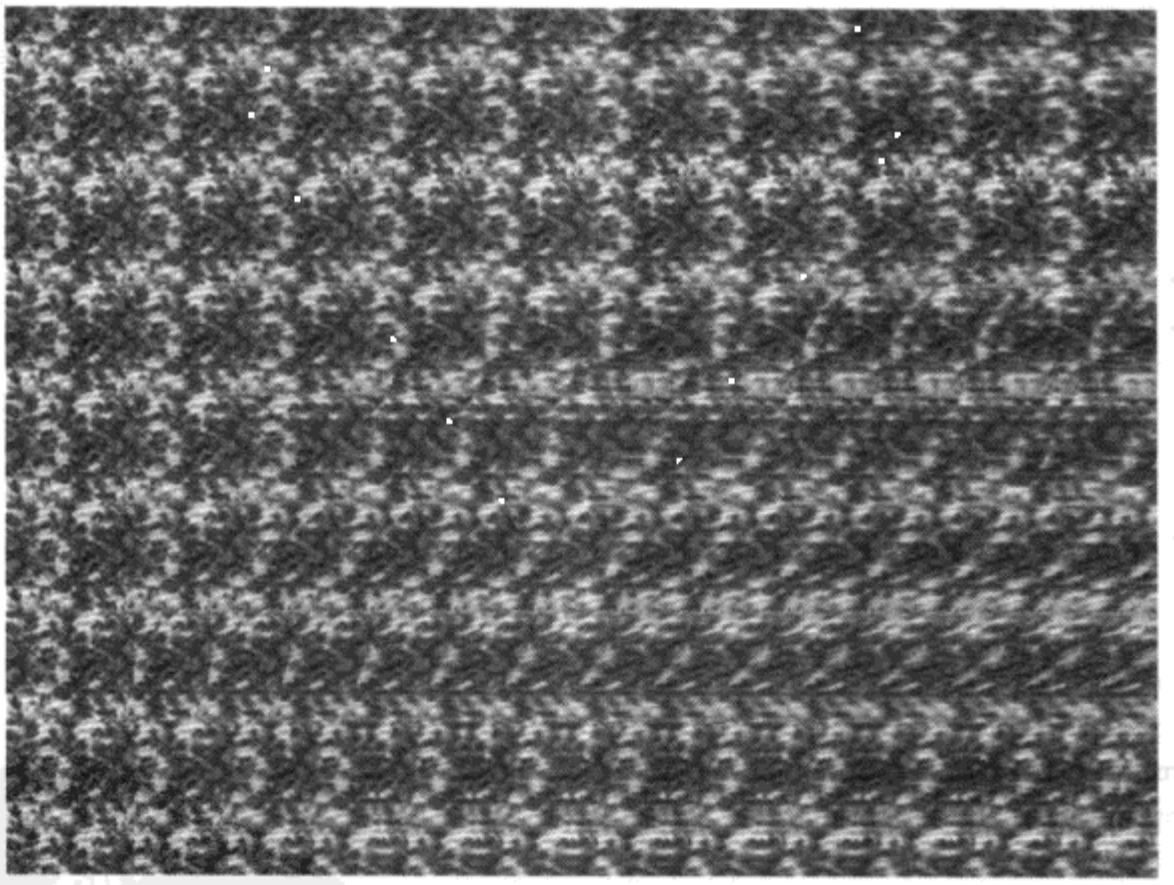


图 41-10　1 帧动画的立体图
由图 41-9（b）三维场景的 z 缓冲产生

观看一个动画的立体图是困难的，因此先看未动画的图，在清楚地看到它时，再把动画打开。这是因为必须使眼睛的交点与图像保持正确的距离，才能够使动画的立体图看得见。不熟悉立体图的人将会发现，还是对静态图像寻找正确的眼睛交点距离比较容易。

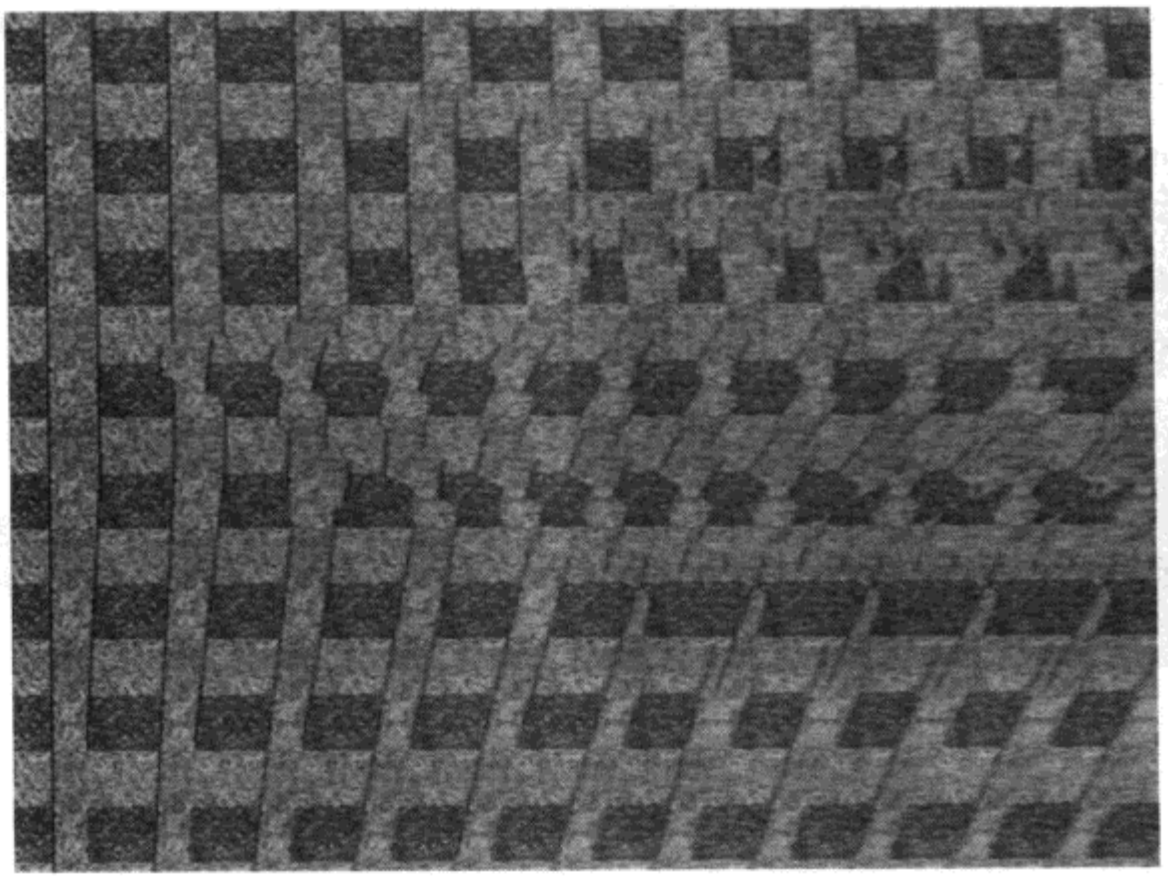


图 41-11 不同颜色平铺的其他视图
也由图 41-9（b）的三维场景的 z 缓冲产生

41.2.4 片元程序

列表 41-1 给出的片元程序，用于产生我们已经讨论的立体图。

列表 41-1 产生立体图的片元程序

```
struct vert2frag
{
  float4 pos      : POSITION;
  float4 texcoord : TEXCOORD0;
};

struct frag2screen
{
  float4 color : COLOR;
};

frag2screen main_frag(
  vert2frag IN,
  uniform sampler2D resmap,    // 结果贴图
  uniform sampler2D depthmap, // 深度贴图

  // [1.0/num_strips, 1.0/(num_strips + 1)]
  uniform float2 strips_info,
  // depth factor (if negative, invert depth)
  uniform float depth_factor)
{
  frag2screen OUT;
```

```
    // 从结果贴图来的纹理坐标
    float2 uv = IN.texcoord.xy;

    // 把纹理坐标转移到深度贴图空间
    // 移开第 1 个条带并得到深度值
    uv.x = (IN.texcoord.x/strips_info.y - 1.0) * strips_info.x;
    float4 tex = tex2D(depthmap,uv);

    // 假如因子为负，颠倒深度
    if (depth_factor < 0.0)
        tex = 1.0 - tex.x;

    // 计算偏移因子
    // (depthmap_value * factor * strip_width)
    float displace = tex.x * abs(depth_factor) * strips_info.y;

    // 把来自结果贴图的纹理坐标转变成
    // 被偏移系数所变换的前面的条带
    uv.x = IN.texcoord.x - strips_info.y + displace;

    // 指定来自结果映射前面条带的输出颜色
    OUT.color = tex2D(resmap, uv);

    return OUT;
}
```

41.3　示例应用程序

示例应用程序使用 OpenGL 和 Cg 支持片元程序。它是一个简单的 Win32 应用，而且源代码很小。为了运行应用程序，开始运行 pStereogram.exe 文件。为了浏览源代码，打开微软的 Visual C++中的 pStereogram.dsw 文件。应用程序使用 GL_ARB_fragment 程序扩展，并且能够在支持这个扩展的任何环境中运行。甚至基于模拟器的片元程序也工作，尽管工作得很慢。

应用程序包括一些选项，以便用于产生新的和定做的立体图。可以装载新的深度贴图（Ctrl+D）、新的平铺图案（Ctrl+T）和新的三维几何体（Ctrl+M）。所支持的图像文件格式是 JPG 和 TGA。应用支持 3D 几何体文件格式 3DS 和 P3D。

在观察菜单中，可以选择以下的几项：

- 纹理过滤（Ctrl+F）：这个选项用于平滑深度范围。如果禁用它，深度值将是不连续的，而且会清楚地看到在深度级别之间的缝隙（深度阶梯）。
- 深度颠倒（Ctrl+I）：这个选项用于颠倒深度值（1−深度）。它通过颠倒深度图像值而工作（白变黑或相反）。有人觉得眼睛交点在图像平面前时看立体图更舒服，有人觉得在后面更舒服（好像看监视器里面的东西）。如果观看进入图像平面的 3D 图像，用这个选项可以获得正确的图像。
- 深度因子（<,>）：这个选项为深度值的增加选择因子。小因子产成浅的图像；大的值

会形成非常深的图像。深度因子不应该比 1.0 大，如果深度贴图使用整个的可用范围（0.0 到 1.0），将会出现失真。

- 条带的数目（–, +）：这个选项指定使用条带的数目。小的值产生少量大块的条带，而大的值产生许多小块的条带。这一点改变为产生 3D 图像，眼睛交点应该靠近图像平面的程度。
- 产生立体图：当选择这项时，片元程序被激活，而且会看到立体图。如果禁用它，片元程序就不工作，而看到的一切就是当前所选择的深度贴图。
- 来自 3D 网格的深度：这个选项使立体图能够动画。启用它时，程序使用的深度贴图由当前选择的 3D 网格物体渲染的 z 缓冲得到。在这个模式中，观看产生的立体图时，可以操纵 3D 场景。一些 3D 场景可能包括不只一个摄像机，而且有些摄像机可能是活动的，因此使用数字键 1、2、…，9 变换摄像机。可以使用 S、X 和箭头键交互地移动场景。也可以通过单击和拖拉鼠标左键旋转视图，以及用鼠标的右键平移视图。

41.4 参考文献

The SIRDS FAQ. **http://www.cs.waikato.ac.nz/~singlis/sirds.html**

The Magic Eye Web site. **http://www.magiceye.com**

Thimbleby, Harold W, Stuart Inglis, and Ian H. Witten. 1994. "Displaying 3D Images: Algorithms for Single Image Random Dot Stereograms." *Journal Computer* 27(10), pp. 38～48. Available online at **http://archive.museophile.sbu.ac.uk/3d/pub/SIRDS-paper.pdf**

第 42 章　变形

Eugene d'Eon
University of Waterloo

三维空间的建模和动画软件包提供各式各样的空间变形工具，例如格子变形和波浪变形。这些工具可以用少量控制使任意一个复杂的网格变形。变形可以用来很快地运行高级的建模操作，而且可以驱动它们的控制以提供强有力的动画装备。在大多数的情况下，有可能把变形计算从 CPU 转卸到 GPU 上去，这样可以在实时应用里使用更先进的动画系统，并且提供更交互的建模工具。在这章中，我们将探讨如何把变形卸载到图形处理器上去，主要是法线的计算问题。

42.1　什么是变形

变形是这样一个操作，它输入顶点集合，产生这些顶点的新坐标（变形后的坐标）。另外，也可以产生新的法线和切线。如果想要使球体变形，把它所有顶点的坐标带入到变形操作，得到新的坐标组。这件工作特别适合于顶点程序，因为它满足了下列要求：

- 变形不创建或删除顶点或边。
- 变形后的顶点坐标不受其他被处理顶点的影响（顶点的平均/平滑操作不是变形，因为它受附近顶点的影响）。
- 变形是确定性的。如果任何两个顶点在变形之前有相同的输入坐标，那么它们的输出坐标相同。也就是说，变形同一个网格多次，每次都得到相同的结果。

非常简单的例子是平移。平移变形取一个输入坐标并输出平移后的坐标。在 Softimage|XSI、Maya 和 3ds max 中的变形，是通过曲线、笼状、格子、弯曲、膨胀、剪裁、扭曲等很多形式进行的。Softimage|XSI 中的随机变形不能满足我们的要求，因为它不是确定性的。我们的法线计算的技术不能用来实现随机网格变形。

大多数的变形是有控制的：由用户所定义的参数给出变形的一些变化。在大多数情况下，控制可以是活动的。波变形有振幅和相位控制。格子变形有格子控制点。

一般而言，我们可以把变形视为向量函数 $f(x, y, z)$，也可以视为把 3 个标量函数放入一个向量中：

$$(f_x(x,y,z), f_y(x,y,z), f_z(x,y,z))$$

这里 f_x、f_y、f_z 是 f 函数的分量。函数 f 取顶点坐标 (x, y, z)，并输出那个顶点的变形坐标。为了使网格变形，我们取各个顶点，把它的坐标输入 $\boldsymbol{f}$，并存储输出，作为新的顶点坐标。或认为 f_x 分量给出原始坐标 (x, y, z) 的新 x 坐标，f_y 给出新 y 坐标，等等。变形函数 f 应该对所有 x、y 和 z 的实际值定义，或至少对包含输入网格的那些值定义。

与空间相关的变形（例如波），在不同的坐标空间中结果也不同，所以将物体空间、世界空间或视点空间中的坐标分别代入 $\boldsymbol{f}$，对在物体空间、世界空间和视点空间中的坐标进行变形。

线性变形，例如旋转、缩放和平移是所有可能变形中的一个简单子集。它们取一组坐标作为输入，然后输出新坐标组（通常以矩阵乘法的形式）。线性变换不是特别有趣的变形，而且我们已经知道如何计算这种变换下的法线。这种变形的定义包括非常大的一组几何操作。

控制

变形操作也取决于变形的控制。控制是影响变形操作的参数。举例来说，波变形的输出取决于波的频率，因此在技术上函数 $\boldsymbol{f}$ 与变量 *freq* 相关。然而，我们不写 $\boldsymbol{f}(x, y, z, freq)$，而是相反，我们把 *freq* 当作给定变形操作中的常数：在网格顶点变形时它不变。因此，$\boldsymbol{f}$ 函数中至少有一个分量保持不变。同样地，用来动画变形的时间参数 t 在 $\boldsymbol{f}$ 中也是一个常数，因为时间不随坐标 x、y、z 而变。

42.2 在 GPU 上的变形

42.2.1 变形的公式

为方便法线的计算，把变形用数学定义（用函数表格）来公式化是有帮助的。如果我们想进行在 y 轴上正向平移两个单位的变形，那么变形 $\boldsymbol{f}$ 是：

$$\boldsymbol{f}(x,y,z) = (f_x, f_y, f_z) = (x, y+2, z)$$

如果我们正在实现一个辐射波的变形，我们可能有

$$\boldsymbol{f}(x,y,z) = (f_x, f_y, f_z) = (x, y+\sin(\sqrt{x^2+z^2}+\lambda), z)$$

这里 λ 是相位控制参数。

注意：f_x 代表 $\boldsymbol{f}$ 函数的 x 分量，不要与 $\partial f / \partial x$ 混淆。

42.2.2 写顶点程序

一旦我们以函数形式写下变形，在顶点程序里进行顶点变形就非常简单了。把控制参数和可能的时间参数以统一输入带入程序（进入常数寄存器）。为了计算顶点位置，我们计算 $f(x, y, z)$，把结果存储为顶点位置的输出。根据希望变形发生在对象空间、世界空间还是视图空间，决定在计算 f 前还是之后应用模型和视图变换。在与空间相关的变形关联到要变形的对象时，我们也可以有附加的坐标变换来置中和转向它。

那么法线呢?

在网格上进行线性变换时，例如平移、旋转、缩放或它们的组合，我们知道取变形的矩阵表达式 $\boldsymbol{M}$，而且通过 $\boldsymbol{M}$ 的逆转置来变换法线，然后再归一化[1]。但是线性变换仅是我们想在顶点程序中实现变形的很小子集。一般而言，因为波变形不是线性变形，它不可能写成矩阵形式（那时，有些常数矩阵 $\boldsymbol{M}$ 可以通过计算 $\boldsymbol{Mv}$ 给出变形位置）。因此我们没有全局矩阵 $\boldsymbol{M}$ 用来转置，从而转换法线。为了转换法线，需要新的技术。

42.2.3　法线的变形

如何在软件中计算法线?

动画软件包首先通过变形转换网格中所有顶点。一旦完成之后，可以在第 2 个通道中遍历顶点，以新变形网格中邻近顶点为基础再计算法线。我们没有带顶点程序的选项。在顶点程序中所处理的每个顶点不知道它的邻居，也没有可以来回察看和计算的第 2 个阶段。

一种近似的数值技术

有一种方法是对每个输入顶点变形 3 个点达到法线变形。求两个非常接近输入顶点的新点。把切线乘以非常小的数后加入顶点坐标得到第 1 点。把副法线乘以非常小的数后加入顶点坐标得到第 2 点。这 3 点定义了两个向量，它们的叉积是原始的法线（等于归一化常数）。然后通过变形操作使这 3 个点变形，而且这 3 个变形点将会定义两个向量，它们的叉积近似于被变形的法线（等于归一化常数）。这 3 个点越靠近，近似值就越正确，但是如果点靠近得太厉害，也会出现数值的准确性问题。我们有更好的办法，可以避免数值上的困难和额外变形点的选择问题。

Jacobian 矩阵

如果变形函数 $\boldsymbol{f}(x, y, z)= (f_x, f_y, f_z)$有连续的一阶偏导数，我们能为每个顶点计算矩阵 $\boldsymbol{J}$，称为 Jacobian 矩阵。我们知道，若函数 $f(x)$的各处是平滑的，那么它可以局部地由一条直线近似（对于某些实数 a，线性函数 $f(x) \approx f'(a)(x-a) + f(a)$）。同样，若图 $f(x,y,z)$各处是平滑的，$\boldsymbol{f}$可以由线性变换 $\boldsymbol{J}$ 加上平移来近似（对于某些点 a 和 $x = (x, y, z)$，$(\boldsymbol{f}(x) \approx \boldsymbol{J}(a)(x - a) + \boldsymbol{f}(a))$。这可以从广义的泰勒 Taylor 定理推出。Jacobian 矩阵定义如下。注意：Jacobian 矩阵时常表示为 $\boldsymbol{Df}(x)$。

$$\boldsymbol{J}(x,y,z) = \begin{pmatrix} \dfrac{\partial f_x}{\partial x} & \dfrac{\partial f_x}{\partial y} & \dfrac{\partial f_x}{\partial z} \\ \dfrac{\partial f_y}{\partial x} & \dfrac{\partial f_y}{\partial y} & \dfrac{\partial f_y}{\partial z} \\ \dfrac{\partial f_z}{\partial x} & \dfrac{\partial f_z}{\partial y} & \dfrac{\partial f_z}{\partial z} \end{pmatrix}$$

这里所有的偏微分都在（x, y, z）处求值。

---

1. 见 OpenGL Programming Guide 中“齐次坐标”一节（出版的书，作者 Addison-Wesley）

例子。假如我们有以下变形：（这里 A 是常数控制参数）

$$f(x,y,z)=(f_x,f_y,f_z)=(\lambda x,z^3,z-y^2)$$

则 Jacobian 会是：

$$J(x,y,z)=\begin{pmatrix}\frac{\partial f_x}{\partial x} & \frac{\partial f_x}{\partial y} & \frac{\partial f_x}{\partial z}\\ \frac{\partial f_y}{\partial x} & \frac{\partial f_y}{\partial y} & \frac{\partial f_y}{\partial z}\\ \frac{\partial f_z}{\partial x} & \frac{\partial f_z}{\partial y} & \frac{\partial f_z}{\partial z}\end{pmatrix}=\begin{pmatrix}\lambda & 0 & 0\\ 0 & 0 & 3z^2\\ 0 & -2y & 1\end{pmatrix}$$

对于每个位置 (x,y,z)，我们现在有线性变换矩阵 $J(x,y,z)$，可以用来变换那个位置的向量。如果在 (x,y,z) 有切线向量 t，那么变形的切线会是 $J(x,y,z)*t$（接着再归一化为单位切线）。

如果我们假定网格的顶点是平滑表面的离散样本，那么由 Jacobian 矩阵在表面上转换的向量是有效的。我们也可以用两种等效的方式使用 $J(x,y,z)$变换法线。

1．$J(x,y,z)$的转置可以在顶点程序中逐顶点地计算，而且用来直接转换法线。假如 n 是输入的单位法线，那么变形后的法线是：

$$n'=normalize[((J(x,y,z))^{-1})^T n]$$

符号 A^{T} 表示矩阵 A 的转置。

2．如果给出每个顶点的单位切线向量 u，那么可以计算出单位副法线 $v=n\times u$。我们可以使用矩阵 $J(x,y,z)$把切线和副法线转换到变形空间。于是变形后的新法线是：

$$n'=normalize[(J(x,y,z)u)\times(J(x,y,z)v)]$$

切线和副法线实际上不需要是单位长度的，因为最终的结果要归一化。

求 3 × 3 矩阵的逆矩阵对于顶点程序不是特别方便，因此方法（2）通常实现起来比较容易，计算也比较快速。如果每个顶点的切线和副法线没有给出，可以在装载网格时在 CPU 上产生或在顶点程序中产生。为了产生切线，计算两个叉积，$n\times(0,0,1)$和 $n\times(0,1,0)$，选择其中较大的结果，结果是对 n 的切线向量。接着副法线由 $v=n\times u$ 给出。

注意：这个技术对整个表面产生不连续的切线和副法线，因此不适用于凹凸贴图以及其他需要连续切线场的技术。

因为用这些方法产生的法线一般不是单位长度，而大多数的照明方程式要求单位法线，所以需要将法线重归一化。

42.3 局限性

- 平滑表面的假设对多个小面的网格是无效的，用软件首先变形网格，然后再计算出的各小面法线不同于平滑表面给出的法线。大体上，变形后每个小面的法线都会改变。但是某些情况下这个方法产生的结果还是可以接受的。
- 为了变换某个位置的法线，需要计算 Jacobian 矩阵的转置，Jacobian 矩阵在那个位置应该是非退化的（即行列式必须是非 0 的）。大多数实际变形情况如此，但是有时变

形设计为整个域上没有最大列的非标准函数。在这种位置不能够计算法线。这章所述的两个技术会在这种退化点上产生 0 向量作为变形的法线。在实际中，数值误差产生的向量非常接近于 0，而后来的归一化操作将产生任意结果。

- 这个方法要求所涉及的函数是可微的，在感兴趣的域上最好能解析可微。因为如果不能解析求函数的导数时，需要使用近似求导技术求偏导数的近似值，这将极大地延长计算，因此非常不好。
- 这个方法受顶点程序的长度所制约，Jacobian 是冗长计算时尤其如此。如果通过一系列变形运算来使一个物体变形，实际上只能在图形处理器上计算最后一个变形。如果另外还有蒙皮或其他复杂操作需要在相同的顶点程序中运行，那么程序可能会有太多指令，或因为计算大量的顶点而运行太慢。

42.4　性能

如前所述，另一个近似计算法线的方法，涉及到对每个输入顶点的 3 个顶点变形，一个是沿着切线非常小的变形，一个是沿着副法线非常小的变形。接着用这 3 个变形顶点来近似变形的切线和副法线，并用叉积给出变形的法线。仅对一组顶点坐标就包含了 3 个以上的变形工作。在许多情况下，计算 Jacobian 矩阵所需要的指令，比变形两组额外坐标的指令更少。在计算 Jacobian 时，往往要重复表达式，而某些表达式与计算顶点坐标的完全相同，因此可以重复使用。也有例外的时候，尤其是如果偏导数涉及到复杂的表达式时。Jacobian 方法不用代入额外坐标，而且避免了许多由于近似方法引起的数值困难。

性能是计算变形所需要指令数的简单函数。像 $4 \times 4 \times 4$ 的格子操作那样复杂的变形需要许多指令，这对实时中可能变形的几何体数目是个限制。在每种情况下，必须考虑到变形的复杂性、应用的 CPU 负荷和 GPU 负荷，以及变形的几何体数量等因素，才能确定在图形处理器上变形是否是较好的解决方法。

42.5　例子：波浪变形

在这里我们考虑做垂直移动的圆形波，见图 4-21，移动基于从 x-z 平面原点开始的半径（在 y 轴上）。函数形式为：

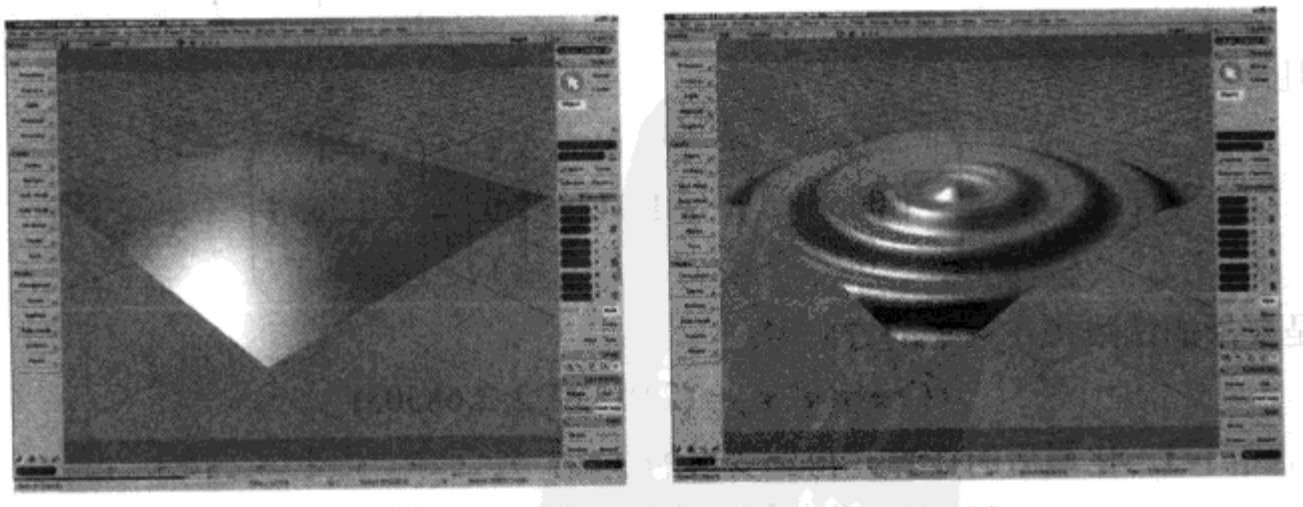


图 42-1　将垂直移动的圆形波变形应用于平面

$$f(x,y,z)=(f_x,f_y,f_z)=(x,y+\sin(v\sqrt{x^2+z^2}+\lambda),z)$$

这里 v 是波频率控制参数，A 是波相位参数。然后我们计算 Jacobian 矩阵：

$$J(x,y,z)=\begin{pmatrix}\frac{\partial f_x}{\partial x} & \frac{\partial f_x}{\partial y} & \frac{\partial f_x}{\partial z}\\ \frac{\partial f_y}{\partial x} & \frac{\partial f_y}{\partial y} & \frac{\partial f_y}{\partial z}\\ \frac{\partial f_z}{\partial x} & \frac{\partial f_z}{\partial y} & \frac{\partial f_z}{\partial z}\end{pmatrix}$$

$$=\begin{pmatrix}1 & 0 & 0\\ \frac{\cos\left(v\sqrt{x^2+z^2}+\lambda\right)}{\sqrt{x^2+z^2}}vx & 1 & \frac{\cos\left(v\sqrt{x^2+z^2}+\lambda\right)}{\sqrt{x^2+z^2}}vz\\ 0 & 0 & 1\end{pmatrix}.$$

假如变形坐标为（3,0,4）的顶点和法线（0,1,0）。让我们选择波频率 v =2，波相位参数 A=0，首先在那个位置上计算 Jacobian 矩阵。

$$J(3,0,4)=\begin{pmatrix}1 & 0 & 0\\ \frac{\cos\left(2\sqrt{3^2+4^2}+0\right)}{\sqrt{3^2+4^2}}2(3) & 1 & \frac{\cos\left(2\sqrt{3^2+4^2}+0\right)}{\sqrt{3^2+4^2}}2(4)\\ 0 & 0 & 1\end{pmatrix}$$

$$\approx\begin{pmatrix}1 & 0 & 0\\ -1.00689 & 1 & -1.34251\\ 0 & 0 & 1\end{pmatrix}.$$

接着，假定切线和副法线为：

$$u=(-1,0,1),v=(1,0,1)$$

然后被转换的切线和副法线是：

$$u'=\begin{pmatrix}1 & 0 & 0\\ -1.00689 & 1 & -1.34251\\ 0 & 0 & 1\end{pmatrix}\begin{pmatrix}-1\\ 0\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -0.335629\\ 1\end{pmatrix}$$

而且

$$v'=\begin{pmatrix}1 & 0 & 0\\ -1.00689 & 1 & -1.34251\\ 0 & 0 & 1\end{pmatrix}\begin{pmatrix}1\\ 0\\ 1\end{pmatrix}=\begin{pmatrix}-1\\ -2.3494\\ 1\end{pmatrix}$$

于是变形的法线将在以下的方向中：

$$Cross(u',v')=(2.01377,2,2.68503)$$

然后我们归一化最后的计算，得到变形的法线：

$$n'=(0.515426,0.511902,0.687235)$$

42.6　小结

我们已经考虑一大类网格变形操作，而且发现了在图形硬件上进行操作的数学工具。变形的 Jacobian 矩阵允许我们在顶点程序中变形法线，而无须担心相邻的变形顶点。这项技术广泛地适用于许多变形操作，只是要考虑到一些局限性。

虽然本书的许多内容处理的是着色效果，但是物体的移动和变形方法与创建逼真的动画图形一样重要。现在，电影人物动画的质量超过电视游戏互动式图形的动画。产生这种质量差距的一方面原因是电影中使用了更多尖端的变形技术，而大多数游戏仅限于使用简单的蒙皮。我们希望随着时间的推移，这章中所描述的技术会有助于动画技术全面用于实时产品。